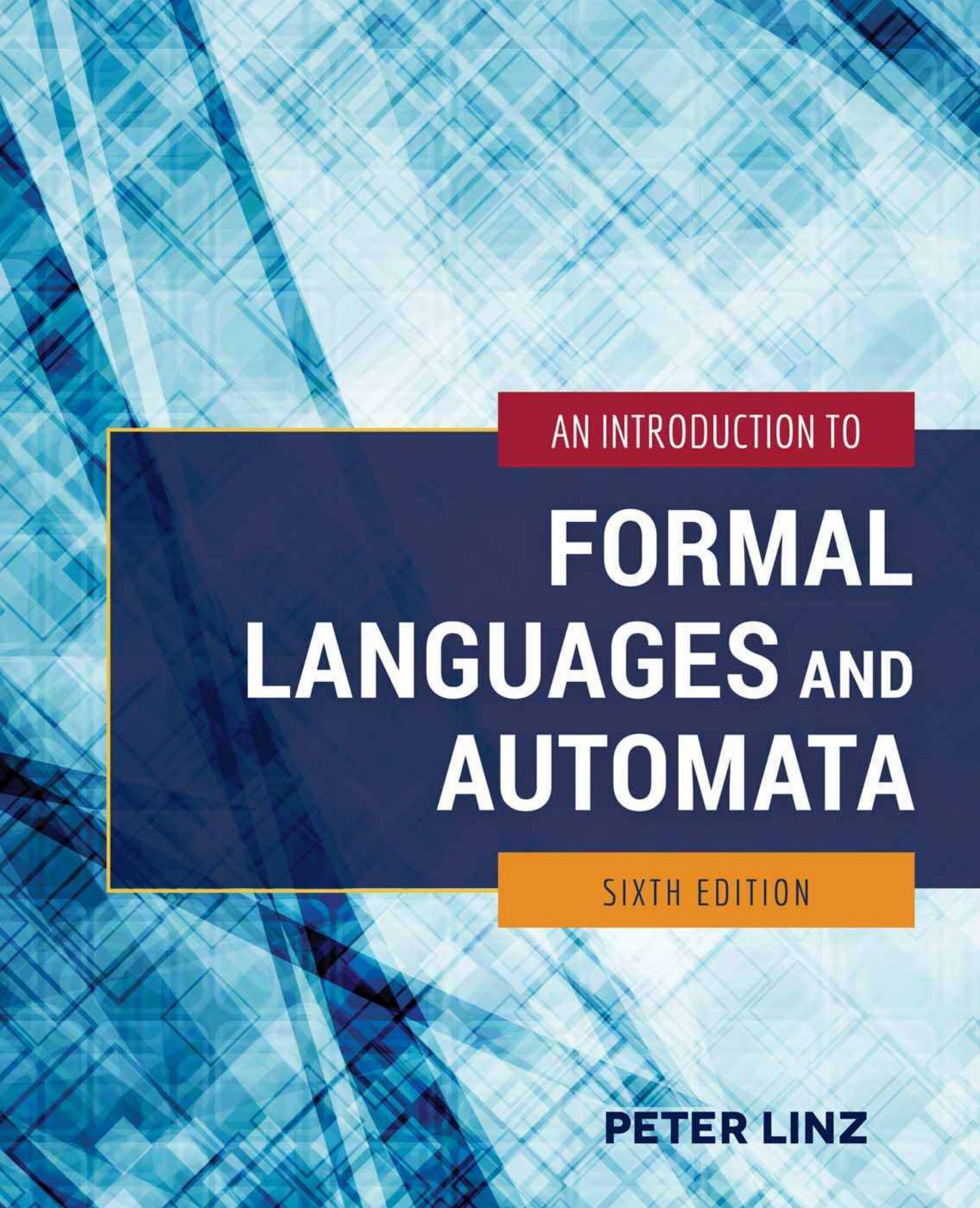




AN INTRODUCTION TO

FORMAL LANGUAGES AND AUTOMATA

SIXTH EDITION

PETER LINZ

BİR GİRİŞ

FORMAL DİLLER VE OTOMATLAR

ALTINCI BASKI

PETER LINZ

CALIFORNIA ÜNİVERSİTESİ DAVIS



JONES & BARTLETT
LEARNING

Dünya Genel Merkezi
Jones & Bartlett Learning
5 Wall Street
Burlington, MA 01803
978-443-5000
info@jblearning.com
www.jblearning.com

Jones & Bartlett Learning kitapları ve ürünleri çoğu kitapçıdan ve çevrimiçi kitap satıcılarından temin edilebilir. Jones & Bartlett Learning ile doğrudan iletişime geçmek için 800-832-0034 numaralı telefonu arayın, 978-443-8000 numaralı faksı gönderin veya web sitemizi ziyaret edin, www.jblearning.com.

Jones & Bartlett Learning yayınlarının toplu alımları için önemli indirimler, şirketler, meslek birlikleri ve diğer nitelikli kuruluşlara sunulmaktadır. Detaylar ve özel indirim bilgileri için yukarıdaki iletişim bilgileri aracılığıyla Jones & Bartlett Learning'in özel satış departmanı ile iletişime geçin veya specialsales@jblearning.com adresine bir e-posta gönderin.

Telif hakkı © 2017 Jones & Bartlett Learning, LLC, Ascend Learning Şirketi Tüm hakları saklıdır. Bu t

elif hakkı ile korunan materyalin hiçbir kısmı, yazılı izin olmadan, fotokopi, kayıt veya herhangi bir bilgi depolama ve geri alma sistemi dahil olmak üzere, elektronik veya mekanik herhangi bir biçimde çoğaltılamaz veya kullanılamaz.

Buradaki içerik, ifadeler, görüşler ve düşünceler, ilgili yazarların tek ifadesidir ve Jones & Bartlett Learning, LLC'nin görüşlerini yansıtmaz. Burada herhangi bir ticari ürün, süreç veya hizmete, ticari ad, marka, üretici veya başka bir şekilde yapılan referans, Jones & Bartlett Learning, LLC tarafından onaylanmasını veya tavsiye edilmesini oluşturmaz veya ima etmez ve bu tür referanslar reklam veya ürün onaylama amaçları için kullanılamaz. Gösterilen tüm markalar, burada belirtilen tarafların markalarıdır. Formal Diller ve Otomatlar, Altıncı Baskı bağımsız bir yayındır ve bu üründe referans verilen markaların veya hizmet markalarının sahipleri tarafından yetkilendirilmemiş, sponsor olunmamış veya başka bir şekilde onaylanmamıştır.

Bu kitapta modellerin yer aldığı görüntüler olabilir; bu modeller, görüntülerde temsil edilen etkinlikleri desteklemekte, temsil etmemekte veya k

Üretim Kredileri
VP, İcra Yayıncı: David D. Cella
Yayıncı: Cathy L. Esperti
Edinimler Editörü: Laura Pagluica
Editoryal Asistan: Taylor Ferracane
Üretim Editörü: Sara Kelly
Pazarlama Direktörü: Andrea DeFronzo
VP, Üretim ve Envanter Kontrolü: Therese Connell
Kompozisyon: S4Carlisle Yayıncılık Pvt. Ltd.
Kapak Tasarımı: Kristin E. Parker
Haklar ve Medya Uzmanı: Jamey O'Quinn
Medya Geliştirme Editörü: Shannon Sheehan
Kapak Görseli: ©saicle/shutterstock
Basım ve Ciltleme: Edwards Brothers Malloy
Kapak Baskısı: Edwards Brothers Malloy

Amerikan Kongre Kütüphanesi Yayın Kataloğu Verileri
Linz, Peter.
Resmi diller ve otomata giriş / Peter Linz, PhD,
California Üniversitesi, Davis, Davis, California
-Altıncı baskı.
sayfalar; cm

Bibliyografik referanslar ve dizin içerir.
ISBN 978-1-284-07724-7(casebound)
1. Resmi diller. 2. Makine teorisi. I. Başlık.
QA267.3.L56 2016
005.13'1-dc23

2015023479

6048

Amerika Birleşik Devletleri'nde basılmıştır

20 19 18 17 16 10 9 8 7 6 5 4 3 2 1

Anne ve Babamın Anısına

İÇİNDEKİLER

ÖNSÖZ

xi

1 TEORİYE GİRİŞ

HESAPLAMANIN

1

1.1 Matematik Ön Hazırlıklar ve Notasyon

3

Setler

3

Fonksiyonlar ve İlişkiler

6

Grafikler ve Ağaçlar

8

Kanıt Teknikleri

9

1.2 Üç Temel Kavram

17

Diller

17

Dilbilgileri

20

Otomatlar

26

1.3 Bazı Uygulamalar*

30

2 SONLU OTOMATLAR

37

2.1 Belirleyici Sonlu Kabul Ediciler

38

Diller ve DFA'lar

41

Regüler Diller

46

2.2 Belirsiz Sonlu Kabul Ediciler

51

Nedensiz Kabul Edici Tanımı

51

Nedensizlik Neden?

55

2.3 Belirleyici ve Nedensiz Eşdeğerlik

Sonsuz Kabul Ediciler

58

2.4 Sonlu Otomatlarda Durum Sayısının Azaltılması* . . .66

3 DÜZENLİ DİLLER VE	
DÜZENLİ GRAMERLER	73
3.1 Düzenli İfadeler	74
Düzenli İfadenin Resmi Tanımı	74
Düzenli İfadelerle İlişkili Diller	75
3.2 Düzenli İfadeler Arasındaki Bağlantı	
ve Düzenli Diller	80
Düzenli İfadeler Düzenli Dilleri Belirler	80
Düzenli Diller için Düzenli İfadeler	82
Basit Desenleri Tanımlamak için Düzenli İfadeler	88
3.3 Düzenli Dilbilgileri	91
Sağ ve Sol-Doğrusal Dilbilgileri	92
Sağ-Doğrusal Dilbilgileri Düzenli Dilleri Üretir	93
Sağ-Doğrusal Dilbilgileri için Düzenli Diller	96
Düzenli Diller ve Düzenli Gramerlerin Eşdeğerliliği . .	97
4 DÜZENLİ DİLLERİN ÖZELLİKLERİ	101
4.1 Düzenli Dillerin Kapatma Özellikleri	102
Basit Küme İşlemleri Altında Kapatma	102
Diğer İşlemler Altında Kapatma	105
4.2 Düzenli Diller Hakkında Temel Sorular	114
4.3 Düzenli Olmayan Dilleri Tanımlama	117
Güvercin Yuva Prensibini Kullanma	117
A Pompalama Lemması	118
5 BAĞIMSIZ DİLLER	129
5.1 Bağımsız Gramerler	130
Bağımsız Dillerin Örnekleri	131
En Soldaki ve En Sağdaki Türevler	133
Türev Ağaçları	134
Cümle Biçimleri ile Türev Ağaçları Arasındaki İlişki . . .	137
5.2 Ayırıştırma ve Belirsizlik	140
Parçalama ve Üyelik	141
Gramerler ve Dillerde Belirsizlik	145
5.3 Bağlamdan Bağımsız Gramerler ve Programlama Dilleri ...	151
6 BAĞLAMDAN BAĞIMSIZ GRAMERLERİN VE NO	
RMAL ŞEKİLLERİN SİMLİFİKASYONU	155
6.1 Gramerleri Dönüştürme Yöntemleri	156
Faydalı Bir İkame Kuralı	157
Gereksiz Üretimlerin Kaldırılması	159
λ-Üretimlerin Kaldırılması	163
Birlik Üretimlerin Kaldırılması	165
6.2 İki Önemli Normal Form	171
Chomsky Normal Form	171
Greibach Normal Form	174
6.3 Bağlamdan Bağımsız Dilbilgileri için Bir Üyelik Algoritması* . .	178

7 İTME OTOMATLARI	181
7.1 Belirsiz İtme Otomatları	182
Bir İtme Otomatının Tanımı	183
Bir İtme Otomatı Tarafından Kabul Edilen Dil	186
7.2 İtişli Otomatlar ve Bağlamdan Bağımsız Diller	191
Bağlamdan Bağımsız Diller için İtme Otomatları	191
İtme Otomatları için Bağlamdan Bağımsız Dilbilgileri	196
7.3 Belirleyici İtme Otomatları ve Belirleyici	
Bağlamdan Bağımsız Diller	203
7.4 Belirleyici Bağlamdan Bağımsız Diller için Dilbilgileri*	207
8 BAĞIMSIZ DİLLERİN 8 ÖZELLİĞİ	213
8.1 İki Pompalama Lemması	214
Bağımsız Diller için Bir Pompalama Lemması	214
Doğrusal Diller için Bir Pompalama Lemması	219
8.2 Kapatma Özellikleri ve Karar Algoritmaları için	
Bağlamdan Bağımsız Diller	223
Bağlamdan Bağımsız Dillerin Kapatılması	223
Bağlamdan Bağımsız Dillerin Bazı Karar Verilebilir Özellikleri ...	227
9 TÜRK MAKİNELERİ	231
9.1 Standart Turing Makinesi	232
Turing Makinesi Tanımı	232
Turing Makineleri Dil Kabul Edicileri Olarak	239
Turing Makineleri Transdüserler Olarak	242
9.2 Karmaşık Görevler İçin Turing Makinelerini Birleştirme	249
9.3 Turing'in Tezi	255
10 DİĞER TURING MAKİNELERİ MODELLERİ	259
10.1 Turing Makinesi Teması Üzerindeki Küçük Varyasyonlar ...	260
Otomat Sınıflarının Eşdeğerliliği	260
Kalma Seçeneğine Sahip Turing Makineleri	261
Yarı Sonsuz Bantlı Turing Makineleri	263
Çevrimdışı Turing Makinesi	265
10.2 Daha Karmaşık Depolama ile Turing Makineleri	268
Çok Bantlı Turing Makineleri	268
Çok Boyutlu Turing Makineleri	271
10.3 Belirsiz Turing Makineleri	273
10.4 Evrensel Turing Makinesi	276
10.5 Doğrusal Sınırlı Otomatlar	281
11 FORMAL DİLLERİN VE OTOMATLARIN HİYERARŞİSİ	285
11.1 Özyinelemeli ve Özyinelemeli Sayılabilir Diller	286
Özyinelemeli Sayılabilir Olmayan Diller	288

Özyinelemeli Olarak Sayılabilir Olmayan Bir Dil	289
Özyinelemeli Olarak Sayılabilir Bir Dil ama Özyinelemeli Değil	291
11.2 Sınırsız Dilbilgileri	293
11.3 Bağlama Duyarlı Dilbilgileri ve Dilleri	299
Bağlam Duyarlı Diller ve Doğrusal Sınırlı Otomatlar	300
Özyinelemeli ve Bağlam Duyarlı Arasındaki İlişki Diller	302
11.4 Chomsky Hiyerarşisi	305
12 ALGORİTMİK HESAPLAMANIN SINIRLARI	309
12.1 Çözülemeyen Bazı Problemler Turing Makineleri Tarafından	310
Hesaplanabilirlik ve Karar Verilebilirlik	310
Turing Makinesi Durdurma Problemi	311
Bir Kararsız Problemi Diğerine İndirmek	315
12.2 Tekrar Edilebilir Diller için Karar Verilemez Problemler Enumerable Diller	319
12.3 Post İletişim Problemi	323
12.4 Bağlamdan Bağımsız Diller için Karar Verilemez Problemler	329
12.5 Verimlilik Sorusu	332
13 DİĞER HESAPLAMA MODELLERİ	335
13.1 Özyinelemeli Fonksiyonlar	337
Primitif Özyinelemeli Fonksiyonlar	338
Ackermann Fonksiyonu	342
μ Özyinelemeli Fonksiyonlar	343
13.2 Post Sistemleri	346
13.3 Yeniden Yazım Sistemleri	350
Matris Gramerleri	350
Markov Algoritmaları	351
L-Sistemleri	353
14 HESAPLAMADA BİR GÖRÜNÜM	355
14.1 Hesaplamanın Verimliliği	356
14.2 Turing Makinesi Modelleri ve Karmaşıklık	358
14.3 Dil Aileleri ve Karmaşıklık Sınıfları	362
14.4 P ve NP Karmaşıklık Sınıfları	365
14.5 Bazı NP Problemleri	367
14.6 Polinom Zamanında Azaltma	370
14.7 NP-Tamamlayıcılığı ve Açık Bir Soru	373

EK A	SONLU DURUM DÖNÜŞTÜRÜCÜLERİ	377
A.1	Genel Çerçeve	377
A.2	Mealy Makineleri	378
A.3	Moore Makineleri	380
A.4	Moore ve Mealy Makinesi Eşdeğerliliği	382
A.5	Mealy Makinesi Minimizasyonu	386
A.6	Moore Makinesi Minimizasyonu	391
A.7	Sonlu Durum Dönüştürücülerinin Sınırlamaları	392
EK B	JFLAP: KULLANILABİLİR BİR ARAÇ	395
Cevaplar	ÇÖZÜMLER VE İPUÇLARI	
SEÇİLMİŞ EGZERSİZLER İÇİN		397
DİĞER OKUMA İÇİN KAYNAKLAR		439
İÇİNDEKİLER		441

ÖNSÖZ

Bu kitap, formel diller, otomata, hesaplanabilirlik ve ilgili konular üzerine bir giriş dersi için tasarlanmıştır. Bu konular, hesaplama teorisi olarak bilinen şeyin önemli bir kısmını oluşturmaktadır. Bu konu üzerine bir ders, bilgisayar bilimi müfredatında artık standarttır ve genellikle programın oldukça erken aşamalarında öğretilmektedir. Bu nedenle, bu kitabın potansiyel okuyucu kitlesi, öncelikle bilgisayar bilimi veya bilgisayar mühendisliği bölümünde öğrenim gören ikinci ve üçüncü sınıf öğrencilerinden oluşmaktadır.

Bu kitapta yer alan materyaller için ön koşullar, bazı yüksek seviyeli programlama dillerine (genellikle C, C++ veya Java™) dair bir bilgi ve veri yapıları ile algoritmaların temellerine aşinalıktır. Küme teorisi, fonksiyonlar, ilişkiler, mantık ve matematiksel akıl yürütmenin unsurlarını içeren bir ayrık matematik dersi gereklidir. Böyle bir ders, standart giriş bilgisayar bilimi müfredatının bir parçasıdır.

Hesaplama teorisinin incelenmesinin birkaç amacı vardır, en önemlisi (1) öğrencileri bilgisayar biliminin temelleri ve ilkeleri ile tanıştırmak, (2) sonraki derslerde faydalı olacak materyali öğretmek ve (3) öğrencilerin formel ve titiz matematiksel argümanlar yürütme yeteneklerini güçlendirmektir. Bu metnin için seçtiğim sunum, ilk iki amaca öncelik vermektedir, ancak üçüncüyü de hizmet ettiğini savunabilirim. Fikirleri net bir şekilde sunmak ve öğrencilere materyal hakkında içgörü kazandırmak için, metnin sezgisel motivasyon ve fikirlerin örnekler aracılığıyla gösterimini vurgulamaktadır. Bir seçim olduğunda, kavram olarak zor olan ama özlü ve şık olan argümanlar yerine, kolayca kavranabilen argümanları tercih ediyorum. Tanımları ve teoremleri kesin bir şekilde ifade ediyorum ve kanıtların motivasyonunu veriyorum, ancak sıklıkla

gündelik ve sıkıcı detayları atlayın. Bunun pedagojik nedenlerle arzu edilir olduğunu düşünüyorum. Birçok kanıt, belirli problemlere özgü farklılıklarla birlikte, indüksiyon veya çelişki gibi heyecan verici olmayan uygulamalardır. Böyle argümanları tam detaylarıyla sunmak yalnızca gereksiz değil, aynı zamanda hikayenin akışını daboşuyor. Bu nedenle, birçok kanıt kısadır ve tamlıkta ısrar eden biri, onları detay açısından yetersiz bulabilir. Bunu bir dezavantaj olarak görmüyorum. Matematiksel beceriler, başkalarının argümanlarını okumaktan değil, bir problemin özünü düşünmekten, noktayı vurgulamak için uygun fikirler keşfetmekten ve ardından bunları kesin detaylarla gerçekleştirmekten gelir. İkincisi kesinlikle öğrenilmesi gereken bir beceridir ve bu metindeki kanıt taslaklarının böyle bir pratik için çok uygun başlangıç noktaları sağladığını düşünüyorum.

Bilgisayar bilimi öğrencileri bazen hesaplama teorisi dersini gereksiz yere soyut ve pratikte hiçbir önemi olmayan bir şey olarak görürler. Onları aksi yönde ikna etmek için, belirli ilgi alanlarına ve güçlü yönlerine, zorlu problemlere karşı azim ve yaratıcılık gibi, başvurmak gerekir. Bu nedenle, benim yaklaşımım problem çözme yoluyla öğrenmeye vurgu yapmaktadır.

Bir problem çözme yaklaşımından kastım, öğrencilerin materyali esasen kavramların arkasındaki motivasyonu gösteren problem türündeki örnekler aracılığıyla öğrenmeleridir; ayrıca bu kavramların teoremler ve tanımlarla bağlantısını da göstermektedir. Aynı zamanda, örnekler, öğrencilerin bir çözüm bulmaları gereken önemsiz bir yön içerebilir. Böyle bir yaklaşımda, ödev alıştırmaları öğrenme sürecinin büyük bir kısmına katkıda bulunur. Her bölümün sonunda yer alan alıştırmalar, materyali aydınlatmak ve örneklemek için tasarlanmış olup, öğrencilerin çeşitli seviyelerde problem çözme yeteneklerini kullanmalarını gerektirir.

Bazı alıştırmalar oldukça basittir, metindeki tartışmanın bıraktığı yerden başlayarak öğrencilerden bir veya iki adım daha devam etmelerini ister. Diğer alıştırmalar ise çok zordur, en iyi zihinleri bile zorlayabilir. Bu tür alıştırmaların iyi bir karışımı, çok etkili bir öğretim aracı olabilir. Öğrencilerden tüm problemleri çözmeleri istenmemelidir, ancak kursun hedeflerini ve eğitmenin bakış açısını destekleyenler verilmelidir. Bilgisayar bilimi müfredatları kurumdan kuruma farklılık gösterir; bazıları teorik tarafı vurgularken, diğerleri neredeyse tamamen pratik uygulamaya yöneliktir. Bu metnin, alıştırmalar dikkatlice öğrencilerin geçmiş ve ilgi alanları göz önünde bulundurularak seçildiği takdirde, bu iki uçtan birine hizmet edebileceğine inanıyorum. Aynı zamanda, eğitmenin öğrencileri, kendilerinden beklenen soyutlama seviyesini bilgilendirmesi gerekmektedir. Bu, özellikle kanıta dayalı alıştırmalar için geçerlidir. “Şunu kanıtlayın” veya “şunu gösterin” dediğimde, öğrencinin bir kanıtın nasıl inşa edileceğini düşünmesi ve ardından net bir argüman üretmesi gerektiğini aklımda tutuyorum. Böyle bir kanıtın ne kadar resmi olması gerektiği eğitmen tarafından belirlenmeli ve öğrencilere bu konuda kursun başında rehberlik edilmelidir.

Metnin içeriği, bir dönemlik bir ders için uygundur. Çoğu malzeme kapsanabilir, ancak bazı vurgu seçimleri yapılması gerekecektir.

Derslerimde genellikle kanıtları geçiřtiriyorum, sonucu makul kılmak için yeteri ncekapsama saęlıyorum ve ardından öğrencilerden geri kalanını kendi başlarını a okumalarını istiyorum. Genel olarak, ancak daha sonra potansiyel zorluklar ol madan tamamen atlanabilecek çok az şey vardır. Birkaç bölüm, bir yıldız ile işare tlenmiş olanlar, daha sonraki materyale kaybı olmadan atlanabilir. Ancak, çoęu malzeme esastır ve kapsanmalıdır.

Ek B, bu kitabın materyalini öğrenme ve öğretme konusunda büyük y ardımcı olan etkileşimli bir yazılım aracı olan JFLAP'ı kısaca tanıtmaktadır. K avramların anlaşılmasına yardımcı olur ve alıştırmaların çözümlerinin gerç ek inşasında büyük zaman tasarrufu saęlar. JFLAP'ın ders entegre etmesin i şiddetle tavsiye ediyorum. JFLAP için öğretmen kaynakları, metnin web si tesinde mevcuttur: go.jblearning.com/LinzJPAK.

Altıncı baskıda, daha zor materyali daha net bir şekilde ortaya koy mak için tasarlanmış bir dizi özellik ekledim. Her bölüm, yalnızca ana k avramları tanıtmakla kalmayıp, bu kavramları daha önce neler olduęu nu ve daha sonra nelerin kapsanacağını da bağlayan bir özetle başlar. Özetler, kitapta geliştirilen hikayenin bir taslağını oluşturur. Zor soyutl amaların gerekli olduęu yerlerde, soyutlamaların belirli biçimlerinin n edenlerini örnekleyen somut tartışmalar ekledim.

Sonunda, eğitimcinin kılavuzu gözden geçirilmiş ve genişletilmiştir, böylece artık kit apta yer alan tüm alıştırmalar için ayrıntılı çözümler içermektedir.

Peter Linz

BÖLÜM

1

HESAPLAMA TEORİSİNE GİRİŞ

BÖLÜM ÖZETİ

Bu bölüm, sizi gelecek olanlara hazırlamaktadır. Bölüm 1.1'de, sonlu matematikten bazı ana fikirleri gözden geçiriyoruz ve metinde kullanılan notasyonu belirliyoruz. Özellikle önemli olan, kanıt teknikleridir; özellikle de tümevarım ile kanıt ve çelişki ile kanıt.

Bölüm 1.2, kitabın merkezinde yer alan fikirlerin ilk bakışını sunmaktadır: otomata, diller, gramerler, bunların tanımları ve ilişkileri. Sonraki bölümlerde, bu fikirleri genişletecek ve farklı türde otomata ve gramerleri inceleyeceğiz.

Bölüm 1.3'te, bu kavramların bilgisayar bilimindeki rolüne dair bazı basit örneklerle karşılaşlıyoruz; özellikle programlama dilleri, dijital tasarım ve metin işleme alanlarında. Bu konuyu burada fazla uzatmayacağız, çünkü bu kavramların uygulamalarıyla diğer birçok Bilgisayar Bilimleri dersinde karşılaşacaksınız.

Bu kitabın konusu, hesaplama teorisi, birkaç konuyu içermektedir: otomata teorisi, biçimsel diller ve gramerler, hesaplanabilirlik ve karmaşıklık. Birlikte, bu materyal bilgisayar biliminin teorik temelini oluşturmaktadır. Genel olarak, otomata, gramerler ve hesaplanabilirliği, bilgisayarların prensipte neler yapabileceğinin incelenmesi olarak düşünebiliriz; karmaşıklık ise pratikte nelerin yapılabileceğini ele almaktadır. Bu kitapta, neredeyse tamamen ilkinde odaklanıyoruz.

bu endişeleri. Çeşitli otomataları inceleyeceğiz, bunların diller ve gramerlerle nasıl ilişkili olduğunu göreceğiz ve dijital bilgisayarların neler yapabileceğini ve neler yapamayacağını araştıracağız. Bu teorinin birçok kullanımı olmasına rağmen, doğası gereği soyut ve matematiksel bir yapıya sahiptir.

Bilgisayar bilimi pratik bir disiplindir. Bu alanda çalışanlar genellikle teorik spekülasyonlar yerine faydalı ve somut sorunlara belirgin bir tercih gösterirler. Bu, gerçek dünyadan zorlu uygulamalarla ilgilenen bilgisayar bilimi öğrencileri için kesinlikle doğrudur. Teorik sorular, yalnızca iyi çözümler bulmaya yardımcı olduklarında ilgi çeker. Bu tutum uygundur, çünkü uygulamalar olmadan bilgisayarlara olan ilgi az olurdu. Ancak bu pratik yönelim göz önüne alındığında, “teoriyi neden çalışmalıyız?” sorusu akla gelebilir.

İlk cevap, teorinin disiplinin genel doğasını anlamamıza yardımcı olan kavramlar ve ilkeler sağlamasıdır. Bilgisayar bilimi alanı, makine tasarımıyla programlamaya kadar geniş bir özel konu yelpazesini içerir. Gerçek dünyada bilgisayarların kullanımı, başarılı bir uygulama için öğrenilmesi gereken çok sayıda özel ayrıntıyı içerir. Bu, bilgisayar bilimini çok çeşitli ve geniş bir disiplin haline getirir. Ancak bu çeşitliliğe rağmen, bazı ortak temel ilkeler vardır. Bu temel ilkeleri incelemek için, bilgisayarların ve hesaplamanın soyut modellerini oluşturuyoruz. Bu modeller, hem donanım hem de yazılım için ortak olan ve bilgisayarlara çalışırken karşılaştığımız birçok özel ve karmaşık yapının temelini oluşturan önemli özellikleri barındırır. Bu tür modeller, gerçek dünya durumlarına hemen uygulanamayacak kadar basit olsa bile, onları incelemekten elde ettiğimiz içgörüler, belirli gelişmelerin temelini oluşturur. Bu yaklaşım, elbette, bilgisayar bilimlerine özgü değildir. Modellerin inşası, herhangi bir bilimsel disiplinin temel unsurlarından biridir ve bir disiplinin faydalılığı genellikle basit ama güçlü teorilerin ve yasaların varlığına bağlıdır.

İkinci ve belki de o kadar belirgin olmayan bir cevap, tartışacağımız fikirlerin bazı acil ve önemli uygulamalara sahip olduğudur. Dijital tasarım, programlama dilleri ve derleyiciler en belirgin örneklerdir, ancak daha birçok alan vardır. Burada incelediğimiz kavramlar, işletim sistemlerinden desen tanımaya kadar bilgisayar biliminin birçok alanında bir ip gibi uzanır.

Üçüncü cevap, okuyucuyu ikna etmeyi umduğumuz bir cevaptır. Konu, entelektüel olarak uyarıcı ve eğlencelidir. Birçok zorlu, bulmaca benzeri problem sunar ve bu da bazı uykusuz gecelere yol açabilir. Bu, saf anlamıyla problem çözmedir.

Bu kitapta, tüm bilgisayarların ve uygulamalarının merkezindeki özellikleri temsil eden modellere bakacağız. Bir bilgisayarın donanımını modellemek için, bir otomaton (çoğulu, otomata) kavramını tanıtıyoruz. Otomaton, dijital bir bilgisayarın tüm vazgeçilmez özelliklerine sahip bir yapıdır. Girdi alır, çıktı üretir, bazı geçici depolama alanlarına sahip olabilir ve girişi çıktıya dönüştürme kararları verebilir.

çıktı. Bir formal dil, programlama dillerinin genel özelliklerinin bir soyutlamasıdır. Bir formal dil, bir dizi sembol ve bu sembollerin cümleler olarak adlandırılan varlıklara birleştirilebileceği bazı oluşum kurallarından oluşur. Bir formal dil, oluşum kurallarının izin verdiği tüm cümlelerin kümesidir. Burada incelediğimiz bazı formal diller, programlama dillerinden daha basit olmasına rağmen, birçok aynı temel özelliğe sahiptir. Formal dillerden programlama dilleri hakkında çok şey öğrenebiliriz. Son olarak, mekanik hesaplamaların kavramını, algoritma teriminin kesin bir tanımını vererek biçimlendireceğiz ve bu tür mekanik yöntemlerle çözülmesi uygun olan (ve olmayan) problem türlerini inceleyeceğiz. Çalışmamız sırasında, bu soyutlamalar arasındaki yakın bağlantıyı gösterecek ve bunlardan çıkarabileceğimiz sonuçları araştıracağız.

Birinci bölümde, bu temel fikirleri çok geniş bir şekilde ele alarak sonraki çalışmalar için zemin hazırlıyoruz. Bölüm 1.1'de, gerekli olacak matematikten ana fikirleri gözden geçiriyoruz. İnsani sezgi, fikirleri keşfederken sıkça rehberimiz olacak, ancak çıkardığımız sonuçlar titiz argümanlara dayalı olacaktır. Bu, bazı matematiksel araçları içerecektir, ancak gereksinimler geniş değildir. Okuyucunun, terimlerin ve küme teorisi, fonksiyonlar ve ilişkilerin temel sonuçlarının makul bir şekilde anlaşılmasına ihtiyacı olacaktır. Ağaçlar ve grafik yapıları sıkça kullanılacak, ancak etiketli, yönlendirilmiş bir grafiğin tanımından öte çok az şey gerekecektir.

Belki de en katı gereksinim, kanıtları takip etme yeteneği ve doğru matematiksel akıl yürütmenin ne olduğunu anlama yetisidir. Bu, çıkarma, tümevarım ve çelişki ile kanıt gibi temel kanıt tekniklerine aşina olmayı kapsar. Okuyucunun bu gerekli arka plana sahip olduğunu varsayacağız. Bölüm 1.1, kullanılacak bazı ana sonuçları gözden geçirmek ve sonraki tartışmalar için notasyonel bir ortak zemin oluşturmak amacıyla eklenmiştir.

Bölüm 1.2'de, dillerin, gramerlerin ve otomataların merkezi kavramlarına ilk bakışımızı atıyoruz. Bu kavramlar, kitap boyunca birçok özel biçimde karşılaşmaktadırlar. Bölüm 1.3'te, bu genel fikirlerin bazı basit uygulamalarını vererek, bu kavramların bilgisayar biliminde yaygın kullanımlara sahip olduğunu örneklemek istiyoruz. Bu iki bölümdeki tartışma sezgisel olacak, katı değil. Daha sonra, tüm bunları çok daha kesin hale getireceğiz; ancak şu anda, amacımız, ele aldığımız kavramların net bir resmini elde etmektir.

1.1 MATEMATİKSEL ÖN HAZIRLIKLAR VE NOTASYON

Setler

A küme, elemanların bir koleksiyonudur, üyelikten başka herhangi bir yapısı yoktur. x 'in S kümesinin bir elemanı olduğunu belirtmek için $x \in S$ yazarız.

x 'in S 'de olmadığını belirten ifade $x \notin S$ şeklinde yazılır. Bir küme, elemanlarının bazı tanımlarını süslü parantezler içinde kaplayarak belirtilebilir; örneğin, $0, 1, 2$ tam sayılar kümesi aşağıdaki gibi gösterilir

$$S = \{0, 1, 2\}.$$

Belirli bir anlam açık olduğunda üç nokta kullanılır. Böylece, $\{a, b, \dots, z\}$ İngiliz alfabesindeki tüm küçük harfleri temsil ederken, $\{2, 4, 6, \dots\}$ tüm pozitif çift tam sayılar kümesini ifade eder. Gerektiğinde, daha açık bir notasyon kullanırsanız, bu durumda yazarız

$$S = \{i : i > 0, i \text{ çift}\} \quad (1.1)$$

son örnek için. Bunu " S , tüm i 'lerin kümesidir, öyle ki sıfırdan büyüktür ve çifttir" şeklinde okuruz; bu, elbette i 'nin bir tam sayı olduğunu ima eder.

Alışılmış küme işlemleri birleşim ($\cup$), kesişim ($\cap$) ve fark ($-$) olarak tanımlanmıştır

$$\begin{aligned} S_1 \cup S_2 &= \{x : x \in S_1 \text{ veya } x \in S_2\}, \\ S_1 \cap S_2 &= \{x : x \in S_1 \text{ ve } x \in S_2\}, \\ S_1 - S_2 &= \{x : x \in S_1 \text{ ve } x \notin S_2\}. \end{aligned}$$

Diğer temel bir işlem, tamamlamadır. Bir kümenin S tamamlayıcı, $\bar{S}$ ile gösterilir ve S içinde olmayan tüm elemanlardan oluşur. Bu anlamı anlamlı kılmak için, tüm olası elemanların evrensel kümesi U nedir bilmemiz gerekir. Eğer U belirtilmişse, o zaman

$$\bar{S} = \{x : x \in U, x \notin S\}.$$

Hiç elemanı olmayan küme, boş küme veya null küme olarak adlandırılır ve $\emptyset$ ile gösterilir. Bir kümenin tanımından,

$$\begin{aligned} S \cup \emptyset &= S - \emptyset = S, \\ S \cap \emptyset &= \emptyset, \\ \overline{\emptyset} &= U, \\ \overline{\bar{S}} &= S. \end{aligned}$$

Aşağıdaki yararlı kimlikler, DeMorgan yasaları olarak bilinir,

$$\overline{S_1 \cup S_2} = \bar{S}_1 \cap \bar{S}_2, \quad (1.2)$$

$$\overline{S_1 \cap S_2} = \bar{S}_1 \cup \bar{S}_2, \quad (1.3)$$

birçok durumda gereklidir.

Bir küme S_1 , eğer S_1 'in her elemanı S 'in de bir elemanı ise S 'nin bir alt kümesi olarak adlandırılır. Bunu şu şekilde yazarız

$$S_1 \subseteq S.$$

Eğer $S_1 \subseteq S$, ancak S , içinde S_1 'de olmayan bir eleman içeriyorsa, o zaman S_1 , S 'nin bir uygun alt kümesi olarak adlandırılır; bunu şu şekilde yazınız

$$S_1 \subset S.$$

Eğer S_1 ve S_2 ortak bir elemene sahip değilse, yani $S_1 \cap S_2 = \emptyset$, o zaman kümeler ayrık olarak adlandırılır.

Bir küme sonlu olarak adlandırılır eğer sonlu sayıda eleman içeriyorsa; aksi takdirde sonsuz'dur. Sonlu bir kümenin boyutu, içindeki eleman sayısıdır; bu $|S|$ ile gösterilir.

Verilen bir kümenin genellikle birçok alt kümesi vardır. Bir kümenin tüm alt kümelerinin kümesine S , S 'nin güç kümesi olarak adlandırılır ve 2^S ile gösterilir. 2^S 'nin bir küme kümesi olduğunu gözlemleyin.

ÖRNEK 1.1

Eğer S kümesi $\{a, b, c\}$ ise, o zaman güç kümesi

$$2^S = \{\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}.$$

Burada $|S| = 3$ ve $|2^S| = 8$. Bu, genel bir sonucun bir örneğidir; eğer S sonlu ise, o zaman

$$|2^S| = 2^{|S|}.$$

Birçok örneğimizde, bir kümenin elemanları diğer kümelerden gelen elemanların sıralı dizileridir. Bu tür kümelere diğer kümelerin Kartezyen çarpımından. İki kümenin Kartezyen çarpımı, kendisi sıralı çiftlerden oluşan bir küme olduğundan, biz yazırız

$$S = S_1 \times S_2 = \{(x, y) : x \in S_1, y \in S_2\}.$$

ÖRNEK 1.2

Let $S_1 = \{2, 4\}$ and $S_2 = \{2, 3, 5, 6\}$. Then

$$S_1 \times S_2 = \{(2, 2), (2, 3), (2, 5), (2, 6), (4, 2), (4, 3), (4, 5), (4, 6)\}.$$

Bir çiftin elemanlarının yazılış sırasının önemli olduğunu unutmayın.

Çift $(4, 2)$ $S_1 \times S_2$ içindedir, ancak $(2, 4)$ değildir.

Notasyon, iki veya daha fazla kümenin Kartezyen çarpımına açık bir şekilde genişletilir; genel olarak

$$S_1 \times S_2 \times \cdots \times S_n = \{(x_1, x_2, \dots, x_n) : x_i \in S_i\}.$$

Bir küme, onu bir dizi alt kümeye ayırarak bölünebilir. Varsayalım ki $S_1, S_2, \dots, S_n$, verilen bir kümenin alt kümeleridir ve aşağıdakiler geçerlidir:

1. Alt kümeler $S_1, S_2, \dots, S_n$ karşılıklı ayrıdır;
2. $S_1 \cup S_2 \cup \dots \cup S_n = S$;
3. hiçbir S_i boş değildir.

O zaman $S_1, S_2, \dots, S_n$, S 'nin bir bölümü olarak adlandırılır.

Fonksiyonlar ve İlişkiler

Bir fonksiyon, bir kümenin elemanlarına başka bir kümenin benzersiz bir elemanını atayan bir kuraldır.

Eğer f bir fonksiyonu belirtirse, o zaman ilk küme f 'nin tanım kümesi olarak adlandırılır ve ikinci küme f 'nin değer kümesidir.

Biz yazalım

$$f : S_1 \rightarrow S_2$$

tanım kümesinin f 'nin S_1 'nin bir alt kümesi olduğunu ve f 'nin değer kümesinin S_2 'nin bir alt kümesi olduğunu belirtmek için. Eğer f 'nin tanım kümesi S_1 'nin tamamı ise, o zaman f 'nin S_1 'de bir toplam fonksiyon olduğunu söyleriz; aksi takdirde f 'ye kısmi fonksiyon denir.

Birçok uygulamada, ilgili fonksiyonların tanım ve değer kümeleri pozitif tam sayılar kümesindedir. Ayrıca, genellikle bu fonksiyonların davranışları, argümanları çok büyük olduğunda ilgi alanımızdadır. Böyle durumlarda, büyüme oranlarının anlaşılması yeterli olabilir ve yaygın bir büyüklük sırası notasyonu kullanılabilir. Let $f(n)$ ve $g(n)$ pozitif tam sayıların bir alt kümesi olan fonksiyonlar olsun. Eğer yeterince büyük n

$$f(n) \leq c|g(n)|,$$

f en fazla g sırasına sahiptir. Bunu şu şekilde yazalım

$$f(n) = O(g(n)).$$

Eğer

$$|f(n)| \geq c|g(n)|,$$

öyleyse f en az g sırasına sahiptir, bunun için

$$f(n) = \Omega(g(n)).$$

Son olarak, eğer c_1 ve c_2 sabitleri varsa böyle ki

$$c_1|g(n)| \leq |f(n)| \leq c_2|g(n)|,$$

f ve g aynı büyüklük sırasına sahiptir, şu şekilde ifade edilir

$$f(n) = \Theta(g(n)).$$

Bu büyüklük sırası notasyonunda, çarpan sabitleri ve n arttıkça önemsiz hale gelen alt sıralı terimleri göz ardı ederiz.

ÖRNEK 1.3

Let

$$\begin{aligned}f(n) &= 2n^2 + 3n, \\g(n) &= n^3, \\h(n) &= 10n^2 + 100.\end{aligned}$$

Then

$$\begin{aligned}f(n) &= O(g(n)), \\g(n) &= \Omega(h(n)), \\f(n) &= \Theta(h(n)).\end{aligned}$$

Order-of-magnitude notasyonunda, = sembolü eşitlik olarak yorumlanmamalıdır ve büyüklük sıralaması ifadeleri sıradan ifadeler gibi ele alınamaz. Aşağıdaki gibi işlemler

$$O(n) + O(n) = 2O(n)$$

mantıklı değildir ve yanlış sonuçlara yol açabilir. Yine de, doğru bir şekilde kullanıldığında, büyüklük sıralaması argümanları etkili olabilir, bunu sonraki bölümlerde göreceğiz.

Bazı fonksiyonlar bir çiftler kümesi ile temsil edilebilir

$$\{(x_1, y_1), (x_2, y_2), \dots\},$$

burada x_i fonksiyonun tanım kümesinde bir elemandır ve y_i onun değerler kümesinde ki karşılık gelen değerdir. Böyle bir kümenin bir fonksiyonu tanımlaması için, her x_i en fazla bir kez bir çiftin ilk elemanı olarak yer alabilir. Eğer bu sağlanmazsa, küme bir ilişki olarak adlandırılır. İlişkiler, fonksiyonlardan daha genel bir kavramdır: Bir fonksiyonda tanım kümesinin her elemanının değerler kümesinde tam olarak bir ilişkili elemanı vardır; bir ilişkide ise değerler kümesinde birden fazla böyle eleman olabilir.

Bir tür ilişki eşdeğerlik ilişkisidir, eşitlik (kimlik) kavramının genelleştirilmesidir. Bir çiftin (x, y) bir eşdeğerlik ilişkisinde olduğunu belirtmek için

$$x \equiv y \text{ yazarız.}$$

$\equiv$ ile gösterilen bir ilişki, üç kuralı sağlıyorsa eşdeğerlik olarak kabul edilir: yansıma kuralı

$$x \equiv x \text{ her } x \text{ için;}$$

simetri kuralı

$$\text{eğer } x \equiv y \text{ ise, o zaman } y \equiv x;$$

ve geçişkenlik kuralı

eğer $x \equiv y$ ve $y \equiv z$ ise, o zaman $x \equiv z$.

ÖRNEK 1.4

Negatif olmayan tam sayılar kümesinde, bir ilişki tanımlayabiliriz

$$x \equiv y$$

eğer ve yalnızca eğer

$$x \bmod 3 = y \bmod 3.$$

Sonra $2 \equiv 5$, $12 \equiv 0$ ve $0 \equiv 36$. Bu açıkça bir eşdeğerlik ilişkisi olup, refleksivite, simetri ve geçişkenliği sağlar.

Eğer S tanımlı bir eşdeğerlik ilişkisi olan bir küme ise, o zaman bu eşdeğerliği kümeyi eşdeğerlik sınıflarına ayırmak için kullanabiliriz. Her eşdeğerlik sınıfı yalnızca eşdeğer elemanları içerir.

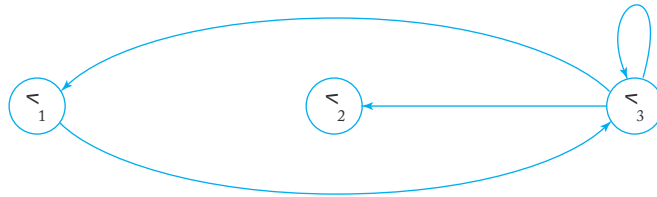
Grafikler ve Ağaçlar

Bir grafik, iki sonlu kümeden oluşan bir yapıdır; bu kümeler $V = \{v_1, v_2, \dots, v_n\}$ köşe kümesi ve $E = \{e_1, e_2, \dots, e_m\}$ kenar kümesidir. Her kenar, V 'den bir köşe çiftidir; örneğin,

$$e_i = (v_j, v_k)$$

v_j 'den v_k 'ye giden bir kenardır. Kenar e_i , v_j için bir giden kenar ve v_k için bir gelen kenar olarak adlandırılır. Bu tür bir yapı aslında yönlendirilmiş bir grafikdir (digraf), çünkü her kenara bir yön (v_j 'den v_k 'ye) ilişkilendiririz. Grafikler etiketlenebilir; etiket, grafiğin parçalarıyla ilişkili bir isim veya diğer bilgilerdir. Hem köşeler hem de kenarlar etiketlenebilir.

Grafikler, köşelerin daireler ve kenarların köşeleri birleştiren oklar la çizgiler olarak temsil edildiği diyagramlarla rahatça görselleştirilir. Köşeleri $\{v_1, v_2, v_3\}$ ve kenarları $\{(v_1, v_3), (v_3, v_1), (v_3, v_2), (v_3, v_3)\}$ olan grafik Şekil 1.1'de gösterilmiştir.



ŞEKİL 1.1

Bir kenar dizisi $(v_i, v_j), (v_j, v_k), \dots, (v_m, v_n)$ v_i 'den v_n 'ye bir yürüyüş olarak adlandırılır. Bir yürüyüşün uzunluğu, başlangıç tepe noktasından son tepe noktasına geçerken geçilen kenarların toplam sayısıdır. Hiçbir kenarın tekrar edilmediği bir yürüyüşe yol denir; bir yol, hiçbir tepe noktasının tekrar edilmediği durumda basit olarak adlandırılır. v_i 'den kendisine tekrar eden kenar olmadan bir yürüyüşe döngü denir ve bu döngünün tabanı v_i 'dir. Eğer döngüde tabandan başka hiçbir tepe noktası tekrar edilmiyorsa, bu durumda basit olduğu söylenir. Şekil 1.1'de, $(v_1, v_3), (v_3, v_2)$ v_1 'den v_2 'ye giden basit bir yoldur. Kenar dizisi $(v_1, v_3), (v_3, v_3), (v_3, v_1)$ bir döngüdür, ancak basit bir döngü değildir. Eğer bir grafiğin kenarları etiketlenmişse, bir yürüyüşün etiketinden bahsedebiliriz. Bu etiket, yol geçildiğinde karşılaşılan kenar etiketlerinin dizisidir. Son olarak, bir tepe noktasından kendisine giden bir kenara döngü denir. Şekil 1.1'de, v_3 tepe noktasında bir döngü bulunmaktadır.

Birçok durumda, iki verilen tepe noktası arasındaki tüm basit yolları (veya bir tepe noktasına dayanan tüm basit döngüleri) bulmak için bir algoritmaya atıfta bulunacağız. Eğer verimlilikle ilgilenmiyorsak, aşağıdaki bariz yöntemi kullanabiliriz. Verilen köşeden başlayarak, yani v_i , tüm çıkan kenarları listeleyin $(v_i, v_k), (v_i, v_l), \dots$. Bu noktada, başlangıç noktası v_i olan tüm birim uzunluktaki yolları elde etmiş oluyoruz. Elde edilen tüm köşeler $v_k, v_l, \dots$ için, inşa ettiğimiz yolda daha önce kullanılan herhangi bir köşeye ulaşmadıkları sürece tüm çıkan kenarları listelleyin. Bunu yaptıktan sonra, başlangıç noktası v_i olan tüm basit iki uzunluktaki yolları elde edeceğiz. Tüm olasılıklar hesaplanana kadar devam ediyoruz. Sadece sonlu sayıda köşe olduğu için, sonunda v_i ile başlayan tüm basit yolları listeleyeceğiz. Bunlardan, istenen köşede bitenleri seçeceğiz.

Ağaçlar, belirli bir grafik türüdür. Ağaç, döngü içermeyen ve kök olarak adlandırılan bir belirgin köşeye sahip olan yönlendirilmiş bir grafikdir; bu kökten her diğer köşeye tam olarak bir yol vardır. Bu tanım, kökün gelen kenarları olmadığını ve bazı köşelerin çıkan kenarları olmadığını ima eder. Bunlara ağacın yaprakları denir. Eğer v_i ile v_j arasında bir kenar varsa, o zaman v_i, v_j 'nin ebeveyni olarak adlandırılır ve v_j, v_i 'nin çocuğu olarak adlandırılır. Her köşeye ilişkili seviye, kökten köşeye giden yolda ki kenar sayısıdır. Ağacın yüksekliği, herhangi bir köşenin en büyük seviye numarasıdır. Bu terimler Şekil 1.2'de gösterilmektedir.

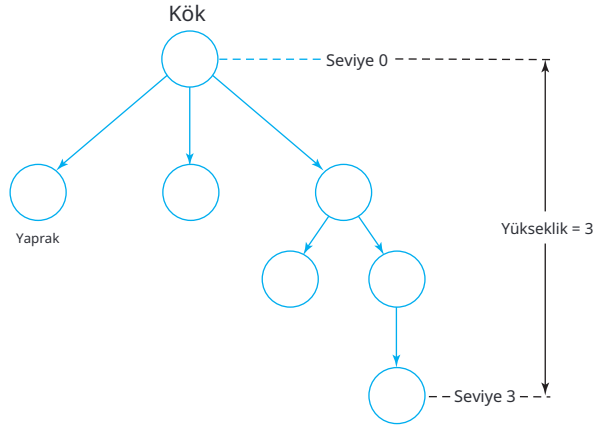
Zaman zaman, her seviyedeki düğümlerle bir sıralama ilişkilendirmek isteriz; bu tür durumlarda sıralı ağaçlardan bahsederiz.

Grafikler ve ağaçlar hakkında daha fazla ayrıntı, çoğu ayrık

matematik kitabında bulunabilir.

Kanıt Teknikleri

Bu metni okumak için önemli bir gereklilik, kanıtları takip etme yeteneğidir. Matematiksel argümanlarda, kabul edilen çıkarımsal akıl yürütme kurallarını kullanırsınız, ve birçok kanıt, bu tür adımların basit bir dizisidir. İki özel kanıt tekniği o kadar sık kullanılır ki, bunları kısaca gözden geçirmek uygundur. Bunlar induksiyonla kanıt ve çelişkiyle kanıttır.



ŞEKİL 1.2

İndüksiyon, bir dizi ifadenin doğruluğunun birkaç özel örneğin doğruluğundan çıkarılabildiği bir tekniktir. Varsayalım ki, doğru olduğun u kanıtlamak istediğimiz bir dizi ifade $P_1, P_2, \dots$ var. Ayrıca, şu durumun da geçerli olduğunu varsayalım:

1. Bazı $k \geq 1$ için, $P_1, P_2, \dots, P_k$ 'nin doğru olduğunu biliyoruz.

2. Problem, herhangi bir $n \geq k$ için, $P_1, P_2, \dots, P_n$

'lerin doğruluğunun P_{n+1} 'nin doğruluğunu ima ettiği şekilde tanımlanmıştır.

O zaman, bu dizideki her ifadenin doğru olduğunu göstermek için indüksiyonu kullanabiliriz.

Bir induksiyon kanıtında, şu şekilde argüman yürütüyoruz: Koşul 1'den, ilk k ifadenin doğru olduğunu biliyoruz. Sonra Koşul 2, bize P_{k+1} 'in de doğru olması gerektiğini söyler. Ama şimdi ilk $k + 1$ ifadenin doğru olduğunu bildiğimize göre, Koşul 2'yi tekrar uygulayarak P_{k+2} 'nin de doğru olması gerektiğini iddia edebiliriz ve bu şekilde devam edebiliriz. Bu argümanı açıkça sürdürmemize gerek yok, çünkü desen açıktır. Akıl yürütme zinciri herhangi bir ifadeye genişletilebilir. Bu nedenle, her ifade doğrudur.

Başlangıç ifadeleri $P_1, P_2, \dots, P_k$ induksiyonun temeli olarak adlandırılır. P_n ile P_{n+1} arasındaki adım **inductive step** olarak adlandırılır. İndüktif adım genellikle **inductive assumption** ile daha kolay hale getirilir; bu, $P_1, P_2, \dots, P_n$ 'nin doğru olduğunu varsaymak ve bu ifadelerin doğruluğunun P_{n+1} 'in doğruluğunu garanti ettiğini savunmaktır. Resmi bir indüktif argümanda, tüm üç kısmı açıkça gösteririz.

ÖRNEK 1.5

İkili ağaç, hiçbir ebeveynin iki çocuktan fazlasına sahip olamayacağı bir ağaçtır. Yüksekliği n olan bir ikili ağacın en fazla 2^n yaprağa sahip olduğunu kanıtlayın.

Kanıt: Eğer yükseklik n olan bir ikili ağacın maksimum yaprak sayısını $l(n)$ ile göstereceksek, o zaman $l(n) \leq 2^n$ olduğunu göstermemiz gerekiyor.

Temel: Açıkça $l(0) = 1 = 2^0$ çünkü yüksekliği 0 olan bir ağacın kök dışında başka düğümü olamaz, yani en fazla bir yaprağı vardır.

İndüktif Varsayım:

$$l(i) \leq 2^i, i = 0, 1, \dots, n. \quad (1.4)$$

İndüktif Adım: Yüksekliği $n + 1$ olan bir ikili ağaç elde etmek için, önceki yüksekliği n olan bir ağaçtan, her bir önceki yaprağın yerine en fazla iki yaprak oluşturabiliriz. Bu nedenle,

$$l(n + 1) = 2 l(n).$$

Şimdi, indüktif varsayımı kullanarak,

$$l(n + 1) \leq 2 \times 2^n = 2^{n+1} \text{ elde ediyoruz.}$$

Böylece, eğer iddiamızın için doğruysa, aynı zamanda $n + 1$ için de doğru olmalıdır. Çünkü herhangi bir sayı olabilir, bu ifade tüm n için doğru olmalıdır. ■

Burada, bu kitapta bir kanıtın sonunu belirtmek için kullanılan sembolü tanıtlıyoruz.

İndüktif akıl yürütme kavramı zorlayıcı olabilir. Programlamada indüksiyon ve özyineleme arasındaki yakın bağlantıyı fark etmek yardımcı olur. Örneğin, herhangi bir pozitif tam sayı olan n için bir fonksiyonun $f(n)$ özyinelemeli tanımı genellikle iki bölümden oluşur. Bir bölüm, $f(n + 1)$ tanımını $f(n)$, $f(n - 1)$, ..., $f(1)$ cinsinden ifade eder. Bu, indüktif adımı karşılar. İkinci kısım, özyinelemeden "kaçış"tır ve bu, $f(1)$, $f(2)$, ..., $f(k)$ değerlerini özyinelemesiz olarak tanımlayarak gerçekleştirilir. Bu, induksiyonun temelini karşılar. İndüksiyonda olduğu gibi, özyineleme, yalnızca birkaç başlangıç değeri verildiğinde ve problemin özyinelemeli doğasını kullanarak, problemin tüm örnekleri hakkında sonuçlar çıkarmamıza olanak tanır.

Bazen, bir sorun zor görünebilir, ta ki ona tam doğru şekilde bakana kadar. Genellikle, ona özyinelemeli bir şekilde bakmak işleri büyük ölçüde basitleştirir.

ÖRNEK 1.6

A set $l_1, l_2, \dots, l_n$ karşılıklı kesişen düz çizgiler, düzlemde birçok ayrı bölgeye ayırır. Tek bir çizgi, düzlemi iki parçaya ayırır, iki çizgi dört bölge oluşturur, üç çizgi yedi bölge yapar ve bu böyle devam eder. Bu, üçe kadar görsel olarak kolayca kontrol edilebilir.

satırlar, ancak satır sayısı arttıkça bir deseni bulmak zorlaşır. Bu sorunu özyinelemeli olarak çözmeye çalışalım.

Şekil 1.3'e bakın, yeni bir satır eklediğimizde ne olduğunu görmek için l_{n+1} mevcut n hatlara. Sol taraftaki bölge l_1 ikiye bölünmüştür, sol taraftaki bölge l_2 de öyle, ve son hatla kadar bu şekilde devam ederiz. Son hatta, sağ taraftaki bölge l_n de öyledir. bölünmüş. Her bir n kesişim, ardından bir yeni bölge üretir, bir tane de sonunda. Yani, eğer $A(n)$ sayısını belirtirsek n çizgiler tarafından üretilen bölgeleri görüyoruz,

$$A(n+1) = A(n) + n + 1, n = 1, 2, \dots,$$

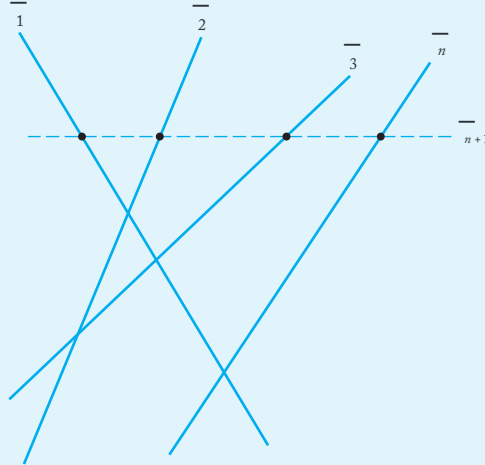
with $A(1) = 2$. From this simple recursion we then calculate $A(2) = 4$, $A(3) = 7$, $A(4) = 11$, and so on.

To get a formula for $A(n)$ and to show that it is correct, we use induction. If we conjecture that

$$A(n) = \frac{n(n+1)}{2} + 1,$$

then

$$\begin{aligned} A(n+1) &= \frac{n(n+1)}{2} + 1 + n + 1 \\ &= \frac{(n+1)(n+2)}{2} + 1 \end{aligned}$$



ŞEKİL 1.3

indüktif adımı haklı çıkarır. Temel kolayca kontrol edilir, argümanı tamamlar.

Bu örnekte, temel, indüktif varsayım ve indüktif adımı tanımlarken biraz daha az resmi olduk, ancak bunlar oradave gereklidir. Sonraki tartışmalarımızın çok resmi hale gelmemesi için, genelde bu ikinci örneğin stilini tercih edeceğiz. Ancak, bir kanıtı takip etmekte veya oluşturmakta zorluk çekiyorsanız, Örnek 1.5'in daha açık formuna geri dönün.

Çelişki ile kanıt, genellikle diğer tüm yöntemler başarısız olduğunda işe yarayan başka bir güçlü tekniktir.

Diyelim ki, bazı ifadelerin doğru olduğunu kanıtlamak istiyoruz.

P doğrudur. O zaman, bir an için P 'nin yanlış olduğunu varsayıyoruz ve bu varsayımın bizi nereye götürdüğüne bakıyoruz. Eğer yanlış olduğunu bildiğimiz bir sonuçta ulaşırsak, başlangıç varsayımını suçlayabiliriz ve P 'nin doğru olması gerektiğine karar verebiliriz. Aşağıda klasik ve şık bir örnek bulunmaktadır.

ÖRNEK 1.7

Rasyonel bir sayı, iki tam sayı n ve m arasındaki oran olarak ifade edilebilen bir sayıdır. n ve m ortak bir çarpana sahip değildir. Rasyonel olmayan bir gerçekte sayıya irrasyonel denir. Gösterin ki $\sqrt{2}$ irrasyoneldir.

Çelişki ile yapılan tüm kanıtlarda olduğu gibi, göstermek istediğimiz şeyin tersini varsayıyoruz. Burada $\sqrt{2}$ 'nin rasyonel bir sayı olduğunu varsayıyoruz, böylece şu şekilde yazılabilir:

$$\sqrt{2} = \frac{m}{n}, \quad (1.5)$$

burada n ve m ortak bir çarpana sahip olmayan tam sayılardır. (1.5)'i yeniden düzenleyerek, şunu elde ederiz:

$$2m^2 = n^2.$$

Buna göre, n^2 çift olmalıdır. Bu, n 'nin çift olduğu anlamına gelir, böylece $n = 2k$ veya

$$2m^2 = 4k^2,$$

ve

$$m^2 = 2k^2.$$

Bu nedenle, m çifttir. Ancak bu, n ve

m 'nin ortak çarpanları olmadığı varsayımımızla çelişiyor. Böylece, m ve n (1.5) içinde var olamaz ve $\sqrt{2}$ rasyonel bir sayı değildir.

Bu örnek, çelişki ile kanıtın özünü sergilemektedir. Belirli bir varsayımda bulunarak, varsayımın veya bazı bilinen gerçeklerin çelişkisine yol açıyoruz. Eğer argümanımızdaki tüm adımlar mantıksal olarak sağlamsa, başlangıç varsayımımızın yanlış olduğu sonucuna varmalıyız.

EGZERSİZLER

1. With $S_1 = \{2, 3, 5, 7\}$, $S_2 = \{2, 4, 5, 8, 9\}$, and $U = \{1 : 10\}$, compute $S_1 \cup S_2$.
2. With $S_1 = \{2, 3, 5, 7\}$ and $S_2 = \{2, 4, 5, 8, 9\}$, compute $S_1 \times S_2$ and $S_2 \times S_1$.
3. For $S = \{2, 5, 6, 8\}$ and $T = \{2, 4, 6, 8\}$, compute $|S \cap T| + |S \cup T|$.
4. What relation between two sets S and T must hold so that $|S \cup T| = |S| + |T|$?
5. Her küme için $S \vee T, S - T = S \cap T$ olduğunu gösterin.
6. DeMorgan yasalarını, Eşitlikler (1.2) ve (1.3) ile kanıtlayın, eğer bir eleman x eşitliğin bir tarafındaki kümede ise, o zaman eşitliğin diğer tarafındaki kümede de olmalıdır.
7. Eğer $S_1 \subseteq S_2$ ise, o zaman $S_2 \subseteq S_1$ olduğunu gösterin.
8. $S_1 = S_2$ olduğunu gösterin, eğer ve yalnızca eğer $S_1 \cup S_2 = S_1 \cap S_2$.
9. Sonuç kümesinin boyutuna göre indüksiyon kullanarak, eğer S sonlu bir küme ise, o zaman $2^{S_1} = 2^{|S_1|}$.
10. S_1 ve S_2 sonlu kümeleri için, eğer $|S_1| = n$ ve $|S_2| = m$ ise, o zaman

$$|S_1 \cup S_2| \leq n + m.$$
11. Eğer S_1 ve S_2 sonlu kümeleri ise, $|S_1 \times S_2| = |S_1| \cdot |S_2|$ olduğunu gösterin.
12. İki küme arasındaki ilişkiyi $S_1 \equiv S_2$ olarak tanımlayın, eğer ve yalnızca $S_1 \equiv S_2$ bu bir eşdeğerlik ilişkisi olduğunu gösterin.
13. Arada bir, birleşim ve kesişim sembollerini toplam işareti Σ ile benzer bir şekilde kullanmamız gerekiyor. Aşağıdaki gibi tanımlıyoruz

$$\bigcup_{p \in \{i, j, k, \dots\}} S_p = S_i \cup S_j \cup S_k \dots$$

birkaç kümenin kesişimi için benzer bir notasyon ile.

Bu notasyon ile genel DeMorgan yasaları şu şekilde yazılır

$$\overline{\bigcup_{p \in P} S_p} = \bigcap_{p \in P} \overline{S_p}$$

ve

$$\overline{\bigcap_{p \in P} S_p} = \bigcup_{p \in P} \overline{S_p}.$$

P sonlu bir küme olduğunda bu kimlikleri kanıtlayın.

14. Gösterin ki

$$S_1 \cup S_2 = \overline{\overline{S_1 \cap S_2}}$$

15. Gösterin ki $S_1 = S_2$ ancak ve ancak

$$(S_1 \cap S_2^c) \cup (\overline{S_1} \cap S_2) = \emptyset.$$

16. Gösterin ki

$$S_1 \cup S_2 - (S_1 \cap S_2) = S_2.$$

17. Dağıtım yasasının geçerli olduğunu gösterin

$$S_1 \cap (S_2 \cup S_3) = (S_1 \cap S_2) \cup (S_1 \cap S_3)$$

küme için geçerlidir.

18. Gösterin ki

$$S_1 \times (S_2 \cup S_3) = (S_1 \times S_2) \cup (S_1 \times S_3).$$

19. S_1 ve S_2 için gerekli ve yeterli koşulları verin, böylece

$$S_1 = (S_1 \cup S_2) - S_2.$$

20. Örnek 1.4'te tanımlanan eşdeğerliği kullanarak küme $\{2, 4, 5, 6, 9, 22, 24, 25, 31, 37\}$ eşdeğerlik sınıflarına ayırın.

21. eğer $f(n) = O(g(n))$ ve $g(n) = O(f(n))$ ise $f(n) = \Theta(g(n))$.

22. Gösterin ki $2^n = O(3^n)$, ancak $2^n \neq \Theta(3^n)$.

23. Aşağıdaki büyüklük sıralaması sonuçlarının geçerli olduğunu gösterin.

(a) $n^2 + 5 \log n = O(n^2)$.

(b) $3^n = O(n!)$.

(c) $n! = O(n^n)$.

24. Gösterin ki $(n^3 - 2n)/(n + 1) = \Theta(n^2)$.

25. Gösterin ki $\frac{n^3}{\log(n+1)} = O(n^3)$ ama $O(n^2)$ değildir.

26. Aşağıdaki argümanda ne yanlış? $x = O(n^4)$, $y = O(n^2)$, bu nedenle $x/y = O(n^2)$.

27. Aşağıdaki argümanda ne yanlış? $x = \Theta(n^4)$, $y = \Theta(n^2)$, bu nedenle $\frac{x}{y} = \Theta(n^2)$.

28. Gösterin ki eğer $f(n) = O(g(n))$ ve $g(n) = O(h(n))$, o zaman $f(n) = O(h(n))$.

29. Gösterin ki eğer $f(n) = O(n^2)$ ve $g(n) = O(n^3)$, o zaman

$$f(n) + g(n) = O(n^3)$$

ve

$$f(n)g(n) = O(n^5).$$

Bu durumda, doğru mu $g(n)/f(n) = O(n)$?

30. Varsayalım ki $f(n) = 2n^2 + n$ ve $g(n) = O(n^2)$. Aşağıdaki argümanda ne yanlış?

$$f(n) = O(n^2) + O(n),$$

so that

$$f(n) - g(n) = O(n^2) + O(n) - O(n^2).$$

Therefore,

$$f(n) - g(n) = O(n).$$

31. Show that if $f(n) = \Theta(\log_2 n)$, then $f(n) = \Theta(\log_{10} n)$.
32. Grafiğin köşeleri $\{v_1, v_2, v_3\}$ ve kenarları $\{(v_1, v_1), (v_1, v_2), (v_2, v_3), (v_3, v_1), (v_2, v_1), (v_3, v_1)\}$ olan bir resim çizin. Tabanı v_1 olan tüm döngüleri sıralayın.
33. Beş köşesi, on kenarı ve döngüsü olmayan bir grafik oluşturun.
34. $G = (V, E)$ herhangi bir grafik olsun. Aşağıdaki iddiayı kanıtlayın: Eğer $v_i \in V$ ve $v_j \in V$ arasında herhangi bir yürüyüş varsa, o zaman bu iki köşe arasında uzunluğu $|V| - 1$ 'den büyük olmayan basit bir yol olmalıdır.
35. Herhangi iki köşe arasında en fazla bir kenar bulunan grafikleri düşünün. Bu koşul altında n köşesi olan bir grafiğin en fazla n^2 kenarı olduğunu gösterin.

36. Gösterin ki

$$\sum_{i=0}^n 2^i = 2^{n+1} - 1.$$

37. Gösterin ki

$$\sum_{i=1}^n \frac{1}{2^i} \leq 2 - \frac{1}{2^n}.$$

38. Her $n \geq 4$ için $2^n < n!$ eşitsizliğini kanıtlayın.
39. The Fibonacci dizisi özyinelemeli olarak tanımlanır

$$f(n+2) = f(n+1) + f(n), n = 1, 2, \dots,$$

ve $f(1) = 1, f(2) = 1$. Gösterin ki

- (a) $f(n) = O(2^n)$,
 (b) $f(n) = \Omega(1.5^n)$.

40. Gösterin ki $\sqrt{8}$ rasyonel bir sayı değildir.
41. Gösterin ki $2 - \sqrt{2}$ irrasyoneldir.
42. Gösterin ki $\sqrt{3}$ irrasyoneldir.
43. Aşağıdaki ifadeleri kanıtlayın veya çürütün.
 (a) Bir rasyonel ve bir irrasyonel sayının toplamı mutlaka irrasyonel olmalıdır.
 (b) İki pozitif irrasyonel sayının toplamı mutlaka irrasyonel olmalıdır.
 (c) Sıfırdan farklı bir rasyonel sayı ile bir irrasyonel sayının çarpımı mutlaka irrasyoneldir.
44. Her pozitif tam sayının asal sayıların çarpımı olarak ifade edilebileceğini gösterin.
45. Sonuç olarak, tüm asal sayıların kümesinin sonsuz olduğunu kanıtlayın.
46. Bir asal çift, iki asal sayının iki ile farklı olduğu bir çifttir. Birçok asal çift vardır, örneğin, 11 ve 13, 17 ve 19. Asal üçlüler, $n \geq 2, n+2, n+4$ olan ve hepsi asal olan üç sayıdır. Tek asal üçlünün (3, 5, 7) olduğunu gösterin.

1.2 ÜÇ TEMEL KAVRAM

Bu kitabın ana temaları üç temel fikirdir: diller, gramerler ve otomata. Çalışmamız sırasında bu kavramlar hakkında ve birbirleriyle olan ilişkileri hakkında birçok sonucu keşfedeceğiz. Öncelikle, terimlerin anlamını anlamalıyız.

Diller

Hepimiz İngilizce ve Fransızca gibi doğal diller kavramına aşinayız. Yine de, çoğumuz “dil” kelimesinin tam olarak ne anlama geldiğini söylemekte zorlanabiliriz. Sözlükler, terimi belirli fikirlerin, gerçeklerin veya kavramların ifadesi için uygun bir sistem olarak gayri resmi bir şekilde tanımlar; bu, semboller ve bunların işlenmesi için kurallar içeren bir kümedir. Bu, bir dilin ne olduğunu anlamamız için sezgisel bir fikir verse de, formel dillerin incelenmesi için yeterli bir tanım değildir. Terim için kesin bir tanıma ihtiyacımız var.

Sonlu, boş olmayan bir sembol kümesi Σ ile başlıyoruz, buna alfabet denir. Bireysel sembollerden diziler oluşturuyoruz, bu diziler alfabetten gelen sembollerin sonlusuzlarıdır. Örneğin, eğer alfabet $\Sigma = \{a, b\}$ ise, o zaman $abab$ ve $aaabbb$ Σ üzerindeki dizilerdir. Az sayıda istisna ile, $a, b, c, \dots$ gibi küçük harfleri Σ 'nin elemanları için ve $u, v, w, \dots$ string isimleri için. Örneğin, yazacağız,

$$w = abaaa$$

adlı dize w belirli bir değere $abaaa$ sahip olduğunu belirtmek için.

İki dize olan w ve v 'nin birleştirilmesi, v 'nin sembollerini w 'nin sağına ekleyerek elde edilen dizedir, yani eğer

$$w = a_1 a_2 \cdots a_n$$

ve

$$v = b_1 b_2 \cdots b_m,$$

o zaman w ve v 'nin birleştirilmesi, wv ile gösterilir,

$$wv = a_1 a_2 \cdots a_n b_1 b_2 \cdots b_m.$$

Bir dizenin tersi, sembollerin ters sırayla yazılmasıyla elde edilir;

Eğer w yukarıda gösterildiği gibi bir dize ise, o zaman ters w^R şudur:

$$w^R = a_n \cdots a_2 a_1.$$

Bir dize olan w 'nin uzunluğu, $|w|$ ile gösterilir ve dizideki sembol sayısını ifade eder. Boş dizeye sıkça atıfta bulunmamız gerekecek; bu, hiç sembol içermeyen bir dizidir. Bu, λ ile gösterilecektir. Aşağıdaki basit ilişkiler

$$|\lambda| = 0,$$

$$\lambda w = w \lambda = w$$

her zaman w için geçerlidir.

Bir w içindeki ardışık sembollerin herhangi bir dizisi, w için bir alt dizi olarak adlandırılır. Eğer

$$w = vu,$$

o zaman v ve u alt dizileri, sırasıyla w için bir ön ek ve bir son ek olarak adlandırılır. Örneğin, eğer $w = abbab$ ise, o zaman $\{\lambda, a, ab, abb, abba, abbab\}$ w için tüm ön eklerin kümesidir, oysa bab, ab, b bazı son ekleridir.

Dizelerin basit özellikleri, uzunlukları gibi, çok sezgisel olupmuhtemelen fazla açıklama gerektirmez. Örneğin, eğer u ve v dizeleri varsa, o zaman birleştirmelerinin uzunluğu, bireysel uzunlukların toplamıdır, yani,

$$|uv| = |u| + |v|. \quad (1.6)$$

Ancak bu ilişki açık olsa da, onu kesin hale getirmek ve kanıtlamak faydalıdır. Bunu yapma teknikleri, daha karmaşık durumlarda önemlidir.

ÖRNEK 1.8

(1.6)'nın herhangi bir u ve v için geçerli olduğunu gösterin. Bunu kanıtlamak için, önce bir dize uzunluğunun tanımına ihtiyacımız var. Bu tanımı,

$$\begin{aligned} |a| &= 1, \\ |wa| &= |w| + 1, \end{aligned}$$

tüm $a \in \Sigma$ ve $w \in \Sigma^*$ üzerindeki herhangi bir dize için. Bu tanım, bir dizinin uzunluğuna dair sezgisel anlayışımızın resmi bir ifadesidir: Tek bir sembolün uzunluğu bir, ve herhangi bir dizinin uzunluğu, ona başka bir sembol eklediğimizde bir artar. Bu resmi tanımla, (1.6)'yı karakterler üzerinden tümevarım ile kanıtlamaya hazırız.

Tanım göre, (1.6) her uzunluktaki u ve uzunluğu 1 olan tüm v için geçerlidir, bu nedenle bir temelimiz var. İndüktif bir varsayım olarak, (1.6)'nın her uzunluktaki u ve uzunluğu 1, 2, ..., n olan tüm v için geçerli olduğunu alıyoruz. Şimdi

uzunluğu $n + 1$ olan herhangi bir v alalım ve onu $v = wa$ olarak yazalım. O zaman,

$$\begin{aligned} |v| &= |w| + 1, \\ |uv| &= |uwa| = |uw| + 1. \end{aligned}$$

İndüktif hipoteze göre (bu geçerlidir çünkü w uzunluğu n 'dir),

$$|uw| = |u| + |w|$$

so that

$$|uv| = |u| + |w| + 1 = |u| + |v|.$$

Bu nedenle, (1.6) tüm u ve uzunluğu $n + 1$ 'e kadar olan tüm v için geçerlidir, böylece indüktif adımı ve argümanı tamamlamış oluyoruz.

If w bir dize ise, o zaman w^n , w 'yi n kez tekrarlayarak elde edilen dizeyi temsil eder. Özel bir durum olarak,

$$w^0 = \lambda,$$

her w için tanımlıyoruz.

Eğer Σ bir alfabeysen, o zaman Σ^* , Σ 'den sıfır veya daha fazla sembolü birleştirilerek elde edilen dize kümesini belirtmek için kullanılır. Küme Σ^* her zaman λ içerir. Boş dizeyi hariç tutmak için,

$\Sigma^+ = \Sigma^* - \{\lambda\}$ olarak tanımlıyoruz.

Σ 'nin sonlu olduğu varsayımı altında, Σ^* ve Σ^+ her zaman sonsuzdur çünkü bu kümelerdeki dizelerin uzunluğu için bir sınır yoktur. Bir dil, çok genel bir şekilde Σ^* 'nin bir alt kümesi olarak tanımlanır. Bir dil L içindeki bir dize, L 'nin bir cümlesi olarak adlandırılacaktır. Bu tanım oldukça geniştir; Σ alfabesi üzerindeki herhangi bir dize kümesi bir dil olarak kabul edilebilir. Daha sonra, belirli dillerin nasıl tanımlanıp açıklanabileceğini inceleyeceğiz; bu, bu oldukça geniş kavrama biraz yapı kazandırmamızı sağlayacaktır. Ancak şu anda, sadece birkaç özel örneğe bakacağız.

ÖRNEK 1.9

$\Sigma = \{a, b\}$. O zaman

$$\Sigma^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, aab, \dots\}.$$

Küme

$$\{a, aa, aab\}$$

Σ üzerinde bir dildir. Sonlu sayıda cümleye sahip olduğu için buna sonlu dil diyoruz. Küme

$$L = \{a^n b^n : n \geq 0\}$$

Σ üzerinde de bir dildir. $aabb$ ve $aaaabbbb$ dizeleri L dilindedir, ancak abb dizisi L içinde değildir. Bu dil sonsuzdur. En ilginç diller sonsuzdur.

Diller kümeler olduğundan, iki dilin birleşimi, kesişimi ve farkı hermen tanımlanır. Bir dilin tamamlayıcı, Σ^* ile ilgili olarak tanımlanır; yani $\bar{L}$, L 'nin tamamlayıcı

$$\bar{L} = \Sigma^* - L.$$

Dilin tersi, tüm dize terslerinin kümesidir, yani,

$$L^R = \{w^R : w \in L\}.$$

İki dilin birleştirilmesi L_1 ve L_2 , L_1 'den herhangi bir elemanı L_2 'den herhangi bir eleman ile birleştirilerek elde edilen tüm dizelerin kümesidir; özellikle,

$$L_1 L_2 = \{xy : x \in L_1, y \in L_2\}.$$

Biz L^n 'yi L kendisiyle n kez birleştirilmiş olarak tanımlıyoruz, özel durumlar ile

$$L^0 = \{\lambda\}$$

ve

$$L^1 = L$$

her dil için L .

Son olarak, bir dilin yıldız-kapatma tanımını yapıyoruz

$$L^* = L^0 \cup L^1 \cup L^2 \dots$$

ve pozitif kapatma olarak

$$L^+ = L^1 \cup L^2 \dots$$

ÖRNEK 1.10

Eğer

$$L = \{a^n b^n : n \geq 0\},$$

then

$$L^2 = \{a^n b^n a^m b^m : n \geq 0, m \geq 0\}.$$

Yukarıdaki n ve m birbirleriyle ilişkili değildir; dizi $aabbbaabbb$

L^2 içindedir.

L 'nin tersini küme notasyonu ile kolayca tanımlayabiliriz

$$L^R = \{b^n a^n : n \geq 0\},$$

ama L veya L^* bu şekilde tanımlamak oldukça daha zordur. Birkaç deneme, karmaşık dillerin tanımlanmasında küme notasyonunun sırtlamalarını hızla size gösterecektir.

Dil bilgileri

Dilleri matematiksel olarak incelemek için, onları tanımlamak için bir mekanizmaya ihtiyacımız var.

Gündelik dil belirsiz ve muğlak olduğundan, İngilizce'deki gayri resmi tanımlar genellikle yetersizdir. Örnek 1.9 ve 1.10'da kullanılan küme notasyonu daha uygundur, ancak sınırlıdır. İlerledikçe, farklı durumlar da yararlı olan birkaç dil tanım mekanizmasını öğreneceğiz.

Burada yaygın ve güçlü bir tanım olan bir dil bilgisi kavramını tanıtıyoruz.

İngilizce için bir dil bilgisi, belirli bir cümlelerin iyi biçimlenip biçimlenmediğini bize söyler. İngilizce dil bilgisinin tipik bir kuralı, “bir cümle e bir isim grubunun ardından bir yüklem içerebilir.” Daha özlü bir şeklide bunu

$$\langle \text{cümle} \rangle \rightarrow \langle \text{isim grubu} \rangle \langle \text{yüklem} \rangle, \text{belir}$$

gin bir yorumla yazarız. Bu, elbette, gerçek cümlelerle başa çıkmak için yeterli değildir. Şimdi, yeni tanıtilan yapılar için tanımlar sağlamalıyız $\langle \text{isim grubu} \rangle$ ve $\langle \text{yüklem} \rangle$. Eğer bunu yaparsak

$$\langle \text{isim grubu} \rangle \rightarrow \langle \text{belirtili artikel} \rangle \langle \text{isim} \rangle,$$

$$\langle \text{yüklem} \rangle \rightarrow \langle \text{fiil} \rangle,$$

ve “bir” ve “the” kelimelerini $\langle \text{belirtili artikel} \rangle$ ile, “çocuk” ve “köpek” kelimelerini $\langle \text{isim} \rangle$ ile, “koşar” ve “yürür” kelimelerini $\langle \text{fiil} \rangle$ ile ilişkilendirirsek, o zaman dil bilgisi bize “bir çocuk koşar” ve “köpek yürür” cümlelerinin doğru biçimlendiğini söyler. Tam bir dil bilgisi verirsek, teorik olarak her doğru cümle e bu şekilde açıklanabilir.

Bu örnek, genel bir kavramın basit kavramlar cinsinden tanımını göstermektedir. En üst düzey kavramla başlıyoruz, burada $\langle \text{cümle} \rangle$, ve onu dilin indirgenemez yapı taşlarına kademeli olarak indiriyoruz. Bu fikirlerin genelleştirilmesi bizi biçimsel dil bilgilerine götürmektedir.

DEFINITION 1.1

Bir dil bilgisi G , bir dördütlü olarak tanımlanır

$$G = (V, T, S, P),$$

burada V , değişkenler olarak adlandırılan nesnelerin sonlu bir kümesidir,

T , terminal semboller olarak adlandırılan nesnelerin sonlu bir kümesidir,

$S \in V$, başlangıç değişkeni olarak adlandırılan özel bir semboldür,

P , üretimlerin sonlu bir kümesidir.

V ve T kümelerinin boş olmayan ve ayrık olduğu varsayılacaktır.

Üretim kuralları bir dil bilgisinin kalbidir; bunlar dil bilgisinin bir dizeyi diğerine nasıl dönüştürdüğünü belirtir ve bu sayede dil bilgisi ile ilişkili bir dili tanımlar. Tartışmamızda tüm üretim kurallarının aşağıdaki biçimde olduğunu varsayacağız

$$x \rightarrow y,$$

burada $x, (V \cup T)^+$ kümesinin bir elemanıdır ve $y, (V \cup T)^*$ kümesindedir. Üretimler şu şekilde uygulanır: Verilen bir dize w aşağıdaki biçimdedir

$$w = u x v,$$

üretim $x \rightarrow y$ bu dize uygulanabilir, ve bunu x ile y değiştirmek için kullanabiliriz, böylece yeni bir dize elde ederiz

$$z = w y v.$$

Bu şu şekilde yazılır

$$w \Rightarrow z.$$

w z türetir veya z w den türetilmiştir deriz. Ardışık dizeler, gramerin üretimlerini rastgele sırayla uygulayarak türetilir. Bir üretim, ne zaman uygulanabilir ise o zaman kullanılabilir ve istenildiği kadar uygulanabilir. Eğer

$$w_1 \Rightarrow w_2 \Rightarrow \dots \Rightarrow w_n,$$

w_1, w_n 'den türetilir ve

$$w_1^* \Rightarrow w_n \text{ yazılır.}$$

$*$, w_1 'den w_n türetmek için belirtilmemiş sayıda adımın (sıfır dahil) atılabilceğini gösterir.

Üretim kurallarını farklı bir sırayla uygulayarak, belirli bir dilbilgisi g genellikle birçok dize üretebilir. Tüm bu terminal dizelerin kümesi, dilbilgisi tarafından tanımlanan veya üretilen dildir.

TANIM 1.2

$G = (V, T, S, P)$ bir dilbilgisi olsun. O zaman küme

$$L(G) = \left\{ w \in T^* : S^* \Rightarrow w \right\}$$

G tarafından üretilen dildir.

eğer $w \in L(G)$, o zaman dizi

$$S \Rightarrow w_1 \Rightarrow w_2 \Rightarrow \dots \Rightarrow w_n \Rightarrow w$$

w cümlesinin bir türevidir. Değişkenler ve terminal semboller içeren $S, w_1, w_2, \dots, w_n$ dizeleri, türetim cümle biçimleri olarak adlandırılır.

ÖRNEK 1.11

Dilbilgisini düşünün

$$G = (\{S\}, \{a, b\}, S, P),$$

P ile verilmiştir

$$S \rightarrow aSb,$$

$$S \rightarrow \lambda.$$

Then

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb,$$

yazabiliriz

$$S^* \Rightarrow aabb.$$

Dize $aabb$, G tarafından üretilen dilde bir cümledir, oysa $aaSbb$ bir cümle biçimidir.

Bir dilbilgisi G , $L(G)$ tamamen tanımlar, ancak dilin dilbilgisinden çok açık bir tanımını almak kolay olmayabilir. Burada, ancak cevap oldukça açıktır.

$$L(G) = \{a^n b^n : n \geq 0\},$$

ve bunu kanıtlamak kolaydır. Kuralın $S \rightarrow aSb$ olduğunu fark edersek, bu özyinelemeli bir kuraldır, bu nedenle bir induksiyonla kanıt önerilmektedir. Öncelikle bütün cümle biçimlerinin

$$w_i = a^i S b^i. \quad (1.7)$$

(1.7)'nin tüm $2i + 1$ veya daha kısa uzunluktaki cümle biçimleri için geçerli olduğunu varsayalım. Başka bir cümle biçimi (cümle olmayan) elde etmek için n yalnızca üretimi uygulayabiliriz $S \rightarrow aSb$. Bu bize

$$a^i S b^i \Rightarrow a^{i+1} S b^{i+1},$$

böylece $2i + 3$ uzunluğundaki her cümle biçimi (1.7) biçiminde de olacaktır. (1.7)'nin açıkça doğru olduğu için $i = 1$, tüm i için induksiyonla geçerlidir. Son olarak, bir cümle elde etmek için üretimi uygulamalıyız $S \rightarrow \lambda$, ve

şunu görüyoruz ki

$$S^* \Rightarrow a^n S b^n \Rightarrow a^n b^n$$

tüm olası türetimleri temsil eder. Böylece, G yalnızca form $a^n b^n$.

Bu formdaki tüm dizelerin türetililebileceğini de göstermemiz gerekiyor. Bu kolaydır; gerekli olduğu kadar $S \rightarrow aSb$ uyguluyoruz, ardından $S \rightarrow \lambda$.

ÖRNEK 1.12

Üreten bir dil bilgisi bulun

$$L = \{ a^n b^{n+1} : n \geq 0 \}$$

Önceki örneğin arkasındaki fikir bu duruma genişletilebilir. Tek yapmamız gereken ekstras üretmektir. Bu, bir üretim ile yapılabilir $S \rightarrow Ab$, diğer üretimlerin seçilmesiyle A önceki örnekteki dili türetebilir. Bu şekilde akıl yürüterek, gramerimizi $G = (\{S, A\}, \{a, b\}, S, P)$ olarak elde ederiz, üretimlerle

$$\begin{aligned} S &\rightarrow Ab, \\ A &\rightarrow aAb, \\ A &\rightarrow \lambda. \end{aligned}$$

Birkaç özel cümle türeterek bunun işe yaradığını kendinize kanıtlayın.

Önceki örnekler oldukça kolaydır, bu nedenle titiz argümanlar gereksiz görünebilir. Ancak genellikle, gayri resmi bir şekilde tanımlanan bir dil için bir gramer bulmak ya da bir gramerle tanımlanan dilin sezgisel bir karakterizasyonunu vermek o kadar da kolay değildir. Belirli bir dilin gerçekten belirli bir gramer G tarafından üretildiğini göstermek için, (a) her $w \in L$ 'nin S 'den G kullanılarak türetililebileceğini ve (b) bu şekilde türetilen her dizinin L içinde olduğunu gösterebilmeliyiz.

ÖRNEK 1.13

$\Sigma = \{a, b\}$ al, ve $n_a(w)$ ve $n_b(w)$ sırasıyla dize w içindeki a ve b sayısını belirtir. O zaman üretimlerle birlikte G grameri

$$\begin{aligned} S &\rightarrow SS, \\ S &\rightarrow \lambda, \\ S &\rightarrow aSb, \\ S &\rightarrow bSa \end{aligned}$$

dili üretir

$$L = \{w : n_a(w) = n_b(w)\}.$$

Bu iddia o kadar açık değil ve ikna edici argümanlar sunmamız gerekiyor.

Öncelikle, G 'nin her cümle biçiminin eşit sayıda olduğunu görmekteyiz a ve b 'nin, çünkü yalnızca a üreten üretimler, yani $S \rightarrow aSb$ ve $S \rightarrow bSa$, aynı anda ab üretir. Bu nedenle, her element of $L(G)$ is in L . Her bir dizeyi görmek biraz daha zor. LG ile türetilebilir.

Problemi genel hatlarıyla inceleyerek başlayalım, çeşitli formlar $w \in L$ alabileceğini düşünelim. Farz edelim w 'a' ile başlıyor ve b . Sonra bu formu alır

$$w = aw_1b,$$

burada w_1 aynı zamanda L içindedir. Bu durumu başlangıçta

$$S \Rightarrow aSb$$

eğer S gerçekten L içinde herhangi bir dize türetiyorsa. Benzer bir argüman eğer wb ile başlıyorsa ve a ile bitiyorsa yapılır. Ancak bu, tüm durumlarda, çünkü bir dizi L aynı sembolle başlayıp bitebilir. Bu tür bir diziyi yazarsak, diyelim $iaabbba$, bunun iki daha kısa dizinin $iaabb$ ve ba birleştirilmesi olarak düşünülebileceğini görürüz, her ikisi de L içindedir. Bu genel olarak doğru mu? Bunun gerçekten böyle olduğunu göstermek için aşağıdaki argümanı kullanabiliriz: Sol uçtan başlayarak, bir a için +1 ve bir b için -1 sayarsak.

Eğer bir dizi w a ile başlıyorsa ve bitiyorsa o zaman sayım, en soldaki sembolden sonra +1 olacaktır ve en sağdaki sembolden hemen önce -1 olacaktır. Bu nedenle, sayım dizinin ortasında bir yerde sıfırdan geçmek zorundadır, bu da böyle bir dizinin

$$w \text{ şeklinde olması gerektiğini gösterir } = w_1w_2,$$

burada hem w_1 hem de w_2 L içindedir. Bu durum, üretim $S \rightarrow SS$ ile ele alınabilir.

Argümanı sezgisel olarak gördüğümüzde, daha titiz bir şekilde ilerlemeye hazırız. Yine indüksiyon kullanıyoruz. Tüm $w \in L$ için varsayalım ki $|w| \leq 2n$, G ile türetilmiştir. Herhangi bir $w \in L$ uzunluğu $2n + 2$ olan. Eğer $w = aw_1b$, o zaman w_1 L içindedir ve $|w_1| = 2n$. Bu nedenle, varsayıma göre,

$$S^* \Rightarrow w_1.$$

Then

$$S \Rightarrow aSb^* \Rightarrow aw_1b = w$$

mümkündür ve $w \in G$ ile türetilebilir. Açıkça, benzer argümanlar eğer $w = bw_1a$. yapılabilir.

Eğer w bu formda değilse, yani aynı

sembolle başlayıp bitiyorsa, o zaman sayım argümanı bize bunun formda olması gerektiğini söyler

$w = w_1w_2$, burada w_1 ve w_2 her ikisi de L içinde ve uzunluğu $2n$ 'den küçük veya eşit olmalıdır. Böylece tekrar görüyoruz ki

$$S \Rightarrow SS^* \Rightarrow w_1S^* \Rightarrow w_1w_2 = w$$

mümkündür.

İndüktif varsayımın açıkça $n=1$ için sağlandığı göz önüne alındığında, bir temelimiz var ve iddia tüm n için doğrudur, argümanımızı tamamlıyoruz.

Normalde, belirli bir dilin onu üreten birçok grameri vardır. Bu gramerler farklı olsa da, bazı anlamlarda eşdeğerdirler. İki gramerin G_1 ve G_2 eşdeğer olduğunu söyleriz eğer aynı dili üretirler, yani eğer

$$L(G_1) = L(G_2).$$

Daha sonra göreceğimiz gibi, iki dilbilgisinin eşdeğer olup olduğunu görmek her zaman kolay değildir.

ÖRNEK 1.14

Dilbilgisini düşünelim $G_1 = (\{A, S\}, \{a, b\}, S, P_1)$, burada P_1 üretimler den oluşmaktadır

$$S \rightarrow aAb \mid \lambda,$$

$$A \rightarrow aAb \mid \lambda.$$

Burada, aynı sol taraflara sahip birkaç üretim kuralının aynı satırda yazıldığı ve alternatif sağ tarafların $\mid$ ile ayrıldığı pratik bir kısayol notasyonu tanıtıyoruz. Bu notasyonda $S \rightarrow aAb \mid \lambda$, iki üretimi $S \rightarrow aAb$ ve $S \rightarrow \lambda$ 'yi temsil eder.

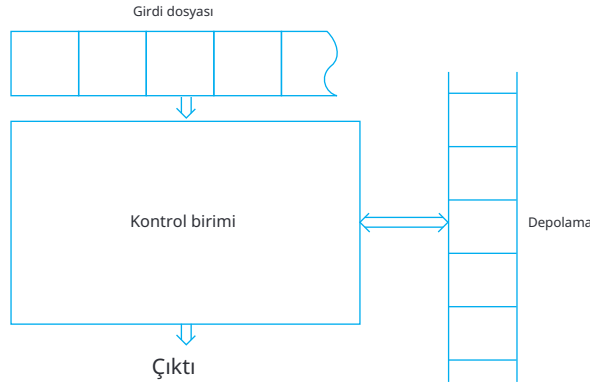
Bu gramer, Örnek 1.11'deki G gramerine eşdeğerdir. Eşdeğerliği kanıtlamak kolaydır, çünkü

$$L(G_1) = \{a^n b^n : n \geq 0\}.$$

Bunu bir alıştırmaya bırakıyoruz.

Otomatlar

Bir otomaton, dijital bir bilgisayarın soyut modelidir. Bu nedenle, her otomaton bazı temel özellikler içerir. Girdi okuma mekanizmasına sahiptir. Girdinin, otomatonun okuyabileceği ancak değiştiremeyeceği, belirli bir alfabede yazılmış bir dize olduğu varsayılacaktır.



ŞEKİL 1.4

Girdi dosyası hücrelere bölünmüştür; her bir hücre bir sembol tutabilir. Girdi mekanizması, girdi dosyasını soldan sağa, bir seferde bir sembol okuyarak okuyabilir. Girdi mekanizması ayrıca girdi dizisinin sonunu (bir dosya sonu durumu algılayarak) tespit edebilir. Otomaton, bir biçimde çıktı üretebilir. Sınırsız sayıda hücreden oluşan geçici bir depolama cihazına sahip olabilir; her hücre bir alfabeden (girdi alfabesiyle aynı olmak zorunda değildir) tek bir sembol tutabilir. Otomaton, depolama hücrelerinin içeriğini okuyabilir ve değiştirebilir. Son olarak, otomatonun, sonlu sayıda iç durumdan birinde olabilen bir kontrol birimi vardır ve bu birim belirli bir şekilde durum değiştirebilir.

Şekil 1.4, genel bir otomatonun şematik temsili göstermektedir.

Bir otomatonun, kesikli bir zaman diliminde çalıştığı varsayılmaktadır. Herhangi bir anda, kontrol birimi bazı iç durumdadır ve giriş mekanizması, giriş dosyasındaki belirli bir sembolü taramaktadır. Kontrol biriminin bir sonraki zaman adımındaki iç durumu, sonraki durum veya geçiş fonksiyonu tarafından belirlenir. Bu geçiş fonksiyonu, mevcut durum, mevcut giriş sembolü ve geçici depolamadaki mevcut bilgi açısından bir sonraki durumu verir. Bir zaman aralığından diğerine geçiş sırasında, çıktı üretilebilir veya geçici depolamadaki bilgi değiştirilebilir. Konfigürasyon terimi, kontrol biriminin, giriş dosyasının ve geçici depolamanın belirli bir durumunu ifade etmek için kullanılacaktır. Otomatonun bir konfigürasyondan diğerine geçişi, bir hareket olarak adlandırılacaktır.

Bu genel model, bu kitapta tartışacağımız tüm otomataları kapsamaktadır. Sınırlı durum kontrolü, tüm özel durumlarda ortak olacaktır, ancak çıktı üretme şekli ve geçici depolamanın doğası nedeniyle farklılıklar ortaya çıkacaktır. Gördüğümüz gibi, geçici depolamanın doğası, farklı türdeki otomataların gücünü belirler.

Sonraki tartışmalar için, belirleyici otomata ile belirsiz otomata arasında ayrım yapmak gerekecektir. Belirleyici bir otomata, her hareketin benzersiz bir şekilde belirlendiği bir otomata olarak tanımlanır.

Mevcut yapılandırmayı bildiğimizde, iç durumu, girişi ve geçici depolamanın içeriğini bildiğimizde, otomatanın gelecekteki davranışını tam olarak tahmin edebiliriz. Belirsiz bir otomata da bu böyle değildir. Her noktada, belirsiz bir otomata birkaç olası hareket yapabilir, bu nedenle yalnızca olası eylemler kümesini tahmin edebiliriz. Farklı türlerdeki belirleyici ve belirsiz otomata arasındaki ilişki, çalışmamızda önemli bir rol oynayacaktır.

Çıktı yanıtı yalnızca basit bir “evet” veya “hayır” ile sınırlı olan bir otomata, bir kabul edici (accepter) olarak adlandırılır. Bir girdi dizesi ile karşılaştığında, bir kabul edici ya a dizeyi kabul eder ya da reddeder. Sembollerin dizilerini çıktı olarak üretebilen daha genel bir otomata ise bir dönüştürücü (transducer) olarak adlandırılır.

EGZERSİZLER

1. w^R içinde kaç tane alt dize aab vardır, burada $w = aabbbab$?
2. Her dize u ve tüm n için $|u \setminus n| = n |u|$ olduğunu göstermek için n üzerinde tümevarım kullanın.
3. Bir dizenin tersinin, yukarıda gayri resmi olarak tanımlandığı gibi, daha kesin bir şekilde tanımlanması, aşağıdaki özyinelemeli kurallarla yapılabilir:

$$\begin{aligned} \emptyset^R &= \emptyset, \\ (wa)^R &= aw^R, \end{aligned}$$

her $a \in \Sigma, w \in \Sigma^*$ için. Bunu kullanarak şunu kanıtlayın:

$$(uv)^R = v^R u^R,$$

her $u, v \in \Sigma^+$ için.

4. Şunu kanıtlayın: $(w^R)^R = w$ her $w \in \Sigma^*$ için.
5. Let $L = \{ab, aa, baa\}$. Aşağıdaki dizelerden hangileri L^* içindedir: $abaabaaabaa$, $aaaabaaaa$, $baaaaabaaaab$, $baaaaabaa$? Hangi dizeler L^4 içindedir?
6. Let $\Sigma = \{a, b\}$ ve $L = \{aa, bb\}$. Kümeler notasyonu kullanarak L^* 'yi tanımlayın.
7. Let L boş olmayan bir alfabede herhangi bir dil olsun. Gösterin ki L ve L^* her ikisi de sonlu olamaz.
8. Hangi diller için $L^* = (L^*)^*$ vardır?
9. Şunu kanıtlayın:

$$(L_1 L_2)^R = L_2^R L_1^R$$

tüm diller için L_1 ve L_2 .

10. Gösterin ki $(L^*)^* = L^*$ tüm diller için.
11. Aşağıdaki iddiaları kanıtlayın veya çürütün.
 - (a) $(L_1 \cup L_2)^R = L_1^R \cup L_2^R$ tüm diller L_1 ve L_2 için.
 - (b) $(L^R)^* = (L^*)^R$ tüm diller L için.

12. Dil için bir dilbilgisi bulun $L = \{a^n, n \text{ çift}\}$.
13. Bir dil için bir dilbilgisi bulun $L = \{a^n, \text{buradan çift ve } n > 3\}$.
14. Σ için dilbilgileri bulun $\Sigma = \{a, b\}$ bu kümeleri üreten
- (a) tam olarak iki a olan tüm dizeler.
 - (b) en az iki a olan tüm dizeler.
 - (c) en fazla üç a olan tüm dizeler.
 - (d) en az üç a olan tüm dizeler.
 - (e) a ile başlayan ve b ile biten tüm dizeler.
 - (f) en çift sayıda b olan tüm dizeler.
- Her durumda, verdiğiniz dilin gerçekten belirtilen dili ürettiğine dair ikna edici argümanlar verin.
15. Dil üretimlerini içeren dilin basit bir tanımını verin.

$$\begin{aligned} S &\rightarrow aaA, \\ A &\rightarrow bS, \\ S &\rightarrow \lambda. \end{aligned}$$

16. Bu üretilere sahip dil neyi üretir?

$$\begin{aligned} S &\rightarrow Aa, \\ A &\rightarrow B, \\ B &\rightarrow Aa. \end{aligned}$$

17. Let $\Sigma = \{a, b\}$. Aşağıdaki dillerden her biri için, onu üreten bir dil bilgisi bulun.

- (a) $L_1 = \{a^n b^m : n \geq 0, m \leq n\}$.
- (b) $L_2 = \{a^{3n} b^{2n} : n \geq 2\}$.
- (c) $L_3 = \{a^{n+3} b^n : n \geq 2\}$.
- (d) $L_4 = \{a^n b^{n-2} : n \geq 3\}$.
- (e) $L_1 L_2$.
- (f) $L_1 \cup L_2$.
- (g) L^3 .
- (h) L_1^* .
- (i) $L_1 - \overline{L_4}$.

18. $\Sigma = \{a\}$ için aşağıdaki dillerin gramerlerini bulun.

- (a) $L = \{w : |w| \bmod 3 > 0\}$.
- (b) $L = \{w : |w| \bmod 3 = 2\}$.
- (c) $w = \{w : |w| \bmod 5 = 0\}$.

19. Dili üreten bir gramer bulun.

$$L = \{ ww^R : w \in \{a, b\}^+ \}.$$

Cevabınız için tam bir gerekçe verin.

20. Üretilen dilde üç dize bulun

$$S \rightarrow aSb|bSa|a.$$

21. Örnek 1.14'teki argümanları tamamlayın, L (G1) verilen dili gerçekten üretebildiğini gösterin.

22. Gramerin $G = (\{S\}, \{a, b\}, S, P)$, üretimlerle

$$S \rightarrow SS|SSS|aSb|bSa|\lambda,$$

Örnek 1.13'teki gramerle eşdeğerdir.

23. Gramerlerin gösterilmesi

$$S \rightarrow aSb|ab|\lambda$$

ve

$$S \rightarrow aaSbb|aSb|ab|\lambda$$

eşdeğerdir.

24. Gramerlerin gösterilmesi

$$S \rightarrow aSb|bSa|SS|a$$

ve

$$S \rightarrow aSb|bSa|a$$

eşdeğer değildir.

1.3 BİRKAÇ UYGULAMA*

Resmi dillerin ve otomataların soyut ve matematiksel doğasını vurgularken, bu kavramların bilgisayar bilimlerinde yaygın uygulamalara sahip olduğu ve aslında birçok uzmanlık alanını bağlayan ortak bir tema olduğu ortaya çıkmaktadır. Bu bölümde, okuyucuya burada incelediğimiz şeyin sadece bir soyutlamalar koleksiyonu olmadığını, aynı zamanda birçok önemli, gerçek problemi anlamamıza yardımcı olan bir şey olduğunu göstermek için bazı basit örnekler sunuyoruz.

Resmi diller ve gramerler, programlama dilleri ile bağlantılı olarak yaygın bir şekilde kullanılmaktadır. Programlamamızın çoğunda, yazdığımız dilin daha az veya daha çok sezgisel bir anlayışıyla çalışıyoruz. Bununla birlikte, alışılmadık bir özellik kullanırken, çoğu zaman, çoğu yerde bulunan sözdizimi diyagramları gibi kesin tanımlara başvurmamız gerekebilir.

programlama metinleri. Eğer bir derleyici yazıyorsak veya bir programın doğruluğu hakkında düşünmek istiyorsak, dilin kesin bir tanımına neredeyse her adım da ihtiyaç vardır. Programlama dillerinin kesin bir şekilde tanımlanabileceği yollar arasında, gramerler belki de en yaygın olarak kullanılanlardır.

Pascal veya C gibi tipik bir dili tanımlayan gramerler çok kapsamlıdır. Örnek olarak, daha büyük bir dilin parçası olan daha küçük bir dili ele alalım.

ÖRNEK 1.15

C dilindeki değişken tanımlayıcıları için kurallar şunlardır:

1. Bir tanımlayıcı, harfler, rakamlar ve alt çizgilerin bir dizisidir.
2. Bir tanımlayıcı bir harf veya alt çizgi ile başlamalıdır.
3. Tanımlayıcılar büyük ve küçük harfleri kabul eder.

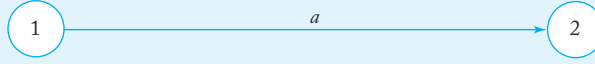
Resmi olarak, bu kurallar bir dilbilgisi ile tanımlanabilir.

$$\begin{aligned} \langle id \rangle &\rightarrow \langle letter \rangle \langle rest \rangle | \langle undrscr \rangle \langle rest \rangle \\ \langle rest \rangle &\rightarrow \langle letter \rangle \langle rest \rangle | \langle digit \rangle \langle rest \rangle | \langle undrscr \rangle \langle rest \rangle | \lambda \\ \langle letter \rangle &\rightarrow a | b | \dots | z | A | B | \dots | Z \\ \langle digit \rangle &\rightarrow 0 | 1 | \dots | 9 \\ \langle undrscr \rangle &\rightarrow - \end{aligned}$$

Bu dilbilgisinde, değişkenler $\langle id \rangle$, $\langle letter \rangle$, $\langle digit \rangle$, $\langle undrscr \rangle$, ve $\langle rest \rangle$. Harfler, rakamlar ve alt çizgi terminaldir. A derivasyonu $a0$ dır

$$\begin{aligned} \langle id \rangle &\Rightarrow \langle letter \rangle \langle rest \rangle \\ &\Rightarrow a \langle rest \rangle \\ &\Rightarrow a \langle digit \rangle \langle rest \rangle \\ &\Rightarrow a0 \langle rest \rangle \\ &\Rightarrow a0. \end{aligned}$$

Programlama dillerinin gramerler aracılığıyla tanımlanması yaygın ve çok faydalıdır. Ancak, genellikle daha pratik olan alternatifler de vardır. Örneğin, bir dili bir kabul edici ile tanımlayabiliriz, kabul edilen her dizeyi dilin bir parçası olarak alarak. Bunu kesin bir şekilde konuşmak için, bir otomatonun daha resmi bir tanımını vermemiz gerekecek. Bunu kısa süre içinde yapacağız; şu anda, daha sezgisel bir şekilde ilerleyelim.



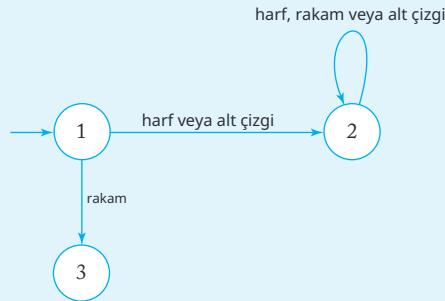
ŞEKİL 1.5

Bir otomaton, köşelerin iç durumları ve kenarların geçişleri gösterdiği bir grafikte temsil edilebilir. Kenarlardaki etiketler, geçiş sırasında ne olduğunu (girdi ve çıktı açısından) gösterir. Örneğin, Şekil 1.5, girdi sembolü a olduğunda Durum 1'den Durum 2'ye geçiş temsil eder. Bu sezgisel resmi akılda tutarak, C tanımlayıcılarını tanımlamanın başka bir yoluna bakalım.

ÖRNEK 1.16

Şekil 1.6, tüm geçerli C tanımlayıcılarını kabul eden bir otomattır. Bazı yorumlamalar gereklidir. Otomatonun başlangıçta Durum 1'de olduğunu varsayıyoruz; bunu, bu duruma bir ok çizerek gösteriyoruz (herhangi bir köşeden başlamayan). Her zamanki gibi, incelenen dize soldan sağa, her adımda bir karakter okunarak ilerlenir. İlk sembol bir harf veya alt çizgi olduğunda, otomaton Durum 2'ye geçer; bundan sonra dizinin geri kalanı önemsizdir. Bu nedenle, Durum 2 kabul eden "evet" durumunu temsil eder. Tersine, eğer ilk sembol bir rakam ise, otomaton Durum 3'e, "hayır" durumuna geçer ve orada kalır. Çözümümüzde, harfler, rakamlar veya

alt çizgiler dışında başka bir girişin mümkün olmadığını varsayıyoruz.



ŞEKİL 1.6

Bir programı bir dilden diğerine çeviren derleyiciler ve diğer çeviriciler, bu örneklerde değinilen fikirlerden geniş ölçüde yararlanmaktadır. Programlama dilleri, gramerler aracılığıyla kesin bir şekilde tanımlanabilir,

örnek 1.15'te olduğu gibi, hem gramerler hem de otomata, belirli bir kod parçasının bir programlama dilinin koşullarını karşıladığını kabul etme karar süreçlerinde temel bir rol oynamaktadır. Yukarıdaki örnek, bunun nasıl yapıldığına dair ilk ipucunu vermektedir; sonraki örnekler bu gözlemi genişletecektir.

Dönüştürücüler Ek A'da kısaca tartışılacaktır; aşağıdaki örnek bu konuyu önizlemektedir.

ÖRNEK 1.17

İkili toplayıcı, herhangi bir genel amaçlı bilgisayarın ayrılmaz bir parçasıdır.

Böyle bir toplayıcı, sayıları temsil eden iki bit dizisini alır ve bunların toplamını çıktı olarak üretir. Basitlik açısından, yalnızca pozitif tam sayılarla uğraştığımızı ve $x = a_0a_1 \dots a_n$

şeklinde bir temsil kullandığımızı varsayalım.

tam sayıyı temsil eder

$$v(x) = \sum_{i=0}^n a_i 2^i.$$

Bu, tersine çevrilmiş olan yaygın ikili temsildir.

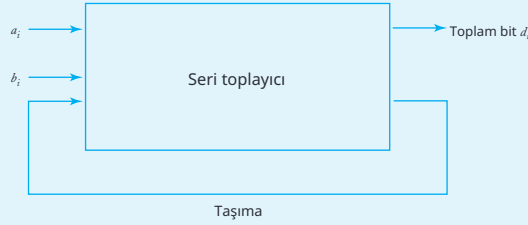
Bir seri toplayıcı, iki böyle sayıyı $x = a_0a_1 \dots a_n$ ve $y = b_0b_1 \dots b_n$, soldan başlayarak bit bit işler. Her bit eklemesi toplam için bir rakam ve bir sonraki daha yüksek

konum için bir taşıma rakamı oluşturur. Bir ikili toplama tablosu (Şekil 1.7) süreci özetler.

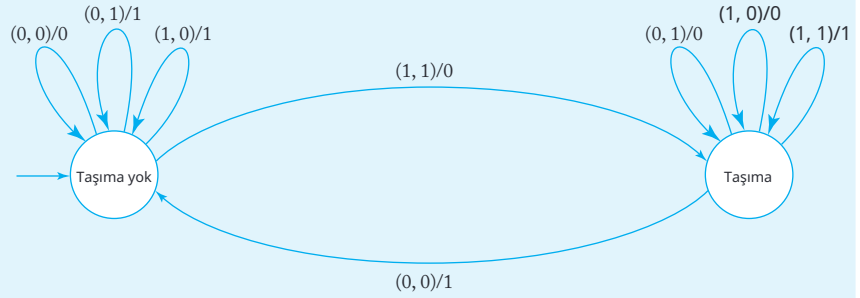
İlk bilgisayarları incelediğimizde gördüğümüz türde bir blok diyagramı Şekil 1.8'de verilmiştir. Bu, bir toplayıcının iki biti kabul eden ve bunların toplam bitini ve olası bir taşıma üreten bir kutu olduğunu bize söyler. Ne yaptığını tanımlar, ancak iç işleyişi hakkında çok az bilgi verir. Bir otomaton (şimdi bir dönüştürücü) bunu çok daha açık hale getirebilir.

		a_i	
		0	1
b_i	0	0 Taşıma yok	1 Taşıma yok
	1	1 Taşıma yok	0 Taşıma

ŞEKİL 1.7



ŞEKİL 1.8



ŞEKİL 1.9

Transdüserin girişi bit çiftleridir (a_i, b_i) ; çıkış bit toplamı d_i olacaktır. Yine, otomatonu bir grafikte temsil ediyoruz şimdi kenarları etiketleyerek $(a_i, b_i)/d_i$. Bir adımdan diğerine taşınan değer otomaton tarafından "taşıma" ve "taşıma yok" olarak etiketlenmiş iki iç durumla hatırlanır. Başlangıçta, transdüser "taşıma yok" durumunda olacaktır. Bir bit çifti $(1, 1)$ ile karşılaşana kadar bu durumda kalacaktır; bu, otomatonu "taşıma" durumuna geçiren bir taşıma oluşturacaktır. Bir taşımanın varlığı, bir sonraki bit çifti okunduğunda dikkate alınır. Seri toplayıcının tam resmi Şekil 1.9'da verilmiştir. Bunun doğru çalıştığını kendinize kanıtlamak için birkaç örnekle bunu takip edin.

Bu örneğin gösterdiği gibi, otomaton, bir devrenin çok yüksek seviyeli, işlevsel tanımı ile transistörler, kapılar ve flip-floplar aracılığıyla mantıksal uygulanması arasında bir köprü işlevi görür. Otomaton, karar mantığını açıkça gösterirken, aynı zamanda kesin matematiksel manipülasyona uygun olacak kadar formaldır. Bu nedenle, dijital tasarım yöntemleri otomata teorisinden gelen kavramlara büyük ölçüde dayanır.

EGZERSİZLER

1. Şifrelerin genellikle çok az kısıtlaması olmasına rağmen, genellikle tamamen serbest değildirler. Diyelim ki belirli bir sistemde, şifreler rastgele uzunlukta olabilir ancak en az bir harf, $a-z$ ve bir rakam 0-9 içermelidir. Böyle yasal şifrelerin kümesini üreten bir dil bilgisi oluşturun.
2. Diyelim ki bazı programlama dillerinde, sayılar aşağıdaki gibi kısıtlanmıştır:
 - (a) Bir sayı işaretli veya işaretsiz olabilir.
 - (b) Değer alanı, ondalık
nokta ile ayrılmış iki boş olmayan kısımdan oluşur.
 - (c) İsteğe bağlı bir üstel alan vardır. Varsa, bu alane harfini içermeli ve ardından işaretli iki basamaklı bir tam sayı gelmelidir.

Böyle sayılar için bir dil bilgisi seti tasarlayın.
3. C dilindeki tam sayı kümesi için bir dil bilgisi verin.
4. C dilinde tam sayılar için bir kabul edici tasarlayın.
5. C'deki tüm reel sabitleri üreten bir dil bilgisi verin.
6. Belirli bir programlama dilinin yalnızca bir harfle başlayan, en az bir ancak en fazla üç rakam içeren ve herhangi bir sayıda harf bulundurmasına izin verdiğini varsayalım. Bu tür bir tanımlayıcı seti için bir dil bilgisi ve bir kabul edici verin.
7. Örnek 1.15'teki dil bilgisini, tanımlayıcıların aşağıdaki kuralları karşılayacak şekilde değiştirin:
 - (a) C kuralları, ancak bir alt çizgi en soldaki sembol olamaz.
 - (b) C kuralları, en fazla bir alt çizgi olabileceği dışında.
 - (c) C kuralları, bir alt çizginin bir rakamla takip edilemeyeceği dışında.
8. Roma rakamları sisteminde, sayılar alfabe $\{M, D, C, L, X, V, I\}$ ile temsil edilir. Sadece düzgün biçimlendirilmiş Roma rakamları olan bu tür dizeleri kabul eden bir kabul edici tasarlayın. Basitlik açısından, dokuz sayısını temsil eden "çıkarma" kuralını, bunun yerine $VIIII$ kullanan bir toplama eşdeğeri ile değiştirin.
9. Bir otomatonun ayrık zaman adımları çerçevesinde çalıştığını varsaydık, ancak bu yön, sonraki tartışmamız üzerinde çok az etkiye sahiptir. Ancak dijital tasarımda, zaman unsuru önemli bir anlam kazanır. Bilgisayarın farklı bölümlerinden gelen sinyalleri senkronize etmek için gecikme devreleri gereklidir. Bir birim gecikme dönüştürücüsü, girişi (sürekli bir sembol akışı olarak görülen) bir zaman birimi sonra basitçe yeniden üreten bir cihazdır. Özellikle, eğer dönüştürücü, zaman t 'de girdi olarak bir sembol a okursa, o zaman

o sembolü zaman $t + 1$ 'de çıktı olarak yeniden üretir. Zaman $t = 0$ 'da, dönüştürücü hiçbir şey çıkartmaz. Bunu, dönüştürücünün girdi $a_1 a_2 \dots$ 'yi çıktı $\lambda a_1 a_2 \dots$ 'ye çevirdiğini belirterek gösteriyoruz.

Bir birim gecikme dönüştürücüsünün nasıl tasarlanabileceğini gösteren bir grafik çizin $\Sigma = \{a, b\}$.

10. Bir n birim gecikme dönüştürücüsü, girişi n zaman birimi sonra yeniden üreten bir dönüştürücüdür; yani, giriş $a_1 a_2 \dots, \lambda n a_1 a_2 \dots$ olarak çevrilir, bu da şu anlama gelir ki, dönüştürücü ilk n zaman dilimi için hiçbir çıktı üretmez.
 - (a) $\Sigma = \{a, b\}$ üzerinde iki birim gecikme dönüştürücüsü inşa edin.
 - (b) Bir n birim gecikme dönüştürücüsünün en az $|\Sigma|^n$ duruma sahip olması gerektiğini gösterin.
11. Bir pozitif tam sayıyı temsil eden bir ikili dizinin iki'nin tamamı, önce her bitin tamamlanması ve ardından en düşüğe bitine bir eklenmesiyle oluşur. Bit dizilerini iki'nin tamamına çeviren bir dönüştürücü tasarlayın, ikili sayının Örnek 1.17'de gösterildiği gibi, daha düşük sıralı bitlerin dizinin son tarafında olduğunu varsayarak.
12. Bir ikili diziyi sekizli sayıya dönüştüren bir dönüştürücü tasarlayın. Örneği n , bit dizisi 001101110 çıktıyı 156 üretmelidir.
13. Let $a_1 a_2 \dots$ bir giriş bit dizisi olsun. Her üç bitlik alt dizinin paritesini hesaplayan bir dönüştürücü tasarlayın. Özellikle, dönüştürücü çıktı üretmelidir

$$\pi_1 = \pi_2 = 0,$$

$$\pi_i = (a_{i-2} + a_{i-1} + a_i) \bmod 2, i = 3, 4, \dots$$

Örneğin, girdi 110111, 000001 üretmelidir.

14. Bit dizilerini kabul eden bir dönüştürücü tasarlayın $a_1 a_2 a_3 \dots$ ve her üç ardışık bitin ikili değerini beş ile mod alarak hesaplayın. Daha spesifik olarak, dönüştürücü $m_1, m_2, m_3, \dots$ üretmelidir, burada

$$m_1 = m_2 = 0,$$

$$m_i = (4a_i + 2a_{i-1} + a_{i-2}) \bmod 5, i = 3, 4, \dots$$

15. Dijital bilgisayarlar genellikle tüm bilgileri bit dizileri ile temsil eder, bazı şifreleme türleri kullanarak. Örneğin, karakter bilgisi iyi bilinen ASCII sistemi kullanılarak şifrelenebilir.

Bu alıştırma için, iki alfabe $\{a, b, c, d\}$ ve $\{0, 1\}$ olarak düşünün, ve birinci alfabeden ikinci alfabe $a \rightarrow 00, b \rightarrow 01, c \rightarrow 10, d \rightarrow 11$ ile tanımlanan bir şifreleme oluşturun. $\{0, 1\}$ üzerindeki dizeleri orijinal mesaja çözmek için bir dönüştürücü inşa edin. Örneğin, girdi 010011 çıktıda bad üretmelidir.

16. Let x and y be two positive binary numbers. Design a transducer whose output is $\max(x, y)$.

BÖLÜM

2

SONLU OTOMATLAR

BÖLÜM ÖZETİ

Bu bölümde, ilk basit otomatonumuz olan bir sonlu durum kabul edeni ile karşılaşıyoruz. Sonludur çünkü yalnızca sonlu bir iç durum kümesine sahiptir ve başka bir belleği yoktur. Bir kabul eden olarak adlandırılır çünkü dizeleri işler ve ya kabul eder ya da reddeder, bu nedenle onu basit birşablon tanıma mekanizması olarak düşünebiliriz.

Deterministik sonlu kabul edici (dfa) ile başlıyoruz. Sıfat “deterministik” otomatonun her zaman yalnızca bir seçeneği olduğunu belirtir. Belirli bir dil türünü tanımlamak için dfa’ları kullanıyoruz, “düzenli dil” olarak adlandırılan ve böylece otomata ile diller arasında ilk bağlantıyı kuruyoruz, bu sonraki tartışmalarda tekrarlanan bir tema.

Sonra, belirsiz otomata (nfa) tanıtıyoruz. Belirli durumlarda, bir nfa birden fazla seçeneğe sahip olabilir ve bu nedenle bir seçim yapıyormuş gibi görünebilir. Deterministik bir dünyada, ya da en azından bilgisayarların deterministik dünyasında, belirsizliğin rolü nedir? Bazen bir nfa’nın en iyi seçimi yapabileceğini söylesek de, bu sadece bir sözel kolaylıktır. Belirsizliğin görselleştirmenin daha iyi bir yolu, bir nfa’nın tüm seçenekleri keşfettiğini (örneğin, bir arama ve geri izleme yöntemiyle) ve tüm seçenekler analiz edilene kadar hiçbir karar vermediğini düşünmektir. Belirsizliğin tanıtmanın ana nedeni, birçok problemin çözümünü basitleştirmesidir.

İki kabul edicinin eşdeğer olduğunu, aynı dili kabul ettiklerinde söyleriz. O zaman nfa'ların sınıfının dfa'ların sınıfına eşdeğer olduğunu da söyleyebiliriz çünkü her nfa için, eşdeğer bir dfa bulabiliriz. Farklı türdeki yapıların eşdeğerliliği, bu kitapta tekrar eden bir temadır.

Herhangi bir düzenli dil için birçok eşdeğer dfa vardır, bu nedenle en az sayıda iç duruma sahip bir dfa bulmak pratik açıdan önemlidir. Bölüm 2.4'te bu nasıl yapılabileceği gösterilmektedir.

İlk bölümde hesaplama ile ilgili temel kavramlara, özellikle otomata tartışmasına kısa ve gayri resmi bir giriş yapıyoruz. Bu noktada, bir otomatonun ne olduğunu ve nasıl bir grafikte temsil edilebileceğini yalnızca genel bir anlayışa sahibiz. İlerlemek için daha kesin olmalı, resmi tanımlar sağlamalı ve titiz sonuçlar geliştirmeye başlamalıyız. Son bölümde tanıtilen genel şemanın basit, özel bir durumu olan sonlu kabul edicilerle başlıyoruz. Bu tür bir otomaton, geçici depolama alanına sahip olmamasıyla karakterizedir. Bir girdi dosyası yeniden yazılamayacağı için, bir sonlu otomatonun hesaplama sırasında "hatırlama" kapasitesi ciddi şekilde sınırlıdır. Kontrol biriminde belirli bir duruma yerleştirerek sınırlı miktarda bilgi saklanabilir. Ancak bu tür durumların sayısı sonlu olduğundan, bir sonlu otomaton yalnızca her zaman saklanacak bilginin kesin olarak sınırlı olduğu durumlarla başa çıkabilir. Örnek 1.16'daki otomaton, bir sonlu kabul edicinin bir örneğidir.

2.1 BELİRLİ SONLU KABUL EDİCİLER

Detaylı olarak incelediğimiz ilk otomaton türü, işlemlerinde belirli olan sonlu kabul edicilerdir. Belirli kabul edicilerin kesin bir resmi tanımla başlıyoruz.

Şekil 1.6 ve 1.9, bazı ortak özelliklere sahip iki basit otomata temsil etmektedir:

- Her ikisi de sonlu sayıda iç duruma sahiptir.
- Her ikisi de bir dizi sembolden oluşan bir girdi dizesini işler.
- Her ikisi de mevcut duruma ve mevcut girdi sembolüne bağlı olarak bir durumdan diğerine geçiş yapar.
- Her ikisi de bazı çıktılar üretir, ancak biraz farklı bir biçimde. Şekil 1.6'daki otomaton yalnızca girişi kabul eder veya reddeder; Şekil 1.9'daki otomaton bir çıktı dizesi üretir.

Ayrıca, her iki otomatonun da her adımda tek bir iyi tanımlanmış geçişe sahip olduğunu unutmayın. Bu özelliklerin tümü aşağıdaki tanıma dahil edilmiştir.

Belirleyici Kabul Ediciler ve Geçiş Grafikleri

TANIM 2.1

Bir belirleyici sonlu kabul edici veyadfa, beşli ile tanımlanır

$$M = (Q, \Sigma, \delta, q_0, F),$$

burada

- Q sonlu bir iç durumlar kümesidir,
- Σ , girdi alfabası olarak adlandırılan sonlu bir semboller kümesidir,
- $\delta : Q \times \Sigma \rightarrow Q$, geçiş fonksiyonu olarak adlandırılan toplam bir fonksiyondur,
- $q_0 \in Q$, başlangıç durumudur,
- $F \subseteq Q$, son durumlar kümesidir.

Deterministik sonlu kabul edici aşağıdaki şekilde çalışır. Başlangıçta, başlangıç durumu q_0 'da olduğu varsayılır ve giriş mekanizması giriş dizisinin en soldaki sembolündedir. Otomatonun her hareketinde, giriş mekanizması bir pozisyon sağa ilerler, bu nedenle her hareket bir giriş sembolünü tüketir. Dizi sona ulaştığında, otomaton son durumla rdan birindeyse dizi kabul edilir.

Aksi takdirde dizi reddedilir. Giriş mekanizması yalnızca soldan sağa hareket edebilir ve her adımda tam olarak bir sembol okur. Bir iç durumdan diğerine geçişler geçiş fonksiyonu

δ ile yönetilir. Örneğin, eğer

$$\delta(q_0, a) = q_1,$$

o zaman eğer dfa durumunda q_0 ve mevcut giriş sembolü a ise, dfa durumuna geçecektir q_1 .

Otomata tartışırken, çalışmak için net ve sezgisel bir resme sahip olmak önemlidir. Sonlu otomata görselleştirmek ve temsil etmek için geçiş grafikleri kullanırız, bu grafikte köşeler durumları, kenarlar ise geçişleri temsil eder. Köşelerdeki etiketler durumların adlarıdır, kenarlarıdaki etiketler ise giriş sembolünün mevcut değerleridir.

Örneğin, eğer q_0 ve q_1 bazı dfa M 'nin iç durumlarıysa, o zaman M ile ilişkili grafik M ile etiketlenmiş bir q_0 ve diğer bir q_1 etiketli bir tepe noktasına sahip olacaktır.

Bir kenar (q_0, q_1) etiketli a , geçiş $\delta(q_0, a) = q_1$ temsil eder. Başlangıç durumu, herhangi bir tepe noktasından kaynaklanmayan gelen etiketlenmemiş bir ok ile tanımlanacaktır. Son durumlar çift daire ile çizilir.

Daha resmi olarak, eğer $M = (Q, \Sigma, \delta, q_0, F)$ deterministik sonlu kabul edici ise, o zaman onunla ilişkili geçiş grafiği G_M tam olarak $|Q|$ tepe noktasına sahiptir, her biri

farklı $q_i \in Q$ ile etiketlenmiştir. Her geçiş kuralı için $\delta(q_i, a) = q_j$, grafik bir kenar sahiptir (q_i, q_j) etiketli a . q_0 ile ilişkili tepe noktasına başlangıç tepe noktası denir, oysa $q_f \in F$ ile etiketlenmiş olanlar son tepe noktalarıdır. Bir dfa'nın $(Q, \Sigma, \delta, q_0, F)$ tanımından geçiş grafiği temsiline ve tersine dönüştürmek basit bir meseledir.

ÖRNEK 2.1

Şekil 2.1'deki grafik dfa'yı temsil eder

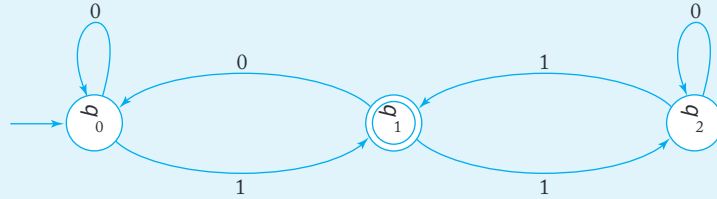
$$M = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_1\}),$$

nerede δ şu şekilde verilmiştir

$$\begin{array}{ll} \delta(q_0, 0) = q_0, & \delta(q_0, 1) = q_1, \\ \delta(q_1, 0) = q_0, & \delta(q_1, 1) = q_2, \\ \delta(q_2, 0) = q_2, & \delta(q_2, 1) = q_1. \end{array}$$

Bu dfa, 01 dizisini kabul eder. Durumdan q_0 başlayarak, sembol 0 okunur ilk olarak. Grafiğin kenarlarına bakarak, otomatonun durumda q_0 kaldığını görüyoruz. Sonra, 1 okunur ve otomaton şu anda q_1 durumuna geçer. Şimdi dizinin sonuna geldik ve aynı zamanda, bir final durumundayız q_1 . Bu nedenle, 01 dizisi kabul edilir. Dfa, diziyi 00 kabul etmez, çünkü iki ardışık 0 okuduktan sonra, durumda q_0 olacaktır. Benzer bir mantıkla, otomatonun

101, 0111 ve 11001 dizilerini kabul edeceğini, ancak 100 veya 1100'ü kabul etmeyeceğini görüyoruz.



ŞEKİL 2.1

Genişletilmiş geçiş fonksiyonu $\delta^*: Q \times \Sigma^* \rightarrow Q$ tanıtmak uygundur. İkinci argüman δ^* bir dizi olup, tek bir sembol değil, ve değeri otomatonun o diziyi okuduktan sonra bulunacağı durumu verir. Örneğin, eğer

$$\delta(q_0, a) = q_1$$

ve

$$\delta(q_1, b) = q_2,$$

then

$$\delta^*(q_0, ab) = q_2.$$

Resmi olarak, δ^* 'yi özyinelemeli olarak tanımlayabiliriz

$$\delta^*(q, \lambda) = q, \quad (2.1)$$

$$\delta^*(q, wa) = \delta(\delta^*(q, w), a), \quad (2.2)$$

her $q \in Q, w \in \Sigma^*, a \in \Sigma$ için. Bunun neden uygun olduğunu görmek için, yukarıdaki basit duruma bu tanımları uygulayalım.

Öncelikle, (2.2)'yi kullanarak

$$\delta^*(q_0, ab) = \delta(\delta^*(q_0, a), b). \quad (2.3)$$

Ama

$$\begin{aligned} \delta^*(q_0, a) &= \delta(\delta^*(q_0, \lambda), a) \\ &= \delta(q_0, a) \\ &= q_1. \end{aligned}$$

Bu ifadeyi (2.3)'e yerleştirerek, elde ederiz

$$\delta^*(q_0, ab) = \delta(q_1, b) = q_2,$$

as expected.

Diller ve DFA'lar

Bir kabul edici için kesin bir tanım yaptıktan sonra, şimdi ilişkili bir dili n ne anlama geldiğini resmi olarak tanımlamaya hazırız. İlişki açıktır: Dil, otomaton tarafından kabul edilen tüm dizelerin kümesidir.

TANIM 2.2

Bir dfa $M = (Q, \Sigma, \delta, q_0, F)$ tarafından kabul edilen dil, M tarafından kabul edilen Σ üzerindeki tüm dizelerin kümesidir. Resmi notasyonla,

$$L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \in F\}.$$

δ ve dolayısıyla δ^* 'nin toplam fonksiyonlar olmasını gerektirdiğimizi unutmayın. Her adımda, benzersiz bir hareket tanımlanır, böylece bu tür bir otomata deterministik demek haklıdır. Bir dfa, Σ^* üzerindeki her diziyi işler ve ya kabul eder ya da etmez. Kabul edilmemesi, dfa'nın son olmayan bir durumda durması anlamına gelir, böylece

$$\overline{L(M)} = \{w \in \Sigma^* : \delta^*(q_0, w) \notin F\}.$$

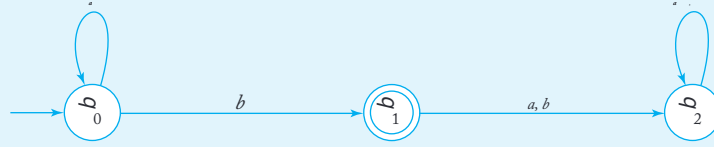
ÖRNEK 2.2

Şekil 2.2'deki dfa'yı düşünün.

Şekil 2.2'yi çizerken, tek bir kenar üzerinde iki etiketin kullanılması na izin verdik. Bu şekilde çoklu etiketli kenarlar, iki veya daha fazla farklı geçişin kısayoludur: Geçiş, giriş sembolü kenar etiketlerinden herhangi biriyle eşleştğinde alınır.

Şekil 2.2'deki otomaton, ilk durumunda q_0 kalır ve ilk b ile karşıl aşana kadar devam eder. Eğer bu, girişin son sembolü ise, o zaman dize kabul edilir. Aksi takdirde, dfa q_2 durumuna geçer ve buradan asla çıkamaz. Durum q_2 bir tuzak durumu'dur. Grafikten açıkça görüyoruz ki, otomaton, bir veya daha fazla a 'nın ardından tek bir b içeren tüm dizeleri kabul eder. Diğer tüm giriş dizeleri reddedilir. Küme notasyonu ile, otomatonun kabul ettiği dil

$$L = \{a^n b : n \geq 0\}.$$



ŞEKİL 2.2

Bu örnekler, sonlu otomata ile çalışırken geçiş grafiklerinin ne kadar kullanışlı olduğunu göstermektedir. Geçiş fonksiyonunun özelliklerine ve (2.1) ile (2.2) aracılığıyla uzantısına dayalı tüm argümanların kesinlikle oluşturulması mümkün olsa da, sonuçlar takip edilmesi zor hale gelmektedir. Tartışmamızda, mümkün olduğunca daha sezgisel olan grafikler kullanıyoruz. Bunu yapmak için, elbette, temsilin bizi yanıltmadığından ve grafiklere dayalı argümanların δ 'nin resmi özelliklerini kullananlar kadar geçerli olduğundan emin olmamız gerekir. Aşağıdaki ön sonuç bize bu güvenceyi vermektedir.

TEOREM 2.1

Bir $M = (Q, \Sigma, \delta, q_0, F)$ deterministik sonlu kabul edici olsun ve G_M onu n la ilişkili geçiş grafiği olsun. O halde her $q_i, q_j \in Q$ ve $w \in \Sigma^+$ için, $\delta^*(q_i, w) = q_j$ eğer ve ancak G_M içinde q_i 'den q_j 'ye w etiketli bir yürüyüş varsa.

Kanıt: Bu iddia, Örnek 2.1 gibi basit durumların incelenmesinden oldukça açıktır. Uzunluk üzerinde bir induksiyon kullanılarak titizlikle kanıtlanabilir.

w uzunluğunun $n + 1$ olduğunu varsayalım ve onu şu şekilde yazalım

$$w = va.$$

Şimdi $\delta^*(q_i, v) = q_k$ olduğunu varsayalım. Çünkü $|v| = n$, G_M 'de q_i 'den q_k 'ye v etiketli bir yürüyüş olmalıdır. Ancak eğer $\delta^*(q_i, w) = q_j$ ise, o zaman M 'nin bir geçişi olmalıdır $\delta(q_k, a) = q_j$, böylece yapısına göre G_M 'de a etiketli bir kenar (q_k, q_j) vardır. Böylece, q_i ile q_j arasında $va = w$ etiketli bir yürüyüş vardır. Sonuç açıkça $n = 1$ için doğrudur, bu nedenle induksiyonla her $w \in \Sigma^+$ için iddia edebiliriz,

$$\delta^*(q_i, w) = q_j \quad (2.4)q$$

q_i ile q_j arasında w ile etiketlenmiş G_M içinde bir yürüyüş olduğunu ima eder.

Bu argüman, böyle bir yolun varlığının (2.4) olduğunu gösterdiği şekilde basit bir şekilde tersine çevrilebilir, böylece

kanit tamamlandı. ■

Yine, teoremin sonucu o kadar sezgisel olarak açıktır ki, resmi bir kanıt gereksiz görünüyor. İki nedenle detayları inceledik. İlk olarak, bu, otomata ile bağlantılı olarak tipik bir induktif kanıtın basit, ancak örnek bir örneğidir. İkincisi, bu sonucun sürekli olarak kullanılacak olması, bunu bir teorem olarak ifade etmek ve kanıtlamak, grafikler kullanarak oldukça güvenle tartışmamıza olanak tanır. Bu, örneklerimizi ve kanıtlarımızı, δ^* özelliklerini kullandığımızdan daha şeffaf hale getirir.

Grafikler otomata görselleştirmek için uygun olsa da, diğer temsil biçimleri de faydalıdır. Örneğin, δ fonksiyonunu bir tabloda temsil edebiliriz. Şekil 2.3'teki tablo, Şekil 2.2'ye eşdeğerdir. Burada satır etiketi mevcut durumu, sütun etiketi ise mevcut girdi sembolünü temsil eder. Tablo içindeki giriş, bir sonraki durumu tanımlar.

	a	b
b_0	b_0	b_1
b_1	b_2	b_2
b_2	b_2	b_2

ŞEKİL 2.3

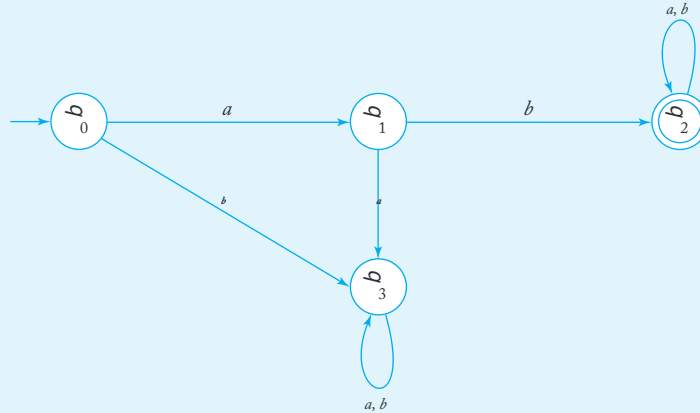
Bu örnekten, bir dfa'nın kolayca bir bilgisayar programı olarak uygulanabileceği açıktır; örneğin, basit bir tablo arama veya bir dizi if ifadesi olarak. En iyi uygulama veya temsil, belirli uygulamaya bağlıdır. Geçiş grafikleri, burada yapmak istediğimiz türden argümanlar için çok uygundur, bu nedenle tartışmalarımızın çoğunda bunları kullanıyoruz.

Resmi olarak tanımlanmamış diller için otomata inşa ederken, daha yüksek seviyeli dillerde programlama ile benzer bir akıl yürütme kullanıyoruz. Ancak bir dfa'nın programlanması zahmetlidir ve bazen böyle bir otomatonun az sayıda güçlü özelliğe sahip olması nedeniyle kavramsal olarak karmaşık hale gelir.

ÖRNEK 2.3

$\Sigma = \{a, b\}$ ile başlayan tüm dizeleri tanıyan bir deterministik sonlu kabul edici bulun.

Buradaki tek sorun, dizinin ilk iki sembolüdür; bunlar okunduktan sonra, başka kararlar alınması gerekmez. Yine de, otomaton karar verilmeden önce tüm diziyi işlemesi gerekir. Bu nedenle, dört durumu olan bir otomaton ile problemi çözebiliriz: bir başlangıç durumu, ab'yi tanımak için iki durum, bir son tuzak durumu ve bir son olmayan tuzak durumu. İlk sembol bir a ise ve ikinci bir ab ise, otomaton son tuzak durumuna geçer, burada kalacaktır çünkü geri kalan girdi önemli değildir. Öte yandan, eğer ilk sembol bir a değilse veya ikincisi bir b değilse, otomaton son olmayan tuzak durumuna girer. Basit çözüm Şekil 2.4'te gösterilmiştir.



ŞEKİL 2.4

ÖRNEK 2.4

Find a dfa that accepts all the strings on $\{0,1\}$, except those 001 alt dizesini içeren.

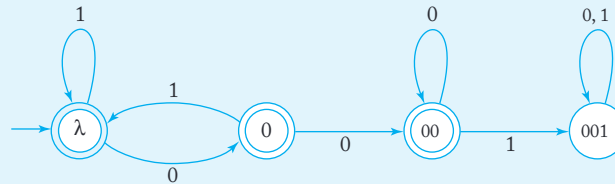
Alt dizi 001'in meydana gelip gelmediğine karar verirken, yalnızca mevcut girdi sembolünü bilmemiz yeterli değildir, ayrıca bunun bir veya iki 0 ile önceden gelip gelmediğini de hatırlamamız gerekir. Bunu, otomatonu belirli durumlara koyarak ve bunları uygun şekilde etiketleyerek takip edebiliriz. Bir programlama dilindeki değişken adları gibi, durum adları da keyfidir ve hafızayı kolaylaştırmak için seçilebilir. Örneğin, iki 0'ın hemen önceki semboller olduğu durum basitçe 00 olarak etiketlenebilir.

String 001 ile başlıyorsa, reddedilmelidir. Bu, başlangıç durumundan bir son durum olmayan duruma 001 etiketli bir yol olması gerektiği anlamına gelir. Kolaylık olması açısından, bu son durum olmayan durum 001 olarak etiketlenmiştir. Bu durum bir tuzak durumu olmalıdır, çünkü sonraki semboller önemli değildir. Diğer tüm durumlar kabul eden durumlardır.

Bu, çözümün temel yapısını bize verir, ancak hala girdi ortasında 001 alt dizesinin bulunması için düzenlemeler eklememiz gerekiyor. Doğru kararı vermek için gereken her şeyi otomatonun hatırlaması için Q ve δ 'yi tanımlamalıyız. Bu durumda, bir sembol okunduğunda, soldaki dizinin bir kısmını bilmemiz gerekiyor; örneğin, önceki iki sembolün 00 olup olmadığını. Durumları ilgili sembollerle etiketlerseniz, geçişlerin ne olması gerektiğini görmek çok kolaydır. Örneğin,

$$\delta(00,0) = 00$$

çünkü bu durum yalnızca üç ardışık 0 varsa ortaya çıkar. Biz yalnızca son iki ile ilgileniyoruz, bu gerçeği dfa'yı 00 durumunda tutarak hatırlıyoruz. Tam bir çözüm Şekil 2.5'te gösterilmiştir. Bu örnekten, durumlar üzerindeki hafıza etiketlerinin işleri takip etmekte ne kadar yararlı olduğunu görüyoruz. 100100 ve 1010100 gibi birkaç dizeyi izleyin, böylece çözümün gerçekten doğru olduğunu görebilirsiniz.



ŞEKİL 2.5

Regüler Diller

Her sonlu otomaton bazı dilleri kabul eder. Tüm olası sonlu otomatonları dikkate alırsak, onlarla ilişkili bir diller kümesi elde ederiz. Bu tür bir diller kümesine bir aile diyeceğiz. Deterministik sonlu kabul edenlerin kabul ettiği dillerin ailesi oldukça sınırlıdır. Bu ailenin dillerinin yapısı ve özellikleri çalışmamız ilerledikçe daha net hale gelecektir; şu anda bu aileye sadece bir isim vereceğiz.

DEFINITION 2.3

Bir dil L yalnızca bir deterministik

sonlu kabul edici M varsa düzenli olarak adlandırılır.

$$L = L(M).$$

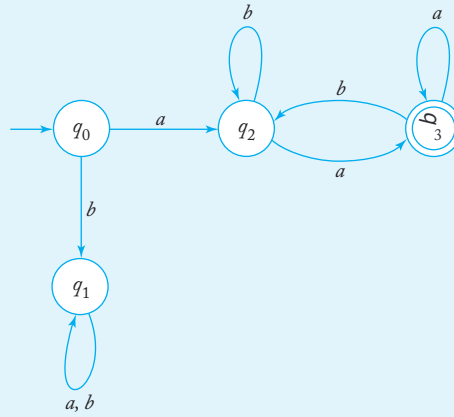
EXAMPLE 2.5

Dili gösterin

$$L = \{ awa : w \in \{a, b\}^* \} \text{ dü}$$

zenlidir.

Bunun veya başka bir dilin düzenli olduğunu göstermek için yapmamız gereken tek şey, onun için bir dfa bulmaktır. Bu dil için bir dfa'nın inşası, Örnek 2.3 ile benzer, ancak biraz daha karmaşıktır. Bu dfa'nın yapması gereken, bir dizinin bir a ile başlayıp bittiğini kontrol etmektir; aradaki kısım önemsizdir. Çözüm, dizinin sonunu test etmenin açık bir yolu olmaması nedeniyle karmaşıklaşır. Bu zorluk, ikinci a ile karşılaşıldığında dfa'yı bir son duruma koyarak aşılır. Eğer bu dizinin sonu değilse ve başka bir b bulunursa, dfa son durumdan çıkar. Tarama bu şekilde devam eder, her a otomasyonu son durumuna geri götürür. Tam çözüm Şekil 2.6'da gösterilmiştir. Yine, bunun neden çalıştığını görmek için birkaç örneği izleyin. Bir veya iki testten sonra, dfa'nın bir diziyi yalnızca bir a ile başlayıp bittiği takdirde kabul ettiği açık olacaktır. Dilin dfa'sını inşa ettiğimiz için, tanım gereği, dilin düzenli olduğunu iddia edebiliriz.

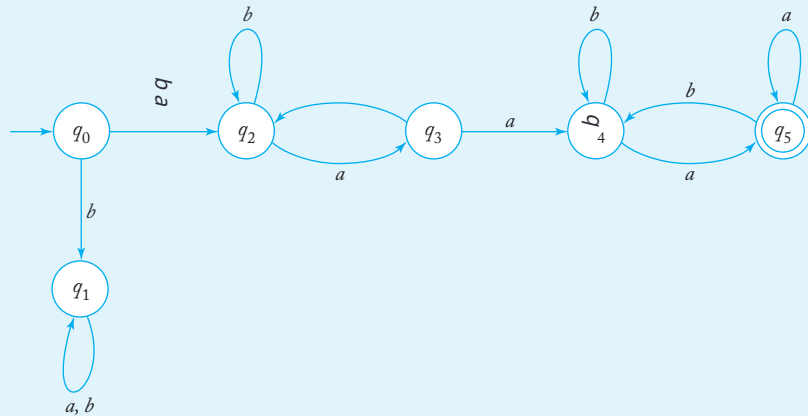


ŞEKİL 2.6

ÖRNEK 2.6

Let L be the language in Example 2.5. Show that L^2 is regular. Again we show that the language is regular by constructing a dfa for it. We can write an explicit expression for L^2 , namely,

$$L^2 = \{aw_1aaw_2a : w_1, w_2 \in \{a, b\}^*\}.$$



ŞEKİL 2.7

Bu nedenle, temelde aynı forma sahip (ancak değer olarak mutlaka aynı olmayan) iki ardışık dizeyi tanıyan bir dfa'ya ihtiyacımız var. Şekil 2.6'daki diyagram bir başlangıç noktası olarak kullanılabilir, ancak q_3 köşesi değiştirilmelidir. Bu durum artık sonlu olamaz çünkü bu noktada, awa biçiminde ikinci bir alt dize aramaya başlamalıyız. İkinci alt dizeyi tanımak için, ilk kısmın durumlarını (yeni adlarla) çoğaltıyoruz ve q_3 ikinci kısmın başlangıcı olarak alıyoruz. Tam dize, aa 'nın bulunduğu her yerde bileşenlerine ayrılabilceğinden, iki ardışık a 'nın ilk oluşumunu otomatonun ikinci kısmına geçişini tetikleyen bir durum olarak kabul ediyoruz. Bunu $\delta(q_3, a) = q_4$ yaparak gerçekleştirebiliriz. Tam çözüm Şekil 2.7'de bulunmaktadır. Bu dfa L^2 'yi kabul eder, bu nedenle düzenlidir.

Son örnek, eğer bir dil L düzenli ise,

$L^2, L^3, \dots$ 'nin de düzenli olduğu varsayımını önermektedir. Bunun gerçekten doğru olduğunu daha sonra göreceğiz.

EGZERSİZLER

- Şekil 2.1'deki dfa tarafından hangi dizelerin 0001, 01101, 00001101 kabul edilmektedir?
- Şekil 2.5'teki grafiği δ - notasyonuna çevirin.
- $\Sigma = \{a, b\}$ için, aşağıdaki kümeleri kabul eden dfa'lar oluşturun:
 - tüm çift uzunluktaki dizeler.
 - uzunluğu 5'ten büyük olan tüm dizeler.
 - çift sayıda a bulunan tüm dizeler.
 - çift sayıda a ve tek sayıda b bulunan tüm dizeler.
- $\Sigma = \{a, b\}$ için, aşağıdaki kümeleri kabul eden dfa'lar oluşturun:
 - tam olarak bir a bulunan tüm dizeler.
 - en az iki a bulunan tüm dizeler.
 - en fazla iki a bulunan tüm dizeler.
 - en az bir b ve tam olarak iki a olan tüm dizeler.
 - tam olarak iki a ve üçten fazla b olan tüm dizeler.
- Diller için dfa'lar verin
 - $L = \{ab^4wb^2 : w \in \{a, b\}^*\}$.
 - $L = \{ab^n a^m : n \geq 3, m \geq 2\}$.

$$(c) L = \{w_1abbw_2 : w_1 \in \{a, b\}^*, w_2 \in \{a, b\}^*\}.$$

$$(d) L = \{ba^n : n \geq 1, n \neq 4\}.$$

6. $\Sigma = \{a, b\}$ ile, $L = \{w_1aw_2 : |w_1| \geq 3, |w_2| \leq 4\}$ için bir dfa verin.

7. $\Sigma = \{a, b\}$ üzerinde aşağıdaki diller için dfa'lar bulun.

$$(a) L = \{w : |w| \bmod 3 = 0\}.$$

$$(b) L = \{w : |w| \bmod 5 = 0\}.$$

$$(c) L = \{w : n_a(w) \bmod 3 < 1\}.$$

$$(d) L = \{w : n_a(w) \bmod 3 < n_b(w) \bmod 3\}.$$

$$(e) L = \{w : (n_a(w) - n_b(w)) \bmod 3 = 0\}.$$

$$(f) L = \{w : (n_a(w) + 2n_b(w)) \bmod 3 < 1\}.$$

$$(g) L = \{w : |w| \bmod 3 = 0, |w| \neq 5\}.$$

8. Bir dizideki koşu, en az iki uzunluğunda, mümkün olduğunca uzun ve tamam en aynı sembolden oluşan bir alt dizedir. Örneğin, dizi abbbaab, uzunluğu üç olan b'lerin ve uzunluğu iki olan a'ların bir koşusunu içerir. Aşağıdaki diller için dfa'ları bulun $\{a, b\}$:

$$(a) L = \{w : w, \text{ uzunluğu üçten az olan koşular içermez}\}.$$

$$(b) L = \{w : \text{her } a\text{'nın koşusu ya iki ya da üç uzunluğundadır}\}.$$

$$(c) L = \{w : \text{en fazla üç uzunluğunda iki } a\text{'nın koşusu vardır}\}.$$

$$(d) L = \{w : \text{tam olarak üç uzunluğunda iki } a\text{'nın koşusu vardır}\}.$$

9. Şunu gösterin ki, Şekil 2.6'yı değiştirirsek, q_3 'ü sonlu olmayan bir durum yapar ve q_3 'ü sonlu durumlar yaparsak, elde edilen dfa L 'yi kabul eder.

10. Önceki egzersizdeki gözlemi genelleştirin. Özellikle, eğer $M = (Q, \Sigma, \delta, q_0, F)$ ve $\overline{M} = (Q, \Sigma, \delta, q_0, Q - F)$ iki dfa ise, o zaman $\overline{L(M)} = L(\overline{M})$.

11. $\{0, 1\}$ üzerinde aşağıdaki gereksinimlerle tanımlanan dize kümesini düşünün. Her biri için, kabul eden bir dfa oluşturun.

(a) Her 00 hemen ardından bir 1 ile takip edilir. Örneğin, dizeler 101, 0010, 0010011001 dilin içindedir, ancak 0001 ve 00100 değildir.

(b) 000 alt dizesini içeren, ancak 0000 içermeyen tüm dizeler.

(c) En soldaki sembol, en sağdaki sembolden farklıdır.

(d) Dört sembolden oluşan her alt dize, en fazla iki 0'a sahiptir. Örneğin, 001110 ve 011001 dilin içindedir, ancak 10010 değildir çünkü alt dizelerinden biri, 0010, üç sıfır içerir.

(e) Sağdan üçüncü sembolün en soldaki sembolden farklı olduğu, beş veya daha uzun uzunluktaki tüm dizeler.

- (f) En soldaki iki sembol ile en sağdaki iki sembolün aynı olduğu tüm dizeler.
- (g) En soldaki iki sembolün aynı, ancak en sağdaki sembolden farklı olduğu, dört veya daha uzun uzunluktaki tüm dizeler.
12. Bir dfa oluşturun ki, yalnızca dizinin, bir tam sayının ikili temsili olarak yorumlandığında, beş ile modülü sıfır olan dizeleri kabul etsin. Örneğin, 0101 ve 1111, sırasıyla 5 ve 15 tam sayılarını temsil eder, kabul edilmelidir.
13. Dilin $L = \{vwv : v, w \in \{a, b\}^*, |v| = 3\}$ düzenli olduğunu gösterin.
14. Dilin $L = \{a^n : n \geq 3\}$ düzenli olduğunu gösterin.
15. Dilin $L = \{a^n : n \geq 0, n = 3\}$ düzenli olduğunu gösterin.
16. Dilin $L = \{a^n : n \text{ ya üçün katı ya da 5'in katıdır}\}$ düzenli olduğunu gösterin.
17. Dilin $L = \{a^n : n \text{ üçün katıdır, ancak 5'in katı değildir}\}$ düzenli olduğunu gösterin.
18. C 'deki tüm reel sayıların kümesinin düzenli bir dil olduğunu gösterin.
19. Eğer L düzenli ise, o zaman $L - \{\lambda\}$ de düzenlidir, gösterin.
20. Eğer L düzenli ise, o zaman $L \cup \{aa\}$ de düzenlidir, tüm $a \in \Sigma$ için gösterin.
21. (2.1) ve (2.2) kullanarak gösterin ki

$$\alpha(q, wv) = \delta^*(\delta^*(q, w), v)$$

her $w, v \in \Sigma^*$.

22. Let L , Şekil 2.2'deki otomaton tarafından kabul edilen dil olsun. L^3 'ü kabul eden bir dfa bulun.
23. Let L , Şekil 2.2'deki otomaton tarafından kabul edilen dil olsun. $L^2 - L$ için bir dfa bulun.
24. Let L , Örnek 2.5'teki dil olsun. L^* 'nin düzenli olduğunu gösterin.
25. Let G_M , bazı dfa M için geçiş grafi olsun. Aşağıdakileri kanıtlayın:
- (a) Eğer $L(M)$ sonsuzsa, o zaman G_M en az bir döngüye sahip olmalıdır ki bu döngüdeki başlangıç tepe noktasından bazı tepe noktalarına bir yol ve döngüdeki bazı tepe noktalarından bir son tepe noktasına bir yol vardır.
- (b) Eğer $L(M)$ sonlu ise, o zaman böyle bir döngü yoktur.
26. Bir işlemi tanımlayalım $truncate$, bu işlem herhangi bir dizinin en sağdaki sembolünü kaldırır. Örneğin, $truncate(aaaba)$ şudur $aaab$. İşlem, dillere genişletilebilir

$$truncate(L) = \{truncate(w) : w \in L\}.$$

Herhangi bir düzenli dil için bir dfa verildiğinde L , bir dfa'nın nasıl inşa edileceğini gösterin $truncate(L)$. Buradan, eğer L bir düzenli dil ise ve içermiyorsa λ , o zaman $truncate(L)$ de düzenlidir.

27. Verilen bir dfa tarafından kabul edilen dil benzersizdir, ancak genellikle bir dili kabul eden birçok dfa vardır. Şekil 2.4'teki dfa ile aynı dili kabul eden tam olarak altı durumu olan bir dfa bulun.
28. Üç durumu olan ve Şekil 2.4'teki dfa'nın dilini kabul eden bir dfa bulabilir misiniz? Eğer bulamazsanız, böyle bir dfa'nın var olamayacağına dair ikna edici argümanlar verebilir misiniz?

2.2 SONBELİRSİZ SONLU KABUL EDİCİLER

Şu ana kadar gördüğümüz otomataları incelediğinizde, her durum ve her girdi sembolü için benzersiz bir geçiş tanımlandığını göreceksiniz. Resmi tanımda, bu, δ 'nin bir toplam fonksiyon olduğunu söyleyerek ifade edilir. Bu nedenle bu otomatalara deterministik diyoruz. Şimdi, birden fazla geçişin mümkün olduğu bazı durumlarda bazı otomatalara seçimler vererek işleri karmaşıktırıyoruz. Bu tür otomatalara sonbelirsiz (nondeterministic) diyeceğiz.

Sonbelirsizlik, ilk bakışta alışılmadık bir fikirdir. Bilgisayarlar deterministik makineler olup, seçim unsuru yerinde görünmemektedir. Yine de, sonbelirsizlik yararlı bir kavramdır, bunu göreceğiz.

Nedensiz Kabul Edici Tanımı

Sonbelirsizlik, bir otomaton için hareketlerin bir seçimidir. Her durumda benzer bir hareket belirlemek yerine, olası hareketlerin bir kümesine izin veriyoruz. Resmi olarak, geçiş fonksiyonunu tanımlayarak bunu gerçekleştiriyoruz, böylece aralığı olası durumlar kümesi olur.

TANIM 2.4

Bir sonbelirsiz sonlu kabul edici (nfa), beşli ile tanımlanır

$$M = (Q, \Sigma, \delta, q_0, F),$$

burada Q, Σ, q_0, F , belirli son kabul edenler için tanımlandığı gibi,

$$\delta : Q \times (\Sigma \cup \{\lambda\}) \rightarrow 2^Q.$$

Bu tanım ile bir dfa tanımı arasında üç ana fark olduğunu unutmayın. Belirsiz bir kabul edici (nondeterministic accepter) durumunda, δ 'nin aralığı 2^Q güç kümesindedir, bu nedenle değeri Q 'nun tek bir elemanı değil, onun bir alt kümesidir. Bu alt küme, geçişle ulaşılabilecek tüm olası durumların kümesini tanımlar. Örneğin, mevcut durum q_1 ise, sembol a okunur ve

$$\delta(q_1, a) = \{q_0, q_2\},$$

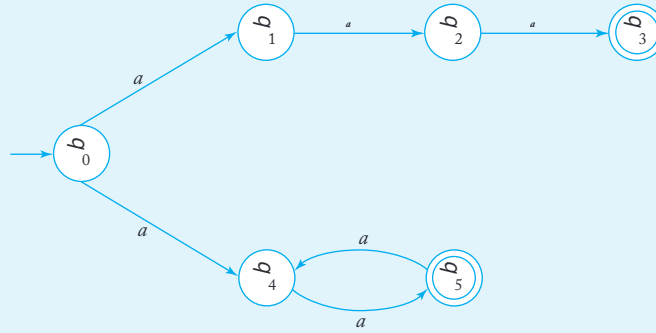
o zaman ya q_0 ya da q_2 nfa'nın bir sonraki durumu olabilir. Ayrıca, δ 'nın ikinci argümanı olarak λ 'yı kabul ediyoruz. Bu, nfa'nın bir giriş sembolü tüketmeden geçiş yapabileceği anlamına gelir. Giriş mekanizmasının yalnızca sağa hareket edebileceğini varsaysak da, bazı hareketlerde sabit kalması mümkündür. Son olarak, bir nfa'da, küme $\delta(q_i, a)$ boş olabilir, bu da bu özel durum için tanımlanmış bir geçişin olmadığı anlamına gelir.

Dfa'lar gibi, belirsiz kabul ediciler geçiş grafikleri ile temsil edilebilir. Düğüm noktaları Q tarafından belirlenirken, etiketli bir kenar (q_i, q_j) a grafikte yalnızca $\delta(q_i, a)$ q_j içtiyorsa bulunur. Ayrıca, a'nın boş dize olabileceğini göz önünde bulundurursak, bazı kenarların λ ile etiketlenmiş olabileceğini unutmayın.

Bir dize, bir nfa tarafından kabul edilir eğer dizinin sonunda makinenin bir son duruma getirecek bazı olası hareketlerin bir dizisi varsa. Bir dize yalnızca, bir son duruma ulaşmak için hiçbir olası hareket dizisi yoksa reddedilir (yani, kabul edilmez). Bu nedenle, belirsizlik, her durumda en iyi hareketin seçilebileceği "sezgisel" bir içgörü olarak görülebilir (nfa'nın her dizeyi kabul etmek istediği varsayılarak).

ÖRNEK 2.7

Şekil 2.8'deki geçiş grafiğini dikkate alın. Bu, q_0 'dan a ile etiketlenmiş iki geçiş olduğu için bir belirsiz kabul edici tanımlar.

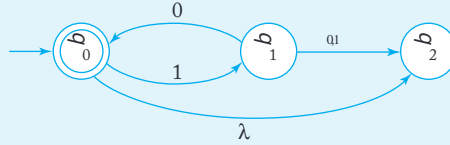


ŞEKİL 2.8

ÖRNEK 2.8

Şekil 2.9'da bir belirsiz otomatın gösterilmektedir. Bu, belirsiz-belirleyici, yalnızca aynı etikete sahip birkaç kenarın bir tepe noktasından kaynaklanmasından değil, aynı zamanda bir λ -geçişine sahip olmasından dolayıdır. Bazı

geçişler, örneğin $\delta(q_2, 0)$, grafikte belirtilmemiştir. Bu, boş küme ile geçiş olarak yorumlanmalıdır, yani $\delta(q_2, 0) = \emptyset$. otomaton, λ , 1010 ve 101010 dizelerini kabul eder, ancak 110 ve 10100'ü kabul etmez. 10 için iki alternatif yürüyüş olduğunu unutmayın, biri q_0 'ye, diğeri ise q_2 'ye götürmektedir. Her ne kadar q_2 son durum olmasa da, dize kabul edilir çünkü bir yürüyüş son duruma götürmektedir.



ŞEKİL 2.9

Yine, geçiş fonksiyonu, ikinci argümanının bir dize olacak şekilde genişletilebilir. Genişletilmiş geçiş fonksiyonu δ^* için, eğer

$$\delta^*(q_i, w) = Q_j,$$

o zaman Q_j , otomatonun başlangıçta q_i durumunda bulunabileceği tüm olası durumların kümesidir ve w okunmuştur. δ^* için, (2.1) ve (2.2) ile benzer bir özyinelemeli tanım mümkündür, ancak özellikle aydınlatıcı değildir. Daha kolay anlaşılabilir bir tanım, geçiş grafikleri aracılığıyla yapılabilir.

TANIM 2.5

Bir nfa için, genişletilmiş geçiş fonksiyonu $\delta^*(q_i, w)$ tanımlanır ki q_j , yalnızca q_i 'den q_j 'ye w ile etiketlenmiş bir yürüyüş varsa içerir. Bu, tüm $q_i, q_j \in Q$ ve $w \in \Sigma^*$ için geçerlidir.

ÖRNEK 2.9

Şekil 2.10 bir nfa'yı temsil etmektedir. Birkaç λ -geçışı ve $\delta(q_2, a)$ gibi bazı tanımsız geçişleri vardır.

Diyelim ki $\delta^*(q_1, a)$ ve $\delta^*(q_2, \lambda)$ bulmak istiyoruz. q_1 'den kendisine doğru i ki λ -geçışı içeren a ile etiketlenmiş bir yürüyüş vardır. Bazı λ -kenarlarını iki kez kullanarak, ayrıca q_0 ve q_2 'ye doğru λ -geçişleri içeren yürüyüşler de olduğunu görüyoruz. Böylece,

$$\delta^*(q_1, a) = \{q_0, q_1, q_2\}.$$

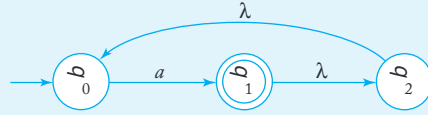
q_2 ile q_0 arasında bir λ -kenarı bulunduğundan, hemen şunu söyleyebiliriz ki $\delta^*(q_2, \lambda)$ içerir q_0 . Ayrıca, herhangi bir duruma kendisinden

hiç hareket etmeden ulaşılabilceğinden ve dolayısıyla hiçbir girdi sembolü kullanılmadığından, $\delta^*(q_2, \lambda)$ aynı zamanda içerir q_2 . Bu nedenle,

$$\delta^*(q_2, \lambda) = \{q_0, q_2\}.$$

Gerekli kadar λ -geçişini kullanarak, ayrıca kontrol edebilirsiniz ki

$$\delta^*(q_2, aa) = \{q_0, q_1, q_2\}.$$



ŞEKİL 2.10

Etiketli yürüyüşler aracılığıyla δ^* tanımı biraz resmi değildir, bu nedenle nu biraz daha yakından incelemek faydalıdır. Tanım 2.5 doğrudur, çünkü her hangi iki tepe noktası v_i ve v_j arasında ya etiketli bir yürüyüş w vardır ya da yoktur, bu da δ^* tamamen tanımlandığını gösterir. Belki de biraz zor görünen şey, bu tanımın her zaman $\delta^*(q_i, w)$ bulmak için kullanılabileceğidir.

Bölüm 1.1'de, iki tepe noktası arasındaki tüm basit yolları bulmak için bir algoritma tanımladık. Bu algoritmayı doğrudan kullanamayız çünkü, Örnek 2.9'un gösterdiği gibi, etiketli bir yürüyüş her zaman basit bir yol değildir. Basit yol algoritmasını değiştirebiliriz, hiçbir tepe noktasının veya kenarın tekrarlanmaması kısıtlamasını kaldırarak. Yeni algoritma artık sırasıyla bir uzunlukta, iki uzunlukta, üç uzunlukta ve daha fazlasında tüm yürüyüşleri üretecektir.

Hala bir zorluk var. Verilen bir w için, etiketli bir yürüyüşün w ne kadar uzun olabileceği hemen belli değil. Örnek 2.9'da, etiketli yürüyüş a, q_1 ile q_2 arasında dört uzunluğundadır. Sorun, yürüyüşü uzatan ancak etiket ve katkıda bulunmayan λ -geçişlerinden kaynaklanmaktadır.

Bu gözlem durumu kurtarıyor: Eğer iki tepe noktası v_i ve v_j arasında herhangi bir etiketli yürüyüş w varsa, o zaman grafikteki λ -kenar sayısı Λ olmak üzere, en fazla $\Lambda + (1 + \Lambda)|w|$ uzunluğunda bir etiketli yürüyüş w olmalıdır. Bunun argümanı şudur: λ -kenarları tekrar edilebilirken, her tekrar eden λ -kenarın, boş olmayan bir sembole etiketlenmiş bir kenar ile ayrıldığı bir yürüyüş her zaman vardır. Aksi takdirde, yürüyüş λ ile etiketlenmiş bir döngü içerir ki bu, yürüyüşün etiketini değiştirmeden basit bir yol ile değiştirilebilir. Bu iddianın resmi kanıtını bir alıştırmaya bırakıyoruz.

Bu gözlemlerle, $\delta^*(q_i, w)$ hesaplamak için bir yöntemimiz var. En fazla $\Lambda + (1 + \Lambda)|w|$ uzunluğundaki tüm yürüyüşleri q_i 'nden başlatıyoruz. Bundan etiketli w olanları seçiyoruz. Seçilen yürüyüşlerin son tepe noktaları, kümenin elemanlarıdır $\delta^*(q_i, w)$.

Belirttiğimiz gibi, δ^* 'yi, deterministik durum için yapıldığı gibi, özyinelemeli bir şekilde tanımlamak mümkündür. Sonuç ne yazık ki çok şeffaf değildir ve bu şekilde tanımlanan genişletilmiş geçiş fonksiyonu ile yapılan argümanlar takip edilmesi zor hale gelir. Daha sezgisel ve daha yönetilebilir olan Tanım 2.5'teki alt ernatifi kullanmayı tercih ediyoruz.

DFA'lar için, bir NFA tarafından kabul edilen dil, genişletilmiş geçiş fonksiyonu ile resmi olarak tanımlanır.

TANIM 2.6

Bir NFA $M = (Q, \Sigma, \delta, q_0, F)$ tarafından kabul edilen dil L , yukarıda belirtilen anlamda kabul edilen tüm dizelerin kümesi olarak tanımlanır. Resmi olarak,

$$L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \cap F \neq \emptyset\}.$$

Kelime olarak, dil, geçiş grafiğinin başlangıç tepe noktasından bazı son tepe noktalarına etiketli w ile bir yürüyüşün olduğu tüm dizeleri içerir.

ÖRNEK 2.10

Şekil 2.9'daki otomatonun kabul ettiği dil nedir? Grafikten, NFA'nı n yalnızca son durumda durabileceği tek yolun, girdinin ya 10 dizisinin tekrarı ya da boş dizi olması olduğu kolayca görülebilir. Bu nedenle, otomaton dilini kabul eder $L = \{(10)^n : n \geq 0\}$.

Bu otomata, dize ile karşılaştığında ne olur $w = 110$? Ön ek 11'i okuduktan sonra, otomata q_2 durumuna gelir, geçiş $\delta(q_2, 0)$ tanımsızdır. Böyle bir duruma ölü yapılandırma diyoruz ve bunu otomatanın daha fazla eylemde bulunmadan durması olarak görsel eştirebiliriz. Ancak, her zaman bu tür görselleştirmelerin kesin olmadığını ve yanlış yorumlama tehlikesi taşıdığını aklımızda tutmalıyız. Kesin olarak söyleyebileceğimiz şey

$$\delta^*(q_0, 110) = \emptyset.$$

Böylece, $w = 110$ işlenerek hiçbir son duruma ulaşılamaz ve bu nedenle dize kabul edilmez.

Nedensizlik Neden?

Belirsiz makineler hakkında düşünürken, sezgisel kavramları kullanırken oldukça dikkatli olmalıyız. Sezgilerimiz bizi yanıltabilir ve sonuçlarımızı desteklemek için kesin argümanlar sunabilmeliyiz.

Belirsizlik zor bir kavramdır. Dijital bilgisayarlar tamamen belirleyicidir; herhangi bir zamanda durumu, girdi ve başlangıç durumundan benzersiz bir şekilde tahmin edilebilir. Bu nedenle, neden belirsiz makineleri incelediğimizi sormak doğaldır. Gerçek sistemleri modellemeye çalışıyoruz, peki neden seçim gibi mekanik olmayan özellikleri dahil edelim? Bu soruya çeşitli şekillerde yanıt verebiliriz.

Birçok deterministik algoritma, bir aşamada seçim yapmayı gerektirir. Tipik bir örnek, bir oyun oynama programıdır. Sıklıkla, en iyi hamle bilinmez, ancak geri izleme ile kapsamlı bir arama kullanılarak bulunabilir. Birden fazla alternatif mümkün olduğunda, birini seçeriz ve en iyi olup olmadığını netleştirene kadar onu takip ederiz. Eğer en iyi değilse, son karar noktasına geri döneriz ve diğer seçenekleri keşfederiz. En iyi seçimi yapabilen bir nondeterministik algoritma, problemi geri izleme olmadan çözebilir, ancak bir deterministik algoritma, biraz ekstra çalışma ile nondeterminizmi simüle edebilir. Bu nedenle, nondeterministik makineler, arama ve geri izleme algoritmalarının modelleri olarak hizmet edebilir.

Nondeterminizm, bazen problemleri kolayca çözmede yardımcı olabilir. Şekil 2.8'deki nfa'ya bakın. Yapılması gereken bir seçim olduğu açıktır. İlk alternatif, dizeyi kabul eder a^3 , ikincisi ise tüm çift sayı a^n 'ları kabul eder. Nfa tarafından kabul edilen dil $\{a^3\} \cup \{a^{2n} : n \geq 1\}$. Bu dil için bir dfa bulmak mümkün olsa da, nondeterminizm oldukça doğaldır. Dil, iki oldukça farklı kümenin birleşimidir ve nondeterminizm, başlangıçta hangi durumu istediğimizi belirlememize olanak tanır. Deterministik çözüm, tanıma o kadar açık bir şekilde bağlı değildir ve bu nedenle bulması biraz daha zordur. Devam ettikçe, nondeterminizmin faydalılığına dair başka ve daha ikna edici örnekler göreceğiz.

Aynı şekilde, belirsizlik, bazı karmaşık dilleri kısaca tanımlamak için etkili bir mekanizmadır. Bir dilbilgisinin tanımının belirsiz bir unsuru içerdiğini unutmayın.

$$S \rightarrow aSb \mid \lambda$$

Herhangi bir noktada birinci veya ikinci üretimi seçebiliriz. Bu, yalnızca iki kural kullanarak birçok farklı dize belirtmemizi sağlar.

Son olarak, belirsizliği tanıtmının teknik bir nedeni vardır. Gördüğümüz gibi, belirli teorik sonuçlar nfa'lar için dfa'lardan daha kolay bir şekilde kurulmaktadır. Bir sonraki önemli sonucumuz, bu iki otomata türü arasında temel bir fark olmadığını göstermektedir. Sonuç olarak, belirsizliğe izin vermek, genellikle resmi argümanları basitleştirir ve sonucun genel geçerliliğini etkilemez.

EGZERSİZLER

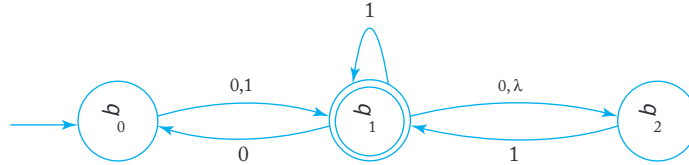
1. C'deki tüm tam sayıları kabul eden bir nfa oluşturun. Oluşumunuzun neden bir nfa olduğunu açıklayın.

2. Bir geçiş grafiğinde w etiketli bir yürüyüş varsa, bu yürüyüşün uzunluğunun $\Lambda + (1 + \Lambda)|w|$ 'den fazla olmaması gerektiği iddiasını detaylı bir şekilde kanıtlayın.
3. Şekil 2.8'de tanımlanan dili kabul eden bir dfa bulun.
4. Şekil 2.8'de tanımlanan dilin tamamlayıcısını kabul eden bir dfa bulun.
5. Şekil 2.9'da, $\delta^*(q_0, 1011)$ ve $\delta^*(q_1, 01)$ bulun.
6. Şekil 2.10'da, $\delta^*(q_0, a)$ ve $\delta^*(q_1, \lambda)$ bulun.
7. Şekil 2.9'daki nfa için, $\delta^*(q_0, 1010)$ ve $\delta^*(q_1, 00)$ bulun.
8. En fazla beş durumu olan bir nfa tasarlayın, küme $\{ab^n : n \geq 0\} \cup \{aban : n \geq 0\}$ için.
9. Üç durumu olan ve $\{ab, abc\}^*$ dilini kabul eden bir nfa oluşturun.
10. Egzersiz 9'un üç durumdan daha azıyla çözülebileceğini düşünüyor musunuz?
11. (a) Dili kabul eden üç durumu olan bir nfa bulun.

$$L = \{a^n : n \geq 1\} \cup \left\{ \sigma_{ak}^m : m \geq 0, k \geq 0 \right\}.$$

(b) (a) kısmındaki dilin, üç durumdan daha azıyla kabul edilebileceğini düşünüyor musunuz?

12. Bir nfa bulunuz, dört durumu olan $L = \{a^n : n \geq 0\} \cup \{b^n a : n \geq 1\}$.
13. Aşağıdaki nfa tarafından hangi dizeler 00, 01001, 10010, 000, 0000 kabul edilmektedir?



14. Şekil 2.10'daki nfa tarafından kabul edilen dilin tamamlayıcı nedir?
15. Let L , Şekil 2.8'deki nfa tarafından kabul edilen dil olsun. $L \cup \{a^5\}$ kabul eden bir nfa bulunuz.
16. L^* için bir nfa bulunuz, burada L , Egzersiz 15'teki dildir.
17. $\{a\}^*$ kabul eden bir nfa bulunuz ve geçiş grafiğinde tek bir kenar kaldırıldığında (başka bir değişiklik olmaksızın), ortaya çıkan otomatonun $\{a\}$ kabul etmesini sağlayacak şekilde olsun.
18. Egzersiz 17 bir dfa kullanılarak çözülebilir mi? Eğer öyleyse, çözümü verin; eğer değilse, sonucunuzu destekleyen ikna edici argümanlar sunun.
19. Aşağıdaki Tanım 2.6'nın bir modifikasyonunu düşünün. Birden fazla başlangıç durumuna sahip bir nfa, beşli ile tanımlanır

$$M = (Q, \Sigma, \delta, Q_0, F),$$

burada $Q_0 \subseteq Q$, olası başlangıç durumlarının bir kümesidir. Böyle bir otomaton tarafından kabul edilen dil

$$L(M) = \{w: \delta^*(q_0, w) \text{ içerir } q_f, \text{ her } q_0 \in Q_0, q_f \in F\}.$$

Birden fazla başlangıç durumuna sahip her nfa için, aynı dili kabul eden tek bir başlangıç durumuna sahip bir nfa'nın var olduğunu gösterin.

20. Egzersiz 19'da $Q_0 \cap F =$ kısıtlamasını yaptığımızı varsayalım. Bu sonuç üzerinde bir etki yaratır mı?

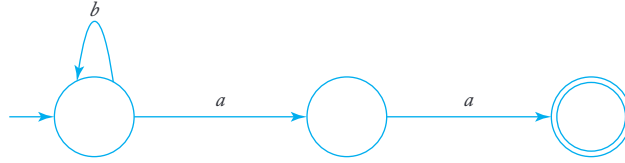
21. Herhangi bir nfa için Tanım 2.5'i kullanarak gösterin

$$\alpha(q, wv) = \bigcup_{p \in \delta^*(q, w)} \alpha(p, v),$$

her $q \in Q$ ve her $w, v \in \Sigma^*$ için.

22. λ -geçisi olmayan (a) ve her $q \in Q$ ve her $a \in \Sigma$ için, $\delta(q, a)$ en fazla bir eleman içeriyorsa, bazen eksik dfa olarak adlandırılır. Bu mantıklıdır çünkü koşullar, hareketlerin asla bir seçim yapılmasını sağlamaz.

$\Sigma = \{a, b\}$ için, aşağıdaki eksik dfa'yı standart bir dfa'ya dönüştürün.



23. Bir L düzenli dilini bazı bir alfabede Σ olarak alalım ve $\Sigma_1 \subset \Sigma$ daha küçük bir alfabe olsun. L_1 'i, yalnızca Σ_1 'den gelen sembollerden oluşan L alt kümesi olarak düşünelim, yani,

$$L_1 = L \cap \Sigma_1^*.$$

L_1 'in de düzenli olduğunu gösterin.

2.3 BELİRLİ VE BELİRSİZ SONLU KABUL EDİCİLERİN EŞDEĞERLİĞİ

Şimdi temel bir soruya geliyoruz. Dfa'lar ve nfa'lar hangi anlamda farklıdır? Açıkça, tanımlarında bir fark vardır, ancak bu, aralarında herhangi bir temel ayrım olduğu anlamına gelmez. Bu soruyu keşfetmek için, otomata arasındaki eşdeğerlik kavramını tanıtırız.

TANIM 2.7

İki sonlu kabul edici, M_1 ve M_2 , eşdeğer olarak kabul edilir eğer

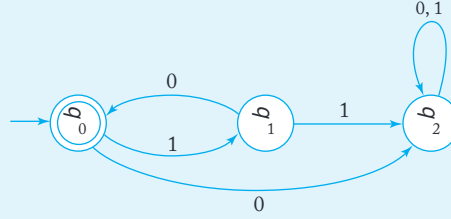
$$L(M_1) = L(M_2),$$

yani, her ikisi de aynı dili kabul ediyorsa.

Belirtildiği gibi, belirli bir dil için genellikle birçok kabul edici vardır, bu nedenle herhangi bir dfa veya nfa birçok eşdeğer kabul ediciye sahiptir.

ÖRNEK 2.11

Şekil 2.11'de gösterilen dfa, Şekil 2.9'daki nfa ile eşdeğerdir çünkü her ikisi de dili kabul eder $\{(10)^n : n \geq 0\}$.



ŞEKİL 2.11

Farklı otomata sınıflarını karşılaştırdığımızda, bir sınıfın diğerinden daha güçlü olup olmadığı sorusu kaçınılmaz olarak ortaya çıkar. “Daha güçlü” derken, bir tür otomatonun, diğer türdeki herhangi bir otomatonun yapamayacağı bir şeyi başarabileceğini kastediyoruz. Bu soruya sonlu kabul ediciler için bakalım. Bir dfa, özünde bir nfa'nın kısıtlı bir türü olduğundan, bir dfa tarafından kabul edilen herhangi bir dilin aynı zamanda bir nfa tarafından da kabul edildiği açıktır. Ancak tersinin bu kadar açık olduğu söylenemez. Belirsizlik ekledik, bu nedenle, prensipte, bir dfa bulamayacağımız bazı nfa'lar tarafından kabul edilen bir dilin var olması en azından mümkündür. Ancak bu durumun böyle olmadığı ortaya çıkıyor. Dfa ve nfa sınıfları eşit derecede güçlüdür: Bir nfa tarafından kabul edilen her dil için, aynı dili kabul eden bir dfa vardır.

Bu sonuç açık değildir ve kesinlikle gösterilmesi gerekir. Argüman, bu kitapta çoğu argüman gibi, yapıcı olacaktır. Bu, herhangi bir nfa'yı eşdeğer bir dfa'ya dönüştürmenin bir yolunu gerçekten verebileceğimizi anlamına gelir. Yapı, anlaşılması zor değildir; fikir netleştikçe, tiz bir argümanın başlangıç noktası haline gelir. Yapının mantığı şudur: Bir nfa bir dize w okuduktan sonra, hangi durumda olacağını tam olarak bilemeyebiliriz, ancak bir dizi olası durumun birinde, diyelim ki $\{q_i, q_j, \dots, q_k\}$ olmalıdır. Aynı dizeyi okuduktan sonra eşdeğer bir dfa kesin bir durumda olmalıdır. Bu iki durumu nasıl eşleştirebiliriz? Cevap güzel bir numara: Dfa'nın durumlarını, w okuduktan sonra eşdeğer dfa'nın $\{q_i, q_j, \dots, q_k\}$ ile etiketlenmiş tek bir durumda olacağı şekilde bir durum seti ile etiketleyin. $|Q|$ durumları için tam olarak $2^{|Q|}$ alt küme olduğundan, karşılık gelen dfa'nın sonlu sayıda durumu olacaktır.

Bu önerilen yapının çoğu, olası durumlar ve girdiler arasındaki karşılığı elde etmek için nfa'nın analizinde yatmaktadır.

Buna resmi tanıma geçmeden önce, basit bir örnekle bunu gösterelim.

ÖRNEK 2.12

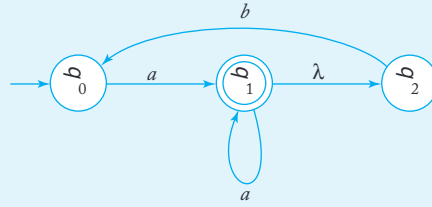
Şekil 2.12'deki nfa'yı eşdeğer bir dfa'ya dönüştürün. nfa, durum q_0 'da başlar, bu nedenle dfa'nın başlangıç durumu $\{q_0\}$ olarak etiketlenecektir. Sonrasında bir a okuduğunda, nfa durum q_1 'de veya bir λ -geçiş yaparak, durum q_2 'de olabilir. Bu nedenle, karşılık gelen dfa'nın bir durumu $\{q_1, q_2\}$ olarak etiketlenmeli ve bir geçiş

$$\delta(\{q_0\}, a) = \{q_1, q_2\} \text{ olmalıdır.}$$

Durum q_0 'da, nfa'nın girdi b olduğunda belirli bir geçişi yoktur; bu nedenle,

$$\delta(\{q_0\}, b) = \emptyset.$$

$\emptyset$ ile etiketlenmiş bir durum, nfa için imkansız bir hareketi temsil eder ve bu nedenle, dizeyi kabul etmemek anlamına gelir. Sonuç olarak, bu durum dfa'da sonlu olmayan bir tuzak durumu olmalıdır.



ŞEKİL 2.12

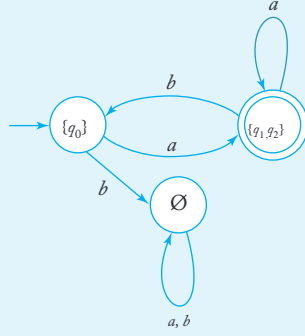
Artık dfa'ya durum $\{q_1, q_2\}$ ekledik, bu nedenle bu durumdan çıkış geçişlerini bulmamız gerekiyor. Unutmayın ki bu dfa durumu, nfa'nın iki olası durumuna karşılık gelir, bu yüzden nfa'ya geri dönmemiz gerekiyor. Eğer nfa, durum q_1 'deyse ve bir a okursa, q_1 'ye gidebilir. Ayrıca, nfa q_1 'den λ -geçiş yaparak q_2 'ye geçebilir. Eğer, aynı girdi, nfa durumundadır q_2 , o zaman belirtilmiş bir geçiş yoktur. Therefore,

$$\delta(\{q_1, q_2\}, a) = \{q_1, q_2\}.$$

Aynı şekilde,

$$\delta(\{q_1, q_2\}, b) = \{q_0\}.$$

Bu noktada, her durumun tüm geçişleri tanımlanmıştır. Sonuç, Şekil 2.13'te gösterildiği gibi, başlangıçta kullandığımız nfa ile eşdeğer bir dfa'dır. Şekil 2.12'deki nfa, herhangi bir dizeyi kabul eder ki $\delta^*(q_0, w)$ contains q_1 . For the corresponding dfa to accept every such w , any etiketi q_1 olan durum son durum haline getirilmelidir.



ŞEKİL 2.13

TEOREM 2.2

Let L , nondeterministic finite accepter $M_N =$

$(Q_N, \Sigma, \delta_N, q_0, F_N)$ tarafından kabul edilen dil olsun. O zaman, aşağıdaki gibi bir deterministic finite accepter $M_D = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D)$ vardır

$$L = L(M_D).$$

Kanıt: M_N verildiğinde, aşağıdaki nfa-to-dfa prosedürünü kullanarak M_D için geçiş grafi G_D 'yi inşa ediyoruz. İnşayı anlamak için, G_D 'nin belirli özelliklere sahip olması gerektiğini unutmayın. Her bir köşe tam olarak $|\Sigma|$ çıkış kenarına sahip olmalıdır, her biri Σ 'nin farklı bir elemanı ile etiketlenmiştir. İnşa sırasında, bazı kenarlar eksik olabilir, ancak prosedür tüm kenarlar mevcut olana kadar devam eder.

prosedür: nfa-to-dfa

1. Bir grafik G_D oluşturun $\{q_0\}$. Bu köşeyi başlangıç

köşesi olarak tanımlayın.

2. Aşağıdaki adımları, daha fazla kenar eksik kalmayana kadar tekrarlayın.

G_D içindeki herhangi bir köşe $\{q_i, q_j, \dots, q_k\}$ alarak, bazıları için dışarı giden kenarı olmayan bir köşe seçin $a \in \Sigma$. Hesaplayın $\delta_N^*(q_i, a), \delta_N^*(q_j, a), \dots, \delta_N^*(q_k, a)$. Eğer

$$\delta_N^*(q_i, a) \cup \delta_N^*(q_j, a) \cup \dots \cup \delta_N^*(q_k, a) = \{q_l, q_m, \dots, q_n\},$$

G_D için $\{q_l, q_m, \dots, q_n\}$ etiketli bir köşe oluşturun, eğer zaten mevcut değilse.

G_D 'ye $\{q_i, q_j, \dots, q_k\}$ 'den $\{q_l, q_m, \dots, q_n\}$ 'ye bir kenar ekleyin ve bunu a ile etiketleyin.

3. GD'nin etiketinde herhangi bir $qf \in FN$ bulunan her durum, bir son köşe olarak tanımlanır.
4. Eğer $M_N^{\lambda^*}$ 'yi kabul ederse, GD 'deki $\{q_0\}$ köşesi de bir son köşe haline gelir.

Bu prosedürün her zaman sona erdiği açıktır. Adım 2'deki döngüde her geçiş, GD 'ye bir kenar ekler. Ancak GD , en fazla $2 \mid Q \mid N \mid \mid \Sigma \mid$ kenarına sahiptir, bu nedenle döngü nihayetinde durur. Yapının doğru cevabı verdiğini göstermek için, girdi dizisinin uzunluğu üzerinde matematiksel induksiyon ile argüman yürütüyoruz.

Herhangi bir v uzunluğun n 'den küçük veya eşit olduğunda, GN 'de q_0 'dan q_i 'ye etiketli v ile bir yürüyüşün varlığı, GD 'de $\{q_0\}$ 'dan bir durumu $Q_i = \{\dots, q_i, \dots\}$ 'ye etiketli v ile bir yürüyüş olduğu anlamına gelir. Şimdi herhangi bir $w = va$ alalım ve GN 'de q_0 'dan q_i 'ye etiketli w ile bir yürüyüşü bakalım. O zaman q_0 'dan q_i 'ye etiketli v ile bir yürüyüş ve q_i 'den q_i 'ye etiketli a ile bir kenar (veya kenar dizisi) olmalıdır. İndüktif varsayımına göre, GD 'de $\{q_0\}$ 'dan q_i 'ye etiketli v ile bir yürüyüş olacaktır. Ancak inşa gereği, Q_i 'den q_i 'yi içeren bir etiketle bazı bir durumdan bir kenar olacaktır. Böylece, indüktif varsayım $n + 1$ uzunluğundaki tüm diziler için geçerlidir. Bu, açıkça $n = 1$ için doğru olduğundan, tüm n için doğrudur. Sonuç olarak, her zaman $\delta_N^*(q_0, w)$ son durum q_f içeriyorsa, $\delta^*_D(q_0, w)$ etiketinin de içerdiği. Kanıtı tamamlamak için, argümanı tersine çeviriyoruz ve eğer $\delta^*_D(q_0, w)$ etiketinde qf varsa, o zaman $\delta_N^*(q_0, w)$ de olmalıdır.

■

Bu kanıttaki argümanlar, doğru olmalarına rağmen, kabul etmek gerekir ki biraz kısa, yalnızca ana adımları göstermektedir. Kitabın geri kalanında bu uygulamayı izleyeceğiz, bir kanıttaki temel fikirleri vurgulayarak ve küçük detayları atlayarak, yerine kendinizin doldurmak isteyebileceği detayları bırakacağız.

Önceki kanıttaki yapılandırma zahmetli ama önemlidir. Başka bir örnek yapalım, böylece tüm adımları anladığımızdan emin olalım.

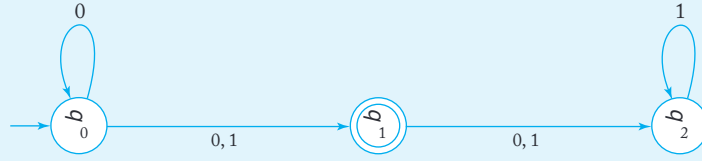
ÖRNEK 2.13

Şekil 2.14'teki nfa'yı eşdeğer bir deterministik makineye dönüştürün.

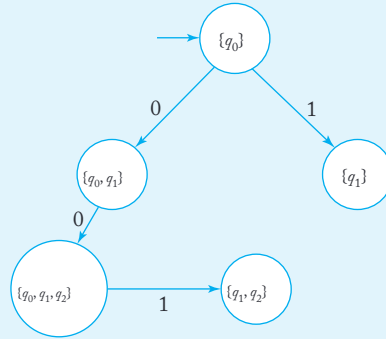
$\delta_N(q_0, 0) = \{q_0, q_1\}$ olduğundan, GD 'de durum $\{q_0, q_1\}$ 'yi tanıtıyoruz ve $\{q_0\}$ ile $\{q_0, q_1\}$ arasında 0 etiketli bir kenar ekliyoruz. Aynı şekilde, $\delta_N(q_0, 1) = \{q_1\}$ 'yi dikkate almak, bize yeni durum $\{q_1\}$ ve onunla $\{q_0\}$ arasında 1 etiketli bir kenar verir.

Artık birkaç eksik kenar var, bu yüzden Teorem 2.2'nin yapısını kullanarak devam ediyoruz. Durum $\{q_0, q_1\}$ 'ye baktığımızda, 0 etiketli bir çıkış kenarının olmadığını görüyoruz, bu yüzden hesaplıyoruz

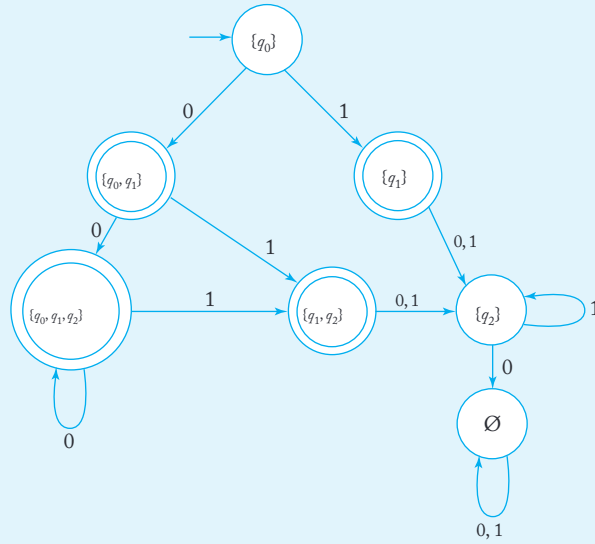
$$\delta_N^*(q_0, 0) \cup \delta_N^*(q_1, 0) = \{q_0, q_1, q_2\}.$$



ŞEKİL 2.14



ŞEKİL 2.15



ŞEKİL 2.16

Bu bize yeni durumu $\{q_0, q_1, q_2\}$ ve geçişi

$$\delta_D(\{q_0, q_1\}, 0) = \{q_0, q_1, q_2\} \text{ verir.}$$

O zaman, $a = 1, i = 0, j = 1, k = 2,$

$$\delta_N^*(q_0, 1) \cup \delta_N^*(q_1, 1) \cup \delta_N^*(q_2, 1) = \{q_1, q_2\}$$

başka bir durumu tanıtmayı gerekli kılar $\{q_1, q_2\}$. Bu noktada, Şekil 2.15'te gösterilen kısmen inşa edilmiş otomata sahibiz. Hala bazı eksik kenarlar olduğu için, Şekil 2.16'da tam çözümü elde eden e kadar devam ediyoruz.

Teorem 2.2'den çıkarabileceğimiz önemli bir sonuç, her nfa tarafından kabul edilen dilin düzenli olduğudur.

EGZERSİZLER

1. Şekil 2.10'daki nfa'yı dfa'ya dönüştürmek için Teorem 2.2'nin yapısını kullanın. Daha doğru dan daha basit bir cevap görebiliyor musunuz?
2. Egzersiz 13, Bölüm 2.2'deki nfa'yı eşdeğer bir dfa'ya dönüştürün.
3. tanımlanan nfa'yı dönüştürün

$$\delta(q_0, a) = \{q_0, q_1\}$$

$$\delta(q_1, b) = \{q_1, q_2\}$$

$$\delta(q_2, a) = \{q_2\}$$

başlangıç durumu q_0 ve son durum q_2 olan eşdeğer bir dfa'ya dönüştürün.

4. tanımlanan nfa'yı dönüştürün

$$\delta(q_0, a) = \{q_0, q_1\}$$

$$\delta(q_1, b) = \{q_1, q_2\}$$

$$\delta(q_2, a) = \{q_2\}$$

$$\delta(q_0, \lambda) = \{q_2\}$$

başlangıç durumu q_0 ve son durum q_2 olan eşdeğer bir dfa'ya dönüştürün.

5. tanımlanan nfa'yı dönüştürün

$$\delta(q_0, a) = \{q_0, q_1\}$$

$$\delta(q_1, b) = \{q_1, q_2\}$$

$$\delta(q_2, a) = \{q_2\}$$

$$\delta(q_1, \lambda) = \{q_1, q_2\}$$

başlangıç durumu q_0 ve son durum q_2 olan eşdeğer bir dfa'ya dönüştürün.

6. Teorem 2.2'nin kanıtındaki argümanları dikkatlice tamamlayın. Eğer $\delta_D^*(q_0, w)$ etiketinde q_f varsa, o zaman $\delta_N^*(q_0, w)$ de q_f içerir.
7. Herhangi bir nfa $M = (Q, \Sigma, \delta, q_0, F)$ için, $L(M)$ 'nin tamamlayıcısının $\{w \in \Sigma^* : \delta^*(q_0, w) \cap F = \emptyset\}$ ile eşit olduğu doğru mu? Eğer öyleyse, kanıtlayın. Değilse, bir karşı örnek verin.
8. Her nfa $M = (Q, \Sigma, \delta, q_0, F)$ için, $L(M)$ 'nin tamamlayıcısının $\{w \in \Sigma^* : \delta^*(q_0, w) \cap (Q - F) = \emptyset\}$ ile eşit olduğu doğru mu? Eğer öyleyse, kanıtlayın; değilse, bir karşı örnek verin.
9. Herhangi bir son durum sayısına sahip nfa için, yalnızca bir son duruma sahip eşdeğer bir nfa'nın var olduğunu kanıtlayın. Benzer bir iddiayı dfa'lar için yapabilir miyiz?
10. λ -geçişleri olmayan ve tek bir son duruma sahip bir nfa bulun ve bu nfa'nın kabul ettiği küme $\{a\} \cup \{b^n : n \geq 2\}$ olsun.
11. λ içermeyen bir düzenli dil L tanımlayın. λ -geçişleri olmayan ve tek bir son duruma sahip bir nfa'nın var olduğunu gösterin ve bu nfa'nın L 'yi kabul ettiğini kanıtlayın.
12. Bir dfa'yı, 2.2. Bölümdeki Alıştırma 18'deki karşılık gelen nfa'ya benzer şekilde çoklu başlangıç durumları ile tanımlayın. Tek bir başlangıç durumu olan eşdeğer bir dfa her zaman var mıdır?
13. Tüm sonlu dillerin düzenli olduğunu kanıtlayın.
14. Eğer L düzenliyse, o zaman L^R de düzenlidir.
15. **Şekil**
2.16'daki dfa tarafından kabul edilen dilin basit bir sözlü tanımını verin. Bunu, verilen dfa ile eşdeğer, ancak daha az duruma sahip başka bir dfa bulmak için kullanın.
16. Herhangi bir dil L olsun. $even(w)$ ifadesini, w 'den çift numaralı konumlardaki harfleri çıkararak elde edilen dize olarak tanımlayın; yani, eğer

$$w = a_1 a_2 a_3 a_4 \dots,$$

o zaman

$$even(w) = a_2 a_4 \dots$$

Buna karşılık olarak, bir dil tanımlayabiliriz

$$even(L) = \{even(w) : w \in L\}.$$

Eğer L düzenliyse, o zaman $even(L)$ de düzenlidir.

17. Bir dil L 'den, her dizedeki en soldaki sembolü kaldırarak yeni bir dil, $chopleft(L)$ oluşturuyoruz. Özellikle,

$$chopleft(L) = \{w : vw \in L, |v| = 1\}.$$

Gösterin ki eğer L düzenli ise, o zaman $chopleft(L)$ de düzenlidir.

18. Bir dil L 'den, her bir dizeden en sağdaki sembolü kaldırarak yeni bir dil $chopright(L)$ oluşturuyoruz. Özellikle,

$$chopright(L) = \{w : wv \in L, |v| = 1\}.$$

Gösterin ki eğer L düzenli ise, o zaman $chopright(L)$ de düzenlidir.

2.4 SONLU OTOMATLARDA DURUM SAYISININ AZALTILMASI*

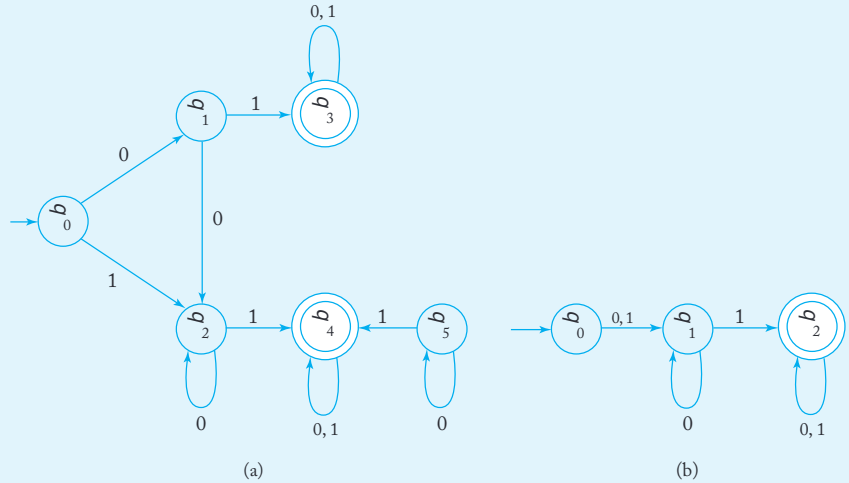
Her dfa benzersiz bir dil tanımlar, ancak tersinin doğru olduğu söylenemez. Verilen bir dil için, onu kabul eden birçok dfa vardır. Bu tür eşdeğer otomatların durum sayısında önemli bir fark olabilir. Şu ana kadar ele aldığımız sorular açısından, tüm çözümler eşit derecede tatmin edicidir, ancak s onuçların pratik bir ortamda uygulanması gerektiğinde, birinin diğerine tercih edilmesi için nedenler olabilir.

ÖRNEK 2.14

Şekil 2.17(a) ve 2.17(b) de gösterilen iki dfa eşdeğerdir, birkaç test dizisi bunu hızla ortaya koyacaktır. Şekil 2.17(a)'nın bazı açıkça gereksiz özelliklerini fark ediyoruz. Durum q_5 , başlangıç durumu q_0 'dan asla ulaşamayacağı için otomatta hiçbir rol oynamaz. Böyle bir durum erişilemezdir ve otomatonun kabul ettiği dili etkilemeden (ona ilişkin tüm geçişlerle birlikte) kaldırılabilir. Ancak q_5 'in kaldırılmasından sonra bile, ilk otomatta bazı gereksiz parçalar vardır. İlk hareketten sonra ulaşılabilir durumlar $\delta(q_0, 0)$,

ilk hareket $\delta(q_0, 1)$. İkinci otomaton bu iki

seçeneği birleştirir.



ŞEKİL 2.17

Kesin teorik bir bakış açısıyla, Şekil 2.17(b) otomatonunu Şekil 2.17(a) otomatonuna tercih etmek için pek az neden vardır. Ancak, basitlik açısından, ikinci alternatif açıkça daha tercih edilir. Hesaplama amacıyla bir otomatonun temsili, durum sayısına orantılı bir alan gerektirir. Depolama verimliliği için, durum sayısını mümkün olduğunca azaltmak arzu edilir. Şimdi bunu başaran bir algoritmayı tanımlıyoruz.

TANIM 2.8

Bir dfa'nın p ve q iki durumu ayırt edilemez olarak adlandırılır eğer

$$\delta^*(p, w) \in F \text{ ise } \delta^*(q, w) \in F,$$

ve

$$\delta^*(p, w) \notin F \text{ ise } \delta^*(q, w) \notin F,$$

tüm $w \in \Sigma^*$ için. Öte yandan, bazı bir dize $w \in \Sigma^*$ varsa

$$\delta^*(p, w) \in F \text{ ve } \delta^*(q, w) \notin F,$$

veya tersine, o zaman durumlar p ve q bir

string w ile ayırt edilebilir.

Açıkça, iki durum ya ayırt edilemez ya da ayırt edilebilir. Ayırt edilemezlik, bir eşdeğerlik ilişkisi özelliklerine sahiptir: Eğer p ve q ayırt edilemezse ve eğer q ve r de ayırt edilemezse, o zaman p ve r de öyle olur ve üç durum da ayırt edilemez.

Bir dfa'nın durumlarını azaltmanın bir yöntemi, ayırt edilemeyen durumları bulmak ve birleştirmeye dayanmaktadır. Öncelikle, ayırt edilebilir durum çiftlerini bulma yöntemini tanımlıyoruz.

prosedür: işaret

1. Tüm erişilemeyen durumları kaldırın. Bu, başlangıç durumundan başlayarak dfa'nın grafiğinin tüm basit yollarını sayarak yapılabilir. Herhangi bir yolun parçası olmayan durum erişilemezdir.
2. Tüm durum çiftlerini (p, q) dikkate alın. Eğer $p \in F$ ve $q \notin F$ veya tersine, çifti (p, q) ayırt edilebilir olarak işaretleyin.
3. Daha önce işaretlenmemiş çiftler işaretlenene kadar aşağıdaki adımı tekrarlayın. Tüm çiftler için (p, q) ve tüm $a \in \Sigma$, hesaplayın $\delta(p, a) = p_a$ ve $\delta(q, a) = q_a$. Eğer (p_a, q_a) çifti ayırt edilebilir olarak işaretlenmişse, (p, q) çifti ayırt edilebilir olarak işaretleyin.

Bu prosedürün tüm ayırt edilebilir çiftleri işaretlemek için bir algoritma oluşturduğumu iddia ediyoruz.

TEOREM 2.3

Uygulanan prosedür işaret, herhangi bir dfa $M = (Q, \Sigma, \delta, q_0, F)$ için sonlanır ve tüm ayırt edilebilir durum çiftlerini belirler.

Kanıt: Açıkça, prosedür sonlanır, çünkü işaretlenebilecek yalnızca sonl usayıda çift vardır. Ayrıca, işaretlenen herhangi bir çiftin durumlarının ayırt edilebilir olduğu da kolayca görülebilir. Geliştirilmesi gereken te k iddia, prosedürün tüm ayırt edilebilir çiftleri bulduğudur.

Öncelikle, durumların q_i ve q_j , yalnızca geçişler varsa, uzunluğun n olan bir dizi ile ayırt e dilebilir olduğunu not edin.

$$\delta(q_i, a) = q_k \quad (2.5)$$

ve

$$\delta(q_j, a) = q_l, \quad (2.6)$$

bazı $a \in \Sigma$, ile q_k ve q_l , uzunluğun $n-1$ olan bir dizi ile ayırt edilebilir. İlk olarak, n 'inci geçişin tamamlanmasında, uzunluğun n veya daha az olan dizi lerle ayırt edilebilen tüm durumların işaretlendiğini göstermek için bunu kull anıyoruz. 2. adımda, λ ile ayırt edilemeyen tüm çiftleri işaretliyoruz, böylece $n = 0$ için bir temelimiz var. Şimdi, iddianın tüm $i = 0, 1, \dots, n-1$ için doğru oldu ğ unu varsayıyoruz. Bu tümevarım varsayımına göre, döngüde n 'inci geçişin b aşında, uzunluğu $n-1$ 'e kadar olan dizilerle ayırt edilebilen tüm durumlar iş aretlenmiştir. Yukarıdaki (2.5) ve (2.6) nedeniyle, bu geçişin sonunda, uzunlu ğ u n kadar olan dizilerle ayırt edilebilen tüm durumlar işaretlenecektir. Dolay ısıyla, her n için, n 'inci geçişin tamamlanmasında, uzunluğu n veya daha az ol an dizilerle ayırt edilebilen tüm çiftlerin işaretlendiğini iddia edebiliriz.

Bu prosedürün tüm ayırt edilebilir durumları işaretlediğini göstermek için, dö ngünün n geçişinden sonra sona erdiğini varsayalım. Bu, n 'inci geçiş sırasında yeni durumların işaretlenmediği anlamına gelir. (2.5) ve (2.6) dan, uzunluğu n olan bir dizi ile ayırt edilebilen durumların olamayacağı sonucuna varılır, ancak daha kısa b ir dizi ile ayırt edilemeyen durumlar vardır. Ancak, yalnızca uzunluğu n olan dizilerl e ayırt edilebilen durumlar yoksa, yalnızca uzunluğu $n+1$ olan dizilerle ayırt edileb ilen durumlar da olamaz ve bu böyle devam eder. Sonuç olarak, döngü sona erdiği nde, tüm ayırt edilebilir çiftler işaretlenmiştir.

Prosedür $mark$, durumları eşdeğer sınıflara ayırarak uygulanabilir. İki durum ayırt edilebilir bulunduğunda, hemen ayrı eşdeğer sınıflara konulurlar.

ÖRNEK 2.15

Şekil 2.18'deki otomata bakın.

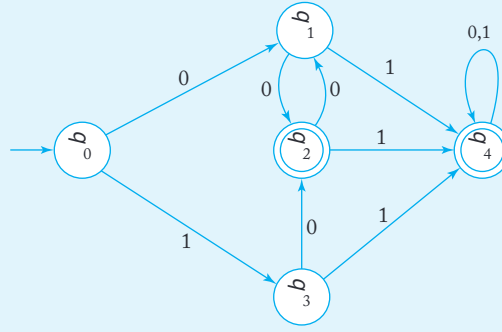
Prosedür_{mark}'ın ikinci adımında, durum kümesini sonlu ve sonsuz durumlar olarak ayırarak iki eşdeğer sınıf elde ederiz $\{q_0, q_1, q_3\}$ ve $\{q_2, q_4\}$. Bir sonraki adımda, hesapladığımızda

$$\delta(q_0, 0) = q_1$$

ve

$$\delta(q_1, 0) = q_2,$$

q_0 ve q_1 'in ayırt edilebilir olduğunu tanırız, bu yüzden onları farklı kümeler koyarız. Böylece $\{q_0, q_1, q_3\}$, $\{q_0\}$ ve $\{q_1, q_3\}$ olarak bölünür. Ayrıca, $\delta(q_2, 0) = q_3$ ve $\delta(q_4, 0) = q_4$ olduğundan, sınıf $\{q_2, q_4\}$ $\{q_2\}$ ve $\{q_4\}$. Geriye kalan hesaplamalar, daha fazla bölmeye gerek olmadığını gösteriyor.



ŞEKİL 2.18

Belirsizlik sınıfları bulunduğunda, minimal dfa'nın inşası basittir.

prosedür: azalt

Bir dfa $M = (Q, \Sigma, \delta, q_0, F)$ verildiğinde, bir azaltılmış dfa $\widehat{M} = (\widehat{Q}, \Sigma, \widehat{\delta},$

$\widehat{q}_0, \widehat{F})$ aşağıdaki gibidir.

1. Eşdeğer sınıfları oluşturmak için prosedürü işaretli kullanın, diyelim $\{q_i, q_j, \dots, q_k\}$, a çıktığı gibi.
2. Her bir $\{q_i, q_j, \dots, q_k\}$ gibi ayırt edilemeyen durumlar kümesi için, bir durum oluşturun etiketli $j \dots k$ için $\widehat{M}$.

3. M için geçiş kuralının her biri şu biçimdedir

$$\delta(q_r, a) = q_p,$$

q_r ve q_p 'nin ait olduğu kümeleri bulun. Eğer $q_r \in \{q_i, q_j, \dots, q_k\}$ ve $q_p \in \{q_l, q_m, \dots, q_n\}$ ise, δ 'ye bir kural ekleyin

$$\widehat{\delta}(i \cdot j \cdots k, a) = l \cdot m \cdots n.$$

4. Başlangıç durumu $\widehat{q}_0$, 0'ı içeren $\widehat{M}$ durumudur.

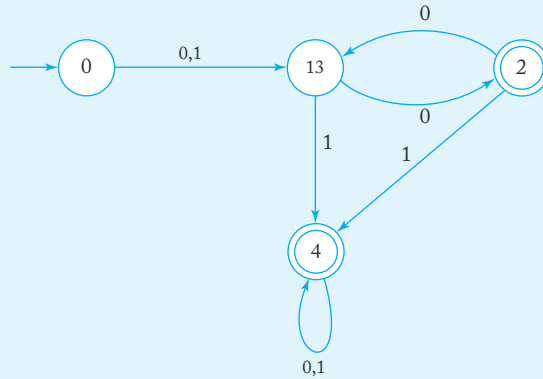
5. $\widehat{F}$, etiketinde i bulunan tüm durumların kümesidir böylece $q_i \in F$.

ÖRNEK 2.16

Örnek 2.15 ile devam ederek, Şekil 2.19'daki durumları oluşturuyoruz. Örneğin,

$$\delta(q_1, 0) = q_2,$$

durum 13'ten durum 2'ye etiketli 0 olan bir kenar vardır. Diğer tüm geçişler kolayca bulunur ve Şekil 2.19'daki minimal dfa'yı verir.



ŞEKİL 2.19

TEOREM 2.4

Herhangi bir dfa M verildiğinde, reduce prosedürünün uygulanması başka bir dfa $\widehat{M}$ üretir böylece

$$L(M) = L(\widehat{M}).$$

Ayrıca, $\widehat{M}$, başka bir dfa'nın daha küçük bir durum sayısına sahip olmadığı anlamında minimaldir ve bu dfa $L(M)$ 'yi de kabul eder.

Kanıt: İki bölüm vardır. Birincisi, reduce ile oluşturulan dfa'nın orijinal dfa ile eşdeğer olduğunu göstermektir. Bu oldukça kolaydır ve dfa'lar ile nfa'lar arasındaki eşdeğerliliği kurarken kullanılan benzer indüktif argümanları kullanabiliriz. Yapmamız gereken tek şey, $\delta^*(q_i, w) = q_j$ olduğunu göstermek ve bu yalnızca $\delta^*(q_i, w)$ etiketini $n \dots j \dots$ biçiminde olması durumunda geçerlidir. Bunu bir alıştırmaya bırakacağız.

İkinci bölüm, $\widehat{M}$ 'nin minimal olduğunu göstermek daha zordur. Farz edelim ki $\widehat{M}$, başlangıç durumu p_0 olan durumlar $\{p_0, p_1, p_2, \dots, p_m\}$ içermektedir. Eşdeğer bir dfa M_1 olduğunu varsayalım, geçiş fonksiyonu δ_1 ve başlangıç durumu q_0 olan, ancak daha az duruma sahip olan $\widehat{M}$ ile eşdeğer. Çünkü $\widehat{M}$ 'de erişilemeyen durum yoktur, bu durumda $w_1, w_2, \dots, w_m$ gibi farklı dizgeler olmalıdır.

$$\widehat{\delta}^*(p_0, w_i) = p_i, i = 1, 2, \dots, m.$$

Ancak M_1 , $\widehat{M}$ 'den daha az duruma sahip olduğundan, bu dizgelerden en az ikisi, diyelim ki w_k ve w_l , olmalıdır.

$$\delta_1^*(q_0, w_k) = \delta_1^*(q_0, w_l).$$

p_k ve p_l ayırt edilebilir olduğundan, bazı dizgeler x olmalıdır ki

$\widehat{\delta}^*(p_0, w_k x) = \widehat{\delta}^*(p_k, x)$ son durumdur ve $\widehat{\delta}^*(q_0, w_l x) = \widehat{\delta}^*(p_l, x)$ bir son olmayan durumdur (veya tersine). Diğer bir deyişle, $w_k x$ $\widehat{M}$ tarafından kabul edilir ve $w_l x$ kabul edilmez. Ancak şunu unutmayın

$$\begin{aligned} \delta_1^*(q_0, w_k x) &= \delta_1^*(\delta_1^*(q_0, w_k), x) \\ &= \delta_1^*(\delta_1^*(q_0, w_l), x) \\ &= \delta_1^*(q_0, w_l x). \end{aligned}$$

Böylece, M_1 ya hem $w_k x$ hem de $w_l x$ kabul eder ya da ikisini de reddeder, bu da $\widehat{M}$ ve M_1 eşdeğerdir varsayımına aykırıdır. Bu çelişki, M_1 'in var olamayacağını kanıtlar. ■

EGZERSİZLER

1. Başlangıç durumu q_0 , son durum q_2 ve

$$\begin{aligned} \delta(q_0, a) &= q_2 & \delta(q_0, b) &= q_2 \\ \delta(q_1, a) &= q_2 & \delta(q_1, b) &= q_2 \\ \delta(q_2, a) &= q_3 & \delta(q_2, b) &= q_3 \\ \delta(q_3, a) &= q_3 & \delta(q_3, b) &= q_1 \end{aligned}$$

Minimal bir eşdeğer dfa bulun.

2. Şekil 2.16'daki dfa'daki durum sayısını minimize edin.
3. Minimal dfa'ları aşağıdaki diller için bulun. Her durumda, sonucun minimal olduğunu kanıtlayın.
 - (a) $L = \{a^n b^m : n \geq 1, m \geq 2\}$.
 - (b) $L = \{a^n b : n \geq 1\} \cup \{b^n a : n \geq 1\}$.
 - (c) $L = \{a^n : n \geq 0, n \neq 2\}$.
 - (d) $L = \{a^n : n \neq 3 \text{ ve } n \neq 4\}$.
 - (e) $L = \{a^n : n \bmod 3 = 1\} \cup \{a^n : n \bmod 5 = 1\}$.
4. reduce prosedürü tarafından üretilen otomatonun deterministik olduğunu gösterin.
5. L , L içindeki herhangi bir w 'nin uzunluğunun en az n olduğu boş olmayan bir dil ise, L 'yi kabul eden herhangi bir dfa'nın en az $n + 1$ duruma sahip olması gerektiğini gösterin.
6. Aşağıdaki varsayımı kanıtlayın veya çürütün. Eğer $M = (Q, \Sigma, \delta, q_0, F)$ düzenli bir dil için minimal bir dfa ise, o zaman $\widehat{M} = (Q, \Sigma, \delta, q_0, Q - F)$ L için minimal bir dfa'dır.
7. Belirsizlik ilişkisi bir eşdeğerlik ilişkisi olduğunu, ancak ayırt edilebilirliğin olmadığını gösterin.
8. Önerilen kanıtın birinci kısmının açık adımlarını gösterin, yani $\widehat{M}$ orijinal dfa ile eşdeğerdir.
9. Aşağıdakileri kanıtlayın: Eğer durumlar q_a ve q_b ayırt edilemezse, ve eğer q_a ve q_c ayırt edilebilirse, o zaman q_b ve q_c ayırt edilebilir olmalıdır.

BÖLÜM

3

DÜZENLİ DİLLER VE DÜZENLİ GRAMERLER

BÖLÜM ÖZETİ

Bu bölümde, düzenli dilleri tanımlamak için iki alternatif yöntemi keşfedeceğiz: düzenli ifadeler ve düzenli gramerler.

Düzenli ifadeler, biçimi tanıdık aritmetik ifadeleri andıran, bazı uygulamalarda basit dize biçimleri nedeniyle kullanışlıdır, ancak sınırlıdır ve daha karmaşık diller için belirgin bir genişletme olanağı yoktur. Öte yandan, düzenli gramerler, birçok farklı gramer türünün özel bir durumu dur.

Bu bölümün amacı, düzenli dilleri tanımlamanın bu üç modunun temel eşdeğerliğini keşfetmektir. Bir biçimden diğerine dönüşümü sağlayan yapılar, bu bölümdeki teoremlerde bulunmaktadır. İlişkileri Şekil 3.19'da gösterilmektedir.

Her düzenli dilin her bir temsiline tamamen diğerlerine dönüştürülebilir olması nedeniyle, mevcut duruma en uygun olanı seçebiliriz. Bunu hatırlamak, işinizi sıklıkla kolaylaştıracaktır.

Tanımımıza göre, bir dil düzenli ise, onun için sonlu bir kabul edici varsa vardır. Bu nedenle, her düzenli dil, bazı dfa veya bazı nfa ile tanımlanabilir. Bu tür bir tanım, örneğin, belirli bir dizeyi belirli bir dilde olup olmadığını gösterme mantığını açıklamak istediğimizde çok yararlı olabilir. Ancak birçok durumda, daha özlü yollar gereklidir.

Düzenli dilleri tanımlamanın bir yolu, düzenli dillerin başka yollarını temsil etmektir. Bu bölümde, düzenli dilleri temsil etmenin diğer yollarına bakıyoruz. Bu temsillerin önemli pratik uygulamaları vardır, bu konu bazı

örnekler ve alıştırmalarda ele alınmaktadır.

3.1 DÜZENLİ İFADELER

Düzenli dilleri tanımlamanın bir yolu, düzenli ifadelerin notasyonudur. Bu notasyon, bazı alfabelerden Σ sembollerinin stringlerinin, parantezlerin ve $+$, $\cdot$ ve $*$ operatörlerinin bir kombinasyonunu kapsar. En basit durum, düzenli ifade a ile gösterilecek olan $\{a\}$ dilidir. Biraz daha karmaşık olan ise, birleşimi belirtmek için $+$ kullandığımız $\{a, b, c\}$ dilidir, ki bunun için düzenli ifade $a+b+c$ 'dir.

Birleştirme için $\cdot$ ve yıldız-kapatma için $*$ benzer bir şekilde kullanılır. İfade $(a+(b \cdot c))^*$, $\{a\} \cup \{bc\}$ dilinin yıldız-kapatmasını ifade eder, yani $\{\lambda, a, bc, aa, abc, bca, bcba, aaa, aabc, \dots\}$ dilidir.

Düzenli İfadenin Resmi Tanımı

Basit bileşenlerden düzenli ifadeler oluşturmak için belirli tekrarlayan kuralları sürekli olarak uygularız. Bu, tanıdık aritmetik ifadeleri oluşturma şeklimize benzer.

DEFINITION 3.1

Verilen bir alfabe Σ olarak alalım. O zaman

1. $\emptyset, \lambda$ ve $a \in \Sigma$ hepsi düzenli ifadeleridir. Bunlara ilkel düzenli ifadeler denir.
2. Eğer r_1 ve r_2 düzenli ifadeler ise, o zaman $r_1+r_2, r_1 \cdot r_2, r_1^*$ ve (r_1) de düzenli ifadelerdir.
3. Bir dize, yalnızca ilkel düzenli ifadelerden (2)'deki kuralların sonlu sayıda uygulanmasıyla türetiliyorsa düzenli bir ifadedir.

EXAMPLE 3.1

$\Sigma = \{a, b, c\}$ için, dize

$$(a+b \cdot c)^* \cdot (c+\emptyset)$$

düzenli bir ifadedir, çünkü yukarıdaki kuralların uygulanmasıyla oluşturulmuştur. Örneğin, eğer $r_1 = c$ ve $r_2 = \emptyset$ alırsak, şunu buluruz:

$c + \emptyset$ ve $(c+\emptyset)$ de düzenli ifadelerdir. Bunu tekrarlayarak, nihayet inde tüm dizeyi üretiriz. Öte yandan, $(a+b +)$ bir düzenli ifade değildir, çünkü ilkel düzenli ifadelere dayanarak yapılamaz.

Düzenli İfadelerle İlişkili Diller

Düzenli ifadeler bazı basit dilleri tanımlamak için kullanılabilir. Eğer r bir düzenli ifade ise, $L(r)$ ifadesi r ile ilişkili dili belirtir.

DEFINITION 3.2

Herhangi bir düzenli ifade r ile gösterilen dil $L(r)$ aşağıdaki kurallarla tanımlanır.

1. $\emptyset$ boş küme gösteren bir düzenli ifadedir,
2. $\lambda\{\lambda\}$ gösteren bir düzenli ifadedir,
3. Her $a \in \Sigma$ için, $a\{a\}$ gösteren bir düzenli ifadedir.

Eğer r_1 ve r_2 düzenli ifadeler ise, o zaman

4. $L(r_1 + r_2) = L(r_1) \cup L(r_2)$,
5. $L(r_1 \cdot r_2) = L(r_1)L(r_2)$,
6. $L((r_1)) = L(r_1)$,
7. $L(r_1^*) = (L(r_1))^*$.

Bu tanımın son dört kuralı, $L(r)$ 'yi daha basit bileşenlere özyinelemeli olarak indirgemek için kullanılır; ilk üçü ise bu özyineleme için sonlandırma koşullarıdır. Verilen bir ifadenin hangi dili belirttiğini görmek için bu kuralları tekrar tekrar uygulayınız.

ÖRNEK 3.2

Dili $L(a^* \cdot (a + b))$ küme notasyonu ile sergileyin.

$$\begin{aligned}
 L(a^* \cdot (a + b)) &= L(a^*) L(a + b) \\
 &= (L(a))^* (L(a) \cup L(b)) \\
 &= \{\lambda, a, aa, aaa, \dots\} \{a, b\} \\
 &= \{a, aa, aaa, \dots, b, ab, aab, \dots\}.
 \end{aligned}$$

Tanım 3.2'deki (4) ile (7) arasındaki kurallarla ilgili bir sorun var. Bu kurallar, r_1 ve r_2 verildiğinde bir dili tam olarak tanımlar, ancak karmaşık bir ifadeyi parçalara ayırırken bazı belirsizlikler olabilir. Örneğin, düzenli ifade $a \cdot b + c$ 'yi ele alalım. Bunu $r_1 = a \cdot b$ ve $r_2 = c$ olarak düşünebiliriz. Bu durumda, $L(a \cdot b + c) = \{ab, c\}$ buluruz. Ancak, tanım 3.2'de $r_1 = a$ ve $r_2 = b + c$ almayı engelleyen hiçbir şey yoktur. Artık farklı bir sonuç elde ediyoruz, $L(a \cdot b + c) = \{ab, ac\}$. Bunu aşmak için, ifade tümüyle parantez içinde olmalıdır, ancak bu karmaşık sonuçlar verir. Bunun yerine, matematikten ve programlama dillerinden tanıdık bir kural kullanıyoruz. Değerlendirme için bir öncelik kuralı seti oluşturuyoruz; burada yıldız-kapatma, birleştirmeden önce gelir ve birleştirme, birleşimden önce gelir. Ayrıca, birleştirme sembolü atlanabilir, böylece $r_1 r_2$ yazabiliriz $r_1 \cdot r_2$.

Biraz pratikle, belirli bir düzenli ifadenin hangi dili belirttiğini hızlıca görebiliriz.

ÖRNEK 3.3

$\Sigma = \{a, b\}$ için, ifade $r = (a + b)^*$

$(a + b)^*$ düzenlidir. B

u, dili belirtir

$$L(r) = \{a, bb, aa, abb, ba, bbb, \dots\}.$$

Bunur'nin çeşitli parçalarını dikkate alarak görebiliriz. İlk parça, $(a + b)^*$, herhangi bir a ve b dizisini temsil eder. İkinci parça, $(a + b)^*$ ya bir a ya da çift b temsil eder. Sonuç olarak, $L(r)$ tüm dizelerin kümesidir $\{a, b\}$, ya bir a ya da bir bb ile sonlanır.

ÖRNEK 3.4

İfade

$$r = (aa)^*(bb)^*b$$

tüm çift sayıda a 'ler ile takip eden

tek sayıda b 'ler içeren tüm dizelerin kümesini belirtir; yani,

$$L(r) = \{a^{2n}b^{2m+1} : n \geq 0, m \geq 0\}.$$

Resmi bir tanım veya küme notasyonundan düzenli bir ifadeye geçmek biraz daha zorlayıcıdır.

ÖRNEK 3.5

$\Sigma = \{0,1\}$ için, $L(r) = \{w \in \Sigma^* : w \text{ en az bir çift ardışı}$

$k \text{ sıfır içerir}\}$ olacak şekilde bir düzenli ifade r verin.

Bir cevap, şöyle bir akıl yürütme ile elde edilebilir: $L(r)$ içindeki her dize bir yerde 00 içermelidir, ancak öncesi ve sonrası tamamen keyfidir. $\{0,1\}$ üzerinde keyfi bir dize $(0+1)^*$ ile gösterilebilir. Bu gözlemleri bir araya getirerek, bir çözüme ulaşırız

$$r = (0+1)^*00(0+1)^*.$$

ÖRNEK 3.6

Dil için bir düzenli ifade bulun

$$L = \{w \in \{0,1\}^* : w \text{ ardışık sıfır çiftine sahip değildir}\}.$$

Bu, Örnek 3.5'e benziyor gibi görünse de, cevap daha zor bir şekle kavuşturulmaktadır. Yararlı bir gözlem, her zaman a0 meydana geldiğinde, hemen ardından bir 1 gelmesi gerektiğidir. Böyle bir alt dize, keyfi sayıda 1 ile önce ve sonra gelebilir. Bu, cevabın $1 \cdots 1 01 \cdots 1$ biçimindeki dizelerin tekrarı ile ilgili olduğunu önermektedir; yani, düzenli ifade $(1^*011^*)^*$ ile belirtilen dil. Ancak, cevap hala eksik, çünkü 0 ile biten veya tamamen 1'lerden oluşan dizeler hesaba katılmamıştır. Bu özel durumları hallettikten sonra cevaba ulaşırız

$$r = (1^*011^*)^*(0+\lambda) + 1^*(0+\lambda).$$

Farklı bir şekilde düşünersek, başka bir cevap bulabiliriz. Eğer L dizgelerin 1 ve 01'in tekrarı olarak görürsek, daha kısa ifade

$$r = (1+01)^*(0+\lambda)$$

ulaşılabilir. İki ifade farklı görünse de, her ikisi de doğrudur, çünkü aynı dili belirtirler. Genel olarak, herhangi bir dil için sınırsız sayıda düzenli ifade vardır.

Bu dilin, Örnek 3.5'teki dilin tamamlayıcı olduğunu unutmayın. Ancak, düzenli ifadeler çok benzer değildir ve diller arasındaki yakın ilişkiyi açıkça önermemektedir.

Son örnek, düzenli ifadelerin eşdeğerlilik kavramını tanıtmaktadır. İki düzenli ifadenin aynı dili belirtmesi durumunda eşdeğer olduğunu söyleyebiliriz. Düzenli ifadeleri basitleştirmek için çeşitli kurallar türetebiliriz (aşağıdaki alıştırmalar bölümünde Alıştırma 20'ye bakın), ancak bu tür manipülasyonlara pek ihtiyaç duymadığımız için bunu sürdürmeyeceğiz.

EGZERSİZLER

1. Uzunluğu beş olan $L((a+bb)^*)$ içindeki tüm dizgeleri bulun.
2. Uzunluğu dörtten az olan $L((ab+b)^*b(a+ab)^*)$ içindeki tüm dizgeleri bulun.
3. Bir nfa bulunuz ki, $\text{dil}L(aa^*(ab+b))^*$ 'yi kabul etsin.
4. Bir nfa bulunuz ki, $\text{dil}L(aa^*(a+b))^*$ 'yi kabul etsin.
5. İfade $((0+1)(0+1)^*)^*00(0+1)^*$, Örnek 3.5'teki dili belirtir mi?
6. $r = (1+01)^*(0+1)^*$ ifadesinin Örnek 3.6'daki dili de belirttiğini gösteriniz. İki başka eşdeğer ifade bulunuz.
7. Set için bir düzenli ifade bulunuz $\{a^n b^m : n \geq 3, m \text{ tek}\}$.
8. Set için bir düzenli ifade bulunuz $\{a^n b^m : (n+m) \text{ tek}\}$.
9. Aşağıdaki diller için düzenli ifadeler veriniz.
 - (a) $L_1 = \{a^n b^m, n \geq 3, m \leq 4\}$.
 - (b) $L_2 = \{a^n b^m : n < 4, m \leq 4\}$.
 - (c) L_1 'in tamamlayıcı.
 - (d) L_2 'nin tamamlayıcı.
10. İfadeler $(\emptyset^*)^*$ ve $a\emptyset$ hangi dilleri belirtmektedir?
11. Dil L 'nin basit bir sözlü tanımını verin $((aa)^*b(aa)^* + a(aa)^*ba(aa)^*)$.
12. L için bir düzenli ifade verin R , burada L Egzersiz 2'deki dildir.
13. L için bir düzenli ifade verin $= \{a^n b^m : n \geq 2, m \geq 1, nm \geq 3\}$.
14. L için bir düzenli ifade bulun $= \{ab^n w : n \geq 4, w \in \{a, b\}^+\}$.
15. Dil için tamamlayıcı bir düzenli ifade bulun Örnek 3.4.
16. $L = \{v w v : v, w \in \{a, b\}^*, |v| = 2\}$ için bir düzenli ifade bulun.
17. $L = \{v w v : v, w \in \{a, b\}^*, |v| \leq 4\}$ için bir düzenli ifade bulun.
18. Düzenli bir ifade bulun

$$L = \{w \in \{0,1\}^* : w \text{ tam olarak bir çift ardışık sıfır içerir}\}.$$
19. $\Sigma = \{a, b, c\}$ için aşağıdaki diller için düzenli ifadeler verin:
 - (a) Tam olarak iki a içeren tüm dizeler.

- (b) En fazla üç a içeren tüm dizeler.
- (c) Σ 'deki her sembolden en az bir kez içeren tüm dizeler

20. Şu diller için düzenli ifadeler yazın $\{0,1\}$:

- (a) 10 ile biten tüm dizeler.
- (b) 10 ile bitmeyen tüm dizeler.
- (c) tek sayıda 0 içeren tüm dizeler.

21. Şu diller için düzenli ifadeler bulun $\{a, b\}$:

- (a) $L = \{w : |w| \bmod 3 = 0\}$.
- (b) $L = \{w : n_a(w) \bmod 3 = 0\}$.
- (c) $L = \{w : n_a(w) \bmod 5 > 0\}$.

22. Aşağıdaki iddiaların tüm düzenli ifadeler için doğru olup olmadığını belirleyin r_1 ve r_2 . Sembol $\equiv$, düzenli ifadelerin eşdeğerliliğini temsil eder, her iki ifadenin de aynı dili tanımladığı anlamında.

- (a) $(r_1^*)^* \equiv r_1^*$
- (b) $r_1^*(r_1 + r_2)^* \equiv (r_1 + r_2)^*$
- (c) $(r_1 + r_2)^* \equiv (r_1^* r_2^*)^*$
- (d) $(r_1 r_2)^* \equiv r_1^* r_2^*$

23. Herhangi bir düzenli ifadeyi r değiştirmek için genel bir yöntem verin $\hat{\cdot}_r$ böyle ki $(L(r))^R = L(\hat{\cdot}_r)$.

24. Örnek 3.6'daki ifadelerin gerçekten belirtilen dili tanımladığını titizlikle kanıtlayın.

25. λ veya $\emptyset$ içermeyen bir düzenli ifade r durumu için, eğer $L(r)$ sonsuz olursa λ ya r 'ye uyması gereken gerekli ve yeterli koşulların bir kümesini verin.

26. Resmi diller, çeşitli iki boyutlu şekilleri tanımlamak için kullanılabilir.

Chain-code dilleri, $\Sigma = \{u, d, r, l\}$ alfabesi üzerinde tanımlanmıştır; burada bu semboller sırasıyla yukarı, aşağı, sağ ve sola doğru birim uzunluğundaki düz çizgileri temsil eder. Bu notasyonun bir örneği $urdl$ olup, birim uzunluğunda kenarları olan kareyi temsil eder. İfadelerden $(rd)^*$, $(urddru)^*$ ve $(ruldr)^*$ ile belirtilen şekillerin resimlerini çizin.

27. Egzersiz 27'de, ifadenin kapalı bir kontur olması için yeterli koşullar nelerdir; yani başlangıç ve bitiş noktalarının aynı olması? Bu koşullar aynı zamanda gerekli midir?

28. Bir düzenli ifade bulun ki, bu ifade, bir ikili tam sayı olarak yorumlandığında değeri 40 veya daha büyük olan tüm bit dizilerini belirtmektedir.

29. Önde 1 bit bulunan ve bir ikili tam sayı olarak yorumlandığında değeri 10 ile 30 arasında olmayan tüm bit dizileri için bir düzenli ifade bulun.

3.2 DÜZENLİ İFADELER VE DÜZENLİ DİLLER ARASINDAKİ BAĞ

Terminolojinin de belirttiği gibi, düzenli diller ile düzenli ifadeler arasındaki bağlantı yakındır. İki kavram esasen aynıdır; her düzenli dil için bir düzenli ifade vardır ve her düzenli ifade için bir düzenli dil vardır. Bunu iki bölümde göstereceğiz.

Düzenli İfadeler Düzenli Dilleri Belirler

Öncelikle, eğer r bir düzenli ifade ise, o zaman $L(r)$ bir düzenli dildir. Tanımımız, bir dilin düzenli olduğunu, bazı dfa'lar tarafından kabul edildiğinde söyler. Nfa'lar ve dfa'lar arasındaki eşdeğerlik nedeniyle, bir dil aynı zamanda bazı nfa'lar tarafından kabul edildiğinde de düzenlidir. Şimdi, herhangi bir düzenli ifade r varsa, bu ifadeyi nfa kabul edildiği bir nfa inşa edebileceğimizi göstereceğiz. Bu inşa, $L(r)$ için özyinelemeli tanıma dayanır. Öncelikle Tanım 3.2'nin (1), (2) ve (3) kısımları için basit otomata inşa edeceğiz, ardından bunların nasıl birleştirileceğini ve daha karmaşık (4), (5) ve (7) kısımlarını uygulamak için kullanılacağını göstereceğiz.

TEOREM 3.1

Bir r düzenli ifade olsun. O zaman $L(r)$ kabul eden bazı belirsiz sonlu kabul edici vardır. Sonuç olarak, $L(r)$ düzenli bir dildir.

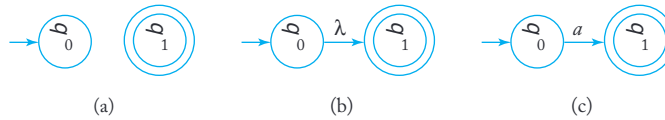
Kanıt: Basit

düzenli ifadeler için dilleri kabul eden otomatarla başlıyoruz $\emptyset, \lambda$ ve $a \in \Sigma$. Bunlar Şekil 3.1(a),

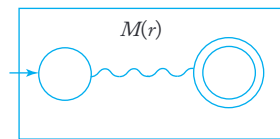
(b) ve (c)'de gösterilmiştir. Şimdi $M(r_1)$ ve

$M(r_2)$ düzenli ifadelerle gösterilen dilleri kabul eden otomatarımız olduğunu varsayıyoruz r_1 ve r_2 ,

sırasıyla. Bu otomatarı açıkça inşa etmemize gerek yoktur, ancak Şekil 3.2'de olduğu gibi şematik olarak temsil edebiliriz. Bu şemada, grafik



ŞEKİL 3.1 (a) nfa $\emptyset$ kabul eder. (b) nfa $\{\lambda\}$ kabul eder. (c) nfa $\{a\}$ kabul eder.

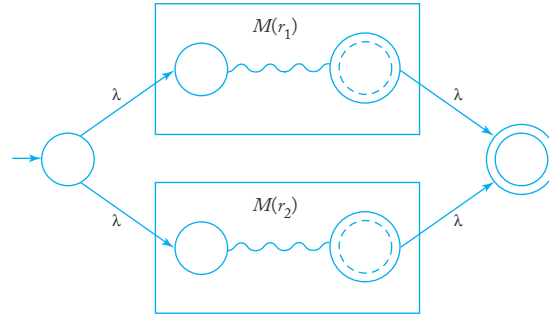


ŞEKİL 3.2 $L(r)$ kabul eden bir nfa'nın şematik temsili.

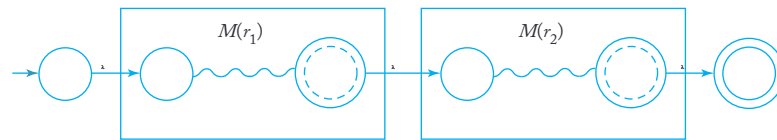
Soldaki köşe başlangıç durumunu, sağdaki köşe ise son durumu temsil eder. Egzersiz 7, Bölüm 2.3'te, her nfa için tek bir son duruma sahip eşdeğer bir nfa olduğunu iddia ediyoruz, bu nedenle yalnızca bir son durum olduğunu varsaymakta bir kaybımız yok. Bu şekilde temsil edilen $M(r_1)$ ve $M(r_2)$ ile, ardından düzenli ifadeler için otomata inşa ediyoruz $r_1 + r_2$, $r_1 r_2$, ve r^* .

1. Yapılar Şekil 3.3'ten 3.5'e kadar gösterilmiştir. Çizimlerde belirtildiği gibi, bileşen makinelerinin başlangıç ve son durumları statülerini kaybeder ve yeni başlangıç ve son durumlarla değiştirilir. Birkaç böyle adımı bir araya getirerek, rastgele karmaşık düzenli ifadeler için otomata inşa edebiliriz.

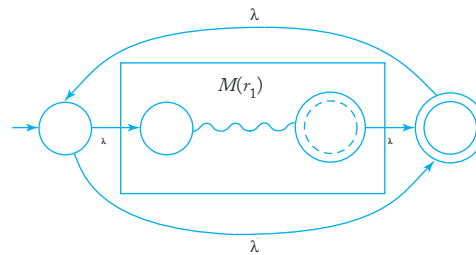
Şekil 3.3'ten 3.5'e kadar olan grafiklerin yorumundan, bu yapının çalıştığı açık olmalıdır. Daha titiz bir şekilde tartışmak için, parçaların durumları ve geçişlerinden birleşik makinenin durumlarını ve geçişlerini oluşturmak için resmi bir yöntem verebiliriz, ardından tümevarım ile kanıtlayabiliriz.



ŞEKİL 3.3 $L(r_1 + r_2)$ için Otomat.



ŞEKİL 3.4 $L(r_1 r_2)$ için Otomat.



ŞEKİL 3.5 $L(r_1^*)$ için Otomat.

Yapının, belirli bir düzenli ifadeyi belirten dili kabul eden bir otomat ürettiği operatör sayısı üzerinedir. Bu noktayı fazla uzatmayacağız, çünkü sonuçların her zaman doğru olduğu oldukça açıktır.

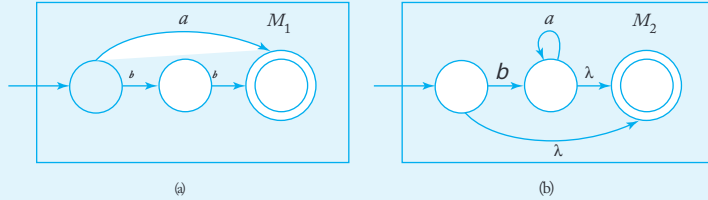
■

ÖRNEK 3.7

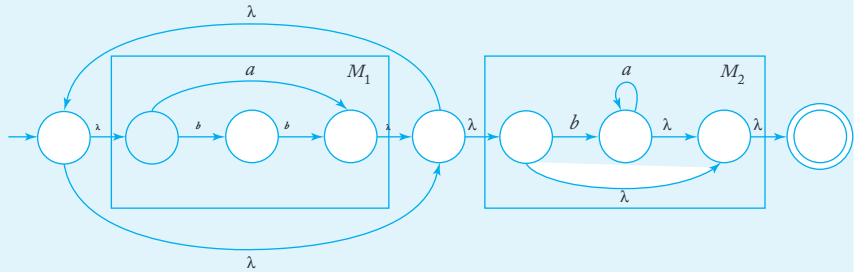
Bir nfa bulun ki $L(r)$ kabul etsin, burada

$$r = (a + bb)^*(ba^* + \lambda).$$

$(a + bb)$ ve $(ba^* + \lambda)$ için otomata, doğrudan ilkelerden inşa edilmiştir, Şekil 3.6'da verilmiştir. Bunları bir araya getirerek Teorem 3.1'deki yapıyı kullanarak, Şekil 3.7'deki çözümü elde ederiz.



ŞEKİL 3.6 (a) $M_1 L(a + bb)$ kabul eder. (b) $M_2 L(ba^* + \lambda)$ kabul eder.



ŞEKİL 3.7 Otomat kabul eder $L((a + bb)^*(ba^* + \lambda))$.

Düzenli Diller için Düzenli İfadeler

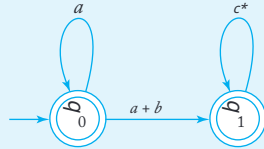
Teorem 3.1'in tersinin geçerli olması ve her düzenli dil için karşılık gelen bir düzenli ifadenin var olması sezgisel olarak mantıklıdır. Çünkü herhangi bir düzenli dilin ilişkili bir nfa'sı ve dolayısıyla bir geçiş grafi vardır, yapmamız gereken tek şey, q0'dan herhangi bir son duruma kadar olan tüm yürüyüşlerin etiketlerini üretebilen bir düzenli ifade bulmaktır. Bu çok zor görünmüyor ama döngülerin varlığıyla karmaşık hale geliyor.

genellikle keyfi olarak, herhangi bir sırayla geçilebilir. Bu, dikkatlice ele alınması gereken bir muhasebe sorunu yaratır. Bunu yapmanın birkaç yolu vardır; bunlardan daha sezgisel olanları, "şu şekilde" olarak adlandırılan bir yan yol gerektirir. genelleştirilmiş geçiş grafikleri (GTG). Bu fikir burada sınırlı bir şekilde kullanıldığı ve daha sonraki tartışmamızda hiçbir rol oynamadığı için, bunu gayri resmi olarak ele alacağız.

Genelleştirilmiş bir geçiş grafiği, kenarlarının düzenli ifadelerle etiketlendiği bir geçiş grafiğidir; aksi takdirde, normal geçiş grafiği ile aynıdır. Başlangıç durumundan bir son duruma kadar olan herhangi bir yürüyüşün etiketi, birkaç düzenli ifadenin birleştirilmesidir ve dolayısıyla kendisi de bir düzenli ifadedir. Bu tür düzenli ifadelerle belirtilen dizeler, genelleştirilmiş geçiş grafiği tarafından kabul edilen dilin bir alt kümesidir ve tam dil, bu tür üretilen tüm alt kümelerin birleşimidir.

ÖRNEK 3.8

Şekil 3.8, genel bir geçiş grafi temsil etmektedir. Bu graf tarafından kabul edilen dil, $L(a^* + a^*(a + b)^*c^*)$ olarak, grafi incelediğimizde açıkça görülmelidir. a ile etiketlenmiş (q_0, q_0) kenarı, herhangi bir sayıda a üretebilen bir döngüdür; yani, $L(a^*)$ 'yi temsil etmektedir. Bu kenarı, grafin kabul ettiği dili değiştirmeden a^* olarak etiketleyebiliriz.



ŞEKİL 3.8

Herhangi bir belirsiz sonlu kabul edicinin grafiği, kenar etiketleri doğru bir şekilde yorumlandığında genel bir geçiş grafi olarak düşünülebilir. Tek bir sembolle etiketlenmiş bir kenar a ,

ifade a ile etiketlenmiş bir kenar olarak yorumlanır, oysa birden fazla sembolle etiketlenmiş bir kenar $a, b, \dots$

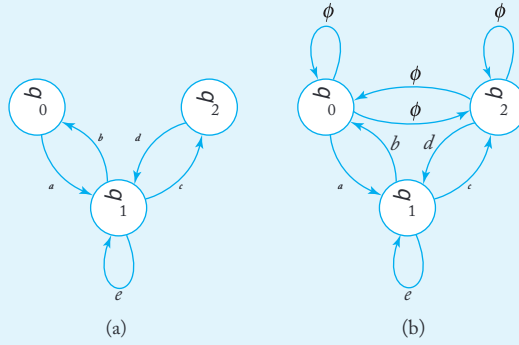
ifade $a + b + \dots$ ile etiketlenmiş bir kenar olarak yorumlanır. Bugözümden, her düzenli dil için onu kabul eden bir genelleştirilmiş geçiş grafiğibulunduğu sonucuna varılır. Tersine, bir genelleştirilmiş geçiş grafiği tarafındankabul edilen her dil düzenlidir. Bir genelleştirilmiş geçiş grafiğindeki her yürüyüşünetiketi bir düzenli ifadeyse, bu Teorem 3.1'in hemen bir sonucugibi görünmektedir.Ancak, argümanda bazı incelikler vardır; bunları burada takip etmeyeceğiz, bununyerine okuyucuyu detaylar için Alıştırma 22, Bölüm 4.3'e yönlendireceğiz.

Genelleştirilmiş geçiş grafiklerinin eşdeğerliliği, kabul edilen dil açısından tanımlanır ve bir sonraki tartışmanın amacı, giderek daha basit GTG'lerin bir dizisini üretmektir.

Bu bağlamda, tam GTG'lerle çalışmanın uygun olacağını bulacağız. Tam bir GTG, tüm kenarların mevcut olduğu bir grafiktir. Eğer bir GTG, bir nfa'dan dönüştürüldükten sonra bazı kenarları eksikse, onları ekleriz ve $\emptyset$ ile etiketleriz. $|V|$ düğümü olan bir tam GTG, tam olarak $|V|^2$ kenara sahiptir.

ÖRNEK 3.9

Şekil 3.9(a) içindeki GTG tam değildir. Şekil 3.9(b), nasıl tamamlandığını göstermektedir.



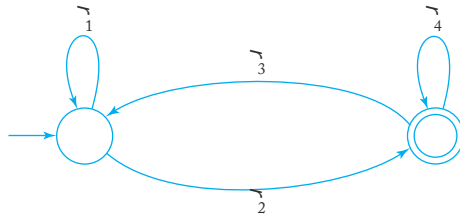
ŞEKİL 3.9

Şimdi, Şekil 3.10'da gösterilen basit iki durumlu tam GTG'ye sahip olduğumuzu varsayalım. Bu GTG'yi zihninizde takip ederek, düzenli ifadenin

$$r = r_1^* r_2 (r_4 + r_3 r_1^* r_2)^* \quad (3.1)$$

tüm olası yolları kapsadığını ve bu nedenle grafik ile ilişkili doğru düzenli ifade olduğunu kendiniz e kanıtlayabilirsiniz.

Bir GTG'nin iki durumdan fazla olduğunda, bir durumu bir seferde k aldirarak eşdeğer bir grafik bulabiliriz. Bunu genel yöntemle geçmeden önce bir örnekle göstereceğiz.



ŞEKİL 3.10

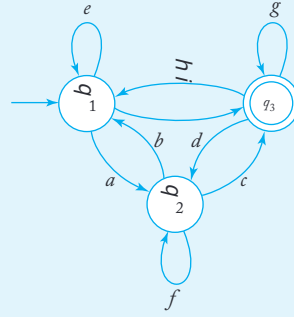
ÖRNEK 3.10

Şekil 3.11'deki tam GTG'yi düşünün. q_2 'yi kaldırmak için önce bazı yeni kenarlar tanıtıyoruz.

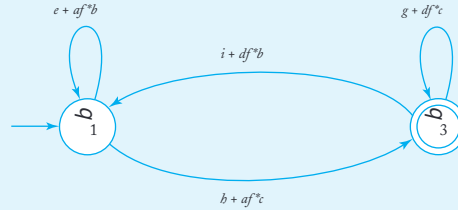
q_1 'den q_1 'e bir kenar oluşturuyoruz ve bunu $e + a f^* b$ olarak etiketliyoruz,
 q_1 'den q_3 'e bir kenar oluşturuyoruz ve bunu $i + a f^* c$ olarak etiketliyoruz,
 q_3 'ten q_1 'e bir kenar oluşturuyoruz ve bunu $i + d f^* b$ olarak etiketliyoruz,
 q_3 'ten q_3 'e bir kenar oluşturuyoruz ve bunu $g + d f^* c$ olarak etiketliyoruz.

Bunu yaptıktan sonra, q_2 'yi ve tüm ilişkili kenarları kaldırıyoruz. Bu, Şekil 3.12'deki GTG'yi verir. İki GTG'nin eşdeğerliğini, $a f^* c$ ve $e^* a b$ gibi düzenli ifadelerin nasıl

oluşturulduğunu görerek keşfedebilirsiniz.



ŞEKİL 3.11



ŞEKİL 3.12

Rastgele GTG'ler için bir durumu birer birer kaldırıyoruz, ta ki sadece iki durum kalana kadar. Sonra son düzenli ifadeyi elde etmek için Denklem (3.1)'i uyguluyoruz. Bu genellikle uzun bir süreçtir, ancak aşağıdaki

prosedürün gösterdiği gibi basittir.

prosedür: nfa-to-rex

1. Bir nfa ile başlayın, durumlar $q_0, q_1, \dots, q_n$ ve başlangıç durumundan farklı tek bir son durum ile.

2. NFA'yı tamamlanmış genel geçiş grafiğine dönüştürün. Let r_{ij} q_i 'den q_j 'ye giden kenarın etiketini temsil etsin.
3. Eğer GTG'nin yalnızca iki durumu varsa, başlangıç durumu q_i ve q_j son durumu ise, ilişkili düzenli ifadesi

$$r = r_{ii}^* r_{ij} (r_{jj} + r_{ji} r_{ii}^* r_{ij})^* . \quad (3.2)$$

4. Eğer GTG'nin üç durumu varsa, başlangıç durumu q_i , son durumu q_j ve üçüncü durumu q_k ise, yeni kenarlar ekleyin, etiketli

$$r_{pq} + r_{pk} r_{kk}^* r_{kq} \quad (3.3)$$

for $p=i, j, q=i, j$. Bu yapıldığında, vertex q_k ve onunla ilişkili kenarlar ı kaldırın.

5. Eğer GTG dört veya daha fazla duruma sahipse, kaldırılacak bir durum q_k seçin. Tüm durum çiftleri için kural 4'ü uygulayın $(q_i, q_j), i / = k, j / = k$. Her adımda basitleştirmeye kurallarını uygulayın

$$r + \emptyset = r,$$

$$r\emptyset = \emptyset,$$

$$\emptyset^* = \lambda,$$

mümkün olduğunca. Bu yapıldığında, durum q_k 'yi kaldırın.

6. Doğru düzenli ifadeyi elde edene kadar Adım 3'ten 5'e kadar tekrarlayın.

ÖRNEK 3.11

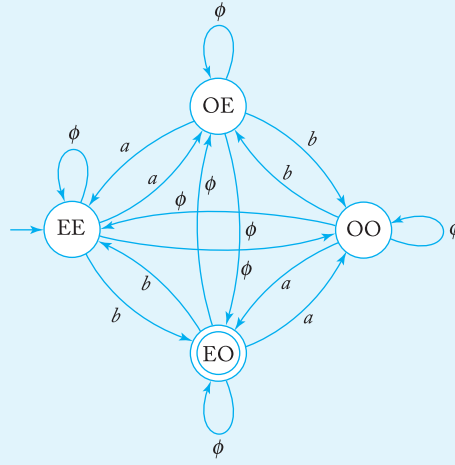
Düzenli bir ifade bul

$$L = \{w \in \{a, b\}^* : n_a(w) \text{ is even and } n_b(w) \text{ is odd}\} .$$

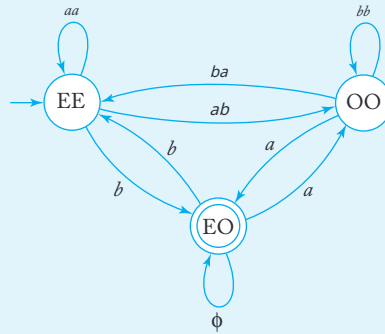
Bu tanımdan doğrudan bir düzenli ifade oluşturmaya çalışmak her türlü zorluğa yol açar. Öte yandan, bir nfa bulmak, köşe etiketleme sini etkili bir şekilde kullandığımız sürece kolaydır. Dörtgenleri, a ve b'nin çift sayısını belirtmek için EE ile, a'nın tek sayısını ve b'nin çift sayısını belirtmek için OE ile etiketliyoruz ve devam ediyoruz. Bunu nla, tam genel geçiş grafiğine dönüştürüldüğünde, çözümü kolayca elde ediyoruz; bu, Şekil 3.13'te gösterilmiştir.

Şimdi, nfa-to-regex prosedürünü kullanarak bir düzenli ifadeye dönüşümü uyguluyoruz. OE durumunu kaldırmak için, Denklem (3.3)'ü uyguluyoruz. EE ile kendisi arasındaki kenar etiketine sahip olacaktır

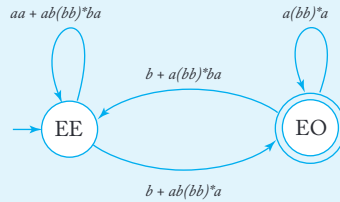
$$r_{EE} = \emptyset + a\emptyset^*a \\ aa.$$



ŞEKİL 3.13



ŞEKİL 3.14



ŞEKİL 3.15

Bu şekilde devam ediyoruz ve Şekil 3.14'te GTG'yi elde ediyoruz. Sonra, OO durumu kaldırılıyor, bu da Şekil 3.15'i veriyor. Son olarak, Denklem (3.2) ile doğru düzenli ifadeyi elde ediyoruz.

Bir nfa'yı düzenli bir ifadeye dönüştürme süreci mekanikama zahmetlidir. Bu, karmaşık ve pratikte pek az kullanışlı olan düzenli ifadelerle yol açar. Bu sürecin sunulmasının ana nedeni, önemli bir sonucun kanıtı için fikir vermesidir.

TEOREM 3.2

Let L düzenli bir dil olsun. O zaman $L = L(r)$ şeklinde bir düzenli ifade vardır.

Kanıt: Eğer L düzenli ise, bunun için bir nfa vardır. Bu nfa'nın tek bir son durumu olduğunu ve bu durumun başlangıç durumundan farklı olduğunu varsayabiliriz. Bu nfa'yı tam bir genel geçiş grafiğine dönüştürüyoruz ve buna *nfa-to-regex* prosedürünü uyguluyoruz. Bu, gerekli düzenli ifadeyi r verir.

Bunun sonucu makul hale getirebilse de, titiz bir kanıt, sürecin her adımının eşdeğer bir GTG ürettiğini göstermemizi gerektirir. Bu, okuyucu ya bıraktığımız teknik bir meseledir. ■

Basit Desenleri Tanımlamak için Düzenli İfadeler

Örnek 1.15 ve Alıştırma 16, Bölüm 2.1'de, sonlu kabul ediciler ile programlama dillerinin bazı daha basit bileşenleri, örneğin tanımlayıcılar veya tam sayılar ve reel sayılar arasındaki bağlantıyı keşfettik. Sonlu otomata ile düzenli ifadeler arasındaki ilişki, bu özellikleri tanımlamak için düzenli ifadeleri de kullanabileceğimiz anlamına gelir. Bu kolayca görülebilir; örneğin, birçok programlama dilinde tam sayı sabitleri kümesi aşağıdaki düzenli ifade ile tanımlanır

$$sdd^*,$$

burada s işareti temsil eder, olası değerleri $\{+, -, \lambda\}$ ve d 0'dan 9'a kadar olan rakamları temsil eder. Tam sayı sabitleri, bazen "desen" olarak adlandırılan, bazı ortak özelliklere sahip nesneler kümesine atıfta bulunan bir terimdir. Desen eşleştirme, verilen bir nesneyi birkaç kategoriden birine atamayı ifade eder. Genellikle, başarılı desen eşleştirmenin anahtarı, desenleri tanımlamak için etkili bir yol bulmaktır. Bu, yalnızca kısaca değinebildiğimiz karmaşık ve kapsamlı bir bilgisayar bilimi alanıdır. Aşağıdaki örnek, şimdiye kadar konuştuğumuz fikirlerin desen eşleştirmede nasıl faydalı bulunduğu dair basitleştirilmiş, ancak yine de öğretici bir gösterimdir.

ÖRNEK 3.12

Bir desen eşleştirme uygulaması metin düzenlemede gerçekleşir. Tüm metin editörleri, belirli bir dizeyi bulmak için dosyaların taranmasına izin verir; çoğu editör, desen aramayı da mümkün kılacak şekilde bunu genişletir. Örneğin, UNIX işletim sistemindeki vi editörü, dosyada ab dizesinin ilk örneğini aramak için bir komut olarak /aba*c/ ifadesini tanır, ardından rastgele sayıda a gelir, ardından bir c gelir. Bu örnekten, desen eşleştirme editörlerinin

normal ifadelerle çalışması gerektiğini görüyoruz.

Böyle bir uygulamadaki zorlu bir görev, dize desenlerini tanıyan etkili bir program yazmaktır. Belirli bir dize için bir dosyada arama yapmak çok basit bir programlama alıştırmasıdır, ancak burada durum daha karmaşıktır. Sınırsız sayıda rastgele karmaşık desenlerle başa çıkamamız gerekiyor; ayrıca, desenler önceden sabitlenmemiştir, ancak çalışma zamanında oluşturulmaktadır. Desen tanıma, girdi kısmının bir parçasıdır, bu nedenle tanıma süreci esnek olmalıdır. Busorunu çözme için genellikle otomata teorisinden fikirler kullanılır.

Eğer desen bir normal ifade ile belirtilmişse, desen-tanım programı bu tanımlanabilir ve Teorem 3.1'deki yapıyı kullanarak eşdeğer bir nfa'ya dönüştürebilir. Daha sonra bu, Teorem 2.2 kullanılarak bir dfa'ya indirgenebilir. Bu dfa, bir geçiş tablosu biçiminde, etkili bir şekilde desen eşleştirme algoritmasıdır. Tüm programcının yapması gereken, tablonun kullanılmasına genel bir çerçeve sağlayan bir sürücü sağlamaktır. Bu şekilde, çalışma zamanında tanımlanan çok sayıda deseni otomatik olarak işleyebiliriz.

Programın verimliliği de dikkate alınmalıdır. Düzenli ifadelerden sonlu otomata inşası, Teorem 2.1 ve 3.1 kullanılarak, genellikle birçok duruma sahip otomata üretir. Eğer bellek alanı bir sorun ise, Bölüm 2.4'te açıklanan durum azaltma yöntemi faydalıdır.

EGZERSİZLER

1. Teorem 3.1'deki yapıyı kullanarak, dili kabul eden bir nfa bulun $L(a^*a+ab)$.
2. Teorem 3.1'deki yapıyı kullanarak, dili kabul eden bir nfa bulun $L((aab)^*ab)$.
3. Teorem 3.1'deki yapıyı kullanarak, dili kabul eden bir nfa bulun $L(ab^*aa+bba^*ab)$.

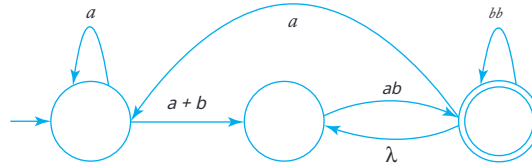
4. Egzersiz 3'teki dilin tamamlayıcısını kabul eden bir nfa bulun.
5. Bir nfa verin ki, dil $L((a+b)^*b(a+bb)^*)$ kabul etsin.
6. Aşağıdaki dfa'ları bulun:

- (a) $L(aa^*+aba^*b^*)$.
- (b) $L(ab(a+ab)^*(a+aa))$.
- (c) $L((abab)^* + (aaa^*+b)^*)$.
- (d) $L(((aa^*)^*b)^*)$.
- (e) $L((aa^*)^*+abb)$.

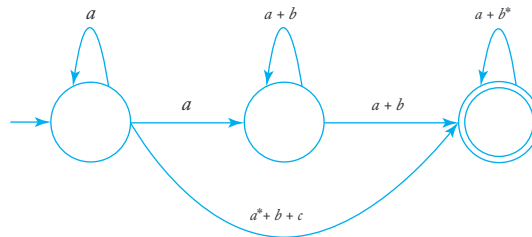
7. Aşağıdaki dfa'ları bulun:

- (a) $L=L(ab^*a^*) \cup L((ab)^*ba)$.
- (b) $L=L(ab^*a^*) \cap L((b)^*ab)$.

8. Find the minimal dfa that accepts $L(abb)^* \cup L(a^*bb^*)$.
9. Find the minimal dfa that accepts $L(a^*bb) \cup L(ab^*ba)$.
10. Aşağıdaki genel geçiş grafiğini dikkate alın.

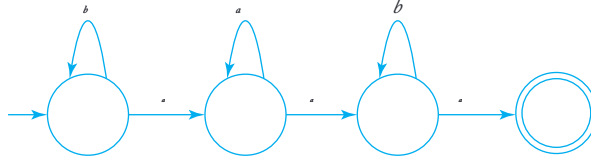


- (a) Sadece iki durumu olan eşdeğer bir genel geçiş grafi bulun.
- (b) Bu graf tarafından kabul edilen dil nedir?
11. Aşağıdaki genel geçiş grafi tarafından hangi dil kabul edilmektedir?

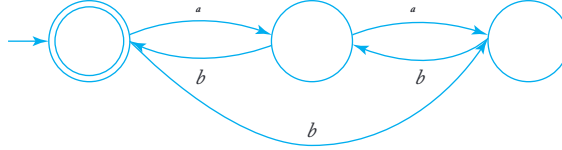


12. Şu otomata tarafından kabul edilen diller için düzenli ifadeleri bulun.

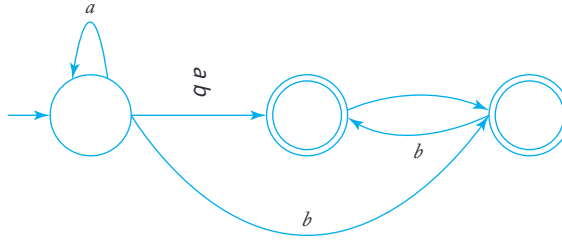
(a)



(b)



(c)



13. Örnek 3.11'i yeniden çalışın, bu sefer önce OO durumunu ortadan kaldırın.

14. Şekil 3.14'teki tüm etiketlerin nasıl elde edildiğini gösterin.

15. Aşağıdaki diller için bir düzenli ifade bulun $\{a, b\}$.

(a) $L = \{w : n_a(w) \text{ ve } n_b(w) \text{ her ikisi de tek}\}$.

(b) $L = \{w : (n_a(w) - n_b(w)) \bmod 3 = 2\}$.

(c) $L = \{w : (n_a(w) - n_b(w)) \bmod 3 = 0\}$.

(d) $L = \{w : 2n_a(w) + 3n_b(w) \text{ çift}\}$.

16. Şekil 3.11 ve 3.12 tarafından önerilen yapının eşdeğer genelleştirilmiş geçiş grafiklerini ürettiğini kanıtlayın.

17. C gerçekte sayılar kümesi için bir düzenli ifade yazın.

18. Theorem 3.1'deki yapıyı kullanarak $L(a\odot)$ ve $L(\odot^*)$ için nfa'lar bulun. Sonuç, bu dillerin tanımıyla tutarlı mı?

3.3 DÜZENLİ GRAMERLER

Düzenli dilleri tanımlamanın üçüncü bir yolu, belirli gramerler aracılığıyla olmaktadır. Gramerler genellikle dilleri belirtmenin alternatif bir yoludur. Her zaman

bir dil ailesini bir otomaton aracılığıyla veya başka bir şekilde tanımladığımızda, dil ailesiyle ilişkilendirebileceğimiz türde bir gramer hakkında bilgi edinmek le ilgileniyoruz. Öncelikle, düzenli dilleri üreten gramerlere bakıyoruz.

Sağ ve Sol-Doğrusal Dilbilgileri

TANIM 3.3

Bir dilbilgisi $G = (V, T, S, P)$ sağ-doğrusal olarak adlandırılır eğer tüm üretimler şu biçimdedir

$$A \rightarrow xB,$$

$$A \rightarrow x,$$

burada $A, B \in V$, ve $x \in T^*$. Bir dilbilgisi sol-doğrusal olarak adlandırılır eğer tüm üretimler şu biçimdedir

$$A \rightarrow Bx,$$

veya

$$A \rightarrow x.$$

Bir düzenli dilbilgisi ya sağ-doğrusal ya da sol-doğrusal olan bir dilbilgisidir.

Düzenli bir dilbilgisinde, en fazla bir değişken herhangi bir üretim inşa tarafında görünür. Ayrıca, o değişken her zaman üretimin sağ tarafının en sağdaki veya en soldaki sembolü olmalıdır.

ÖRNEK 3.13

Gramer $G_1 = (\{S\}, \{a, b\}, S, P_1)$, P_1 olarak verilmiştir

$$S \rightarrow abS|a$$

doğru-lineerdir. Gramer $G_2 = (\{S, S_1, S_2\}, \{a, b\}, S, P_2)$, üretimlerle

$$S \rightarrow S ab,$$

$$S_1 \rightarrow S_1 ab|S_2,$$

$$S \rightarrow a,$$

sol-lineerdir. Hem G_1 hem de G_2 düzenli gramerlerdir. Seri

$$S \Rightarrow abS \Rightarrow ababS \Rightarrow ababa$$

G_1 ile bir türetimdir. Bu tek örnekten, $L(G_1)$ düzenli ifade $r = (ab)^*a$ ile belirtilen dil olduğunu tahmin etmek kolaydır. Benzer şekilde, $L(G_2)$ düzenli dil olduğunu görebiliriz $L(aab(ab)^*)$.

ÖRNEK 3.14

Grammer $G = (\{S, A, B\}, \{a, b\}, S, P)$ üretimleri ile

$$\begin{aligned} S &\rightarrow A, \\ A &\rightarrow aB|\lambda, \\ B &\rightarrow Ab, \end{aligned}$$

düzenli değildir. Her üretim ya sağdan-lineer ya dasoldan-lineer formda olmasına rağmen, gramer kendisi ne sağdan-lineer ne de soldan-lineer olup, bu nedenle düzenli değildir. Gramer, bir lineer gramer örneğidir. Lineer bir gramer, en fazla bir değişkenin herhangi bir üretim in sağ tarafında yer alabileceği, bu değişkenin konumu üzerinde herhangi bir kısıtlama olmaksızın bir gramerdir. Açıkça, düzenli bir gramer her zaman lineerdir, ancak tüm lineer gramerler düzenli değildir.

Bir sonraki hedefimiz, düzenli gramerlerin düzenli dillerle ilişkili olduğunu ve her düzenli dil için bir düzenli gramer olduğunu göstermektir. Böylece, düzenli gramerler düzenli diller hakkında konuşmanın başka bir yoludur.

Sağ-Doğrusal Dilbilgileri Düzenli Diller Üretir

Öncelikle, sağ-doğrusal bir dilbilgisi tarafından üretilen bir dilin her zaman düzenli olduğunu gösteriyoruz. Bunu yapmak için, sağ-doğrusal bir dilbilgisinin türevlerini taklit eden bir nfa inşa ediyoruz. Sağ-doğrusal bir dilbilgisinin cümle biçimlerinin, tam olarak bir değişkenin bulunduğu ve bu değişkenin en sağdaki sembol olarak yer aldığı özel bir biçime sahip olduğunu unutmayın. Şimdi, bir türevde bir adımda

$$ab \cdots cD \quad ab \cdots cdE,$$

üretim $D \rightarrow dE$ kullanarak ulaştığımızı varsayalım. İlgili nfa, bir sembol d ile karşılaşıldığında durum D' den durum E' ye geçerek bu adımı taklit edebilir. Bu şemada, otomatonun durumu cümle biçimindeki değişkeni karşılamakla birlikte, işlenmiş olan dize kısmi cümle biçiminin terminal ön eki ile aynıdır. Bu basit fikir, aşağıdaki teoremin temelini oluşturur.

TEOREM 3.3

$G = (V, T, S, P)$ sağ-doğrusal bir dilbilgisi olsun. O zaman $L(G)$ düzenli bir dildir.

Kanıt: $V = \{V_0, V_1, \dots\}$ olduğunu, $S = V_0$ olduğunu ve $V_0 \rightarrow v_1 V_i, V_i \rightarrow v_2 V_j, \dots$ veya $V_n \rightarrow v_l, \dots$ biçiminde üretimlere sahip olduğumuzu varsayıyoruz. Eğer $w, L(G)$ içi nde bir dize ise, o zaman üretimlerin biçimi nedeniyle

$$\begin{aligned} V_0 &\Rightarrow v_1 V_i \\ &\Rightarrow v_1 v_2 V_j \\ &\stackrel{*}{\Rightarrow} v_1 v_2 \cdots v_k V_n \\ &\Rightarrow v_1 v_2 \cdots v_k v_l = w. \end{aligned} \quad (3.4)$$

İnşa edilecek otomaton, her bir v 'yi sırayla tüketerek türevi yeniden üretecektir. Otomatonun başlangıç durumu V_0 olarak etiketlenecek ve her v değişken V_i için bir son olmayan durum V_i olarak etiketlenecektir. Her üretim için

$$V_i \rightarrow a_1 a_2 \cdots a_m V_j,$$

otomaton, V_i ve V_j 'yi bağlamak için geçişlere sahip olacaktır, yani δ , böyle tanımlanacaktır ki

$$\delta^*(V_i, a_1 a_2 \cdots a_m) = V_j.$$

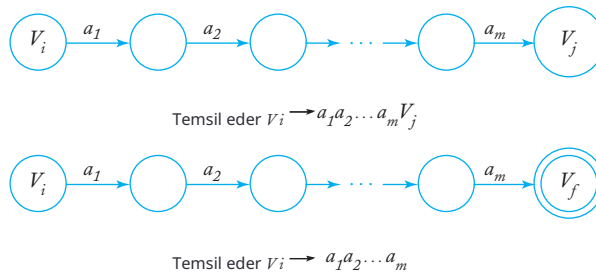
Her üretim için

$$V_i \rightarrow a_1 a_2 \cdots a_m,$$

otomatonun karşılık gelen geçişi

$$\delta^*(V_i, a_1 a_2 \cdots a_m) = V_f,$$

burada V_f bir son durumdur. Bunu yapmak için gereken ara durumlar önemli değildir ve rastgele etiketler verilebilir. Genel şema Şekil 3.16' da gösterilmiştir. Tam otomaton, bu tür bireysel parçalardan bir araya getirilmiştir.



ŞEKİL 3.16

Şimdi varsayalım ki $w \in L(G)$ böylece (3.4) sağlanır. nfa'da inşa gereği, bir yol vardır V_0 ile V_i arasında v_1 ile etiketlenmiş, bir yol V_i ile V_j v_2 ile etiketlenmiş, ve bu şekilde devam eder, böylece açıkça

$$V_f \in \delta^*(V_0, w)$$

ve w M tarafından kabul edilir.

Tam tersine, w 'nin M tarafından kabul edildiğini varsayalım. M 'nin inşa edilme şekli nedeniyle, w 'yi kabul etmek için otomatonun $V_0, V_i, \dots, V_f$ durumları arasında bir dizi durumdan geçmesi gerekir ve $v_1, v_2, \dots$ etiketli yolları kullanması gerekir. Bu nedenle, w 'nin şu formda olması gerekir

$$w = v_1 v_2 \cdots v_k v_l$$

ve türetim

$$V_0 \Rightarrow V_i \quad v_1 v_2 V_j \xRightarrow{*} v_1 v_2 \cdots v_k V_k \Rightarrow v_1 v_2 \cdots v_k v_l$$

mümkündür. Dolayısıyla $w, L(G)$ içindedir ve teorem kanıtlanmıştır. ■

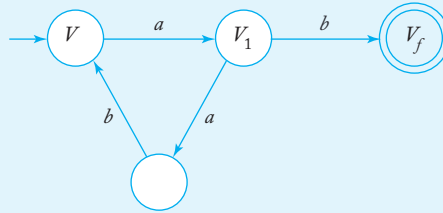
ÖRNEK 3.15

Gramer tarafından üretilen dili kabul eden sonlu bir otomaton inşa edin

$$\begin{aligned} V_0 &\rightarrow aV_1 \\ &\rightarrow abV \mid b, \end{aligned}$$

burada V_0 başlangıç değişkenidir. Geçiş grafiğine V_0, V_1 ve V_f köşeleri ile başlıyoruz. İlk üretim kuralı, V_0 ile V_1 arasında a etiketli bir kenar oluşturur. İkinci kural için, V_1 ile

V_0 arasında ab etiketli bir yol olması için ek bir köşe tanıtmalıyız. Son olarak, V_1 ile V_f arasında b etiketli bir kenar eklememiz gerekiyor ve bu, Şekil 3.17'de gösterilen otomatonu verir. Gramer tarafından üretilen ve otomaton tarafından kabul edilen dil, düzenli dildir $L((aab)^* ab)$.



ŞEKİL 3.17

Sağ-Doğru Dilbilgileri için Düzenli Diller

Her düzenli dilin bazı sağ-doğru dilbilgileri tarafından üretilebileceğini göstermek için, dilin dfa'sından başlıyoruz ve Teorem 3.3'te gösterilen yapıyı tersine çeviriyoruz. Şimdi dfa'nın durumları dilbilgisinin değişkenleri haline gelir ve geçişlere neden olan semboller üretimlerdeki terminallerle haline gelir.

TEOREM 3.4

If L, Σ alfabeti üzerinde düzenli bir dil ise, o zaman $L = L(G)$ olacak şekilde bir sağ-lineer gramer $G = (V, \Sigma, S, P)$ vardır.

Kanıt: $M = (Q, \Sigma, \delta, q_0, F)$ bir dfa olsun ki L 'yi kabul eder. Varsayıyoruz ki $Q = \{q_0, q_1, \dots, q_n\}$ ve $\Sigma = \{a_1, a_2, \dots, a_m\}$. Sağ-lineer gramer $G = (V, \Sigma, S, P)$ ile

$$V = \{q_i, q_j\}, q$$

ve $S = q_0$. Her geçiş için

$$\delta(q_i, a_j) = q_k$$

üzerine M, P üretimi koyuyoruz

$$q_i \rightarrow a_j q_k. \quad (3.5)$$

Ek olarak, eğer $q_k \in F$ içindeyse, P 'ye üretimi ekliyoruz.

$$q_k \rightarrow \lambda. \quad (3.6)$$

Bu şekilde tanımlanan G 'nin her dizeyi üretebileceğini ilk olarak gösteriyoruz L . Dikkate al $w \in L$ biçiminde

$$w = a_i a_j \cdots a_k a_l.$$

Bu dizgiyi kabul etmek için M , hareketler yapmalıdır

$$\delta(q_0, a_i) = q_p,$$

$$\delta(q_p, a_j) = q_r,$$

$$\vdots$$

$$\delta(q_r, a_k) = q_t,$$

$$\delta(q_t, a_l) = q_f \in F.$$

Yapı itibarıyla, dilbilgisi bunların her biri için bir üretime sahip olacaktır. Şöyle. Bu nedenle, türetmeyi yapabiliriz

$$\begin{aligned} q_0 &\Rightarrow a_i q_p \Rightarrow a_i a_j q_r \xRightarrow{*} a_i a_j \cdots a_k q_t \\ &\Rightarrow a_i a_j \cdots a_k a_l \quad f \Rightarrow a_i a_j \cdots a_k a_l, \end{aligned} \quad (3.7)$$

gramer G ile $w \in L(G)$.

Tam tersine, eğer $w \in L(G)$ ise, o zaman türetilmesi (3.7) biçiminde olmalıdır. Ancak bu,

$$\delta^*(q_0, a_i a_j \cdots a_k a_l) = q_f,$$

kanıtın tamamlanmasını gerektirir. ■

Bir gramer inşa etme amacıyla, M 'nin bir dfa olma kısıtlamasının Teorem 3.4'ün kanıtı için gerekli olmadığını not etmek faydalıdır. Ufak bir değişiklik ile, aynı yapı, eğer M bir nfa ise kullanılabilir.

ÖRNEK 3.16

$L(aab^*a)$ için sağ-lineer bir dil bilgisi oluşturun. Bir nfa için geçiş fonksiyonu, ilgili dil bilgisi üretimleri ile birlikte Şekil 3.18'de verilmiştir. Sonuç, Teorem 3.4'teki yapıyı takip ederek elde edilmiştir. Dizeaaba, oluşturulan dil bilgisi ile türetilir.

$$q_0 \Rightarrow aq_1 \Rightarrow aaq_2 \Rightarrow aabq_2 \Rightarrow aabaq_f \Rightarrow aaba.$$

$\delta(q_0, a) = \{q_1\}$	$q_0 \rightarrow aq_1$
$\delta(q_1, a) = \{q_2\}$	$q_1 \rightarrow aq_2$
$\delta(q_2, b) = \{q_2\}$	$q_2 \rightarrow bq_2$
$\delta(q_2, a) = \{q_f\}$	$q_2 \rightarrow aq_f$
$q_f \in F$	$q_f \rightarrow \lambda$

ŞEKİL 3.18

Regüler Dillerin ve Regüler Gramerlerin Eşdeğerliliği

Önceki iki teorem, regüler diller ile sağ-lineer gramerler arasındaki bağlantıyı kurmaktadır. Regüler diller ile sol-lineer gramerler arasında da benzer bir bağlantı kurulabilir, böylece regüler gramerler ile regüler dillerin tam eşdeğerliliği gösterilmiş olur.

TEOREM 3.5

Bir dil L regülerdir eğer ve ancak bir sol-lineer gramer G vardır ki $L = L(G)$.

Kanıt: Sadece ana fikri özetliyoruz. Herhangi bir sol-lineer gramer verildiğinde, üretimlerin formu

$$A \rightarrow Bv,$$

veya

$$A \rightarrow v,$$

ondan bir sağ-lineer dilbilgisi $\hat{G}$ inşa ediyoruz her böyle üretimi G ile değiştirerek

$$A \rightarrow v^R B,$$

veya

$$A \rightarrow v^R,$$

sırasıyla. Birkaç örnek, hızlı bir şekilde $L(G) =$

$\left(L(\hat{G}) \right)^R$. Sonra, herhangi bir düzenli dilin tersinin de düzenli olduğunu söyleyen 12. Egzersiz, Bölüm 2.3'ü kullanıyoruz.

$\hat{G}$ sağ-lineer olduğundan, $L(\hat{G})$

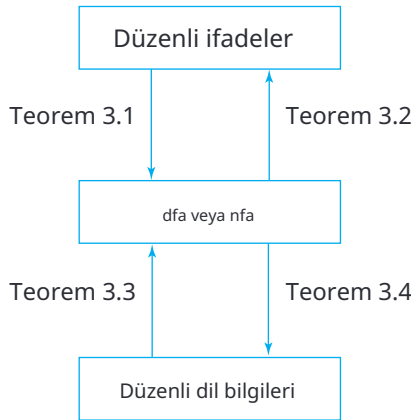
düzenlidir. Ama o zaman $L\left(\left(\hat{G}\right)^R\right) = L(G)$. ■

Teorem 3.4 ve 3.5'i bir araya getirerek, düzenli diller ile düzenli dilbilgileri arasındaki eşdeğerliliğe ulaşıyoruz.

TEOREM 3.6

Bir dil L düzenlidir eğer ve yalnızca bir düzenli dilbilgisi G varsa böylece $L = L(G)$.

Artık düzenli dilleri tanımlamanın birkaç yolu var: dfa'lar, nfa'lar, düzenli ifadeler ve düzenli dilbilgileri. Bazı durumlarda bunlardan biri ya da diğeri en uygun olabilir, ancak hepsi eşit derecede güçlüdür.



ŞEKİL 3.19

Her biri düzenli bir dilin tam ve belirsiz olmayan tanımını verir. Bu bölümdeki dört teorem, Şekil 3.19'da gösterildiği gibi, bu kavramlar arasındaki bağlantıyı kurar.

EGZERSİZLER

1. Gramer tarafından üretilen dili kabul eden bir dfa oluşturun.

$$\begin{aligned} S &\rightarrow abA, \\ A &\rightarrow baB, \\ &\mid bb. \end{aligned}$$

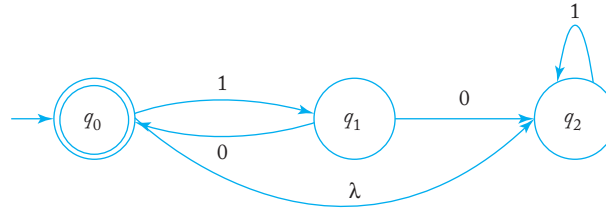
2. Gramer tarafından üretilen dili kabul eden bir dfa oluşturun.

$$\begin{aligned} S &\rightarrow \mid A, \\ A &\rightarrow baB, \\ B &\rightarrow aA \mid bb. \end{aligned}$$

3. Dili üreten düzenli bir gramer bulun $L(aa^*(ab+a)^*)$.
4. Egzersiz 1'deki dil için soldan-lineer bir gramer oluşturun.
5. Dil için sağdan ve soldan-lineer gramerler oluşturun.

$$L = \{ \text{ }^n b \mid n \geq 3, m \geq 2 \}$$

6. Dil için sağdan-lineer bir gramer oluşturun $L((aaab^*ab)^*)$.
7. $\Sigma = \{a, b\}$ üzerinde, en fazla ikia içeren tüm dizeleri oluşturan düzenli bir gramer bulun.
8. Teorem 3.5'te, kanıtlayın ki $L(\widehat{G}) = (L(G))^R$.
9. Bir nfa'dan doğrudan elde edilebilecek bir sol-lineer gramer için bir yapı önerin.
10. Yukarıdaki alıştırmalar tarafından önerilen yapıyı kullanarak aşağıdaki nfa için bir sol-lineer gramer oluşturun.



11. Dil için bir sol-lineer gramer bulun $L((aaab^*ba)^*)$.
12. Dil için bir düzenli gramer bulun $L = \{a^n b^m : n+m \text{ tek}\}$.

13. Düzenli bir dil üreten bir düzenli dil bilgisi bulun

$$L = \{w \in \{a, b\}^* : n_a(w) + 3n_b(w) \text{ is odd}\}.$$

14. Aşağıdaki diller için düzenli dil bilgileri bulun $\{a, b\}$:

- (a) $L = \{w : n_a(w) \text{ çift}, n_b(w) \geq 4\}.$
- (b) $L = \{w : n_a(w) \text{ ve } n_b(w) \text{ her ikisi de çift}\}.$
- (c) $L = \{w : (n_a(w) - n_b(w)) \bmod 3 = 1\}.$
- (d) $L = \{w : (n_a(w) - n_b(w)) \bmod 3 \neq 1\}.$
- (e) $L = \{w : (n_a(w) - n_b(w)) \bmod 3 \neq 0\}.$
- (f) $L = \{w : |n_a(w) - n_b(w)| \text{ tek}\}.$

15. Herhangi bir λ içermeyen düzenli dil için, üretimlerinin aşağıdaki biçimlerle sınırlı olduğu bir sağ-lineer dil bilgisi olduğunu gösterin.

$$A \rightarrow aB,$$

veya

$$A \rightarrow a,$$

$A, B \in V$ ve $a \in T$.

16. $L(G) = \emptyset$ olan herhangi bir düzenli dil bilgisi G 'nin en az bir üretime sahip olması gerektiğini gösterin.

$$A \rightarrow x$$

$A \in V$ ve $x \in T^*$.

17. C 'deki tüm reel sayıları üreten bir düzenli dil bilgisi bulun.

18. $G_1 = (V_1, \Sigma, S_1, P_1)$ sağ-lineer ve $G_2 = (V_2, \Sigma, S_2, P_2)$ sol-lineer dil bilgisi olsun ve V_1 ile V_2 'nin ayrık olduğunu varsayalım. Aşağıdaki lineer dil bilgisini düşünün: $G = (\{S\} \cup V_1 \cup V_2, \Sigma, S, P)$, burada $S, V_1 \cup V_2$ 'de yoktur ve $P = \{S \rightarrow S_1 \mid S_2\} \cup P_1 \cup P_2$. $L(G)$ 'nin düzenli olduğunu gösterin.

BÖLÜM

4

REGÜLER DİLLERİN ÖZELLİKLERİ

BÖLÜM ÖZETİ

Artık tüm düzenli dillerin hangi özelliklere sahip olduğunu, düzenli dillerin birleştirildiğinde (örneğin, iki düzenli dilin birleşimini veya kesişimini oluşturduğumuzda) ne olduğunu ve bir dilin düzenli olup olmadığını nasıl anlayabileceğimizi inceliyoruz. Sözde pompalama lemması, hala basit ama düzenli olmayan dillerin var olduğunu göstermemizi sağlar. Bu, daha karmaşık dil sınıflarını inceleme ihtiyacını ortaya koyacaktır.

Düzenli dilleri tanımladık, bunların temsil edilebileceği bazı yolları inceledik ve faydalılıklarına dair birkaç örnek gördük. Şimdi düzenli dillerin ne kadar genel olduğu sorusunu gündeme getiriyoruz. Her resmi dilin düzenli olabileceği mümkün mü? Belki de herhangi bir küme, çok karmaşık olsa da, bir sonlu otomaton tarafından kabul edilebilir. Kısa süre içinde göreceğimiz gibi, bu varsayımın cevabı kesinlikle hayır. Ancak bunun neden böyle olduğunu anlamak için düzenli dillerin doğasına daha derinlemesine bakmalı ve tüm ailenin hangi özelliklere sahip olduğunu görmeliyiz.

Gündeme getirdiğimiz ilk soru, düzenli diller üzerinde işlemler gerçekleştirildiğinde ne olduğudur. İncelediğimiz işlemler, birleştirme gibi basit küme işlemleri ile her bir dil dizisinin değiştirildiği işlemleri içerir; örneğin, Bölüm 2.1, Alıştırma 24'te olduğu gibi. Ortaya çıkan dil hala düzenli mi? Buna bir kapanış sorusu diyoruz.

Kapatma özellikleri, çoğunlukla teorik ilgi alanı olmasına rağmen, karşılaşacağımız çeşitli dil aileleri arasında ayrım yapmamıza yardımcı olur.

Dil aileleri hakkında ikinci bir soru seti, belirli özellikler üzerinde karar verme yeteneğimizle ilgilidir. Örneğin, bir dilin sonlu olup olmadığını anlayabilir miyiz? Gördüğümüz gibi, bu tür sorular düzenli diller için kolayca yanıtlanabilir, ancak diğer dil aileleri için o kadar kolay değildir.

Son olarak, önemli bir soruyu ele alıyoruz: Verilen bir dilin düzenli olup olmadığını nasıl anlayabiliriz? Eğer dil gerçekten düzenliyse, her zaman bunun için bir dfa, düzenli ifade veya düzenli gramer göstererek bunu kanıtlayabiliriz. Ancak değilse, başka bir saldırı hattına ihtiyacımız var. Bir dilin düzenli olmadığını göstermek için, düzenli dillerin genel özelliklerini incelemek bir yoldur; yani, tüm düzenli dillerin paylaştığı özellikler. Eğer böyle bir özelliği biliyorsak ve aday dilin bunu taşımadığını gösterebiliyorsak, o zaman dilin düzenli olmadığını anlayabiliriz.

Bu bölümde, düzenli dillerin çeşitli özelliklerine bakıyoruz. Bu özellikler, düzenli dillerin neler yapabileceği ve neler yapamayacağı hakkında bize çok şey anlatıyor. Daha sonra, diğer dil aileleri için aynı sorulara baktığımızda, bu özelliklerdeki benzerlikler ve farklılıklar, çeşitli dil ailelerini karşılaştırmamıza olanak tanıyacak.

4.1 DÜZENLİ DİLLERİN KAPANMA ÖZELLİKLERİ

Aşağıdaki soruyu düşünün: İki düzenli dil L_1 ve L_2 verildiğinde, birleşimleri de düzenli midir? Belirli durumlarda, cevap açık olabilir, buradaysa genel olarak problemi ele almak istiyoruz. Tüm düzenli diller için bu doğru mu? L_1 ve L_2 ? Cevap evet, bunu ifade ettiğimiz bir gerçek, düzenli diller ailesinin birleşim altında kapalı olduğunu söyleyerek. Diller üzerindeki diğer tür işlemler hakkında benzer sorular sorabiliriz; bu, dillerin genel olarak kapanış özelliklerinin incelenmesine yol açar.

Farklı işlemler altında çeşitli dil ailelerinin kapanış özellikleri önemli teorik bir ilgi alanıdır. İlk bakışta, bu özelliklerin pratik anlamı ne olabilir, net olmayabilir. Elbette, bazıları çok az pratik anlam taşıırken, bir çok sonuç faydalıdır. Dil ailelerinin genel doğası hakkında bize fikir vererek, kapanış özellikleri diğer, daha pratik soruları yanıtlamamıza yardımcı olur. Bu bölümde daha sonra bunun örneklerini göreceğiz (Teorem 4.7 ve Örnek 4.13).

Basit Küme İşlemleri Altında Kapanış

Ortak küme işlemleri, örneğin birleşim ve kesişim altında düzenli dillerin kapanışını inceleyerek başlıyoruz.

TEOREM 4.1

Eğer L_1 ve L_2 düzenli diller ise, o zaman $L_1 \cup L_2, L_1 \cap L_2, L_1 L_2, L_1^*,$ ve L_1^* de düzenli dillerdir. Düzenli diller ailesinin birleşim, kesişim, birleştirme, tamamlayıcı ve yıldız-kapanış altında kapalı olduğunu söyleriz.

Kanıt: Eğer L_1 ve L_2 düzenli ise, o zaman $L_1 = L(r_1)$ ve $L_2 = L(r_2)$ düzenli ifa deler r_1 ve r_2 vardır. Tanıma göre, $r_1 + r_2, r_1 r_2,$ ve r_1^* düzenli ifadeler olup sırasıyla $L_1 \cup L_2, L_1 L_2,$ ve L_1^* dillerini belirtir. Böylece, birleşim, birleştirme ve yıldız-kapanış altında kapanış hemen görülmektedir.

Tamamlayıcı altında kapanışı göstermek için $M = (Q, \Sigma, \delta, q_0, F)$ olan bir dfa alalım ki bu L_1 'i kabul eder. O zaman dfa

$$\widehat{M} = (Q, \Sigma, \delta, q_0, F)$$

L_1 'i kabul eder. Bu oldukça basittir; zaten 2.1. Bölümdeki 10. Egzersiz' de sonucu önermiştik. Bir dfa tanımında, δ^* 'nin toplam bir fonksiyon olduğunu varsaydık, böylece $\delta^*(q_0, w)$ tümü için tanımlıdır.

$w \in \Sigma^*$. Sonuç olarak ya $\delta^*(q_0, w) \in F$ son durumdur, bu durumda $w \in L_1$, ya da $\delta^*(q_0, w) \notin F$ ve $w \notin L_1$.

Kesişim altında kapalı olmanın gösterilmesi biraz daha fazla çalışma gerektirir. Let $L_1 = L(M_1)$ ve $L_2 = L(M_2)$, burada $M_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1)$ ve $M_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2)$ dfa'lar. Biz M_1 ve M_2 'den birleştirilmiş bir otomat $\widehat{M} = (Q, \Sigma, \delta, q_0, F)$ inşa ediyoruz, durum kümesi $Q = Q_1 \times Q_2$ çiftlerinden oluşur (q_i, p_j) , ve geçiş fonksiyonu δ öyle tanımlanır ki $\widehat{M}$ durumundadır (q_i, p_j) her zaman M_1 durumundayken q_i ve M_2 durumundayken p_j . Bu, alarak gerçekleştirilir

$$\delta((q_i, p_j), a) = (q_k, p_l),$$

her zaman

$$\delta(q_i, a) = q_k$$

ve

$$\delta(p_j, a) = p_l$$

$\widehat{F}$, tüm (q_i, p_j) çiftlerinin kümesi olarak tanımlanır, böylece $q_i \in F_1$ ve $p_j \in F_2$. O zaman $w \in L_1 \cap L_2$ olduğu gösterilmesi basit bir meseledir, eğer ve yalnızca $\widehat{M}$ tarafından kabul ediliyorsa. Sonuç olarak, $L_1 \cap L_2$ düzenlidir. ■

Kesişim altında kapalı olmanın kanıtı, yapıcı bir kanıtın iyi bir örneğidir. Sadece istenen sonucu kurmakla kalmaz, aynı zamanda kesişim için sonlu bir kabul edici nasıl inşa edileceğini de açıkça gösterir.

iki düzenli dil. Yapılandırıcı kanıtlar bu kitap boyunca yer almaktadır; bunlar, son uçlar hakkında bize içgörü sağladıkları ve genellikle pratik algoritmalar için başlangıç noktası oldukları için önemlidir. Burada, birçok durumda olduğu gibi, kısa ama yapılandırıcı olmayan (ya da en azından o kadar açıkça yapılandırıcı olmayan) argümanlar vardır. Kesişim altında kapama için, DeMorgan'ın kanunuyla başlıyoruz, Denklem (1.3), her iki tarafın tamamlayıcısını alarak. Sonra

$$L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$$

herhangi bir diller L_1 ve L_2 . Şimdi, eğer L_1 ve L_2 düzenliyse, o zaman tamamlayıcı altında kapama ile, L_1 ve L_2 de düzenlidir. Birlikte kapama kullanarak, sonra $L_1 \cup L_2$ düzenlidir. Bir kez daha tamamlayıcı altında kapama kullanarak, görürüz ki

$$\overline{\overline{L_1} \cup \overline{L_2}} = L_1 \cap L_2$$

düzenlidir.

Aşağıdaki örnek aynı fikrin bir varyasyonudur.

ÖRNEK 4.1

Düzenli diller ailesinin fark altında kapalı olduğunu gösterin.

Diğer bir deyişle, eğer L_1 ve L_2 düzenliyse, o zaman

$L_1 - L_2$ mutlaka düzenli de olmalıdır.

Gerekli küme kimliği, bir küme farkının tanımından hemen açıktır

$$L_1 - L_2 = L_1 \cap \overline{L_2}.$$

Regular olan L_2 'nin varlığı, L_2 'nin de regular olduğu anlamına gelir. O zaman, regular dillerin kesişim altında kapalı olması nedeniyle,

$L_1 \cap L_2$ 'nin regular olduğunu biliyoruz ve argüman tamamlanmıştır.

Bir dizi diğer kapanış özelliği, temel argümanlarla doğrudan türetilebilir.

TEOREM 4.2

Regular diller ailesi, tersine çevirme altında kapalıdır.

Kanıt: Bu teoremin kanıtı, Bölüm 2.3'te bir alıştırmaya olarak önerilmişti. İşte detaylar. L düzenli bir dil olduğunu varsayalım. O zaman bunun için tek bir son durumlu bir nfa inşa ederiz. Bölüm 2.3'teki Alıştırma 7'ye göre, bu her zaman mümkündür. Bu nfa'nın geçiş grafi içinde başlangıç noktasını son nokta, son noktayı başlangıç noktası yaparız ve tüm kenarların yönünü tersine çeviririz. Bu oldukça basit bir meseledir

değiştirilmiş nfa'nın w^{R_i} 'yi kabul ettiğini göstermek, yalnızca orijinal nfa'nın w 'yi kabul etmesi durumunda mümkündür. Bu nedenle, değiştirilmiş nfa L^{R_i} 'yi kabul eder, tersine çevirme altında kapalı olduğunu kanıtlar. ■

Diğer İşlemler Altında Kapanış

Dillere yönelik standart işlemlerin yanı sıra, diğer işlemleri tanımlamak ve bunlar için kapanış özelliklerini araştırmak mümkündür. Bu tür birçok sonuç vardır; yalnızca iki tipik olanı seçiyoruz. Diğerleri bu bölümün sonunda yer alan alıştırmalarda incelenmektedir.

DEFİNİSYON 4.1

Σ ve Γ alfabeleri olsun. O zaman bir fonksiyon

$$h : \Sigma \rightarrow \Gamma^*$$

homomorfizm olarak adlandırılır. Kelimelerle, bir homomorfizm, tek bir harfin bir dizi ile değiştirildiği bir yerleştirmedir.

Fonksiyonun tanım kümesi

h açık bir şekilde dizelere uzatılır; eğer

$$w = a_1 a_2 \cdots a_n,$$

o zaman

$$h(w) = h(a_1) h(a_2) \cdots h(a_n).$$

Eğer $L \subseteq \Sigma^*$ üzerinde bir dil ise, o zaman homomorfik görüntüsü şöyle tanımlanır

$$h(L) = \{h(w) : w \in L\}.$$

ÖRNEK 4.2

$\Sigma = \{a, b\}$ ve $\Gamma = \{a, b, c\}$ olarak tanımlayın h şöyle tanımlansın

$$h(a) = ab,$$

$$h(b) = bbc.$$

O zaman $h(aba) = abbbcab$. $L = \{aa, aba\}$ dilinin homomorfik görüntüsü $h(L) = \{abab, abbbcab\}$.

Eğer bir dil L için bir düzenli ifade r varsa, o zaman $h(L)$ için bir düzenli ifade elde edilebilir, bu da homomorfizmin her Σ sembolüne uygulayarak yapılır.

ÖRNEK 4.3

$\Sigma = \{a, b\}$ ve $\Gamma = \{b, c, d\}$ alalım. h 'yi tanımlayın

$$\begin{aligned} h(a) &= dbcc, \\ h(b) &= bdc. \end{aligned}$$

L , tarafından gösterilen düzenli dil ise

$$r = (a + b^*)(aa)^*,$$

o zaman

$$r_1 = (dbcc + (bdc)^*)(dbccdbcc)^*$$

düzenli dil $h(L)$ olarak gösterilir.

Herhangi bir homo-

omorfizm altında düzenli dillerin kapanışı ile ilgili genel sonuç, bu örnekten açık bir şekilde ortaya çıkmaktadır.

TEOREM 4.3

h bir homomorfizma olsun. Eğer L bir düzenli dil ise, o zaman homomorfik resmi $h(L)$ de düzenlidir. Bu nedenle düzenli diller ailesi herhangi bir homomorfizma altında kapalıdır.

Kanıt: L , bazı düzenli ifadelerle gösterilen bir düzenli dil olsun r . $h(r)$ bulmak için, her sembol $a \in \Sigma$ için $h(a)$ ile değiştirilir. Sonucun bir düzenli ifade olduğunu doğrudan düzenli bir ifadenin tanımına başvurarak göstermek mümkündür. Elde edilen ifadenin $h(L)$ olduğunu görmek de oldukça kolaydır. Yapmamız gereken tek şey, her $w \in L(r)$ için, karşılık gelen $h(w)$ 'nin $L(h(r))$ içinde olduğunu ve tersine, her v 'nin $L(h(r))$ içinde olduğunu ve böylece $v = h(w)$ olduğunu göstermektir. Ayrıntıları bir alıştırmaya bırakarak, $h(L)$ 'nin düzenli olduğunu iddia ediyoruz. ■

DEFINITION 4.2

Let L_1 and L_2 be languages on the same alphabet. Then the **right quotient** of L_1 with L_2 is defined as

$$L_1 / L_2 = \{x : xy \in L_1 \text{ for some } y \in L_2\}. \quad (4.1)$$

To form the right quotient of L_1 with L_2 , we take all the strings in L_1 that have a suffix belonging to L_2 . Every such string, after removal of this suffix, belongs to L_1/L_2 .

EXAMPLE 4.4

If

$$L_1 = \{a^n b^m : n \geq 1, m \geq 0\} \cup \{ba\}$$

ve

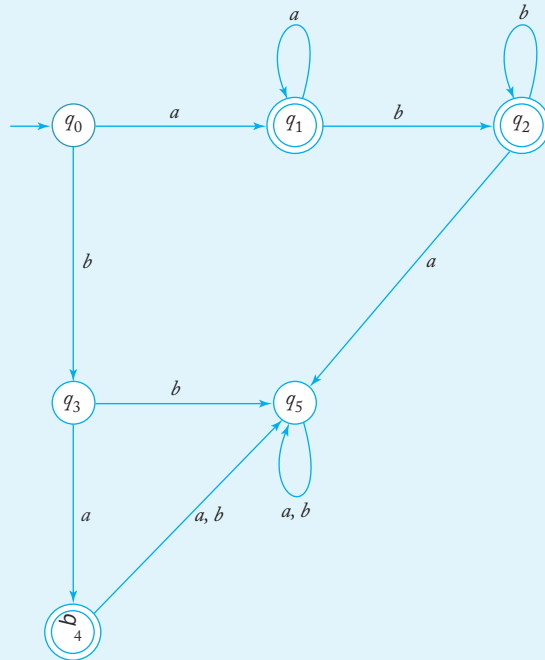
$$L_2 = \{b^m : m \geq 1\},$$

o zaman

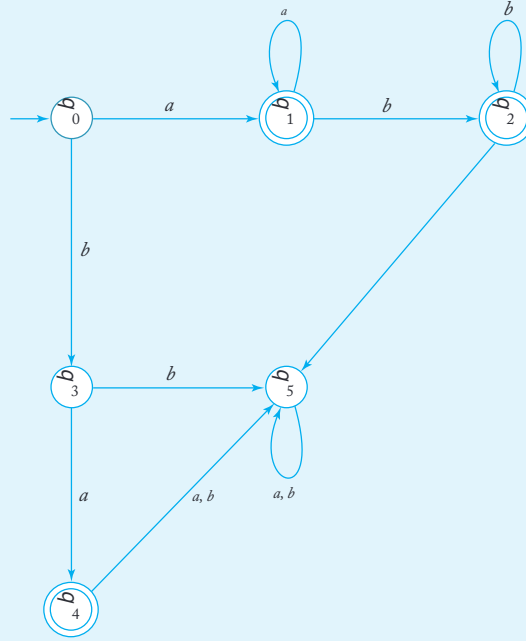
$$L_1/L_2 = \{a^n b^m : n \geq 1, m \geq 0\}.$$

The strings in L_2 consist of one or more b 's. Therefore, we arrive at the answer by removing one or more b 's from those strings in L_1 that terminate with at least one b .

Note that here L_1 , L_2 , and L_1/L_2 are all regular. This suggests that the right quotient of any two regular languages is also regular. We will prove this in the next theorem by a construction that takes the dfa's for L_1 and L_2 and constructs from them a dfa for L_1/L_2 . Before we



ŞEKİL 4.1



ŞEKİL 4.2

inşaata tam olarak tanımlayın, bu örneğe nasıl uygulandığını görelim. L_1 için bir dfa ile başlıyoruz; otomata $M_1 = (Q, \Sigma, \delta, q_0, F)$ ŞEKİL 4.1'de. L_1/L_2 için bir otomatanın herhangi bir ön eki kabul etmesi gerektiğinden string'lerin L_1 içinde, M_1 'i kabul edecek şekilde değiştirmeye çalışacağız eğer herhangi bir y (4.1)'i sağlayan varsa. Zorluk, bazı y olup olmadığını bulmaktan gelir y öyle ki $xy \in L_1$ ve $y \in L_2$. Bunu çözmek için, her $q \in Q$ için, etiketli v bir son duruma yürüyüş olup olmadığını belirliyoruz böylece $v \in L_2$. Eğer öyleyse, herhangi bir x için $\delta(q_0, x) = q$ L_1/L_2 içinde olacaktır. Biz otomata uygun şekilde değiştiriyoruz q 'yu son durum yapacak şekilde.

Bu durumu mevcut durumumuza uygulamak için, her durumu kontrol ediyoruz $q_0, q_1, q_2, q_3, q_4, q_5$ herhangi bir q_1, q_2 veya

q_4 'ye etiketli bb^* bir yürüyüş olup olmadığını görmek için. Sadece q_1 ve q_2 uygun olduğunu görüyoruz; q_0, q_3, q_4 uygun değildir. Elde edilen otomata L_1/L_2 için Şekil 4.2'de gösterilmiştir. İnceleyin ve yapının ϵ alıştığını görün. Fikir, bir sonraki teoreme genelleştirilmiştir.

TEOREM 4.4

Eğer L_1 ve L_2 düzenli diller ise, o zaman L_1/L_2 de düzenlidir. Düzenli diller ailesinin, düzenli bir dil ile sağ bölme altında kapalı olduğunu söyleriz.

Kanıt: $L_1 = L(M)$ olarak tanımlayalım, burada $M = (Q, \Sigma, \delta, q_0, F)$ bir dfa'dır. Başka bir dfa $\widehat{M} = (Q, \Sigma, \delta, q_0, \widehat{F})$ aşağıdaki gibi inşa ediyoruz. Her $q_i \in Q$ için, bir $y \in L_2$ olup olmadığını belirleyin, böylece

$$\delta^*(q, y) = q \in F.$$

Bu, dfa'ların $M_i = (Q, \Sigma, \delta, q_i, F)$ incelenmesiyle yapılabilir. Otomaton M_i , başlangıç durumu q_0 yerine q_i ile değiştirilmiş M 'ye eşittir. Şimdi, $L(M_i)$ içinde L_2 'de de bulunan bir y olup olmadığını belirliyoruz. Bunun için, Teorem 4.1'de verilen iki düzenli dilin kesişimi için inşaatı kullanabiliriz, $L_2 \cap L(M_i)$ için geçiş grafiğini buluyoruz. Başlangıç tepe noktası ile herhangi bir son tepe noktası arasında bir y ol varsa, o zaman $L_2 \cap L(M_i)$ boş değildir. Bu durumda, q_i 'yi $\widehat{F}$ 'ye ekleyin. Her $q_i \in Q$ için bunu tekrarlayarak $\widehat{F}$ 'yi belirliyoruz ve böylece $\widehat{M}$ 'yi inşa ediyoruz.

$L(\widehat{M}) = L_1/L_2$ olduğunu kanıtlamak için, xL_1/L_2 'nin herhangi bir elemanı olsun. O zaman $xy \in L_1$ olacak şekilde bir $y \in L_2$ olmalıdır. Bu,

$$\delta^*(q_0, xy) \in F,$$

öyle ki, bazı $q \in Q$ olmalıdır ki

$$\delta^*(q, y) = q$$

ve

$$\delta^*(q, y) \in F.$$

Bu nedenle, inşaat gereği, $q \in \widehat{F}$ ve $\widehat{M}$, çünkü $\delta^*(q_0, x) \in \widehat{F}$ dedir.

Tam tersine, herhangi bir $x \in \widehat{M}$ tarafından kabul edildiğinde, şunu elde ederiz

$$\delta^*(q_0, x) = q \in \widehat{F}.$$

Ancak yine de yapısı gereği, bu, L_2 içinde bir $y \in$ olduğunu ima eder. $\delta^*(q, y) \in F$. Bu nedenle, $xy \in L_1$ içindedir ve $x \in L_1/L_2$ içindedir. Bu nedenle şunu sonuçlandırıyoruz ki

$$L(\widehat{M}) = L_1/L_2,$$

ve buradan L_1/L_2 düzenlidir. ■

ÖRNEK 4.5

L_1/L_2 bul

$$L_1 = L(a^*baa^*),$$

$$L_2 = L(ab^*).$$

Öncelikle L_1 kabul eden bir dfa buluyoruz. Bu kolaydır ve bir çözüm Şekil 4.3'te verilmiştir. Örnek, inşaatın resmiyetlerini atlayabileceğimiz kadar basittir. Şekil 4.3'teki grafikten oldukça açık bir şekilde

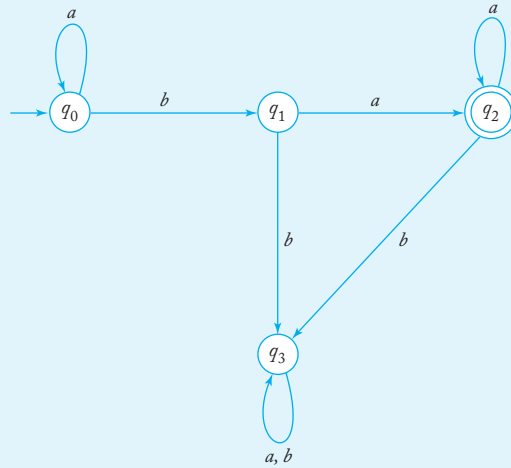
$$L(M_0) \cap L_2 = \emptyset,$$

$$L(M_1) \cap L_2 = \{a\} \neq \emptyset,$$

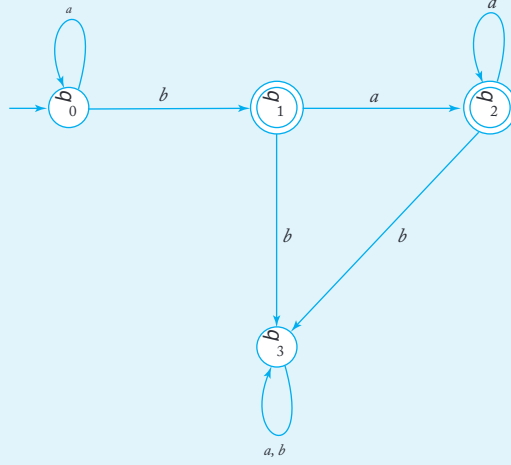
$$L(M_2) \cap L_2 = \{a\} \neq \emptyset,$$

$$L(M_3) \cap L_2 = \emptyset.$$

Bu nedenle, otomatonun kabul ettiği L_1/L_2 belirlenmiştir. Sonuç Şekil 4.4'te gösterilmiştir. Bu, düzenli ifadeyle belirtilen dili kabul eder $a^*b + a^*baa^*$, bu da a^*baa^* olarak basitleştirilebilir. Böylece $L_1/L_2 = L(a^*baa^*)$.



ŞEKİL 4.3



ŞEKİL 4.4

EGZERSİZLER

1. $\Sigma = \{a, b\}$ için, aşağıdaki dillerin tamamlayıcısı için düzenli ifadeleri bulun:

(a) $L = L(aa^*bb^*)$

(b) $L = L(a^*b^*)^*$

2. $L_1 = L(ab^*aa)$, $L_2 = L(a^*bba^*)$ olarak alalım. $(L_1 \cup L_2)^* L_2$.

3. L için bir dfa'dan L^* için bir sonlu otomaton nasıl inşa edileceğini gösterin.

4. Bir dil ailesinin birleşim ve tamamlayıcı altında kapalı olduğunu gösterirseniz, kesim altında da kapalı olması gerektiğini gösterin.

5. Teorem 4.1'de kesim altında kapalı olmanın yapısal kanıtının detaylarını doldurun.

6. Teorem 4.1'deki yapıyı kullanarak kabul eden nfa'lar bulun.

(a) $L((ab)^*a^*) \cap L(baa^*)$

(b) $L(ab^*a^*) \cap L(a^*b^*a)$

7. Örnek 4.1'de, düzenli diller için fark altında kapalı olduğunu gösterdik, bu nunla birlikte kanıt yapısal değildi. Bu sonucu yapısal bir argümanla sağl ayın, Teorem 4.1'deki kesim argümanında kullanılan yaklaşımı takip eder ek.

8. Teorem 4.3'ün kanıtında, $h(tr)$ bir düzenli ifade olduğunu gösterin. Sonra $h(tr)$ 'nin $h(L)$ 'yi belirttiğini gösterin.

9. Gösterin ki, düzenli diller ailesi sonlu birleşim ve kesişim altında kapalıdır, yani eğer $L_1, L_2, \dots, L_n$ düzenli ise,

$$L_U = \bigcup_{i=1,2,\dots,n} L_i$$

ve

$$L_I = \bigcap_{i=1,2,\dots,n} L_i$$

de düzenlidir.

10. İki kümenin S_1 ve S_2 simetrik farkı,

$$S_1 \ominus S_2 = \{x : x \in S_1 \text{ veya } x \in S_2, \text{ ancak } x \text{ hem } S_1 \text{ hem de } S_2 \text{ içinde değildir}\}.$$

Gösterin ki düzenli diller ailesi simetrik fark altında kapalıdır.

11. İki dilin nor'u

$$\text{nor}(L_1, L_2) = \{w : w \notin L_1 \text{ and } w \notin L_2\}.$$

Gösterin ki düzenli diller ailesi nor işlemi altında kapalıdır.

12. İki dilin tamamlayıcı veya (*cor*) tanımlayın

$$\text{cor}(L_1, L_2) = \{w : w \in \bar{L}_1 \text{ or } w \in \bar{L}_2\}.$$

Gösterin ki düzenli diller ailesi cor işlemi altında kapalıdır.

13. Aşağıdakilerden hangileri tüm düzenli diller ve tüm homomorfizmalar için doğrudur?

(a) $h(L_1 \cup L_2) = h(L_1) \cup h(L_2).$

(b) $h(L_1 \cap L_2) = h(L_1) \cap h(L_2).$

(c) $h(L_1 L_2) = h(L_1) h(L_2).$

14. Let $L_1 = L(a^* b a a^*)$ and $L_2 = L(a b a^*)$. Find L_1 / L_2 .

15. Gösterin ki $L_1 = L_1 L_2 / L_2$ tüm diller için doğru değildir L_1 ve L_2 .

16. Diyelim ki $L_1 \cup L_2$ düzenli ve L_1 sonludur. Bu durumda L_2 'nin düzenli olduğunu sonuçlandırabilir miyiz?

17. Eğer L düzenli bir dilse, ispatlayın ki $L_1 = \{uv : u \in L, |v| = 2\}$ de düzenlidir.

18. Eğer L düzenli bir dilse, ispatlayın ki dil $\{uv : u \in L, v \in L^R\}$ de düzenlidir.

19. Bir dilin L_1 sol bölümünün L_2 ile ilgili tanımı

$$L_2 / L_1 = \{y : x \in L_2, xy \in L_1\}.$$

Bir düzenli dilin sol bölümünün düzenli bir dil ile kapalı olduğunu gösterin.

20. “Eğer L_1 düzenli ise ve $L_1 \cup L_2$ de düzenli ise, o zaman L_2 de düzenli olmalıdır” ifadesi tüm L_1 ve L_2 için doğru olsaydı, o zaman tüm diller düzenli olurdu.

21. Bir dilin kuyruğu, dizelerinin tüm son eklerinin kümesi olarak tanımlanır, yani,

$$\text{tail}(L) = \{y : xy \in L \text{ for some } x \in \Sigma^*\}.$$

Eğer L düzenli ise, o zaman $\text{tail}(L)$ de düzenlidir.

22. Bir dilin başı, dizelerinin tüm ön eklerinin kümesidir, yani,

$$\text{head}(L) = \{x : xy \in L \text{ for some } y \in \Sigma^*\}.$$

Bu işlemin altında düzenli diller ailesinin kapalı olduğunu gösterin.

23. Dizeler ve diller üzerinde üçüncü bir işlem tanımlayın.

$$\text{third}(a_1a_2a_3a_4a_5a_6\cdots) = a_3a_6\cdots$$

Bu tanımın diller için uygun uzantısı ile. Bu işlemin altında düzenli diller ailesinin kapanışını kanıtlayın.

24. Bir dize için $a_1a_2\cdots a_n$ işlemini kaydırma olarak tanımlayın.

$$\text{shift}(a_1a_2\cdots a_n) = a_2\cdots a_na_1.$$

Bundan, bir dil üzerindeki işlemi şu şekilde tanımlayabiliriz.

$$\text{shift}(L) = \{v : v = \text{shift}(w) \text{ for some } w \in L\}.$$

Düzenliliğin kaydırma işlemi altında korunduğunu gösterin.

25. Tanımlayın.

$$\text{exchange}(a_1a_2\cdots a_{n-1}a_n) = a_na_2\cdots a_{n-1}a_1,$$

ve

$$\text{exchange}(L) = \{v : v = \text{exchange}(w) \text{ for some } w \in L\}.$$

Gösterin ki düzenli diller ailesi değişim altında kapalıdır.

26. Dil L için $\min$ tanımı

$$\min(L) = \{w \in L : \text{there is no } u \in L, v \in \Sigma^+, \text{ such that } w = uv\}.$$

Düzenli diller ailesinin $\min$ işlemi altında kapalı olduğunu gösterin.

27. G_1 ve G_2 iki düzenli gramer olsun. Diller için düzenli gramerlerin nasıl türetilceğini gösterin

(a) $L(G_1) \cup L(G_2).$

(b) $L(G_1)L(G_2).$

(c) $L(G_1)^*.$

4.2 REGÜLER DİLLER HAKKINDA TEMEL SORULAR

Şimdi çok temel bir konuya geliyoruz: Bir dil L ve bir dizew verildiğinde, w 'nin L 'nin bir elemanı olup olmadığını belirleyebilir miyiz? Bu, üyelik sorusudur ve buna cevap vermek için bir yöntem üyelik algoritması olarak adlandırılır.* Verimli üyelik algoritmalarını bulamadığımız dillerle çok az şey yapılabilir. Üyelik algoritmalarının varlığı ve doğası, sonraki tartışmalarda büyük bir endişe kaynağı olacaktır; bu, genellikle zor bir meseledir. Ancak düzenli diller için bu kolay bir meseledir.

Öncelikle “bir dil verildiğinde...” derken tam olarak ne demek istediğimizi düşünelim. Birçok argümanda, bunun belirsiz olmaması önemlidir. Regüler dilleri tanımlamak için birkaç yol kullandık: gayri resmi sözlü tanımlar, küme notasyonu, sonlu otomata, düzenli ifadeler ve düzenli gramerler. Sadece son üçü, teoremlerde kullanılmak üzere yeterince iyi tanımlanmıştır. Bu nedenle, bir düzenli dilin standart bir temsilde verildiğini söyleriz, eğer ve yalnızca bir sonlu otomata, bir düzenli ifade veya bir düzenli gramerle tanımlanıyorsa.

TEOREM 4.5

Herhangi bir düzenli dilin standart temsili L üzerinde ve herhangi bir $w \in \Sigma^*$ için, w 'nin L 'de olup olmadığını belirlemek için bir algoritma vardır.

Kanıt: Dili bir dfa ile temsil ederiz, ardından w 'yi bu otomatonun kabul edip etmediğini test ederiz. ■

Diğer önemli sorular, bir dilin sonlu mu yoksa sonsuz mu olduğu, bir dilin aynı olup olmadığı ve bir dilin diğerinin alt kümesi olup olmadığıdır. En azından düzenli diller için, bu sorular kolayca yanıtlanır.

TEOREM 4.6

Standart temsil ile verilen bir düzenli dilin boş, sonlu veya sonsuz olup olmadığını belirlemek için bir algoritma vardır.

Kanıt: Dili bir dfa'nın geçiş grafi olarak temsil edersek, cevap açıktır. Başlangıç tepe noktasından herhangi bir son tepe noktasına basit bir yol varsa, o zaman dil boş değildir.

* Daha sonra “algoritma” teriminin ne anlama geldiğini netleştireceğiz. Şu anda, bunu bir bilgisayar programı yazılabilecek bir yöntem olarak düşünün.

Dilinin sonsuz olup olmadığını belirlemek için, bazı döngülerin tabanı olan tüm köşeleri bulun. Eğer bunlardan herhangi biri başlangıç köşesinden son köşeye giden bir yol üzerindeyse, dil sonsuzdur. Aksi takdirde, sonludur.

İki dilin eşitliği sorusu da önemli bir pratik meseledir. Genellikle bir programlama dilinin birkaç tanımı vardır ve farklı görünümüne rağmen aynı dili tanımlayıp tanımlamadıklarını bilmemiz gerekir. Bu genellikle zor bir problemdir; düzenli diller için bile argüman açık değildir. Cümle cümle karşılaştırma yaparak tartışmak mümkün değildir, çünkü bu yalnızca sonlu diller için geçerlidir. Normal ifadeleri, gramerleri veya dfa'ları inceleyerek cevabı görmek de kolay değildir. Şık bir çözüm, zaten belirlenmiş kapanış özelliklerini kullanır.

TEOREM 4.7

İki düzenli dilin standart temsilleri L_1 ve L_2 verildiğinde, $L_1 = L_2$ olup olmadığını belirlemek için bir algoritma vardır.

Kanıt: L_1 ve L_2 kullanarak dili tanımlıyoruz

$$L_3 = (L_1 \cap \overline{L_2}) \cup (\overline{L_1} \cap L_2).$$

Kapanış ile, L_3 düzenlidir ve L_3 'ü kabul eden bir dfa^M bulabiliriz. M'ye sahip olduğumuzda, Teorem 4.6'daki algoritmayı kullanarak

L_3 'ün boş olup olmadığını belirleyebiliriz. Ancak Egzersiz 8, Bölüm 1.1'den, $L_3 = \emptyset$ yalnızca $L_1 = L_2$ olduğunda olduğunu görüyoruz. ■

Bu sonuçlar, açık ve şaşırtıcı olmalarına rağmen temeldir. Düzenli diller için, Teorem 4.5'ten 4.7'ye kadar ortaya atılan sorular kolayca ya nitlanabilir, ancak diğer dil aileleriyle uğraşırken bu her zaman böyle değildir. Daha sonra birkaç kez bu tür sorularla karşılaşacağız. Biraz öngöründe bulunursak, cevapların giderek daha zor hale geldiğini ve sonunda bulunmasının imkansız hale geldiğini göreceğiz.

EGZERSİZLER

Bu bölümdeki tüm alıştırmalar için, düzenli dillerin standart temsiline verildiğini varsayın.

1. Verilen bir düzenli dil L ve bir dize $w \in L$, $w^R \in L$ olup olmadığını nasıl belirleyeceğinizi gösterin.
2. Bir düzenli dil L 'nin, $w^R \in L$ olacak şekilde herhangi bir dize w içerip içermediğini belirlemek için bir algoritma sergileyin.

3. Bir dilin *palindrome* dili olduğu söylenir eğer $L = L^R$. Verilen bir düzenli dilin palindrome dili olup olmadığını belirlemek için bir algoritma bulun.
4. Herhangi bir verilen w ve herhangi bir düzenli diller L_1 ve L_2 için $w \in L_1 - L_2$ olup olmadığını belirlemek için bir algoritmanın var olduğunu gösterin.
5. Herhangi bir düzenli diller L_1 ve L_2 için $L_1 \subseteq L_2$ olup olmadığını belirlemek için bir algoritmanın var olduğunu gösterin.
6. Herhangi bir düzenli diller L_1 ve L_2 için L_1 'in L_2 'nin proper alt kümesi olup olmadığını belirlemek için bir algoritmanın var olduğunu gösterin.
7. Herhangi bir düzenli dil L için $\lambda \in L$ olup olmadığını belirlemek için bir algoritmanın var olduğunu gösterin.
8. Bir algoritmanın var olduğunu gösterin, böylece herhangi bir düzenli dil L için $L = \Sigma^*$ olup olmadığını belirleyebilirsiniz.
9. Herhangi üç düzenli dil verildiğinde, $L, L_1, L_2, L = L_1 L_2$ olup olmadığını belirleyen bir algoritma gösterin.
10. Herhangi bir düzenli dil L verildiğinde, $L = L^*$ olup olmadığını belirleyen bir algoritma gösterin.
11. L, Σ üzerinde bir düzenli dil olsun ve w, Σ^* içinde herhangi bir dize olsun. L 'nin w 'nin bir alt dizesi olduğu herhangi bir w içerip içermediğini belirlemek için bir algoritma bulun; yani, $w = u \hat{w} v$, $u, v \in \Sigma^*$, ve $|uv| > 0$.
12. $\text{tail}(L)$ işlemi şu şekilde tanımlanır:

$$\text{tail}(L) = \{v : uv \in L, u, v \in \Sigma^*\}.$$

Herhangi bir düzenli L için $L = \text{tail}(L)$ olup olmadığını belirleyen bir algoritmanın var olduğunu gösterin.

13. $\Sigma = \{a, b\}$ üzerinde herhangi bir düzenli dil L olsun. L 'nin çift uzunlukta herhangi bir dize içerip içermediğini belirlemek için bir algoritmanın var olduğunu gösterin.
14. Her düzenli dil L için $|L| \geq 5$ olup olmadığını belirleyebilen bir algoritmanın var olduğunu gösterin.
15. Bir düzenli dil L 'nin sonlu sayıda çift uzunlukta dizeler içerip içermediğini belirlemek için bir algoritma bulun.
16. Bir düzenli gramer G verildiğinde, $L(G) = \Sigma^*$ olup olmadığını belirleyebilen bir algoritmayı tanımlayın.
17. İki düzenli gramer G_1 ve G_2 verildiğinde, birinin nasıl belirlenebileceğini gösterin. $L(G_1) \cup L(G_2) = \Sigma^*$
18. İki düzenli dilin eşit olup olmadığını belirlemek için bir algoritma tanımlayın.
19. İki düzenli ifadenin eşit olup olmadığını belirlemek için bir algoritma tanımlayın.

4.3 DÜZENLİ OLMAYAN DİLLERİ TANIMLAMA

Düzenli diller sonsuz olabilir, çoğu örneğimiz bunu göstermiştir. Ancak, düzenli dillerin sonlu belleğe sahip otomata ile ilişkilendirilmesi, bir düzenli dilin yapısı üzerinde bazı sınırlamalar getirir. Düzenliliğin sağlanabilmesi için bazı dar kısıtlamalara uyulması gerekir. Bir dilin düzenli olduğunu anlamak için, herhangi bir dize işlenirken, hatırlanması gereken bilginin her aşamada kesinlikle sınırlı olması gerektiği sezgisi vardır. Bu doğrudur, ancak anlamlı bir şekilde kullanılabilmesi için kesin olarak gösterilmesi gerekir. Bunu yapmanın birkaç yolu vardır.

Güvercin Yuvası Prensibini Kullanma

“Güvercin yuvası prensibi” terimi, matematikçiler tarafından aşağıdaki basit gözlemi ifade etmek için kullanılır. Eğer n nesneyi m kutuya (güvercin yuvalarına) koyarsak ve eğer $n > m$ ise, o zaman en az bir kutuda birden fazla nesne olmalıdır. Bu, o kadar açık bir gerçek ki, ondan ne kadar derin sonuçlar elde edilebileceği şaşırtıcı.

ÖRNEK 4.6

$\text{Dil } L = \{a^n b^n : n \geq 0\}$ düzenli mi? Cevap hayır, çünkü bunu çelişki ile kanıtlayarak gösteriyoruz.

Diyelim ki L düzenli. O zaman bunun için bazı $\text{dfa } M = (Q, \{a, b\}, \delta, q_0, F)$ vardır. Şimdi $\delta^*(q_0, a^i)$ için $i = 1, 2, 3, \dots$ bakın. Sınırsız sayıda i olduğu için, ancak M 'de sonlu sayıda durum bulunduğu için, güvercin yuvası prensibi bize bazı durumların, diyelim ki q , olması gerektiğini söyler.

$$\delta^*(q_0, a^i) = q$$

ve

$$\delta^*(q_0, a^j) = q,$$

ilen $j = m$. Ama M $a^m b^n$ kabul ettiğinden, şunları elde etmemiz gerekir:

$$\delta^*(q, b^n) = q_f \in F.$$

Bundan şunu çıkarabiliriz ki:

$$\begin{aligned} \delta^*(q_0, a^m b^n) &= \delta^* \delta^*(q_0, a^m), b^n \\ &= \delta^*(q, b^n) \\ &= q_f. \end{aligned}$$

Bu, başlangıçtaki varsayım olan M 'nin $a^m b^n$ 'yi yalnızca eğer $n = m$ ise kabul ettiği gerçeğiyle çelişiyor ve bize L 'nin düzenli olamayacağı sonucuna varmamıza yol açıyor.

Bu argümanda, güvercin yuvası prensibi, sonlu bir otomatonun sınırlı bir belleğe sahip olduğunu söylediğimizde ne demek istediğimizi açıkça ifade etmenin bir yoludur. Tüm $a^n b^n$ 'yi kabul etmek için, bir otomaton tüm ön ekler a^n ve a^m arasında ayırım yapmak zorunda kalır. Ancak bunu yapmak için yalnızca sonlu sayıda iç durum olduğundan, bazı n ve m için bu ayırım yapılamaz.

Bu tür bir argümanı çeşitli durumlarda kullanmak için, onu genel bir teorem olarak kodlamak uygundur. Bunu yapmanın birkaç yolu vardır; burada verdiğimiz belki de en ünlü olanıdır.

Regüler diller için geçiş grafikleri hakkında bildiklerimiz şunlardır:

- Eğer geçiş grafiğinde döngü yoksa, dil sonludur ve dolayısıyla regüldür.
- Eğer geçiş grafiğinde boş olmayan bir etikete sahip bir döngü varsa, dil sonsuzdur. Tersine, her sonsuz regüler dilin böyle bir döngüye sahip bir dfa'sı vardır.
- Eğer bir döngü varsa, bu döngü ya atlanabilir ya da keyfi sayıda tekrar edilebilir. Yani, eğer döngünün etiketi v ise ve eğer dize $w_1 v w_2$ dilin içindeyse, o zaman dizeler $w_1 w_2, w_1 v v w_2, w_1 v v v w_2$,
ve devamı da dilin içindedir.
- Dfa'da bu döngünün nerede olduğunu bilmiyoruz, ancak eğer dfa'nın durumları m ise, döngü m sembolü okunduğunda girilmiş olmalıdır.

Eğer bir dil L için, bu özelliğe sahip olmayan en az bir dize w varsa, L regüler olamaz. Bu gözlem, bir teorem olarak resmi olarak pompa lemması olarak ifade edilebilir.

Bir Pompa Lemması

Aşağıdaki sonuç, düzenli diller için bilinen pompa lemması, başka bir biçimde güvercin yuvası ilkesini kullanır. Kanıt, n köşesi olan bir geçiş grafiğinde, n uzunluğunda veya daha uzun bir yürüyüşün mutlaka bazı köşeleri tekrar etmesi gerektiği, yani bir döngü içermesi gerektiği gözlemi üzerine kuruludur.

TEOREM 4.8

Bir L sonsuz düzenli dil olsun. O zaman, herhangi bir $w \in L$ için $|w| \geq m$ olan pozitif bir tam sayı m vardır ve bu, aşağıdaki gibi ayrıştırılabilir:

$$w = xyz$$

ile

$$|xy| \leq m,$$

ve

$$|y| \geq 1,$$

öyle ki

$$w_i = xy^i z, \quad (4.2)$$

tüm $i = 0, 1, 2, \dots$ için L içinde de bulunmaktadır.

Bunu başka bir şekilde ifade etmek gerekirse, L içindeki yeterince uzun her dize, orta kısmın rastgele sayıda tekrarıyla başka bir L dizisine dönüşecek şekilde üç parçaya ayrılabilir. Orta dizeye “pompalama” denir, bu nedenle bu sonucun adı pompalama lemmasıdır.

Kanıt: Eğer L düzenliyse, onu tanıyan bir dfa vardır. Böyle bir dfa'nın $q_0, q_1, q_2, \dots, q_n$ etiketli durumları olsun. Şimdi, L içinde $|w| \geq m = n + 1$ olan bir dize w alalım. Çünkü L sonsuz olarak varsayılmıştır, bu her zaman yapılabilir. Otomatonun w 'yi işlerken geçtiği durumlar kümesini düşünelim, diyelim

$$q_0, q_i, q_j, \dots, q_f.$$

Bu dizinin tam olarak $|w| + 1$ girişi olduğundan, en az bir durumun tekrar edilmesi gerekir ve böyle bir tekrar n . hamlesinden daha geç başlamamalıdır. Bu nedenle, dizi şöyle görünmelidir

$$q_0, q_i, q_j, \dots, q_r, \dots, q_r, \dots, q_f,$$

w içinde x, y, z alt dizileri olmalıdır ki

$$\delta^*(q_0, x) = q_r,$$

$$\delta^*(q_r, y) = q_r,$$

$$\delta^*(q_r, z) = q_f,$$

$|xy| \leq n + 1 = m$ ve $|y| \geq 1$. Buradan hemen şunu çıkarıyoruz ki

$$\delta^*(q_0, xz) = q_f,$$

ve ayrıca

$$\delta^*(q_0, xy^3z) = q_f,$$

$$\delta^*(q_0, xy^3z) = q_f,$$

ve devam ederek, teoremin kanıtını tamamlıyoruz. ■

Pompala lema'sını yalnızca sonsuz diller için verdik. Sonlu diller, her ne kadar her zaman düzenli olsa da, pompalanamaz çünkü pompalama otomatik olarak sonsuz bir küme oluşturur. Teorem sonlu diller için geçerlidir,

ancak bu boş bir ifadedir. Pompala lema'sındaki m , en uzun diziden daha büyük alınmalıdır, böylece hiçbir dizi pompalanamaz.

Pompala lema'sı, Örnek 4.6'daki güvercin yuvası argümanı gibi, belirli dillerin düzenli olmadığını göstermek için kullanılır. Gösterim her zaman ϵ ilişkisi ile yapılır. Burada ifade ettiğimiz gibi pompala lema'sında, bir dilin düzenli olduğunu kanıtlamak için kullanılabilecek hiçbir şey yoktur. Her ne kadar pompalanan herhangi bir dizinin orijinal dilde olması gerektiğini gösterebilseydik (ve bu genellikle oldukça zordur), Teorem 4.8'in ifadesinde, bu durumdan dilin düzenli olduğu sonucuna varmamıza izin veren hiçbir şey yoktur.

ÖRNEK 4.7

Pompa lemmasını kullanarak $L = \{a^n b^n : n \geq 0\}$ düzenli olmadığını gösterin. L düzenli olduğunu varsayalım, böylece pompa lemması geçerli olmalıdır. m değerini bilmiyoruz, ama ne olursa olsun, her zaman $n = m$ seçebiliriz. Bu nedenle, alt dize y tamamen a 'lerden oluşmalıdır. Varsayalım ki $|y| = k$. O zaman, Denklem (4.2) kullanılarak elde edilen dize $i = 0$ olduğunda

$$w_0 = a^{m-k} b^m$$

ve açıkça L içinde değildir. Bu, pompa lemmasıyla çelişir ve böylece L 'nin düzenli olduğu varsayımının yanlış olması gerektiğini gösterir.

Pompa lemmasını uygularken, teoremin ne söylediğini aklımızda tutmalıyız. Bizim için bir m ve ayrıştırma xyz varlığının garantisi vardır, ancak bunların ne olduğunu bilmiyoruz. Sadece pompa lemmasının bazı özel değerler için ihlal edilmesi nedeniyle bir çelişkiye ulaştığımızı iddia edemeyiz m veya xyz . Öte yandan, pompa lemması her $w \in L$ ve her i için geçerlidir. Bu nedenle, pompa lemması bir w veya i için bile ihlal edilirse, o zaman dilim düzenli olamaz.

Doğru argüman, bir rakibe karşı oynadığımız bir oyun olarak görseleştirilebilir. Amacımız, pompa lemmasının bir çelişkisini kurarak oyunu kazanmaktır, rakip ise bizi engellemeye çalışır. Oyunda dört hamle vardır.

1. Rakip m seçer.
2. Verilen m ile, uzunluğu m 'ye eşit veya daha büyük olan L içindeki bir dize w seçeriz. Herhangi bir w seçmekte serbestiz, şartları $w \in L$ ve $|w| \geq m$.
3. Rakip, ayrıştırmayı xyz seçer, şartları $|xy| \leq m, |y| \geq 1$. Rakibin, oyunu kazanmamız için en zor hale getirecek seçimi yaptığını varsaymamak zorundayız.

4. Pompalı dizeyi w^i , Denklem (4.2)'de tanımlandığı gibi, L içinde olmayacak şekilde seçmeye çalışıyoruz. Bunu yapabilirsek, oyunu kazanırız.

Rakiplerin seçimleri ne olursa olsun kazanmamıza olanak tanıyan bir strateji, lisanın düzenli olmadığını kanıtlamakla eşdeğerdir. Bu bağlamda, Adım 2 kritik öneme sahiptir. Rakibi belirli bir parçalama seçimini yapmaya zorlayamasa da, w 'yi öyle seçebiliriz ki, rakip Adım 3'te çok kısıtlı kalır ve bu da bizetamponlama lemmasını ihlal etmemizi sağlayacak şekilde x, y ve z 'yi seçme zorunluluğu getirir.

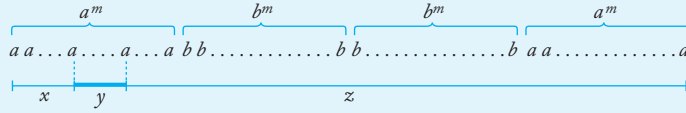
ÖRNEK 4.8

Gösterin ki

$$L = \{ww^R : w \in \Sigma^*\}$$

düzenli değildir.

Her ne olursa olsun m rakip 1. Adımda seçerse, her zaman bir w seçebiliriz, Şekil 4.5'te gösterildiği gibi. Bu seçim nedeniyle ve $|xy| \leq m$ gerekliliğinden dolayı, rakip 3. Adımda bir y seçmekte kısıtlıdır. tamamen a 'lerden oluşur. 4. Adımda, $i = 0$ kullanıyoruz. Elde edilen dize bu modanın solda sağdan daha az a var ve bu nedenle şu şekilde olamaz ww^R . Bu nedenle, L düzenli değildir.



ŞEKİL 4.5

Not edin ki eğer bir w çok kısa, o zaman rakip bir y çift sayıda b ile seçmişiz. Bu durumda, son adımda pompa lemmasının ihlaline ulaşamamış olurduk. Ayrıca, tüm a 'lerden oluşan bir dize seçersek, diyelim ki, başarısız olurduk.

$$w = a^{2m},$$

rakip sadece seçmek zorundadır L . Bizi yenmek için

$$y = aa.$$

Şimdi w_i L içindedir tüm i için, ve kaybediyoruz.

Pompala lema uygulamak için rakibin yanlış bir hamle yapacağı varsayamayız. Eğer, seçtiğimiz durumda $w = a^{2m}$ ise, rakip seçerse

$$y = a,$$

w_0 o zaman w_0 tek uzunlukta bir dizedir ve bu nedenle L içinde değildir. Ancak herhangi bir argüman, rakibin bu kadar uyumlu olduğunu varsayıyorsa, otomatik olarak yanlıştır.

ÖRNEK 4.9

Şimdi $\Sigma = \{a, b\}$. Dil

$$L = \{w \in \Sigma^* : n_a(w) < n_b(w)\}$$

düzenli değildir.

Diyelim k verildi. Seçim yaparken w üzerinde tam özgürlüğe sahip olduğumuz için $w = a^m b^{m+1}$ olarak seçiyoruz. Şimdi, w büyük olamaz, rakip hiçbir şey yapamaz, sadece tüm a 'larla bir y seçer, yani

$$y = a^k, \quad 1 \leq k \leq m.$$

Şimdi $i = 2$ kullanarak pompalıyoruz. Ortaya çıkan dize

$$w_2 = a^{m+2} b^{m+1}$$

L içinde değildir. Bu nedenle, pompalama lemması ihlal edilmiştir ve L düzenli değildir.

ÖRNEK 4.10

Dil

$$L = \{a^n b^k : n > k, k \geq 0\}$$

düzenli değildir.

Verilen m , dize olarak seçiyoruz

$$w = (ab)^{m+1} a^m,$$

w L içindedir. Kısıtlama $|xy| \leq m$ nedeniyle, hem x hem de y stringin ab 'lerden oluşan kısmında olmalıdır. Seçim x argümanı etkilemez, bu yüzden y ile neler yapılabileceğine bakalım. Eğer rakibimiz $y = a$ seçerse, $i = 0$ alırız ve dize elde ederiz ki bu $L \setminus ((ab)^* a^*)$ içinde değildir. Eğer rakip $y = ab$ seçerse, yine $i = 0$ seçebiliriz. Şimdi dizeyi $(ab)^m a^m$ alıyoruz, bu da L içinde değildir. Aynı şekilde, rakibin herhangi bir olası seçimiyle başa çıkabiliriz, böylece

iddiamızı kanıtlamış oluruz.

ÖRNEK 4.11

Gösterin ki

$$L = \{a^n : n \text{ is a perfect square}\}$$

bu düzenli değildir.

Rakibin m seçimine göre, biz seçiyoruz

$$w = a^{m^2}.$$

Eğer $w = xyz$ ayrıştırması ise, o zaman açıkça

$$y = a^k$$

$1 \leq k \leq m$ ile. Bu durumda,

$$w_0 = a^{m^2 - k}$$

Ancak $m^2 - k > (m - 1)^2$, bu nedenle $w_0 \notin L$ içinde olamaz. Bu nedenle, bu dil düzenli değildir.

Bazı durumlarda, kapanış özellikleri, verilen bir problemizaten sınıflandırdığımız bir problemle ilişkilendirmek için kullanılabilir. Bu, doğrudan pompa lemmasının uygulanmasından daha basit olabilir.

ÖRNEK 4.12

Dilin gösterilmesi

$$L = \{a^n b^k c^{n+k} : n \geq 0, k \geq 0\}$$

düzenli değildir.

Pompa lemmasını doğrudan uygulamak zor değildir, ancak homomorfizm altında kapanışı kullanmak daha da kolaydır. Alın

$$h(a) = a, h(b) = a, h(c) = c,$$

o zaman

$$\begin{aligned} h(L) &= \{a^{n+k} c^{n+k} : n + k \geq 0\} \\ &= \{a^i c^i : i \geq 0\}. \end{aligned}$$

Ancak bu dilin düzenli olmadığını biliyoruz; bu nedenle, L de düzenli olamaz.

ÖRNEK 4.13

Dilin gösterilmesi

$$L = \{ \quad : n / l \}$$

düzenli değildir.

Burada pompa lemmasını doğrudan uygulamak için biraz yaratıcılığa ihtiyacımız var. $n = l + 1$ veya $n = l + 2$ olan bir dize seçmek yeterli olmayacaktır, çünkü rakibimiz her zaman dizeyi dilin dışına pompalamayı imkansız hale getirecek bir ayrıştırma seçebilir (yani, a'ların ve b'lerin eşit sayıda olduğu şekilde pompalamak). Daha dikkatli olmalıyız.

yaratıcı. Had $n=m!$ alalım ve $l = (m + 1)!$. Eğer rakip şimdi bir y (zorunlu olarak tüm a'lerden oluşan) uzunluğu $k < m$, biz $m!$ kez bir dize oluşturmak için $m! + (i-1)k$ a' s. Eğer i seçebilirsek, pompa lemmasının bir çelişkisinin elde edebiliriz.

$$m! + (i - 1)k \quad + 1)!$$

Bu her zaman mümkündür çünkü

$$i = 1 + \frac{m}{k}$$

ve $k \leq m$. Bu nedenle sağ taraf bir tam sayıdır ve pompa lema koşullarını ihlal etmeyi başardık.

Ancak, bu problemi çözmenin çok daha şık bir yolu vardır. Diyelim ki L düzenliydi. O zaman, Teorem 4.1'e göre, L ve dil

$$L_1 = \overline{L} \cap (a^*b^*)$$

de düzenli olurdu. Ancak $L_1 = \{a^n b^n : n \geq 0\}$, bu da zaten düzensiz olarak sınıflandırdığımız bir dildir. Sonuç olarak, L düzenli olamaz.

Pompala lema anlaması zor ve uygularken yanlış yola sapmak kolaydır. İşte bazı yaygın tuzaklar. Bunlara dikkat edin.

Bir hata, bir dilin düzenli olduğunu göstermek için pompa lemmasını kullanmaya çalışmaktır. Bir dildeki hiçbir dize L pompalama yapılamayacağını gösterebilirseniz bile, L 'nin düzenli olduğu sonucuna varamazsınız. Pompa lemması, bir dilin düzenli olmadığını kanıtlamak için yalnızca kullanılabilir.

Başka bir hata, (genellikle istemeden) bir dizi ile başlamaktır L . Örneğin, diyelim ki bunu göstermeye çalışıyoruz

$$L = \{a^n \mid n \text{ is a prime number}\} \quad (4.3)$$

düzenli değildir. "Verilen m , let $w = a^m \dots$," ile başlayan bir argüman yanlıştır çünkü m mutlaka asal değildir. Bu tuzaktan kaçınmak için, ihtiyacımız var

"Başlamak için şöyle bir şey: "Verilen m , let $w = a^M$, burada M m 'den büyük bir asal sayıdır."

Sonunda, belki de en yaygın hata, ayrıştırma hakkında bazı varsayımlarda bulunmaktır xyz . Ayrıştırma hakkında söyleyebileceğimiz tek şey, pompa lemmasının bize söylediğidir, yani y boş değildir ve $|xy| \leq m$; yani y , dizinin sol ucundan m sembolü içinde olmalıdır. Başka bir şey, argümanı geçersiz kılar. (4.3) numaralı denklemin dilinin düzenli olmadığını kanıtlamaya çalışırken tipik bir hata, $y = a^k$, burada k tek sayıdır demektir. O zaman elbette $w = xz$, çift uzunlukta bir dizedir ve dolayısıyla L içinde değildir. Ancak k üzerindeki varsayım izin verilmez ve kanıt yanlıştır.

Ancak pompa lemmasının teknik zorluklarını ustaca aşsanız bile, onu nasıl kullanacağınızı tam olarak görmek zor olabilir. Pompa lemması, karmaşık kurallara sahip bir oyun gibidir. Kuralların bilgisi esastır, ancak bu tek başına iyi bir oyun oynamak için yeterli değildir. Kazanmak için iyi bir stratejiye gereksinim vardır. Bu kitapta daha zor olan bazı durumlarda pompa lemmasını doğru bir şekilde uygulayabilerseniz, tebrik edilmelisiniz.

EGZERSİZLER

1. Dilin gösterilmesi

$$L = \{a^n b^k c^n : n \geq 0, k \geq 0\}$$

düzenli değildir.

2. Dilin gösterilmesi

$$L = \{a^n b^k c^n : n \geq 0, k \geq n\}$$

düzenli değildir.

3. Dilin gösterilmesi

$$L = \{a^n b^k c^n d^k : n \geq 0, k > n\}$$

düzenli değildir.

4. Dilin gösterilmesi

$$L = \{w : n_a(w) = n_b(w)\}$$

düzenli değildir. L^* düzenli mi?

5. Aşağıdaki dillerin düzenli olmadığını kanıtlayın:

$$(a) L = \{a^n b^l a^k : k \leq n + l\}.$$

$$(b) L = \{a^n b^l a^k : k \neq n + l\}.$$

$$(c) L = \{a^n b^l a^k : n=l \vee a^l \neq k\}.$$

- (d) $L = \{a^n b^l : n \geq l\}$.
- (e) $L = \{w : n_a(w) \neq n_b(w)\}$.
- (f) $L = \{ww : w \in \{a, b\}^*\}$.
- (g) $L = \{w^R w w w^R : w \in \{a, b\}^*\}$.

6. $\Sigma = \{a\}$ üzerinde aşağıdaki dillerin düzenli olup olmadığını belirleyin:

- (a) $L = \{a^n : n \geq 2, \text{ asal bir sayı}\}$.
- (b) $L = \{a^n : n \text{ asal bir sayı değildir}\}$.
- (c) $L = \{a^n : n = k^3 \text{ için bazı } k \geq 0\}$.
- (d) $L = \{a^n : n = 2^k \text{ için bazı } k \geq 0\}$.
- (e) $L = \{a^n : n \text{ iki asal sayının çarpımıdır}\}$.
- (f) $L = \{a^n : n \text{ ya asal ya da iki veya daha fazla asal sayıların çarpımıdır}\}$.
- (g) L^* , burada L (a) kısmındaki dildir.

7. Aşağıdaki dillerin düzenli olup olmadığını belirleyin:

- (a) $L = \{a^n b^n : n \geq 1\} \cup \{a^n b^m : n \geq 1, m \geq 1\}$.
- (b) $L = \{a^n b^n : n \geq 1\} \cup \{a^n b^{n+2} : n \geq 1\}$.

8. Dilin gösterilmesi

$$L = \{a^n b^n : n \geq 0\} \cup \{a^n b^{n+1} : n \geq 0\} \cup \{a^n b^{n+2} : n \geq 0\}$$

düzenli değildir.

9. Dilin gösterilmesi

$$L = \{a^n b^{n+k} : n \geq 0, k \geq 1\} \cup \{a^{n+k} b^n : n \geq 0, k \geq 3\}$$

düzenli değildir.

10. Dil $L = \{w \in \{a, b, c\}^* : |w| = 3 n_a(w)\}$ düzenli mi?

11. Dili göz önünde bulundurun

$$L = \{a^n : n \text{ is not a perfect square}\}.$$

(a) Bu dilin düzenli olmadığını pompalama

lemmasını doğrudan uygulayarak gösterin.

(b) Sonra düzenli dillerin kapama özelliklerini kullanarak aynı şeyi gösterin.

12. Dilin düzenli olmadığını

$$L = \{a^{n!} : n \geq 1\}$$

nı gösterin.

13. Pompala lema'sını doğrudan uygulayarak Örnek 4.12'deki sonucu gösterin.

14. Aşağıdaki pompalama lema'sının versiyonunu kanıtlayın. Eğer L düzenli ise, o zaman bir m vardır ki, her $w \in L$ uzunluğum m ' den büyük olan w , aşağıdaki gibi ayrıştırılabilir

$$w = xyz,$$

$|yz| \leq m$ ve $|y| \geq 1$ olacak şekilde, böylece xy^iz tümü için L içindedir.

15. Pompala lema'sının aşağıdaki genellemesini kanıtlayın, bu da Teorem 4.8'i ve Alıştırma 1'i özel durumlar olarak içerir.

If L düzenli ise, o zaman her biri için aşağıdakiler geçerlidir yeterince uzun $w \in L$ ve her birinin ayrıştırmaları $w = u_1 v u_2$, ile $u_1, u_2 \in \Sigma^*, |v| \geq m$. Orta dize v şu şekilde yazılabilir $v = xyz$, ile $|xy| \leq m, |y| \geq 1$, böylece $u_1 x y^i z u_2 \in L$ tümü $i = 0, 1, 2, \dots$

16. Şu dilin düzenli olmadığını gösterin:

$$L = \{a^n b^k : n > k\} \cup \{a^k : k \neq -1\}.$$

17. Aşağıdaki ifadeyi kanıtlayın veya çürütün: Eğer L_1 ve L_2 düzensiz diller ise, o zaman $L_1 \cup L_2$ de düzensizdir.

18. Aşağıdaki dilleri düşünün. Her biri için, düzenli olup olmadığına dair bir varsayımda bulunun. Sonra varsayımınızı kanıtlayın.

(a) $L = \{a^n b^l a^k : n + l + k > 5\}$

(b) $\{a^n b^l : n > 5, l > 3, k \leq \}$.

$= \{a^n b^l : n/l \text{ bir tam sayıdır}\}.$

(d) $L = \{a^n b^l : n + l \text{ asal bir sayıdır}\}.$

(e) $L = \{a^n b^l : n \leq l \leq 2n\}.$

(f) $L = \{a^n b^l : n \geq 100, l \leq 100\}.$

(g) $L = \{a^n b^l : |n - l| = 2\}$

19. Bu dil düzenli mi?

$$\{w_1 c w_2 : w_1, w_2 \in \{a, b\}^*, w \neq w_2\}.$$

20. Let L_1 ve L_2 düzenli diller olsun. Bu dil

$$L = \{w : w \in L_1, w^R \in L_2\}$$

kesinlikle düzenli mi?

21. Örnek 4.8'deki dile doğrudan güvercin yuvası argümanını uygulayın.

22. Aşağıdaki diller düzenli mi?

(a) $= \{u w w^R v : u, v, w \in \{a, b\}^*\}.$

$= \{u w w^R v : u, v, w \in \{a, b\}^*, |u| \geq |v|\}.$

23. Bu dil düzenli mi?

$$L = \{ww^Rv : v, w \in \{a, b\}^+\}.$$

24. Let P be an infinite but countable set, and associate with each $p \in P$ a language L_p . The smallest set containing every L_p is the union over the infinite set P ; it will be denoted by $\bigcup_{p \in P} L_p$. Show by example that the family of regular languages is not closed under infinite union.

25. Consider the argument in Section 3.2 that the language associated with any generalized transition graph is regular. The language associated with such a graph is

$$L = \bigcup_{p \in P} L(r_p),$$

where P is the set of all walks through the graph and r_p is the expression associated with a walk p . The set of walks is generally infinite, so that in light of Exercise 24 it does not immediately follow that L is regular. Show that in this case, because of the special nature of P , the infinite union is regular.

26. Is the family of regular languages closed under infinite intersection?

27. Suppose that we know that $L_1 \cup L_2$ and L_1 are regular. Can we conclude from this that L_2 is regular?

28. Egzersiz 27, Bölüm 3.1'deki zincir kodu dilinde, L , tüm $w \in \{u, r, l, d\}^*$ olan ve dikdörtgenleri tanımlayan küme olsun. L 'nin düzenli bir dil olmadığını gösterin.

29. Let $L = \{a^n b^m : n \geq 100, m \leq 50\}$.

(a) Pompalama lemmasını kullanarak L 'nin düzenli olduğunu gösterebilir misiniz?

(b) Pompalama lemmasını kullanarak L 'nin düzenli olmadığını gösterebilir misiniz?

Cevaplarınızı açıklayın.

30. Gramerin ürettiği dilin $S \rightarrow aSS|b$ düzenli olmadığını gösterin.

BÖLÜM

5

BAĞIMSIZ DİLLER

BÖLÜM ÖZETİ

Düzenli dillerin sınırlamalarını keşfettikten sonra, şimdi daha karmaşık dilleri incelemeye devam ediyoruz. Yeni bir gramer türü tanımlayarak, bağımsız gramerler ve ilişkili bağımsız diller üzerinde çalışıyoruz. Bağımsız gramerler hala oldukça basit olsalar da, düzenli olmadığını bildiğimiz bazı dillerle başa çıkabilirler. Bu, düzenli diller ailesinin, bağımsız diller ailesinin uygun bir alt kümesi olduğunu gösterir.

Burada karşılaşılan iki önemli kavram, üyelik sorusu ve ayrıştırma : Üyelik problemi (bir dilbilgisi G ve bir dize w verildiğinde, w 'nin $L(G)$ için de olup olmadığını bulmak) burada düzenli dillerden çok daha karmaşık tır. Ayrıştırma da öyledir; bu, yalnızca üyeliği değil, aynı zamanda w 'ye g ötüren belirli türetmeyi bulmayı da içerir. Bu konular bu bölümde yalnızca kısaca ele alınmaktadır, ancak sonraki bölümlerde çok daha derinlemesine incelenecektir. Burada yalnızca kaba kuvvet ayrıştırmasına bakıyoruz. Kaba kuvvet ayrıştırması çok genel bir yöntemdir; yani, dilbilgisinin biçiminden bağımsız olarak çalışır, ancak o kadar verimsizdir ki nadiren kullanılır.

Önemi esasen teoriktir, çünkü bağlamdan bağımsız dil ayrıştırmanın, en azından prensipte, her zaman mümkün olduğunu gösterir. Daha sonra daha verimli üyelik ve ayrıştırma algoritmalarını inceleyeceğiz.

Son bölümde, tüm dillerin düzenli olmadığını keşfettik. Düzenli diller belirli basit kalıpları tanımlamadı etkili olsa da, düzensiz dillerin örnekleri için çok uzağa bakmaya gerek yoktur. Bu sınırlamaların programlama dilleriyle olan ilgisi, bazı örnekleri yeniden yorumladığımızda belirgin hale gelir. Eğer $L = \{a^n b^n : n \geq 0\}$ içinde a için bir sol parantez ve b için bir sağ parantez koyarsak, o zaman $(())$ ve $((()))$ gibi parantez dizileri L içindedir, ancak $(())$ değil dir. Bu nedenle dil, programlama dillerinde bulunan basit bir tür iç içe yapı tanımlar ve bazı programlama dillerinin özelliklerinin düzenli dillerin ötesinde bir şeye ihtiyaç duyduğunu gösterir. Bunu ve diğer daha karmaşık özellikleri kapsamak için dil ailesini genişletmemiz gerekir. Bu, bizi bağlamdan bağımsız dilleri ve dilbilgilerini düşünmeye yönlendirir.

Bu bölüme, bağlamdan bağımsız gramerleri ve dilleri tanımlayarak başlıyoruz, tanımları bazı basit örneklerle açıklıyoruz. Sonra, önemli bir üyeliğin sorununu ele alıyoruz; özellikle, belirli bir dize ile belirli bir gramerin türetilip türetilmediğini nasıl anlayabileceğimizi soruyoruz. Bir cümleyi gramatik türetilmesiyle açıklamak, doğal dillerin incelenmesinden çoğumuz için tanıdık bir durumdur ve buna ayrıştırma denir. Ayrıştırma, cümle yapısını tanımlamanın bir yoludur. Bir cümlenin anlamını anlamamız gerektiğinde önemlidir; örneğin, bir dilden diğerine çeviri yaparken olduğu gibi. Bilgisayar biliminde, bu, yorumlayıcılar, derleyiciler ve diğer çeviri programları için geçerlidir.

Bağlamdan bağımsız diller konusu, belki de programlama dillerinin uygulandığında formel dil teorisinin en önemli yönüdür. Gerçek programlama dilleri, bağlamdan bağımsız diller aracılığıyla zarif bir şekilde tanımlanabilen birçok özelliğe sahiptir. Formül dil teorisinin bağlamdan bağımsız diller hakkında söyledikleri, programlama dillerinin tasarımı ve verimli derleyicilerin inşasında önemli uygulamalara sahiptir. Bunu Kısım 5.3'te kısaca ele alıyoruz.

5.1 BAĞIMSIZ KURALLAR

Bir düzenli dildeki üretimler iki şekilde kısıtlanmıştır: Sol taraf tek bir değişken olmalı, sağ taraf ise özel bir forma sahip olmalıdır.

Daha güçlü dilbilgileri oluşturmak için, bu kısıtlamalardan bazılarını gevşetmemiz gerekir. Sol taraftaki kısıtlamayı koruyarak, sağ tarafta herhangi bir şeye izin verdiğimizde, bağımsız dilbilgileri elde ederiz.

TANIM 5.1

Bir dilbilgisi $G = (V, T, S, P)$ bağımsız olarak adlandırılırsa, P içindeki tüm üretimlerin formu

$$A \rightarrow x,$$

şu şekildedir $A \in V$ ve $x \in (V \cup T)^*$.

Bir dil L , yalnızca $L = L(G)$ şeklinde bir bağlam-dışı dilbilgisi G varsa bağlam-dışı olarak kabul edilir.

Her düzenli dilbilgisi bağlam-dışıdır, bu nedenle düzenli bir dil de bir bağlam-dışı dildir. Ancak, $\{a^n b^n\}$ gibi basit örneklerden bildiğimiz gibi, düzensiz diller vardır. Bu dilin bir bağlam-dışı dilbilgisi tarafından üretilebileceğini Örnek 1.11'de zaten göstermiştik, bu nedenle düzenli diller ailesinin, bağlam-dışı diller ailesinin uygun bir alt kümesi olduğunu görüyoruz.

Bağlam-dışı dilbilgileri, bir üretimin solundaki değişkenin, bir cümle biçiminde herhangi bir zamanda görüldüğünde yer değiştirebilmesi gerçeğinden adını alır. Bu, cümle biçiminin geri kalanındaki sembollere (bağlama) bağlı değildir. Bu özellik, üretimin sol tarafında yalnızca bir değişkenin bulunmasına izin vermenin sonucudur.

Bağlamdan Bağımsız Dillerin Örnekleri

ÖRNEK 5.1

Üslup $G = (\{S\}, \{a, b\}, S, P)$, üretimlerle

$$S \rightarrow aSa,$$

$$S \rightarrow bSb,$$

$$S \rightarrow \lambda,$$

bağlamdan bağımsızdır. Bu gramerde tipik bir türetim

$$S \Rightarrow aSa \Rightarrow aaSaa \Rightarrow aabSbaa \Rightarrow aabbbaa.$$

Bu ve benzer türetimler,

$$L(G) = \{ww^R : w \in \{a, b\}^*\}.$$

Dilin bağlamdan bağımsız olduğunu, ancak Örnek 4.8'de gösterildiği gibi, düzenli olmadığını açıkça ortaya koymaktadır.

ÖRNEK 5.2

Üslup G , üretimlerle

$$S \rightarrow abB,$$

$$A \rightarrow aaBb,$$

$$B \rightarrow bbAa,$$

$$A \rightarrow \lambda,$$

bağlamdan bağımsızdır. Bunu göstermek okuyucuya bırakılmıştır.

$$L(G) = \{ab(bbaa)^n bba(ba)^n : n \geq 0\}.$$

Yukarıdaki her iki örnek de yalnızca bağlamdan bağımsız değil, aynı zamanda lineerdir. Düzenli ve lineer gramerler açıkça bağlamdan bağımsızdır, ancak bir bağlamdan bağımsız gramer mutlaka lineer değildir.

ÖRNEK 5.3

Dil

$$L = \{a^n b^m : n \neq m\}$$

bağlamdan bağımsızdır.

Bunu göstermek için, dil için bir bağlamdan bağımsız gramer üretmemiz gerekiyor. $n = m$ durumu Örnek 1.11'de çözülmüştür ve bu çözüm üzerine inşa edebiliriz. $n > m$ durumunu ele alalım. Öncelikle eşit sayıda a ve b içeren bir dize oluşturuyoruz, ardından sola ekstra a ekliyoruz.

Bunu şu şekilde yapıyoruz

$$S \rightarrow AS_1,$$

$$S_1 \rightarrow aS_1b|\lambda,$$

$$A \rightarrow aA|a.$$

$n < m$ durumu için benzer bir akıl yürütme kullanabiliriz ve cevabı alırız.

$$S \rightarrow AS_1|S_1B,$$

$$S_1 \rightarrow aS_1b|\lambda,$$

$$A \rightarrow aA|a,$$

$$B \rightarrow bB|b.$$

Ortaya çıkan gramer bağlamdan bağımsızdır, dolayısıyla L , bağlamdan bağımsız bir dildir. Ancak, gramer lineer değildir.

Burada verilen dilbilgisi biçimi, örnekleme amacıyla seçilmişti r; bu dil için birçok başka eşdeğer bağlamdan bağımsız dilbilgisi vardır. Aslında, bu dil için bazı basit lineer olanlar da mevcuttur. Bu bölümün sonunda, 26. Alıştırmada bunlardan birini bulmanız isteniyor.

ÖRNEK 5.4

Üretimlere sahip dilbilgisini düşünün

$$S \rightarrow aSb | SS | \lambda.$$

Bu, bağlamdan bağımsız olan ancak lineer olmayan başka bir dilbilgisidir. $L(G)$ için deki bazı dizeler abaabb, aababb ve ababab'dır. Bunun doğru olduğunu varsaymak ve kanıtlamak zor değildir.

$$L = \{w \in \{a, b\}^* : n_a(w) = n_b(w) \text{ and } n_a(v) \geq n_b(v), \text{ where } v \text{ is any prefix of } w\}. \quad (5.1)$$

Programlama dilleri ile bağlantıyı açıkça görebiliriz eğer a ve b 'yi sırasıyla sol ve sağ parantez ile değiştirirsek. L dili, $()$ ve $()()$ gibi dizeleri içerir ve aslında yaygın programlama dilleri için tüm düzgün iç içe geçmiş parantez yapılarının kümesidir.

Burada da birçok başka eşdeğer dilbilgisi vardır. Ancak, Örnek 5.3'e kıyasla, lineer olanların olup olmadığını görmek o kadar kolay değildir. Bu sorunun yanıtını verebilmemiz için 8. Bölümü beklememiz gerekecek.

En Soldaki ve En Sağdaki Türevler

Lineer olmayan bir dilbilgisinde, bir türev birden fazla değişken içeren cümle biçimlerini içerebilir. Bu tür durumlarda, değişkenlerin değiştirilme sırası konusunda bir seçim yapma hakkımız vardır. Örneğin, dilbilgisini ele alalım $G = (\{A, B, S\}, \{a, b\}, S, P)$ üretimlerle

1. $S \rightarrow AB.$
2. $A \rightarrow aaA.$
3. $A \rightarrow \lambda.$
4. $B \rightarrow Bb.$
5. $B \rightarrow \lambda.$

Bu dilin grameri $L(G) = \{a^{2n}b^m : n \geq 0, m \geq 0\}$.
 Bunu kendinize kanıtlamak için birkaç türetme yapın.
 Şimdi iki türetmeyi düşünün

$$S \xRightarrow{1} AB \xRightarrow{2} aaAB \xRightarrow{3} aaB \xRightarrow{4} aaBb \xRightarrow{5} aab$$

ve

$$S \xRightarrow{1} AB \xRightarrow{4} ABb \xRightarrow{2} aaABb \xRightarrow{5} aaAb \xRightarrow{3} aab.$$

Hangi üretimin uygulandığını göstermek için, üretimleri numaralandırdık ve uygun numarayı $\Rightarrow$ sembolüne yazdık. Buradan, iki türetmenin yalnızca aynı cümleyi üretmekle kalmayıp, aynı zamanda tam olarak aynı üretileri kullandığını görüyoruz. Fark tamamen üretimlerin uygulandığı sıradadır. Bu tür alakasız faktörleri ortadan kaldırmak için, genellikle de değişkenlerin belirli bir sırayla değiştirilmesini talep ederiz.

TANIM 5.2

Bir türetme, her adımda cümle biçimindeki en soldaki değişkenin değiştirilmesi durumunda en soldaki olarak adlandırılır. Her adımda en sağdaki değişken değiştirilirse, türetmeye en sağdaki denir.

ÖRNEK 5.5

Üretimlere sahip dilbilgisini düşünün

$$\begin{aligned} S &\rightarrow aAB, \\ A &\rightarrow bBb, \\ B &\rightarrow A|\lambda. \end{aligned}$$

Sonra

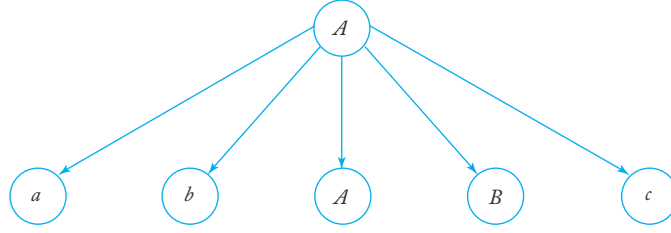
$$S \Rightarrow aAB \Rightarrow abBbB \Rightarrow abAbB \Rightarrow abbBbbB \Rightarrow abbbbB \Rightarrow abbbb$$

string $abbbb$ için en soldan türetimdir. Aynı stringin en sağdan türetimi ise

$$S \Rightarrow aAB \Rightarrow aA \Rightarrow abBb \Rightarrow abAb \Rightarrow abbBbb \Rightarrow abbbb.$$

Türetim Ağaçları

Türetimleri göstermenin ikinci bir yolu, üretimlerin kullanıldığı sıradan bağımsız olarak, bir türetim veya ayrıştırma ağacı ile gösterilmektedir. Bir türetim ağacı



ŞEKİL 5.1

Üretimlerin sol tarafları ile etiketlenmiş düğümlere sahip sıralı bir ağaç ve bir düğümün çocuklarının onun karşılık gelen sağ taraflarını temsil ettiği bir ağaç. Örneğin, Şekil 5.1, bir üretimi temsil eden bir türetim ağacının bir bölümünü göstermektedir.

$$A \rightarrow abABc.$$

Bir türetim ağacında, bir üretimin sol tarafında bulunan bir değişkenle etiketlenmiş bir düğümün çocukları, o üretimin sağ tarafındaki sembollerden oluşur. Başlangıç sembolü ile etiketlenmiş kök düğümden başlayarak ve yapraklar da sonlanan terminal sembollere kadar, bir türetim ağacı her değişkenin türetimde nasıl değiştirildiğini gösterir. Aşağıdaki tanım bu kavramı kesinleştirir.

TANIM 5.3

Let $G = (V, T, S, P)$ bir bağlamdan bağımsız bir dilbilgisi olsun. Sıralı bir ağaç, G için bir türetim ağacıdır eğer ve ancak aşağıdaki özelliklere sahipse.

1. Kök, S ile etiketlenmiştir.
2. Her yaprak, $T \cup \{\lambda\}$ kümesinden bir etikete sahiptir.
3. Her iç tepe (yaprak olmayan bir tepe) V kümesinden bir etiket taşır.
4. Eğer bir tepe $A \in V$ ile etiketlenmişse ve çocukları soldan sağa doğru $a_1, a_2, \dots, a_n$ ile etiketlenmişse, o zaman P , aşağıdaki biçimde bir üretim içermelidir

$$A \rightarrow a_1 a_2 \cdots a_n.$$

5. λ ile etiketlenmiş bir yaprak, kardeşi yoktur; yani, λ ile etiketlenmiş bir çocuğa sahip bir tepe başka çocuklara sahip olamaz.

Ağaç, 3, 4 ve 5 özelliklerine sahip, ancak 1'in mutlaka geçerli olmadığı ve 2 özelliğinin yerine

- 2a. Every leaf has a label from $V \cup T \cup \{\lambda\}$,

kısmi türetim ağacı olarak adlandırılır.

Ağacın yapraklarını soldan sağa okuyarak elde edilen semboller dizisi, karşılaşılan herhangi bir λ 'yi atlayarak, ağacın verimi olarak adlandırılır. Tanımlayıcı terim soldan sağa kesin bir anlam kazanabilir. Verim, ağacın derinlik öncelikli bir şekilde geçildiğinde karşılaşılan terminal dizisi dir; her zaman en soldaki keşfedilmemiş dal alınır.

ÖRNEK 5.6

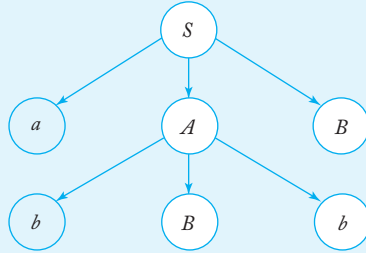
Üretimleri olan G dilini düşünün.

$$S \rightarrow aAB,$$

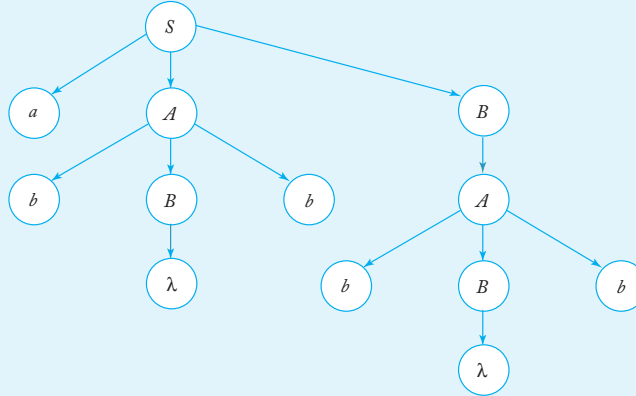
$$A \rightarrow bBb,$$

$$B \rightarrow A|\lambda.$$

Şekil 5.2'deki ağaç, G için bir kısmi türetim ağacıdır, Şekil 5.3'teki ağaç ise bir türetim ağacıdır. İlk ağacın verimi olan dizgi $abBbB, G'$ nin bir cümle biçimidir. İkinci ağacın verimi ise ağaç, $abbbb, L(G)$ cümlesidir.



ŞEKİL 5.2



ŞEKİL 5.3

Cümle Biçimleri ile Türetim Ağaçları Arasındaki İlişki

Türetim ağaçları, bir türetimin çok açık ve kolayca anlaşılabilir bir tanımı verir. Sonlu otomata için geçiş grafikleri gibi, bu açıklık, argümanlar oluştururken büyük bir yardımcıdır. Öncelikle, türetimler ile türetim ağaçları arasındaki bağlantıyı kuralıyız.

TEOREM 5.1

Bir $G = (V, T, S, P)$ bağlamdan bağımsız bir dilbilgisi olsun. O zaman her $w \in L(G)$ için n, G 'nin bir türetim ağacı vardır ve bu ağacın çıktısı w 'dir. Tersine, herhangi bir türetim ağacının çıktısı $L(G)$ içindedir. Ayrıca, eğer $t \in G$ için kökü S ile etiketlenmiş herhangi bir kısmi türetim ağacı ise, o zaman $t \in G$ 'nin çıktısı G 'nin bir cümle biçimidir.

Kanıt: Öncelikle, $L(G)$ içindeki her cümle biçimi için karşılık gelen bir kısmi türetim ağacının olduğunu gösteriyoruz. Bunu türetim adımlarının sayısına göre indüksiyonla yapıyoruz. Bir temel olarak, iddia edilen sonucun bir adımda türetilen her cümle biçimi için doğru olduğunu not ediyoruz. Çünkü $S \Rightarrow u$, bir türetim $S \rightarrow u$ olduğu anlamına gelir, bu da Tanım 5.3'ten hemen çıkar.

Her türetim adımında türetilen cümle biçimi için kısmi bir türetim ağacının olduğunu varsayalım. Şimdi $n + 1$ adımda türetilen herhangi bir w , şöyle olmalıdır

$$S \xRightarrow{*} xAy, \quad x, y \in (V \cup T)^*, \quad A \in V,$$

n adımda ve

$$xAy \Rightarrow xa_1a_2 \cdots a_my = w, \quad a_i \in V \cup T.$$

İndüktif varsayımına göre, çıktısı xAy olan bir kısmi türetim ağacı vardır ve dilbilgisinin $A \rightarrow a_1a_2 \cdots a_m$ üretimi olmalıdır, A ile etiketlenmiş yaprağı genişlettiğimizde, çıktısı $xa_1a_2 \cdots a_my = w$ olan bir kısmi türetim ağacı elde ederiz. Bu nedenle, indüksiyonla, sonucun tüm cümle biçimleri için doğru olduğunu iddia ediyoruz.

Benzer bir şekilde, her kısmi türetim ağacının bazı cümle biçimlerini temsil ettiğini gösterebiliriz. Bunu bir alıştırmaya bırakacağız.

Türetim ağacı, yaprakları terminal olan bir kısmi türetim ağacı olduğundan, $L(G)$ içindeki her cümle için G 'nin bir türetim ağacının çıktısı olduğu ve her türetim ağacının çıktısının $L(G)$ içinde olduğu sonucuna varılır.

Türetim ağaçları, bir cümle elde etmek için hangi üretimlerin kullanıldığını gösterir, ancak bunların uygulanma sırasını vermez. Türetim ağaçları, bu sıralamanın önemsiz olduğunu yansıtan herhangi bir türetimi temsil edebilir; bu gözlem, önceki tartışmadaki bir boşluğu kapatmamıza olanak tanır. Tanım gereği, herhangi bir $w \in L(G)$ bir türetime sahiptir, ancak biz henüz

sol en soldan veya sağ en soldan bir türetim olduğunu iddia etmedik. Ancak, bir türetim ağacına sahip olduğumuzda, ağacı sol en soldaki değişkenin her zaman önce genişletildiği şekilde inşa edilmiş olarak düşünerek her zaman sol en soldan bir türetim elde edebiliriz. Birkaç detayı doldurduğumuzda, herhangi bir $w \in L(G)$ için bir sol en soldan ve bir sağ en soldan türetim olduğu sonucuna ulaşırız (detaylar için bu bölümün sonunda yer alan Alıştırma 25'e bakınız).

EGZERSİZLER

1. Bağlamdan bağımsız gramerleri aşağıdaki diller için bulun:
 - (a) $L = a^n b^n, n$ çift.
 - (b) $L = a^n b^n, n$ tek.
 - (c) $L = a^n b^n, n$ üçün katıdır.
 - (d) $L = a^n b^n, n$ üçün katı değildir.
2. $w = aaabbaaa$ için Örnek 5.1'deki grameri kullanarak bir türetim ağacı gösterin.
3. Örnek 5.1'deki türetime karşılık gelen türetim ağacını çizin.
4. $w = abbbaabbaba$ için Örnek 5.2'deki gramer için bir türetim ağacı verin.
5. Örnek 5.2'deki argümanları tamamlayın, verilen dilin gramer tarafından üretildiğini gösterin.
6. Örnek 5.4'teki gramerin gerçekten 5.1. Denklemden tanımlanan dili ürettiğini gösterin.
7. Örnek 5.2'deki dil düzenli mi?
8. Teorem 5.1'deki kanıtı tamamlayarak, kökü S olan her kısmi türetim ağacının veriminin G 'nin bir cümle biçimi olduğunu gösterin.
9. Aşağıdaki diller için bağlamdan bağımsız gramerler bulun ($n \geq 0, m \geq 0$).
 - (a) $L = \{a^n b^m : n \leq m + 3\}$.
 - (b) $L = \{a^n b^m : n = m - 1\}$.
 - (c) $L = \{a^n b^m : n = 2m\}$.
 - (d) $L = \{a^n b^m : 2n \leq m \leq 3n\}$.
 - (e) $L = \{w \in \{a, b\}^* : n_a(w) = n_b(w)\}$.
 - (f) $L = \{w \in \{a, b\}^* : n_a(w) \geq n_b(w), \text{ where } w \text{ is any prefix of } w\}$.
 - (g) $L = \{w \in \{a, b\}^* : n_a(w) = 2n_b(w) + 1\}$.
 - (h) $L = \{w \in \{a, b\}^* : n_a(w) = n_b(w) + 2\}$.

10. Gramer hangi dili üretir

$$S \rightarrow aSB|bSA$$

$$S \rightarrow \lambda$$

$$A \rightarrow a$$

$$B \rightarrow b$$

?

11. Gramer hangi dili üretir

$$S \rightarrow aaSbb|SS$$

$$S \rightarrow \lambda$$

12. Aşağıdaki diller için bağlamdan bağımsız gramerler bulun ($n \geq 0, m \geq 0, k \geq 0$):

$$(a) L = \{a^n b^m c^k : n = m \text{ or } m \leq k\}.$$

$$(b) L = \{a^n b^m c^k : n = m \text{ or } m \neq k\}.$$

$$(c) L = \{a^n b^m c^k, k = n + m\}.$$

$$(d) L = \{a^n b^m c^k, k = |n - m|\}.$$

$$(e) L = \{a^n b^m c^k, k = n + 2m\}.$$

$$(f) \quad , k \neq n +$$

13. Find a context-free grammar for

$$L = \{w \in \{[a, b, c]^* . n_a(w) + n_b(w) \neq n_c(w)\}.$$

14. Show that $L = \{w \in \{a, b, c\}^* : |w| = 3n_a(w)\}$ is a context-free language.15. Find a context-free grammar for $\Sigma = \{a, b\}$ for the language

$$L = \{a^n w w^R b^n \mid (w \in \Sigma^*, n \geq 1)\}.$$

16. Let $L = \{a^n b^n : n \geq 0\}$.(a) Show that L^2 is context-free.(b) Gösterin ki L^k tüm $k > 1$ için bağlamdan bağımsızdır.(c) L ve L^* bağlamdan bağımsız olduğunu gösterin.17. Let L_1 , Alıştırma 12(a)'daki dil ve L_2 , Alıştırma 12(d)'deki dil olsun. $L_1 \cup L_2$ 'nin bağlamdan bağımsız bir dil olduğunu gösterin.

18. Aşağıdaki dilin bağlamdan bağımsız olduğunu gösterin:

$$L = \{uvw v^R : u, v, w \in \{a, b\}^*, |u| = |w| = 2\}.$$

19. Örnek 5.1'deki dilin tamamlayıcısının bağlamdan bağımsız olduğunu gösterin.

20. Alıştırma 12(c)'deki dilin tamamlayıcısının bağlamdan bağımsız olduğunu gösterin.

21. $Dil L = \{ w_1 c w_2 : w_1, w_2 \in \{ a, b \}^+, w_1 / = w_2^R \}$, $\Sigma = \{ a, b, c \}$, bağlamdan bağımsızdır.

22. Gramer ile dize $aabbbb$ için bir türetim ağacı gösterin.

$$S \rightarrow AB|\lambda,$$

$$A \rightarrow aB,$$

$$B \rightarrow Sb.$$

Bu gramer tarafından üretilen dilin sözlü tanımını verin.

23. Üretimlere sahip dilbilgisini düşünün

$$S \rightarrow aaB,$$

$$A \rightarrow bBb|\lambda,$$

$$B \rightarrow Aa.$$

Dize $aabbabba$ nın bu gramer tarafından üretilen dilde olmadığını gösterin.

24. İki tür parantez içeren düzgün yerleştirilmiş parantez yapılarının ne anlama gelebileceğini tanımlayın, diyelim ki $()$ ve $[\]$. Bu durumda, düzgün yerleştirilmiş dizelerin tuitif olarak $([\])$, $([[\]])$ $[(\)]$, ancak $(())$ veya $([])$ değildir. Tanımınızı kullanarak, tüm düzgün yerleştirilmiş parantezleri üreten bir bağlamdan bağımsız gramer verin.

25. Alfabe $\{a, b\}$ üzerindeki tüm düzenli ifadelerin kümesi için bir bağlamdan bağımsız gramer bulun.

26. Bağlamdan bağımsız bir dilbilgisi bulun ki, bu dilbilgisi tüm üretim kurallarını üretebilsin bağlamdan bağımsız dilbilgileri için $T = \{a, b\}$ ve $V = \{A, B, C\}$.

27. G bir bağlamdan bağımsız dilbilgisi ise, her $w \in L(G)$ için en soldan ve en sağdan türetimlerin olduğunu kanıtlayın. Böyle türetimleri bir türetim ağacından bulmak için bir algoritma verin.

28. Örnek 5.3 için dilin lineer bir dilbilgisini bulun.

29. $G = (V, T, S, P)$ olsun, her bir üretiminin $A \rightarrow v$ biçiminde olduğu bir bağlamdan bağımsız dilbilgisi olsun, burada $|v| = k > 1$. Herhangi bir $w \in L(G)$ için türetim ağacının yüksekliğinin h olduğunu gösterin.

$$\log_k |w| \leq h \leq \frac{(|w| - 1)}{k - 1}.$$

5.2 PARSE ETME VE BELİRSİZLİK

Şu ana kadar dilbilgilerinin üretken yönlerine odaklandık. Bir dilbilgisi G verildiğinde, bu dilbilgisi kullanılarak türetilen dizelerin kümesini inceledik. Pratik uygulama durumlarında, dilbilgisinin analitik yönüyle de ilgileniyoruz: Bir terminal dizisi w verildiğinde, bunun olup olmadığını bilmek istiyoruz.

veya değil $w \in L(G)$ içinde mi. Eğer öyleyse, w için bir türetim bulmak isteyebiliriz. Bize w 'ün $L(G)$ içinde olup olmadığını söyleyebilen bir algoritma, bir üyelik algoritmasıdır. Terim ayrıştırma, bir dizi üretim bulmayı tanımlar; böylece bir $w \in L(G)$ türetilir.

Ayrıştırma ve Üyelik

Bir dize $w \in L(G)$ içinde verildiğinde, onu oldukça belirgin bir şekilde ayrıştırabiliriz: Sistematik olarak tüm olası (örneğin, en soldaki) türetimleri oluşturuyoruz ve bunların herhangi birinin w ile eşleşip eşleşmediğine bakıyoruz. Özellikle, birinci turda, tüm üretilere bakarak başlıyoruz

$$S \rightarrow x,$$

bir adımda S 'den türetililecek tüm x 'leri buluyoruz. Eğer bunların hiçbirinin w ile eşleşmiyorsa, bir sonraki tura geçiyoruz; bu turda, her x 'nin en soldaki değişkenine uygun tüm üretimleri uyguluyoruz. Bu, bize bir dizi cümle biçimi verir; bunların bazıları belki de w 'ye yol açabilir. Her sonraki turda, yine tüm en soldaki değişkenleri alıyoruz ve bütün olası üretimleri uyguluyoruz. Bu cümle biçimlerinden bazıları, w 'nin asla türetilmeyeceği gerekçesiyle reddedilebilir, ancak genel olarak, her turda bir dizi olası cümle biçimimiz olacaktır. İlk turdan sonra, tek bir üretim uygulayarak türetililebilen cümle biçimlerine sahibiz; ikinci turdan sonra, daha iki adımda türetililebilen cümle biçimlerine sahibiz ve bu şekilde devam eder. Eğer $w \in L(G)$ ise, o zaman sonlu uzunlukta bir en soldaki türetimi olmalıdır. Böylece, yöntem sonunda w 'nin en soldaki türetimi ini verecektir.

Aşağıda referans olarak, buna kapsamlı arama ayrıştırması veya zorlayıcı ayrıştırma diyeceğiz. Bu, bir yukarıdan aşağıya ayrıştırma biçimidir ve bunu kökten aşağıya doğru bir türetim ağacının inşası olarak görebiliriz.

ÖRNEK 5.7

Grameri dikkate al

$$S \rightarrow SS \mid aSb \mid bSa \mid \lambda$$

ve dizi $w = aabb$. Birinci tur bize verir

1. $S \Rightarrow SS$,
2. $S \Rightarrow aSb$,
3. $S \Rightarrow bSa$,
4. $S \Rightarrow \lambda$.

Bu son iki madde, bariz nedenlerden ötürü daha fazla değerlendirmeden çıkarılabilir. İkinci tur, cümle biçimlerini verir.

$$S \Rightarrow SS \Rightarrow SSS,$$

$$S \Rightarrow SS \Rightarrow aSbS,$$

$$S \Rightarrow SS \Rightarrow bSaS,$$

$$S \Rightarrow SS \Rightarrow S,$$

cümle biçim 1'deki en soldaki S yerine tüm geçerli yer değiştirmeleri koyarak elde edilenlerdir. Benzer şekilde, cümle biçim 2'den de cümle biçimleri elde ederiz.

$$S \Rightarrow aSb \Rightarrow aSSb,$$

$$S \Rightarrow aSb \Rightarrow aaSbb,$$

$$S \Rightarrow aSb \Rightarrow abSab,$$

$$S \Rightarrow aSb \Rightarrow ab.$$

Yine, bunların birçoğu yarışmadan çıkarılabilir. Bir sonraki turda, diziden gerçek hedef dizesini buluyoruz.

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb.$$

Bu nedenle, $aabb$, dikkate alınan gramer tarafından üretilen dilde bulunmaktadır.

Kapsamlı arama ayrıştırmasının ciddi kusurları vardır. En bariz olanı sıkıcılığıdır; verimli ayrıştırmanın gerekli olduğu yerlerde kullanılmamalıdır. Ancak verimlilik ikincil bir mesele olsa bile, daha önemli bir itiraz vardır. Yöntem her zaman bir $w \in L(G)$ ayrıştırırsa da, $L(G)$ içinde olmayan dizgeler için asla sonlanmaması mümkündür. Bu, kesinlikle önceki örnekte olduğu gibidir; $w = abb$ ile, yöntem durdurmak için bir yol eklemediğimiz sürece deneme cümle biçimlerini sonsuza kadar üretmeye devam edecektir.

Tam arama ayrıştırmasının sonlanmama problemi, dilbilgisinin alabileceği biçimi kısıtladığımızda nispeten kolay bir şekilde aşılabılır. Örnek 5.7'yı incelediğimizde, zorluğun üretimlerden kaynaklandığını görüyoruz. $S \rightarrow \lambda$; bu üretim, ardışık cümle biçimlerinin uzunluğunu azaltmak için kullanılabilir, böylece ne zaman duracağımızı kolayca anlayamayız. Eğer böyle bir üretimimiz yoksa, o zaman çok daha az zorlukla karşılaşırız. Aslında, dışlamak istediğimiz iki tür üretim vardır, bunlar şu şekildedir: $A \rightarrow \lambda$ ve $A \rightarrow B$ biçimindeki ifadeler. Bir sonraki bölümde göreceğimiz gibi, bu kısıtlama elde edilen gramerlerin

güçlerini önemli bir şekilde etkilemez.

ÖRNEK 5.8

Gramer

$$S \rightarrow SS \mid aSb \mid bSa \mid ab \mid ba$$

verilen gereksinimleri karşılar. Boş dize olmadan Örnek 5.7'deki dili üretir.

Verilen herhangi bir $w \in \{a, b\}^+$, kapsamlı arama ayrıştırma yöntemi her zaman $|w|$ turda sona erecektir. Bu açıktır çünkü cümle biçiminin uzunluğu her turda en az bir sembol artar. $|w|$ turdan sonra ya bir ayrıştırma üretmişizdir ya da $w \notin L(G)$ olduğunu biliyoruz.

Bu örnekteki fikir genelleştirilebilir ve genel olarak bağlamdan bağımsız diller için bir teorem haline getirilebilir.

TEOREM 5.2

Varsayalım ki $G = (V, T, S, P)$ bağlamdan bağımsız bir dilbilgisi olup herhangi bir kuralı yoktur

$$A \rightarrow \lambda,$$

veya

$$A \rightarrow B,$$

burada $A, B \in V$. O zaman kapsamlı arama ayrıştırma yöntemi, herhangi bir $w \in \Sigma^*$ için ya bir ayrıştırma üretir ya da w için ayrıştırmanın mümkün olmadığını bize söyler.

Kanıt: Her cümle biçimi için, hem uzunluğunu hem de terminal sembollerin sayısını dikkate alın. Türetme sürecindeki her adım, bunlardan en az birini artırır. Bir cümle biçiminin uzunluğu veya terminal sembollerin sayısı $|w|$ 'yi aşamayacağından, bir türetme daha fazla içeremez $2|w|$ turlar, bu sırada ya başarılı bir ayrıştırmamız var ya da w olamaz gramer tarafından oluşturulmalıdır. ■

Tam arama yöntemi, çözümlemenin her zaman yapılabileceğine dair teorik bir garanti verse de, pratikteki kullanışlılığı sınırlıdır çünkü ürettiği cümle biçimlerinin sayısı aşırı derecede büyük olabilir. Üretilen cümle biçimlerinin sayısı durumdan duruma değişir; kesin bir genel sonuç belirlenemez, ancak bazı kaba üst sınırlar koyabiliriz.

Üzerinde. Eğer kendimizi en soldaki türetmelerle sınırlarsak, bir turdan sonra en fazla $|P|$ cümle biçimimiz olabilir, ikinci turdan sonra en fazla $|P|^2$ cümle biçimimiz olabilir ve bu şekilde devam eder. Teorem 5.2'nin kanıtında, ayrıştırmanın $2|w|$ turdan fazla olamayacağını gözlemledik; bu nedenle, toplam cümle biçimlerinin sayısı aşamaz.

$$\begin{aligned} M &= |P| + |P|^2 + \dots + |P|^{2|w|} \\ &= O(P^{2|w|+1}). \end{aligned} \quad (5.2)$$

Bu, kapsamlı arama ayrıştırmasının işinin, dizinin uzunluğuyla birlikte üstel olarak büyüebileceğini gösteriyor ve bu yöntemin maliyetini karşılanamaz hale getiriyor. Elbette, Denklem (5.2) sadece bir sınırdır ve genellikle cümle biçimlerinin sayısı çok daha küçüktür. Yine de, pratik gözlemler, kapsamlı arama ayrıştırmasının çoğu durumda çok verimsiz olduğunu göstermektedir.

Bağlamdan bağımsız gramerler için daha verimli ayrıştırma yöntemlerinin inşası, derleyicilerle ilgili bir dersin konusunu oluşturan karmaşık bir meseledir. Bunu burada yalnızca bazı izole sonuçlar dışında takip etmeyeceğiz.

TEOREM 5.3

Her bağlamdan bağımsız gramer için, herhangi bir $w \in L(G)$ ayrıştıran ve adımların sayısı $|w|^3$ ile orantılı olan bir algoritma vardır.

Bunu başarmak için birkaç bilinen yöntem vardır, ancak bunların hepsi yeterince karmaşıktır ki, bazı ek sonuçlar geliştirmeden bile tanımlayamayız. Bölüm 6.3'te bu soruyu tekrar kısaca ele alacağız. Daha fazla ayrıntı Harrison 1978 ve Hopcroft ve Ullman 1979'da bulunabilir. Bunu detaylı bir şekilde takip etmemek için bir neden, bu algoritmaların bile tatmin edici olmamasıdır. Dizenin uzunluğunun üçüncü kuvveti ile artan bir iş yükü olan bir yöntem, üstel bir algoritmadan daha iyi olsa da, yine de oldukça verimsizdir ve buna dayanan bir ayrıştırıcı, orta uzunlukta bir programı analiz etmek için aşırı miktarda zamana ihtiyaç duyacaktır. İstedığımız şey, dizinin uzunluğuna orantılı bir zaman alan bir ayrıştırma yöntemidir. Böyle bir yöntemle lineer zaman ayrıştırma algoritması diyoruz. Genel olarak bağlamdan bağımsız diller için herhangi bir lineer zaman ayrıştırma yöntemi bilmiyoruz, ancak kısıtlı, ancak önemli, özel durumlar için böyle algoritmalar bulunabilir.

DEFINITION 5.4

Bağlamdan bağımsız bir dilbilgisi $G = (V, T, S, P)$ basit bir dilbilgisi veya s-dilbilgisi olarak adlandırılır eğer tüm üretimleri aşağıdaki biçimdedir

$$A \rightarrow ax,$$

burada $A \in V, a \in T, x \in V^*$, ve herhangi bir çift (A, a) en fazla bir kez P içinde yer alır.

EXAMPLE 5.9

Gramer

$$S \rightarrow aS | bSS | c$$

bir s-dilbilgisidir. Bu dilbilgisi

$$S \rightarrow aS | bSS | aSS | c$$

bir s-dilbilgisi değildir çünkü çift (S, a) iki üretimde yer alır:
 $S \rightarrow aS$ ve $S \rightarrow aSS$.

S-dilbilgileri oldukça kısıtlayıcı olmasına rağmen, bazı ilgi alanlarına sahiptir. Önümüzdeki bölümde göreceğimiz gibi, yaygın programlama dillerinin birçok özelliği s-dilbilgileri ile tanımlanabilir.

Eğer G bir s-dilbilgisi ise, o zaman $L(G)$ içindeki herhangi bir dize w , çaba olarak $|w|$ ile orantılı bir şekilde ayrıştırılabilir. Bunu görmek için, kapsamlı arama yöntemine ve dize $w = a_1 a_2 \cdots a_n$ bakın. Solda en fazla bir kural olabileceğinden ve sağda a_1 ile başladığından, türetim

$$S \Rightarrow a_1 A_1 A_2 \cdots A_m.$$

Sonra, değişken için yerine koyuyoruz A_1 , ancak yine de en fazla bir

seçenek olduğundan, sahip olmalıyız

$$S \Rightarrow^* a_1 a_2 B_1 B_2 \cdots A_2 \cdots A_m.$$

Bundan, her adımın bir terminal sembolü ürettiğini ve dolayısıyla tüm sürecin en fazla $|w|$ adımda tamamlanması gerektiğini görüyoruz.

Gramerler ve Dillerde Belirsizlik

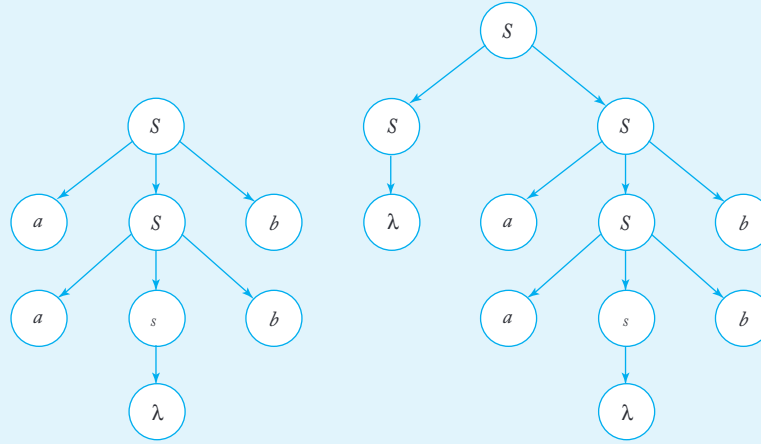
Argümanımız temelinde, herhangi bir $w \in L(G)$ verildiğinde, kapsamlı arama ayrıştırması w için bir türetim ağacı üretecektir. “Bir” türetim ağacı dememizin sebebi “tek” türetim ağacı değil, birden fazla farklı türetim ağacının var olma olasılığıdır. Bu duruma belirsizlik denir.

TANIM 5.5

Bir bağlamdan bağımsız gramer G , en az iki farklı türetim ağacına sahip olan bazı $w \in L(G)$ varsa belirsiz olarak adlandırılır. Alternatif olarak, belirsizlik, iki veya daha fazla en soldaki veya en sağdaki türetimlerin varlığını ima eder.

ÖRNEK 5.10

Örnek 5.4'teki gramer, üretimlerle $S \rightarrow aSb \mid SS \mid \lambda$, belirsizdir. Cümle $aabb$, Şekil 5.4'te gösterilen iki türetim ağacına sahiptir.



ŞEKİL 5.4

Belirsizlik, doğal dillerin yaygın bir özelliğidir; burada çeşitli şekillerde tolere edilir ve ele alınır. Programlama dillerinde, her ifadenin yalnızca bir yorumu olması gerektiğinden, belirsizlik mümkün olduğunda ortadan kaldırılmalıdır. Genellikle bunu, dilbilgisini eşdeğer, belirsiz olmayan bir biçimde yeniden yazarak başarabiliriz.

ÖRNEK 5.11

Dilbilgisi $G = (V, T, E, P)$ ile düşünün

$$V = \{E, I\},$$

$$T = \{a, b, c, +, *, (,)\},$$

ve üretimler

$$E \rightarrow I,$$

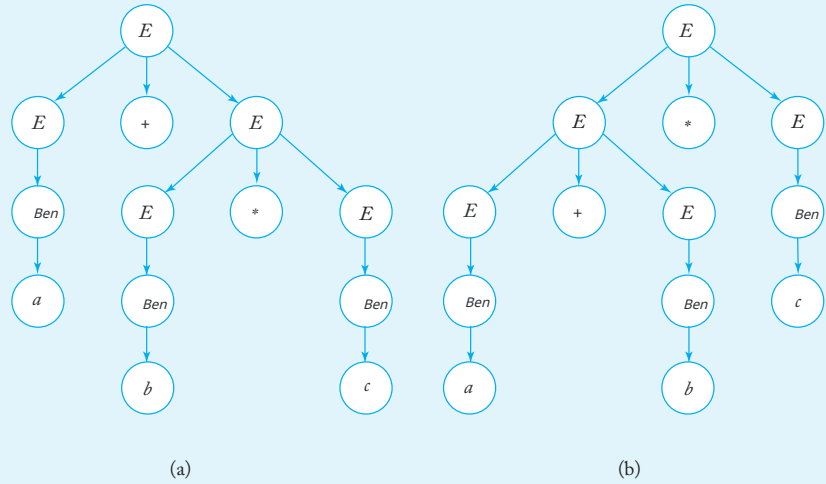
$$E \rightarrow E + E,$$

$$E \rightarrow E * E,$$

$$E \rightarrow (E),$$

$$I \rightarrow a \mid b \mid c.$$

Dizgeler $(a+b)*c$ ve $a*b+c$, $L(G)$ içindedir. Bu dilbilgisinin, C benzeri programlama dilleri için kısıtlı bir aritmetik ifadeler alt kümesi ürettiğini görmek kolaydır. Dilbilgisi belirsizdir. Örneğin, dizge $a+b*c$, Şekil 5.5'te gösterildiği gibi iki farklı türetim ağacına sahiptir.



ŞEKİL 5.5 $a+b*c$ için iki türetim ağacı.

Belirsizliği çözmenin bir yolu, programlama kılavuzlarında olduğu gibi, + ve * operatörleri ile öncelik kurallarını ilişkilendirmektir. Çünkü * normalde +'dan daha yüksek önceliğe sahiptir, Şekil 5.5(a)'yı doğru ayrıştırma olarak alırız, çünkü bu, $b*c$ 'nin toplama işlemi yapılmadan önce değerlendirilmesi gereken bir alt ifadenin olduğunu gösterir. Ancak, bu çözüm tamamen dilbilgisi dışıdır. Sadece bir ayrıştırmanın mümkün olacağı şekilde dilbilgisini yeniden yazmak daha iyidir.

ÖRNEK 5.12

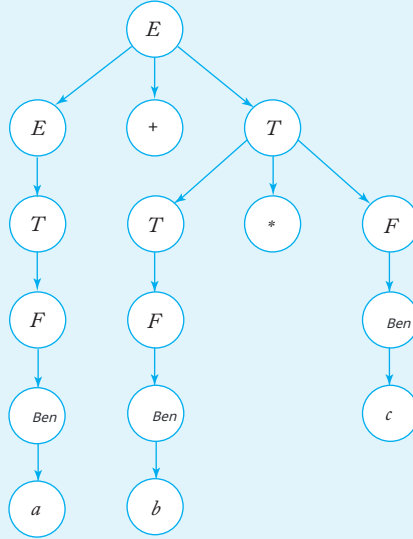
Örnek 5.11'deki dilbilgisini yeniden yazmak için yeni değişkenler tanıtıyoruz, V 'yi $\{E, T, F, I\}$ olarak alıyor ve üretimleri

$$\begin{aligned} E &\rightarrow T, \\ T &\rightarrow F, \\ F &\rightarrow I, \\ E &\rightarrow E + T, \\ T &\rightarrow T * F, \end{aligned}$$

$$F \rightarrow (E),$$

$$I \rightarrow a | b | c.$$

Bir cümlenin türetim ağacı $a+b*c$ Şekil 5.6'da gösterilmiştir. Bu dize için başka bir türetim ağacı mümkün değildir: Dilbilgisi belirsiz değildir. Ayrıca, Örnek 5.11'deki dilbilgisi ile eşdeğerdir. Bu belirli durumda bu iddiaları haklı çıkarmak çok zor değil, ancak, genel olarak, belirli bir bağlamdan bağımsız dilbilgisinin belirsiz olup olmadığı veya iki belirli bağlamdan bağımsız dilbilgisinin eşdeğer olup olmadığı soruları çok zor yanıtlanır. Aslında, daha sonra bu soruların her zaman çözülebileceği genel algoritmaların olmadığını göstereceğiz.



ŞEKİL 5.6

Önceki örnekte belirsizlik, eşdeğer belirsiz olmayan bir dilbilgisi b ulmakla ortadan kaldırılabilirdi anlamında dilbilgisinden kaynaklanı yordu. Ancak bazı durumlarda, bu mümkün değildir çünkü belirsizlik dilin içindedir.

TANIM 5.6

Eğer L , belirsiz bir gramerin var olduğu bir bağlamdan bağımsız bir dil i se, o zaman L 'nin belirsiz olduğu söylenir. Eğer L 'yi üreten her gramer b elirsizse, o zaman dilin kendiliğinden belirsiz olduğu söylenir.

Kendiliğinden belirsiz bir dili sergilemek bile oldukça zor bir meseledir. Burada yapabileceğimiz en iyi şey, kendiliğinden belirsiz olduğu konusunda makul bir iddia ile bir örnek vermektir.

ÖRNEK 5.13

Dil

$$L = \{a^n b^n c^m\} \cup \{a^n b^m c^m\},$$

n ve m pozitif olmayan ile, doğası gereği belirsiz bir bağlamdan bağımsız dildir.

L 'nin bağlamdan bağımsız olduğu kolayca gösterilebilir. Dikkat edin ki

$$L = L_1 \cup L_2,$$

L_1 'nin

$$\begin{aligned} S_1 &\rightarrow S_1 c | A, \\ A &\rightarrow a A b | \lambda \end{aligned}$$

ve L_2 'nin başlangıç sembolü S_2 olan benzer bir gramerle verildiği üretimlerdir

$$\begin{aligned} S_2 &\rightarrow a S_2 | B, \\ B &\rightarrow b B c | \lambda. \end{aligned}$$

O zaman L , bu iki gramere ek üretimle birlikte üretilir

$$S \rightarrow S_1 | S_2.$$

Gramer belirsizdir çünkü dize $a^n b^n c^n$ iki farklı üretim, biri $S \Rightarrow S_1$ ile, diğeri $S \Rightarrow S_2$ ile başlar. Bu Elbette, bu durumun L 'nin doğası gereği belirsiz olduğu anlamına gelmediği, çünkü onun için başka belirsiz olmayan gramerler mevcut olabilir. Ama bir şekilde L_1 ve L_2 'nin çelişkili gereksinimleri var; ilki a 'ların ve b 'lerin sayısına kısıtlama getirirken, ikincisi b 'ler için aynı şeyi yapıyor. ve c 'ler. Birkaç deneme, bu gereksinimleri tek bir kural setinde birleştirmenin imkansız olduğunu hızlıca kanıtlayacaktır. $n = m$ 'yi benzersiz bir şekilde kapsayan. Ancak, titiz bir argüman oldukça teknik. Bir kanıt Harrison 1978'de bulunabilir.

EGZERSİZLER

1. Gösterin ki

$$S \rightarrow aS|bS|cA$$

$$A \rightarrow aA|bS$$

bir s-gramerdir.

2. Her s-gramerin belirsiz olmadığını gösterin.
3. Bir s-gramer bulun $L(aaa^*b+ab^*)$.
4. Bir s-gramer bulun $L=\{a^n b^n : n \geq 2\}$.
5. Bir s-gramer bulun $L=\{a^n b^{2n} : n \geq 2\}$.
6. Bir s-gramer bulun $L=\{a^n b^{n+1} : n \geq 1\}$.
7. $G=(V, T, S, P)$ bir s-gramer olsun. P'nin maksimum boyutu için bir ifade verin $|V|$ ve $|T|$.
8. Aşağıdaki gramerin belirsiz olduğunu gösterin:

$$S \rightarrow AB|aaaB,$$

$$A \rightarrow a|Aa,$$

$$B \rightarrow b.$$

9. Egzersiz 8'deki gramerle eşdeğer belirsiz olmayan bir gramer oluşturun.
10. $((a+b)^*c+d)$ için türetim ağacını verin, Örnek 5.12'deki grameri kullanarak. $a^*b+((c+d))$ için türetim ağacını verin, Örnek 5.12'deki grameri kullanarak. $((a+b)^*c) + a+b$ için türetim ağacını verin, Örnek 5.12'deki grameri kullanarak.
13. Bir düzenli dilin doğası gereği belirsiz olamayacağını gösterin.
14. $\Sigma=\{a, b\}$ üzerinde tüm düzenli ifadeleri üreten belirsiz olmayan bir gramer verin.
15. Bir düzenli gramerin belirsiz olması mümkün mü?
- Dil $L = \{ww^R : w \in \{a, b\}^*\}$ doğası gereği belirsiz değildir.

17. Aşağıdaki gramerin belirsiz olduğunu gösterin:

$$S \rightarrow aSbS|bSaS|\lambda.$$

18. Örnek 5.4'teki gramerin belirsiz olduğunu, ancak onun tarafından gösterilen dilin belirsiz olmadığını gösterin.
19. Örnek 1.13'teki gramerin belirsiz olduğunu gösterin.
20. Örnek 5.5'teki dilbilgisinin belirsiz olmadığını gösterin.

21. Örnek 5.5'teki dilbilgisi ile $abbbbbb$ dizisini ayrıştırmak için kapsamlı arama ayrıştırma yöntemini kullanın. Genel olarak, bu dilde herhangi bir w dizisini ayrıştırmak için kaç tane gerekecektir?
22. Dizi $aabbababb$ dilbilgisi tarafından üretilen dilde midir $S \rightarrow aSS|b$?
23. Örnek 1.14'teki dilbilgisinin belirsiz olmadığını gösterin.
24. Aşağıdaki sonucu kanıtlayın. $G = (V, T, S, P)$ her $A \in V$ 'nin en fazla bir üretimin sol tarafında yer aldığı bir bağlamdan bağımsız dilbilgisi olsun. O zaman G belirsizdir.
25. Teorem 5.2'nin koşullarını sağlayan, Örnek 5.5'teki ile eşdeğer bir dilbilgisi bulun.

5.3 KONTEKST-ÜCRETLİ DİLLER VE PROGRAMLAR DİLLERİ

Resmi diller teorisinin en önemli kullanımlarından biri, programlama dillerinin tanımında ve bunlar için yorumlayıcılar ve derleyiciler inşa etmede yer almaktadır. Buradaki temel sorun, bir programlama dilini kesin bir şekilde tanımlamak ve bu tanıma, verimli ve güvenilir çeviri programları yazmanın başlangıç noktası olarak kullanmaktır. Hem düzenli hem de konteks-ücretsiz diller, bunu başarmada önemlidir. Gördüğümüz gibi, düzenli diller, programlama dillerinde meydana gelen belirli basit kalıpların tanınmasında kullanılır, ancak bu bölümün girişinde tartıştığımız gibi, daha karmaşık yönleri modellemek için konteks-ücretsiz dillere ihtiyacımız var.

Diğer birçok dilde olduğu gibi, bir programlama dilini bir dil bilgisi ile tanımlayabiliriz. Programlama dilleri üzerine yazarken, dil bilgilerini belirtmek için kullanılan bir gelenek olan *Backus-Naur* formu veya BNF kullanmak gelenekseldir. Bu form esasen burada kullandığımız notasyona benzer, ancak görünümü farklıdır. BNF'de, değişkenler üçgen parantezler içinde yer alır. Terminal semboller herhangi bir özel işaret olmadan yazılır. BNF ayrıca $|$ gibi yan semboller de kullanır, bizim yaptığımız gibi. Böylece, Örnek 5.12'deki dil bilgisi BNF'de şu şekilde görünebilir

$$\begin{aligned}\langle expression \rangle &::= \langle term \quad expression \rangle + \langle term \rangle, \\ \langle term \rangle &::= \langle factor \rangle | \langle term \rangle * \langle factor \rangle,\end{aligned}$$

ve devam eder. Semboller $+$ ve $*$ terminaldir. Sembol $|$ alternatif olarak kullanılır, bizim notasyonumuzdaki gibi, ancak $::=$ yerine $\rightarrow$ kullanılır. BNF programlama dillerinin tanımlarında, üretimin amacını açık hale getirmek için daha açık değişken kimlikleri kullanma eğilimindedir. Ancak, iki notasyon arasında önemli bir fark yoktur.

C benzeri programlama dillerinin birçok kısmı, kısıtlı biçimlerdeki bağlamdan bağımsız dil bilgileri ile tanımlanabilir. Örneğin, *while*

C'deki ifadesi şu şekilde tanımlanabilir

$$\langle while_statement \rangle ::= while \langle expression \rangle \langle statement \rangle.$$

Burada anahtar kelime *while* bir terminal semboldür. Diğer tüm terimler değişkenlerdir, bunlar hala tanımlanmalıdır. Bunu Tanım 5.4 ile kontrol edersek, bunun bir s-dil bilgisi üretimi gibi görüldüğünü görürüz. Değişken $\langle while_statement \rangle$ solda her zaman terminal *while* sağda ile ilişkilidir. Bu nedenle, böyle bir ifade kolayca ve verimli bir şekilde ayrıştırılır. Burada programlama dillerinde anahtar kelimeleri neden kullandığımızı görebiliriz. Anahtar-kelimeler yalnızca bir programın okuyucusunu yönlendirebilecek bazı görsel yapılar sağlama kla kalmaz, aynı zamanda bir derleyicinin işini de çok daha kolay hale getirir.

Ne yazık ki, tipik bir programlama dilinin tüm özellikleri bir s-gramer ile ifade edilemez. Yukarıdaki $\langle expression \rangle$ için kurallar bu türden değildir, bu nedenle ayrıştırma daha az belirgin hale gelir. O zaman hangi gramer kurallarına izin verebileceğimiz ve yine de verimli bir şekilde ayrıştırabileceğimiz sorusu ortaya çıkar. Derleyicilerde, LL ve LR gramerleri olarak adlandırılanların kapsamlı bir şekilde kullanımı yapılmıştır. Bu gramerler, bir programlama dilinin daha az belirgin özelliklerini ifade etme yeteneğine sahiptir, yine de lineer zamanda ayrıştırmamıza olanak tanır. Bu basit bir mesele değildir ve çoğu, tartışmamızın kapsamının ötesindedir. Bu konuyu Bölüm 6'da kısaca ele alacağız, ancak amacımız için böyle gramerlerin var olduğunu ve yaygın olarak incelendiğini anlamak yeterlidir.

Bununla bağlantılı olarak, belirsizlik meselesi ek bir önem kazanır. Bir programlama dilinin tanımı belirsiz olmamalıdır, aksi takdirde bir program farklı derleyiciler tarafından işlendiğinde veya farklı sistemlerde çalıştırıldığında çok farklı sonuçlar verebilir. Örnek 5.11'de gösterildiği gibi, naif bir yaklaşım gramerde kolayca belirsizlik yaratabilir. Böyle hatalardan kaçınmak için belirsizlikleri tanıyabilmeli ve ortadan kaldırabilmeliyiz. İlgili bir soru, bir dilin doğası gereği belirsiz olup olmadığıdır. Bu amaç için ihtiyaç duyduğumuz şey, bağlamdan bağımsız gramerlerde belirsizlikleri tespit eden ve kaldıran algoritmalar ile bir bağlamdan bağımsız dilin doğası gereği belirsiz olup olmadığını belirleyen algoritmalarıdır. Ne yazık ki, bunlar çok zor görevlerdir ve en genel anlamda imkansızdır, daha sonra göreceğimiz gibi.

Bir programlama dilinin bağlamdan bağımsız bir gramerle modellenebilmesi en yönleri genellikle onun sözdizimi olarak adlandırılır. Ancak, bu anlamda sözdizimsel olarak doğru olan tüm programların aslında kabul edilebilir programlar olmadığı normalde geçerlidir. C için, yaygın BNF tanımı, aşağıdaki gibi yapıları izin verir:

char *a, b, c;*

takip eden

c = 3.2;

Bu kombinasyon, “bir karakter değişkenine gerçek bir değer atanamaz” kuralını ihlal ettiği için C derleyicileri tarafından kabul edilmez. Bağlamdan bağımsız

gramerler, tür çakışmalarının izin verilmeyebileceği gerçeğini ifade edemez. Böyle kurallar, belirli bir yapının anlamını nasıl yorumladığımızla ilgili olduğu için programlama dili semantiğinin bir parçasıdır.

Programlama dili semantiği karmaşık bir meseledir. Programlama dili semantiğinin tanımı için bağlamdan bağımsız gramerler kadar zarif ve özlü bir şey yoktur ve dolayısıyla bazı anlamsal özellikler kötü tanımlanmış veya belirsiz olabilir. Programlama dilleri ve formel dil teorisi açısından programlama dili semantiğini tanımlamak için etkili yöntemler bulmak sürekli bir endişe kaynağıdır. Birçok yöntem önerilmiştir, ancak bunların hiçbirisi bağlamdan bağımsız dillerin sözdizimi için olduğu kadar evrensel olarak kabul edilmemiş ve başarılı olmamıştır.

EGZERSİZLER

1. Belirli bir programlama dilinde sayıların aşağıdaki kurallara göre oluşturulduğunu varsayalım:

(a) Bir işaret isteğe bağlıdır.

(b) Değer alanı, ondalık noktasıyla ayrılmış iki boş olmayan kısımdan oluşur.

(c) Bir üstel alan isteğe bağlıdır. Varsa, üstel alane harfi ve işaretli iki basamaklı bir tam sayıdan oluşmalıdır.

Bu gereksinimleri biçimsel hale getiren bir dil bilgisi bulun.

2. Aşağıdaki yapılar için biçimsel tanımlar için C hakkında bir kitaba danışın:

Literal

For ifadesi

If-else ifadesi

Do ifadesi

Bileşik ifade

Return ifadesi

3. C'nin bağlamdan bağımsız dil bilgileriyle tanımlanamayan özelliklerine örnekler bulun.

BÖLÜM

6

SÖZCÜKLERİN KONTEXT-ÜCRETSİZ GRAMERLERİN

VE NORMAL ŞEKİLLERİNİN BASİTLEŞTİRİLMESİ

BÖLÜM ÖZETİ

Bölüm 5'te, bağlamdan bağımsız diller için üyelik ve parsing'in önemli sorunlarıyla karşılaştık. Kaba kuvvetle parsing her zaman mümkün olsa da, çok verimsizdir ve bu nedenle pratik değildir. Gerçek uygulamalar, derleyiciler gibi, daha verimli yöntemlere ihtiyaç duyar. Bu, bağlamdan bağımsız bir üretimin sağ tarafının sınırsız biçimi nedeniyle zorlaşır, bu yüzden gramerin gücünü azaltmadan sağ tarafı kısıtlayıp kısıtlamayacağımızı araştırıyoruz.

Yapabileceğimiz ilk şey, belirli türdeki üretimler hakkında endişelenmemiz gerekmediğini göstermektir. Özellikle, bir bağlamdan bağımsız dilbilgisi sağ tarafında λ bulunan üretilere sahipse, böyle λ üretimleri olmayan eşdeğer bir dilbilgisi bulabiliriz. Benzer bir şekilde, sağda yalnızca bir değişken bulunan birim üretimleri ve bir dize türetilmesinde asla gerçekleşemeyecek gereksiz üretimleri kaldırabiliriz.

Ayrıca, normal formları, yani çok kısıtlı ancak yine de herhangi bir bağlamdan bağımsız dilbilgisinin normal formda bir eşdeğeri olduğu anlamında genel olan dilbilgisel formları tartışıyoruz. Birçok türde normal form tanımlanabilir; burada yalnızca en yararlı iki tanesini tartışıyoruz: Chomsky normal formu ve Greibach normal formu.

Bağlamdan bağımsız dilleri daha derinlemesine inceleyebilmemiz için bazı teknik konulara dikkat etmemiz gerekiyor. Bir bağlamdan bağımsız dilbilgisinin tanımı, bir üretimin sağ tarafında hiçbir kısıtlama getirmez. Ancak, tam özgürlük gerekli değildir ve aslında bazı argümanlarda dezavantajdır. Teorem 5.2'de, dilbilgisel formlar üzerindeki belirli kısıtlamaların sağladığı kolaylığı görüyoruz; aşağıdaki gibi kuralları ortadan kaldırmak $A \rightarrow \lambda$ ve $A \rightarrow B$ argümanları daha kolay hale getirir. Birçok durumda, dilbilgisine daha katı kısıtlamalar getirmek istenir. Bu nedenle, rastgele bir bağlamdan bağımsız dilbilgisini, biçimi üzerinde belirli kısıtlamaları karşılayan eşdeğer bir dilbilgisine dönüştürme yöntemlerine bakmamız gerekiyor. Bu bölümde, sonraki tartışmalarda faydalı olacak birkaç dönüşüm ve yer değiştirme üzerinde duruyoruz.

Bağlamdan bağımsız dilbilgileri için normal biçimleri de araştırıyoruz. Normal bir biçim, kısıtlı olmasına rağmen, herhangi bir dilbilgisinin eşdeğer bir normal biçim versiyonuna sahip olmasını sağlayacak kadar geniştir. Bunlardan en faydalı olan iki tanesini tanıtıyoruz: Chomsky normal biçimi ve Greibach normal biçimi. Her ikisinin de birçok pratik ve teorik kullanımı vardır. Chomsky normal biçiminin ayrıştırmaya doğrudan bir uygulaması Bölüm 6.3'te verilmiştir.

Bu bölümdeki materyalin biraz sıkıcı doğası, birçok argümanın manipülatif olmasından ve az sezgisel içgörü sağlamasından kaynaklanmaktadır. Bizim amacımız için, bu teknik yön nispeten önemsizdir ve rahatça okunabilir. Çeşitli sonuçlar önemlidir; bunlar daha sonraki tartışmalarda birçok kez kullanılacaktır.

6.1 GRAMERLERİ DÖNÜŞTÜRME YÖNTEMLERİ

Öncelikle dilbilgileri ve dillerle ilgili biraz rahatsız edici bir konuyu gündeme getiriyoruz: boş dizenin varlığı. Boş dize, birçok teorem ve kanıtta oldukça özel bir rol oynamaktadır ve genellikle ona özel bir dikkat vermek gerekmektedir. Onu tamamen göz önünden kaldırmayı tercih ediyoruz, yalnızca λ içermeyen dillere bakıyoruz. Bunu yaparken, aşağıdaki değerlendirmelerden de göreceğimiz gibi, genel geçerliliğimizi kaybetmiyoruz.

Let L herhangi bir bağlamdan bağımsız dil olsun ve $G = (V, T, S, P)$ $L - \{\lambda\}$ için bir bağlamdan bağımsız dilbilgisi olsun. O zaman, V 'ye yeni değişken S_0 ekleyerek, S_0 'ı başlangıç değişkeni yaparak ve P 'ye üretimleri ekleyerek elde ettiğimiz dilbilgisi

$$S_0 \rightarrow S|\lambda$$

üretir L . Bu nedenle, $L - \{\lambda\}$

için yapabileceğimiz herhangi bir önemsiz sonuç neredeyse kesinlikle L 'ye aktarılacaktır. Ayrıca, herhangi bir bağlamdan bağımsız dilbilgisi verildiğinde

$$G, G \text{ elde etme yöntemi vardır} \text{ böylece } L(\hat{G}) = L(G) - \{\lambda\}$$

(bu bölümün sonunda 15 ve 16. alıştırmalara bakınız). Sonuç olarak, pratikte bağlamdan bağımsız diller arasında hiçbir fark yoktur.

λ içeren ve içermeyenler. Bu bölümün geri kalanında, aksi belirtilmedikçe, tartışmamızı λ -ücretsiz dillerle sınırlayacağız.

Kullanışlı Bir Yerine Koyma Kuralı

Yerine koymalar yoluyla eşdeğer dilbilgileri oluşturmayı yöneten birçok kural vardır. Burada, dilbilgilerini çeşitli şekillerde basitleştirmek için çok yararlı olan bir kural veriyoruz. "Basitleştirme" terimini tam olarak tanımlama yacağız, ancak yine de kullanacağız. Bununla kastettiğimiz, istenmeyen belirlirli türdeki üretimlerin kaldırılmasıdır; bu süreç, kuralların sayısında mutlak bir azalma ile sonuçlanmaz.

TEOREM 6.1

$G = (V, T, S, P)$ bir bağlamdan bağımsız dilbilgisi olsun. P 'nin aşağıdaki biçimde bir üretim içerdiğini varsayalım

$$A \rightarrow x_1 B x_2.$$

A ve B farklı değişkenler olsun ve

$$B \rightarrow y_1 | y_2 | \cdots | y_n$$

P içinde B 'nin sol tarafı olarak bulunan tüm üretimlerin kümesi olsun. Let $\hat{G} = (V, T, S, \hat{P})$ gramer olsun ki $\hat{P}$, silerek inşa edilir

$$A \rightarrow x_1 B x_2 \quad (6.1)$$

P 'den, ve ona ekleyerek

$$A \rightarrow x_1 y_1 x_2 | x_1 y_2 x_2 | \cdots | x_1 y_n x_2.$$

Sonra

$$L(\hat{G}) = L(G).$$

Kanıt: $w \in L(G)$ olduğunu varsayalım, böylece

$$S \xRightarrow{*}_G w.$$

Türetme işareti üzerindeki alt simge $\Rightarrow$ burada farklı gramerlerle yapılan türetmeleri ayırt etmek için kullanılır. Eğer bu türetme, üretim (6.1) ile ilgili değilse, o zaman açıkça

$$S \xRightarrow{*}_{\hat{G}} w.$$

Eğer öyleyse, (6.1) ilk kez kullanıldığında türetime bakın. Tanıtılan B son unda değiştirilmelidir; bunun hemen yapıldığını varsayarak hiçbir şey ka ybetmiyoruz (bu bölümün sonunda Egzersiz 18'e bakın). Böylece

$$S \xRightarrow{*}_G u_1 A u_2 \Rightarrow_G u_1 x_1 B x_2 u_2 \Rightarrow_G u_1 x_1 y_j x_2 u_2.$$

Ancak dilbilgisi $\hat{G}$ ile elde edebiliriz

$$S \xRightarrow{*}_{\hat{G}} u_1 A u_2 \Rightarrow_{\hat{G}} u_1 x_1 y_j x_2 u_2.$$

Böylece G ve $\hat{G}$ ile aynı cümle biçimine ulaşabiliriz. (6.1) daha sonra te krar kullanılırsa, argümanı tekrarlayabiliriz. Bu durumda, üretimin uy gulanma sayısı üzerinde indüksiyon ile

$$S \xRightarrow{*}_{\hat{G}} w.$$

Bu nedenle, eğer $w \in L(G)$, o zaman $w \in L(\hat{G})$.

Benzer bir mantıkla, eğer $w \in L(\hat{G})$, o zaman $w \in L(G)$, kanıtı tamamlanır. ■

Teorem 6.1, basit ve oldukça sezgisel bir yer değiştirme kuralıdır: Bi r üretim $A \rightarrow x_1 B x_2$, yerine B 'nin bir adımda türettiği tüm dizelerle değişt iildiği üretim kümesini koyarsak bir dilbilgisinden çıkarılabilir. Bu sonuç ta, A ve B 'nin farklı değişkenler olması gerekmektedir. Durumun $A=B$ ol duğu kısmen bu bölümün sonunda Egzersiz 25 ve 26'da ele alınmaktadı r.

ÖRNEK 6.1

Consider $G = (\{A, B\}, \{a, b, c\}, A, P)$ with productions

$$A \rightarrow a | aaA | abBc,$$

$$B \rightarrow abbA | b.$$

Önerilen değişimi kullanarak değişken B için, gram-mar $\hat{G}$ üretimleri ile elde ediyoruz

$$A \rightarrow a | aaA | ababbAc | abbc,$$

$$B \rightarrow abbA | b.$$

Yeni gram-mar $\hat{G}, G$ ile eşdeğerdir. Dizi $aaabbc$, türetimi vardır

$$A \Rightarrow aaA \Rightarrow aaabBc \Rightarrow aaabbc$$

iG , ve karşılık gelen türetim

$$A \Rightarrow aaA \Rightarrow aaabbc$$

$i\hat{G}$.

Bu durumda, değişken B ve ona bağlı üretimler, artık herhangi bir türetimde rol oynayamaz hale gelse de, gram-ırmada hâlâ bulunmaktadır. Şimdi, böyle gereksiz üretimlerin bir gram-ırmadan nasıl çıkarılacağını göstereceğiz.

Gereksiz Üretimlerin Kaldırılması

Bir gram-ırmadan asla herhangi bir türetimde yer almayacak üretimleri kaldırmak her zaman istenir. Örneğin, tüm üretim kümesi

$$\begin{aligned} S &\rightarrow aSb \mid \lambda \mid A, \\ A &\rightarrow aA, \end{aligned}$$

üretim $S \rightarrow A$ açıkça bir rol oynamaz, çünkü A terminal bir diziye dönüşürülemez. A , S 'den türetilen bir dizide yer alabilir, ancak bu asla bir c ümleye yol açamaz. Bu üretimi kaldırmak, dili etkilemez ve herhangi bir tanıma göre bir basitleştirme değildir.

DEFINITION 6.1

Bir $G = (V, T, S, P)$ bağlamdan bağımsız bir dilbilgisi olsun. Bir değişken $A \in V$, yalnızca en az bir $w \in L(G)$ olduğu durumlarda yararlı olarak adlandırılır.

$$S \xRightarrow{*} xAy \xRightarrow{*} w, \quad (6.2)$$

with $x, y \in (V \cup T)^*$. Kelimelerle, bir değişken yalnızca en az bir türetimde bulunuyorsa faydalıdır.

Faydalı olmayan bir değişken useless denir.

Bir üretim, herhangi bir faydasız değişken içeriyorsa faydasızdır.

ÖRNEK 6.2

Bir değişken işe yaramaz olabilir çünkü bir terminal elde etmenin yolu yoktur. ondan bir dize. Az önce bahsedilen durum bu türdendir. Bir değişkenin işe yaramaz olmasının bir diğer nedeni, bir gramerde gösterilmektedir.

başlangıç sembolü S ve üretimlerle

$$\begin{aligned} S &\rightarrow A, \\ A &\rightarrow aA|\lambda, \\ B &\rightarrow bA, \end{aligned}$$

değişken B işe yaramaz ve üretim $B \rightarrow bA$ de öyle. B bir terminal dize türetebilse de, $S \Rightarrow^* xBy$ elde etmenin bir yolu yoktur.

Bu örnek, bir değişkenin neden işe yaramaz olduğunu gösteren iki nedeni açıklamaktadır: yabaşlangıç sembolünden ulaşılamadığı için ya da bir terminal dize türetemediği için. İşe yaramaz değişkenleri ve ürettiği mleri kaldırma prosedürü, bu iki durumu tanımaya dayanmaktadır. Genel durumu ve ilgili teoremi sunmadan önce, bir başka örneğe bakalım.

ÖRNEK 6.3

$G=(V, T, S, P)$ içindeki gereksiz sembolleri ve üretimleri ortadan kaldırın, burada $V=\{S, A, B, C\}$ ve $T=\{a, b\}$ olup, P şunlardan oluşmaktadır

$$\begin{aligned} S &\rightarrow aS | A | C, \\ A &\rightarrow a, \\ B &\rightarrow aa, \\ C &\rightarrow aCb. \end{aligned}$$

Öncelikle, bir terminal dizeye yol açabilecek değişkenler kümesini tanımlıyoruz. Çünkü $A \rightarrow a$ ve $B \rightarrow aa$, değişkenler A ve B bu kümeye aittir. Ayrıca S de bu kümeye aittir, çünkü $S \Rightarrow A \Rightarrow a$. Ancak, bu argüman C için yapılamaz, bu nedenle onu gereksiz olarak tanımlıyoruz. C 'yi ve ilgili üretimlerini kaldırarak, G_1 gramerine ulaşırız değişkenler $V_1=\{S, A, B\}$, terminaller $T=\{a\}$ ve üretimler

$$\begin{aligned} S &\rightarrow aS | A, \\ A &\rightarrow a, \\ B &\rightarrow aa. \end{aligned}$$

Sonra, başlangıç değişkeninden ulaşılamayan değişkenleri ortadan kaldırmak istiyoruz. Bunun için, değişkenler için bir bağımlılık grafiği çizebiliriz. Bağımlılık grafikleri, karmaşık ilişkileri görselleştirmenin bir yoludur ve birçok uygulamada bulunur. Bağımsız gramerler için, bir bağımlılık grafiğinin köşeleri değişkenlerle etiketlenmiştir,

Bir köşe C ve D arasında yalnızca, aşağıdaki biçimde bir üretim varsa bir kenar bulunur:

$$C \rightarrow xDy.$$

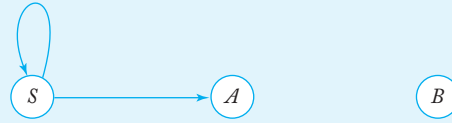
V_1 için bir bağımlılık grafiği Şekil 6.1'de gösterilmiştir. Bir değişken yalnızca S ile etiketlenmiş köşeden o değişkenle etiketlenmiş köşeye bir yol varsa kullanışlıdır. Bizim durumumuzda, Şekil 6.1, B 'nin gereksiz olduğunu göstermektedir.

Onu ve etkilenen üretimleri ve terminalleri kaldırarak, nihai cevaba $\hat{G} = (V, \hat{T}, S, \hat{P})$ ulaşırız $\hat{V} = \{S, A\}$, $\hat{T} = \{a\}$ ve üretimler

$$S \rightarrow aS|A,$$

$$A \rightarrow a.$$

Bu sürecin formalizasyonu genel bir yapı ve ilgili teoremi ortaya çıkarır.



ŞEKİL 6.1

TEOREM 6.2

$G = (V, T, S, P)$ bağlamdan bağımsız bir dilbilgisi olsun. O zaman, gereksiz değişkenler veya üretimler içermeyen eşdeğer bir dilbilgisi $\hat{G} = (V, \hat{T}, S, \hat{P})$ vardır.

Kanıt: Dilbilgisi $\hat{G}$, iki bölümden oluşan bir algoritma ile G 'den türetilir. İlk bölümde, yalnızca değişkenler A içeren bir ara dilbilgisi $G_1 = (V_1, T_2, S, P_1)$ oluşturuyoruz.

$$A \xRightarrow{*} w \in T^*$$

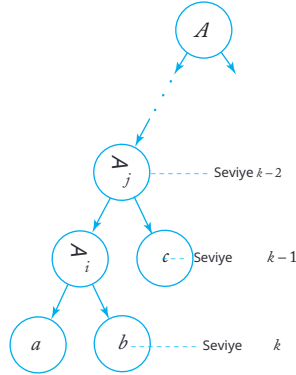
Algoritmadaki adımlar şunlardır:

1. V_1 'yi $\emptyset$ olarak ayarlayın.
2. V_1 'ye daha fazla değişken eklenmediği sürece aşağıdaki adımı tekrarlayın. Her $A \in V$ için, P 'nin aşağıdaki biçimde bir üretimi varsa

$$A \rightarrow x_1 x_2 \cdots x_n, \text{ with all } x_i \text{ in } V_1 \cup T,$$

A 'yı V_1 'ye ekleyin.

3. TP_1 olarak, P içindeki tüm sembollerin $(V_1 \cup T)$ içinde olduğu tüm üretimleri al.



ŞEKİL 6.2

Bu prosedürün sona erdiği açıktır. Ayrıca, eğer $A \in V_1$, o zaman $A^* \Rightarrow w \in T^*$, G_1 ile olası bir türetimdir. Kalan sorun, her A için $A^* \Rightarrow w = ab \dots$ türetimden önce V_1 'ye eklenip eklenmediğidir. Bunu görmek için, her hangi bir A 'yı ele alalım ve o türetime karşılık gelen kısmi türetim ağacına bakalım (Şekil 6.2). Seviye k 'de yalnızca terminal semboller vardır, bu nedenle seviye $k-1$ 'deki her değişken A_i , algoritmanın Adım 2'sinin ilk geçişinde V_1 'ye eklenecektir. Seviye $k-2$ 'deki herhangi bir değişken ise, Adım 2'nin ikinci geçişinde V_1 'ye eklenecektir. Adım 2'de üçüncü kez, seviye $k-3$ 'teki tüm değişkenler eklenecek, ve bu şekil de devam edecektir. Algoritma, ağaçta henüz V_1 'de olmayan değişkenler varken sona eremez. Bu nedenle A nihayetinde V_1 'ye eklenecektir.

Yapının ikinci bölümünde, nihai cevabı G elde ediyoruz

G_1 için. Değişken bağımlılık grafiğini G_1 için çizeriz ve buradan S 'den ulaşılabilen tüm değişkenleri buluruz. Bunlar, aralarındaki üretimlerle birlikte değişken kümesinden çıkarılır. Ayrıca, bazı yararlı üretimlerde yer almayan herhangi bir terminali de ortadan kaldırabiliriz. Sonuç, dil bilgisi $\hat{G} = (V, \hat{T}, S, \hat{P})$ olacaktır.

Yapı nedeniyle, $\hat{G}$ gereksiz semboller veya üretimler içermez. Ayrıca, her $w \in L(G)$ için bir türetimimiz vardır

$$S \xRightarrow{*} xAy \xRightarrow{*} w.$$

$\hat{G}$ 'nin yapısı A ve tüm ilişkili üretimleri koruduğundan, türetim için gereken her şeye sahibiz

$$S \xRightarrow{*}_{\hat{G}} xAy \xRightarrow{*}_{\hat{G}} w.$$

Dil bilgisi $\hat{G}$, üretimlerin çıkarılmasıyla G' 'den inşa edilmiştir, bu nedenle $\hat{P} \subseteq P$. Sonuç olarak $L(\hat{G}) \subseteq L(G)$. İki sonucu bir araya getirdiğimizde, G ve $\hat{G}$ 'nin eşdeğer olduğunu görüyoruz. ■

Removing λ -Productions

Bir tür üretim, bazen istenmeyen bir durumdur; bu durumda sağ taraf boş dizedir.

DEFINITION 6.2

Bir bağlamdan bağımsız dilbilgisinin aşağıdaki formdaki herhangi bir üretimi

$$A \rightarrow \lambda$$

bir λ -üretim olarak adlandırılır. Herhangi bir değişken A için türetim

$$A \xRightarrow{*} \lambda \quad (6.3)$$

mümkünse nullable olarak adlandırılır.

Bir dilbilgisi, λ içermeyen bir dil üretebilir, ancak bazı λ -üretileri veya nullable değişkenleri olabilir. Bu tür durumlarda, λ -üretileri kaldırılabilir.

EXAMPLE 6.4

Grameri dikkate al

$$\begin{aligned} S &\rightarrow aS_1b, \\ S_1 &\rightarrow aS_1b|\lambda, \end{aligned}$$

başlangıç değişkeni S ile. Bu dilbilgisi λ -ücretsiz dili üretir. $\{a^n b^n : n \geq 1\}$. λ üretimi $S_1 \rightarrow \lambda$, S_1 'in bulunduğu yerlerde λ ile değiştirilerek elde edilen yeni üretimlerin eklenmesinden sonra kaldırılabilir. Sağda. Bunu yaparak grameri elde ederiz.

$$\begin{aligned} S &\rightarrow aS_1b|ab, \\ S_1 &\rightarrow aS_1b|ab. \end{aligned}$$

Bu yeni gramere, orijinal olanla aynı dili ürettiğini kolayca gösterebiliriz.

Daha genel durumlarda, λ üretimleri için ikameler benzer, ancak daha karmaşık bir şekilde yapılabilir.

TEOREM 6.3

$G, L(G)$ 'de olmayan herhangi bir bağlamdan bağımsız gramer olsun. O zaman λ üretimleri olmayan eşdeğer bir gramer $\hat{G}$ vardır.

Kanıt: Öncelikle, aşağıdaki adımları kullanarak G 'nin tüm nullable değişkenlerinin kümesini VN buluyoruz.

1. Tüm üretimler için $A \rightarrow \lambda$, A 'yi V_N 'ye ekleyin.
2. Aşağıdaki adımı, V_N 'ye daha fazla değişken eklenmediği sürece tekrarlayın.
Tüm üretimler için

$$B \rightarrow A_1 A_2 \cdots A_n,$$

$A_1, A_2, \dots, A_n$ V_N 'de ise, B 'yi V_N 'ye ekleyin.

Küme V_N bulunduğunda, $\hat{P}$ 'yi oluşturmak için hazırız. Bunu yapmak için, P 'deki tüm üretilere bakıyoruz.

$$A \rightarrow x_1 x_2 \cdots x_m, m \geq 1,$$

Her $x_i \in V \cup T$ olan üretimler için. P 'deki her böyle üretim için, o üretimi $\hat{P}$ 'ye ekliyoruz ve nullable değişkenleri λ ile tüm olası kombinasyonlarda değiştirilerek üretilenleri de ekliyoruz. Örneğin, eğer x_i ve x_j her ikisi de nullable ise, $\hat{P}$ 'de x_i 'nin λ ile değiştirildiği bir üretim, x_j 'nin λ ile değiştirildiği bir üretim ve her ikisinin de λ ile değiştirildiği bir üretim olacaktır. Bir istisna vardır: Eğer tüm x_i nullable ise, üretim $A \rightarrow \lambda$ 'ye eklenmez.

Bu dilin $\hat{G}$ 'nin G ile eşdeğer olduğu argümanı açıktır ve okuyucuya bırakılacaktır. ■

ÖRNEK 6.5

λ üretimleri olmayan bir bağlamdan bağımsız dilbilgisi bulun ve bu dilbilgisinin eşdeğeri olan dilbilgisi aşağıda tanımlanmıştır.

$$\begin{aligned} S &\rightarrow ABaC, \\ A &\rightarrow BC, \\ B &\rightarrow b|\lambda, \\ C &\rightarrow D|\lambda, \\ D &\rightarrow d. \end{aligned}$$

Teorem 6.3'teki inşanın ilk adımından, nullable değişkenlerin A, B, C olduğunu buluyoruz. Ardından, inşanın ikinci adımını takip ederek, elde ediyoruz

$$\begin{aligned} S &\rightarrow ABaC \mid BaC \mid AaC \mid ABa \mid aC \mid Aa \mid Ba \mid a, \\ A &\rightarrow B \mid C \mid BC, \\ B &\rightarrow b, \\ C &\rightarrow D, \\ D &\rightarrow d. \end{aligned}$$

Birlik Üretimlerini Kaldırma

Teorem 5.2'de gördüğümüz gibi, her iki tarafın da tek bir değişken olduğu üretimler bazen istenmeyen durumlardır.

TANIM 6.3

Bir bağlamdan bağımsız dilbilgisinin aşağıdaki formdaki herhangi bir üretimi

$$A \rightarrow B,$$

burada $A, B \in V$, bir birlik üretimi olarak adlandırılır.

Birlik üretimlerini kaldırmak için, Teorem 6.1'de tartışılan yer değiştirme kurallını kullanıyoruz. Bir sonraki teoremdeki inşanın gösterdiği gibi, bunu dikkatli bir şekilde ilerleyerek yapabiliriz.

TEOREM 6.4

$G = (V, T, S, P)$ herhangi bir λ -produksiyon içermeyen bağlamdan bağımsız bir dilbilgisi olsun. O zaman bir bağlamdan bağımsız dilbilgisi $\hat{G} = (\hat{V}, \hat{T}, S, \hat{P})$ birim üretim i olmayan ve G ile eşdeğer olan.

Kanıt: Açıkça, herhangi bir birim üretim biçimi $A \rightarrow A$, dilbilgisi üzerinde etki si olmadan kaldırılabilir ve yalnızca $A \rightarrow B$ 'yi düşünmemiz gerekiyor, burada A ve B farklı değişkenlerdir. İlk bakışta, doğrudan $x_1 = x_2 = \lambda$ ile birlikte Teorem 6.1'i kullanabileceğimiz gibi görünebilir.

$$A \rightarrow B$$

ile

$$A \rightarrow y_1 \mid y_2 \mid \cdots \mid y_n.$$

Ancak bu her zaman işe yaramayacak; özel durumda

$$\begin{aligned} A &\rightarrow B, \\ B &\rightarrow A, \end{aligned}$$

birim üretimler kaldırılmamaktadır. Bunu aşmak için, önce her A için, tüm değişkenler B buluruz ki

$$A \xRightarrow{*} B. \quad (6.4)$$

Bunu, dilbilgisi bir birim üretim $C \rightarrow D$ olduğunda bir kenar (C, D) ile bir bağımlılık grafiği çizerek yapabiliriz; o zaman (6.4) her zaman A ve B arasında bir yürüyüş olduğunda geçerlidir. Yeni dilbilgisi $\hat{G}$, önce $\hat{P}$ 'ye $\hat{P}$ 'nin tüm birim olmayan üretimlerini koyarak oluşturulur. Sonra, (6.4)'ü sağlayan tüm A ve B için, $\hat{P}$ 'ye ekleriz

$$A \rightarrow y_1 | y_2 | \cdots | y_n,$$

burada $B \rightarrow y_1 | y_2 | \cdots | y_n$, $\hat{P}$ 'ye B 'nin solda olduğu tüm kuralların kümesidir. Not edin ki $B \rightarrow y_1 | y_2 | \cdots | y_n$, $\hat{P}$ 'den alındığı için, hiçbir y_i tek bir değişken olamaz, böylece son adımda hiçbir birim üretim oluşturulmaz.

Sonuçta elde edilen dilbilgisinin orijinal olanla eşdeğer olduğunu göstermek için, Teorem 6.1'deki aynı akıl yürütmeyi takip edebiliriz. ■

ÖRNEK 6.6

Tüm birim üretimleri kaldırın

$$\begin{aligned} S &\rightarrow Aa|B, \\ B &\rightarrow A|bb, \\ A &\rightarrow a|bc|B. \end{aligned}$$

Birim üretimler için bağımlılık grafi Şekil 6.3'te verilmiştir; buradan $S^* \Rightarrow A$, $S^* \Rightarrow B$, $B^* \Rightarrow A$ ve $A^* \Rightarrow B$ olduğunu görüyoruz. Bu nedenle, orijinal birim olmayan üretilere ekliyoruz

$$\begin{aligned} S &\rightarrow Aa, \\ A &\rightarrow a|bc, \\ B &\rightarrow bb, \end{aligned}$$

yeni kurallar

$$\begin{aligned} S &\rightarrow a|bc|bb, \\ A &\rightarrow bb, \\ B &\rightarrow a|bc, \end{aligned}$$

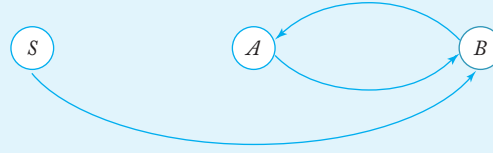
eşdeğer dilbilgisini elde etmek için

$$S \rightarrow a | bc | bb | Aa,$$

$$A \rightarrow a | bb | bc,$$

$$B \rightarrow a | bb | bc.$$

Birimin kaldırılması, B ve ilişkili üretimleri işe yaramaz hale getirdiğini unutmayın.



ŞEKİL 6.3

Tüm bu sonuçları bir araya getirerek, bağlamdan bağımsız diller için dilbilgilerini n işe yaramaz üretimlerden, λ üretimlerinden ve birim üretimlerinden arındırılabilceğini gösterebiliriz.

TEOREM 6.5

L , λ içermeyen bir bağlamdan bağımsız dil olsun. O zaman L 'yi üreten ve hiçbir işe yaramaz üretim, λ üretimi veya birim üretimi olmayan bir bağlamdan bağımsız dilbilgisi vardır.

Kanıt: 6.2, 6.3 ve 6.4 Teoremlerinde verilen prosedürler bu tür üretimleri sırayla kaldırır. Dikkate alınması gereken tek nokta, bir tür üretimi n kaldırılmasının başka bir tür üretimi tanıtabilmesidir; örneğin, λ üretimlerini kaldırma prosedürü yeni birim üretimleri oluşturabilir. Ayrıca, Teorem 6.4, dilbilgisinin λ üretimlerine sahip olmamasını gerektirir. Ancak, birim üretimlerinin kaldırılmasının λ üretimleri oluşturmadığını (bu bölümün sonunda 18. Alıştırma) ve işe yaramaz üretimlerin kaldırılmasının λ üretimleri veya birim üretimleri oluşturmadığını (bu bölümün sonunda 19. Alıştırma) unutmayın. Bu nedenle, aşağıdaki adım dizisini kullanarak tüm istenmeyen üretimleri kaldırabiliriz:

1. Remove λ -productions.
2. Remove unit-productions.
3. Remove useless productions.

The result will then have none of these productions, and the theorem is proved. ■

EGZERSİZLER

1. Eliminate the variable B from the grammar

$$S \rightarrow aSB|bB$$

$$B \rightarrow aA|b.$$

2. Show that the two grammars

$$S \rightarrow abAB|ba,$$

$$A \rightarrow aaa,$$

$$B \rightarrow aA|bb$$

ve

$$S \rightarrow abAaA|abAbb|ba,$$

$$A \rightarrow aaa$$

are equivalent.

3. Complete the proof of Theorem 6.1 by showing that

$$S \xRightarrow{*}_{\hat{G}} w$$

implies

$$S \xRightarrow{*}_G w.$$

4. Örnek 6.1'de, hem orijinal hem de değiştirilmiş grameri kullanarak dize $ababbac$ için bir türetim ağacı gösterin.
5. Teorem 6.1'de, neden A ve B 'nin farklı değişkenler olduğunu varsaymak gereklidir?
6. Gramerden tüm gereksiz üretimleri ortadan kaldırın

$$S \rightarrow aS|AB|\lambda,$$

$$A \rightarrow bA,$$

$$B \rightarrow AA.$$

Bu gramer hangi dili üretir?

7. Gereksiz üretimleri ortadan kaldırın

$$S \rightarrow a|aA|B|C,$$

$$A \rightarrow aB|\lambda,$$

$$B \rightarrow Aa,$$

$$C \rightarrow cCD,$$

$$D \rightarrow ddd|Cd.$$

8. Tüm λ -üretimlerini ortadan kaldırın

$$\begin{aligned} S &\rightarrow aSSS, \\ S &\rightarrow bb|\lambda. \end{aligned}$$

9. Tüm λ -üretimlerini ortadan kaldırın

$$\begin{aligned} S &\rightarrow AaB|aaB, \\ A &\rightarrow \lambda, \\ B &\rightarrow bbA|\lambda. \end{aligned}$$

10. Gramerden tüm birim üretimlerini, tüm gereksiz üretimleri ve tüm λ -üretimlerini kaldırın

$$\begin{aligned} S &\rightarrow aA|aBB, \\ A &\rightarrow aaA|\lambda, \\ B &\rightarrow bB|bbC, \\ C &\rightarrow B. \end{aligned}$$

Bu gramer hangi dili üretir?

11. Egzersiz 7'deki dilbilgisi kurallarından tüm birim üretimlerini kaldırın.
 12. Teorem 6.3'ün kanıtını tamamlayın.
 13. Teorem 6.4'ün kanıtını tamamlayın.
 14. Örnek 5.4'teki gramerden λ üretimlerini kaldırmak için Teorem 6.3'teki yapıyı kullanın. Sonuçta elde edilen gramer hangi dili üretir?
 15. Üretimleri olan gramer G' 'yi düşünün.

$$\begin{aligned} S &\rightarrow A|B, \\ A &\rightarrow \lambda, \\ B &\rightarrow aBb, \\ B &\rightarrow b. \end{aligned}$$

Teorem 6.3'teki algoritmayı G' 'ye uygulayarak bir gramer $\hat{G}$ oluşturun.

$L(G)$ ile $L(\hat{G})$ arasındaki fark nedir?

16. Varsayalım ki $G, \lambda \in L(G)$ olan bir bağlamdan bağımsız gramerdir. Teorem 6.3'teki yapıyı uygularsak, yeni bir gramer $\hat{G}$ elde ederiz böylece $L(\hat{G}) = L(G) - \{\lambda\}$.
 17. λ üretimlerinin kaldırılmasının daha önce var olmayan birim üretimlerini nasıl ortaya çıkardığına dair bir örnek verin.
 18. G, λ üretimleri olmayan ancak bazı birim üretimleri olabilen bir gramer olsun. Teorem 6.4'ün yapısının o zaman herhangi bir λ üretimi ortaya çıkar madığını gösterin.
 19. Bir dilbilgisi λ -produksiyonları ve birim-produksiyonları yoksa, o zaman Teorem 6.2'nin inşasıyla gereksiz üretimlerin kaldırılmasının böyle üretimleri tanıtmadığını gösterin.

20. Teorem 6.1'in kanıtında yapılan B değişkeninin görüldüğü anda değiştirilebil eceği iddiasını gerekçelendirin.

21. Varsayalım ki bir bağlamdan bağımsız bir dilbilgisi $G = (V, T, S, P)$ aşağıdaki biçimde bir üreti me sahiptir

$$A \rightarrow xy,$$

burada $x, y \in (V \cup T)^+$. Bu kuralın

$$A \rightarrow By,$$

$$B \rightarrow x,$$

burada $B \notin V$ ile değiştirilmesi durumunda, elde edilen dilbilgisinin orijinal olanla eşdeğer olduğunu kanıtlayın.

22. Teorem 6.2'de önerilen gereksiz üretimlerin kaldırılması prosedürünü dü şünün. İki kısmın sırasını tersine çevirin, önce S' 'den ulaşılamayan değişke nleri kaldırın, ardından terminal dize vermeyenleri kaldırın. Yeni prosedür hala doğru çalışıyor mu? Eğer öyleyse, bunu kanıtlayın.

Eğer değilse, bir karşı örnek verin.

23. Bir dilbilgisinin basitleştirme terimini kesin olarak tanımlamak mümkündür. Bu, bir dilbilgisinin karmaşıklığı kavramını tanıtarak yapılabilir. Bu birçok şekilde yapılabil ir; bunlardan biri, üretim kurallarını veren tüm dizelerin uzunluğu aracılığıyla dır. Örneğin, şunu kullanabiliriz

$$complexity(G) = \sum_{A \rightarrow v \in P} \{1 + |v|\}.$$

Gereksiz üretimlerin kaldırılmasının her zaman bu anlamda karmaşıklığı azalt tığını gösterin. λ -üretimlerinin ve birim üretimlerinin kaldırılması hakkında ne söyleyebilirsiniz?

24. Bir bağlamdan bağımsız dilbilgisi G , belirli bir dil L için minimal olarak adlandırılır eğer $karmaşıklık(G) \leq karmaşıklık(\hat{G})$ herhangi bir $\hat{G}$ üreten L için. Gereksiz üretimlerin kaldırılmasının mutlaka minimal bir dilbilgisi üretmediğini ör nekle gösterin.

25. Sonucu kanıtlayın. Let $G = (V, T, S, P)$ bir bağlamdan bağımsız gramer olsun. Sol tarafları belirli bir değişkenin (örneğin,

A) bazı üretim kümesini iki ayrı alt kümeye ayırın

$$A \rightarrow Ax_1 | Ax_2 | \dots | Ax_n,$$

$$A \rightarrow y_1 | y_2 | \dots | y_m,$$

burada $x_i, y_i \in (V \cup T)^*$ içindedir, ancak A herhangi bir y_i 'nin ön eki değildir. Gram eri düşünün $\hat{G} = (V \cup \{Z\}, T, S, \hat{P})$, burada $Z \notin V$ ve $\hat{P}$, sol tarafta A olan tū m üretimlerin yerine konulmasıyla elde edilir.

$$A \rightarrow y_i | y_i Z, \quad i = 1, 2, \dots, m,$$

$$Z \rightarrow x_i | x_i Z, \quad i = 1, 2, \dots, n.$$

$$O \text{ zaman } L(G) = L(\hat{G}).$$

26. Önceki egzersizin sonucunu kullanarak grameri yeniden yazın

$$\begin{aligned} A &\rightarrow Aa|aBc|\lambda, \\ B &\rightarrow Bb|bc \end{aligned}$$

öyle ki artık $A \rightarrow Ax$ veya $B \rightarrow Bx$ biçiminde üretimleri olmasın.

27. Şu karşıtlığı kanıtlayın Egzersiz 23. Sol tarafta değişken A içeren üretim k ümesi iki ayrı alt kümeye bölünsün

$$A \rightarrow x_1 A | x_2 A | \dots | x_n A$$

ve

$$A \rightarrow y_1 | y_2 | \dots | y_m,$$

burada A herhangi bir y_i 'nin son eki değildir. Bu üretimleri değiştirerek el de edilen dilbilgisinin

$$\begin{aligned} A &\rightarrow y_i | Z y_i, \quad i = 1, 2, \dots, m, \\ Z &\rightarrow x_i | Z x_i, \quad i = 1, 2, \dots, n, \end{aligned}$$

orijinal dilbilgisi ile eşdeğer olduğunu gösterin.

6.2 İKİ ÖNEMLİ NORMAL ŞEKİL

Bağlamdan bağımsız dilbilgileri için kurabileceğimiz birçok normal şekil vardı r. Bunlardan bazıları, geniş kullanışlılıkları nedeniyle, kapsamlı bir şekilde ince lenmiştir. Bunlardan ikisini kısaca ele alıyoruz.

Chomsky Normal Form

Bir tür normal form arayabiliriz ki, burada bir üretimin sağındaki sem bol sayısı kesinlikle sınırlıdır. Özellikle, bir üretimin sağındaki dizinin en fazla iki sembolden oluşmasını isteyebiliriz. Bunun bir örneği Cho msky normal formudur.

DEFINITION 6.4

Bir bağlamdan bağımsız dilbilgisi, tüm üretimlerin aşağıdaki formda olduğu durumlarda Chomsky normal formundadır.

$$A \rightarrow BC$$

veya

$$A \rightarrow a,$$

burada $A, B, C \in V$ 'dir ve $a \in T$ 'dir.

EXAMPLE 6.7

Gramer

$$\begin{aligned} S &\rightarrow AS|a, \\ A &\rightarrow SA|b \end{aligned}$$

Chomsky normal formundadır. Dilbilgisi

$$\begin{aligned} S &\rightarrow AS|AAS, \\ A &\rightarrow SA|aa \end{aligned}$$

değildir; hem üretimler $S \rightarrow AAS$ hem de $A \rightarrow aa$, Tanım 6.4'ün koşullarını ihlal etmektedir.

TEOREM 6.6

Herhangi bir bağlamdan bağımsız dilbilgisi $G = (V, T, S, P)$ ile $\lambda \in L(G)$ eşdeğer bir dilbilgisi $\hat{G} = (\hat{V}, \hat{T}, \hat{S}, \hat{P})$ Chomsky normal formunda bulunmaktadır.

Kanıt: Teorem 6.5'ten dolayı, genel olarak kayıp olmaksızın şunu varsayabiliriz ki G λ -üretimleri ve birim-üretimleri yoktur. Yapının $\hat{G}$ iki adımda yapılacaktır.

Adım 1: Bir dilbilgisi $G_1 = (V_1, T, S, P_1)$ oluşturun G içindeki tüm üretimleri P biçiminde dikkate alarak

$$A \rightarrow x_1 x_2 \cdots x_n, \quad (6.5)$$

her bir x_i ya V ya da T içinde bir semboldür. Eğer $n = 1$ ise, o zaman x_1 bir terminal olmalıdır çünkü birim üretimlerimiz yok. Bu durumda, üretimi P_1 içine koyun. Eğer $n \geq 2$ ise, her $a \in T$ için yeni değişkenler B_a tanıtır. Her P üretimi (6.5) biçiminde ise, üretimi P_1 içine koyarız.

$$A \rightarrow C_1 C_2 \cdots C_n,$$

nerede

$$C_i = x_i \text{ if } x_i \text{ is in } V,$$

ve

$$C_i = B_a \text{ if } x_i = a.$$

Her bir B_a için ayrıca P_1 üretime de koyuyoruz

$$B_a \rightarrow a.$$

Algoritmanın bu kısmı, sağ tarafı birden fazla uzunluğa sahip olan üretimlerden tüm terminalleri kaldırarak, bunları yeni tanıtılan değişkenlerle değiştirmektedir. Bu adımın sonunda, tüm üretimlerinin formu olan bir dilbilgisi G_1 elde ederiz

$$A \rightarrow a, \quad (6.6)$$

veya

$$A \rightarrow C_1 C_2 \cdots C_n, \quad (6.7)$$

nerede $C_i \in V_1$.

Teorem 6.1'in kolay bir sonucudur ki

$$L(G_1) = L(G).$$

Adım 2: İkinci adımda, gerekli olduğunda üretimlerin sağ taraflarının uzunluğunu azaltmak için ek değişkenler tanıtıyoruz. Öncelikle (6.6) formundaki tüm üretimleri ve (6.7) formundaki tüm üretimleri $n = 2$ olanları $\hat{P}$ ye koyuyoruz. $n > 2$ için, yeni değişkenler $D_1, D_2, \dots$ tanıtıyoruz ve $\hat{P}$ ye şu üretimleri koyuyoruz

$$\begin{aligned} A &\rightarrow C_1 D_1, \\ D_1 &\rightarrow C_2 D_2, \\ &\vdots \\ D_{n-2} &\rightarrow C_{n-1} C_n. \end{aligned}$$

Açıkça, elde edilen dilbilgisi $\hat{G}$ Chomsky normal formundadır. Teorem 6.1'in tekrar eden uygulamaları, $L(G_1) = L$ olduğunu gösterecektir $(\hat{G})$, böylece

$$L(\hat{G}) = L(G).$$

Bu biraz gayri resmi argüman kolayca daha kesin hale getirilebilir. Biz bunu okuyucuya bırakacağız. ■

ÖRNEK 6.8

Üretimlerle grameri

$$\begin{aligned} S &\rightarrow ABa, \\ A &\rightarrow aab, \\ B &\rightarrow Ac \end{aligned}$$

Chomsky normal formuna dönüştürün.

Teorem 6.6'nın inşası gereği, gramerin herhangi bir λ -üretimi veya herhangi bir birim üretimi yoktur.

Adım 1'de, yeni değişkenler B_a, B_b, B_c tanıtlıyoruz ve algoritmayı kullanarak elde ediyoruz

$$\begin{aligned} S &\rightarrow ABB_a, \\ A &\rightarrow B_a B_a B_b, \\ B &\rightarrow AB_c, \\ B_a &\rightarrow a, \\ B_b &\rightarrow b, \\ B_c &\rightarrow c. \end{aligned}$$

İkinci adımda, ilk iki üretimi normal forma getirmek için ek değişkenler tanıtlıyoruz ve nihai sonucu elde ediyoruz

$$\begin{aligned} S &\rightarrow AD_1, \\ D_1 &\rightarrow BB_a, \\ &\rightarrow B_a D_2, \\ D_2 &\rightarrow B_a B_b, \\ B &\rightarrow AB_c, \\ B_a &\rightarrow a, \\ B_b &\rightarrow b, \\ &\rightarrow c. \end{aligned}$$

Greibach Normal Formu

Diğer yararlı bir dilbilgisel form Greibach normal formu'dur. Burada, bir üretimin sağ taraflarının uzunluğu üzerinde kısıtlamalar koymuyoruz, ancak terim inallerin ve değişkenlerin görünebileceği pozisyonlar üzerinde kısıtlamalar koyuyoruz. Greibach normal formunu haklı çıkaran argümanlar biraz karmaşık ve çok şeffaf değildir. Benzer şekilde, verilen bir bağlamdan bağımsız dilbilgisine eşdeğer bir Greibach normal formu oluşturmak zahmetlidir. Bu nedenle, bu konuyu çok kısaca ele alıyoruz. Yine de, Greibach normal formunun bir çok teorik ve pratik sonucu vardır.

TANIM 6.5

Bir bağlamdan bağımsız dilbilgisi, tüm üretimlerin aşağıdaki formda olduğu durumlarda Greibach normal formunda olduğu söylenir

$$A \rightarrow ax,$$

burada $a \in T$ ve $x \in V^*$.

Bu durumu Tanım 5.4 ile karşılaştırdığımızda, formun $A \rightarrow ax$ hem Greibach normal formunda hem de s -dilbilgilerinde yaygın olduğunu görüyoruz, ancak Greibach normal formu (A, a) çiftinin en fazla bir kez meydana gelmesi kısıtlamasını taşımamaktadır. Bu ek özgürlük, Greibach normal formuna s -dilbilgilerinin sahip olmadığı bir genel gösterilme kazandırır.

Bir dilbilgisi Greibach normal formunda değilse, yukarıda karşılaşılan bazı tekniklerle bu formda yeniden yazabiliriz. İşte iki basit örnek.

ÖRNEK 6.9

Gramer

$$\begin{aligned} S &\rightarrow AB, \\ A &\rightarrow aA \mid bB \mid b, \\ B &\rightarrow b \end{aligned}$$

Greibach normal formunda değildir. Ancak, Teorem 6.1'de verilen ikameyi kullanarak, hemen eşdeğer dilbilgisini elde ederiz.

$$\begin{aligned} S &\rightarrow aAB \mid bBB \mid bB, \\ A &\rightarrow aA \mid bB \mid b, \\ B &\rightarrow b, \end{aligned}$$

Greibach normal formundadır.

ÖRNEK 6.10

Dilbilgisini

$$S \rightarrow abSb \mid aa$$

Greibach normal formuna dönüştürün.

Burada, Chomsky normal formunun inşasında tanıtilen benzer bir cihaz kullanabiliriz. Yeni değişkenler A ve

B 'yi tanıtlıyoruz; bunlar sırasıyla a ve b için esasen eşanlamlılardır. Terminal sembollerin ilgili değişkenleriyle değiştirilmesi, eşdeğer dilbilgisine yol açar.

$$\begin{aligned} S &\rightarrow aBSB \mid aA, \\ A &\rightarrow a, \\ B &\rightarrow b, \end{aligned}$$

Greibach normal formundadır.

Genel olarak, verilen bir dilbilgisinin Greibach normal formuna dönüştürülmesi veya bunun her zaman yapılabileceğinin kanıtı basit bir mesele değildir. Burada Greibach normal formunu tanıtıyoruz çünkü bu, bir sonraki bölümde önemli bir sonucun teknik tartışmasını basitleştirecektir. Ancak, kavramsal bir bakış açısıyla, Greibach normal formunun tartışmamızda daha fazla bir rolü yoktur, bu nedenle sadece aşağıdaki genel sonucu kanıtsız olarak aktarıyoruz.

TEOREM 6.7

Her $\lambda \in L(G)$ olan bir bağlamdan bağımsız dilbilgisi G için, Greibach normal formunda eşdeğer bir dilbilgisi $\tilde{G}$ vardır.

EGZERSİZLER

1. Dilbilgisini $S \rightarrow aSS \mid a \mid b$ biçiminden Chomsky normal formuna dönüştürün.
2. Dilbilgisini $S \rightarrow aSb \mid Sab \mid ab$ biçiminden Chomsky normal formuna dönüştürün.
3. Dilbilgisini

$$S \rightarrow aSaaA \mid A, A \rightarrow abA \mid bb$$

Chomsky normal formuna dönüştürün.

4. Üretimlere sahip dilbilgisini dönüştürün

$$\begin{aligned} S &\rightarrow baAB, \\ A &\rightarrow bAB \mid \lambda, \\ B &\rightarrow BAa \mid A \mid \lambda \end{aligned}$$

Chomsky normal formuna dönüştürün.

5. Dilbilgisini

$$\begin{aligned} S &\rightarrow AB \mid aB, \\ A &\rightarrow abb \mid \lambda, \\ B &\rightarrow bbA \end{aligned}$$

Chomsky normal formuna dönüştürün.

6. $G = (V, T, S, P)$ herhangi bir λ üretimi veya birim üretimi olmayan bağlamdan bağımsız dilbilgisi olsun. P 'deki herhangi bir üretimin sağında maksimum sembol sayısını k olarak tanımlayın. Chomsky normal formunda, en fazla $(k - 1) \mid P \mid + \mid T \mid$ üretim kuralına sahip eşdeğer bir dilbilgisi olduğunu gösterin.
7. Egzersiz 5 için dilbilgisi bağımlılık grafiğini çizin.

8. Doğrusal bir dil, doğrusal bir dilbilgisi (tanım için, Örnek 3.14'e bakın) bulunan bir dildir. L, λ içermeyen herhangi bir doğrusal dil olsun. Üretimlerinin tümünün aşağıdaki formlardan birine sahip olduğu $G = (V, T, S, P)$ adlı bir dilbilgisi olduğunu gösterin.

$$\begin{aligned} A &\rightarrow aB, \\ A &\rightarrow Ba, \\ A &\rightarrow a, \end{aligned}$$

$a \in T, A, B \in V$ olacak şekilde, $L = L(G)$ olacak şekilde.

9. Her bağlamdan bağımsız dilbilgisi $G = (V, T, S, P)$ için, tüm üretimlerin aşağıdaki formda olduğu eşdeğer bir dilbilgisi olduğunu gösterin.

$$A \rightarrow aBC,$$

veya

$$A \rightarrow \lambda,$$

nerede $a \in \Sigma \cup \{\lambda\}, A, B, C \in V$.

10. Dilbilgisini

$$S \rightarrow aSb|bSa|a|b|ab$$

Greibach normal formuna dönüştürün.

11. Aşağıdaki grameri Greibach normal formuna dönüştürün.

$$S \rightarrow aSb|ab|bb$$

12. Dilbilgisini

$$S \rightarrow ab|aS|aaS|aSS$$

Greibach normal formuna dönüştürün.

13. Dilbilgisini

$$\begin{aligned} S &\rightarrow ABb|a|b \\ A &\rightarrow aaA|B \\ B &\rightarrow bAb \end{aligned}$$

Greibach normal formuna dönüştürün.

14. Her lineer gramer, tüm üretimlerin

şu şekilde görüldüğü bir forma dönüştürülebilir mi. $A \rightarrow ax$, burada $a \in T$ ve $x \in V^* \setminus \{\lambda\}$?

15. Bir bağlamdan bağımsız gramer, tüm üretim

kuralları aşağıdaki deseni sağlıyorsa iki-standart formda olarak kabul edilir:

$$\begin{aligned} A &\rightarrow aBC, \\ A &\rightarrow aB, \\ A &\rightarrow a, \end{aligned}$$

nerede $A, B, C \in V$ ve $a \in T$.

Grameri $G = (\{S, A, B, C\}, \{a, b\}, S, P)$ ile P şu şekilde verilmiştir

$$\begin{aligned} S &\rightarrow aSA, \\ A &\rightarrow bABC, \\ B &\rightarrow bb, \\ C &\rightarrow aBC \end{aligned}$$

iki-standart forma dönüştürün.

- 16.** İki-standart form genel bir formdur; herhangi bir bağlamdan bağımsız dil bilgisi G için $\lambda \in L(G)$ olduğunda, iki-standart formda eşdeğer bir dil bilgisi vardır. Bu iddiayı kanıtlayın.

6.3 BAĞLAMDAN BAĞIMSIZ DİL BİLGİLERİ İÇİN BİR ÜYELİK ALGORİTMASI*

Bölüm 5.2'de, herhangi bir ayrıntı vermeden, bağlamdan bağımsız dil bilgileri için üyelik ve ayrıştırma algoritmalarının var olduğunu iddia ediyoruz ve bunların bir dizeyi w ayrıştırmak için yaklaşık $|w|/3$ adım gerektirdiğini belirtiyoruz. Şimdi bu iddiayı haklı çıkarmak için bir konumdayız. Burada tanımlayacağımız algoritma, kökenini oluşturan J. Cocke, D. H. Younger ve T. Kasami'den dolayı CYK algoritması denir. Algoritma yalnızca dil bilgisi Chomsky normal formundaysa çalışır ve bir problemi aşağıdaki şekilde daha küçük bir dizi probleme ayırarak başlatılır. Chomsky normal formunda bir dil bilgisi $G = (V, T, S, P)$ ve bir dize olduğunu varsayalım

$$w = a_1 a_2 \cdots a_n.$$

Alt dizeleri tanımlıyoruz

$$w_{ij} = a_i \cdots a_j,$$

ve V 'in alt kümesini

$$V_{ij} = \left\{ A \in V : A \xRightarrow{*} w_{ij} \right\}.$$

Açıkça, $w \in L(G)$ yalnızca $S \in V_{1n}$ olduğunda geçerlidir.

V_{ij} 'yi hesaplamak için, $A \in V_{ii}$ yalnızca G bir üretim içeriyorsa gözlemleyin $A \rightarrow a_i$. Bu nedenle, V_{ii} tüm $1 \leq i \leq n$ için hesaplanabilir w ve dil bilgisi üretimlerinin incelenmesiyle. Devam etmek için, $j > i$ olduğunda, $A w_{ij}$ 'yi türetir yalnızca $A \rightarrow BC$ üretimi varsa, $B^* \Rightarrow w_{ik}$ ve $C^* \Rightarrow w_{k+1j}$ için bazı k ile $i \leq k, k < j$. Diğer bir deyişle,

$$V_{ij} = \bigcup_{k \in \{i, i+1, \dots, j-1\}} \{A : A \rightarrow BC, \text{ with } B \in V_{ik}, C \in V_{k+1, j}\}. \quad (6.8)$$

(6.8)'deki indekslerin incelenmesi,

tüm V_{ij} 'yi hesaplamak için kullanılabileceğini gösterir eğer sırayla devam edersek

1. $V_{11}, V_{22}, \dots, V_{nn}$ 'yi hesaplayın,
2. $V_{12}, V_{23}, \dots, V_{n-1,n}$ 'yi hesaplayın,
3. Hesapla $V_{13}, V_{24}, \dots, V_{n-2,n}$,

ve devamı.

ÖRNEK 6.11

String'in $w =$
gramer tarafından

$aabbb$ üretilen dil içindedir

$$\begin{aligned} S &\rightarrow AB, \\ A &\rightarrow BB|a, \\ B &\rightarrow AB|b. \end{aligned}$$

Öncelikle şunu belirtelim ki $w_{11}=a$, bu nedenle V_{11} tüm değişkenlerin kümesidir hemen türet a , yani $V_{11}=\{A\}$. Çünkü $w_{22}=a$, biz de şunu elde ederiz $V_{22}=\{A\}$ ve benzer şekilde,

$$V_{11} = \{A\}, V_{22} = \{A\}, V_{33} = \{B\}, V_{44} = \{B\}, V_{55} = \{B\}.$$

Şimdi (6.8) kullanarak elde ediyoruz

$$V_{12} = \{A : A \rightarrow BC, B \in V_{11}, C \in V_{22}\}.$$

$V_{11}=\{A\}$ ve $V_{22}=\{A\}$ olduğundan, küme, sağ tarafı AA olan bir üretimin sol tarafında yer alan tüm değişkenleri içerir. Hiçbiri yok, V_{12} boş. Sonra,

$$V_{23} = \{A : A \rightarrow BC, B \in V_{22}, C \in V_{33}\},$$

gereken sağ taraf AB , ve bizde $V_{23}=\{S, B\}$. Bu doğrultuda basit bir argüman sunar

$$\begin{aligned} V_{12} &= \emptyset, V_{13} = \{S, B\}, V_{34} = \{A\}, V_{45} = \{A\}, \\ V_{24} &= \{A\}, V_{35} = \{S, B\}, \\ V_{14} &= \{A\}, V_{25} = \{S, B\}, \\ V_{35} &= \{S, B\}, \end{aligned}$$

öyle ki $w \in L(G)$.

CYK algoritması, burada tanımlandığı gibi, Chomsky normal formun da bir dilin üyeliğini belirler. Ögelerin nasıl türetildiğini takip etmek için bazı eklemelerle, bir ayrıştırma yöntemine dönüştürülebilir. CYK üyelik algoritmasının $O(n^3)$ adım gerektirdiğini görmek için, tam olarak $n(n+1)/2$ kümesinin V_{ij} hesaplanması gerektiğine dikkat edin. Her biri (6.8)'de en fazla n terimin değerlendirilmesini içerir, bu nedenle iddia edilen sonuç ortaya çıkar.

EGZERSİZLER

1. CYK algoritmasını kullanarak dizelerin $abb, bbb, aabba$ ve $abbbb$, Örnek 6.11'deki dilin grameri tarafından üretilip üretilmediğini belirleyin.
2. CYK algoritmasını kullanarak dizeyi aab çözümlemek için Örnek 6.11'deki grameri kullanın.
3. Egzersiz 2'de kullanılan yaklaşımı kullanarak, CYK üyelik algoritmasının nasıl bir ayrıştırma yöntemi haline getirilebileceğini gösterin.
4. CYK yöntemini kullanarak, dize $w = aaabbbb$ 'nin dilde olup olmadığını belirleyin gramer tarafından üretilmiştir $S \rightarrow aSb \mid b$.

BÖLÜM

7

PUSHDOWN OTOMATLAR

BÖLÜM ÖZETİ

Regular dillerin tartışmasında, regular dilleri keşfetmenin birkaç yolunu gördük: sonlu otomata, regular gramerler ve regular ifadeler. Bağlamdan bağımsız dilleri bağlamdan bağımsız gramerler aracılığıyla tanımladıktan sonra, bağlamdan bağımsız diller için başka seçeneklerin olup olmadığını soruyoruz. Görünüşe göre regular ifadelerin bir analoğu yoktur, ancak pushdown otomata bağlamdan bağımsız dillerle ilişkili otomatalardır.

Pushdown otomata, esasen bir yığın depolama ile sonlu otomatalardır. Bir yığının tanım gereği sonsuz uzunluğu olduğundan, bu, sınırlı bir belleğin neden olduğu sonlu otomataların sınırlamasını aşar.

Pushdown otomata, eğer belirsiz olmalarına izin verilirse, bağlamdan bağımsız gramerlerle eşdeğerdir. Ayrıca belirleyici pushdown otomata tanımlayabiliriz, ancak onlarla ilişkili dil ailesi, bağlamdan bağımsız dillerin uygun bir alt kümesidir.

Bağlamdan bağımsız dillerin bağlamdan bağımsız gramerler aracılığıyla tanımlanması kullanışlıdır; bu, programlama dili tanımında BN F kullanımında gösterilmiştir. Bir sonraki soru, bağlamdan bağımsız dillerle ilişkilendirilebilecek bir otomata sınıfının olup olmadığıdır. Gördüğümüz gibi, sonlu otomatalar tüm bağlamdan bağımsız dilleri tanıyamaz.

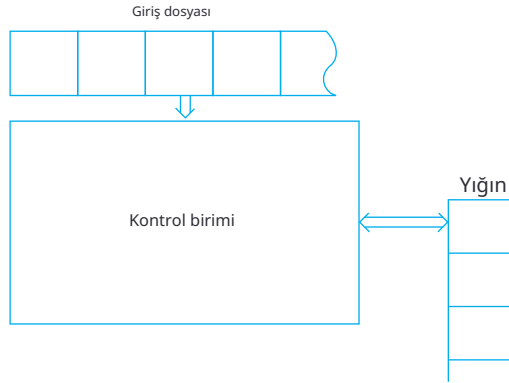
Sezgisel olarak, bunun nedeni sonlu otomataların kesinlikle sonlu bellikleri olmasıdır; oysa bir bağlamdan bağımsız dilin tanınması, sınırsız miktarda bilgi depolamayı gerektirebilir. Örneğin, dilin bir dizesini tararken $L = \{a^n b^n : n \geq 0\}$, yalnızca tüm a 'ların ilk b 'den önce geldiğini kontrol etmekle kalmayıp, aynı zamanda a 'ların sayısını da saymalıyız. Çünkü n sınırsızdır, bu sayım sonlu bir bellekle yapılamaz.

Sınırsız sayım yapabilen bir makine istiyoruz. Ancak diğer örneklerden, örneğin $\{ww^R\}$, gördüğümüz gibi, sınırsız sayım yeteneğinden daha fazlasına ihtiyacımız var: Sembollerin bir dizisini ters sırayla depolama ve eşleştirme yeteneğine ihtiyacımız var. Bu, sınırsız depolama sağlayan ancak bir yığın gibi çalışmakla sınırlı olan bir depolama mekanizması olarak bir yığın denememiz gerektiğini önermektedir. Bu, bize yığın otomatları (pda) olarak adlandırılan bir makine sınıfı verir.

Bu bölümde, itme otomata ile bağlamdan bağımsız diller arasında aki bağlantıyı inceliyoruz. Öncelikle, itme otomatalarının belirsiz bir şekilde hareket etmesine izin verirse, tam olarak bağlamdan bağımsız diller ailesini kabul eden bir otomata sınıfı elde ettiğimizi gösteriyoruz. Ancak burada, deterministik ve belirsiz versiyonlar arasında artık bir eşdeğerlik olmadığını da göreceğiz. Deterministik itme otomata sınıfı, bağlamdan bağımsız dillerin uygun bir alt kümesini oluşturan yeni bir dil ailesini tanımlar. Bu, programlama dillerinin ele alınması için önemli bir aile olduğundan, bölümü deterministik bağlamdan bağımsız dillerle ilişkili gramerlere kısa bir girişle sonlandırıyoruz.

7.1 BELİRSİZ İTME OTOMATALARI

Bir itme otomatasının şematik temsili Şekil 7.1'de verilmiştir. Kontrol biriminin her hareketi, giriş dosyasından bir sembol okurken, aynı zamanda yığın içeriğini de alışıldık şekilde değiştirir.



ŞEKİL 7.1

yığın işlemleri. Kontrol biriminin her hareketi, mevcut giriş sembolü ile yığının en üstündeki sembol tarafından belirlenir. Hareketin sonucu, kontrol biriminin yeni bir durumu ve yığının en üstündeki bir değişikliktir.

Bir Pushdown Otomatonun Tanımı

Bu sezgisel kavramı biçimlendirmek, bize bir pushdown

otomatonun kesin tanımını verir.

TANIM 7.1

Bir belirsiz pushdown kabul edici (npda) sep-

tuple ile tanımlanır

$$M = (Q, \Sigma, \Gamma, \delta, q_0, z, F),$$

nerede

Q kontrol biriminin sonlu iç durumlar kümesidir,

Σ girdi alfabesidir,

Γ , yığın alfabesi olarak adlandırılan sonlu semboller kümesidir,

$\delta : Q \times (\Sigma \cup \{\lambda\}) \times \Gamma \rightarrow Q \times \Gamma^*$ sonlu alt küme kümesinin geçiş

fonksiyonudur,

$q_0 \in Q$ kontrol biriminin başlangıç durumudur,

$z \in \Gamma$ yığın başlangıç sembolüdür,

$F \subseteq Q$ son durumlar kümesidir.

δ 'nın tanım aralığının karmaşık resmi daha yakından incelenmeyi gerektirir. δ 'nın argümanları kontrol biriminin mevcut durumu, mevcut girdi sembolü ve yığının üstündeki mevcut semboldür.

Sonuç, (q, x) çiftleri kümesidir; burada q kontrol biriminin bir sonraki durumu ve x daha önce orada bulunan tek sembolün yerine yığının üstüne konulan bir dizgedir. δ 'nın ikinci argümanının λ olabileceğini unutmayın; bu, bir girdi sembolü tüketen bir hareketin mümkün olduğunu gösterir. Bu tür bir hareketi λ -geçiş olarak adlandıracağız. Ayrıca, δ 'nın bir yığın sembolü gerektirecek şekilde tanımlandığını unutmayın; yığın boşsa hiçbir hareket mümkün değildir. Son olarak, δ 'nın aralığının elemanlarının sonlu bir alt küme olması gerekliliği önemlidir çünkü $Q \times \Gamma^*$ sonsuz bir kümedir ve dolayısıyla sonsuz alt kümelere sahiptir. Bir npda hareketleri için birkaç seçeneğe sahip olabilir, ancak bu seçim, sınırlı bir olasılık kümesi ile kısıtlanmalıdır.

ÖRNEK 7.1

Bir npda'nın geçiş kuralları kümesinin içerdiğini varsayalım

$$\delta(q_1, a, b) = \{(q_2, cd), (q_3, \lambda)\}.$$

Herhangi bir zamanda kontrol birimi durum q_1 'de ise, okunan girdi sembolü a ise ve yığının üstündeki sembol b ise, o zaman iki şeyden biri olabilir: (1) kontrol birimi durum q_2 'ye geçer ve dize cd b 'yi yığının üstünde değiştirir, veya (2) kontrol birimi durum q_3 'e geçer ve sembol b yığının üstünden kaldırılır. Notasyonumuzda bir dizinin yığına eklenmesinin sembol sembol olarak, dizinin sağ ucundan başlayarak yapıldığını varsayıyoruz.

ÖRNEK 7.2

Bir npda düşünün

$$Q = \{q_0, q_1, q_2, q_3\},$$

$$\Sigma = \{a, b\},$$

$$\Gamma = \{\lambda, 1\},$$

$$z = 0,$$

$$F = \{q_3\},$$

ilk durum ile q_0 ve

$$\delta(q_0, a, 0) = \{(q_1, 10), (q_3, \lambda)\},$$

$$\delta(q_0, \lambda, 0) = \{(q_3, \lambda)\},$$

$$\delta(q_1, a, 1) = \{(q_1, 11)\},$$

$$\delta(q_1, b, 1) = \{(q_2, \lambda)\},$$

$$\delta(q_2, b, 1) = \{(q_2, \lambda)\},$$

$$\delta(q_2, \lambda, 0) = \{(q_3, \lambda)\}.$$

Bu otomatonun eylemi hakkında ne söyleyebiliriz?

Öncelikle, geçişlerin tüm olası giriş ve yığın sembolleri kombi nasyonları için belirtilmediğini fark edin. Örneğin, $\delta(q_0, b, 0)$ için bir giriş verilmemiştir. Bunun yorumu, belirsiz geçişin boş küme olduğu ve npda için ölü bir yapılandırmayı temsil ettiği şeklindedir.

Önemli geçişler şunlardır

$$\delta(q_1, 1) = \{(q_1, 11)\},$$

bir a okunduğunda yığına 1 ekleyen ve

$$\delta(q_2, b, 1) = \{(q_2, \lambda)\},$$

bir b ile karşılaşıldığında 1 çıkaran. Bu iki adım, a 'ların sayısını sayar ve bu sayıyı b 'lerin sayısı ile karşılaştırır. Kontrol birimi, ilk b ile karşılaşana kadar q_1 durumundadır; o zaman q_2 durumuna geçer. Bu, son a 'dan önce hiçbir b 'nin gelmediğini garanti eder. Kalan geçişleri analiz ettikten sonra, npda'nın yalnızca girdi dizesi dildeyse q_3 son durumunda sona ereceğini görüyoruz.

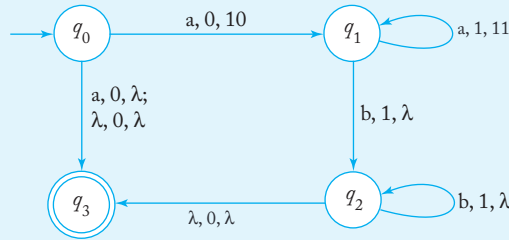
$$L = \{a^n b^n : n \geq 0\} \cup \{a\}.$$

Sonlu otomata ile bir benzetme yaparsak, npda'nın yukarıdaki dili kabul ettiğini söyleyebiliriz. Elbette, böyle bir iddiada bulunmadan önce, npda'nın bir dili kabul etmesiyle neyi kastettiğimizi tanımlamalıyız.

npda'ları temsil etmek için geçiş grafikleri de kullanabiliriz. Bu temsil biçiminde, grafiğin kenarlarını üç şekilde etiketleriz: mevcut girdi sembolü, yığının üstündeki sembol ve yığının üstünü değiştiren dize.

ÖRNEK 7.3

Örnek 7.2'deki npda, Şekil 7.2'deki geçiş grafi ile temsil edilmektedir.



ŞEKİL 7.2

Geçiş grafikleri npda'ları tanımlamak için uygun olsa da, argümanlar oluşturmak için daha az faydalıdır. Hem iç durumları hem de yığın içeriğini takip etmemiz gerektiği gerçeği, geçiş grafiklerinin resmi akıl yürütme için faydasını sınırlar. Bunun yerine, bir npda'nın ardışık yapılandırmalarını tanımlamak için özlü bir notasyon sunuyoruz.

bir dize işlenirken. Herhangi bir zamanda ilgili faktörler, kontrol biriminin mevcut durumu, okunmamış girdi dizesinin kısmı ve yığının mevcut içeriğidir. Bunlar birlikte npda'nın ilerleyebileceği tüm olası yolları tamamen belirler. Üçlü

$$(q, w, u),$$

burada q kontrol biriminin durumu, w okunmamış girdi dizesinin kısmı ve u yığın içeriğidir (en soldaki sembol yığının üstünü gösterir), bir anlık tanım olarak adlandırılır pushdown otomatu. Bir anlık tanımdan diğerine geçiş, sembol- ile gösterilecektir; böylece

$$(q_1, aw, bx) \vdash (q_2, w, yx)$$

mümkündür eğer ve yalnızca eğer

$$(q_2) \in \delta(q, a, b).$$

Rastgele adım sayısını içeren hareketler ifade

$\vdash^*$

$$(q_1, w_1, x) \vdash^* (q_2, w_2, x_2)$$

bir dizi adımda olası bir yapılandırma değişikliğini gösterir.¹ Bir den fazla otomatonun dikkate alındığı durumlarda, belirli otomaton M tarafından hareketin yapıldığını vurgulamak için $\vdash^*_M$ kullanacağız.

Bir Pushdown Otomatonu Tarafından Kabul Edilen Dil

TANIM 7.2

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$ bir belirsiz pushdown otomatonu olsun. M tarafından kabul edilen dil, kümedir

$$L(M) = \left\{ w \in \Sigma^* : (q_0, w, z) \vdash^*_M (p, \lambda, u), p \in F, u \in \Gamma^* \right\}.$$

Kelime olarak, M tarafından kabul edilen dil, yerleştirilebilecek tüm dizelerin kümesidir. M bir dize sonunda nihai bir duruma. Nihai yığın içeriği u bu kabul tanımına ilişkin değildir.

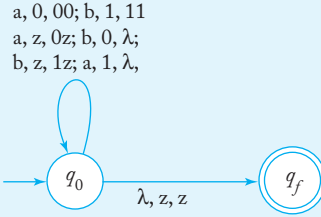
¹Belirsizlik nedeniyle, böyle bir değişiklik elbette gerekli değildir.

ÖRNEK 7.4

Bir dil için npda oluşturun

$$L = \{w \in \{a, b\}^* : n_a(w) = n_b(w)\}.$$

Örnek 7.2'de olduğu gibi, bu problemin çözümü, a ve b'nin sayısını saymayı içerir, bu da bir yığın ile kolayca yapılır. Burada a'ların ve b'lerin sırası hakkında endişelenmemize bile gerek yok. Bir a okunduğunda yığına bir sayıcı sembol, diyelim ki 0, ekleyebiliriz, ardından bir b bulunduğunda yığından bir sayıcı sembolü çıkarabiliriz. Bunun tek zorluğu, w'nin a'lardan daha fazla b'ye sahip bir ön eki varsa, kullanacak bir a0 bulamayacağız. Ama bu kolayca düzeltilebilir; eşleştirecek b'leri saymak için negatif bir sayıcı sembol, diyelim ki 1, kullanabiliriz. Tam çözüm, Şekil 7.3'teki geçiş grafiğinde verilmiştir.



ŞEKİL 7.3

Dizgiyi *baab* işlerken, npda hareketleri yapar

$$\begin{aligned} (q_0, baab, z) &\vdash (q_0, aab, 1z) \vdash (q_0, ab, z) \\ &\vdash (q_0, b, 0z) \vdash (q_0, \lambda, z) \vdash (q_f, \lambda, z) \end{aligned}$$

ve bu nedenle dizgi kabul edilir.

ÖRNEK 7.5

Dili kabul eden bir npda oluşturmak için

$$L = \{ww^R : w \in \{a, b\}^+\},$$

sembollerin bir yığın (stack) üzerinden ters sırayla alındığı gerçeğini kullanıyoruz. Dizginin ilk kısmını okurken, ardışık sembolleri yığına itiyoruz. İkincikısımda, mevcut giriş sembolünü yığının tepe kısmındaki sembol ile karşılaştırıyoruz,

iki sembol eşleştiği sürece devam ediyoruz. Semboller, yığına itil dikleri sıranın tersine alınırsa, tam bir eşleşme yalnızca girişin w^R biçiminde olması durumunda sağlanır.

Bu önerideki belirgin bir zorluk, dizginin ortasını bilmememizdir; yani, nerede w biter ve w^R başlar. Ancak otomatonun belirsiz doğası bu konuda bize yardımcı olur; npda ortanın neresi olduğunu doğru bir şekilde tahmin eder ve o noktada durum değiştirir. Bir çözüm, problemi şu şekilde verir: $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$, burada

$$\begin{aligned} Q &= \{q_0, q_1, q_2\}, \\ \Sigma &= \{a, b\}, \\ \Gamma &= \{a, b, z\}, \\ F &= \{q_2\}. \end{aligned}$$

Geçiş fonksiyonu birkaç parçadan oluştuğu şekilde görselleştirilebilir: yığına w 'yi itmek için bir küme,

$$\begin{aligned} \delta(q_0, a, a) &= \{(q_0, aa)\}, \\ \delta(q_0, b, a) &= \{(q_0, ba)\}, \\ \delta(q_0, a, b) &= \{(q_0, ab)\}, \\ \delta(q_0, b, b) &= \{(q_0, bb)\}, \\ \delta(q_0, a, z) &= \{(q_0, az)\}, \\ \delta(q_0, b, z) &= \{(q_0, bz)\}, \end{aligned}$$

bir küme, dizinin ortasını tahmin etmek için, burada npda durumdan q_0 'dan q_1 'e geçer,

$$\begin{aligned} \delta(q_0, \lambda, a) &= \{(q_1, a)\}, \\ \delta(q_0, \lambda, b) &= \{(q_1, b)\}, \end{aligned}$$

bir küme eşleştirmek için w^R yığının içeriğine karşı,

$$\begin{aligned} \delta(q_1, a, a) &= \{(q_1, \lambda)\}, \\ \delta(q_1, b, b) &= \{(q_1, \lambda)\}, \end{aligned}$$

ve nihayet

$$\delta(q_1, \lambda, z) = \{(q_2, z)\},$$

başarılı bir eşleşmeyi tanımak için.

abba kabul etme hareketleri dizisi

$$\begin{aligned} (q_0, abba, z) &\vdash (q_0, bba, az) \vdash (q_0, ba, baz) \\ &\vdash (q_1, ba, baz) \vdash (q_1, a, az) \vdash (q_1, \lambda, z) \vdash (q_2, z). \end{aligned}$$

String'in ortasını bulmak için belirsiz alternatif, üçüncü hamlede alınır. O aşamada, pda anlık tanımlara sahiptir (q_0, ba, baz) ve bir sonraki hamlesi için iki seçeneği vardır. Biri $\delta(q_0, b, b) = \{(q_0, bb)\}$ kullanmak ve hamle yapmak

$$, ba, baz \vdash (q_0, a, bbaz),$$

ikincisi yukarıda kullanılan, yani $\delta(q_0, \lambda, b) = \{(q_1, b)\}$. Sadece ikinci si girişi kabul etmeye götürür.

ALISTIRMALAR

1. Bir pda bulun ki dil $L = \{a^n b^{2n} : n \geq 1\}$
2. Egzersiz 1'deki pda tarafından $aabbbb$ kabulü için anlık tanımların dizisini gösterin.
3. Aşağıdaki düzenli dilleri kabul eden npda'lar oluşturun:
 - (a) $L_1 = L(aaa^*bab)$.
 - (b) $L_2 = L(aab^*aba^*)$.
 - (c) L_1 ve L_2 'nin birleşimi.
 - (d) $L_1 - L_2$.
 - (e) L_1 ve L_2 'nin kesişimi.
4. Örnek 7.2'deki pda ile aynı dili kabul eden dört durumdan daha az duruma sahip bir pda bulun.
5. Örnek 7.5'teki pda'nın, içinde bulunmayan herhangi bir dizeyi kabul etmediğini kanıtlayın. $\{ww^R\}$.
6. Şu dilleri kabul eden npda'lar oluşturun $\Sigma = \{a, b, c\}$:
 - (a) $L = \{a^n b^{3n} : n \geq 0\}$.
 - (b) $L = \{w c w^R : w \in \{a, b\}^*\}$.
 - (c) $L = \{a^n b^m c^{n+m} : n \geq 0, m \geq 0\}$.
 - (d) $L = \{a^n b^{n+m} c^m : n \geq 0, m \geq 1\}$.
 - (e) $L = \{a^3 b^n c^n : n \geq 0\}$.
 - (f) $L = \{b^m : n \leq m \leq 3n\}$.
 - (g) $L = \{w : n_a(w) = n_b(w)\}$.
 - (h) $L = \{w : n_a(w) = 2\}$.
 - (i) $L = \{w : n_a(w) + n_b(w) = n_c(w)\}$.

$$(j) \quad L = \{w : 2n_a(w) \leq n_b(w) \leq 3n_a(w)\}.$$

$$(k) \quad L = \{w : n_a(w) < n_b(w)\}.$$

7. Bir npda oluşturun, bu dil $L = \{a^n b^m : n \geq 0, n \neq m\}$.

8. $\Sigma = \{a, b, c\}$ üzerinde bir npda bulun, bu dil

$$L = \{w_1 c w_2 : w_1, w_2 \in \{a, b\}^*, w_1 \neq w_2^R\}.$$

9. $L(a^*)$ ve Egzersiz 8'deki dilin birleştirilmesi için bir npda bulun.

10. Bir npda bulunuz dil için $L = \{ab(ab)^n ba(ba)^n : n \geq 0\}$.

11. DFA'nın PDA ile aynı dili kabul eden bir DFA bulmak mümkün mü?

$$M = (\{q_0, q_1\}, \{a, b\}, \{z\}, \delta, q_0, z, \{q_1\}),$$

ile

$$\delta(q_0, a, z) = \{(q_1, z)\},$$

$$\delta(q_0, b, z) = \{(q_0, z)\},$$

$$\delta(q_1, a, z) = \{(q_1, z)\},$$

$$\delta(q_1, b, z) = \{(q, z)\}?$$

12. PDA tarafından hangi dil kabul edilmektedir?

$$M = (\{q_0, q_1, q_2, q_3, q_4, q_5, z\}, \delta, q_0, z),$$

ile

$$\delta(q_0, b, z) = \{(q_1, 1z)\},$$

$$\delta(q_1, b, 1) = \{(q_1, 11)\},$$

$$\delta(q_2, a, 1) = \{(q_3, \lambda)\},$$

$$\delta(q_3, a, 1) = \{(q_4, \lambda)\},$$

$$\delta(q_4, a, z) = \{(q_4, z), (q_5, z)\}?$$

13. NPDA tarafından hangi dil kabul edilmektedir $M = (\{q_0, q_1, q_2\}, \{a, b\}, \{a, b, z\}, q_0, z, \{q_2\})$.

$$\delta(q_0, a, z) = \{(q_1, a, (q, \lambda))\},$$

$$\delta(q_1, b, a) = \{(q_1, b)\},$$

$$\delta(q_1, b, b) = \{(q_1, b)\},$$

$$\delta(q_1, a, b) = \{(q_2, \lambda)\}?$$

14. Örnek 7.4'te NPDA'nın hangi dili kabul ettiğini öğrenmek için $F = \{q_0, q_f\}$ kullanırsak?

15. Yukarıdaki 13. Egzersizde npda tarafından kabul edilen dil nedir eğer $F = \{q_0, q_1, q_2\}$?

16. İki iç durumu aşmayan bir npda bulun ve bu dilin kabul edildiğini gösterin $L(aa^*^*)$.

17. Örneğin 7.2'de verilen değeri $\delta(q_2, \lambda, 0)$ ile değiştirirsek

$$\delta(q_2, \lambda, 0) = \{(q_0, \lambda)\}.$$

Bu yeni PDA tarafından kabul edilen dil nedir?

18. Bir kısıtlı npda'yı, her hareketinde yığın uzunluğunu en fazla bir sembol artırabilen olarak tanımlayabiliriz, Tanım 7.1'i şu şekilde değiştirerek

$$\delta : Q \times (\Sigma \cup \{\lambda\}) \times \Gamma \rightarrow 2^{Q \times (\Gamma \cup \Gamma \cup \{\lambda\})}.$$

Bunun yorumu, δ 'nin aralığının, (q_i, ab) , (q_i, a) veya (q_i, λ) biçimindeki çiftler kümesinden oluştuğudur. Her npda M için, böyle bir kısıtlı npda M' 'nin var olduğunu gösterin ki $L(M) = L(M')$.

19. Dil kabulü için Tanım 7.2'ye bir alternatif, giriş dizesinin sonuna ulaşıldığında yığının boş olmasını gerektirmektir. Resmi olarak, bir npda M , yığın boş olduğunda dil $N(M)$ kabul eder denir eğer

$$N(M) = \left\{ w \in \Sigma^* : (q_0, w, z) \vdash^* p, \lambda, \lambda \right\},$$

p , Q 'daki herhangi bir eleman ise. Bu kavramın, herhangi bir npda M için, $L(M) = N^*M$ olacak şekilde bir npda M' 'nin var olduğu anlamında Tanım 7.2 ile etkili olarak eşdeğer olduğunu gösterin. (M) , ve tersine.

7.2 PUSHDOWN OTOMATLARI VE KONTEXT-SİZ DİLLER

Önceki bölümdeki örneklerde, bazı tanıdık kontext-siz diller için pushdown otomatalarının var olduğunu gördük. Bu bir tesadüf değil. Kontext-siz diller ile belirsiz pushdown kabul edicileri arasında genel bir ilişki vardır ve bu, sonraki iki ana sonuçta ortaya konulmuştur. Her bağlamdan bağımsız dil için bir npda'nın bunu kabul ettiğini ve tersine, herhangi bir npda tarafından kabul edilen dilin bağlamdan bağımsız olduğunu göstereceğiz.

Bağlamdan Bağımsız Diller için Yığın Otomatları

Öncelikle, her bağlamdan bağımsız dil için bir npda'nın bunu kabul ettiğini gösteriyoruz. Temel fikir, dildeki herhangi bir dize için sol en soldan türetme gerçekleştirebilen bir npda inşa etmektir. Argümanı bira z basitleştirmek için, dilin Greibach normal formunda bir gramer tarafından üretildiğini varsayıyoruz.

Şimdi inşa edeceğimiz pda, cümle biçiminin sağ kısmındaki değişkenleri yığında tutarak türetimi temsil edecekken, tamamen terminallerden oluşan sol kısmın, okunan girdi ile aynı olmasını sağlayacaktır.

Yığın üzerinde başlangıç sembolünü koyarak başlıyoruz. Bunun ardından, bir üretimin uygulanmasını simüle etmek için $A \rightarrow ax$, yığın üzerinde değişken A en üstte olmalı ve terminal a girdi sembolü olarak bulunmalıdır. Yığındaki değişken kaldırılır ve değişken dizisi x ile değiştirilir. Bunu başarmak için δ ne olmalı, görmek kolaydır. Genel argümanı sunmadan önce, basit bir örneğe bakalım.

ÖRNEK 7.6

Bir dilin kabul edildiği bir pda oluşturun

bir dilbilgisi ile üretimlerle

$$S \rightarrow aSbb|a.$$

Öncelikle grameri Greibach normal formuna dönüştürüyoruz, üretimleri değiştirerek

$$\begin{aligned} S &\rightarrow aSA|a, \\ A &\rightarrow bB, \\ B &\rightarrow b. \end{aligned}$$

Karşılık gelen otomatonun üç durumu olacaktır $\{q_0, q_1, q_2\}$, ile ilk durum q_0 ve son durum q_2 . Öncelikle, başlangıç sembolü S konulur yığın tarafından

$$\delta(q_0, \lambda, z) = \{(q_1, Sz)\}.$$

Üretim $S \rightarrow aSA$, yığınının S 'yi çıkararak ve yerine SA koyarak pda'da simüle edilecektir, bu sırada a okunacaktır. girdi. Benzer şekilde, kural $S \rightarrow a$ pda'nın bir a okumasına neden olmalıdır basitçe S kaldırırken. Böylece, iki üretim pda'da

şu şekilde temsil edilir

$$\delta(q_1, a, S) = \{(q_1, SA), (q_1, \lambda)\}.$$

Benzer bir şekilde, diğer üretimler verir

$$\begin{aligned} \delta(q_1, b, A) &= \{(q_1, B)\}, \\ \delta(q_1, b, B) &= \{(q_1, \lambda)\}. \end{aligned}$$

Yığın başlangıç sembolünün yığının üstünde görünmesi, türetimin tamamlandığını ve pda'nın son durumuna geçtiğini gösterir.

$$\delta(q, \lambda, z) = \{(q, \lambda)\}.$$

Bu örneğin yapısı diğer durumlara uyarlanabilir ve genel bir sonuca yol açar.

TEOREM 7.1

Herhangi bir bağlamdan bağımsız dil L için, öyle bir npda M vardır ki

$$L = L(M).$$

Kanıt: Eğer L bir λ içermeyen bağlamdan bağımsız dil ise, bunun için Greibach normal formunda bir bağlamdan bağımsız gramer vardır. Let $G = (V, T, S, P)$ bu gramer. Daha sonra, bu gramerde en soldaki türetmeleri simüle eden bir npda inşa ederiz. Önerildiği gibi, simülasyon, cümle biçiminin işlenmiş kısmının yığında olduğu ve herhangi bir cümle biçiminin terminal önekinin giriş dizesinin karşılık gelen ön ekiyle eşleştiği şekilde yapılacaktır.

Özellikle, npda olacak

$$M = (\{q_0, q_1, q_f\}, T, V \cup \{z\}, \delta, q_0, z, \{q_f\}),$$

burada $z \notin V$. M 'nin girdi alfabesinin G 'nin terminal kümesiyle aynı olduğunu ve yığın alfabasının gramerin değişkenler kümesini içerdiğini unutmayın.

Geçiş fonksiyonu şunları içerecektir

$$\delta(q_0, \lambda, z) = \{(q_1, Sz)\}, \quad (7.1)$$

ilk hareketten sonra M , yığının türetimin başlangıç sembolü S içerdiğinden emin olmak için. (Yığının başlangıç sembolü z , türetimin sonunu tespit etmemizi sağlamak için bir işarettir.) Ayrıca, geçiş kuralları kümesi öyle ki

$$(q_1, u) \in \delta(q_1, a, A), \quad (7.2)$$

her zaman

$$A \rightarrow au$$

is $in P$. Bu, girişi a okur ve değişkeni A yığınınından kaldırır,

yerine u ile değiştirir. Bu şekilde, tüm türevleri simüle etmesine olanak tanıyan geçişleri üretir.

Son olarak, elimizde

$$\delta(q_1, \lambda, z) = \{(q_f, z)\}, \quad (7.3)$$

son duruma M ulaşmak için.

Herhangi bir $w \in L(G)$ kabul ettiğini göstermek için, kısmi en soldaki

türetimi dikkate alalım

$$\begin{aligned} S &\xRightarrow{*} a_1 a_2 \cdots a_n A_1 A_2 \cdots A \\ &\Rightarrow a_1 a_2 \cdots a_n b B_1 \cdots B_k A_2 \cdots A \end{aligned}$$

If M bu türevi simüle edecekse, o zaman $a_1a_2 \cdots a_n$ okuduktan sonra, yığın şu şekide olmalıdır $A_1A_2 \cdots A_m$. Türevin bir sonraki adımını almak için, G bir üretime sahip olmalıdır

$$A_1 \rightarrow bB_1 \cdots B_k.$$

Ancak yapı öyle ki, o zaman M bir geçiş kuralına sahiptir ki

$$(q_1, B \quad \quad \quad) \in \delta(q_1, b, A_1),$$

şu şekilde ki yığın artık $B_1 \cdots B_k A_2 \cdots A_m a_1 a_2 \cdots a_n b$ okuduktan sonra içer mektedir.

Bir basit indüksiyon argümanı, türetimdeki adım sayısını gösterir sonra eğer

$$S \xRightarrow{*} w,$$

o zaman

$$(q_1, w, Sz) \vdash^* (q_1, \lambda, z).$$

(7.1) ve (7.3) kullanarak şunları elde ederiz

$$(q_0, w, z) \vdash (q_1, w, Sz) \vdash^* (q, \lambda, z) \vdash (q, \lambda, z),$$

öyle ki $L(G) \subseteq L(M)$.

Şunu kanıtlamak için $L(M) \subseteq L(G)$, $w \in L(M)$ alalım. O zaman tanıma göre

$$(q_0, w, z) \vdash^* (q_f, \lambda, u).$$

Ancak q_0 'dan q_1 'e geçmenin sadece bir yolu ve q_1 'den

q_f 'ye geçmenin de sadece bir yolu vardır. Bu nedenle, şunları elde etmeliyiz

$$(q_1, w, Sz) \vdash^* (q_1, \lambda, z).$$

Şimdi $w = a_1a_2a_3 \cdots a_n$ olarak yazalım. O zaman

$$(q_1, a_1a_2a_3 \cdots a_n, Sz) \vdash^* (q_1, \lambda, z) \quad (7.4)$$

ilk adım (7.2) biçiminde bir kural olmalıdır ki

$$(q_1, a_1a_2a_3 \cdots a_n, Sz) \vdash (q_1, a_2a_3 \cdots a_n, u_1z).$$

Ancak o zaman dilbilgisi $S \rightarrow a_1u_1$ biçiminde bir kurala sahiptir, böylece

$$S \Rightarrow a \quad .$$

Bunu tekrarlayarak $u_1 = Au_2$ yazıyoruz, şunları elde ederiz

$$(q_1, a_2a_3 \cdots a_n, Au_2z) \vdash (q_1, a_3 \cdots a_n, u_3u_2z),$$

bu, $A \rightarrow a_2u_3$ 'ün dilbilgisi içinde olduğunu ve

$$S \xrightarrow{*} a_1a_2u_3u_2.$$

Bu, yığın içeriğinin (z hariç) her noktada cümle biçiminin eşleşmeyen kısmıyla aynı olduğunu oldukça net bir şekilde ortaya koyar, böylece (7.4) şunu ima eder

$$S \xrightarrow{*} a_1a_2 \cdots a_n.$$

Sonuç olarak, $L(M) \subseteq L(G)$, dil λ içermiyorsa kanıtı tamamlar.

Eğer $\lambda \in L$ ise, inşa edilen npda'ya geçiş ekliyoruz

$$\delta(q_0, \lambda, z) = \{q_f, z\}$$

boş dize de kabul edilecek şekilde. ■

ÖRNEK 7.7

Dilbilgisini düşünün

$$\begin{aligned} S &\rightarrow aA, \\ A &\rightarrow aABC \mid bB \mid a, \\ B &\rightarrow b, \\ C &\rightarrow c. \end{aligned}$$

Dilbilgisi zaten Greibach normal formunda olduğundan, önceki teoremin inşasını hemen kullanabiliriz. Kurallara ek olarak

$$\delta(q_0, \lambda, z) = \{(q_1, Sz)\}$$

ve

$$\delta(q_1, \lambda, z) = \{(q_f, z)\},$$

pda'nın geçiş kuralları da olacak

$$\begin{aligned} \delta(q_1, a, S) &= \{(q_1, A)\}, \\ \delta(q_1, a, A) &= \{(q_1, ABC), (q_1, \lambda)\}, \\ \delta(q_1, b, A) &= \{(q_1, B)\}, \\ \delta(q_1, b, B) &= \{(q_1, \lambda)\}, \\ \delta(q_1, c, C) &= \{(q_1, \lambda)\}. \end{aligned}$$

M'nin aaabc'yi işlerken yaptığı hareketlerin sırası

$$\begin{aligned}
 (q_0, aaabc, z) &\vdash (q_1, aaabc, Sz) \\
 &\vdash (q_1, abc, Az) \\
 &\vdash (q_1, abc, ABCz) \\
 &\vdash (q_1, bc, BCz) \\
 &\vdash (q_1, c, Cz) \\
 &\vdash (q_1, \lambda, z) \\
 &\vdash (q_f, \lambda, z).
 \end{aligned}$$

Bu, türetime karşılık gelir

$$S \Rightarrow aA \Rightarrow aaABC \Rightarrow aaaBC \Rightarrow aaabC \Rightarrow aaabc.$$

Argümanları basitleştirmek için, Teorem 7.1'deki kanıtın gramerin Greibach normal formunda olduğu varsayılmıştır. Bunu yapmak zorunda değiliz; general bir bağlamdan bağımsız gramerden benzer ve yalnızca biraz daha karmaşık bir yapı oluşturabiliriz. Örneğin, aşağıdaki biçimdeki üretimler için

$$A \rightarrow Bx,$$

yığınlardan A'yı kaldırır ve Bx ile değiştiririz, ancak hiçbir girdi

sembolü tüketmeyiz. Aşağıdaki biçimdeki üretimler için

$$A \rightarrow abCx,$$

öncelikle girdi içindeki ab'yi yığındaki benzer bir dize ile eşleştirmeliyiz ve ardından A'yı Cx ile değiştirmeliyiz. Yapının detaylarını ve ilişkili kanıtı bir alıştırmaya bırakıyoruz.

Yığın Otomatları için Bağlamdan Bağımsız Gramerler

Teorem 7.1'in tersi de doğrudur. İlgili yapı hemen kendini önermektedir: Teorem 7.1'deki süreci tersine çevirerek dilbilgisinin pda'nın hareketlerini simüle etmesini sağlayın. Bu, yığın içeriğinin cümle biçiminin değişken kısmında yansıtılması gerektiği anlamına gelirken, işlenmiş girdi cümle biçiminin terminal ön ekidir. Bunu çalıştırmak için oldukça fazla ayrıntıya ihtiyaç vardır.

Tartışmayı mümkün olduğunca basit tutmak için, söz konusu npda'nın aşağıdaki gereksinimleri karşıladığını varsayacağız:

1. Tek bir son durum q_f vardır ve bu yalnızca yığın boş olduğunda girilir;

2. Bir $a \in \Sigma \cup \{\lambda\}$, tüm geçişler $\delta(q_i, a, A) = \{c_1, c_2, \dots, c_n\}$ biçiminde olmalıdır, burada

$$c_i = (q_j, \lambda), \quad (7.5)$$

veya

$$c_i = (q_j, BC). \quad (7.6)$$

Yani, her hareket yığın içeriğini bir sembol ile ya artırır ya da azaltır.

Bu kısıtlamalar çok sert görünebilir, ancak öyle değildir.

Herhangi bir npda için, 1 ve 2 özelliklerine sahip eşdeğer bir npda'nın var olduğu gösterilebilir. Bu eşdeğerlik, Bölüm 7.1'deki Alıştırmalar 18 ve 19'da kısmen incelenmiştir. Burada bunu daha fazla keşfetmemiz gerekiyor, ancak yine de argümanları bir alıştırmaya bırakacağız (bu bölümün sonunda Alıştırma 17'ye bakın). Bunu verili olarak alarak, şimdi npda tarafından kabul edilen dil için bir bağlamdan bağımsız dil bilgisi oluşturuyoruz.

Belirtildiği gibi, cümle biçiminin yığın içeriğini temsil etmesini istiyoruz. Ancak npda'nın yapılandırması aynı zamanda bir iç durumu da içerir ve bu da cümle biçiminde hatırlanmalıdır. Bunu nasıl yapabileceğimizi görmek zor, burada verdiğimiz yapı biraz karmaşık.

Bir an için, değişkenlerinin $(q_i A q_j)$ biçiminde olduğu ve üretimlerin

$$(q_i A q_j) \xRightarrow{*} v,$$

npda'nın, v 'yi okurken ve q_i durumundan q_j durumuna geçerken, yığınının A 'yı silmesi durumunda ve yalnızca bu durumda geçerli olan bir gramer bulabileceğimizi varsayalım. Burada "silme", A 'nın ve etkilerinin (yani, onun yerine geçen tüm ardışık dizelerin) yığından çıkarılması anlamına gelir; A 'nın altında bulunan sembolü en üste getirir. Böyle bir gramer bulabilirsek ve başlangıç sembolü olarak $(q_0 z q_f)$ seçersek, o zaman

$$(q_i z q_f) \xRightarrow{*}$$

npda'nın, w 'yi okurken z 'yi çıkardığı (boş bir yığın oluşturduğu) durumunda ve yalnızca bu durumda geçerli olacaktır. w ve q_0 'dan q_f 'ye geçerken. Ama bu, npda'nın w 'yi kabul etme şeklidir. Bu nedenle, gramer tarafından üretilen dil, npda tarafından kabul edilen dille aynı olacaktır.

Bu koşulları karşılayan bir gramer oluşturmak için, npda'nın yapabileceği farklı geçiş türlerini inceliyoruz. (7.5) A 'nın anında silinmesini içerdiğinden, gramerin karşılık gelen bir üretimi olacaktır.

$$(q_i A q_j) \rightarrow a.$$

(7.6) türündeki üretimler, kurallar kümesini üretir.

$$(q_i A q_k) \rightarrow a \quad j B q_l \quad (q_l C q_k),$$

burada q_k ve q_l Q içindeki tüm olası değerleri alır. Bunun nedeni, A silmek için önce onu BC ile değiştirmemizdir, bir a okurken ve durumdan q_i ye q_j ye geçerken. Ardından, q_j den q_l ye geçiyoruz, B 'yi silerek, sonra q_l den q_k ye geçiyoruz, C 'yi silerek.

Son adımda, çok fazla eklemişiz gibi görünebilir, çünkü bazı durumlar q_l , silerken B 'den q_j 'ye ulaşılabilir. Bu doğrudur, ancak bu dilbilgisi üzerinde bir etki yaratmaz. Ortaya çıkan değişkenler ($q_j B q_l$) gereksiz değişkenlerdir ve dilbilgisi tarafından kabul edilen dili etkilmez.

Son olarak, başlangıç değişkeni olarak ($q_0 z q_f$) alıyoruz, burada q_f npda'nın tek final durumudur.

ÖRNEK 7.8

Geçişleri olan npda'yı düşünün

$$\delta(q_0, a, z) = \{(q_0, Az)\},$$

$$\delta(q_0, a, A) = \{(q_0, A)\},$$

$$\delta(q_0, b, A) = \{(q_1, \lambda)\},$$

$$\delta(q_1, \lambda, z) = \{(q_2, \lambda)\}.$$

Başlangıç durumu olarak q_0 ve final durumu olarak q_2 kullanarak, npda yukarıdaki 1. koşulu sağlar, ancak 2. koşulu sağlamaz. İkincisini sağlamak için yeni bir durum q_3 ve önce yığından A 'yı kaldırdığımız, ardından bir sonraki hamlede yerine koyduğumuz bir ara adım tanıtlıyoruz. Yeni geçiş kural seti şudur:

$$\delta(q_0, a, z) = \{(q_0, Az)\},$$

$$\delta(q_3, \lambda, z) = \{(q_0, Az)\},$$

$$\delta(q_0, a, A) = \{(q_3, \lambda)\},$$

$$\delta(q_0, b, A) = \{(q_1, \lambda)\},$$

$$\delta(q_1, \lambda, z) = \{(q_2, \lambda)\}.$$

Son üç geçiş (7.5) biçimindedir, böylece karşılık gelen üretimleri verir.

$$(q_0 A q_3) \rightarrow a, \quad (q_0 A q_1) \rightarrow b, \quad (q_1 z q_2) \rightarrow \lambda.$$

İlk iki geçişten üretim setini elde ederiz.

$$(q_0 z q_0) \rightarrow a (q_0 A q_0) (q_0 z q_0) \mid a (q_0 A q_1) (q_1 z q_0) \mid$$

$$a (q_0 A q_2) (q_2 z q_0) \mid a (q_0 A q_3) (q_3 z q_0),$$

$$(q_0 z q_1) \rightarrow a (q_0 A q_0) (q_0 z q_1) \mid a (q_0 A q_1) (q_1 z q_1) \mid$$

$$a (q_0 A q_2) (q_2 z q_1) \mid a (q_0 A q_3) (q_3 z q_1),$$

$$\begin{aligned}
(q_0 z q_2) &\rightarrow a(q_0 A q_0) (q_0 z q_2) | a(q_0 A q_1) (q_1 z q_2) | \\
&\quad a(q_0 A q_2) (q_2 z q_2) | a(q_0 A q_3) (q_3 z q_2), \\
(q_0 z q_3) &\rightarrow a(q_0 A q_0) (q_0 z q_3) | a(q_0 A q_1) (q_1 z q_3) | \\
&\quad a(q_0 A q_2) (q_2 z q_3) | a(q_0 A q_3) (q_3 z q_3), \\
(q_3 z q_0) &\rightarrow (q_0 A q_0) (q_0 z q_0) | (q_0 A q_1) (q_1 z q_0) | (q_0 A q_2) (q_2 z q_0) | (q_0 A q_3) (q_3 z q_0), \\
(q_3 z q_1) &\rightarrow (q_0 A q_0) (q_0 z q_1) | (q_0 A q_1) (q_1 z q_1) | (q_0 A q_2) (q_2 z q_1) | (q_0 A q_3) (q_3 z q_1), \\
(q_3 z q_2) &\rightarrow (q_0 A q_0) (q_0 z q_2) | (q_0 A q_1) (q_1 z q_2) | (q_0 A q_2) (q_2 z q_2) | (q_0 A q_3) (q_3 z q_2), \\
(q_3 z q_3) &\rightarrow (q_0 A q_0) (q_0 z q_3) | (q_0 A q_1) (q_1 z q_3) | (q_0 A q_2) (q_2 z q_3) | (q_0 A q_3) (q_3 z q_3).
\end{aligned}$$

Bu oldukça karmaşık görünüyor, ancak basitleştirilebilir. Sol taraf ta herhangi bir üretimde yer almayan bir değişkengereksiz olmalıdır, bu nedenle hemen $(q_0 A q_0)$ ve $(q_0 A q_2)$ kaldırabiliriz. Ayrıca, değiştirilen npda'nın geçiş grafiüzerinden bakarak, q_1 'den q_0 'a, q_1 'den q_1 'e, q_1 'den q_3 'e ve q_2 'den q_2 'ye bir yol olmadığını görüyoruz, bu da ilişkili değişkenlerin de gereksiz olduğunu gösteriyor. Tüm bugereksiz üretimleri kaldırdığımızda, elimizde çok daha kısa bir dil bilgisi kalıyor.

$$\begin{aligned}
(q_0 A q_3) &\rightarrow a, \\
(q_0 A q_1) &\rightarrow b, \\
(q_1 z q_2) &\rightarrow \lambda, \\
(q_0 z q_0) &\rightarrow a(q_0 A q_3)(q_3 z q_0), \\
(q_0 z q_1) &\rightarrow a(q_0 A q_3)(q_3 z q_1), \\
(q_0 z q_2) &\rightarrow a(q_0 A q_1)(q_1 z q_2) | a(q_0 A q_3)(q_3 z q_2), \\
(q_0 z q_3) &\rightarrow a(q_0 A q_3)(q_3 z q_3), \\
(q_3 z q_0) &\rightarrow (q_0 A q_3)(q_3 z q_0), \\
(q_3 z q_1) &\rightarrow (q_0 A q_3)(q_3 z q_1), \\
(q_3 z q_2) &\rightarrow (q_0 A q_1)(q_1 z q_2) | (q_0 A q_3)(q_3 z q_2), \\
(q_3 z q_3) &\rightarrow (q_0 A q_3)(q_3 z q_3),
\end{aligned}$$

başlangıç değişkeni $(q_0 z q_2)$ ile.

ÖRNEK 7.9

String $w = ab$ olarak düşünün. Bu, Örnek

7.8'deki pda tarafından kabul edilir, ardışık konfigürasyonlarla

$$\begin{aligned}
(q_0, ab, z) &\vdash (q_0, ab, Az) \\
&\vdash (q_3, b, z)
\end{aligned}$$

$$\vdash (q_0, b, Az)$$

$$\vdash (q_1, \lambda, z)$$

$$\vdash (q_2, \lambda, \lambda).$$

Karşılık gelen türetim G ile

$$\begin{aligned} (q_0 z q_2) &\Rightarrow (q_0 z q_2) \\ &\Rightarrow aa (q_3 z q_2) \\ &\Rightarrow aa (q_0 A q_1) (q_1 z q_2) \\ &\Rightarrow aab (q_1 z q_2) \\ &\Rightarrow aab. \end{aligned}$$

Aşağıdaki teoremin kanıtındaki adımlar, pda'nın ardışık anlık tanımlamaları ile türetimdeki cümle biçimleri arasındaki ilişkiyi fark ederseniz daha kolay anlaşılacaktır. Her cümle biçiminin en soldaki değişkenindeki ilk q_i , pda'nın mevcut durumudur, ortadaki sembollerin dizisi ise yığın içeriği ile aynıdır.

Yapı, oldukça karmaşık bir dil bilgisi üretse de, geçiş kuralları verilen koşulları sağlayan herhangi bir pda'ya uygulanabilir. Bu, genel sonucun kanıtının temelini oluşturur.

TEOREM 7.2

If $L = L(M)$ for some npda M , then L is a context-free language.

Proof: Assume that $M = (Q, \Sigma, \Gamma, \delta, q_0, z, \{q_f\})$ satisfies conditions 1 and 2 above. We use the suggested construction to get the grammar $G = (V, T, S, P)$, with $T = \Sigma$ and V consisting of elements of the form $(q_i c q_j)$. Bu elde edilen dilbilgisinin, tüm $q_i, q_j \in$

$Q, A \in \Gamma, X \in \Gamma^*, u, v \in \Sigma^*$ için geçerli olduğunu göstereceğiz,

$$(q_i, uv, AX) \vdash^* (q_j, v, X) \quad (7.7)$$

anlamına gelir

$$(q_i A q_j) \Rightarrow^* u,$$

ve tersine.

İlk kısım, npda'nın, sembol A ve etkilerinin, u 'yu okurken ve durumu q_i 'den q_j 'ye geçerken yığınlardan çıkarılabileceği durumlarda, değişkenin $(q_i A q_j)$ u 'yu türetebileceğini göstermektir. Bu,

görmek zor değil çünkü dilbilgisi bunu yapmak için açıkça inşa edilmiştir. Bunu kesinleştirmek için yalnızca hareket sayısı üzerinde bir induksiyon yapmamız gerekiyor.

Tersine, türetimdeki tek bir adımı düşünün,

$$(q_i A q_k) \Rightarrow (q_j \quad \quad \quad).$$

npda için karşılık gelen geçişi kullanarak

$$\delta(q_i, a, A) = \{(q_j, BC), \dots\}, \quad (7.8)$$

A'nın yığından çıkarılabileceğini, BC'nin konulabileceğini, a'yı okurken, kontrol biriminin durum q_i 'den q_j 'ye geçtiğini görüyoruz. Benzer şekilde, eğer

$$(q_i A q_j) \Rightarrow a, \quad (7.9)$$

o zaman karşılık gelen bir geçiş olmalıdır

$$\delta(q, a, A) = \{ \quad \quad \quad, \lambda \} \quad (7.10)$$

burada A yığınının çıkarılabilir. Bunu gördüğümüzde, $(q_i A q_j)$ den türetilen cümle biçimlerinin, (7.7)'nin elde edilebileceği npda'nın olası konfigürasyonlarının bir dizisini tanımladığını görüyoruz.

Dikkat edin ki $(q_i A q_j) \Rightarrow a (q_j B q_l) (q_l C q_k)$ bazıları için mümkün olabilir $(q_j B q_l) (q_l C q_k)$ için, bu formda karşılık gelen bir geçiş yoktur (7.8) veya (7.10). Ancak, bu durumda, sağdaki değişkenlerden en az biri işe yaramaz olacaktır. Terminal bir dizeye giden tüm cümle biçimleri için, verilen argüman geçerlidir.

Eğer şimdi sonucu uyguluyorsak

$$(q_0, w, z) \vdash^* (q_f, \lambda, \lambda),$$

bunun yalnızca ve yalnızca şu durumda mümkün olduğunu görüyoruz

$$(q_0 z q_f) \xRightarrow{*} w.$$

Sonuç olarak $L(M) = L(G)$. ■

ALISTIRMALAR

1. Gramer tarafından tanımlanan dili kabul eden bir pda oluşturun $S \rightarrow abSb \mid \lambda$.
2. Gramer tarafından tanımlanan dili kabul eden bir pda oluşturun $S \rightarrow aSSSab \mid \lambda$.

3. Gramer tarafından üretilen dili kabul eden bir npda oluşturun

$$S \rightarrow aSbb|abb.$$

4. Örnek 7.6'da oluşturulan pda'nın aabb veaaabbbb dizelerini kabul ettiğini ve her iki dizinin de verilen gramer tarafından üretilen dilde olduğunu gösterin.
5. Prove that the pda in Example 7.6 accepts the language $L = \{a^{n+1}b^{2n} : n \geq 0\}$.
6. Gramer tarafından üretilen dili kabul eden bir npda oluşturun
 $S \rightarrow aSSS|ba.$
7. Construct an npda corresponding to the grammar

$$S \rightarrow aABB|aAA,$$

$$A \rightarrow aBB|b,$$

$$B \rightarrow bBB|A.$$

8. Construct an npda that will accept the language generated by the grammar $G = (\{S, A\}, \{a, b\}, S, P)$, with productions $S \rightarrow AA|a, A \rightarrow SA|ab$.
9. Show that Theorems 7.1 and 7.2 imply the following. For every npda M , there exists an npda $\widehat{M}$ with at most three states, such that $L(M) = L(\widehat{M})$.
10. Show how the number of states of $\widehat{M}$ in the above exercise can be reduced to two.
11. Find an npda with two states for the language $L = \{a^n b^{n+1} : n \geq 0\}$.
12. İki durumu olan bir npda bulun ve kabul etsin $L = \{a^n b^{2n} : n \geq 2\}$.
13. Örnek 7.8'deki npda'nın $L(aa^*b)$ kabul ettiğini gösterin.
14. Örnek 7.8'deki dilin $L(aa^*b)$ üretilen gramer olduğunu gösterin.
15. Örnek 7.8'de, değişkenin $(q_0 z q_1)$ gereksiz olduğunu gösterin.
16. npda $M = (\{q_0, q_1\}, \{a, b\}, \{A, z\}, \delta, q_0, z, \{q_1\})$ tarafından kabul edilen dili üreten bir bağlamdan bağımsız gramer bulun, geçişlerle birlikte

$$\delta(q_0, a, z) = \{(q_0, Az)\}$$

$$\delta(q_0, b, A) = (q_0, AA) ,$$

$$\delta(q_0, a, A) = (q_1, \lambda) .$$

17. Her npda için, Teorem 7.2'nin önsözündeki 1 ve 2 koşullarını sağlayan eşdeğer bir npda'nın var olduğunu gösterin.
18. Teorem 7.2'nin kanıtının tam detaylarını verin.
19. Teorem 7.1'in kanıtında kullanılabilecek herhangi bir bağlamdan bağımsız gramere bir yapı verin.
20. Örnek 7.8'deki dilbilgisi hala gereksiz değişkenler içeriyor mu?

7.3 BELİRLİ YIĞIT OTOMATLARI VE BELİRLİ BAĞLAM DAN BAĞIMSIZ DİLLER

A belirli bir yığıt kabul edici (dpda) hareketinde asla bir seçim yapmayan bir yığıt otomati olarak tanımlanır. Bu, Tanım 7.1'in bir modifikasyonu ile sağlanabilir.

TANIM 7.3

Bir yığıt otomati $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$ belirli olarak adlandırılır. Eğer Tanım 7.1'de tanımlanan bir otomatsa, her $q \in Q$ için, $a \in \Sigma \cup \{\lambda\}$ ve $b \in \Gamma$ koşullarına tabi ise,

1. $\delta(q, a, b)$ en fazla bir eleman içerir,
2. eğer $\delta(q, \lambda, b)$ boş değilse, o zaman $\delta(q, c, b)$ her $c \in \Sigma$ için boş olmalıdır.

Bu koşullardan ilki, herhangi bir verilen girdi sembolü

ve herhangi bir yığın üstü için en fazla bir hareket yapılabilceğini gerektirir. İkinci koşul ise, bir λ -hareketinin mümkün olduğu bir yapılandırmada, girdi tüketen alternatifin mevcut olmamasıdır.

Bu tanım ile belirleyici sonlu otomatonun karşılık gelen tanımı arasındaki farkı not etmek ilginçtir. Geçiş fonksiyonunun tanım kümesi, λ -geçişlerini korumak istediğimiz için, Tanım 7.1'de olduğu gibi $Q \times \Sigma \times \Gamma$ yerine hala aynı kalmaktadır. Yığının üstü, bir sonraki hareketi belirlemede bir rol oynadığından, λ -geçişlerinin varlığı otomatik olarak belirsizlik anlamına gelmez. Ayrıca, bir dpda'nın bazı geçişleri boş küme, yani tanımsız olabilir, bu nedenle ölü yapılandırmalar olabilir. Bu tanımı etkilemez; belirsizlik için tek kriter, her zaman en fazla bir olası hareketin mevcut olmasıdır.

TANIM 7.4

Bir dil L , yalnızca $L = L(M)$ olacak şekilde bir dpda M varsa, belirleyici bağlamdan bağımsız bir dil olarak adlandırılır.

ÖRNEK 7.10

Dil

$$L = \{a^n b^n : n \geq 0\}$$

belirleyici bir bağlamdan bağımsız dildir. $pda M = (\{q_0, q_1, q_2\}, \{a, b\}, \{0, 1\}, \delta, q_0, 0, \{q_0\})$ ile

$$\delta(q_0, a, 0) = \{(q_1, 10)\}$$

$$\delta(q_1, a, 1) = \{(q_1, 11)\}$$

$$\delta(q_1, b, 1) = \{(q, \lambda)\},$$

$$\delta(q_2, b, 1) = \{(q_2, \lambda)\},$$

$$\delta(q_2, \lambda, 0) = \{(q, \lambda)\}$$

verilen dili kabul eder. Tanım 7.3'ün koşullarını sağlar ve bu nedenle belirleyicidir.

Şimdi Örnek 7.5'e bakın. Oradaki npda belirleyici değildir çünkü

$$\delta(q_0, a, a) = \{(q_0, aa)\}$$

ve

$$\delta(q, \lambda, a) = \{(q, a)\}$$

Tanım 7.3'ün 2. koşulunu ihlal eder. Bu, elbette, dilin $\{ww^R\}$ kendisini n belirsiz olduğu anlamına gelmez, çünkü eşdeğer bir dpda olasılığı vardır. Ancak dilin gerçekten belirleyici olmadığı bilinmektedir. Bu ve sonraki örnekten, sonlu otomataların aksine, belirleyici ve belirsiz itme otomatalarının eşdeğer olmadığını görmekteyiz. Belirleyici olmayan bağlamdan bağımsız diller vardır.

ÖRNEK 7.11

Olsun

$$L_1 = \{a^n b^n : n \geq 1\}$$

ve

$$L_2 = \{a^n b^{2n} : n \geq 1\}.$$

Argümanın açık bir modifikasyonu, L_1 'in bir bağlamdan bağımsız dil olduğunu gösterir ve L_2 'nin de bağlamdan bağımsız olduğunu ortaya koyar. Dil

$$L = L_1 \cup L_2$$

bağlamdan bağımsızdır. Bu, bir sonraki bölümde sunulacak genel bir teorinin sonucudur, ancak bu noktada kolayca mantıklı hale getirilebilir. $G_1 = (V_1, T, S_1, P_1)$ ve $G_2 = (V_2, T, S_2, P_2)$ bağlamdan bağımsız gramerleri olsun, böylece $L_1 = L(G_1)$ ve $L_2 = L(G_2)$ olarak tanımlanır. Eğer varsayarsak ki

V_1 ve V_2 ayrık ve $S \in V_1 \cup V_2$ ise, o zaman, ikisini birleştirerek, grammar $G = (V_1 \cup V_2 \cup \{S\}, T, S, P)$, where

$$P = P_1 \cup P_2 \cup \{S \rightarrow S_1 | S_2\},$$

generates $L_1 \cup L_2$. Bu noktada oldukça net olmalı, ancak argümanın detayları Bölüm 8'e ertelenecektir. Bunu kabul edersek, L 'nin bağlamdan bağımsız olduğunu görüyoruz. Ancak L , belirleyici bir bağlamdan bağımsız dil değildir. Bu makul görünüyor, çünkü pda ya birini eşleştirmek zorundadır b ya da ikiye karşı a , bu nedenle başlangıçta bir seçim yapması gerekiyor girdi L_1 veya L_2 içindedir. Seçimin kesin olarak yapılabileceği bir bilgi, dizinin başında mevcut değildir.

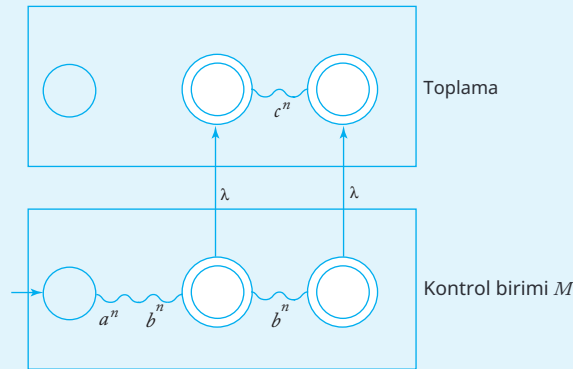
Elbette, bu tür bir argüman, aklımızda belirli bir algoritmaya dayanmaktadır; bu, bizi doğru varsayıma yönlendirebilir, ancak hiçbir şeyi kanıtlamaz. Her zaman, başlangıçta bir seçimden kaçınan tamamen farklı bir yaklaşım olasılığı vardır. Ancak, bunun olmadığını ve L 'nin gerçekten de belirsiz olduğunu görmekteyiz. Bunu görmek için önce aşağıdaki iddiayı kuruyoruz. Eğer L deterministik bir bağlamdan bağımsız dil olsaydı, o zaman

$$\hat{L} = L \cup \{a^n b^n c^n : n \geq 0\}$$

bağlamdan bağımsız bir dil olacaktır. Bunu, bir dpda M verildiğinde, L için bir npda $\hat{M}$ inşa ederek gösteriyoruz. for L .

Yapının arkasındaki fikir, kontrol birimine

M benzer bir parçanın eklenmesidir; burada giriş sembolü b tarafından neden olunan geçişler, benzerleri ile giriş c için değiştirilir. Kontrolün bu yeni parçası birim, M okuduktan sonra $a^n b^n$ girişi yapılabilir. İkinci kısım ilk bölümün b^n ile yaptığı gibi, süreç c^n ile aynı şekilde yanıt verir bu $a^n b^{2n}$ şimdi ayrıca kabul eder $a^n b^n c^n$. Şekil 7.4, inşaatı grafiksel olarak tanımlar; resmi bir argüman takip eder.



ŞEKİL 7.4

O halde $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$ ile

$$Q = \{q_0, q_1, \dots, q_n\}.$$

O zaman $\widehat{M} = (\widehat{Q}, \Sigma, \Gamma, \widehat{\delta}, z, \widehat{F})$ ile

$$\widehat{Q} = Q \cup \{\widehat{q}_0, \widehat{q}_1, \dots, \widehat{q}_n\},$$

$$\widehat{F} = F \cup \{\widehat{q}_i : q_i \in F\},$$

ve $\widehat{\delta}$ kullanılarak oluşturulmuştur

$$\widehat{\delta}(q_f, \lambda, s) = \{(\widehat{q}_f, s)\},$$

tüm $q_f \in F, s \in \Gamma$ için ve

$$\widehat{\delta}(\widehat{q}_i, c, s) = \{(\widehat{q}_j, u)\},$$

herkes için

$$\delta(q_i) = \{(j, u)\},$$

$q_i \in Q, s \in \Gamma, u \in \Gamma^*$. M kabul etmek için $a^n b^n$ olması gerekir

$$(q_0, a^n b^n, z) \vdash^* (q_i, \lambda, u),$$

$q_i \in F$. Çünkü M deterministiktir, bu da şunu gerektirir

$$(q_0, a^n b^{2n}, z) \vdash_M^* (q_i, b^n, u),$$

kabul etmesi için $a^n b^{2n}$ olması gerekir

$$(q_i, b^n, u) \vdash_M^* (q_j, \lambda, u_1),$$

bazı $q_j \in F$. Ama o zaman, inşa ile

$$(\widehat{q}, c^n, u) \vdash^* \neg (\widehat{q}_j, \lambda, u_1),$$

öyle ki $\widehat{M}$ kabul edecek $a^n b^n c^n$. Başka dizelerin $\widehat{L}$ dışında $\widehat{M}$ tarafından kabul edilmediği gösterilmelidir; bu, bu bölümün sonunda birkaç çalıştırmada ele alınmaktadır. Sonuç olarak $\widehat{L} = L(\widehat{M})$, öyle ki $\widehat{L}$ bağlamdan bağımsızdır. Ama bir sonraki bölümde (Örnek 8.1) gösterilecektir ki $\widehat{L}$ bağlamdan bağımsız değildir. Bu nedenle, L deterministik bağlamdan bağımsız bir dil olduğu varsayımımız yanlıştır.

ALISTIRMALAR

1. Gösterin ki $L = \{a^n b^m, n > m\}$ belirleyici bir bağlamdan bağımsız dildir.
2. Gösterin ki $L = \{a^n b^{2n}, n \geq 0\}$ belirleyici bir bağlamdan bağımsız dildir.
3. Gösterin ki $L = \{a^n b^m, n < m\}$ belirleyici bir bağlamdan bağımsız dildir.
4. Gösterin ki $L = \{a^n b^m, m \geq n + 3\}$ belirleyicidir.
5. $Dil L = \{a^n b^n, n \geq 1\} \cup \{b\}$ deterministik mi?
6. $Dil L = \{a^n b^n, n \geq 1\} \cup \{a\}$ deterministik mi?
7. Örnek 7.4'teki yığın otomatonunun deterministik olmadığını, ancak örnekteki dilin yine de deterministik olduğunu gösterin.
8. Egzersiz 1'deki dil L için, L^* 'nin deterministik bir bağlamdan bağımsız bir dil olduğunu gösterin.
9. Aşağıdaki dilin deterministik olmadığını öne sürmek için nedenler verin:

$$L = \{a^n b^m c^k : n = m \text{ or } m = k\}.$$

10. $Dil L = \{a^n b^m, n = m \text{ veya } n = m + 1\}$ deterministik mi?
11. Dil $\{wcw^R : w \in \{a, b\}^*\}$ deterministik mi?
12. Egzersiz 11'deki dil deterministik olmasına rağmen, yakından ilişkili dil $L = \{ww^R, w \in \{a, b\}^*\}$ nondeterministik olarak bilinir. Bu ifadenin makul olmasını sağlayan argümanlar verin.
13. Gösterin ki $L = \{w \in \{a, b\}^* : n_a(w) \neq n_b(w)\}$ deterministik bir bağlamdan bağımsız bir dildir.
14. Gösterin ki $L = \{a^n b^m, n < 2m\}$ belirleyici bağlamdan bağımsız bir dildir.
15. Gösterin ki eğer L_1 belirleyici bağlamdan bağımsızsa ve L_2 düzenliyse, o zaman $Dil L_1 \cup L_2$ belirleyici bağlamdan bağımsızdır.
16. Gösterin ki, Egzersiz 15'in koşulları altında, $L_1 \cap L_2$ deterministik bağlamdan bağımsız bir dildir.
17. Bir deterministik bağlamdan bağımsız dilin tersinin deterministik olmadığına dair bir örnek verin.

7.4 DETERMINİSTİK BAĞLAMdan BAĞIMSIZ DİLLER İÇİN GRAMERLER*

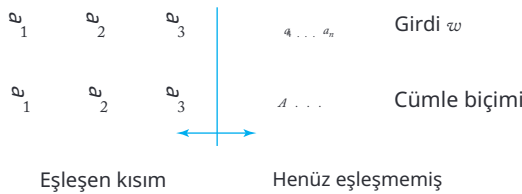
Deterministik bağlamdan bağımsız dillerin önemi, verimli bir şekilde ayrıştırılabilmelerinde yatmaktadır. Bunu, itme otomatonunu bir ayrıştırma cihazı olarak görerek sezgisel olarak anlayabiliriz. Geri izleme olmadığı için, bunun için kolayca bir bilgisayar programı yazabiliriz ve bunun verimli çalışmasını bekleyebiliriz. Ancak, içindeki geçişleri olabileceğinden, hemen bunun lineer zamanlı bir ayrıştırıcı üreteceğini iddia edemeyiz, ama yine de bizi doğru yolda tutar.

Bunu sürdürmek için, deterministik bağlamdan bağımsız dillerin tanımları için hangi gramerlerin uygun olabileceğine bakalım. Burada, derleyicilerin incelenmesinde önemli bir konuya giriyoruz, ancak ilgi alanlarımızın biraz dışında. Önemli bazı sonuçlara yalnızca kısa bir giriş yapacağız ve daha kapsamlı bir inceleme için okuyucuyu derleyicilerle ilgili kitaplara yönlendireceğiz.

Diyelim ki, yukarıdan aşağıya ayrıştırma yapıyoruz ve belirli bir cümle için en soldaki türevini bulmaya çalışıyoruz. Tartışma açısından, Şekil 7.5'te gösterilen yaklaşımı kullanıyoruz. Giriş w soldan sağa tararken, terminal ön ekinin w 'ye kadar taranan sembol ile eşleştiği bir cümle biçimi geliştiriyoruz. Ardışık sembolleri eşleştirmeye devam etmek için, her adımda hangi üretim kuralının uygulanacağını tam olarak bilmek istiyoruz. Bu, geri izlemeyi önleyecek ve bize verimli bir ayrıştırıcı verecektir. Soru, bunu yapmamıza izin veren gramerlerin olup olmadığıdır. Genel bir bağlamdan bağımsız gramer için bu geçerli değildir, ancak gramerin biçim i kısıtlanırsa, amacımıza ulaşabiliriz.

İlk örnek olarak, Tanım 5.4'te tanıtılan s-grammer'ları ele alalım. Oradaki tartışmadan, ayrıştırmanın her aşamasında hangi üretimin uygulanması gerektiğini tam olarak bildiğimiz açıktır. Varsayalım ki $w = w_1 w_2$ vesentensiyel formu $w_1 A x$ geliştirdik. Sentensiyel formun bir sonraki sembolünü w 'de ki bir sonraki sembolle eşleştirmek için, sadece w_2 'nin en soldaki sembolüne, diyelim ki a 'ya bakarız. Eğer gramerde $A \rightarrow ay$ kuralı yoksa, dize w dilin bir parçası değildir. Eğer böyle bir kural varsa, ayrıştırma devam edebilir. Ancak bu durumda yalnızca bir tane böyle kural vardır, bu nedenle seçim yapılmaz bir durum yoktur.

Her ne kadar s-grammer'lar faydalı olsa da, programlama dillerinin sözdiziminin tüm yönlerini yakalamak için çok kısıtlayıcıdır. Fikri genelleştirmemiz gerekiyor, böylece temel ayrıştırma özelliğini kaybetmeden daha güçlü hale geliyor. Bir tür gramer LL gramer olarak adlandırılır. Bir LL gramere sahip olduğumuzda, girdi üzerinde sınırlı bir parçaya bakarak (taranan sembol artı onu takip eden sonlu sayıda sembolden oluşan) hangi üretim kuralının kullanılacağını tam olarak tahmin edebilme özelliğine hala sahibiz. Terim LL , derleyicilerle ilgili kitaplarda standart bir kullanımdır; ilk L , girişin soldan sağa doğru tarandığını ifade eder; ikinci L en soldaki türevlerin oluşturulduğunu gösterir. Her s-gramer bir LL gramerdir, ancak kavram daha geneldir.



ŞEKİL 7.5

ÖRNEK 7.12

Gramer

$$S \rightarrow aSb|ab$$

s-gramer değildir, ancak bir LL gramerdir. Hangi üretimin uygulanacağını belirlemek içingiriş dizisinin iki ardışık sembolüne bakılır. Eğer ilki bir a ve ikincisi bir b ise, üretim $S \rightarrow ab$ uygulanmalıdır. Aksi takdirde, kural $S \rightarrow aSb$ kullanılmalıdır.

Bir gramere $LL(k)$ gramer denir eğer mevcut taranan sembol ve “ile riye bakma” olarak sonraki $k-1$ sembol verildiğinde doğru üretimi benzersiz bir şekilde tanımlayabiliyorsak. Örnek 7.12, bir $LL(2)$ gramer örneğidir.

ÖRNEK 7.13

Gramer

$$S \rightarrow SS|aSb|ab$$

Örnek 7.12'deki dilin pozitif kapanışını üretir. Örnek 5.4'te belirtil diği gibi, bu düzgün iç içe geçmiş parantez yapılarının dilidir. Bu gramer, herhangi bir k için bir $LL(k)$ gramer değildir.

Bunun neden böyle olduğunu görmek için, iki uzunluktan daha uzun dizilerin türetilmesine bakın.

Başlamak için, iki olası üretim mevcuttur.

$S \rightarrow SS$ ve $S \rightarrow aSb$. Tarayıcı sembol bize hangisinin doğru olduğunu söylemiyor. Şimdi bir ileri bakış kullanıp ilk iki sembolü dikkate alalım, bunların

aa. Bu, bize yapma imkanı veriyor mu

doğru karar mı? Cevap hala hayır, çünkü gördüğümüz şey, hem $aabb$ hem de $aabbab$ dahil olmak üzere bir dizi dizinin ön eki olabilir.

ilk durumda, $S \rightarrow aSb$ ile başlamalıyız, ikinci durumda ise gerekli kullanmak $S \rightarrow SS$. Dil bilgisi bu nedenle bir $LL(2)$

dil bilgisi değildir. Benzer bir şekilde, ne kadar çok

ileri bakış sembolümüz olursa olsun, her zaman bazı durumlar vardır ki çözülemez.

Bu dil bilgisi hakkındaki gözlem, dilin deterministik olmadığı v eya bunun için bir LL dil bilgisi olmadığı anlamına gelmez. Biz Orijinal dilin gramerinin başarısızlık nedenini analiz edersek, dil için bir LL grameri bulmak. Zorluk, temel desenin kaç kez tekrarlandığını tahmin edemememizdir $a^n b^n$, dizinin sonuna ulaşana kadar, ancak gramer anında bir karar gerektirir. Grameri yeniden yapmak bu zorluğu aşar.

Gramer

$$S \rightarrow aSbS|\lambda$$

orijinal gramerle neredeyse eşdeğer bir LL-grameridir.

Bunu görmek için, $w=abab$ olan en soldaki türetimi düşünün. O zaman

$$S \Rightarrow aSbS \Rightarrow abS \Rightarrow abaSbS \Rightarrow ababS \Rightarrow abab.$$

Hiçbir seçeneğimiz olmadığını görüyoruz. İncelenen girdi sembolü a olduğunda, $S \rightarrow aSbS$ kullanmalıyız; sembol b olduğunda veya dizinin sonundaysak, $S \rightarrow \lambda$ kullanmalıyız.

Ancak sorun henüz tamamen çözülmedi çünkü yenigramer boş dize üretebiliyor. Bunu, bazı boş olmayan stringlerin üretilmesini sağlamak için yeni bir başlangıç değişkeni S_0 ve bir üretim tanımlayarak düzeltiyoruz. Nihai sonuç

$$\begin{aligned} S_0 &\rightarrow aSbS, \\ S &\rightarrow aSbS|\lambda \end{aligned}$$

o zaman orijinal gramerle eşdeğer bir LL-grameridir.

Bu LL gramerlerinin bu gayri resmi tanımı, basit örnekleri anlama için yeterli olsa da, herhangi bir titiz sonuç geliştirilmesi için daha kesin bir tanıma ihtiyacımız var. Tartışmamızı böyle bir tanımla sonlandırıyoruz.

DEFINITION 7.5

Let $G = (V, T, S, P)$ bir bağlamdan bağımsız gramer olsun. Eğer her sol en önde türetim çifti için

$$\begin{aligned} S &\xRightarrow{*} w_1Ax_1 \Rightarrow w_1y_1x_1 \xRightarrow{*} w_1w_2, \\ S &\xRightarrow{*} w_1Ax_2 \Rightarrow w_1y_2x_2 \xRightarrow{*} w_1w_3, \end{aligned}$$

with $w_1, w_2, w_3 \in T^*$, w_2 ve w_3 'ün en soldaki k sembollerinin eşitliği $y_1 = y_2$ 'yi ifade ediyorsa, o zaman G LL (k) grameri olarak adlandırılır. (Eğer $|w_2|$ veya $|w_3|$ k 'den küçükse, o zaman k bu değerlerin daha küçüğü ile değiştirilir.)

Tanım, daha önce belirtilenleri kesinleştirir. Eğer en soldaki türetim aşamasında (w_1Ax) girdi için sonraki k sembollerini biliyorsak, türetimdeki bir sonraki adım benzersiz bir şekilde belirlenir (bu, $y_1 = y_2$ ile ifade edilir).

LL gramerleri konusu, derleyici çalışmalarında önemli bir konudur. Birçok programlama dili LL gramerleri ile tanımlanabilir,

ve birçok derleyici LL ayrıştırıcıları kullanılarak yazılmıştır. Ancak LL gramerleri, tüm deterministik bağlamdan bağımsız dilleri ele almak için yeterince genel değildir. Sonuç olarak, daha genel deterministik gramerlere ilgi vardır. Özellikle, türetim ağacını alttan yukarı inşa eden LR gramerleri olarak adlandırılan gramerler önemlidir. Bu konu hakkında, derleyicilerle ilgili kitaplarda (örneğin, Hunter 1981) veya formel diller için ayrıştırma yöntemlerine özel olarak adanmış kitaplarda (Aho ve Ullman 1972 gibi) çok sayıda materyal bulunmaktadır.

ALISTIRMALAR

1. Aşağıdaki diller için LL gramerleri verin, $\Sigma = \{a, b, c\}$ varsayarak:
 - (a) $L = \{ a^n b^m c^{n+m} : n \geq 1, m \geq 1 \}$
 - (b) $L = \{ a^{n+2} b^m c^{n+m} : n \geq 1, m \geq 1 \}$
 - (c) $L = \{ a^n b^{n+2} c^m : n \geq 1, m > 2 \}$
2. Örnek 7.13'teki ikinci dilbilgisinin bir LL dilbilgisi olduğunu ve orijinal dilbilgisi ile eşdeğer olduğunu gösterin.
3. Örnek 1.13'te verilen $L = \{w : n_a(w) = n_b(w)\}$ dilbilgisinin bir LL dilbilgisi olmadığını gösterin.
4. Dil L için bir LL dilbilgisi oluşturun $(aa^*ba) \cup L(abb^*)$.
5. Herhangi bir LL dilbilgisinin belirsiz olmadığını gösterin.
6. Eğer G bir $LL(k)$ dilbilgisi ise, o zaman $L(G)$ deterministik bir bağlam-özgür dildir.
7. Deterministik bir bağlamdan bağımsız dilin asla içsel olarak belirsiz olmadığını gösterin.
8. G Greibach normal formunda bir bağlamdan bağımsız dilbilgisi olsun. Herhangi bir verilen k için, G 'nin bir $LL(k)$ dilbilgisi olup olmadığını belirleyen bir algoritma tanımlayın.

BÖLÜM

8

KONTEXT-SIZ DİLLERİN ÖZELLİKLERİ

BÖLÜM ÖZETİ

Bu bölümde, bağlamdan bağımsız dillerin genel özelliklerini inceleyerek, düzenli dillerin özellikleriyle karşılaştırıyoruz. Burada, kapama ve karar algoritmaları söz konusu olduğunda, bu özelliklerin bağlamdan bağımsız diller için daha karmaşık olma eğiliminde olduğunu görüyoruz. Bu, bağlamdan bağımsız diller için bir pompalama lemması ve lineer diller için bir pompalama lemması olmak üzere iki pompalama lemması için özellikle doğrudur. Temel fikir, düzenli diller için olduğu gibi aynı olsa da, belirli argümanlar genellikle çok daha fazla ayrıntı içerir.

Bağlamdan bağımsız diller ailesi, biçimsel diller hiyerarşisinde merkezi bir konum işgal etmektedir. Bir yandan, bağlamdan bağımsız diller, düzenli ve belirleyici bağlamdan bağımsız diller gibi önemli ancak sınırlı dil ailelerini içermektedir. Diğer yandan, bağlamdan bağımsız dillerin özel bir durumu olduğu daha geniş dil aileleri bulunmaktadır. Dil aileleri arasındaki ilişkiyi incelemek ve benzerliklerini ve farklılıklarını sergilemek için çeşitli ailelerin karakteristik özelliklerini araştırıyoruz. Bölüm 4'te olduğu gibi, çeşitli işlemler altında kapama, aile üyelerinin özelliklerini belirlemek için algoritmalar ve pompalama lemmaları gibi yapısal sonuçlara bakıyoruz. Bunların hepsi bize

farklı aileler arasındaki ilişkileri anlamanın yanı sıra belirli dilleri uygun bir kategoriye sınıflandırmak için bir araç sağlamaktadır.

8.1 İKİ POMPALAMA LEMASI

Teorem 4.8'de verilen pompalama lemaları, belirli dillerin düzenli olmadığını göstermek için etkili bir araçtır. Diğer dil aileleri için de benzer pompalama lemaları bilinmektedir. Burada, biri genel bağlamdan bağımsız diller için, diğer i ise sınırlı bir bağlamdan bağımsız dil türü için olmak üzere iki böyle sonucu tartışacağız.

Regular diller için pompalama lemması, tümregular dildeki dizelerin belirli bir tekrarlayıcı deseni sergilemesi gerektiği gerçeğine dayanmaktadır. Eğer bir dil L , bu deseni takip etmeyen bir dize içeriyorsa, o zaman L regular olamaz. Benzer bir akıl yürütme, bağlamdan bağımsız diller için pompalama lemmasına götürür. Regular diller için, desen, temsil eden dfa'nın sonluluğunun bir sonucudur; bağlamdan bağımsız dillerde ise, desen en kolay şekilde dilbilgisi aracılığıyla görülür.

Bir değişkenin, $A^* \Rightarrow vAy$ anlamında tekrar ettiğini varsayalım, $A^* \Rightarrow x$. Türemiş cümle, o zaman vxy alt dizesini içerecektir. Ayrıca, dilde $x, vvx, vvy, vvvxy$ gibi alt dizeleri içeren cümleler de olacaktır.

Aslında, bağlamdan bağımsız diller için pompalama lemması, regular diller için pompalama lemmasından farklı değildir, ancak uygulaması, pompalanan dizenin artık iki ayrı bölümden oluşması ve dizenin herhangi bir yerinde meydana gelebilmesi gerçeğiyle karmaşıklaşmaktadır.

Bağlamdan Bağımsız Diller için Bir Pompalama Lemması

TEOREM 8.1

L sonsuz bir bağlamdan bağımsız dil olsun. O zaman, herhangi bir $w \in L$ için $|w| \geq m$ olan pozitif bir tam sayı m vardır ve bu dize şu şekilde ayrıştırılabilir

$$w = uvxyz, \quad (8.1)$$

with

$$|vxy| \leq m, \quad (8.2)$$

and

$$|vy| \geq 1, \quad (8.3)$$

such that

$$uv^i xy^i z \in L, \quad (8.4)$$

for all $i = 0, 1, 2, \dots$. This is known as the pumping lemma for context-free languages.

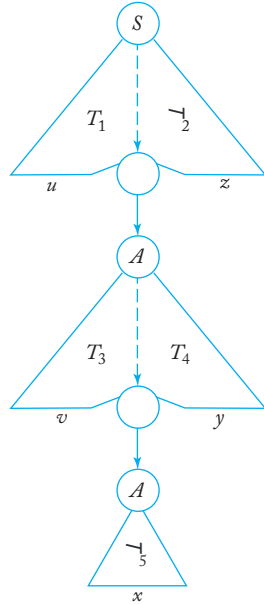
Kanıt: Dili $L - \{ \lambda \}$ düşünün ve bunun için bir gramer G olduğunu varsayın, birim üretimleri veya λ üretimleri olmadan. Herhangi bir üretimin sağ tarafındaki dizinin uzunluğu, diyelim k ile sınırlıdır, bu nedenle herhangi bir $w \in L$ için türetme uzunluğu en az $|w|/k$ olmalıdır. Bu nedenle, L sonsuz olduğundan, keyfi uzunlukta türetmeler ve karşılık gelen türetme ağaçları vardır.

Şimdi böyle yüksek bir türetim ağacını ve kökten bir yaprağa kadar yeterince uzun bir yolu düşünün. G 'deki değişken sayısı sonlu olduğundan, bu yolda tekrar eden bir değişken olmalıdır; bu, Şekil 8.1'de şematik olarak gösterilmiştir. Şekil 8.1'deki türetim ağacına karşılık gelen türetim

$$S \Rightarrow^* uAz \Rightarrow^* uvAyz \Rightarrow^* uvxyz,$$

u, v, x, y , ve z , tümü terminal dizeleridir. Yukarıdan görüyoruz ki $A^* \Rightarrow vAy$ ve $A^* \Rightarrow x$, bu nedenle tüm dizeler $uv^i xy^i z, i = 0, 1, 2, \dots$, dil bilgisi tarafından üretilebilir ve bu nedenle L içindedir. Ayrıca, türetimlerde $A^* \Rightarrow vAy$ ve $A^* \Rightarrow x$, hiçbir değişkenin tekrar etmediğini varsayabiliriz.

Bunu görmek için, Şekil 8.1'deki türetim ağacının taslağına bakın. alt ağaç T_5 içinde hiç bir değişken tekrarlanmıyor; aksi takdirde bu tekrarlayan değişkene argümanı uygulayabilirdik. Benzer şekilde, hiçbir değişkenin tekrarlanmadığını varsayabiliriz.



ŞEKİL 8.1

alt ağaçlarda T_3 ve T_4 . Bu nedenle, dizelerin uzunlukları v, x , veya yalnızca a dilbilgisinin üretimlerine bağlıdır ve w bağımsız olarak sınırlanabilir, böylece (8.2) geçerlidir. Son olarak, birim üretim yoktur ve λ üretimleri yoktur, v ve y ikisi de boş dizeler olamaz, (8.3) verir.

Bu, (8.1) ile (8.4) arasındaki argümanı tamamlar. ■

Bu pompalama lemması, bir dilin bağlamdan bağımsız diller ailesine ait olmadığını göstermede faydalıdır. Uygulaması, genel olarak pompalama lemmalarının tipik bir örneğidir; belirli bir dilin bazı ailelere ait olmadığını göstermek için olumsuz bir şekilde kullanılır. Teorem 4.8'de olduğu gibi, doğru argüman, zeki bir rakibe karşı bir oyun olarak görselleştirilebilir. Ancak şimdi kurallar bizim için biraz daha zor hale getiriyor. Düzenli diller için, uzunluğu m ile sınırlı olan alt dize xy , w 'ün sol ucundan başlar. Bu nedenle, pompalanabilen alt dize y , w 'ün başlangıcından m sembol içinde yer alır. Bağlamdan bağımsız diller için, yalnızca bir sınırlamamız vardır. $|vxy|$. Alt dize u , vxy 'yi takip eden, keyfi olarak uzun olabilir. Bu, karşı taraf için ek bir özgürlük sağlar ve Teorem

8.1 ile ilgili argümanları biraz daha karmaşık hale getirir.

ÖRNEK 8.1

Gösterin ki dil

$$L = \{a^n b^n c^n : n \geq 0\}$$

bağlamdan bağımsız değildir.

Bir kez düşman m 'yi seçtiğinde, $a^m b^m c^m$ dizisini seçiyoruz, k i bu L 'de bulunmaktadır. Düşmanın şimdi birkaç seçeneği var. Eğer sadece a 'ları içeren vxy 'yi seçerse, o zaman pompalı dizi açıkça L 'de olmayacaktır. Eğer v ve y 'nin eşit sayıda a 'lar ve b 'lerden oluşacak şekilde bir ayrıştırma seçerse, o zaman pompalı dizi $a^k b^k c^m$ ile oluşturulabilir ve yine L 'de olmayan bir dizi üretmiş oluyoruz.

L . Aslında, rakibin bizi kazanmaktan alıkoyabileceği tek yol, seçmek vxy böylece v 'nin aynı sayıda a 'ları, b 'leri ve c 'leri olmasıdır. Ancak mümkün değil çünkü kısıtlama (8.2) nedeniyle. Bu nedenle, L bağlamdan bağımsız değildir.

Bu argümanı dil üzerinde denersek $L = \{a^n b^n\}$, başarısız oluruz, çünkü dil bağlamdan bağımsızdır. Eğer L , böylece $w = a^m b^m$, düşman $v = a^k$ veya $y = b^k$ seçebilir. Şimdi, hangi i seçersek seçelim, elde edilen pompalı dize w_i L içindedir.

Ancak, bunun L 'nin bağlamdan bağımsız olduğunu kanıtlamadığını unutmayın; tek söyleyebileceğimiz, bu konudan herhangi bir sonuç elde edemediğimizdir.

pompa lema. OL bağlamdan bağımsız olmalıdır, bu başka bir argümandan gelmelidir, örneğin bir bağlamdan bağımsız dilbilgisi inşası gibi.

Bu argüman, Örnek 7.11'de yapılan bir iddiayı da haklı çıkarıyor ve bu örnekteki bir boşluğu kapatmamıza olanak tanıyor. Dil

$$\hat{L} = \{a^n b^n\} \cup \{a^n b^{2n}\} \cup \{a^n b^n c^n\}$$

bağlamdan bağımsız değildir. Dize $ambmcm^L$ içindedir, ancak pompalama sonucu değildir.

ÖRNEK 8.2

Dili dikkate al

$$L = \{ww : w \in \{a, b\}^*\}.$$

Bu dil, Örnek 5.1'in bağlamdan bağımsız diline çok benzer görünse de, bağlamdan bağımsız değildir.

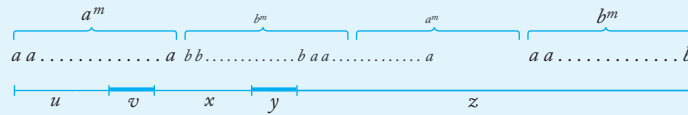
Alınan dize

$$a^m b^m a^m b^m.$$

Şimdi düşmanın vxy 'yi seçebileceği birçok yol var, ancak bunların hepsi için kazanan bir karşı hamlemiz var. Örneğin, Şekil 8.2'deki seçim için $i = 0$ kullanarak aşağıdaki biçimde bir dize elde edebiliriz

$$a^k b^j a^m b^m, k < m \text{ or } j < m,$$

bu L 'de yoktur. Düşmanın diğer seçimleri için benzer argümanlar yapılabilir. Sonuç olarak, L 'nin bağlam serbest olmadığı sonuclandırıyoruz.



ŞEKİL 8.2

ÖRNEK 8.3

Gösterin ki dil

$$L = \{a^{n!} : n \geq 0\}$$

bağlamdan bağımsız değildir.

Rakibin m seçimine göre $a = a^{m!}$ alıyoruz. Açıkça, her ne kadar ayrıştırma olursa olsun, biçimi $v = a^k, y = a^l$ olmalıdır. O zaman $w_0 = uxx$ uzunluğu $m! - (k+l)$ dir. Bu dize L içinde yalnızca

$$m! - (k+l) = j!$$

bazı j için geçerlidir. Ancak bu imkansızdır, çünkü $k+l \leq m$ ile,

$$m - (k+l) > (m-1)!$$

Bu nedenle, dil bağlamdan bağımsız değildir.

ÖRNEK 8.4

Gösterin ki dil

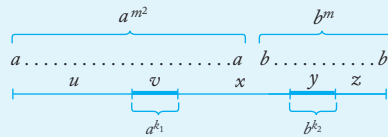
$$L = \{a^n b^j : n = j^2\}$$

bağlamdan bağımsız değildir.

Teorem 8.1'de m verildiğinde, dize olarak $a^{m^2} b^m$ seçiyoruz. Rakip $\mathfrak{P}$ mdi birkaç seçeneğe sahip. Düşünmeyi gerektiren tek seçenek, Şekil 8.3'te gösterilendir. Pompalama i kez yeni bir dize üretecektir $(i-1)k_1 a'$ lar ve $m + (i-1)k_2 b'$ ler ile. Eğer rakip alırsa $k_1 / = 0, k_2 / = 0, i = 0$ alabiliriz. Çünkü

$$\begin{aligned} (m - k_2)^2 &\leq (m - 1)^2 \\ &= m^2 - 2m + 1 \\ &< m^2 - k_1, \end{aligned}$$

sonuç L içinde değildir. Eğer rakip $k_1 = 0, k_2 / = 0$ veya $k_1 / = 0, k_2 = 0$ ise, yine $i = 0$ ile, pompalanan dize L içinde değildir. Bunu gözlemleyerek L 'nin bağlamdan bağımsız bir dil olmadığını söyleyebiliriz.



ŞEKİL 8.3

Doğrusal Diller için Bir Pompalama Lemması

Daha önce doğrusal ve doğrusal olmayan bağlamdan bağımsız gramerler arasında bir ayrım yaptık. Şimdi benzer bir ayrımı diller arasında yapıyoruz.

TANIM 8.1

Bir bağlamdan bağımsız dil L doğrusal olarak adlandırılır eğer $L = L(G)$ şeklinde bir doğrusal bağlamdan bağımsız gramer G varsa.

Açıkça, her doğrusal dil bağlamdan bağımsızdır, ancak tersinin doğru olup olmadığını henüz belirlemedik.

ÖRNEK 8.5

$Dil L = \{a^n b^n : n \geq 0\}$ bir doğrusal dildir. Bunun için bir doğrusal gramer Örnek 1.11'de verilmiştir. $Dil L = \{w : n_a(w) = n_b(w)\}$ için Örnek 1.13'te verilen gramer doğrusal değildir, bu nedenle ikinci dil mutlaka doğrusal değildir.

Elbette, belirli bir gramerin doğrusal olmaması, onun tarafından üretilen dilin doğrusal olmadığı anlamına gelmez. Bir dilin doğrusal olmadığını kanıtlamak istiyorsak, eşdeğer bir doğrusal gramerin mevcut olmadığını göstermeliyiz. Bunu, doğrusal diller için yapısal özellikler belirleyerek ve ardından bazı bağlamdan bağımsız dillerin gerekli bir özelliğe sahip olmadığını göstererek alışıldık şekilde ele alıyoruz.

TEOREM 8.2

Let L sonsuz bir lineer dil olsun. O zaman, herhangi bir $w \in L$ için, $|w| \geq m$ olan pozitif bir tam sayım vardır ve bu $w = uvxyz$ olarak ayrıştırılabilir.

$$|vxy| \leq m, \quad (8.5)$$

$$|vy| \geq 1, \quad (8.6)$$

such that

$$uv^i xy^i z \in L, \quad (8.7)$$

for all $i = 0, 1, 2, \dots$

Bu teoremin sonuçlarının Teorem 8.1'den farklı olduğunu unutmayın, çünkü (8.2) (8.5) ile değiştirilmiştir. Bu, pompalanması gereken dizelerin v ve

y şimdiki sembolü içinde sol ve sağ w uçlarının arasında yer alması gerektiğini belirtir. Orta dize x rastgele uzunlukta olabilir.

Kanıt: Dil lineerdir, bu nedenle onun için bir lineer dilbilgisi G vardır. Argüman için G 'nin birim üretimleri ve λ üretimleri olmadığını varsayma uygundur. Teorem 6.3 ve 6.4'ün kanıtlarının incelenmesi, birim üretimlerinin ve λ üretimlerinin kaldırılmasının dilbilgisinin lineerliğini bozmadığını açıkça göstermektedir. Bu nedenle, G 'nin gerekli özelliğe sahip olduğunu varsayabiliriz.

Şimdi bir dize $w \in L(G)$ türetimini düşünelim.

$$S \xRightarrow{*} uAz \xRightarrow{*} uvAyz \xRightarrow{*} wxyz = w.$$

Şu anda, her $w \in L(G)$ için bir değişken A olduğunu varsayalım, böylece

1. kısmi türevde $S^* \Rightarrow uAz$ değişken tekrarı yoktur,
2. kısmi türevde $S^* \Rightarrow uAz^* \Rightarrow uvAyzA$ dışında değişken tekrarı yoktur,
3. A tekrarı ilk m adımda gerçekleşmelidir, burada m dilbilgisine bağlı olabilir, bununla birlikte w 'ya bağlı değildir.

Bu doğruysa, o zaman u, v, y, z uzunlukları w bağımsız olarak sınırlı olmalıdır. Bu da (8.5), (8.6) ve (8.7)'nin geçerli olması gerektiği anlamına gelir.

Argümanı tamamlamak için, yukarıdaki koşulların her lineer dil için geçerli olduğunu göstermemiz gerekiyor. Değişkenlerin ortaya çıkabileceği dizilere baktığımızda bunun zor olmadığını görebiliriz. Burada detayları atlayacağız, bunu bir alıştırmaya bırakıyoruz (bu bölümün sonunda Alıştırma 16'ya bakın). ■

ÖRNEK 8.6

Dil

$$L = \{w : n_a(w) = n_b(w)\}$$

lineer değildir.

Bunu göstermek için, dilin lineer olduğunu varsayalım ve bu dizeye Teorem 8.2'yi uygulayalım.

$$w = a^m b^{2m} a^m.$$

Eşitsizlik (8.5), bu durumda dizelerin u, v, y, z tamamen a 'lardan oluşması gerektiğini göstermektedir. Bu diziyi bir kez pompalarsak, $a^{m+k} b^{2m} a^{m+l}$ elde ederiz, burada $yak \geq 1$ ya da $l \geq 1$, bu sonuç L içinde değildir. Teorem 8.2'nin bu çelişkisi, dilin lineer olmadığını kanıtlar.

Bu örnek, bağlamdan bağımsız ve lineer diller arasındaki ilişkiyle ilgili ortaya atılan genel soruyu yanıtlamaktadır. Lineer diller ailesi, bağlamdan bağımsız diller ailesinin doğru bir alt kümesidir.

ALISTIRMALAR

1. Gösterin ki dil

$$L = \{w : n_a(w) < n_b(w) < n_c(w)\}$$

bağlamdan bağımsız değildir.

2. Gösterin ki dil

$$L = \{w \in \{a, b, c\}^* : n_a(w) = n_b(w) \geq n_c(w)\}$$

bağlamdan bağımsız değildir.

3. Dilin $L = \{a^n b^i c^j d^k : n + k \leq i + j\}$ bağlamdan bağımsız olup olmadığını belirleyin.4. Gösterin ki dil $L = \{a^n : n \text{ asal bir sayıdır}\}$ bağlam serbest değildir.5. Dil $L = \{a^n b^m : m = 2^n\}$ bağlam serbest mi?6. Gösterin ki dil $L = \{a^n b^m : n \geq 0\}$ bağlamdan bağımsız değildir.7. Gösterin ki aşağıdaki diller $\Sigma = \{a, b, c\}$ bağlam serbest değildir:

(a) $L = \{a^n b^j : n \leq j^2\}.$

(b) $L = \{a^n b^j : n \geq (j - 1)^3\}.$

(c) $L = \{a^n b^j c^k : k = jn\}.$

(d) $L = \{a^n b^j c^k : k > n, k > j\}.$

(e) $L = \{a^n b^j c^k : n < j, n \leq k \leq j\}.$

(f) $L = \{w : n_a(w) = n_b(w) * n_c(w)\}.$

(g) $L = \{w \in \{a, b, c\}^* : n_a(w) + n_b(w) = 2n_c(w), n_a(w) = n_b(w)\}.$

(h) $L = \{a^n b^m : n \text{ and } m \text{ are both prime}\}.$

(i) $L = \{a^n b^m : n \text{ is prime or } m \text{ is prime}\}.$

(j) $L = \{a^n b^m : n \text{ is prime and } m \text{ is not prime}\}.$

8. Aşağıdaki dillerin bağlam serbest olup olmadığını belirleyin:

(a) $L = \{a^n w w^R b^n : n \geq 0, w \in \{a, b\}^*\}.$

(b) $L = \{a^n b^j a^n b^j : n \geq 0, j \geq 0\}.$

(c) $L = \{a^n b^j a^j b^n : n \geq 0, j \geq 0\}.$

$$(d) L = \{a^n b^j a^k b^l : n + j \leq k + l\}.$$

$$(e) L = \{a^n b^j a^k b^l : n \leq k, j \leq l\}.$$

$$(f) L = \{a^n b^n c^j : n \geq j\}.$$

$$(g) L = \{a^n b^n c^k, k = 2n\}.$$

9. Teorem 8.1'de, gram-

mar G'nin özellikleri cinsinden m için bir sınır bulun.

10. Aşağıdaki dilin bağlam serbest olup olmadığını belirleyin:

$$L = \{a^m c w_2 : m \geq 1, w_2 \in \{a, b\}^*, w_1 \neq w_2\}.$$

11. Dilin $L = \{a^n b^n a^m : n \geq 0, m \geq 0\}$ bağlam serbest olduğunu ancak

lineer olmadığını gösterin.

12. Aşağıdaki dilin lineer olmadığını gösterin.

$$L = \{w : n_a(w) \leq n_b(w)\}.$$

13. Dilin $L = \{w \in \{a, b, c\}^* : n_a(w) + n_b(w) = n_c(w)\}$ bağlam serbest olduğunu, ancak lineer olmadığını gösterin.

14. Dilin $L = \{a^n b^j : j \leq n \leq 2j - 1\}$ lineer midir?

15. Dilin $L = \{a^n b^m c^n\} \cup \{a^n b^n c^m\}$ lineer midir?

16. Let G be a linear grammar with k variables. Show that when we write any sequence of variables, there must be some variable A that repeats so that

- (a) The first occurrence of A must be in position $p \leq k$,
- (b) The repetition of A must be no later than $q \leq k + 1$, and
- (c) There can be no other repeating variable between positions p and q .

17. Justify the claim made in Theorem 8.2 that for any linear language (not containing λ) there exists a linear grammar without λ -productions and unit-productions.

18. Consider the set of all strings $a^n b^m$, where a and b are positive decimal integers such that $a < b$. The set of strings then represents all possible decimal fractions. Determine whether or not this is a context-free language.

19. Show that the complement of the language in Exercise 7 is not context free.

20. Is the language $L = \{a^{nm} : n \text{ and } m \text{ are prime numbers}\}$ context free?

21. It is known that the language

$$L = \{a^n b^n c^m : n \neq m\}$$

is not context free. (See the next exercise.) Show that, in spite of this, it is not possible to use Theorem 8.1 to prove it.

22. Ogden'in lemması Teorem 8.1'in bir uzantısıdır ve pompalama lemması oyununu oynama şeklinde bazıdağişiklikler gerektirir. Özellikle,(a) Herhangi

bir $w \in L$ seçebilirsiniz $|w| \geq m$, ancak w içinde en az m sembolü işaretli emelisiniz. Hangi sembolleri işaretleyeceğinizi seçebilirsiniz.

- (b) Rakip, ek kısıtlama ile birlikte $w = uvxyz$ ayrıştırmasını seçmelidir; $yavx$ ya da xy en az biri işaretli pozisyona sahip olmalıdır.

Teorem 8.1'in, tüm sembollerin w içinde işaretlendiği Ogden'in lemmasının özel bir durumu olduğunu unutmayın. Ogden'in lemmasının, önceki alıştırmadaki dilin bağlam serbest olmadığını kanıtlamak için nasıl kullanılabileceğini gösterin ve buradan Ogden'in lemmasının Teorem 8.1'den daha güçlü olduğunu sonucunu çıkarın.

8.2 BAĞLAMA ÖZELLİKLERİ VE KARAR ALGORİTMALARI BAĞLAM SERBEST DİLLER İÇİN

Bölüm 4'te belirli işlemler altında kapanışa ve düzenli diller ailesinin özelliklerini belirlemek için algoritmalara baktık. Genel olarak, orada ortaya atılan soruların kolay cevapları vardı. Aynı soruları bağlam serbest diller hakkın da sorduğumuzda, daha fazla zorlukla karşılaşyoruz. Öncelikle, düzenli diller için geçerli olan kapanış özellikleri her zaman bağlam serbest diller için geçerli değildir. Geçerli olduğunda, bunları kanıtlamak için gereken argümanlar genellikle oldukça karmaşıktır. İkincisi, bağlam serbest diller hakkında sezgisel olarak basit ve önemli birçok soru yanıtlanamaz. Bu ifade ilk başta şaşırtıcı görünebilir ve ilerledikçe açıklanması gerekecektir. Bu bölümde, en önemli sonuçların sadece bir örneğini sunuyoruz.

Bağlamdan Bağımsız Dillerin Kapatılması

TEOREM 8.3

Bağlamdan bağımsız diller ailesi, birleşim, birleştirme ve yıldız-kapatma altında kapalıdır.

Kanıt: L_1 ve L_2 , sırasıyla bağlamdan bağımsız gramerler $G_1 = (V_1, T_1, S_1, P_1)$ ve $G_2 = (V_2, T_2, S_2, P_2)$ tarafından üretilen iki bağlamdan bağımsız dildir. Genel olarak, kümelerin V_1 ve V_2 ayrık olduğunu varsayabiliriz.

Şimdi, gramer tarafından üretilen dil $L(G_3)$ üzerinde düşünelim

$$U \cup V \cup \{S_3\} \quad U \cup T, S_3 \in V_3,$$

burada S_3 , $V_1 \cup V_2$ içinde olmayan bir değişkendir. G_3 'ün üretimleri, G_1 ve G_2 'nin tüm üretimleri ile birlikte, her iki grameri de kullanmamıza olanak tanıyan alternatif bir başlangıç üretimidir. Daha kesin olarak,

$$P_3 = P_1 \cup P_2 \cup \{S_3 \rightarrow _1 | S_2\}.$$

Açıkça, G_3 bir bağlamdan bağımsız gramerdir, dolayısıyla $L(G_3)$ bir bağlamdan bağımsız dildir. Ancak,

$$L(G_3) = L_1 \cup L_2. \quad (8.8)$$

Örneğin, diyelim ki $w \in L_1$. O zaman

$$S_3 \Rightarrow S_1 \xRightarrow{*} w$$

gramer G_3 içinde olası bir türetmedir. Benzer bir argüman $w \in L_2$ için de yapılabilir. Ayrıca, eğer $w \in L(G_3)$ ise, o zaman ya

$$S_3 \Rightarrow S_1 \quad (8.9)$$

veya

$$S_3 \Rightarrow S_2 \quad (8.10)$$

türetimin ilk adımı olmalıdır. (8.9) kullanıldığını varsayalım. S_1 'den türetilen cümle biçimleri, V_1 'de değişkenler içerir ve V_1 ile V_2 ayrık olduğundan, türetim

$$S_1 \xRightarrow{*} w$$

sadece P_1 'deki üretimleri içerebilir. Bu nedenle $w \in L_1$ 'de olmalıdır. Alternatif olarak, eğer (8.10) önce kullanılırsa, o zaman $w \in L_2$ 'de olmalıdır ve bu da $L(G_3)$ 'nin

L_1 ve L_2 'nin birleşimi olduğu anlamına gelir.

Sonraki adımda,

$$G_4 = (V_1 \cup V_2 \cup \{S_4\}, T_1 \cup T_2, S_4, P_4).$$

Burada yine S_4 yeni bir değişkendir ve

$$P_4 = P_1 \cup P_2 \cup \{S_4 \rightarrow S_1 _2\}.$$

O zaman

$$L(G_4) = L(G_1) _2)$$

kolayca takip edilir.

Son olarak, $L(G_5)$ 'yi düşünün

$$G_5 = (V_1 \cup \{S_5\}, T_1, S_5, P_5),$$

burada S_5 yeni bir değişkendir ve

$$P = P_1 \cup \{S_5 \rightarrow S_1 S_5 | \lambda\}.$$

O zaman

$$L(G_5) = L(G_1)^*.$$

Bu nedenle, bağlamdan bağımsız diller ailesinin birleşim, birleştirme ve yıldız-kapama altında kapalı olduğunu göstermiş olduk. ■

TEOREM 8.4

Bağlamdan bağımsız diller ailesi, kesişim ve tamamlayıcı altında kapalı değildir.

Kanıt: İki dili göz önünde bulundurun

$$L_1 = \{a^n b^n c^m : n \geq 0, m \geq 0\}$$

ve

$$= \{a^n b^m c^m : n \geq 0, m \geq 0\}.$$

L_1 ve L_2 'nin bağlamdan bağımsız olduğunu göstermek için birkaç yol vardır. Örneğin, L_1 için bir dilbilgisi

$$\begin{aligned} S &\rightarrow S_1 S_2, \\ S_1 &\rightarrow a S_1 b | \lambda, \\ S_2 &\rightarrow c S_2 | \lambda. \end{aligned}$$

Alternatif olarak, L_1 'in iki bağlamdan bağımsız dilin birleştirilmesi olduğunu not ediyoruz, bu nedenle Teorem 8.3'e göre bağlamdan bağımsızdır. Ancak

$$L_1 \cap L_2 = \{a^n b^n c^n : n \geq 0\},$$

bağlamdan bağımsız olmadığını zaten göstermiş olduğumuz bir dildir. Böylece, bağlamdan bağımsız diller ailesi kesişim altında kapalı değildir.

Teoremin ikinci kısmı Teorem 8.3 ve küme

kimliği ile takip edilmektedir.

$$L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}.$$

Eğer bağlamdan bağımsız diller ailesi tamamlayıcı altında kapalı olsaydı, yukarıdaki ifadenin sağ tarafı herhangi bir bağlamdan bağımsız L_1 ve L_2 için bir bağlamdan bağımsız dil olurdu. Ancak bu, iki bağlamdan bağımsız dilin kesişiminin mutlaka bağlamdan bağımsız olmadığını gösterdiğimizle çelişiyor. Sonuç olarak, bağlamdan bağımsız diller ailesi tamamlayıcı altında kapalı değildir. ■

İki bağlamdan bağımsız dilin kesişimi, bağlamdan bağımsız olmayan bir dil üretebilirken, eğer dillerden biri düzenli ise kapama özelliği geçerlidir.

TEOREM 8.5

Let L_1 bağlamdan bağımsız bir dil ve L_2 düzenli bir dil olsun. O zaman $L_1 \cap L_2$ bağlamdan bağımsızdır.

Kanıt: Let $M_1 = (Q, \Sigma, \Gamma, \delta_1, q_0, z, F_1)$ bağlamdan bağımsız dil L_1 kabul eden bir n pda ve $M_2 = (P, \Sigma, \delta_2, p_0, F_2)$ düzenli dil L_2 kabul eden bir dfa olsun. Bir push-down automaton $\widehat{M} = (Q, \Sigma, \Gamma, \widehat{\delta}, \widehat{q}_0, z, \widehat{F})$ inşa ediyoruz ki bu, M_1 ve M_2 'nin paralel hareketini simüle eder: Giriş dizisinden bir sembol okunduğunda, $\widehat{M}$ aynı anda M_1 ve M_2 'nin hareketlerini gerçekleştirir. Bu amaçla şunu alıyoruz:

$$\begin{aligned}\widehat{Q} &= Q \times P, \\ \widehat{q}_0 &= (q_0, p_0), \\ \widehat{F} &= F_1 \times F_2,\end{aligned}$$

ve tanımla $\widehat{\delta}$ böylece

$$((q_k, p_l), x) \in \widehat{\delta}((q_i, p_j), a, b)$$

eğer ve yalnızca eğer

$$(q_k, x) \in \delta_1(q_i, a, b)$$

ve

$$\delta_2(p_j, a) = p$$

Bu durumda, eğer $a = \lambda$ ise, o zaman $p_j = p_l$ olmasını da gerektiririz. Diğer bir deyişle, $\widehat{M}$ durumları, belirli bir girdi dizesini okuduktan sonra M_1 ve M_2 'de buluna bilecek ilgili durumları temsil eden çiftlerle (q_i, p_j) etiketlenmiştir. Bu, şu şekilde gösterilmesi kolay bir induksiyon argümanıdır

$$((q_0, p_0), w, z) \vdash_{\widehat{M}}^* ((q_r, p_s), \lambda, x),$$

ile $q_r \in F_1$ ve $p_s \in F_2$ eğer ve yalnızca eğer

$$(q_0, w, z) \vdash^* q_r, \lambda, x,$$

ve

$$\delta^*((q_r, p_s), w) = p.$$

Bu nedenle, bir dize $\widehat{M}$ tarafından kabul edilir eğer ve yalnızca eğer M_1 ve M_2 tarafından kabul ediliyorsa, yani $L(M_1) \cap L(M_2) = L_1 \cap L_2$ içindeyse. ■

Bu teoremin ele aldığı özellik, düzenli kesişim altında kapama olarak adlandırılır. Teoremin sonucu nedeniyle, bağlamdan bağımsız diller ailesinin düzenli kesişim altında kapalı olduğunu söyleriz. Bu kapama özelliği, belirli dillerle bağlantılı argümanları basitleştirmek için bazen faydalıdır.

ÖRNEK 8.7

Gösterin ki dil

$$L = \{a^n b^n : n \geq 0, n \neq 100\}$$

bağlamdan bağımsızdır.

Bu iddiayı, dil için bir pda veya bağlamdan bağımsız bir dilbilgisi oluşturarak kanıtlamak mümkündür, ancak süreç zahmetlidir. Teorem 8.5 ile çok daha düzenli bir argüman elde edebiliriz.

Olsun

$$L_1 = \{a^{100} b^{100}\}.$$

O zaman, çünkü L_1 sonludur, düzenlidir. Ayrıca, görmek kolaydır ki

$$L = \{a^n b^n : n \geq 0\} \cap L_1^c.$$

Bu nedenle, tamamlayıcı altında düzenli dillerin kapanışı ve düzenli kesişim altında bağlamdan bağımsız dillerin kapanışı, istenen sonuç ortaya çıkar.

ÖRNEK 8.8

Gösterin ki dil

$$L = \{w \in \{a, b, c\}^* : n_a(w) = n_b(w) = n_c(w)\}$$

bağlamdan bağımsız değildir.

Pompala lema bunun için kullanılabilir, ancak yine de düzenli kesişim altında kapanış kullanarak çok daha kısa bir argüman elde edebiliriz. Varsayalım ki L bağlamdan bağımsızdır. O zaman

$$\cap L(a^* b^* c^*) = \{a^n b^n c^n : n \geq 0\}$$

da bağlamdan bağımsız olacaktır. Ama bunun böyle olmadığını zaten biliyoruz. Sonuç olarak L bağlamdan bağımsız değildir.

Dillerin kapanış özellikleri, biçimsel diller teorisinde önemli bir rol oynamaktadır ve bağlamdan bağımsız diller için daha birçok kapanış özelliği kurulabilir. Bu bölümün sonunda yer alan çalıştırmalarda bazı ek sonuçlar incelenmektedir.

Bağlamdan Bağımsız Dillerin Bazı Karar Verilebilir Özellikleri

Teorem 5.2 ve 6.6'yı bir araya getirerek, bağlamdan bağımsız diller için bir üyelik algoritmasının varlığını zaten kurmuş olduk. Bu,

Herhangi bir dil ailesinin pratikte faydalı olan temel bir özelliği olan k ur s. Diğerbağlamdan bağımsız dillerin basit özellikleri de belirlenebili r. Bu tartışmanın amacı için,dilinin dilbilgisi ile tanımlandığını varsayıy oruz.

TEOREM 8.6

Bağlamdan bağımsız bir dilbilgisi $G = (V, T, S, P)$ verildiğinde, $L(G)$ boş mu d eğil mi karar vermek için bir algoritma vardır.

Kanıt: Basitlik açısından, $\lambda \in L(G)$ olduğunu varsayalım. Eğer böyle değilse, ar gümanda küçük değişiklikler yapılması gerekecektir. Gereksiz sembolleri ve üre timleri kaldırma algoritmasını kullanıyoruz. Eğer S gereksiz bulunursa, o zaman $L(G)$ boştur; aksi takdirde, $L(G)$ en az bir eleman içerir. ■

TEOREM 8.7

Bağlamdan bağımsız bir dilbilgisi $G = (V, T, S, P)$ verildiğinde, $L(G)$ sonsuz mu değil mi belirlemek için bir algoritma vardır.

Kanıt: G 'nin λ -üretimleri, birim üretimleri ve gereksiz sembolleri içerm ediğini varsayıyoruz. Dilbilgisinin, bazı $A \in V$ için bir türetim olduğu anl amında tekrarlayan bir değişkeni olduğunu varsayalım.

$$A \xRightarrow{*} xAy.$$

Çünkü G 'nin λ -üretimleri ve birim üretimleri olmadığı varsayılmaktadır, o zaman x ve y aynı anda boş olamaz. Çünkü A ne boşaltılabilir ne de gereksiz bir sembol dür, bu durumda

$$S \xRightarrow{*} uAv \xRightarrow{*} w$$

ve

$$A \xRightarrow{*} z,$$

u, v ve z T^* içindedir. Ama o zaman

$$S \xRightarrow{*} uAv \xRightarrow{*} u x^n A y^n v \xRightarrow{*} u x^n z y^n v$$

tüm n için mümkündür, böylece $L(G)$ sonsuzdur.

Eğer hiçbir değişken asla tekrar edemezse, o zaman herhangi bir türetimin uzunl uğu $|V|$ ile sınırlıdır. Bu durumda, $L(G)$ sonludur.

Bu nedenle, $L(G)$ sonlu olup olmadığını belirlemek için bir algoritma elde etmek üzere, yaln ı zca dilbilgisinin bazı tekrarlayan değişkenlere sahip olup olmadığını belirlememiz gerekiyor.

Bu, değişkenler için bir bağımlılık grafiği çizerek basitçe yapılabiliröyl e ki, her zaman karşılık gelen bir üretim olduğunda bir kenar (A, B) b ulunur.

$$A \rightarrow xBy.$$

O zaman bir döngünün tabanında bulunan herhangi bir değişken tekrarlayan bir değişkendir. Dolayısıyla, dil bilgisi yalnızca bağımlılık grafi bir döngüye sahipse tekrarlayan bir değişkene sahiptir.

Artık bir dilbilgisinin tekrar eden bir değişkeni olup olmadığını belirlemek için bir algoritmamız olduğuna göre, bir dilbilgisinin tekrar eden bir değişkeni olup olmadığını belirlemek için bir algoritmamız var. $L(G)$ sonsuzdur. ■

Biraz şaşırtıcı bir şekilde, bağlamdan bağımsız dillerin diğer basit özellikleri bu kadar kolay bir şekilde ele alınamaz. Teorem 4.7'de olduğu gibi, iki bağlamdan bağımsız dilbilgisinin aynı dili üretip üretmediğini belirlemek için bir algoritma arayabiliriz. Ancak, böyle bir algoritmanın olmadığı ortaya çıkıyor. Şu anda, "algoritma yoktur" ifadesinin anlamını doğru bir şekilde tanımlamak için gerekli teknik makinaya sahip değiliz, ancak sezgisel anlamı açıktır. Bu, da ha sonra geri döneceğimiz önemli bir noktadır.

ALISTIRMALAR

1. Örnek 8.8'deki dilin tamamlayıcı bağlamdan bağımsız mı?
2. Teorem 8.4'teki dil L_1 üzerinde düşünün. Bu dilin lineer olduğunu gösterin.
3. Bağlamdan bağımsız diller ailesinin homomorfizm altında kapalı olduğunu gösterin.
4. Lineer diller ailesinin homomorfizm altında kapalı olduğunu gösterin.
5. Bağlamdan bağımsız diller ailesinin tersine kapalı olduğunu gösterin.
6. Görüştüğümüz dil ailelerinden hangileri tersine kapalı değildir?
7. Bağlamdan bağımsız diller ailesinin genel olarak fark altında kapalı olmadığını, ancak düzenli fark altında kapalı olduğunu gösterin; yani, eğer L_1 bağlamdan bağımsız ve L_2 düzenli ise, o zaman $L_1 - L_2$ bağlamdan bağımsızdır.
8. Deterministik bağlamdan bağımsız diller ailesinin düzenli fark altında kapalı olduğunu gösterin.
9. Lineer diller ailesinin birleşim altında kapalı olduğunu, ancak birleştirme altında kapalı olmadığını gösterin.
10. Lineer diller ailesinin kesişim altında kapalı olmadığını gösterin.
11. Deterministik bağlamdan bağımsız diller ailesinin birleşim ve kesişim altında kapalı olmadığını gösterin.
12. Tamamlayıcı bağlamdan bağımsız olmayan bir bağlamdan bağımsız dil örneği verin.
13. Eğer L_1 lineer ve L_2 düzenli ise, o zaman $L_1 L_2$ lineer bir dildir.
14. Açıkça tanımlanmış bağlamdan bağımsız diller ailesinin birleşim altında kapalı olmadığını gösterin.

15. Açıkça tanımlanmış bağlamdan bağımsız diller ailesinin kesişim altında kapalı olma adığını gösterin.
16. Let L deterministik bir bağlamdan bağımsız dil olsun ve yeni bir dil tanımlayın $L_1 = \{w: aw \in L, a \in \Sigma\}$. L_1 'in mutlaka deterministik bir bağlamdan bağımsız dil olduğu doğru mu?
17. Dilin $L = \{a^n b^n : n \geq 0, n \text{ 5'in katı değildir}\}$ bağlamdan bağımsız olduğunu gösterin.
18. Aşağıdaki dilin bağlamdan bağımsız olduğunu gösterin:

$$L = \{w \in \{a, b\}^* : n_a(w) = n_b(w) ; w \text{ aab alt dizesini içermez}\}.$$
19. Deterministik bağlamdan bağımsız diller ailesi homomorfizm altında kapalı mı?
20. Teorem 8.5'teki tümevarım argümanının ayrıntılarını verin.
21. Herhangi bir verilen bağlamdan bağımsız dil bilgisi için G ,
 $\lambda \in L(G)$ olup olmadığını belirleyebilen bir algoritma verin.
22. Bir bağlamdan bağımsız dil bilgisinin ürettiği dilin, belirli bir sayıdan n daha kısa herhangi bir kelime içerip içermediğini belirlemek için bir algoritmanın var olduğunu gösterin.
23. Bir bağlamdan bağımsız dilin, herhangi bir çift uzunlukta dize içerip içermediğini belirlemek için bir algoritmanın var olduğunu gösterin.
24. Bir L_1 bağlamdan bağımsız bir dil ve L_2 düzenli olsun. Ortak bir öğeye sahip olup olmadıklarını belirlemek için bir algoritmanın var olduğunu gösterin.

BÖLÜM

9

TURING MACHINES

BÖLÜM ÖZETİ

Burada, bir boyutlu, sınırsız bir bantla bağlı olan son durum kontrol birimi olan önemli bir kavram olan Turing makinesi tanıtıyoruz. Turing makinesi hala çok basit bir yapı olmasına rağmen, çok güçlüdür ve bir yığın otomatu ile çözülemeyen birçok problemi çözmemizi sağlar. Bu, Turing'in Tezi, Turing makinelerinin, prensipte herhangi bir bilgisayar kadar güçlü olan en genel otomata türleri olduğunu iddia eder.

Şu ana kadar yaptığımız tartışmada bazı temel fikirlerle karşılaştık, özellikle düzenli ve bağlamdan bağımsız dillerin kavramları ve bunların son otomata ve yığın kabul edenlerle ilişkisi. Çalışmamız, düzenli dillerin bağlamdan bağımsız dillerin uygun bir alt kümesini oluşturduğunu ve dolayısıyla yığın otomalarının son otomatalardan daha güçlü olduğunu ortaya koymuştur. Ayrıca, bağlamdan bağımsız dillerin, programlama dillerinin incelenmesi açısından temel olmasına rağmen, kapsamlarının sınırlı olduğunu gördük. Bu, son bölümde, bazı basit dillerin, örneğin $\{a^n b^n c^n\}$ ve $\{ww\}$, bağlamdan bağımsız olmadığını gösteren sonuçlarımızla netleştirdi. Bu, bizi bağlamdan bağımsız dillerin ötesine bakmaya ve bu örnekleri içeren yeni dil ailelerini nasıl tanımlayabileceğimizi araştırmaya yönlendiriyor. Bunu yapmak için, bir otomatonun genel resmine geri dönüyoruz. Sonlu otomata ile yığın otomatasını karşılaştırdığımızda, doğasının

geçici depolama onların arasındaki farkı yaratır. Eğer depolama yoksa sonlu bir otomata elde ederiz; eğer depolama bir yığınsa, daha güçlü biritme otomatasına sahibiz. Bu gözlemden yola çıkarak, otomata daha esnek bir depolama verildiğinde daha güçlü dil aileleri keşfetmeyi ekleyebiliriz. Örneğin, Şekil 1.3'teki genel şemada iki yığın, üç yığın, bir kuyruk veya başka bir depolama cihazı kullanırsak ne olur? Her depolama cihazı yeni bir otomata türü tanımlar mı ve bununla birlikte yeni bir dil ailesi oluşturur mu? Bu yaklaşım birçok soru ortaya çıkarır, bunların çoğu ise ilginç değildir. Daha iddialı bir soru sormak ve bir otomatın kavramının ne kadar ileri götürülebileceğini düşünmek daha öğreticidir. En güçlü otomatalar ve hesaplamanın sınırları hakkında ne söyleyebiliriz? Bu, temel bir Turing makinesi kavramına ve dolayısıyla mekanik veya algoritmik hesaplama fikrinin kesin bir tanımına yol açar.

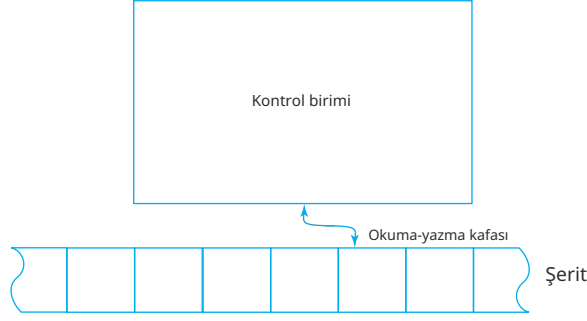
Çalışmamıza bir Turing makinesinin resmi tanımıyla başlıyoruz, ardından bazı basit programlar yaparak nelerin dahil olduğunu anlamaya çalışıyoruz. Sonraki aşamada, bir Turing makinesinin mekanizmasının oldukça basit olduğunu, ancak kavramın çok karmaşık süreçleri kapsayacak kadar geniş olduğunu savunuyoruz. Tartışma, mevcut bilgisayarlar tarafından gerçekleştirilen hesaplama süreçleri gibi herhangi bir hesaplama sürecinin bir Turing makinesinde yapılabileceğini savunan Turing tezi ile doruğa ulaşıyor.

9.1 STANDART TURING MAKİNESİ

Çeşitli karmaşık ve sofistike depolama cihazlarına sahip otomata türlerini hayal edebilmek de, bir Turing makinesinin depolama sistemi aslında oldukça basittir. Bir dizi hücreden oluşan tek boyutlu bir dizi olarak görselleştirilebilir; her biri tek bir sembol tutabilir. Bu dizi her iki yönde sonsuz bir şekilde uzanır ve dolayısıyla sınırsız miktarda bilgi tutma kapasitesine sahiptir. Bilgi herhangi bir sırayla okunabilir ve değiştirilebilir. Bu tür bir depolama cihazına, eski bilgisayarlarda kullanılan manyetik bantlara benzerliğinden dolayı bir bant denir.

Turing Makinesinin Tanımı

Bir Turing makinesi, geçici depolaması bir bant olan bir otomattır. Bu bant, her biri bir sembol tutma kapasitesine sahip hücrelere bölünmüştür. Şerit ile ilişkili olan, şeritte sağa veya sola hareket edebilen ve her hareketinde tek bir sembol okuyup yazabilen bir okuma-yazma kafasıdır. Bölüm 1'in genel şemasından biraz sapmak için, Turing makinesi olarak kullandığımız otomatonun ne bir girdi dosyası ne de herhangi bir özel çıktı mekanizması olacaktır. Gerekli olan her türlü girdi ve çıktı, makinenin şeridinde yapılacaktır. Daha sonra, bu genel modelimizin Bölüm 1.2'deki bu değişikliğinin pek bir önemi olmadığını göreceğiz. Girdi dosyasını koruyabilirdik



ŞEKİL 9.1

ve herhangi bir sonucun etkileneyeceği bir özel çıktı mekanizmasını dahil etmeden, çıkardığımız sonuçların hiçbirini etkilemeyecek şekilde tutabilirdik, ancak ortaya çıkan otomatonu tanımlamak biraz daha kolay olduğu için bunları dışarıda bırakıyoruz.

Turing makinesinin sezgisel bir görselleştirmesini veren bir diyagram Şekil 9.1 'de gösterilmektedir. Tanım 9.1, kavramı kesin hale getirir.

DEFINITION 9.1

Bir Turing makinesi M şu şekilde tanımlanır

$$M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$$

burada

Q iç durumlar kümesidir,

Σ girdi alfabesidir,

Γ tape alphabet olarak adlandırılan sonlu bir sembol kümesidir,

δ geçiş fonksiyonudur,

$\square \in \Gamma$, boş olarak adlandırılan özel bir semboldür,

$q_0 \in Q$ başlangıç durumudur,

$F \subseteq Q$ son durumlar kümesidir.

Bir Turing makinesinin tanımında, $\Sigma \subseteq \Gamma - \{\square\}$, yani giriş alfabesinin bant alfabesinin bir alt kümesi olduğunu varsayıyoruz, boş sembolü içermemektedir. Boş semboller, kısa süre içinde anlaşılacak nedenlerden dolayı giriş olarak dışlanmıştır. Geçiş fonksiyonu δ şu şekilde tanımlanır:

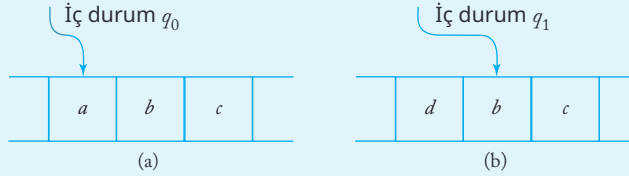
$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, \square\}$$

Genel olarak, $\delta, Q \times \Gamma$ üzerinde tanımlı bir kısmi fonksiyondur; yorumu, bir Turing makinesinin nasıl çalıştığını belirten ilkedir. δ 'nin argümanları, kontrol biriminin mevcut durumu ve okunan mevcut bant sembolüdür. Sonuç, kontrol biriminin yeni bir durumu, eski sembolü değiştiren yeni bir bant sembolü ve bir hareket sembolüdür, L veya R . Hareket sembolü, okuma-yazma kafasının yeni sembol bant üzerine yazıldıktan sonra bir hücre sola mı yoksa sağa mı hareket ettiğini gösterir.

ÖRNEK 9.1

Şekil 9.2, hareketten önceki ve sonraki durumu göstermektedir.

$$\delta(q_0, a) = (q_1, d, R).$$



ŞEKİL 9.2 Taşınmadan önce (a) ve taşınmadan sonra (b) durum.

Bir Turing makinesini oldukça basit bir bilgisayar olarak düşünebiliriz. O, sıranırlı bir belleğe sahip bir işlem birimine ve sınırsız kapasiteye sahip bir ikincil depolama alanına sahip bir bantta bulunur. Bu tür bir bilgisayarın gerçekleştirebileceği talimatlar oldukça sınırlıdır: Bantta bir sembolü algılayabilir ve sonucu kullanarak ne yapacağına karar verebilir. Makinenin gerçekleştirebileceği tek eylemler mevcut sembolü yeniden yazmak, kontrol durumunu değiştirmek ve okuma-yazma kafasını hareket ettirmektir. Bu küçük talimat seti karmaşık şeyler yapmak için yetersiz görünebilir, ancak durum böyle değildir. Turing makineleri prensipte oldukça güçlüdür. Geçiş fonksiyonu δ bu bilgisayarın nasıl hareket ettiğini tanımlar ve genellikle makinenin “programı” olarak adlandırırız.

Her zamanki gibi, otomaton verilen başlangıç durumunda bazı bilgilerle bantta başlar. Ardından, geçiş fonksiyonu δ tarafından kontrol edilen bir dizi adım geçer. Bu süreçte, banttaki herhangi bir hücrenin içeriği birçok kez incelenebilir ve değiştirilebilir. Sonunda, tüm süreç sona erebilir; bu, bir Turing makinesinde onu duraklama durumu'na koyarak gerçekleştirilir. Bir Turing makinesi, δ 'nın tanımlı olmadığı bir yapılandırmaya ulaştığında durakladığı söylenir; bu, çünkü δ kısmi bir fonksiyondur. Aslında, herhangi bir son durum için geçişlerin tanımlanmadığını varsayarsanız, bu nedenle Turing makinesi bir son duruma girdiğinde duraklayacaktır.

ÖRNEK 9.2

Turing makinesini düşünün

$$Q = \{q_0, q_1\},$$

$$\Sigma = \{a, b\},$$

$$\Gamma = \{a, b, \square\},$$

$$F = \{q_1\},$$

ve

$$\delta(q_0, a) = (q_0, b, R),$$

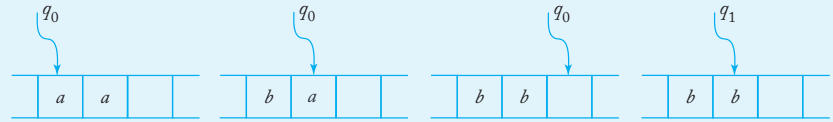
$$\delta(q_0, b) = (q_0, \square, L),$$

$$\delta(q_0, \square) = (q_1, \square, L).$$

Eğer bu Turing makinesi, okuma-yazma kafasının altında sembol a ile q_0 durumunda başlatılırsa, uygulanabilir geçiş kuralı $\delta(q_0, a) = (q_0, b, R)$ olacaktır. Bu nedenle, okuma-yazma kafası a 'yı b ile değiştirecek, ardından bantta sağa hareket edecektir. Makine q_0 durumunda kalacaktır. Herhangi bir sonraki a ile b ile değiştirilecektir, ancak b 'ler değiştirilmez. Ne zaman makine ilk boşluğa geldiğinde, bir hücre sola hareket edecek, ardından son durumda q_1 duracaktır.

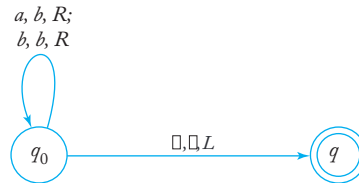
Şekil 9.3, basit bir başlangıç

konfigürasyonu için sürecin birkaç aşamasını göstermektedir.



ŞEKİL 9.3 Hareketlerin bir dizisi.

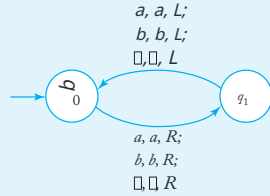
Önceki gibi, Turing makinelerini temsil etmek için geçiş grafikleri kullanabiliriz. Şimdi grafiğin kenarlarını üç öge ile etiketliyoruz: mevcut bantsembolü, onun yerine geçen sembol ve okuma-yazma başlığının hareket edeceği yön. Örnek 9.2'deki Turing makinesi, Şekil 9.4'teki geçiş grafiği ile temsil edilmektedir.



ŞEKİL 9.4

ÖRNEK 9.3

Şekil 9.5'teki Turing makinesine bakın. Ne olacağını görmek için, bir tipik durumu izleyebiliriz. Varsayalım ki, bant başlangıçta $ab\dots$ i çeriyor, okuma-yazma kafası a üzerinde. Makine daha sonra a okur, ancak onu değiştirmez. Bir sonraki durumu q_1 ve okuma-yazma kafası sağa hareket eder, böylece artık b üzerinde. Bu sembol de okunur ve değiştirilmez. Makine tekrar q_0 durumuna geri döner ve okuma-yazma kafası sola hareket eder. Artık tam olarak ilk duruma geri döndük ve hareket dizisi tekrar başlar. Bu durumdan, bant üzerindeki başlangıç bilgisi ne olursa olsun, makinenin sonsuz bir süre çalışacağı, okuma-yazma kafasının sırayla sağa ve sonra sola hareket edeceği, ancak bantta hiçbir değişiklik yapmayacağı açık. Bu, durmayan bir Turing makinesinin bir örneğidir. Programlama terminolojisiyle, Turing makinesinin sonsuz döngü içinde olduğunu söyleriz.



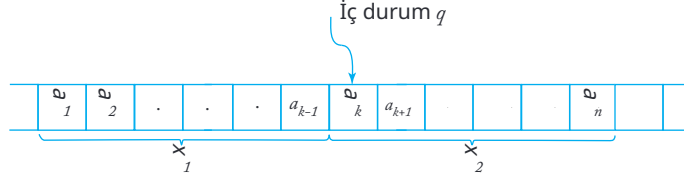
ŞEKİL 9.5

Bir Turing makinesinin birkaç farklı tanımını yapmak mümkün olduğundan, modelimizin ana özelliklerini özetlemek faydalıdır, buna standart Turing makinesi diyeceğiz:

1. Turing makinesi, her iki yönde de sınırsız bir bantta çalışır, sola ve sağa herhangi bir sayıda hareket etmeye olanak tanır.
2. Turing makinesi, her yapılandırma için en fazla bir hareket tanımlayan δ ile deterministiktir.
3. Özel bir giriş dosyası yoktur. Başlangıçta bantta belirli bir içerik olduğunu varsayıyoruz. Bunun bir kısmı giriş olarak kabul edilebilir. Aynı şekilde, özel bir çıktı cihazı yoktur. Makine durduğunda, banttaki bazı veya tüm içerikler çıktı olarak görülebilir.

Bu kurallar, esasen sonraki tartışmaların kolaylığı için seçilmiştir. Bölüm 10'da, Turing makinelerinin diğer versiyonlarına bakacağız ve bunların standart modelimizle olan ilişkisini tartışacağız.

Burada, pda'lar durumunda olduğu gibi, bir Turing makinesinin konfigürasyonlarının bir dizisini sergilemenin en uygun yolu, anlık tanım fikrini kullanmaktır. Herhangi bir konfigürasyon tamamen



ŞEKİL 9.6

kontrol biriminin mevcut durumu, banttaki içerikler ve okuma-yazma başlığının pozisyonu notasyonunu kullanacağız.

$$x_1 q x_2$$

veya

$$a_1 a_2 \cdots a_{k-1} q a_k a_{k+1} \cdots a_n$$

makinenin anlık tanımını ifade eder q bantta Şekil 9.6'da gösterilmiştir.

Semboller $a_1, \dots, a_n$ banttaki içerikleri gösterirken q kontrol biriminin durumunu tanımlar. Bu konvansiyon, okuma-yazma başlığının konumu, sembolü içeren hücrenin üzerindedir hemen ardından q .

Anlık tanım, okuma-yazma başlığının sağında ve solunda yalnızca sonlu bir miktar bilgi verir. Belirtilmemiş bant kısmının boşluklar içerdiği varsayılır; genellikle bu tür boşluklar önemsizdir ve anlık tanımda açıkça gösterilmez. Ancak, boşlukların konumu tartışma için önemliyse, boş sembol anlık tanımda görünebilir. Örneğin, anlık tanım q_w okuma-yazma başlığının w 'nin ilk sembolünün hemen solundaki hücrede olduğunu ve bu hücrenin bir boşluk içerdiğini gösterir.

ÖRNEK 9.4

Şekil 9.3'te çizilen resimler, anlık tanımların dizisine karşılık gelir $q_0 a a, b q_0 a, b b q_0, b q_1 b$.

Bir konfigürasyondan diğerine geçiş- ile gösterilecektir. Böylece,

$$\delta(q_1, c) = (q_2, e, R),$$

o zaman hareket

$$a b q_1 c d \vdash a b e q_2 d$$

iç durum q_1 olduğunda, bant $abcd$ içerdiğinde ve

okuma-yazma kafası c üzerinde olduğunda hareket yapılır. Sembol $\vdash^*$, keyfi sayıda hareketin alışılmış anlamına gelir. Alt simgeler, örneğin $\vdash_M$, birkaç makineyi ayırt etmek için argümanlarda kullanılır.

ÖRNEK 9.5

Şekil 9.3'teki Turing makinesinin eylemi

$$q_0aa \vdash bq_0a \vdash bbq_0 \square \vdash bq_1b$$

veya

$$q_0aa \vdash^* bq_1b.$$

Daha fazla tartışma için, yapılan çeşitli gözlemleri resmi bir şekilde özetlemek uygundur.

TANIM 9.2

Bir Turing makinesi olarak $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ olsun. O zaman herhangi bir dize $a_1 \cdots a_k - 1 q_1 a_k a_{k+1} \cdots a_n$, burada $a_i \in \Gamma$ ve $q_1 \in Q$, M 'nin anlık deskripsiyonudur. Bir hareket

$$a_1 \cdots a_{k-1} q_1 a_k a_{k+1} \cdots a_n \vdash a_1 \cdots a_{k-1} b q_2 a_{k+1} \cdots a_n$$

mümkündür eğer ve yalnızca eğer

$$\delta(q_1, a_k) = (q_2, b, R).$$

Bir hareket

$$a_1 \cdots a_{k-1} q_1 a_k a_{k+1} \cdots a_n \vdash a_1 \cdots q_2 a_{k-1} b a_{k+1} \cdots a_n$$

mümkündür eğer ve yalnızca eğer

$$\delta(q_1, a_k) = (q_2, b, L).$$

M , bazı başlangıç konfigürasyonlarından $x_1 q_i x_2$ itibaren durdurulacağı söylenir eğer

$$x_1 q_i x_2 \vdash^* y_1 q_j a y_2$$

herhangi bir q_j ve a için, bunun için $\delta(q_j, a)$ tanımsızdır. Bir durma durumuna götüren konfigürasyonlar dizisi hesaplama olarak adlandırılacaktır.

Örnek 9.3, bir Turing makinesinin asla durmayabileceği olasılığını göstermektedir, kaçamadığı sonsuz bir döngüde ilerlemektedir. Bu durum, Turing makineleri tartışmasında temel bir rol oynamaktadır, bu yüzden özel bir notasyon kullanıyoruz. Bunu

$$x_1 q x_2 \vdash^* \infty,$$

ilk konfigürasyondan başlayarak $x_1 q x_2$, makinenin

bir döngüye girdiğini ve asla durmadığını belirtmektedir.

Turing Makineleri Dil Kabul Edicileri Olarak

Turing makineleri, aşağıdaki anlamda kabul ediciler olarak görülebilir. Bir dizew, şeritte yazılıdır ve kullanılmayan kısımlar boşluklarla doldurulmuştur. Makine, okuma-yazma başlığı q_0 ile en soldaki w sembolünün üzerinde başlangıç durumunda başlatılır. Eğer, bir dizi hareketten sonra, Turing makinesi son duruma girer ve durursa, o zaman w kabul edilmiş sayılır.

TANIM 9.3

Bir $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ Turing makinesi olsun. O zaman M tarafında n kabul edilen dil

$$L(M) = \left\{ w \in \Sigma^+ : q_0 w \vdash^* x q_f x_2 \text{ for some } q_f \in F, x_1, x_2 \in \Gamma^* \right\}$$

Bu tanım, giriş w 'ün şeritte yazılı olduğunu ve her iki tarafında boşluklar bulunduğunu belirtmektedir. Boşlukların girişten çıkarılmasının nedeni şimdi netleşiyor: Bu, tüm girişin, sağda ve solda boşluklarla sınırlı iyi tanımlanmış bir şerit bölgesine kısıtlandığını garanti eder. Bu kural olmadan, makine, girişi araması gereken bölgeyi sınırlayamaz; ne kadar çok boşluk görürse görsün, şeritte başka bir yerde boş olmayan bir girişin olmadığından asla emin olamazdı.

Tanım 9.3, eğer $w \in L(M)$ ise ne olması gerektiğini bize söyler. Diğer herhangi bir giriş için sonuç hakkında hiçbir şey söylemez. Eğer $w \in L(M)$ 'de değilse, iki şeyden biri olabilir: Makine, son olmayan bir durumda durabilir veya sonsuz bir döngüye girebilir ve asla durmaz. Makinenin durmadığı her dize, tanım gereği $L(M)$ 'de değildir.

ÖRNEK 9.6

$\Sigma = \{0,1\}$ için, 00^* ile gösterilen dili kabul eden bir Turing makinesi tasarlayın.

Bu, Turing makinesi programlamada kolay bir alıştırmadır. Girişin sol ucundan başlayarak, her sembolü okuruz ve bunun bir 0 olup olmadığını kontrol ederiz. Eğer öyleyse, sağa doğru hareket etmeye devam ederiz. Eğer sadece 0 ile karşılaşmadan boş bir alana ulaşırsak, işlemi sonlandırır ve dizeyi kabul ederiz. Eğer girişte herhangi bir yerde 1 varsa, dize $L(00^*)$ içinde değildir ve bir sonlanmamış durumda dururuz. Hesaplamayı takip etmek için, iki iç durum $Q = \{q_0, q_1\}$ ve bir son durum $F = \{q_1\}$ yeterlidir. Geçiş fonksiyonu olarak alabiliriz

$$\begin{aligned}\delta(q_0, 0) &= (q_0, R), \\ \delta(q_0, \square) &= (q_1, \square, R)\end{aligned}$$

Okuma-yazma kafasının altında bir 0 olduğu sürece, kafa sağa hareket edecektir. Eğer herhangi bir zamanda 1 okunursa, makine sonlanmamış durumda q_0 'da duracaktır, çünkü $\delta(q_0, 1)$ tanımsızdır. Turing makinesinin, boş bir alanda q_0 durumunda başlatıldığında da son durumda durduğunu unutmayın. Bunu λ kabulü olarak yorumlayabiliriz, ancak teknik nedenlerden dolayı boş dize Tanım 9.3'e dahil edilmemiştir.

Daha karmaşık dillerin tanınması daha zordur. Turing makinelerinin ilkel bir talimat setine sahip olması nedeniyle, daha yüksek seviyeli bir dilde kolayca programlayabildiğimiz hesaplamalar, genellikle bir Turing makinesinde zahmetlidir. Yine de, bu mümkündür ve kavram anlaşılması kolaydır, bir sonraki örnekler bunu göstermektedir.

ÖRNEK 9.7

$\Sigma = \{a, b\}$ için, kabul eden bir Turing makinesi tasarlayın

$$L = \{a^n b^n : n \geq 1\}$$

Sezgisel olarak, problemi aşağıdaki şekilde çözüyoruz. En soldaki a ile başlayarak, onu bir sembol ile, diyelim k_x ile değiştirilerek işaretliyoruz. Ardından, okuma-yazma kafasının en soldaki b 'yi bulmak için sağa hareket etmesine izin veriyoruz, bu da başka bir sembol ile, diyelim k_y ile değiştirilerek işaretleniyor. Sonrasında, tekrar en soldaki a 'ya gidiyoruz, onu x ile değiştiriyoruz, ardından en soldaki b 'ye geçip onu y ile değiştiriyoruz ve bu şekilde devam ediyoruz. Bu şekilde gidip gelerek, her a 'yı karşılık gelen bir b ile eşleştiriyoruz. Eğer bir süre sonra hiç a veya b kalmazsa, o zaman dize L içinde olmalıdır.

Detayları çalışarak, tam bir çözüme ulaşırız ki $Q = \{q_0, q_1, q_2, q_3, q_4\}$, $F = \{q_4\}$, $\Sigma = \{a, b\}$, $\Gamma = \{a, b, x, y\}$. The

geçişler birkaç parçaya ayrılabilir. Küme

$$\delta(q_0, a) = (q_1, x, R),$$

$$\delta(q_1, a) = (q_1, a, R),$$

$$\delta(q_1, y) = (q_1, y, R),$$

$$\delta(q_1, b) = (q_1, y, L)$$

en soldaki a ile bir x değiştirir, ardından okuma-yazma kafasının ilk b 'ye sağa hareket etmesine neden olur, onu y ile değiştirir. y yazıldığında, makine bir duruma giriyor q_2 , bu da bir a başarıyla paired with a b .

Bir sonraki geçiş seti, bir x ile karşılaşılana kadar yönü tersine çevirir, okuma-yazma kafasını en soldaki a üzerine yeniden konumlandırır ve kontrolü başlangıç durumuna geri döndürür.

$$\delta(q_2, y) = (q_2, y, L),$$

$$\delta(q_2, a) = (q_2, a, L),$$

$$\delta(q_2, x) = (q_0, x, R).$$

Şu anda başlangıç durumuna geri döndük q_0 , bir sonraki a ile başa çıkmaya hazırız ve b .

Bu hesaplama bölümünden bir geçişten sonra, makine kısmi hesaplamayı gerçekleştirmiş olacaktır

$$q_0 a a \dots a b b \dots b \vdash^* x q_0 a \dots a y b \dots b,$$

te a ile eşleşmiş bir tek b olmasını sağlamak için. İki geçişten sonra, kısmi hesaplamayı tamamlamış olacağız

$$q_0 a a \dots a b b \dots b \vdash^* x x q_0 \dots a y y \dots b,$$

ve benzeri, eşleştirme sürecinin düzgün bir şekilde yürütüldüğünü belirtmektedir.

Girdi bir dize olduğunda $a^n b^n$, yeniden yazma bu şekilde devam eder, yalnızca silinecek daha fazla a kalmadığında durur. Bakarken en soldaki a için, okuma-yazma kafası makine ile birlikte

durum q_2 'de sola hareket eder. Bir x ile karşılaşıldığında, yön tersine döner ve te a 'ya ulaşır. Ama şimdi, bir a bulmak yerine bir y bulacaktır. Sonlandırmak için, tüm a 'nın ve b 'nin yerini aldı (to

giriş tespit etme, burada bir $a b$ izler). Bu, şu şekilde yapılabilir

$$\delta(q_0, y) = (q_3, y, R),$$

$$\delta(q_3, y) = (q_3, y, R),$$

$$\delta(q_3, \square) = (q_4, \square, R).$$

Bir dilde olmayan bir dize girdiğimizde, hesaplama sonuçlanmamış bir durumda duracaktır. Örneğin, makineye bir dize verirsek $a^n b^m$, $n > m$ ise, makine sonunda durumda

q_1 boş bir alanla karşılaşacaktır. Bu durumda geçiş belirtilmediği için duracaktır. Dilde olmayan diğer girdiler de sonuçlanmamış bir durma durumuna yol açacaktır (bu bölümün sonunda Alıştırma 4'e bakınız).

Belirli girdi $aabb$ aşağıdaki ardışık anlık tanımları verir:

$$\begin{aligned} q_0 a a b b &\vdash x q_1 a b b \vdash x a q_1 b b \vdash x q_2 a y b \\ &\vdash q_2 x a y b \vdash x q_0 a y b \vdash x x q_1 y b \\ &\vdash x x y q_1 b \vdash x x q_2 y y \vdash x q_2 x y y \\ &\vdash x x q_0 y y \vdash x x y q_3 y \vdash x x y y q_3 \quad \square \\ &\vdash x x y y q_4. \end{aligned}$$

Bu noktada Turing makinesi son durumda durur, bu nedenle dize $aabb$ kabul edilir.

Bu programı birkaç başka dize ile izlemeye teşvik ediliyorsunuz L , ayrıca L içinde olmayan bazı dizelerle.

ÖRNEK 9.8

Kabul eden bir Turing makinesi tasarlayın

$$L = \{a^n b^n c^n : n \geq 1\}.$$

Örnek 9.7'de kullanılan fikirler bu duruma kolayca aktarılabilir. Her a, b ve c 'yi sırasıyla x, y ve z ile değiştirecek şekilde eşleştiriyoruz. Sonunda, tüm orijinal sembollerin yeniden yazıldığını kontrol ediyoruz. Kavramsal olarak önceki örneğin basit bir uzantısı olmasına rağmen, gerçek programı yazmak zahmetlidir. Bunu biraz uzun ama basit bir alıştırma olarak bırakıyoruz. Dikkat edin ki, $\{a^n b^n\}$ bağlamdan bağımsız bir dildir ve $\{a^n b^n c^n\}$ değildir, ancak çok benzer yapılarla Turing makineleri tarafından kabul edilebilirler.

Bu örnekten çıkarabileceğimiz bir sonuç, bir Turing makinesinin bağlamdan bağımsız olmayan bazı dilleri tanıyabilmesidir; bu, Turing makinelerinin yığın otomatalarından daha güçlü olduğuna dair ilk işarettir.

Turing Makineleri Transdüserler Olarak

Şu ana kadar transdüserleri incelemek için pek bir nedenimiz olmadı; dil teorisinde,

kabul edenler oldukça yeterlidir. Ancak kısa süre içinde göreceğimiz gibi, Turing makineleri sadece dil kabul edenler olarak ilginç değildir, aynı zamanda genel olarak di-

varsayalım ki $w(x)$ ve $w(y)$ şeritte tekil notasyonda, tek bir 0 ile ayrıl-
mıştır, okuma-yazma başlığı en soldaki sembolde bulunmaktadır
 $w(x)$. Hesaplamadan sonra, $w(x+y)$ bantta bir tek 0 ile birlikte yer
alacak ve okuma-yazma kafası sonucun sol ucunda konumlanaca-
k. Bu nedenle, hesaplamayı gerçekleştiren bir Turing makinesi ta-
sarlamak istiyoruz.

$$q_0 w(x) 0 w(y) \vdash^* q_f w(x+y) 0,$$

burada q_f son durumdur. Bunun için bir program oluşturmak görece
basittir. Yapmamız gereken tek şey, ayırıcı 0'ı $w(y)$ sağ
ucuna taşımaktır, böylece toplama, iki dizeyi birleştirmekten başka bir şey
olmaz. Bunu başarmak için, biz oluşturuyoruz

$M = (Q, \Sigma, \Gamma, \delta, q_0, F)$, $Q = \{q_0, q_1, q_2, q_3, q_4\}$, $F = \{q_4\}$, ve

$$\delta(q_0, 1) = (q_0, 1, R),$$

$$\delta(q_0, 0) = (q_1, \text{ }, R),$$

$$\delta(q_1, \text{ }_1 1) = (q_1, \text{ }, R),$$

$$\delta(q_1, \square) = (q_2, \square, L),$$

$$\delta(q_2, 1) = (q_3, \text{ }, L),$$

$$\delta(q_3, 1) = (q_3, \text{ }, L),$$

$$\delta(q_3, \square) = (q_4, \square, R).$$

0'ı sağa hareket ettirdiğimizde geçici olarak ekstra bir 1 oluşturuyoruz, bu
durum makineyi q_1 durumuna koyarak hatırlanır. Bu
geçiş $\delta(q_2, 1) = (q_3, 0, R)$ bu durumu hesaplamamızın sonunda kaldı-
rmak için gereklidir. Bu, 11'e 111 eklemek için anlık tanımlar dizis-
inden görülebilir:

$$\begin{aligned} q_0 111011 &\vdash 1 q_0 11011 \vdash 11 q_0 1011 \vdash 111 q_0 011 \\ &\vdash 1111 q_1 11 \vdash 11111 q_1 1 \vdash 111111 q_1 \square \\ &\vdash 11111 q_1 1 \vdash 1111 q_3 10 \\ &\vdash^* q_3 \square 111110 \vdash 111110. \end{aligned}$$

Tekil notasyon, pratik hesaplamalar için zahmetli olsa da, Turing
makinelere programlamak için çok uygundur. Ortaya çıkan progr-
amlar, binary veya ondalık gibi başka bir temsil kullanmış olsaydı
çok daha kısa ve basit olurdu.

Sayıları toplamak, herhangi bir bilgisayarın temel işlemlerinden biridir, ve daha karmaşık talimatların sentezinde rol oynar. Diğer temel işlemler, dizeleri kopyalamak ve basit karşılaştırmalardır. Bunlar Turing makinesinde de kolayca yapılabilir.

ÖRNEK 9.10

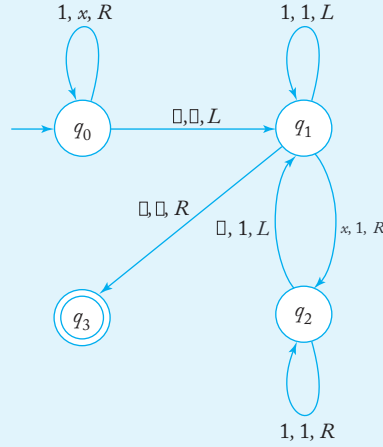
1'lerin dizelerini kopyalayan bir Turing makinesi tasarlayın. Daha kesin olarak, hesaplamayı gerçekleştiren bir makine bulun

$$q_0 w \vdash^* q_f w w,$$

herhangi bir $w \in \{1\}^+$.

Problemi çözmek için aşağıdaki sezgisel süreci uyguluyoruz:

1. Her 1'i x ile değiştirin.
2. En sağdaki x bulun ve onu 1 ile değiştirin.
3. Mevcut boş olmayan bölgenin sağ ucuna gidin ve orada a 1 oluşturun.
4. Adım 2 ve 3'ü, daha fazla x kalmayana kadar tekrarlayın.



ŞEKİL 9.7

Çözüm, Şekil 9.7'deki geçiş grafiğinde gösterilmektedir. Çözümün doğru olduğunu ilk başta görmek biraz zor olabilir, bu yüzden programı basit dize 11 ile izleyelim. Bu durumda gerçekleştirilen hesaplama

$$\begin{aligned}
 q_0 11 &\vdash x q_0 1 \vdash x x q_0 \square \vdash x q_1 x \\
 &\vdash x 1 q_2 \square \vdash x q_1 11 \vdash q_1 x 11 \\
 &\vdash 1 q_2 11 \vdash 11 q_2 1 \vdash 111 q_2 \square \\
 &\vdash 11 q_1 11 \vdash 1 q_1 111 \\
 &\vdash q_1 1111 \vdash q_1 \square 1111 \vdash q_3 1111.
 \end{aligned}$$

ÖRNEK 9.11

Let x and y be two positive integers represented in unary notation. Bir Turing makinesi inşa edin ki, eğer $x \geq y$ ise q_y durumunda duracak, ve eğer $x < y$ ise q_n durumunda duracak. Daha spesifik olarak, makine hesaplamayı gerçekleştirecektir

$$\begin{aligned}
 q_0 w(x) 0 w(y) &\vdash^* q_y w(x) 0 w(y) && \text{if } x \geq y, \\
 q_0 w(x) 0 w(y) &\vdash^* q_n w(x) 0 w(y) && \text{if } x < y.
 \end{aligned}$$

Bu problemi çözmek için, Örnek 9.7'deki fikri bazı küçük değişikliklerle kullanabiliriz. a'ların ve b'lerin eşleştirilmesi yerine, bölücü 0'ın solundaki her 1'i sağındaki 1 ile eşleştiriyoruz. Eşleştirme nin sonunda, bantta ya

$$xx \cdots 110xx \cdots x \square$$

veya

$$xx \cdots xx0xx \cdots x11 \square,$$

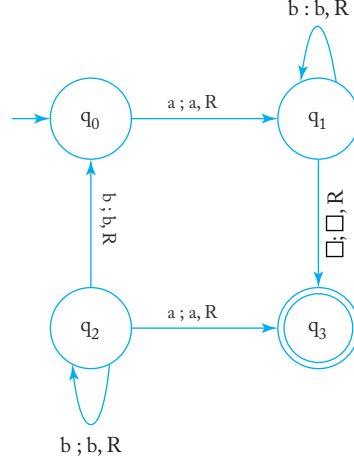
olacak, $x > y$ veya $y > x$ olup olmadığına bağlı olarak. İlk durumda, başka bir 1 eşleştirmeye çalıştığımızda, çalışma alanının sağında boşlukla karşılaşırız. Bu, durum q_y 'ye girmek için bir sinyal olarak kullanılabilir. Bu İkinci durumda, soldaki tüm 1'ler değiştirildiğinde, sağda hala bir 1 buluyoruz. Bunu, diğer duruma girmek için kullanıyoruz q_n . Bu programın tamamı basittir ve bir alıştırmaya bırakılmıştır.

Bu örnek, bir Turing makinesinin aritmetik karşılaştırmalara dayalı kararlar vermek üzere programlanabileceği önemli noktasını vurgulamaktadır. Bu tür basit kararlar, makine dilinde yaygındır.

bilgisayarların, bir aritmetik işlemin sonucuna bağlı olarak alternatif talimat akışlarının girildiği durumlarda.

ALISTIRMALAR

1. Bir Turing makinesi inşa edin, $L = L(aaaa^*b^*)$ kabul etsin.
2. Bir Turing makinesi inşa edin, dilin tamamlayıcısını kabul etsin $L = L(aaaa^*b^*)$. $\Sigma = \{a, b\}$ varsayalım.
3. Üçten fazla durumu olmayan bir Turing makinesi tasarlayın, $L(a(a+b)^*)$ kabul etsin. $\Sigma = \{a, b\}$ varsayalım. Bunuiki durumlu bir makine ile yapmak mümkün mü?
4. Örnek 9.7'deki Turing makinesinin, giriş olarak $kabab$ ve $aaabbbb$ verildiğinde ne yaptığını belirleyin.
5. Örnek 9.7'deki Turing makinesinin sonsuz döngüye girdiği herhangi bir girdi var mı?
6. Turing makinesinin geçiş grafi aşağıdaki şekilde olan makine hangi dili kabul eder?



7. Örnek 9.10'da eğer dize w 1'den başka herhangi bir sembol içeriyorsa ne olur?
8. Aşağıdaki dilleri kabul edecek Turing makineleri inşa edin $\{a, b\}$:
 - (a) $L = L(aaba^*b)$.
 - (b) $L = \{w : |w| \text{ tek}\}$.
 - (c) $L = \{w : |w| \text{ 4'ün katı}\}$.

- (d) $L = \{a^n b^m : n \geq 2, n = m\}$
 (e) $L = \{w : n_a(w) \neq n_b(w)\}$.
 (f) $L = \{a^n b^m a^{n+m} : n \geq \quad \geq \}$
 (g) $L = \{a^n b^n a^n b^n : n \neq 0\}$.
 (h) $L = \{a^n b^{2n} : n \geq 1\}$.

Her problem için bir geçiş grafiği verin; ardından cevaplarınızı birkaç test örneği ile kontrol edin.

9. Bir dili kabul eden bir Turing makinesi tasarlayın.

$$L = \{ww : w \in \{a, b\}^+\}.$$

Bir fonksiyonu hesaplamak için bir Turing makinesi oluşturun.

$$f(w) = w^R$$

burada $w \in \{0, 1\}^+$.

11. Bir fonksiyonu hesaplayan bir Turing makinesi tasarlayın.

$$\begin{aligned} f(w) &= 1 \text{ if } w \text{ is even} \\ &= 0 \text{ if } w \text{ is odd.} \end{aligned}$$

12. Bir fonksiyonu hesaplayan bir Turing makinesi tasarlayın.

$$\begin{aligned} f(x) &= x - 2 \text{ if } x > 2 \\ &= 0 \text{ if } x \leq 2. \end{aligned}$$

13. Unary olarak temsil edilen x ve y pozitif tam sayıları için aşağıdaki fonksiyonları hesaplamak üzere Turing makineleri tasarlayın:

- (a) $f(x) = 2x + 1$.
 (b) $f(x, y) = x + 2y$.
 (c) $f(x, y) = 2x + 3y$.
 (d) $\quad \text{mod } 5$.
 (e) $f(x) = \lfloor \frac{x}{2} \rfloor$, burada $\lfloor \frac{x}{2} \rfloor$, $x/2$ 'den küçük veya eşit olan en büyük tam sayıyı ifade eder.

14. Turing makinesi tasarlayın, $\Gamma = \{0, 1, \sqcup\}$, bu makine, herhangi bir hücrede boş veya 1 ile başlatıldığında, yalnızca bantında bir 0 varsa duracaktır.

15. Örnek 9.8 için tam bir çözüm yazın.

16. Örnek 9.10'daki Turing makinesinin 111 girişi ile karşılaştığında geçtiği a nlık tanımlar dizisini gösterin. Bu makine bantında 110 ile başlatıldığında ne olur?

17. Örnek 9.10'daki Turing makinesinin gerçekten belirtilen hesaplamayı gerçekleştirdiğine dair ikna edici argümanlar verin.
18. Örnek 9.11'deki detayları tamamlayın.
19. Örnek 9.9'da x ve y 'yi ikili olarak temsil etmeye karar vermiş olsaydık. Turing makinesi programını bu temsil ile belirtilen hesaplamayı yapmak için yazın.
20. Bu bölümdeki tüm örneklerin yalnızca bir sondurumu olduğunu fark etmiş olabilirsiniz. Herhangi bir Turing makinesi için, aynı dili kabul eden yalnızca bir son durumu olan başka bir makinenin var olduğu genel olarak doğru mu?

9.2 KARMAŞIK GÖREVLER İÇİN Turing MAKİNELERİNİ BİRLEŞTİRME

Bir Turing makinesinde, tüm bilgisayarlarda bulunan bazı önemli işlemlerin nasıl yapılabileceğini açıkça gösterdik. Dijital bilgisayarlarda, bu tür ilkel işlemler daha karmaşık talimatların yapı taşları olduğundan, bu temel işlemlerin Turing makinesinde nasıl bir araya getirilebileceğine bakalım. Turing makinelerinin nasıl birleştirilebileceğini göstermek için, programlamada yaygın bir uygulamayı takip ediyoruz. Yüksek seviyeli bir tanımlama ile başlıyoruz, ardından programı üzerinde çalıştığımız gerçek dile kadar kademeli olarak rafine ediyoruz. Turing makinelerini yüksek seviyede birkaç şekilde tanımlayabiliriz; blok diyagramları veya sahte kod, sonraki tartışmalarda en sık kullanacağımız iki yaklaşımdır. Bir blok diyagramında, hesaplamaları işlevinin tanımlandığı ancak iç detaylarının gösterilmediği kutular içinde kapsülleriz. Bu tür kutuları kullanarak, aslında inşa edilebileceklerini dolaylı olarak iddia ediyoruz. İlk örnek olarak, Örnek 9.9 ve 9.11'deki makineleri birleştiriyoruz.

ÖRNEK 9.12

Bir fonksiyonu hesaplayan bir Turing makinesi tasarlayın.

$$f(x, y) = \begin{cases} x + y & \text{if } x \geq y, \\ 0 & \text{if } x < y. \end{cases}$$

Tartışma açısından, x ve y 'nin unary temsilde pozitif tam sayılar olduğunu varsayalım. Sıfır değeri 0 ile temsil edilecek ve şeridin geri kalanı boş kalacaktır.

$f(x, y)$ hesaplaması, Şekil 9.8'deki diyagram aracılığıyla yüksek seviyede görselleştirilebilir. Diyagram, önce Örnek 9.11'deki gibi bir karşılaştırma makinesi kullandığımızı gösterir.

$x \geq y$ olup olmadığını. Eğer öyleyse, karşılaştırmacı bir başlangıç sinyali gönderir toplayıcıya, bu da $x+y$ hesaplar. Aksi takdirde, her 1'i boş bir alana çevirecek bir silme programı başlatılır.

Sonraki tartışmalarda, Turing makinelerinin bu tür yüksek seviyeli, siyah-diyagram temsillerini sıkça kullanacağız. Bu, karşılık gelen kapsamlı δ setlerinden kesinlikle daha hızlı ve daha net. Bu yüksek seviyeli görüşü kabul etmeden önce, bunu haklı çıkarmalıyız. Örneğin, karşılaştırmacının toplayıcıya bir başlangıç sinyali gönderdiğini söylemek ne anlama geliyor? Tanım 9.1'de bu olasılığı sunan hiçbir şey yok. Yine de, bu doğrudan bir şekilde yapılabilir.

Karşılaştırmacı için program C , Sınavda önerildiği gibi yazılır-ple 9.11, C ile indekslenmiş durumlara sahip bir Turing makinesi kullanarak. Toplayıcı için, A ile indekslenmiş durumları kullanarak Örnek 9.9'daki fikri kullanıyoruz. Silgi için E , E ile indekslenmiş durumları olan bir Turing makinesi inşa ediyoruz. C tarafından yapılacak hesaplamalar şunlardır:

$$q_{C,0}w(x)0w(y) \vdash^* q_{A,0}w(x)0w(y) \quad \text{if } x \geq y,$$

ve

$$q_{C,0}w(x)0w(y) \vdash^* q_{E,0}w(x)0w(y) \quad \text{if } x < y.$$

Eğer $q_{A,0}$ ve $q_{E,0}$ sırasıyla A ve E için başlangıç durumları olarak alırsak, görüyoruz ki C ya A ya da E ile başlıyor.

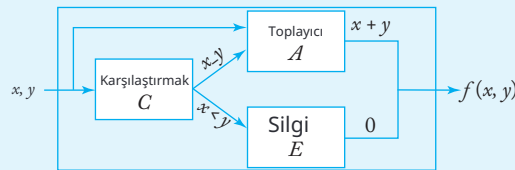
Toplayıcı tarafından gerçekleştirilen hesaplamalar şunlardır:

$$q_{A,0}w(x)0w(y) \vdash^* q_{A,f}w(x+y)0,$$

ve silgi E olacak

$$q_{E,0}w(x)0w(y) \vdash^* q_{E,f}0.$$

Sonuç, C 'nin eylemini birleştiren tek bir Turing makinesidir C , A ve E Şekil 9.8'de belirtildiği gibi.



ŞEKİL 9.8

Turing makinelerinin başka bir yararlı, yüksek seviyeli görünümü, psödo kodu içerir. Bilgisayar programlamasında, psödo kodu, anlamını anladığımızı iddia ettiğimiz tanımlayıcı ifadeler kullanarak bir hesaplamayı özetlemenin bir yoludur. Bu tanım bilgisayarda kullanılabilir olmasa da, gerektiğinde uygun dile çevirebileceğimizi varsayıyoruz. Psödo kodunun basit bir türü, daha düşük seviyedeki ifadelerin bir dizisi için tek bir ifade kısayolu olan makro talimat fikriyle örneklendirilir. Öncelikle makro talimatı, daha düşük seviyedeki dil açısından tanımlıyoruz. Ardından, makro talimatı bir programda kullanıyoruz ve ilgili düşük seviyedeki kodun makro talimatının her bir örneği için yerleştirileceğini varsayıyoruz. Bu fikir, Turing makinesi programlamasında çok yararlıdır.

ÖRNEK 9.13

Makro talimatı dikkate al

if a *then* q_j *else* q_k ,

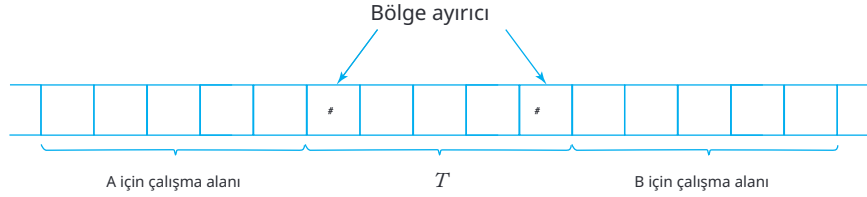
şu yorumla. Eğer Turing makinesi bir a okursa, mevcut durumuna bakılmaksızın, bant içeriğini değiştirmeden veya okuma-yazma kafasını hareket ettirmeden q_j durumuna geçmelidir. Eğer okunan sembol bir a değilse, makine hiçbir şeyi değiştirmeden q_k durumuna geçmelidir.

Bu makro talimatı uygulamak, Turing makinesinin birkaç görece bariz adımını gerektirir.

$$\begin{aligned}\delta(q_i, a) &= (q_{j0}, a, R) \text{ for all } q_i \in Q, \\ \delta(q_i, b) &= (q_{k0}, b, R) \text{ for all } i \in \text{ } \text{ and all } b \in \Gamma - \{a\}, \\ \delta(\text{ }, c) &= (\text{ }, c, L) \text{ for all } c \in \Gamma, \\ \delta(\text{ }, c) &= (q_k, c, L) \text{ for all } c \in \Gamma\end{aligned}$$

Durumlar q_{j0} ve q_{k0} , standart bir Turing makinesinde okuma-yazma kafasının her hareketle pozisyon değiştirmesi nedeniyle ortaya çıkan karmaşıklıkları ele almak için tanımlanan yeni durumlardır. Makro talimatında durumu değiştirmek istiyoruz, ancak okuma-yazma kafasını olduğu yerde bırakmak istiyoruz. Kafanın sağa hareket etmesine izin veriyoruz, ancak makineyi q_{j0} veya q_{k0} durumuna alıyoruz. Bu, istenen q_j veya q_k durumuna girmeden önce sola hareket edilmesi gerektiğini gösterir.

Bir adım daha ileri giderek, makro talimatlarını alt programlarla değiştirebiliriz. Normalde, bir makro talimatı her birinde gerçek kodla değiştirilir.



ŞEKİL 9.9

oluşum, oysa bir alt program, gerektiğinde tekrar tekrar çağrılan tek bir kod parçasıdır. Alt programlar, yüksek seviyeli programlama dilleri için temeldir, ancak Turing makineleri ile de kullanılabilir.

Bu olasılığı sağlamak için, bir Turing makinesinin başka bir Turing makinesi tarafından tekrar tekrar çağrılabilen bir alt program olarak nasıl kullanılabileceğini kısaca özetleyelim. Bu, çağırılan programın yapılandırması hakkında bilgi depolama yeteneği gibi yeni bir özellik gerektirir, böylece alt programdan döndüğümüzde yapılandırma yeniden oluşturulabilir. Örneğin, makine A durumu q_i makine B 'yi çağırdığında, B tamamlandığında, programı yeniden başlatmak istiyoruz.

A durumunda q_i , okuma-yazma kafası (B 'nin işlemleri sırasında hareket etmiş olabilir) orijinal yerinde. Diğer zamanlarda, A durumundan B 'yi çağırabiliriz, bu durumda kontrol bu duruma geri dönmelidir. Kontrolaktarım sorununu çözmek için, A 'dan B 'ye ve tersine bilgi geçirebilmemiz gerekir, A 'nın kontrolü geri aldığı anda konfigürasyonunu yeniden oluşturabilmemiz gerekir.

B , ve geçici olarak askıya alınmış A hesaplamalarının B 'nin yürütülmesi tarafından etkilenmediğinden emin olun. Bunu çözmek için, şeridi Şekil 9.9'da gösterildiği gibi birkaç bölgeye ayırabiliriz.

Önce A , B için gerekli bilgileri (örneğin, A 'nın mevcut durumu, B için argümanlar) bir bölgede T bandta yazar. A daha sonra B 'nin başlangıç durumuna geçiş yaparak kontrolü B 'ye devreder. Transferden sonra, B , girdi bulmak için T kullanacaktır. B 'nin çalışma alanı T ve A için çalışma alanından ayrıdır, bu nedenle herhangi bir müdahale gerçekleşemez. B tamamlandığında, ilgili sonuçları bölge T 'ye döndürecek; burada A bunu bulmayı bekleyecektir. Bu şekilde, iki program gerekli şekilde etkileşimde bulunabilir. Not edilmelidir ki, bu, bir alt program çağrıldığında gerçek bir bilgisayarda olarlara çok benzer.

Artık Turing makinelerini, bu pseudocode'u gerçek bir Turing makinesi programına nasıl çevireceğimizi (en azından teorik olarak) bildiğimiz süreç, pseudocode'da programlayabiliriz.

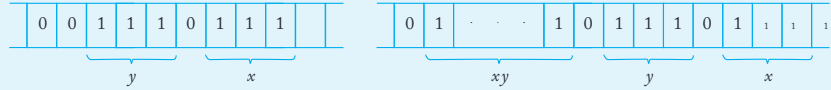
ÖRNEK 9.14

İki pozitif tam sayıyı unary notasyonda çarpan bir Turing makinesi tasarlayın.

Bir çarpma makinesi, toplama ve kopyalama sırasında karşılaştığımız fikirleri birleştirerek inşa edilebilir. Başlangıç ve bitiş band içeriğinin Şekil 9.10'da gösterildiği gibi olacağını varsayalım. Çarpma işlemi, çarpan x içindeki her 1 için çarpan y 'nin tekrar tekrar kopyalanması olarak görselleştirilebilir; bu sayede y dizisi, kısmen hesaplanmış çarpıma uygun sayıda eklenir. Aşağıdaki pseudocode, sürecin ana adımlarını göstermektedir.

1. Aşağıdaki adımları tekrarlayın x artık 1 içermediği sürece.
 x içinde 1 bul ve onu başka bir sembol a ile değiştir.
 En soldaki 0'ı 0 y ile değiştir.
2. Tüm a 'ları 1 ile değiştir.

Bu pseudocode belirsiz olsa da, fikir o kadar basit ki bunun yapılabileceğinden şüphe olmamalıdır.



ŞEKİL 9.10

Bu örneklerin tanımlayıcı doğasına rağmen, Turing makinelerinin, prensipte oldukça ilkel olmalarına rağmen, birçok şekilde birleştirilerek oldukça güçlü hale getirilebileceğini varsaymak çok da uzak bir ihtimal değildir. Örneklerimiz, herhangi bir şeyi kanıtladığımızı iddia edebilmemiz için yeterince genel ve detaylı değildi, ancak bu noktada Turing makinelerinin bazı oldukça karmaşık şeyler yapabileceği mantıklı görünmektedir.

ALISTIRMALAR

1. Örnek 9.14 için tam çözümü yazın.
2. Pozitif ve negatif tam sayıları unary notasyonda temsil etmek için bir konvansiyon belirleyin. Belirlediğiniz konvansiyon ile, $x - y$ hesaplamak için bir çıkarıcı inşasını taslağını çizin.
3. Toplayıcılar, çıkarıcılar, karşılaştırmacılar, kopyalayıcılar veya çarpanlar kullanarak, aşağıdaki fonksiyonları hesaplayan Turing makineleri için blok diyagramları çizin:

(a) $f(n) = n(n+2)$.

(b) $f(n) = n^4$.

(c) $f(n) = 2^n$.

(d) $f(n) = n!$.

(e) $f(n) = 2^{n!}$.

tüm pozitif tam sayılar n için.

4. Bir blok diyagramı kullanarak, tüm $w_1, w_2, w_3 \in \{1\}^+$ için tanımlanan bir fonksiyon f 'nin uygulanmasını taslak olarak çizin.

$$f(w_1 \dots w_n) = i,$$

burada i , eğer iki w 'nin aynı uzunlukta olmadığı durumlarda $|w| = \max(|w_1|, |w_2|, |w_3|)$ olacak şekilde tanımlanır, aksi takdirde $i = 0$ 'dır.

5. Turing makineleri için, aşağıdaki dilleri kabul eden “yüksek seviyeli” bir tanım sağlayın $\{a, b\}$. Her problem için, uygulanması makul derecede kolay olduğunu düşündüğünüz uygun makro talimatlar setini tanımlayın. Ardından bunları çözüm için kullanın.

(a) $\{ww^Rw\}$.

(b) $L = \{w_1w_2 : w_1 \neq w_2 : |w_1| = |w_2|\}$.

(c) (b) kısmındaki dilin tamamlayıcısı.

(d) $L = \{b^m : n = m^2, m \geq 1\}$.

(e) $L = \{a^n : n \text{ asal bir sayıdır}\}$.

6. Rasyonel sayıları bir Turing makinesinde temsil etmek için bir yöntem önerin, sonra bu tür sayıları toplama ve çıkarma yöntemini taslağını çizin.
7. Genel ondalık notasyonda verilen pozitif tam sayılar x ve y için toplama ve çarpma işlemlerini gerçekleştirebilen bir Turing makinesinin inşasını çizin.
8. Makro talimatın bir uygulamasını verin.

$$\text{searchleft}(a, q, q_j),$$

Bu, makinenin mevcut konumunun solundaki bandını sembol a için ilk oluşumu araması gerektiğini belirtir. Eğer a bir boşluktan önce karşılaşırsa, makine q_i durumuna geçmelidir, aksi takdirde q_j durumuna geçmelidir.

9. $\Sigma = \{a, b\}$ dilini kabul eden bir Turing makinesi tasarlamak için searchleft makro talimatını kullanın $L(ab^*ab^*a)$.

9.3 TURING'İN TEZİ

Önceki tartışma, bir Turing makinesinin daha basit parçalardan nasıl inşa edilebileceğini göstermenin yanı sıra, bu tür düşük seviyeli otomata ile çalışmanın olumsuz bir yönünü de göstermektedir. Bir blok diyagramını veya sahte kodu karşılaştıran gelen Turing makinesi programına çevirmek için çok az hayal gücü veya yaratıcılık gerekirken, bunu yapmak zaman alıcı, hata yapmaya açık ve anlayışımıza pek bir katkı sağlamamaktadır. Bir Turing makinesinin komut seti o kadar kısıtlıdır ki, sıradan bir problem için herhangi bir argüman, çözüm veya kanıt oldukça zahmetlidir.

Şimdi bir ikilemle karşı karşıyayız: Turing makinelerinin yalnızca sınırladığımız açık programlar için basit işlemleri değil, aynı zamanda blok diyagramları veya sahte kodla tanımlanabilir daha karmaşık süreçleri de gerçekleştirebileceğini iddia etmek istiyoruz. Bu tür iddiaları sınırlamalara karşı savunmak için, ilgili programları açıkça göstermeliyiz. Ancak bunu yapmak hoş değil ve dikkat dağınıktır ve mümkünse kaçınılmalıdır. Bir şekilde, uzun, düşük seviyeli kod yazmadan Turing makineleri hakkında makul bir şekilde titiz bir tartışma yürütmenin bir yolu bulunmak istiyoruz. Ne yazık ki, bu durumdan çıkmanın tamamen tatmin edici bir yolu yoktur; yapabileceğimiz en iyi şey makul bir uzlaşmaya ulaşmaktır. Böyle bir uzlaşmayı nasıl elde edebileceğimizi görmek için, biraz felsefi bir konuya döneceğiz.

Önceki bölümdeki örneklerden bazı basit sonuçlar çıkarabiliriz. İlk olarak, Turing makinelerinin itme otomata (pushdown automata) göre daha güçlü olduğu görünmektedir (bu konuda bir yorum için, bu bölümün sonunda yer alan alıştırmaya bakın). Örnek 9.8'de, bağlamdan bağımsız olmaya bir dil için bir Turing makinesinin inşasını taslak olarak çizdik ve bu dil için dolayısıyla hiçbir itme otomatası mevcut değildir. Örnekler 9.9, 9.10 ve 9.11, Turing makinelerinin bazı basit aritmetik işlemleri gerçekleştirebileceğini, dize manipülasyonları yapabileceğini ve bazı basit karşılaştırmalar yapabileceğini göstermektedir. Tartışma ayrıca, ilkel işlemlerin daha karmaşık problemleri çözmek için nasıl birleştirilebileceğini, birkaç Turing makinesinin nasıl bir araya getirilebileceğini ve bir programın başka bir program için nasıl alt program olarak işlev görebileceğini de göstermektedir. Bu şekilde de çok karmaşık işlemler inşa edilebileceğinden, bir Turing makinesinin gücünün tipik bir bilgisayara yaklaşıma başladığını düşünebiliriz.

Farz edelim ki, bir anlamda, Turing makineleri tipik bir dijital bilgisayarla güç açısından eşittir? Böyle bir hipotezi nasıl savunabilir veya çürütebiliriz? Savunmak için, giderek daha zorlaşan bir problem dizisi alabilir ve bunların nasıl bazı Turing makineleri tarafından çözüldüğünü gösterebiliriz. Ayrıca, belirli bir bilgisayarın makine dili talimat setini alabilir ve set içindeki tüm talimatları yerine getirebilen bir Turing makinesi tasarlayabiliriz. Bu kesinlikle sabrımızı zorlayacaktır, ancak hipotezimizin doğru olması durumunda prensipte mümkün olmalıdır. Yine de, bu yöndeki her başarı hipotezin doğruluğuna olan inancımızı güçlendirse de, bir kanıt sağlamaz. Zorluk, gerçekte yatan bir gerçekte yatmaktadır.

“tipik bir dijital bilgisayar”dan ne kastedildiğini tam olarak bilmediğimiz ve kesin bir tanım yapma imkanımızın olmadığıdır.

Problemi diğer taraftan da ele alabiliriz. Bilgisayar programı yazabilecek eğimiz ancak hiçbir Turing makinesinin var olamayacağını gösterebileceğimiz bir prosedür bulmaya çalışabiliriz. Eğer bu mümkün olsaydı, hipotezi reddetmek için bir temelimiz olurdu. Ancak henüz kimse bir karşı örnek üretilmiş değil; bu tür denemelerin başarısız olduğu gerçeği, bunun yapılabileceğine dair dolaylı bir kanıt olarak alınmalıdır. Turing makinelerinin prensipte herhangi bir bilgisayar kadar güçlü olduğu yönünde her belirti vardır.

Bu tür argümanlar, A. M. Turing ve diğerlerini 1930'ların ortalarında ünlü Turing tezi olarak adlandırılan varsayıma yönlendirdi. Bu hipotez, mekanik yollarla gerçekleştirilebilen her hesaplamanın bir Turing makinesi tarafından yapılabileceğini belirtir.

Bu kapsamlı bir ifadedir, bu nedenle Turing tezinin ne olduğunu akılda tutmak önemlidir. Bu, kanıtlanabilir bir şey değildir. Bunu yapmak için “mekanik yollar” terimini kesin bir şekilde tanımlamamız gerekir. Bu, başka bir soyut model gerektirecek ve bizi öncekinden daha ileriye götürmeyecektir. Turing tezi, mekanik bir hesaplamanın neyi oluşturduğuna dair daha doğru bir tanım olarak görülmelidir: Bir hesaplama, yalnızca bir Turing makinesi tarafından gerçekleştirilebiliyorsa mekaniktir.

Bu tutumu alırsak ve Turing teoremini basitçe bir tanım olarak görürsek, bu tanımın yeterince geniş olup olmadığını sorgularız. Şu anda bilgisayarlarla yaptığımız her şeyi (ve gelecekte yapabileceğimizi şeyleri) kapsayacak kadar geniş mi? Kesin bir “evet” mümkün değil, ancak bunun lehine çok güçlü kanıtlar var. Turing teoremini mekanik hesaplamanın tanımı olarak kabul etmenin bazı argümanları şunlardır:

1. Mevcut herhangi bir dijital bilgisayarda yapılabilen her şey, bir Turing makinesi tarafından da yapılabilir.
2. Henüz, sezgisel olarak bir algoritma olarak düşündüğümüz bir problemi çözmek için bir Turing makinesi programının yazılamayacağına dair bir öneri sunan kimse olmadı.
3. Mekanik hesaplama için alternatif modeller önerilmiştir, ancak bunların hiçbiri Turing makinesi modelinden daha güçlü değildir.

Bu argümanlar dolaylıdır ve Turing’in teoremi bunlarla kanıtlanamaz. Olasılığına rağmen, Turing’in teoremi hala bir varsayımdır. Ancak Turing’in teoremini basit bir keyfi tanım olarak görmek önemli bir noktayı kaçırır. Bir anlamda, Turing’in teoremi bilgisayar bilminde fizik ve kimyanın temel yasalarıyla aynı rolü oynar.

Klasik fizik, örneğin, büyük ölçüde Newton’un hareket yasalarına dayanmaktadır. Onlara yasalar dememize rağmen, mantıksal bir zorunluluğa sahip değillerdir; aksine,

fiziksel dünyanın çoğunu açıklayan makul modellere sahiptirler. Onları kabul ediyoruzçünkü onlardan çıkardığımız sonuçlar, deneyimlerimize gözlemlerimizle uyumludur. Bu tür yasaların doğru olduğu kanıtlanamaz, ancak geçersiz kılınma olasılığı vardır. Eğer bir deneysel sonuç, yasalar temelinde bir sonuca aykırıysa, geçerliliklerini sorgulamaya başlayabiliriz. Öte yandan, bir yasaı geçersiz kılma çabalarının tekrarı, ona olan güvenimizi artırır. Bu, Turing'in tezi için de geçerlidir, bu yüzden bunu bilgisayar biliminin temel yasalarından biri olarak değerlendirmek için bazı nedenlerimiz var. Ondan çıkardığımız sonuçlar, gerçek bilgisayarlar hakkında bildiklerimizle uyumludur ve şimdiye kadar, tüm geçersiz kılma girişimleri başarısız olmuştur. Her zaman, birinin Turing makineleri tarafından kapsanmayan bazı ince durumları açıklayacak başka bir tanım ortaya koyma olasılığı vardır, bununla birlikte, bu durumlar mekanik hesaplama konusundaki sezgisel anlayışımızın kapsamı içindedir. Böyle bir durumda, sonraki tartışmalarımızın bazıları önemli ölçüde değiştirilmek zorunda kalacaktır. Ancak, bunun olma olasılığı çok küçük görünmektedir.

Turing'in tezini kabul ettikten sonra, bir algoritmanın kesin bir tanımını verme konumundayız.

DEFINITION 9.5

Bir algoritma için bir fonksiyon $f: D \rightarrow R$ bir Turing makinesi M dir, bu makine verilen herhangi bir $d \in D$ girişi ile bantında, sonunda doğru cevap $f(d) \in R$ ile bantında durur. Özellikle, şunu talep edebiliriz ki

$$q_0 d \vdash_M^* q_f f(d), q_f \in F,$$

for all $d \in D$.

Bir Turing makinesi programı ile bir algoritmayı tanımlamak, "bir algoritma vardır . . ." veya "hiçbir algoritma yoktur . . ." gibi iddiaları titizlikle kanıtlamamıza olanak tanır. Ancak, nispeten basit problemler için bile açıkça bir algoritma oluşturmak oldukça uzun bir süreçtir. Bu tür hoş olmayan olasılıkları önlemek için, Turing'in teziyle başvurabiliriz ve herhangi bir bilgisayarda yapabileceğimiz her şeyin bir Turing makinesinde de yapılabileceğini iddia edebiliriz.

Sonuç olarak, Tanım 9.5'te "Turing makinesi" yerine "C programı" kullanabiliriz. Bu, algoritmaları sergileme yükünü önemli ölçüde hafifletecektir. Aslında, daha önce yaptığımız gibi, bir adım daha ileri gidecek ve eğer bize zorluk çıkarırsa, bunlar için bir Turing makinesi programı yazabileceğimiz varsayımıyla, sözlü tanımlamaları veya blok diyagramlarını algoritmalar olarak kabul edeceğiz. Bu, tartışmayı büyük ölçüde basitleştiriyor, ancak açıkça eleştiriye açık hale getiriyor. "C programı" iyi tanımlanmışken, "açık sözlü tanım" değildir ve var olmayan algoritmaların varlığını iddia etme tehlikesiyle karşı karşıyayız. Ancak bu tehlike, sahip olduğumuz gerçeklerle büyük ölçüde dengelenmektedir.

tartışmayı basit ve sezgisel olarak net tutabiliriz ve bazı oldukça karmaşık süreçler için özlü tanımlar verebiliriz. Bu iddiaların geçerliliği hakkında herhangi bir şüphesi olan okuyucu, uygun bir program yazarak bu şüpheleri giderebilir.

ALISTIRMALAR

1. Yukarıdaki tartışmada, bir noktada Turing makinelerinin, yığın otomatalarından daha güçlü görüldüğünü belirttik. Çünkü bir Turing makinesinin bandı her zaman bir yığın gibi davranacak şekilde ayarlanabilir, bu nedenle aslında Turing makinelerinin daha güçlü olduğunu iddia edebildik. Böyle bir iddianın haklı çıkarılması için ele alınması gereken önemli bir faktör hangi argümanda dikkate alınmamıştır?

BÖLÜM

10

DİĞER MODEL TURING MAKİNELERİ

BÖLÜM ÖZETİ

Bu bölümde, standart Turing makinesinin karmaşık hale getirilebileceği birçok yolu inceleyerek Turing'in tezine esasen meydan okuyoruz. Çeşitli depolama cihazlarına bakıyoruz ve hatta belirsizliğe izin veriyoruz. Bu karmaşıklıkların hiçbirisi standart makinenin temel gücünü artırmaz, bu da Turing'in tezine güvenilirlik kazandırır.

Standart bir Turing makinesinin tanımı, tek olası tanım değildir; eşit derecede iyi hizmet edebilecek alternatif tanımlar vardır.

Turing makinesinin gücü hakkında çıkarabileceğimiz sonuçlar, büyük ölçüde seçilen belirli yapıya bağımsızdır. Bu bölümde, standart Turing makinesinin, tanımlayacağımız bir anlamda, diğer, daha karmaşık modellerle eşdeğer olduğunu gösteren birkaç varyasyonu inceliyoruz.

Turing'in tezini kabul edersek, standart Turing makinesini daha karmaşık bir depolama cihazı vererek karmaşıklıklaştırmamızın otomatonun gücü üzerinde herhangi bir etkisi olmayacağını bekleriz. Böyle bir yeni düzenekte gerçekleştirilebilecek herhangi bir hesaplama, yine mekanik bir hesaplama kategorisine girecek ve dolayısıyla standart bir model tarafından yapılabilecektir. Ancak, daha karmaşık modelleri incelemek öğreticidir; eğer başka bir nedeni yoksa, beklenen sonucun açık bir gösterimi, Turing makinesinin gücünü gösterecek ve böylece Turing'in tezine olan güvenimizi artıracaktır.

Tanım 9.1'in temel modelinde birçok varyasyon mümkündür. Örneğin, birden fazla bant veya birkaç boyutta uzanan bantlara sahip Turing makinelerini düşünebiliriz.

Sonraki tartışmalarda faydalı olacak varyantları dikkate alacağız.

Belirsiz Turing makinelerini de inceliyoruz ve bunların belirleyici olanlardan daha güçlü olmadığını gösteriyoruz. Bu beklenmedik bir durumdur, çünkü Turing'in tezi yalnızca mekanik hesaplamaları kapsar ve belirsizlikteki zeki tahminleri ele almaz. Turing'in tezinin hemen çözmediği bir diğer sorun, bir makinenin farklı zamanlarda farklı programları çalıştırmasıdır. Bu, "yeniden programlanabilir" veya "evrensel" Turing makinesi fikrine yol açar.

Son olarak, sonraki bölümler için hazırlık olarak, lineer sınırlı otomataları inceliyoruz. Bunlar sonsuz bir şeridi olan Turing makineleridir, ancak şeridi yalnızca kısıtlı bir şekilde kullanabilirler.

10.1 TURING MAKİNESİ TEMASINDAKİ KÜÇÜK VARYASYONLAR

Öncelikle Tanım 9.1'deki bazı nispeten küçük değişiklikleri ele alıyoruz ve bu değişikliklerin genel kavramda herhangi bir fark yaratıp yaratmadığını araştırıyoruz. Herhangi bir tanıma değiştirdiğimizde, yeni bir otomata türü tanıtlıyoruz ve bu yeni otomataların daha önce karşılaştığımız otomatalardan gerçek anlamda farklı olup olmadığını sorguluyoruz. Bir otomata sınıfı ile diğerleri arasında temel bir fark ne anlama geliyor? Tanımlarında belirgin farklılıklar olsa da, bu farklılıkların ilginç sonuçları olmayabilir. Belirleyici ve belirsiz sonlu otomatalar durumunda bunun bir örneğini gördük. Bunların tanımları oldukça farklıdır, ancak her ikisi de tam olarak düzenli diller ailesi ile özdeşleştiği anlamında eşdeğerdir. Bunu genelleyerek, otomata sınıfları için eşdeğerlilik veya eşdeğersizlik tanımlayabiliriz.

Otomatların Sınıflarının Eşdeğerliliği

İki otomata veya otomata sınıfları için eşdeğerliliği tanımladığımızda, eşdeğerliliğin ne anlama geldiğini dikkatlice belirtmeliyiz. Bu bölümün geri kalanında, nfa 'lar ve dfa 'lar için belirlenen önceliği takip ediyoruz ve diller kabul etme yeteneğine göre eşdeğerliliği tanımlıyoruz.

TANIM 10.1

İki otomata, aynı dili kabul ediyorsa eşdeğerdir. İki otomata sınıfını C_1 ve C_2 düşünelim. Eğer C_1 içindeki her otomata M_1 için

bir otomata M_2C_2 içinde öyle ki

$$L(M_1) = L(M_2),$$

en azından C_2, C_1 kadar güçlüdür deriz. Eğer tersinin de geçerli olduğu u veher M_2 için C_2 içinde bir M_1C_1 içinde öyle ki $L(M_1) = L(M_2)$ varsa, C_1 ve C_2 eşdeğerdir deriz.

Otomatların eşdeğerliliğini belirlemenin birçok yolu vardır. Teorem 2.2'nin yapısı, dfa'lar ve nfa'lar için bunu yapar. Turing makineleri ile bağlantılı olarak eşdeğerliliği göstermek için genellikle simülasyon önemli tekniğini kullanırız.

Let M bir otomaton olsun. Başka bir otomaton $\widehat{M}$, M 'nin hesaplamasını simüle edebiliriz deriz eğer $\widehat{M}$, M 'nin hesaplamasını aşağıdaki şekilde taklit edebiliyorsa. Let $d_0, d_1, \dots M$ 'nin hesaplamasının anlık tanımlarının dizisi olsun, yani,

$$\vdash_1 \vdash \dots \vdash_n \dots$$

Sonra $\widehat{M}$ bu hesaplamayı simüle eder eğer bir

$$\widehat{d}_0^* \vdash \widehat{d}_1^* \vdash \dots \vdash \widehat{d}_n^* \dots$$

burada $\widehat{d}_0, \widehat{d}_1, \dots$ anlık tanımlamalardır, böylece her biri M ile benzersiz bir yapılandırma ile ilişkilidir. Diğer bir deyişle, eğer $\widehat{M}$ tarafından gerçekleştirilen hesaplamayı biliyorsak, ondan tam olarak ne hesaplamalar M yapmış olacağını belirleyebiliriz, verilen ilgili başlangıç yapılandırmasıyla.

Tek bir hareketin simülasyonuna d_i dikkat edin. $d_{i+1} M$ için birkaç hareketi içerebilir. M içindeki ara yapılandırmalar $d_i \xrightarrow{M} d_{i+1} M$ ile herhangi bir yapılandırmaya karşılık gelmeyebilir, ancak bu, hangi yapılandırmaların M için ilgili olduğunu belirleyebiliyorsak hiçbir şeyi etkilemez. Eğer M tarafından yapılan hesaplamadan M ne yapacağını belirleyebiliyorsak, simülasyon doğrudur. Eğer M her hesaplamayı simüle edebiliyorsa M , o zaman MM simüle edebiliriz. Eğer MM simüle edebiliyorsa, o zaman işler M ve M aynı dili kabul edecek şekilde düzlenenebilir ve iki otomat eşdeğerdir. İki otomat sınıfının eşdeğerliğini göstermek için, bir sınıftaki her makine için, onu simüle edebilen ikinci sınıfta bir makine olduğunu ve tersinin de geçerli olduğunu gösteriyoruz.

Turing Makineleri ile Bir Bekleme Seçeneği

Standart bir Turing makinesinin tanımında, okuma-yazma kafası ya sağa ya da sola hareket etmelidir.

Bazen, hücre içeriğini yeniden yazdıktan sonra okuma-yazma kafasının yerinde kalmasını sağlamak için üçüncü bir seçenek sunmak uygundur.

Bu nedenle, Bekleme Seçeneği olan bir Turing makinesini tanımlamak için Tanım 9.1'deki δ yerine

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$$

okuma-yazma kafasının hareket etmediğini belirten S ile değiştirebiliriz. Bu seçenek, otomatonun gücünü artırmaz.

TEOREM 10.1

Bekleme seçeneğine sahip Turing makineleri sınıfı, standart Turing makineleri sınıfına eşdeğerdir.

Kanıt: Bekleme seçeneğine sahip bir Turing makinesi, açıkça standart modelin bir uzantısı olduğundan, herhangi bir standart Turing makinesinin bir bekleme seçeneğine sahip olanla simüle edilebileceği açıktır.

Karşıtını göstermek için, let $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ bir Turing makinesi olsun ve bu, standart bir Turing makinesi tarafından simüle edilsin $\hat{M} = (Q, \Sigma, \Gamma, \hat{\delta}, q_0, F)$. Her M hareketi için, simüle eden makine aşağıda kileri yapar. Eğer M 'nin hareketi stay-option'ı içermiyorsa, simüle eden makine bir hareket yapar, temelde simüle edilecek harekete benzer. Eğer S M 'nin hareketinde yer alıyorsa, o zaman $\hat{M}$ iki hareket yapacaktır: İlk olarak sembolü yeniden yazar ve okuma-yazma başlığını sağa hareket ettirir; ikinci olarak okuma-yazma başlığını sola hareket ettirir, bant içeriğini değiştirmeden. Simüle eden makine, M 'den $\hat{\delta}$ 'yi aşağıdaki gibi tanımlayarak inşa edilebilir: Her geçiş için

$$\delta(q_i, a) = (q_j, b, L \text{ or } R)$$

we put into $\hat{\delta}$

$$\hat{\delta}(\hat{q}_i, a) = (\hat{q}_i, b, L \text{ or } R).$$

Her S -geçiş için

$$\delta(q_i, a) = (q_j, b, S)$$

we put into $\hat{\delta}$ the corresponding transitions

$$\hat{\delta}(\hat{q}_i, a) = (\hat{q}_{jS}, b, R)$$

ve

$$\hat{\delta}(\hat{q}_{jS}, c) = (\hat{q}_j, c, L),$$

tüm $c \in \Gamma$ için.

Her M hesaplamasının karşılık gelen bir hesaplaması olduğu oldu kça açıktır $\widehat{M}$, böylece $\widehat{M}$, M simüle edebilir. ■

Simülasyon, otomata eşdeğerliğini gösterme konusunda standart bir tekniktir ve tanımladığımız formülasyon, yukarıdaki teoremin gösterdiği gibi, süreci kesin bir şekilde konuşmayı ve eşdeğerlik hakkında teoremler kanıtlamayı mümkün kılar. Sonraki tartışmamızda, simülasyon kavramını sıkça kullanıyoruz, ancak genellikle her şeyi titiz ve ayrıntılı bir şekilde tanımlama çabası içinde değiliz. Turing makineleri ile tam simülasyonlar genellikle zahmetlidir. Bunu önlemek için, tartışmamızı tanımlayıcı tutuyoruz, teorem-kanıt biçiminde değil. Simülasyonlar yalnızca geniş bir çerçevede verilmiştir, ancak bunların nasıl titiz hale getirilebileceğini görmek zor olmamalıdır. Okuyucu, her simülasyonu daha yüksek seviyeli bir dilde veya sahte kodda taslak olarak çizmenin öğretici olduğunu görecektir.

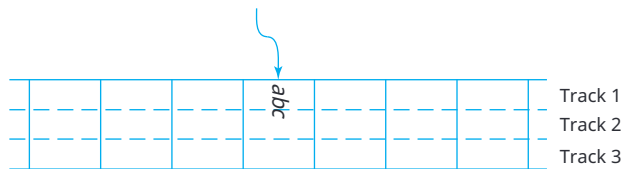
Diğer modelleri tanıtmadan önce, standart Turing makinesi hakkında bir yorum yapalım. Tanım 9.1'de her bant sembolünün yalnızca tek bir karakter değil, bir bileşik karakter olabileceği örtük olarak belirtilmiştir. Bu, bant sembollerinin daha basit bir alfabeden üçlüler olduğu genişletilmiş bir Şekil 9.1 (Şekil 10.1) çizilerek daha açık hale getirilebilir.

Resimde, şeridin her hüresini üç parçaya ayırdık, "track" olarak adlandırılan, her biri üçlülerden bir üye içeren. Bugörselleştirmeye dayanarak, böyle bir otomata bazen çoklu "track" ile Turing makinesi denir, bununla birlikte, bu bakış açısı Tanım 9.1'i hiçbir şekilde genişletmez, çünkü yapmamız gereken tek şey Γ 'yı her sembolün birkaç parçadan oluştuğu bir alfabeyle dönüştürmektir.

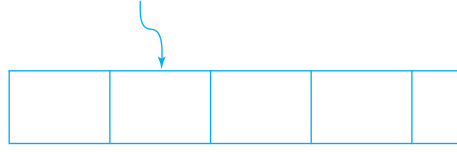
Ancak, diğer Turing makinesi modelleri bir tanım değişikliği içerir, bu nedenle standart makine ile eşdeğerliğin gösterilmesi gerekir. Burada bazen standart tanım olarak kullanılan iki böyle modele bakıyoruz. Daha az yaygın olan bazı varyantlar bu bölümün sonunda yer alan alıştırmalarda incelenmektedir.

Yarı Sonsuz Şeritli Turing Makineleri

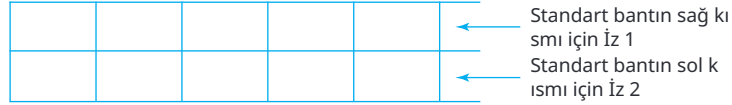
Pek çok yazar, Şekil 9.1'deki modeli standart olarak görmemekte, bir yönü sınırsız olan bir şerit ile kullanılan bir model tercih etmektedir. Bunu, sol bir sınırı olan bir şerit olarak görselleştirebiliriz (Şekil 10.2). Bu Turing makinesi



ŞEKİL 10.1



ŞEKİL 10.2



ŞEKİL 10.3

okuma-yazma kafası sınırda olduğunda sol hareketin izin verilmediği dışında, standart modelimize aynıdır.

Bu kısıtlamanın makinenin gücünü etkilemediğini görmek zor değildir. Standart bir Turing makinesi M simüle etmek için, yarı sonsuz bir bantla bir makine $\widehat{M}$ kullanıyoruz; bu, Şekil 10.3'te gösterilen düzenlemeyi kullanır.

Simüle eden makine $\widehat{M}$ iki şeritli bir banda sahiptir. Üst şeritte, M 'nin bantındaki bazı referans noktasının sağındaki bilgileri tutarız. Referans noktası, örneğin, hesaplamanın başlangıcındaki okuma-yazma kafasının konumu olabilir. Alt şerit, M 'nin bantının sol kısmını ters sırayla içerir. $\widehat{M}$, üst şeritteki bilgileri yalnızca M 'nin okuma-yazma kafası referans noktasının sağında olduğu sürece kullanacak şekilde programlanmıştır ve M bantının sol kısmına geçtiğinde alt şeritte çalışacaktır. Bu ayrım, $\widehat{M}$ 'nin durum kümesini iki parçaya ayırarak yapılabilir; diyelim ki Q_U ve Q_L : ilki üst şeritte çalışırken, ikincisi alt şeritte kullanılacaktır. Bir şeritten diğerine geçişi kolaylaştırmak için bantın sol sınırına özel son işaretleyiciler $\#$ yerleştirilmiştir. Örneğin, simüle edilecek makinenin ve simüle eden makinenin Şekil 10.4'te gösterilen ilgili konfigürasyonlarda olduğunu ve simüle edilecek hareketin

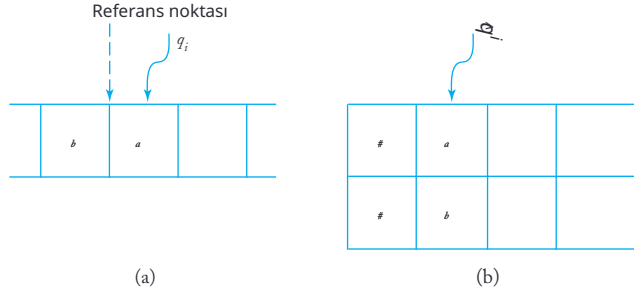
$$\delta(q_i, a) = (q_j, c, L).$$

Simüle eden makine önce geçiş aracılığıyla hareket edecektir

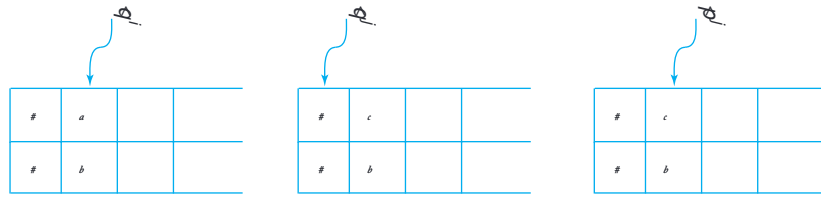
$$\widehat{\delta}(\widehat{q}_i, (a, b)) = (\widehat{q}_j, (c, b), L),$$

burada $\widehat{q}_i \in Q_U$. Çünkü $\widehat{q}_i \in Q_U$ 'ya aittir, bu noktada yalnızca üst şeritteki bilgiler dikkate alınır. Şimdi, simüle eden makine $(\#, \#)$ durumunda $\widehat{q}_j \in Q_U$ 'yu göstermektedir. Sonra bir geçiş kullanır

$$\widehat{\delta}(\widehat{q}_j, (\#, \#)) = (\widehat{p}_j, (\#, \#), R),$$



ŞEKİL 10.4(a) Simüle edilecek makine. (b) Simüle eden makine.



ŞEKİL 10.5 Simülasyonda konfigürasyonların dizisi $\delta(q_i, a) = (q_j, c, L)$.

ile $\hat{p}_j \in Q_L$, Şekil 10.5'te gösterilen konfigürasyona yerleştirerek. Şimdi makine Q_L 'den bir durumda ve alt rayda çalışacak. Simülasyonun ayrıntıları basittir.

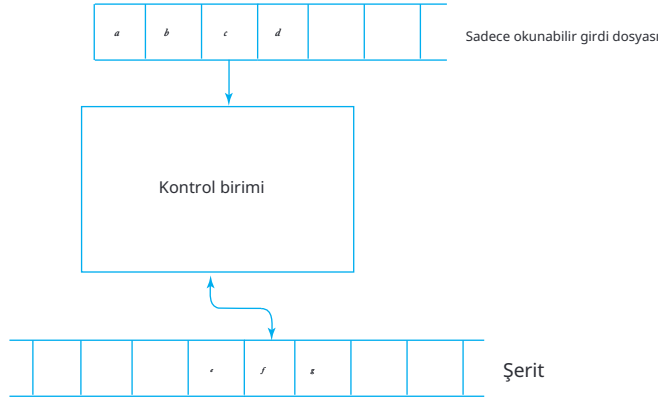
Çevrimdışı Turing Makinesi

Bölüm 1'deki bir otomatonun genel tanımı, bir girdi dosyası ve geçici depolama içeriyordu. Tanım 9.1'de, basitlik nedenleriyle girdi dosyasını çıkardık ve bunun Turing makinesi kavramını etkilemediğini iddia ettik. Şimdi bu iddiayı genişletiyoruz.

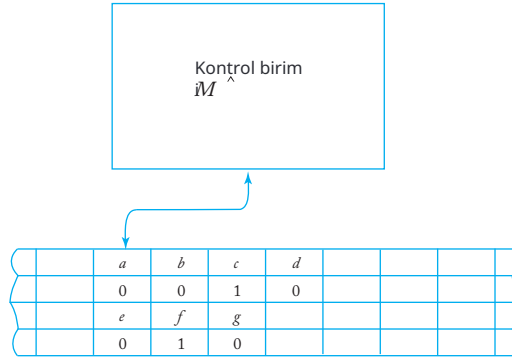
Eğer girdi dosyasını tekrar resme koyarsak, buna çevrimdışı Turing makinesi denir. Böyle bir makinede, her hareketiçsel durum, girdi dosya sından şu anda okunan ve okuma-yazma başlığı tarafından görülen şeyle yönetilir. Çevrimdışı bir makinenin şematik temsili Şekil 10.6'da gösterilmiştir. Çevrimdışı Turing makinesinin resmi bir tanımı kolayca yapılabilir, ancak bunu bir alıştırma olarak bırakacağız. Kısaca yapmak istediğimiz, çevrimdışı Turing makineleri sınıfının standart makineler sınıfına eşdeğer olduğunu belirtmektir.

Öncelikle, herhangi bir standart Turing makinesinin davranışı, bazı çevrimdışı modeller tarafından simüle edilebilir. Simüle eden makinenin yapması gereken tek şey, girdi dosyasından girişi şeride kopyalamaktır. Ardından, standart makineyle aynı şekilde devam edebilir.

Bir standart makine tarafından bir çevrimdışı makine M simülasyonu, $\hat{M}$ daha uzun bir tanım gerektirir. Bir standart makine,



ŞEKİL 10.6



ŞEKİL 10.7

Şekil 10.7'de gösterilen dört parçalı düzeni kullanarak bir çevrimdışı makinenin hesaplanması Bu resimde, gösterilen bant içeriği Şekil 10.6'nın belirli konfigürasyonunu temsil etmektedir. $\hat{M}$ 'nin dört parçasının her biri simülasyonda belirli bir rol oynamaktadır. İlk parça girişi, ikinci parça girişin okunduğu konumu, üçüncü parça M 'nin bantını ve dördüncü parça M 'nin okuma-yazma başlığının konumunu göstermektedir.

M 'nin her hareketinin simülasyonu, $\hat{M}$ 'nin bir dizi hareketini gerektirir. Belirli bir standart konumdan, örneğin sol uçtan başlayarak ve ilgili bilgilerin özel uç işaretçileriyle işaretlendiği durumda, $\hat{M}$, M 'nin giriş dosyasının okunduğu konumu bulmak için parça 2'yi arar. Parça 1'deki karşılık gelen hücrede bulunan sembol, kontrol biriminin içine yerleştirilerek hatırlanır.

$\hat{M}$ bu amaç için seçilen bir duruma dönüştürülür. Sonra, 4. iz, okuma-yazma kafasının M konumu için aranır. Hatırlanan girdi ve 3. izdeki sembol ile artık M 'nin ne yapması gerektiğini biliyoruz. Bu bilgi, uygun bir iç durum ile tekrar $\hat{M}$ tarafından hatırlanır. Sonra, tüm dört iz

tape'inin $\widehat{M}$ hareketini yansıtacak şekilde değiştirilir. Son olarak, okuma-yazma kafası $\widehat{M}$ bir sonraki hareketin simülasyonu için standart konumuna geri döner.

ALISTIRMALAR

1. Yarı sonsuz bir bant ile bir Turing makinesinin resmi tanımını verin.
2. Çevrimdışı bir Turing makinesinin resmi tanımını verin.
3. Herhangi bir belirli hareket sırasında, bant sembolünü değiştirebilen veya okuma-yazma kafasını hareket ettirebilen, ancak her ikisini birden yapamayan bir Turing makinesini düşünün.
 - (a) Böyle bir makinenin resmi tanımını verin.
 - (b) Böyle makinelerin sınıfının standart Turing makineleri sınıfına eşdeğer olduğunu gösterin.
4. Önceki egzersizde tanımlanan türde bir makinenin hareketleri nasıl simüle edebileceğinizi gösterin.

$$\delta(q_i, a) = (q_j, b, R)$$

$$\delta(q_i, b_k)$$

standart bir Turing makinesinin.

5. Her hareketin okuma-yazma kafasının bir hücreden daha fazla sola veya sağa hareket etmesine izin verdiği bir Turing makinesi modelini düşünün; hareketin mesafesi ve yönü, δ 'nin argümanlarından biridir. Böyle bir otomatonun kesin bir tanımını verin ve onu standart bir Turing makinesi ile simüle edin.
6. Silinmeyen bir Turing makinesi, boş olmayan bir sembolü boş bir sembole değiştiremeyen bir makinedir. Bu, eğer

$$\delta(q_i, a) = (q_j, \square, L \text{ or } R),$$

o zaman a olmalıdır. Böyle bir kısıtlama getirmenin genel birliği kaybettirmediğini gösterin.

7. Boş yazamayan bir Turing makinesini düşünün; yani, tüm $\delta(q_i, a) = (q_j, b, L \text{ veya } R)$ için, $b \in \Gamma - \{\square\}$ içinde olmalıdır. Böyle bir makinenin standart bir Turing makinesini nasıl simüle edebileceğini gösterin.
8. Bir Turing makinesinin yalnızca bir son durumda durabileceği gereksinimini getirdiğimizi varsayalım: yani, tüm (q, a) çiftleri için $\delta(q, a)$ tanımlı olmasını istiyoruz, $a \in \Gamma$ ve $q \in F$. Bu, Turing makinesinin gücünü kısıtlar mı?
9. Bir Turing makinesinin her zaman okuduğu sembolden farklı bir sembol yazması gerektiği kısıtlamasını getirdiğimizi varsayalım: yani, eğer

$$\delta(q_i, a) = (q, b, L),$$

o zaman a ve b farklı olmalıdır. Bu kısıtlama otomatonun gücünü azaltır mı?

10. Geçişlerin yalnızca okuma-yazma kafasının doğrudan altında bulunan hücreye değil, aynı zamanda hemen sağdaki ve soldaki hücrelere de bağlı olabileceği standart Turing makinesinin bir versiyonunu düşünün. Böyle bir makinenin resmi tanımını yapın, ardından standart bir Turing makinesi tarafından simülasyonunu taslak olarak çizin.
11. Farklı bir karar sürecine sahip bir Turing makinesini düşünün; burada geçişler, mevcut bant sembolü belirli bir kümenin elemanlarından biri değilse yapılır. Örneğin,

$$\delta(q_i, \{a, b\}) = (q_j, c, R)$$

mevcut bant sembolü ne a ne de b ise belirtilen hareketi gerçekleştirecektir.

Bu kavramı biçimlendirin ve bu tür bir Turing makinesinin standart bir Turing makinesi ile eşdeğer olduğunu gösterin.

12. Önceki egzersizde tanımlanan türde bir Turing makinesi için bir program yazın; bu makine $L(aa^*bb^*)$ kabul etsin. $\Sigma = \{a, b\}$ ve $\Gamma = \{a, b, \}$ varsayalım.

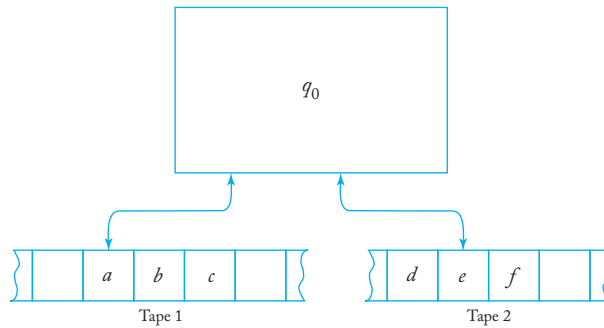
10.2 Daha Karmaşık Depolama ile Turing Makineleri

Standart bir Turing makinesinin depolama cihazı o kadar basittir ki, daha karmaşık depolama cihazları kullanarak güç kazanmanın mümkün olduğunu düşünebilirsiniz. Ancak durum böyle değildir; bunu şimdi iki örnekle açıklayacağız.

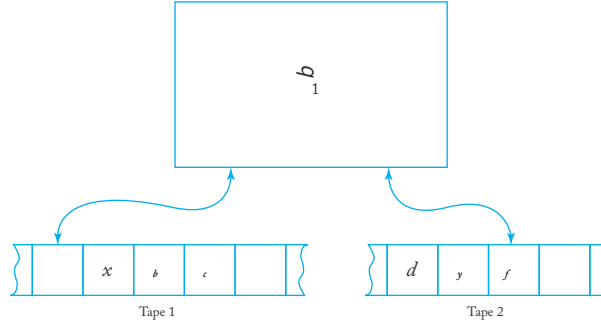
Çok Bantlı Turing Makineleri

Çok bantlı bir Turing makinesi, her biri bağımsız olarak kontrol edilen okuma-yazma kafasına sahip birkaç bantla donatılmış bir Turing makinesidir (Şekil 10.8).

Bir çok bantlı Turing makinesinin resmi tanımı, değiştirilmiş bir geçiş fonksiyonu gerektirdiğinden, Tanım 9.1'den daha ileri gider. Genellikle, bir n -bant makinesini $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ olarak tanımlarız; burada $Q, \Sigma, \Gamma, q_0, F$



ŞEKİL 10.8



ŞEKİL 10.9

Tanım 9.1'de olduğu gibidir, ancak

$$\delta : Q \times \Gamma^n \rightarrow Q \times \Gamma^n \times \{L, R\}^n$$

tüm bantlarda neler olduğunu belirtir. Örneğin, eğer $n = 2$ ise, Şekil 10.8'de gösterilen mevcut yapılandırma ile, o zaman

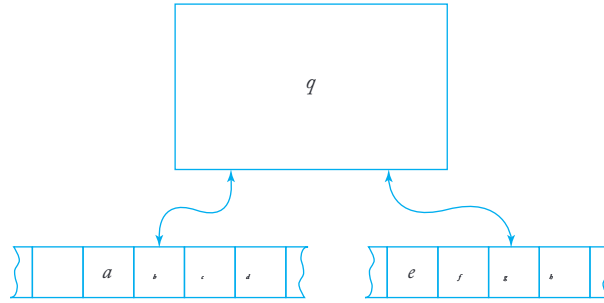
$$\delta(q_0, a, e) = (q_1, x, y, L, R)$$

aşağıdaki gibi yorumlanır. Geçiş kuralı yalnızca makine q_0 durumunda olduğunda ve ilk okuma-yazma kafası a gördüğünde ve ikincisi e gördüğünde uygulanabilir. İlk banttaki sembol, o zaman x ile değiştirilir ve okuma-yazma kafası sola hareket eder. Aynı zamanda, ikinci banttaki sembol y olarak yeniden yazılır ve okuma-yazma kafası sağa hareket eder. Kontrol birimi daha sonra durumunu q_1 olarak değiştirir ve makine, Şekil 10.9'da gösterilen yeni yapılandırmaya geçer.

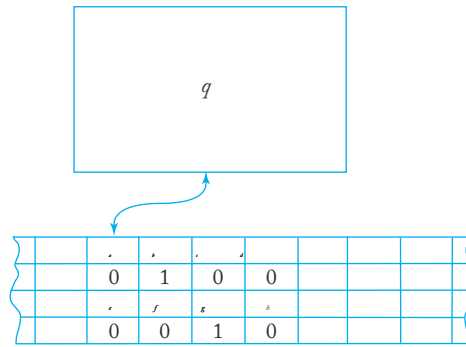
Çok bantlı ve standart Turing makineleri arasındaki eşdeğerliliği göstermek için, herhangi bir çok bantlı Turing makinesi M 'nin standart bir Turing makinesi $\widehat{M}$ tarafından simüle edilebileceğini ve tersine, herhangi bir standart Turing makinesinin bir çok bantlı makine tarafından simüle edilebileceğini savunuyoruz. Bu iddianın ikinci kısmının açıklamaya ihtiyacı yoktur, çünkü her zaman yalnızca bir bantla faydalı iş yapan bir çok bantlı makine çalıştırmayı seçebiliriz. Tek bantlı bir makine tarafından çok bantlı bir makinenin simülasyonu biraz daha karmaşıktır, ancak kavramsal olarak basittir.

Örneğin, Şekil 10.10'da gösterilen yapılandırmadaki iki bantlı makineyi düşünün. Simüle edilen tek bantlı makine dörtşeride sahip olacaktır (Şekil 10.11). İlk şerit, M 'nin bant 1'inin içeriğini temsil eder. İkinci şeridin boş olmayan kısmı, M 'nin okuma-yazma başlığının konumunu işaret eden tek bir 1 dışında tamamen sıfırlardan oluşmaktadır. Şerit 3 ve 4, M 'nin bant 2'si için benzer bir rol oynamaktadır. Şekil 10.11, ilgili $\widehat{M}$ yapılandırmaları için (yani, belirtilen forma sahip olanlar) M 'nin benzersiz bir karşılık gelen yapılandırmasının olduğunu açıkça göstermektedir.

Bir çok bantlı makinenin tek bantlı bir makine ile temsil edilmesi, çevrimdışı bir makinenin simülasyonunda kullanılanla benzerdir. Gerçek



ŞEKİL 10.10



ŞEKİL 10.11

simülasyondaki adımlar da çok benzer, tek fark daha fazla bantın dikkate alınmasıdır. Çevrimdışı makinelerin simülasyonu için verilen taslak, bu duruma küçük değişikliklerle aktarılır ve geçiş fonksiyonu $\delta^{\widehat{M}}$ için, geçiş fonksiyonu δ^M 'den nasıl inşa edileceğini öneren bir prosedür sunar. İnşayı kesin hale getirmek zor değildir, ancak çok yazı yazmayı gerektirir. Kesinlikle, $\widehat{M}$ 'nin hesaplamaları uzun ve karmaşık görünme eğilimindedir, ancak bu sonuca etki etmez. M üzerinde yapılabilecek her şey, $\widehat{M}$ üzerinde de yapılabilir.

Şu noktayı akılda tutmak önemlidir. Birden fazla bantlı bir Turing makinesinin standart bir Turing makinesinden daha güçlü olmadığını iddia ettiğimizde, bu makinelerin neler yapabileceği hakkında bir ifade vermekteyiz, özellikle hangi dillerin kabul edilebileceği konusunda.

ÖRNEK 10.1

Dili $\{a^n b^n\}$.
bu dilin bir Turing

Örnek 9.7'de, bir

makinesi tarafından kabul edilebileceği zahmetli bir yöntemdir. İki bantlı bir makine kullanmak işi çok

daha kolay. Başlangıçta bir dize $a^n b^m$, hesaplamanın başında bant 1'e yazılmıştır. Sonra tüm a 'ları okuyoruz, bunları bant 2'ye kopyalıyoruz. a 'ların sonuna geldiğimizde, bant 1'deki b 'leri bant 2'ye kopyalan a 'larla eşleştiriyoruz. Bu şekilde, okuma-yazma kafasının tekrar tekrar ileri geri hareket etmesine gerek kalmadan a 'ların ve b 'lerin eşit sayıda olup olmadığını belirleyebiliriz.

Unutmayın ki, Turing makinelerinin çeşitli modelleri, yalnızca şeyleri yapabilme yetenekleri açısından eşdeğer kabul edilir, programlamanın kolaylığı veya dikkate alabileceğimiz diğer verimlilik ölçütleri açısından değil. Bu önemli noktaya 14. Bölümde döneceğiz.

Çok Boyutlu Turing Makineleri

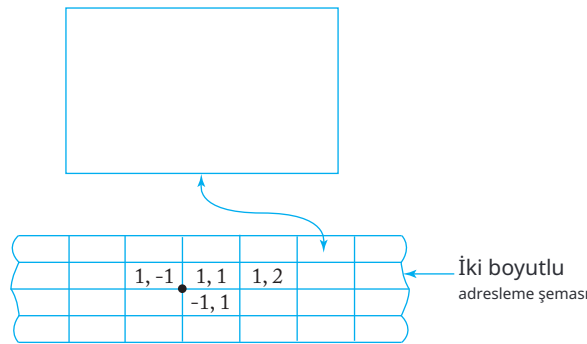
Çok boyutlu bir Turing makinesi, şeridin birden fazla boyutta sonsuz uzandığı şekilde görülebileceği bir makinedir. İki boyutlu bir Turing makinesinin diyagramı Şekil 10.12'de gösterilmiştir.

İki boyutlu bir Turing makinesinin resmi tanımı, aşağıdaki biçimde bir geçiş fonksiyonu δ içerir

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, U, D\},$$

burada U ve D okuma-yazma kafasının yukarı ve aşağı hareketini belirtir.

Bu makineyi standart bir Turing makinesinde simüle etmek için, Şekil 10.13'te gösterilen iki izli modeli kullanabiliriz. Öncelikle, bir sıralama ile ilişkilendiriyoruz



ŞEKİL 10.12

	a					b				
	1	#	2	#	1	0	#	-	3	#

ŞEKİL 10.13

veya iki boyutlu bantın hücreleri ile adres. Bu, bir dizi şeklinde yapılabilir, örneğin, Şekil 10.12'de gösterildiği gibi iki boyutlu bir şekilde. Simüle edici makinenin iki izli bantı, bir izde hücre içeriklerini depolamak ve diğerinde iliskili adresi tutmak için kullanılacaktır. Şekil 10.12'deki şemada, hücre (1, 2)'nin a içerdiği ve hücre (10, -3)'ün b içerdiği yapı Şekil 10.13'te gösterilmektedir. Bir karmaşıklığa dikkat edin: Hücre adresi, keyfi olarak büyük tam sayılar içerebilir, bu nedenle adres izinin adresleri depolamak için sabit boyutlu bir alan kullanması mümkün değildir. Bunun yerine, alanları sınırlamak için bazı özel semboller kullanarak değişken alan boyutu düzenlemesi yapılmalıdır, resimde gösterildiği gibi.

Simülasyonun her hareketinin başlangıcında, iki boyutlu makinenin okuma-yazma kafası M ve simüle eden makinenin okuma-yazma kafası $\widehat{M}$ her zaman karşılık gelen hücrelerde bulunmaktadır. Bir hareketi simüle etmek için, simüle eden makine $\widehat{M}$ önce M 'ye hareket etmesi gereken hücrenin adresini hesaplar. İki boyutlu adresleme şemasını kullanarak, bu basit bir hesaplamadır. Adres hesaplandıktan sonra, $\widehat{M}$ bu adrese sahip hücreyi 2. traktta bulur ve ardından hücre içeriğini M 'nin hareketini hesaba katarak şekilde değiştirir. Yine, verilen M için, $\widehat{M}$ için basit bir yapı vardır.

ALISTIRMALAR

- İki bantlı Turing makinesinin resmi tanımını verin; ardından aşağıdaki dilleri kabul eden programlar yazın. $\Sigma = \{a, b, c\}$ olduğunu ve girişin başlangıçta tamamen bant 1'de olduğunu varsayın.

(a) $L = \{a^n b^n c^n, n \geq 1\}$.

(b) $L = \{a^n b^n c^m, m > n\}$.

(c) $L = \{ww : w \in \{a, b\}^*\}$.

(d) $L = \{ww^R w : w \in \{a, b\}^*\}$.

(e) $L = \{n_a(w) = n_b(w) = n_c(w)\}$.

(f) $L = \{n_a(w) = n_b(w) \geq n_c(w)\}$.

- Birçok bantlı çevrimdışı Turing makinesi olarak adlandırılacak bir tanım yapın ve bunun standart bir Turing makinesi tarafından nasıl simüle edilebileceğini açıklayın.
- Çok başlı bir Turing makinesi, tek bir bant ve tek bir kontrol birimi olan ancak birden fazla, bağımsız okuma-yazma kafasına sahip bir Turing makinesi olarak görsel eşitirilebilir. Çok başlı bir Turing makinesinin resmi tanımını verin ve ardından böyle bir makinenin standart bir Turing makinesi ile nasıl simüle edilebileceğini gösterin.
- Çok başlı çok bantlı bir Turing makinesinin resmi tanımını verin. Ardından, böyle bir makinenin standart bir Turing makinesi tarafından nasıl simüle edilebileceğini gösterin.

5. Tek bir bantı olan ancak her biri tek bir okuma-yazma kafasına sahip birden fazla kontrol birimi bulunan bir Turing makinesinin resmi tanımını verin. Böyle bir makinenin çok bantlı bir makine ile nasıl simüle edilebileceğini tartışın.

10.3 BELİRSİZ TURING MAKİNELERİ

Turing'in tezi, belirli bant yapısının Turing makinesinin gücü için önemli olduğunu makul kılarken, belirsizlik için aynı şey söylenemez. Belirsizlik, bir seçim unsuru içerdiğinden ve bu nedenle mekanik olmaya n bir tat taşıdığından, Turing'in tezine başvurmak uygun değildir. Bir Turing makinesinin gücüne belirsizliğin hiçbir şey ekmediğini savunmak istiyorsak, belirsizliğin etkisini daha ayrıntılı bir şekilde incelemeliyiz. Yine simülasyona başvuruyoruz, belirsiz davranışın deterministik olarak ele alınabileceğini gösteriyoruz.

DEFINITION 10.2

Belirsiz bir Turing makinesi, Tanım 9.1'de verilen bir otomattır, tek farkı δ artık bir fonksiyondur.

$$\delta : Q \times \Gamma \rightarrow 2^{Q \times \Gamma \times \{L, R\}}.$$

Her zaman belirsizlik söz konusu olduğunda, δ aralığı, makine tarafından seçilebilecek olası geçişlerin bir kümesidir.

ÖRNEK 10.2

Bir Turing makinesi geçişleri şu şekilde belirtilmişse

$$\delta(q_0, a) = \{(q_1, b, R), (q_2, c, L)\},$$

belirsizdir. Hareketler

$$q_0aaa \vdash bq_1aa$$

ve

$$q_0aaa \vdash q_2\Box caa$$

her ikisi de mümkündür.

		$\times$	a	b_0	a	a	$+$	b	b	b_1	a	$\times$	
--	--	----------	-----	-------	-----	-----	-----	-----	-----	-------	-----	----------	--

ŞEKİL 10.14

Belirsizliğin hesaplama fonksiyonlarında ne rol oynadığı net olmadığından, belirsiz otomata genellikle kabul ediciler olarak görülür. Bir belirsiz Turing makinesi, herhangi bir olası hareket dizisi varsa kabul eder denir.

$$q_0 w \vdash^* x_1 q_f x_2,$$

$q_f \in F$ ile. Bir belirsiz makine, son olmayan bir duruma veya sonsuz bir döngüye yol açabilecek hareketlere sahip olabilir. Ancak, her zaman olduğu gibi belirsizlikte, bu alternatifler önemsizdir; bizim ilgilendiğimiz tek şey, kabul ile sonuçlanan bazı hareket dizilerinin varlığıdır.

Bir belirsiz Turing makinesinin, deterministik bir makineden daha güçlü olmadığını göstermek için, belirsizlik için bir deterministik eşdeğer sağlamamız gerekir. Bununla ilgili bir ipucu verdik. Belirsizlik, deterministik bir geri izleme algoritması olarak görülebilir ve bir deterministik makine, geri izleme ile ilgili muhasebe işlemlerini yönetebildiği sürece belirsiz bir makineyi simüle edebilir. Bunu basitçe nasıl yapabileceğimizi görmek için, belirsizliğin alternatif bir görüşünü düşünelim; bu, birçok argümanda faydalıdır: Bir belirsiz makine, gerektiğinde kendini çoğaltma yeteneğine sahip biri olarak görülebilir. Birden fazla hareket mümkün olduğunda, makine ihtiyaç duyduğu kadar kopya üretir ve her kopyaya alternatiflerden birini gerçekleştirme görevini verir. Bu belirsizlik görüşü, sınırsız çoğaltmanın kesinlikle günümüz bilgisayarlarının gücünde olmadığı düşünüldüğünde, özellikle mekanik olmayan bir görünüm sergileyebilir.

Yine de, bir simülasyon mümkündür.

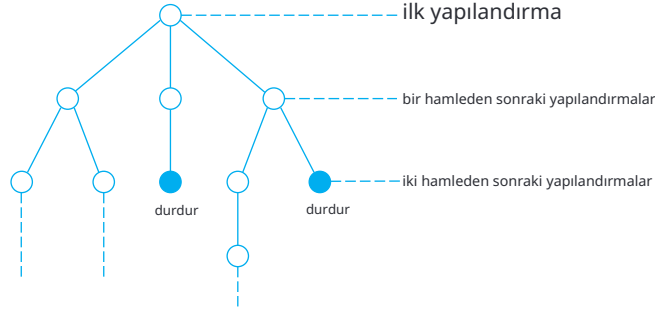
Simülasyonu görselleştirmenin bir yolu, standart bir Turing makinesi kullanmaktır, belirsiz makinenin tüm olası anlık tanımlarını şeridinde tutarak, bazı kurallara göre ayrılmıştır. Şekil 10.14, iki yapılandırmanın aq_0aa ve bbq_1a nasıl görünebileceğini göstermektedir. Sembol $\times$ ilgili alanını sınırlamak için kullanılırken, $+$ bireysel anlık tanımları ayırır. Simüle eden makine, tüm aktif yapılandırmalara bakar ve bunları belirsiz makinenin programına göre günceller. Yeni yapılandırmalar veya genişleyen anlık tanımlar, $\times$ işaretleyicilerini hareket ettirmeyi gerektirir. Detaylar kesinlikle sıkıcıdır, ancak görselleştirmesi zor değildir. Bu simülasyona dayanarak, her belirsiz Turing makinesi için eşdeğer bir belirli standart makinenin var olduğunu sonucuna varıyoruz.

TEOREM 10.2

Belirli Turing makineleri sınıfı ile belirsiz Turing makineleri sınıfı eşdeğerdir.

Kanıt: Yukarıda önerilen yapıyı kullanarak, herhangi bir belirsiz Turing makinesinin belirli bir makine tarafından simüle edilebileceğini gösterin. ■

Daha sonra pratik durumlarda belirsizliğin etkisini yeniden değerlendireceğiz, bu nedenle bazı yorumlar eklememiz gerekiyor. Her zamanki gibi, belirsizlik alternatifler arasında bir seçim olarak görülebilir. Bu, bir karar ağacı olarak görseleştirilebilir (Şekil 10.15).



ŞEKİL 10.15

Böyle bir yapılandırma ağacının genişliği, dallanma faktörüne bağlıdır, yani her hamlede mevcut olan seçeneklerin sayısına. Eğer k maksimum dallanmayı belirtirse, o zaman

$$M = k^n \quad (10.1)$$

n hamle sonrasında var olabilecek maksimum yapılandırma sayısıdır.

Sonraki amaçlar için, dil kabulünün tanımını ayrıntılı olarak açıklama k ve ayrıca üyelik sorununu da dahil etmek gereklidir.

TANIM 10.3

Bir belirsiz Turing makinesi M , bir dili L kabul eder denir eğer, tüm $w \in L$ için, mümkün olan yapılandırmalardan en az biri w 'yi kabul ediyorsa. Kabul etmeyen yapılandırmalara giden dallar olabilir, bazıları ise makineyi sonsuz bir döngüye sokabilir. Ancak bunlar kabul için alakasızdır.

Bir belirsiz Turing makinesi M , bir dili L karar verir denir eğer, tüm $w \in \Sigma^*$ için, kabul veya reddetmeye giden bir yol varsa.

ALISTIRMALAR

1. Belirli bir Turing makinesi tarafından belirsiz bir Turing makinesinin simülasyonunu tartışın. Yeni makinelerin nasıl oluşturulduğunu, aktif makinelerin nasıl tanımlandığını ve duraklayan makinelerin nasıl daha fazla değerlendirmeden çıkarıldığını açıkça belirtin.
2. Belirli Turing makineleri için aşağıdaki dilleri kabul eden programlar yazın. Her durumda, belirsizliğin görevi nasıl basitleştirdiğini açıklayın.

- (a) $L = \{ ww : w \in \{a, b\}^+ \}$.
- (b) $L = \{ ww^R w : w \in \{a, b\}^+ \}$.
- (c) $L = \{ xww^R y : x, y, w \in \{a, b\}^+, |x| \geq |y| \}$.
- (d) $L = \{ w : n_a(w) = n_B(w) = n_c(w) \}$.
- (e) $L = \{ a^n : n \text{ asal bir sayı değildir} \}$.

3. İki yığınlı otomaton, iki bağımsız yığına sahip olan belirsiz bir itme otomatonudur. Böyle bir otomatonu tanımlamak için, Tanım 7.1'i değiştiriyoruz, böylece

$$\delta : Q \times (\Sigma \cup \{\lambda\}) \times \Gamma \times \Gamma \rightarrow \text{finite subsets of } Q \times \Gamma^* \times \Gamma^*.$$

Bir hareket, iki yığının üstlerine bağlıdır ve bu iki yığına yeni değerlerin itildiği sonuçlanır. İki yığınlı otomaton sınıfının, Turing makineleri sınıfına eşdeğer olduğunu gösterin.

10.4 BİR EVRENSEL TURING MAKİNESİ

Aşağıdaki argümanı Turing'in tezi aleyhine düşünün: "Tanım 9.1'de sunulan bir Turing makinesi, özel amaçlı bir bilgisayardır. Bir kez tanımlandığında, makine belirli bir tür hesaplamayı gerçekleştirmekle sınırlıdır. Öte yandan, dijital bilgisayarlar, farklı zamanlarda farklı işler yapmak üzere programlanabilen genel amaçlı makinelerdir. Sonuç olarak, Turing makineleri genel amaçlı dijital bilgisayarlarla eşdeğer olarak kabul edilemez.

Bu itiraz, yeniden programlanabilir bir Turing makinesi tasarlayarak aşılabılır, buna evrensel Turing makinesi denir. Evrensel Turing makinesi M_m , herhangi bir Turing makinesinin M tanımını ve bir dize w 'yi girdi olarak aldığı anda, M 'nin w üzerindeki hesaplamasını simüle edebilen bir otomattır. Böyle bir M_m 'yi inşa etmek için önce Turing makinelerini tanımlamanın standart bir yolunu seçeriz. Genel olarak kayıptan kaçınmadan,

$$Q = \{q_1, q_2, \dots, q_n\},$$

q_1 başlangıç durumu, q_2 tek final durumu ve

$$\Gamma = \{a_1, a_2, \dots, a_m\},$$

a_1 boşluğu temsil eder. Daha sonra q_1 'in 1 ile, q_2 'nin 11 ile temsil edildiği bir kodlama seçeriz ve benzeri şekilde, a_1 'in 1 olarak, a_2 'nin 11 olarak kodlandığını varsayalım. 1'ler arasında ayırıcı olarak 0 sembolü kullanılacaktır. Bu konvansiyonla tanımlanan başlangıç ve final durumu ile boşluk, herhangi bir Turing makinesi yalnızca δ ile tamamen tanımlanabilir. Geçiş fonksiyonu, argümanlar ve sonuç belirli bir sıraya göre bu şemaya göre kodlanır. Örneğin, $\delta(q_1, a_2) = (q_2, a_3, L)$ şu şekilde görünebilir:

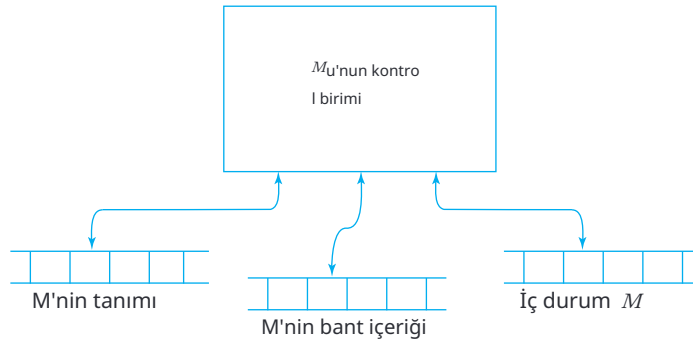
... 1 0 1 1 0 1 1 0 1 1 0 1 0 ...

Bu, herhangi bir Turing makinesinin bir dize üzerinde sonlu bir kodlamaya sahip olduğunu ve herhangi bir M kodlaması verildiğinde, bunu benzersiz bir şekilde çözebileceğimizi gösterir. Bazı dizeler herhangi bir Turing makinesini temsil etmeyecektir (örneğin, 00011 dizisi), bunları kolayca tespit edebiliriz, bu nedenle endişe kaynağı değildir.

Evrensel bir Turing makinesi M_u , $\{0,1\}$ içeren bir girdi alfabesine ve Şekil 10.16'da gösterildiği gibi çok bantlı bir makine yapısına sahiptir.

Herhangi bir girdi M ve w için, bant 1 M 'nin kodlanmış tanımını saklayacaktır. Bant 2, M 'nin bant içeriğini içerecek ve bant 3 M 'nin iç durumunu içerecektir. M_u , M 'nin konfigürasyonunu belirlemek için önce bant 2 ve 3'ün içeriğine bakar. Ardından, bu konfigürasyonda M 'nin ne yapacağını görmek için bant 1'e danışır. Son olarak, bant 2 ve 3, hareketin sonucunu yansıtacak şekilde değiştirilecektir.

Gerçek bir evrensel Turing makinesi inşa etmek makul bir durumdur (örneğin, Denning, Dennis ve Qualitz 1978'e bakın), ancak süreç ilginç değildir. Bunun yerine Turing'in hipotezine başvurmayı tercih ediyoruz. Uygulamanın açıkça bazı programlama dilleri kullanılarak yapılabileceği;



ŞEKİL 10.16

Aslında, Bölüm 9.1, Egzersiz 1'de önerilen program, daha yüksek seviyeli bir dilde evrensel bir Turing makinesinin bir gerçekleştirilmesidir. Bu nedenle, bunun standart bir Turing makinesi tarafından da yapılabileceğini bekliyoruz. O zaman, herhangi bir program verildiğinde, o program tarafından belirtilen hesaplamaları gerçekleştirebilen bir Turing makinesinin varlığını iddia etme hakkına sahibiz ve bu nedenle genel amaçlı bir bilgisayar için uygun bir modeldir.

Her Turing makinesinin 0'lar ve 1'ler dizisi ile temsil edilebileceği gözlemlenebilir önemli sonuçlar doğurmaktadır. Ancak bu sonuçları keşfetmeden önce, bazı küme teorisi sonuçlarını gözden geçirmemiz gerekiyor.

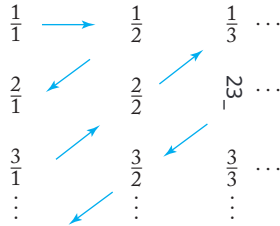
Bazı kümeler sonludur, ancak ilginç olan çoğu küme (ve diller) sonsuzdur. Sonsuz kümeler için, sayılabilir kümeler ile sayılamaz kümeler arasında ayrım yaparız. Bir kümenin elemanları pozitif tam sayılarla bire bir karşılıştırılabilirse, o kümenin sayılabilir olduğu söylenir. Bununla, kümenin elemanlarının belirli bir sırayla yazılabileceği anlamına gelir; diyelim ki, $x_1, x_2, x_3, \dots$, böylece kümenin her elemanının bazı sonlu bir indeksi vardır. Örneğin, tüm çift tam sayılar kümesi $0, 2, 4, \dots$ sırasıyla yazılabilir. Herhangi bir pozitif tam sayı $2n$, $n + 1$ konumunda yer aldığından, küme sayılabilir. Bu çok şaşırtıcı olmamalı, ancak daha karmaşık örnekler var, bazıları sezgisel olarak karşıt görünebilir. Pozitif tam sayılar olan p ve q biçimindeki tüm kesirlerin kümesini ele alalım. Bu kümenin sayılabilir olduğunu göstermek için nasıl sıralamalıyız? Bu diziyi kullanamayız

$$\frac{1}{1}, \frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \dots$$

çünkü o zaman³ asla görünmeyecek. Bu, kümenin sayılabilir olmadığı anlamına gelmez; bu durumda, kümenin aslında sayılabilir olduğunu göstermek için akılcıca bir sıralama yolu vardır. Şekil 10.17'de gösterilen şekle bakın, ve okları takip ederek karşılaştığınız sırayla elemanı yazın. Bu bize verir

$$\frac{1}{1}, \frac{1}{2}, \frac{2}{1}, \frac{3}{1}, \frac{2}{2}, \dots$$

Burada eleman³-yedinci sırada yer alır ve her elemanın dizide bir konumu vardır. Bu nedenle küme sayılabilir.



ŞEKİL 10.17

Bu örnekten, bir kümenin sayılabilir olduğunu kanıtlayabileceğimizi görmekteyiz eğer öğelerinin bir sırada yazılabileceği bir yöntem üretebilirsek. Bu tür bir yöntem sayım prosedürü diyoruz. Sayım prosedürü bir tür mekanik süreç olduğundan, bunuresmi olarak tanımlamak için bir Turing makinesi modeli kullanabiliriz.

DEFINITION 10.4

Bir S , bazı alfabelerden Σ üzerinde bir dizi dize olsun. O zaman S için bir sıralama prosedürü, adım dizisini gerçekleştirebilen bir Turing makinesidir.

$$q_0 \vdash^* q_s x_1 \# s_1 \vdash^* q_s x_2 \# s_2 \dots,$$

$x_i \in \Gamma^* - \{ \# \}, s_i \in S$ şeklinde, böylece S içindeki herhangi bir s , sonlu sayıda adımda üretilir. Durum q_s , S 'ye üyeliği belirten bir durumdur; yani, ne zaman q_s girilirse, $\#$ 'dan sonraki dize S içinde olmalıdır.

Her küme sayılabilir değildir. Bir sonraki bölümde göreceğimiz gibi, bazı sayılabilir olmayan kümeler vardır. Ancak bir sıralama prosedürü mevcut olan herhangi bir küme sayılabilir, çünkü sıralama gerekli diziyi verir.

Kesin olarak konuşursak, bir sıralama prosedürü bir algoritma olarak adlandırılmaz, çünkü S sonsuz olduğunda sonlanmayacaktır. Yine de, iyi tanımlanmış ve öngörülebilir sonuçlar ürettiği için anlamlı bir süreç olarak kabul edilebilir.

EXAMPLE 10.3

$\Sigma = \{a, b, c\}$ olsun. Eğer elemanlarını bir sırayla üreten bir sıralama prosedürü bulabilirsek, $S = \Sigma^+$ sayılabilir olduğunu gösterebiliriz, örneğin, bir sözlükte görünecekleri sırayla. Ancak, sözlüklerde kullanılan sıra, değişiklik olmadan uygun değildir. Bir sözlükte, a ile başlayan tüm kelimeler, b dizesinden önce listelenir. Ancak sonsuz sayıda a kelimesi olduğunda, b 'ye asla ulaşamayız, bu da tanım 10.4'te belirtilen herhangi bir dize sonlu sayıda adımda listelenmesi koşulunu ihlal eder.

Bunun yerine, ilk kriter olarak dizinin uzunluğunu alıp, ardından eşit uzunluktaki tüm dizelerin alfabetik sıralamasını takip ettiğimiz değiştirilmiş bir sıralama kullanabiliriz. Bu, diziyi veren bir sıralama prosedürüdür.

$$a, b, c, aa, ab, ac, ba, bb, bc, ca, cb, cc, aaa, \dots$$

Bu tür bir sıralama için birkaç kullanıma olacağından, buna uygun sıralama adını vereceğiz.

Önceki tartışmanın önemli bir sonucu, Turing makinelerinin sayıla bilir olduğudur.

TEOREM 10.3

Tüm Turing makinelerinin kümesi, sonsuz olmasına rağmen, sayılabilir.

Kanıt: Her Turing makinesini 0 ve 1 kullanarak kodlayabiliriz. Bu kodlama ile, aşağıdaki sıralama prosedürünü oluşturuyoruz.

1. Uygun sıralamada $\{0,1\}^+$ bir sonraki diziyi üretin.
2. Üretilen diziyi kontrol edin ve bir Turing makinesini tanımlayıp tanımlamadığını görün. Eğer tanımlıyorsa, onu Tanım 10.4'te belirtilen biçimde bantta yazın. Eğer tanımlamıyorsa, diziyi göz ardı edin.
3. Adım 1'e geri dön.

Her Turing makinesinin sonlu bir tanımı olduğundan, bu süreçle herhangi bir belirli makine sonunda üretilecektir. ■

Özel Turing makinelerinin sıralaması, kullandığımız kodlamaya bağlıdır; eğer farklı bir kodlama kullanırsak, farklı bir sıralama beklemeliyiz. Ancak bu önemsizdir ve sıralamanın kendisinin önemsiz olduğunu gösterir. Önemli olan, bazı bir sıralamanın varlığıdır.

ALISTIRMALAR

1. Verilen yöntemi kullanarak kodlamayı verin,

$$\delta(q_1, a_1) = (q_1, a_1, R),$$

$$\delta(q_1, a_2) = (q_3, a_1, L),$$

$$\delta(q_3, a_1) = (q_2, a_2, L).$$

2. Eğer a 1, b 11, R 1, L 11 olarak kodlanıyorsa, 011010111011010 dizisini çözün.
3. Bir diziyi $\{0,1\}^+$ inceleyen ve bunun bir kodlanmış Turing makinesini temsil edip etmediğini belirleyen bir algoritma çizin.
4. Doğru sırada $\{0,1\}^+$ kümesini listeleyen bir Turing makinesi programı çizin.
5. Egzersiz 4'te $0^i 1^j$ indeksinin değeri nedir?
6. Aşağıdaki kümenin doğru sırada listeleneceği bir Turing makinesi tasarımı çizin:

$$L = \{a^n b^n : n \geq 1\}.$$

7. Örnek 10.3 için, her w için doğru sıralamada indeksini veren bir fonksiyon $f(w)$ bulun.
8. İşlem kümesinin tüm üçlülere, (i, j, k) ile i, j, k pozitif tam sayılar, sayılabilir olduğunu gösterin.
9. Varsayalım ki S_1 ve S_2 sayılabilir kümelerdir. O zaman $S_1 \cup S_2$ ve $S_1 \times S_2$ de sayılabilir olduğunu gösterin.
10. Sonlu sayıda sayılabilir kümenin Kartezyen çarpımının sayılabilir olduğunu gösterin.

10.5 DOĞRUSAL SINIRLI OTOMAT

Standart Turing makinesinin gücünü bant yapısını karmaşıktırarak genişletmek mümkün olmasa da, bantın kullanım şekli kısıtlanarak sınırlamak mümkün dür. Bunu, yığılma otomatikleri ile daha önce gördük. Bir yığılma otomatı ği, bantın bir yığın gibi kullanılmak üzere kısıtlandığı bir belirsiz Turing makinesi olarak kabul edilebilir. Bant kullanımını başka şekillerde de kısıtlayabiliriz; örneğin, bantın yalnızca sonlu bir kısmının çalışma alanı olarak kullanılmasına izin verebiliriz. Bunun, bizi sonlu otomatiklere geri götürdüğü gösterilebilir (bu bölümün sonunda Yer alacak Alıştırma 3'e bakın), bu yüzden bunu daha fazla takip etmemize gerek yok. Ancak, bant kullanımını sınırlamanın daha ilginç bir duruma yol açan bir yolu vardır: Makinenin yalnızca girdi tarafından işgal edilen bant kısmını kullanmasına izin veriyoruz.

Bu nedenle, uzun giriş dizileri için daha fazla alan mevcuttur, kısa olanlara göre, yeni bir makine sınıfı oluşturur, yani lineer sınırlı otomata (veya lba).

Bir lineer sınırlı otomaton, standart bir Turing makinesi gibi, sınırsız bir şeride sahiptir, ancak şeridin ne kadarının kullanılabileceği, girişin bir fonksiyonudur. Özellikle, şeridin kullanılabilir kısmını tam olarak giriş tarafından alınan hücrelerle sınırlıyoruz.¹ Bunu sağlamak için, girişi iki özel sembol ile sınırlı olarak hayal edebiliriz, yani sol uç işareti [ve sağ uç işareti]. Bir girdi w için, Turing makinesinin başlangıç konfigürasyonu, anlık tanım $q_0[w]$ ile verilir. Uç işaretleri yeniden yazılamaz ve okuma-yazma kafası [in soluna veya

sağındaki] hareket edemez. Bazen okuma-yazma kafasının uç işaretlerinden "sektiğini" söyleriz.

¹Bazı tanımlarda, şeridin kullanılabilir kısmı giriş uzunluğunun bir katıdır, burada kat, dile bağlı olabilir, ancak girişi etkilemez. Burada yalnızca giriş dizisinin tam uzunluğunu kullanıyoruz, ancak yalnızca bir şeritte giriş olan çoklu şeritli makinelere izin veriyoruz.

DEFINITION 10.5

Bir lineer sınırlı otomaton, Tanım 10.2'de olduğu gibi, bir nondeterministik Turing makinesi $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ olup, Σ 'nin $[\text{ ve }]$ sembollerini içermesi kısıtlamasına tabidir. Böylece $\delta(q_i, [)$ yalnızca eşya biçiminde $(q_j, [, R)$ içerebilir ve $\delta(q_i,])$ yalnızca şu biçimde $(q_j,] , L)$ öğeleri içerebilir.

DEFINITION 10.6

Bir dize w , bir lineer sınırlı otomaton tarafından kabul edilir, eğer mümkün bir hareket dizisi varsa

$$q_0 [w] \vdash^* x_1 f x_2$$

bazı $q_f \in F, x_1, x_2 \in \Gamma^*$. LBA tarafından kabul edilen dil, tüm bu tür kabul edilebilir dizelerin kümesidir.

Bu tanımda bir lineer sınırlı otomatonun nondeterministik olduğu varsayılmaktadır. Bu sadece bir kolaylık meselesi değil, LBA'ların tartışması için esastır.

EXAMPLE 10.4

Dil

$$L = \{a^n b^n c^n : n \geq 1\}$$

bazı lineer sınırlı otomaton tarafından kabul edilir. Bu, Örnek 9.8'deki tartışmadan gelmektedir. Orada belirtilen hesaplama, orijinal girdinin dışındaki alanı gerektirmediğinden, bir lineer sınırlı otomaton tarafından gerçekleştirilebilir.

EXAMPLE 10.5

Bir dil kabul eden lineer sınırlı otomaton bulun

$$L = \{a^{n!} : n \geq 0\}.$$

Problemi çözmenin bir yolu, a 'nın sayısını ardışık olarak 2, 3, 4, ..., bölmektir, ta ki dizeyi ya kabul edebilelim ya da reddedebilelim. Eğer girdi

bir L içindedir, sonunda tek bir a kalacaktır; eğer değilse, bir nokta ada sıfırdan farklı bir kalıntı ortaya çıkacaktır. Tanım 10.5'in bir örnek anlamını belirtmek için çözümü taslak olarak çizeriz. Lineer sınırlı bir otomatonun şeridi çoklu izli olabilir, ekstra izler çalışma alanı olarak kullanılabilir. Bu problem için iki izli bir şerit kullanabiliriz. İlk iz, bölme işlemi sırasında kalan a sayısını içerir ve ikinci iz mevcut bölene içerir (Şekil 10.18). Gerçek çözüm oldukça basittir. İkinci izdeki bölene kullanarak, ilk izdeki a sayısını böleriz, yani bölmenin katları dışındaki tüm semboller kaldırırız. Bunun ardından, bölene bir artırırız ve ya sıfırdan farklı bir kalıntı bulana kadar ya da tek bir a kalana kadar devam ederiz.

	[	a	a	.	.	.	.	.	.	incelenecek a'lar
	[			a					]	Mevcut bölene

ŞEKİL 10.18

Son iki örnek, lineer sınırlı otomatonların, bağlam serbest dillerin iki si de olmadığı için, itme otomatonlarından daha güçlü olduğunu önermektedir. Böyle bir varsayımı kanıtlamak için, herhangi bir bağlam serbest dilin lineer sınırlı bir otomaton tarafından kabul edilebileceğini göstermemiz gerekiyor. Bunu daha sonra biraz dolaylı bir şekilde yapacağız; daha doğrudan bir yaklaşım, bu bölümün sonunda 6 ve 7 numaralı alıştırmalarda önerilmektedir. Turing makineleri ile lineer sınırlı otomatonlar arasındaki ilişki hakkında bir varsayımda bulunmak o kadar kolay değildir. Örnek 10.5 gibi problemler, girişin uzunluğuna orantılı bir miktar geçici alan mevcut olduğundan, her zaman lineer sınırlı bir otomaton tarafından çözülebilir. Aslında, hiçbir lineer sınırlı otomaton tarafından kabul edilemeyecek somut ve açıkça tanımlanmış bir dil bulmak oldukça zordur. Bölüm 11'de, lineer sınırlı otomatonlar sınıfının, sınırsız Turing makineleri sınıfından daha az güçlü olduğunu göstereceğiz, ancak bunun bir gösterimi çok daha fazla çalışma gerektirir.

ALISTIRMALAR

1. Lineer sınırlı otomaların aşağıdaki dilleri kabul etmek için nasıl inşa edilebileceğini açıklayın:

$$(a) L = \{ a^n : n = m^2, m \geq 1 \}.$$

$$(b) L = \{ a^n : n \text{ asal bir sayıdır} \}.$$

$$(c) L = \{ ww : w \in \{ a, b \}^+ \}.$$

$$(d) L = \{ w^n : w \in \{a, b\}^+, n \geq 2 \}.$$

$$(e) L = \{ ww^R : w \in \{a, b\}^+ \}.$$

$$(f) L = \{ w w v v^R : w, v \in \{a, b\}^+ \}.$$

$$(g) L = \{ ww^R : n_a(w) = n_b(w) \}.$$

(h) c. kısımda verilen dilin pozitif kapanışı.

(i) c. kısımda verilen dilin tamamlayanı.

2. Her bağlamdan bağımsız dil için, yığın içindeki sembol sayısının, girdi dizisinin uzunluğunu birden fazla aşmadığı bir kabul eden pda'nın var olduğunu gösterin.
3. Yukarıdaki alıştırmadaki gözlemi kullanarak, λ içermeyen herhangi bir bağlamdan bağımsız dilin bazı lineer sınırlı otomatonlar tarafından kabul edildiğini gösterin.
4. Deterministik bir lineer sınırlı otomaton tanımlamak için, Tanım 10.5'i kullanabiliriz, ancak Turing makinesinin deterministik olmasını gerektiririz. Alıştırma 4'teki çözümlerinizi inceleyin. Çözümler tamamen deterministik lineer sınırlı otomatonlar mı? Değilse, öyle olan çözümler bulmaya çalışın.

BÖLÜM

11

BİR HİYERARŞİ F ORMAL DİLLER VE OTO MATLAR

BÖLÜM ÖZETİ

Bu bölümde, Turing makineleri ile diller arasındaki bağlantıyı inceliyoruz. Dil kabulünü nasıl tanımladığınıza bağlı olarak, birkaç farklı dil ailesi elde ediyoruz: rekürsif, rekürsif olarak sayılabilir ve bağlam duyarlı diller. Düzenli ve bağlamdan bağımsız dillerle birlikte, bunlar Chomsky hiyerarşisi olarak adlandırılan düzgün bir şekilde iç içe geçmiş bir ilişki oluşturur.

Şimdi ana ilgimize, formel dillerin incelenmesine geri dönüyoruz. Hedefimiz, Turing makineleri ile ilişkili dilleri ve bazı kısıtlamalarını incelemektir. Turing makineleri her türlü algoritmik hesaplama yapabildiğinden, onlarla ilişkili diller ailesinin oldukça geniş olduğunu bulmayı bekliyoruz. Bu aile, yalnızca düzenli ve bağlamdan bağımsız dilleri değil, aynı zamanda bu ailelerin dışında kalan çeşitli örnekleri de içermektedir. Önemli bir soru, bazı Turing makineleri tarafından kabul edilmeyen dillerin olup olmadığıdır. Bu soruyu, Turing makinelerinden daha fazla dil olduğunu göstererek yanıtlayacağız, böylece bazı diller için Turing makinelerinin olmadığı sonucuna varabiliriz.

Kanıt kısa ve şıktır, ancak yapıcı değildir ve probleme dair çok az içgörüyü sunar. Bu nedenle, Turing makineleri tarafından tanınmayan dillerin varlığını daha açık bir şekilde ortaya koyacağız.

örnekler, aslında bize böyle bir dili tanımlama imkanı veren örneklerdir. Başka bir araştırma yolu, Turing makineleri ile belirli türdeki dilbilgeleri arasındaki ilişkiye bakmak ve bu dilbilgeleri ile düzenli ve bağlamdan bağımsız dilbilgeleri arasında bir bağlantı kurmaktır. Bu, bir dilbilgeleri hiyerarşisine ve bunun aracılığıyla dil ailelerini sınıflandırma yöntemine yol açar. Bazı küme teorisi diyagramları, çeşitli dil aileleri arasındaki ilişkileri açıkça göstermektedir.

Kesin olarak konuşursak, bu bölümdeki birçok argüman yalnızca boş dizeyi içermeyen diller için geçerlidir. Bu kısıtlama, tanımladığımız şekliyle Turing makinelerinin boş dizeyi kabul edememesi gerçeğinden kaynaklanmaktadır. Tanımı yeniden ifade etmek veya tekrar eden bir feragat ekleme zorunda kalmamak için, bu bölümde tartışılan dillerin, aksi belirtilmedikçe, λ içermediği varsayımını sessizce kabul ediyoruz. λ 'yi dahil edecek şekilde her şeyi yeniden ifade etmek basit bir meseledir, ancak bunu okuyucuya bırakacağız.

11.1 ÖZYİNELEMELİ VE ÖZYİNELEMELİ SAYILABİLİR DİLLER

Başlangıçta Turing makineleri ile ilişkili diller için bazı terimlerle başlıyoruz. Bunu yaparken, kabul eden bir Turing makinesi bulunan diller ile bir üyelik algoritması bulunan diller arasında önemli bir ayrım yapmıyoruz. Bir Turing makinesi, kabul etmediği bir girdi üzerinde durmak zorunda olmadığından, birincisi ikincisini gerektirmez.

DEFINITION 11.1

A language L is said to be **recursively enumerable** if there exists a Turing machine that accepts it.

Bu tanım, yalnızca her $w \in L$ için bir Turing makinesi M bulunduğunu ima eder,

$$q_0 w \vdash_M^* x_1 q_f x_2,$$

bir q_f son durum ile. Tanım, ne olacağı hakkında hiçbir şey söylemiyor $w \notin L$; makinenin son durum dışında durması veya asla durmaz ve sonsuz bir döngüye girer. Daha talepkar olabiliriz ve makinenin herhangi bir verilen girdinin dilinde olup olmadığını

bize söylemesini isteyebiliriz.

DEFINITION 11.2

Bir dil $L \subseteq \Sigma^+$ üzerinde özyinelemeli olarak adlandırılırsa, her $w \in L$ için durduđu ve w 'yi kabul eden bir Turing makinesi M varsa. Diğer bir deyişle, bir dil yalnızca bir üyelik algoritması varsa özyinelemelidir.

Bir dil özyinelemeli ise, o zaman kolayca inşa edilebilen bir sıralama prosedürü vardır. Varsayalım ki M , özyinelemeli bir dil L içindeki üyeliğı belirleyen bir Turing makinesidir. Öncelikle, tüm dizeleri Σ^+ içinde uygun sırayla üreten başka bir Turing makinesi, diyelim ki $\widehat{M}$, inşa ediyoruz. Bu dizeler üretilirken, bunlar M için girdi haline gelir ve M , yalnızca dizeler L içinde olduğunda bantta yazacak şekilde değıştirilir.

Her özyinelemeli sıralanabilir dil için de bir sıralama prosedürü olduğü gör mek o kadar kolay değıldir. Önceki argümanı olduğü gibi kullanamayız, çünkü eğer bazı $w_j \in L$ içinde değılirse, makine M , bantında w_j ile başlatıldığında, asla durm ayabilir ve bu nedenle sıralamada w_j 'yi takip eden dizelere asla ulaşamaz. Bunun olmamasını sağlamak için, hesaplama farklı bir şekilde gerçekleştirilir. Öncelikle M 'yi w_1 üretmesi için yönlendiririz ve M 'ye üzerinde bir hareket yapmasını sağılarız. Sonra M 'yi w_2 üretmesi için yönlendiririz ve M 'ye w_2 üzerinde bir hareket yapmasını, ardından w_1 üzerinde ikinci hareket yapmasını sağılarız.

Bundan sonra, w_3 üretiriz ve w_3 üzerinde bir adım atarız, ardından w_2 üzerinde ikinci adım, w_1 üzerinde üçüncü adım atarız ve bu şekilde devam ederiz. Uygulama sırası Şekil 11.1'de gösterilmiştir. Buradan, M 'nin asla sonsuz bir döngüye girmeyeceğı açıktır. Herhangi bir $w \in L$, $\widehat{M}$ tarafından üretilir ve M tarafından sonlu sayıda adımda kabul edil diğinden, L içindeki her dize sonunda M tarafından üretilir.

Herhangi bir sayım prosedürü için mevcut olan her dilin, özyinelemeli olarak sayılabilir olduğünü görmek kolaydır.

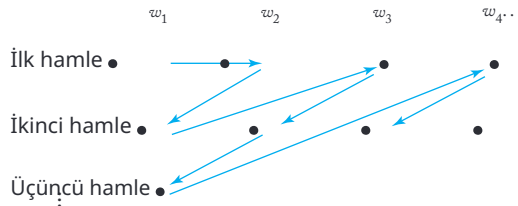
Verilen girdi dizesini, sayım prosedürü tarafından üretilen ardışık dizelerle karşılaştırıyoruz.

Eğer $w \in L$,

L , sonunda bir eşleşme elde edeceğız ve süreç sonlandırılabilir.

Tanımlar 11.1 ve 11.2, ya özyinelemeli ya da özyinelemeli olarak sayılabilir dillerin doğası hakkında çok az bilgi verir.

Bu tanımlar



ŞEKİL 11.1

İsimler, Turing makineleri ile ilişkili dil ailelerine, ancak bu ailelerdeki temsilci dillerin doğası hakkında hiçbir bilgi vermez. Ayrıca, bu diller arasındaki ilişkiler veya daha önce karşılaştığımız dil aileleri ile bağlantıları hakkında da pek fazla bilgi vermezler. Bu nedenle, hemen “Hesaplanabilir ama hesaplanamaz diller var mı?” ve “Bir şekilde tanımlanabilir, ancak hesaplanabilir olmayan diller var mı?” gibi sorularla karşı karşıyayız. Bazı cevaplar sağlayabileceğiz, ancak bu soruları, özellikle de ikincisini, açıklamak için çok belirgin örnekler sunamayacağız.

Döngüsel Olarak Sayılmayan Diller

Döngüsel olarak sayılmayan dillerin varlığını çeşitli yollarla kanıtlayabiliriz. Bunlardan biri çok kısa olup, matematiğin çok temel ve zarif bir sonucunu kullanır.

TEOREM 11.1

Bir S sonsuz sayılabilir küme olsun. O zaman onun güç kümesi 2^S sayılabilir değildir.

Kanıt: $S = \{s_1, s_2, s_3, \dots\}$ olsun. O zaman 2^S 'ye ait herhangi bir eleman t , 0 ve 1'lerden oluşan bir dizi ile temsil edilebilir; burada 1, yalnızca s_i t içinde bulunduğu pozisyon i 'nde yer alır. Örneğin, küme $\{s_2, s_3, s_6\}$ 01100100..., olarak temsil edilirken, $\{s_1, s_3, s_5, \dots\}$ 10101... olarak temsil edilir. Açıkça, 2^S 'nin herhangi bir elemanı böyle bir dizi ile temsil edilebilir ve böyle bir dizi, 2^S 'nin benzersiz bir elemanını temsil eder. Farz edelim ki 2^S sayılabilir; o zaman elemanları bir sırayla yazılabilir, diyelim ki $t_1, t_2, \dots$, ve bunları bir tabloya girebiliriz, Şekil 11.2'de gösterildiği gibi. Bu tabloda, ana köşegen üzerindeki elemanları alalım ve her girişi tamamlayalım, yani 0'ı 1 ile ve tam tersini değiştirin. Şekil 11.2'deki örnekte, elemanlar 1100... olduğundan, 0011... elde ederiz.

t_1	1	0	0	0	0	...
t_2	1	1	0	0	0	...
t_3	1	1	0	1	0	...
t_4	1	1	0	0	1	...
...						
...						

ŞEKİL 11.2

sonuç. Diyagonal boyunca yeni sıra, bazı elemanların 2^S , diyelim k_{t_i} için bazı i . Ancak bu t_1 olamaz çünkü t_1 'den s_1 ile farklıdır. Aynı nedenle t_2, t_3 veya sıralamadaki herhangi bir başka giriş olamaz. Bu çelişki, yalnızca 2^S 'nin sayılabilir olduğu varsayımını atarak ortadan kaldırılabilecek bir mantıksal çıkıma yaratır. ■

Bu tür bir argüman, bir tablonun diyagonal elemanlarının manipülasyonunu içerdiği için diyagonalizasyon olarak adlandırılır. Bu teknik, gerçek sayıların sayılabilir olmadığını göstermek için kullanan matematikçi G. F. Cantor'a atfedilir. Önümüzdeki birkaç bölümde, benzer bir argümanı birkaç bağlamda göreceğiz. Teorem 11.1, en saf haliyle diyagonalizasyondur.

Bu sonucun hemen bir sonucu olarak, bazı anlamlarda, dillerden daha az Turing makinesi olduğunu gösterebiliriz, bu nedenle bazı dillerin özyinelemeli olarak sayılabilir olmadığını söylemek zorundayız.

TEOREM 11.2

Boş olmayan herhangi bir Σ için, özyinelemeli olarak sayılabilir olmayan diller vardır.

Kanıt: Bir dil, Σ^* 'nin bir alt kümesidir ve bu tür her alt küme bir dildir. Bu nedenle, tüm dillerin kümesi tam olarak 2^{Σ^*} 'dir. Σ^* sonsuz olduğundan, Teorem 11.1, Σ üzerindeki tüm dillerin kümesinin sayılabilir olmadığını bize söyler. Ancak tüm Turing makinelerinin kümesi sıralanabilir, bu nedenle tüm tekrarlanabilir sayılabilir dillerin kümesi sayılabilir. Bu bölümün sonunda Yer alacak 16. Egzersiz, Σ üzerinde tekrarlanabilir sayılabilir olmayan bazı dillerin var olması gerektiğini ima eder. ■

Bu kanıt, kısa ve basit olmasına rağmen birçok açıdan tatmin edici değildir. Tamamen yapıcı değildir ve bazı dillerin tekrarlanabilir sayılabilir olmadığını bize söylese de, bu dillerin nasıl görünebileceği hakkında hiçbir his vermez. Sonrakı sonuçlar setinde, sonuca daha açık bir şekilde bakacağız.

Tekrar Sayılabilir Olmayan Bir Dil

Doğrudan algoritmik bir şekilde tanımlanabilen her dil, bir Turing makinesi tarafından kabul edilebilir ve dolayısıyla tekrarlanabilir sayılabilir olduğundan, tekrarlanabilir sayılabilir olmayan bir dilin tanımı dolaylı olacaktır. Yine de, bu mümkündür. Argüman, diyagonalizasyon temasının bir varyasyonunu içerir.

TEOREM 11.3

Tamamlayıcısı geri döngüsel olarak sayılabilir olmayan bir geri döngüsel olarak sayılabilir dil vardır.

Kanıt: $\Sigma = \{a\}$ olarak alalım ve bu giriş alfabesi ile tüm Turing makinelerinin kümesini düşünelim. Teorem 10.3'e göre, bu küme sayılabilir, bu nedenle onun elemanlarıyla bir sıralama $M_1, M_2, \dots$ ilişkilendirebiliriz. Her Turing makinesi M_i için, ilişkili bir geri döngüsel olarak sayılabilir dil $L(M_i)$ vardır. Tersine, Σ üzerindeki her geri döngüsel olarak sayılabilir dil için, onu kabul eden bir Turing makinesi vardır.

Şimdi aşağıdaki gibi tanımlanan yeni bir dil $\bar{L}$ düşünelim. Her $i \geq 1$ için, dize a^i , eğer ve yalnızca $a^i \in L(M_i)$ ise $\bar{L}$ içindedir. Dilin $\bar{L}$ iyi tanımlandığı açıktır, çünkü ifade $a^i \in L(M_i)$ ve dolayısıyla $a^i \in L$, ya doğru ya da yanlış olmalıdır. Sonra ki adımda, dilin tamamlayıcısını L düşünelim,

$$\bar{L} = \{a^i : a^i \notin L\}, \quad (11.1)$$

bu da iyi tanımlıdır ancak, göstereceğimiz gibi, geri döngüsel olarak sayılabilir değildir.

Bunu çelişki ile göstereceğiz, varsayımımızdan başlayarak $\bar{L}$, özyinelemeli olarak sayılabilir. Eğer bu doğruysa, o zaman bazı Turing makinesi, diyelim ki M_k , olmalıdır

$$\bar{L} = L(M_k). \quad (11.2)$$

Dizeyi a^k düşünün. Isitin $\bar{L}$ veya L içinde mi? Farz edelim ki $a^k \in \bar{L}$. (11.2) ile bu, şunu ifade eder:

$$a^k \notin L(M_k).$$

Ancak (11.1) şimdi şunu ifade ediyor:

$$a^k \notin \bar{L}.$$

Alternatif olarak, eğer a^k 'nin L içinde olduğunu varsayıyorsak, o zaman $a^k \in L$ ve (11.2) şunu ifade eder ki:

$$a^k \notin L(M_k).$$

Ancak (11.1)'den şunu elde ediyoruz:

$$a^k \in \bar{L}.$$

Çelişki kaçınılmazdır ve L 'nin özyinelemeli olarak sayılabilir olduğu varsayımımızın yanlış olduğunu sonulandırmalıyız.

Teoremin belirtilen kanıtını tamamlamak için, hala şunu göstermeliyiz ki: $\bar{L}$ özyinelemeli olarak sayılabilir. Bunun için, Turing makineleri için bilinen sayım prosedürünü kullanabiliriz. Verilen a^i , önce i 'yi sayarak buluyoruz.

Sonra, Turing makineleri için sayım prosedürünü kullanarak M_i 'yi buluyoruz. Son olarak, onun tanımını a^i ile birlikte evrensel Turing makinesi M_u 'ye veriyoruz ki bu, M 'nin a^i üzerindeki eylemini simüle eder. Eğer $a^i L$ içinde ise, M_u tarafından gerçekleştirilen hesaplama nihayetinde duracaktır. Bunun birleşik etkisi, her $a^i \in L$ yi kabul eden bir Turing makinesidir. Bu nedenle, Tanım 11.1'e göre, L özyinelemeli olarak sayılabilir. ■

Bu teoremin kanıtı, (11.1) aracılığıyla, özyinelemeli olarak sayılabilir olmayan iyi tanımlanmış bir dili açıkça sergilemektedir. Bu, L 'nin kolay, sezgisel bir yorumunun olduğu anlamına gelmez; bu dilin birkaç önemsiz üyesinden fazlasını sergilemek zor olacaktır. Yine de, L düzgün bir şekilde tanımlanmıştır.

Özyinelemeli Ama Özyinelemeli Olmayan Bir Dil

Sonraki adımda, bazı dillerin özyinelemeli olduğunu ancak özyinelemeli olmadığını gösteriyoruz. Yine, bunu oldukça dolaylı bir şekilde yapmamız gerekiyor. Öncelikle, yardımcı bir sonuç belirleyerek başlıyoruz.

TEOREM 11.4

Eğer bir dil L ve onun tamamlayıcı $\bar{L}$ her ikisi de özyinelemeli ise, o zaman her iki dil de özyinelemelidir. Eğer L özyinelemeli ise, o zaman $\bar{L}$ de özyinelemelidir ve dolayısıyla her ikisi de özyinelemelidir.

Kanıt: Eğer L ve $\bar{L}$ her ikisi de özyinelemeli ise, o zaman L ve $\bar{L}$ için sırasıyla özyineleme prosedürleri olarak hizmet eden Turing makineleri M ve $\bar{M}$ vardır. İlk makine $w_1, w_2, \dots$ üretecek, ikincisi ise $\bar{w}_1, \bar{w}_2, \dots$ üretecektir. Şimdi herhangi bir $w \in \Sigma^+$ verildiğini varsayalım. Öncelikle M 'nin w_1 üretmesine izin veriyoruz ve bunu w ile karşılaştırıyoruz. Eğer aynı değillerse, o zaman $\bar{M}$ 'nin $\bar{w}_1$ üretmesine izin veriyoruz ve tekrar karşılaştırıyoruz. Devam etmemiz gerekiyorsa, bir sonraki adımda M 'nin w_2 üretmesine izin veriyoruz, ardından $\bar{M}$ 'nin $\bar{w}_2$ üretmesine izin veriyoruz ve bu şekilde devam ediyoruz. Herhangi bir $w \in \Sigma^+$ ya M ya da $\bar{M}$ tarafından üretilcektir, bu nedenle sonunda bir eşleşme elde edeceğiz. Eşleşen dize M tarafından üretilmişse, $w \in L$ 'ye aittir, aksi takdirde $\bar{L}$ 'dedir. Bu süreç, hem L hem de $\bar{L}$ için bir üyelik algoritmasıdır, bu nedenle her ikisi de özyinelemelidir.

Karşıtını varsayalım, o zaman L rekürsiftir. O halde bunun için bir üyelik algoritması vardır. Ancak bu, sonucunu basitçe tamamlayarak $\bar{L}$ için bir üyelik algoritması haline gelir. Bu nedenle, $\bar{L}$ rekürsiftir. Her rekürsif dilin rekürsif olarak sayılabilir olduğunu bildiğimizden, kanıt tamamlanmıştır. ■

Bundan, rekürsif olarak sayılabilir diller ailesi ile rekürsif diller ailesinin aynı olmadığını doğrudan sonuçlandırıyoruz. Teorem 11.3'teki dil L birincisinde, ancak ikincisinde değildir.

TEOREM 11.5

Rekürsif olmayan bir rekürsif olarak sayılabilir dil vardır; yani, rekürsif diller ailesi, rekürsif olarak sayılabilir diller ailesinin bir alt kümesidir.

Kanıt: Teorem 11.3'teki dil L dikkate alın. Bu dil rekürsif olarak sayılabilir, ancak tamamlayıcı değildir. Bu nedenle, Teorem 11.4'e göre, rekürsif değildir, aradığımız örneği bize verir. ■

Bundan, bir üyelik algoritması oluşturmanın mümkün olmadığı iyi tanımlanmış dillerin gerçekten var olduğunu görüyoruz.

ALISTIRMALAR

1. Gerçek sayıların kümesinin sayılabilir olmadığını kanıtlayın.
2. Özyinelemeli olarak sayılabilir olmayan tüm dillerin kümesinin sayılabilir olmadığını kanıtlayın.
3. Let L be a finite language. Show that then L^+ is recursively enumerable. Suggest an enumeration procedure for L^+ .
4. Let L be a context-free language. Show that L^+ is recursively enumerable and suggest an enumeration procedure for it.
5. Show that if a language is not recursively enumerable, its complement cannot be recursive.
6. Show that the family of recursively enumerable languages is closed under union.
7. Özyinelemeli sayılabilir diller ailesi kesişim altında kapalı mıdır?
8. Özyinelemeli diller ailesinin birleşim ve kesişim altında kapalı olduğunu gösterin.
9. Özyinelemeli ve özyinelemeli diller ailelerinin tersine çevrilme altında kapalı olduğunu gösterin.
10. Özyinelemeli diller ailesi birleştirme altında kapalı mıdır?
11. Bir bağlamdan bağımsız dilin tamamlayıcısının özyinelemeli olması gerektiğini kanıtlayın.
12. Let L_1 be recursive and L_2 recursively enumerable. Show that $L_2 - L_1$ is necessarily recursively enumerable.
13. Varsayalım ki L öyle bir kümedir ki, onun elemanlarını uygun sırayla sıralayan bir Turing makinesi vardır. Bunun, L 'nin rekürsif olduğu anlamına geldiğini gösterin.
14. Eğer L rekürsif ise, bu durumda L^+ 'nin de rekürsif olması zorunlu mudur?
15. Turing makineleri için belirli bir kodlama seçin ve bununla birlikte, Teorem 11.3'teki L dilinin bir elemanını bulun.

16. Let S_1 be a countable set, S_2 a set that is not countable, and $S_1 \subset S_2$. Show that S_2 must then contain an infinite number of elements that are not in S_1 .
17. Exercise 16'da, aslında $S_2 - S_1$ kümesinin sayılabilir olamayacağını gösterin.
18. Theorem 11.1'deki argüman neden S sonlu olduğunda başarısız olur?
19. Tüm irrasyonel sayıların kümesinin sayılabilir olmadığını gösterin.

11.2 KISITLAMASIZ DİLLER

Tekrar sayılabilir diller ile gramerler arasındaki bağlantıyı araştırmak için, Bölüm 1'deki bir gramerin genel tanımına geri dönüyoruz. Tanım 1.1'de üretim kurallarının herhangi bir biçimi almasına izin verilmişti, ancak daha sonra belirli gramer türlerini elde etmek için çeşitli kısıtlamalar getirilmiştir. Genel biçimi alıp hiçbir kısıtlama getirmezsek, kısıtlamasız gramerler elde ederiz.

DEFINITION 11.3

Bir dilbilgisi $G = (V, T, S, P)$ sınırsız olarak adlandırılırsa, tüm üretimler şu biçimdedir

$$u \rightarrow v,$$

burada $u \in (V \cup T)^+$ içinde ve $v \in (V \cup T)^*$ içinde yer alır.

Sınırsız bir dilbilgisinde, esasen üretimler üzerinde hiçbir koşul getirilmez. Sol veya sağda herhangi sayıda değişken ve terminal bulunabilir ve bunlar herhangi bir sırayla yer alabilir. Tek bir kısıtlama vardır: λ bir üretimin sol tarafında kullanılamaz.

Gördüğümüz gibi, sınırsız dilbilgileri, şimdiye kadar incelediğimiz ınırlı formlar olan düzenli ve bağlamdan bağımsız dilbilgilerinden çok daha güçlüdür. Aslında, sınırsız dilbilgileri, mekanik yollarla tanıyabileceğimiz en büyük dil ailesine karşılık gelir; yani, sınırsız dilbilgileri tam olarak özyinelemeli olarak sayılabilir diller ailesini üretir. Bunu iki bölümde gösteriyoruz; ilki oldukça basit, ancak ikincisi uzun bir yapı içerir.

THEOREM 11.6

Sınırsız bir dilbilgisi tarafından üretilen herhangi bir dil özyinelemeli olarak sayılabilir.

Kanıt: Dilbilgisi, dildeki tüm dizeleri sistematik olarak saymak için bir prosedür tanımlar. Örneğin, tüm w 'yi L içinde listeleyebiliriz, böylece

$$S \Rightarrow w,$$

yani, w bir adımda türetilir. Üretim kümesi sonlu olduğundan, böyle s onlu sayıda dize olacaktır. Sonra, iki adımda türetilen tüm w 'yi L içinde listeleyeceğiz

$$S \Rightarrow x \Rightarrow w,$$

ve devam eder. Bu türetmeleri bir Turing makinesinde simüle edebiliriz ve bu nedenle dil için bir sayım prosedürüne sahibiz. Bu nedenle, geri döngü sel olarak sayılabilir. ■

Geri döngüsel olarak sayılabilir diller ile sınırsız gramerler arasınd aki bu ilişki şaşırtıcı değildir. Gramer, iyi tanımlanmış bir algoritmik sü reçle dizeleri üretir, bu nedenle türetmeler bir Turing makinesinde ya pılabilir. Tersini göstermek için, herhangi bir Turing makinesinin nasıl sınırsız bir gramerle taklit edilebileceğini açıklıyoruz.

Bir Turing makinesi $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ verildi ve $L(G) = L(M)$ ola cak şekilde bir gramer G üretmek istiyoruz. Yapının arkasındaki fikir o ldukça basittir, ancak uygulaması notasyon açısından karmaşık hale g elir.

Turing makinesinin hesaplaması, anlık tanımların dizisiyle tanımla nabilir

$$q_0 w \vdash^* x q_f y, \quad (11.3)$$

karşılık gelen gramere, aşağıdaki özelliğe sahip olacak şekilde düzenlemeye çalışacağı z

$$q_0 w \xRightarrow{*} x q_f y \quad (11.4)$$

eğer ve yalnızca (11.3) geçerliyse. Bunu yapmak zor değildir; daha zor ol an, (11.4) ile gerçekten istediğimiz şey arasındaki bağlantıyı kurmaktır, y ani,

$$S \xRightarrow{*} w$$

tüm w için (11.3)'ü tatmin eden. Bunu başarmak için, geniş hatlarıyla aşağıda ki özelliklere sahip bir dil bilgisi oluşturuyoruz:

1. S , tüm w için $q_0 w$ türetebilir $\in \Sigma^+$.
2. (11.4) yalnızca (11.3) geçerli olduğunda mümkündür.

3. Bir dize $xq_f y$ ile $q_f \in F$ üretildiğinde, dil bilgisi bu diziyi orijinal w 'ye dönüştürür.

Türetimlerin tam dizisi ise

$$S \xRightarrow{*} q_0 w \xRightarrow{*} xq_f y \xRightarrow{*} w. \quad (11.5)$$

Yukarıdaki türetimdeki üçüncü adım sorunlu olandır. Dil bilgisi, ikinci adım sırasında w değiştirilirse bunu nasıl hatırlayabilir? Bunu, kodlanmış dizeleri öyle bir şekilde kodlayarak çözüyoruz ki, kodlanmış versiyon başlangıçta w 'ye iki kopya sahiptir. İlk kopya saklanırken, ikinci kopya (11.4)'teki adımlarda kullanılır. Nihai bir konfigürasyona girildiğinde, dil bilgisi saklanan w dışında her şeyi siler.

w 'nin iki kopyasını üretmek ve M 'nin durum sembolünü (sonunda dil bilgisi tarafından kaldırılması gereken) işlemek için, $tüma \in \Sigma \cup \{\}, b \in \Gamma$ ve $tümi \in Q$ için $q_i \in Q$ olan değişkenler Vab ve $Vaib$ tanıtlıyoruz. Değişken Vab iki sembol $üa$ ve b kodlarken, $Vaib$ ile birlikte durum q_i 'yi de kodlar.

(11.5) içindeki ilk adım, (kodlanmış formda) şu şekilde gerçekleştirilebilir

$$S \rightarrow V_{\square \square} S | S V_{\square \square} | T, \quad (11.6)$$

$$T \rightarrow T V_{aa} | V_{a0a}, \quad (11.7)$$

$tüma \in \Sigma$. Bu üretimler, dilbilgisinin herhangi bir dizeyi $q_0 w$, rastgele sayıda baştaki ve sondaki boşluklarla kodlanmış bir sürümünü üretmesine olanak tanır.

İkinci adım için, her geçiş için

$$\delta(q_i, c) = (q_j, d, R)$$

M , dilbilgisi üretimlerine ekliyoruz

$$V_{aic} V_{pq} \rightarrow V_{ad} \quad pjq, \quad (11.8)$$

$tüma, p \in \Sigma \cup \{\}, q \in \Gamma$. Her bir

$$\delta(q_i) = (q, d, L)$$

M 'nin içinde G

$$V_{pq} V_{aic} \rightarrow V_{pj} V_{ad} \quad (11.9)$$

$tüma, p \in \Sigma \cup \{\}, q \in \Gamma$.

İkinci adımda, M bir son duruma girerse, dilbilgisi, ilk indekslerde saklanan w dışında her şeyden kurtulmalıdır.

Bu nedenle, her $q_j \in F$ için, üretimler ekliyoruz

$$V_{ajb} \rightarrow a, \quad (11.10)$$

for all $a \in \Sigma \cup \{\square\}$, $b \in \Gamma$. This creates the first terminal in the string, which then causes a rewriting in the rest by

$$cV_{ab} \rightarrow ca, \quad (11.11)$$

$$V_{abc} \rightarrow ac, \quad (11.12)$$

for all $a, c \in \Sigma \cup \{\square\}$, $b \in \Gamma$. We need one more special production

$$\square \rightarrow \lambda. \quad (11.13)$$

This last production takes care of the case when M moves outside that part of the tape occupied by the input w . To make things work in this case, we must first use (11.6) and (11.7) to generate

$$\square \dots \square q_0 w \square \dots \square,$$

representing all the tape region used. The extraneous blanks are removed at the end by (11.13).

Aşağıdaki örnek, bu karmaşık yapıyı göstermektedir. Örnekteki her adımı dikkatlice kontrol edin, böylece çeşitli üretimlerin ne yaptığını ve neden gerekli olduğunu görebilirsiniz.

ÖRNEK 11.1

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ be a Turing machine with

$$\begin{aligned} Q &= \{q_0, q_1\}, \\ \Gamma &= \{a, b, \square\}, \\ \Sigma &= \{a, b\}, \\ F &= \{q_1\}, \end{aligned}$$

ve

$$\begin{aligned} \delta(q_0, a) &= (q_0, a, R), \\ \delta(q_0, \square) &= (q_1, \square, L). \end{aligned}$$

Bu makine kabul eder $L(aa^*)$.

Şimdi hesaplamayı düşünün

$$q_0 aa \vdash aq_0 a \vdash aaq_0 \square \vdash aq_1 a \square, \quad (11.14)$$

bu dizgiyi aa kabul eden. Bu dizgiyi G ile elde etmek için, önce (11.6) ve (11.7) biçimindeki kuralları kullanarak uygun başlangıç

dizgisini elde ederiz,

$$S \Rightarrow SV_{\square\square} \Rightarrow TV_{\square\square} \Rightarrow TV_{aa}V_{\square\square} \Rightarrow V_{a0a}V_{aa}V_{\square\square}.$$

Son cümle biçimi, Turing makinesinin hesaplamasını taklit eden t üretim kısmı için başlangıç noktasıdır. İlk indekslerin dizisinde orijinal girdi aa ve kalan indekslerde başlangıç anlık tanımı q_0aa içerir. Sonra, uyguluyoruz

$$V_{a0a}V_{aa} \rightarrow V_{aa}V_{a0a},$$

ve

$$V_{a0a}V_{\square\square} \rightarrow V_{aa}V_{\square0\square},$$

hangi (11.8)'in belirli örnekleridir ve

$$V_{aa}V_{\square0\square} \rightarrow V_{a1a}V_{\square\square}$$

(11.9)'dan gelmektedir. O zaman türetimdeki sonraki adımlar

$$V_{a0a}V_{aa}V_{\square\square} \Rightarrow V_{aa}V_{a0a}V_{\square\square} \Rightarrow V_{aa}V_{aa}V_{\square0\square} \Rightarrow V_{aa}V_{a1a}V_{\square\square}.$$

ilk indekslerin sırası aynı kalır, her zaman ilk girişi hatırlayarak. Diğer indekslerin sırası ise

$$0aa\square, a0a\square, aa0\square, a1a\square,$$

(11.14)'teki anlık tanımların sırasına eşdeğerdir.

Son olarak, (11.10) ile (11.13) son adımlarda kullanılır

$$V_{aa}V_{a1a}V_{\square\square} \Rightarrow V_{aa}aV_{\square\square} \Rightarrow V_{aa}a\square \Rightarrow aa\square \Rightarrow aa.$$

(11.6) ile (11.13) arasında tanımlanan yapı, aşağıdaki sonucun kanıtının temelidir.

TEOREM 11.7

Her bir özyinelemeli olarak sayılabilir dil L için, böyle bir sınırsız gramer G vardır ki $L = L(G)$.

Kanıt: Açıklanan yapı, bunun garantisini verir

$$x \vdash y,$$

o zaman

$$e(x) \Rightarrow e(y),$$

burada $e(x)$ verilen kurala göre bir dizinin kodlamasını belirtir. Adım sayısına göre bir induksiyon ile, o zaman gösterebiliriz ki

$$e(q_0w) \xRightarrow{*} e(y)$$

eğer ve yalnızca eğer

$$q_0w \vdash^* y.$$

Ayrıca, her olası başlangıç konfigürasyonunu üretebileceğimizi ve w yalnızca M son bir konfigürasyona girdiğinde doğru bir şekilde yeniden yapılandırıldığını göstermeliyiz. Detaylar, çok zor olmayan, bir alıştırmaya bırakılmıştır. ■

Bu iki teorem, yapmayı amaçladığımız şeyi kurar. Sınırsız gramerlerle ilişkili diller ailesinin, özyinelemeli olarak sayılabilir diller ailesiyle aynı olduğunu gösterir.

ALISTIRMALAR

1. Kısıtlanmamış dil bilgisi

$$\begin{aligned} S &\rightarrow S_1B, \\ S_1 &\rightarrow aS_1b, \\ bB &\rightarrow bbbB, \\ aS_1b &\rightarrow aa, \\ B &\rightarrow \lambda \end{aligned}$$

hangi dili türetiyor?

2. $L(01(01)^*)$ için bir Turing makinesi inşa edin, ardından Teorem 11.7'deki yapıyı kullanarak bunun için bir kısıtlanmamış dil bilgisi bulun. Elde edilen dil bilgisi ile 0101 için bir türetim verin.
3. Teorem 11.7'deki yapıyı kullanarak aşağıdaki diller için kısıtlanmamış dil bilgileri bulun:

- (a) $L = L(aaa^*bb^*)$.
- (b) $L = L\{(ab)^*b^*\}$.
- (c) $L = \{w : n_a(w) = n_b(w)\}$.

4. Bir türev için başlangıç noktası olarak tek bir değişken yerine sonlu bir dize kümesinin kullanılabileceği gramerlerde bir varyasyonu düşünün. Bu kavramı biçimlendirin, ardından bu tür gramerlerin burada kullandığımız sınırsız gramerlerle nasıl ilişkili olduğunu araştırın.

Bu kavramı biçimlendirin, ardından bu tür gramerlerin burada kullandığımız sınırsız gramerlerle nasıl ilişkili olduğunu araştırın.

5. Örnek 11.1'de, oluşturulan gramerin içinde ab bulunan herhangi bir cümle üretilmeyeceğini kanıtlayın.
6. Teorem 11.7'nin kanıtının ayrıntılarını verin.
7. Her sınırsız dilbilgisi için, tüm üretimlerinin aşağıdaki biçimde olduğu eşdeğer bir sınırsız dilbilgisi vardır:

$$u \rightarrow v,$$

$$u, v \in (V \cup T)^+ \text{ ve } |u| \leq |v|, \text{ veya}$$

$$A \rightarrow \lambda,$$

$$A \in V.$$

8. Egzersiz 7'nin sonucunun, ek koşullar eklendiğinde de geçerli olduğunu gösterin: $|u| \leq 2$ ve $|v| \leq 2$.
9. Bazı yazarlar, sınırsız dilbilgileri için tanımımız 11.3 ile tam olarak aynı olmayan bir tanım verir. Bu alternatif tanımda, bir sınırsız dilbilgisinin üretimlerinin aşağıdaki biçimde olması gerekmektedir:

$$x \rightarrow y,$$

burada

$$x \in (V \cup T)^* V (V \cup T)^*,$$

ve

$$y \in (V \cup T)^*.$$

Fark, burada sol tarafın en az bir değişken içermesi gerektiğidir.

Bu alternatif tanımın, her bir tür için eşdeğer bir gramerin bulunduğu anlamda, kullandığımız tanımla temelde aynı olduğunu gösterin.

11.3 KONTEKST-DUYARLI GRAMERLER VE DİLLER

Sınırlı, bağlamdan bağımsız gramerler ile genel, sınırsız gramerler arasında, "biraz sınırlı" gramerlerin büyük bir çeşitliliği tanımlanabilir. Tüm durumlar ilginç sonuçlar vermez; ancak ilginç sonuçlar verenler arasında, bağlamdan duyarlı gramerler önemli bir dikkat çekmiştir. Bu gramerler, Bölüm 10.5'te tanıttığımız, sınırlı bir Turing makinesi sınıfı ile ilişkili dilleri üretir, lineer sınırlı otomatalar.

TANIM 11.4

Bir gramer $G = (V, T, S, P)$ bağlamdan duyarlı olarak adlandırılırsa, tüm üretimle r aşağıdaki biçimdedir:

$$x \rightarrow y,$$

burada $x, y \in (V \cup T)^+$ ve

$$|x| \leq |y|. \quad (11.15)$$

Bu tanım, bu tür bir gramerin bir yönünü açıkça göstermektedir; bu d araltıcı değildir, çünkü ardışık cümle biçimlerinin uzunluğu asla azalmaz. B u tür gramerlerin neden bağlamdan duyarlı olarak adlandırılması gerektiğ i daha az açıktır, ancak (örneğin, Salomaa 1973'e bakın) tüm bu gramerleri n, tüm üretimlerin aşağıdaki biçimde olduğu normal bir forma yeniden ya zılabileceği gösterilebilir.

$$xAy \rightarrow xvy.$$

Bu, üretimin

$$A \rightarrow v$$

yalnızca bir dize bağlamında A'nın meydana geldiği durumlarda uygulanabilir x solda ve sağda dize y bulunmaktadır. Terminolojiyi kullanırken Bu özel yorumdan kaynaklanan, formun kendisi burada bizim için pek ilgi çekici değil ve tamamen Tanım 11.4'e dayanacağız.

Bağlam Duyarlı Diller ve Doğrusal Sınırlı Otomatlar

Terminolojinin belirttiği gibi, bağlam duyarlı gramerler aynı adı taşıya n bir dil ailesi ile ilişkilidir.

DEFINITION 11.5

A language L bağlam duyarlı olarak adlandırılırsa, bağlam duyarlı bir grammer G var ise, öyle ki $L = L(G)$ veya $L = L(G) \cup \{\lambda\}$.

Bu tanımda, boş dizeyi yeniden tanıtıyoruz. Tanım 11.4 $x \rightarrow \lambda$ ifadesinin yasak olduğunu belirtir, bu nedenle bir bağlam duyarlı dilbilgisi asla boş dize içeren bir dil üretemez. Yine de, her

λ olmadan bağlamdan bağımsız dil, bir bağlamdan duyarlı dilin özel bir durumu tarafından üretilebilir, örneğin Chomsky veya Greibach normal formunda olan bir dil tarafından, her ikisi de Tanım 11.4'ün koşullarını karşılar. Boş dizeyi bağlamdan duyarlı bir dilin tanımına dahil ederek (ancak dilbilgi sine değil), bağlamdan bağımsız diller ailesinin, bağlamdan duyarlı diller ailesinin bir alt kümesi olduğunu iddia edebiliriz.

ÖRNEK 11.2

$Dil L = \{a^n b^n c^n : n \geq 1\}$ bağlam duyarlı bir dildir. Bunu, dil için bir bağlam duyarlı gramer sergileyerek gösteriyorz. Böyle bir gramer

$$\begin{aligned} S &\rightarrow abc|aAbc, \\ Ab &\rightarrow bA, \\ Ac &\rightarrow Bbcc, \\ bB &\rightarrow Bb, \\ aB &\rightarrow aa|aaA. \end{aligned}$$

Bunun nasıl çalıştığını, bir türetmeye bakarak görebiliriz $a^3b^3c^3$.

$$\begin{aligned} S &\Rightarrow aAbc \Rightarrow abAc \Rightarrow abBbcc \\ &\Rightarrow aBbbcc \Rightarrow aaAbbcc \Rightarrow aabAbcc \\ &\Rightarrow aabbAcc \Rightarrow aabbBcccc \\ &\Rightarrow aabBbbccc \Rightarrow aaBbbbccc \\ &\Rightarrow aaabbbccc. \end{aligned}$$

Çözüm, değişkenleri A ve B haberci olarak etkili bir şekilde kullanır. Bir A solda oluşturulur, sağa doğru ilk c 'ye gider, burada başka bir b ve c oluşturur. Ardından, karşılık gelen a 'yı oluşturmak için haberci B 'yi sola geri gönderir. Süreç, bir Turing makinesini $dil L$ 'yi kabul edecek şekilde programlamaya benzer.

Önceki örnekteki dil bağlam serbest olmadığından, bağlam serbest diller ailesinin, bağlam duyarlı diller ailesinin doğru bir alt kümesi olduğunu görüyoruz. Örnek 11.2, bağlam duyarlı bir gramer bulmanın, nispeten basit örnekler için bile kolay bir iş olmadığını da göstermektedir. Genellikle çözüm, bir Turing makinesi programıyla başlamak ve ardından bunun için eşdeğer bir gramer bulmakla en kolay elde edilir. Birkaç örnek, dil bağlam duyarlı olduğunda, karşılık gelen Turing makinesinin öngörülebilir alan gereksinimlerine sahip olduğunu gösterecektir; özellikle, b u, lineer sınırlı bir otomaton olarak görülebilir.

TEOREM 11.8

Herhangi bir bağlam duyarlı dil L , λ içermiyorsa, öyle bir lineer sınırlı otomaton M vardır ki $L = L(M)$.

Kanıt: Eğer L bağlam duyarlı ise, o zaman $L - \{\lambda\}$ için bir bağlam duyarlı gramer vardır. Bu gramerdeki türetimlerin bir lineer sınırlı otomaton tarafından simüle edilebileceğini gösteriyoruz. Lineer sınırlı otomatonun iki yolu olacaktır, biri girdi dizisini w içerecek, diğeri ise G kullanılarak türetilen cümle biçimlerini içerecektir. Bu argümanın önemli bir noktası, hiçbir olası cümle biçiminin uzunluğunun $|w|$ 'den büyük olamayacağıdır. Dikkat edilmesi gereken bir diğer nokta ise, lineer sınırlı otomatonun tanım gereği belirsiz olmasıdır. Bu, argümanda gereklidir, çünkü doğru üretimin her zaman tahmin edilebileceğini ve hiçbir verimsiz alternatifin izlenmesi gerekmediğini iddia edebiliriz. Bu nedenle, Teorem 11.6'da tanımlanan hesaplama, yalnızca w tarafından işgal edilen alan dışında bir alan kullanmadan gerçekleştirilebilir; yani, bu bir lineer sınırlı otomaton tarafından yapılabilir. ■

TEOREM 11.9

Eğer bir dil L bazı lineer sınırlı otomaton M tarafından kabul ediliyorsa, o zaman L 'yi üreten bir bağlam duyarlı gramer vardır.

Kanıt: Buradaki yapı, Teorem 11.7'dekiyle benzerdir. Teorem 11.7'de üretilen tüm üretimler, (11.13) hariç, sözleşme yapmayan üretimlerdir.

$$\square \rightarrow \lambda.$$

Ancak bu üretim atlanabilir. Bu, Turing makinesinin orijinal girdinin sınırlarının dışına çıkması gerektiğinde gereklidir, ki bu burada söz konusu değildir. Bu gereksiz üretim olmadan elde edilen gramer, sözleşme yapmayan bir gramerdir ve argümanı tamamlar. ■

Özyinelemeli ve Bağlam Duyarlı Diller Arasındaki İlişki

Teorem 11.9, her bağlam duyarlı dilin bazı Turing makineleri tarafından kabul edildiğini ve dolayısıyla özyinelemeli olarak sayılabilir olduğunu bize söyler. Teorem 11.10 bu durumdan kolayca çıkar.

TEOREM 11.10

Her bağlam duyarlı dil L özyinelemelidir.

Kanıt:Bağlam duyarlı dil L ile ilişkili bir bağlam duyarlı gramer G düşünün ve w 'nin bir türevini inceleyin.

$$S \Rightarrow x_1 \Rightarrow x_2 \Rightarrow \cdots \Rightarrow x_n \Rightarrow w.$$

Herhangi bir genel kayıp olmaksızın, tek bir türevdeki tüm cümle biçimlerinin farklı olduğunu varsayabiliriz; yani, $x_i \neq x_j$ için tüm $i \neq j$. Argümanımızın özü, herhangi bir türevdeki adım sayısının $|w|$ 'nin sınırlı bir fonksiyonu olduğudur. Biliyoruz ki

$$|x_j| \leq |x_{j+1}|,$$

çünkü G sözleşme yapmayan bir yapıdır. Eklememiz gereken tek şey, yalnızca G ve w 'ye bağlı olarak bazı m 'lerin var olduğudur, böylece

$$|x_j| < |x_{j+m}|,$$

her j için, $m = m(|w|) \vee |w|$ 'nin sınırlı bir fonksiyonu. Bu, $|w|$ 'nin sonlu olmasının, belirli bir uzunluktaki yalnızca sonlu sayıda dize olduğu anlamına gelmesinden kaynaklanmaktadır. Bu nedenle, $w \in L$ 'nin bir türetilmesinin uzunluğu en fazla $|w|$ olacaktır.

Bu gözlem, bize hemen L için bir üyelik algoritması verir.

Uzunluğu $|w|$ kadar olan tüm türetmeleri kontrol ediyoruz. G 'nin üretim kümesi sonlu olduğundan, bunların yalnızca sonlu sayıda vardır. Eğer bunlardan herhangi biri w 'yi veriyorsa, o zaman $w \in L$ 'dir, aksi takdirde değildir. ■

TEOREM 11.11

Bağlam duyarlı olmayan bir rekürsif dil vardır.

Kanıt: $T = \{a, b\}$ üzerinde tüm bağlam duyarlı gramerler kümesini düşünelim. Her gramerin, aşağıdaki biçimde bir değişken kümesine sahip olduğu bir kural kullanabiliriz

$$V = \{V_0, V_1, V_2, \dots\}.$$

Her bağlam duyarlı gramer, üretimleriyle tamamen tanımlanır; bunları tek bir dize olarak yazılmış olarak düşünebiliriz

$$x_1 \rightarrow y_1; x_2 \rightarrow y_2; \dots; x_m \rightarrow y_m.$$

Bu dizeye şimdi homomorfizmayı uyguluyoruz

$$h(a) = 010,$$

$$h(b) = 01^20,$$

$$h(\rightarrow) = 01^30,$$

$$h(;) = 01^40,$$

$$h(V_i) = 01^{i+5}0.$$

Bu nedenle, herhangi bir bağlama duyarlı dil, L 'den bir dize ile benzer siz bir şekilde temsil edilebilir. Ayrıca, temsil, verilen herhangi bir dize için en fazla bir bağlama duyarlı dilin buna karşılık geldiği anlamında tersine çevrilebilir.

Şimdi $\{0,1\}^+$ üzerinde uygun bir sıralama tanıtalım, böylece dizeleri $w_1, w_2, \dots$ vb. sırasıyla yazabiliriz. Verilen bir dize w_j , bir bağlama duyarlı dil tanımlamaya bilir; eğer tanımlıyorsa, dil G_j olarak adlandırılır. Sonra, bir dil L tanımlıyoruz

$$L = \{w_i : w_i \text{ bir bağlama duyarlı dil } G_i \text{ tanımlar ve } w_i \in L(G_i)\}.$$

L iyi tanımlıdır ve aslında tekrarlardır. Bunu görmek için, bir üyelik algoritması inşa ediyoruz. w_i verildiğinde, bunun bir bağlama duyarlı dil G_i tanımlayıp tanımlamadığını kontrol ediyoruz. Eğer tanımlamıyorsa, o zaman $w_i \notin L$. Eğer dize bir dil tanımlıyorsa, o zaman $L(G_i)$ tekrarlardır ve $w_i \in L(G_i)$ olup olmadığını bulmak için Teorem 11.10'un üyeli k algoritmasını kullanabiliriz. Eğer değilse, o zaman w_i L 'e aittir.

Ancak L bağlama duyarlı değildir. Eğer öyle olsaydı, bazı w_j 'ler için $L = L(G_j)$ olurdu. O zaman w_j 'nin $L(G_j)$ 'de olup olmadığını sorabiliriz. Eğer $w_j \in L(G_j)$ olduğunu varsayarsak, o zaman tanım gereği w_j L 'de değildir. Ama $L = L(G_j)$ olduğundan, bir çelişki elde ederiz. Tersine, eğer $w_j \notin L(G_j)$ olduğunu varsayarsak, o zaman tanım gereği $w_j \in L$ 'dir ve başka bir çelişki elde ederiz. Bu nedenle, L 'nin bağlama duyarlı olmadığını sonucuna varmalıyız. ■

Teorem 11.11'deki sonuç, lineer sınırlı otomata

'nın gerçekten Turing makinelerinden daha az güçlü olduğunu göstermektedir, çünkü yalnızca bir doğru alt kümesini kabul ederler.

Aynı sonuçtan, lineer sınırlı otomata'nın pushdown otomata'dan daha güçlü olduğu sonucuna varılmaktadır. Bağlamdan bağımsız diller, bağlamdan bağımsız gramerler tarafından üretilmektedir ve bağlamdan duyarlı dillerin bir alt kümesidir. Çeşitli örnekler gösterdiği gibi, bunlar doğru bir alt kümedir. Lineer sınırlı otomata ile bağlamdan duyarlı dillerin bir yanda, pushdown otomata ile bağlamdan bağımsız dillerin diğer yanda özdeşleştiği nedeniyle, bir pushdown otomata tarafından kabul edilen herhangi bir dilin aynı zamanda bazı lineer sınırlı otomata tarafından da kabul edildiğini ve bazı lineer sınırlı otomata tarafından kabul edilen dillerin ise hiçbir pushdown otomata tarafından kabul edilmediğini görmekteyiz.

ALISTIRMALAR

1. Egzersiz 1, Bölüm 11.2'deki kısıtlanmamış gramerle eşdeğer bir bağlamdan duyarlı gramer bulun.
2. Aşağıdaki diller için bağlamdan duyarlı gramerler bulun:

(a) $L = \{a^{n+1}b^nc^{n-1} : n \geq 1\}.$

- (b) $L = \{a^n b^n a^{2n} : n \geq 1\}$.
- (c) $L = \{a^n b^m c^n d^m : n \geq 1, m \geq 1\}$.
- (d) $L = \{ww : w \in \{a, b\}^+\}$.
- (e) $L = \{a^n b^n c^n d^n : n \geq 1\}$.

3. Aşağıdaki diller için bağlamdan duyarlı gramerler bulun:

- (a) $L = \{w : n_a(w) = n_b(w) = n_c(w)\}$.
- (b) $L = \{w : n_a(w) = n_b(w) < n_c(w)\}$.

4. Show that the family of context-sensitive languages is closed under union.
5. Show that the family of context-sensitive languages is closed under reversal.
6. Form in Theorem 11.10, give explicit bounds for m as a function of $|w|$ and $|V \cup T|$.
7. Without explicitly constructing it, show that there exists a context-sensitive grammar for the language $L = \{wuw^R : w, u \in \{a, b\}^+, |w| \geq |u|\}$.

11.4 CHOMSKY HİYERARŞİSİ

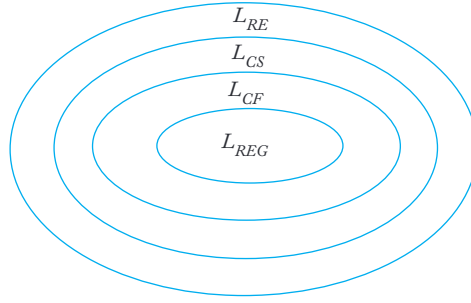
Artık aralarında tekrar sayılabilir diller (L_{RE}), bağlam duyarlı diller (L_{CS}), bağlamdan bağımsız diller (L_{CF}) ve düzenli diller (L_{REG}) bulunan birçok dil ailesiyle karşılaştık.

Bu aileler arasındaki ilişkiyi sergilemenin bir yolu,

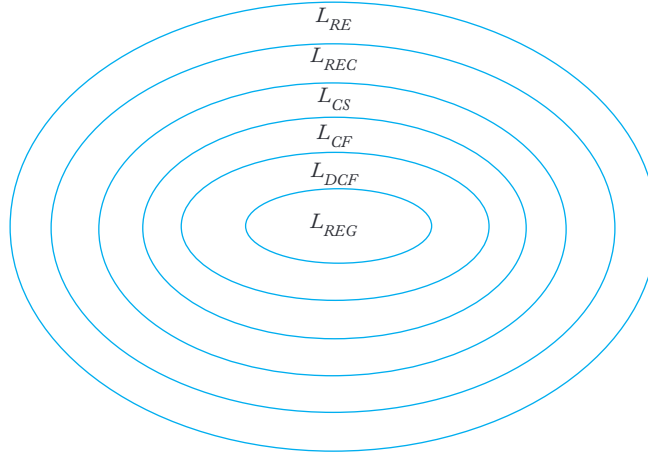
Chomsky hiyerarşisidir. Formel dil teorisinin kurucularından Noam Chomsky, dört dil türüne, tip 0'dan tip 3'e kadar, ilk bir sınıflandırma sağlamıştır. Bu orijinal terminoloji devam etmiştir ve sık sık atıflarını bulmak mümkündür, ancak sayısal türler aslında incelediğimiz dil aileleri için farklı isimlerdir. Tip 0 dilleri, kısıtlamasız gramerler tarafından üretilen, yani tekrar sayılabilir dillerdir.

Tip 1, bağlam duyarlı dillerden oluşur, tip 2, bağlamdan bağımsız dillerden oluşur ve tip 3, düzenli dillerden oluşur. Gördüğümüz gibi, her dil ailesi tip_i , tip_{i-1} 'in ailelerinin uygun bir alt kümesidir. Bir diyagram (Şekil 11.3) bu ilişkiyi açıkça sergilemektedir. Şekil 11.3, orijinal Chomsky hiyerarşisini göstermektedir. Ayrıca bu resme uyan birkaç başka dil ailesiyle de karşılaştık. Belirleyici bağlamdan bağımsız dillerin (L_{DCF}) ve özyinelemeli dillerin (L_{REC}) ailelerini de dahil ederek, Şekil 11.4'te gösterilen genişletilmiş hiyerarşiye ulaşırız.

Diğer dil aileleri tanımlanabilir ve Şekil 11.4'teki yerleri incelenebilir, ancak ilişkileri her zaman Şekil 11.3 ve 11.4'teki gibi düzgün bir şekilde iç içe geçmiş bir yapıya sahip değildir. Bazı durumlarda, ilişkiler tamamen anlaşılamamaktadır.



ŞEKİL 11.3



ŞEKİL 11.4

ÖRNEK 11.3

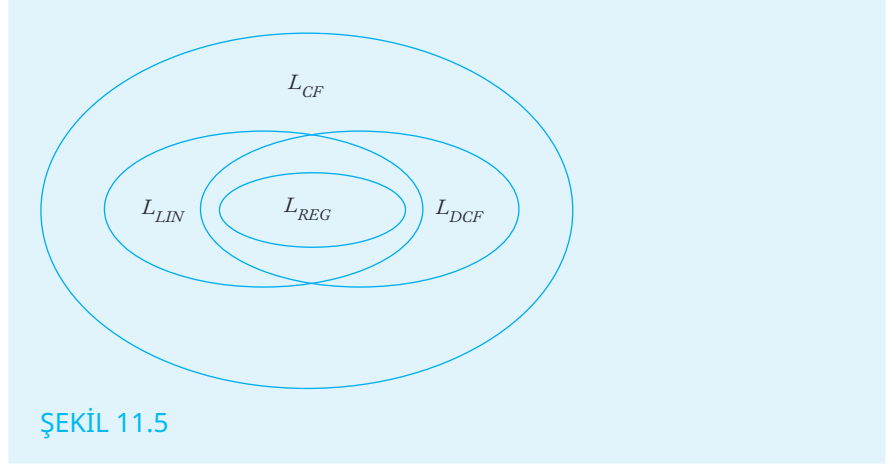
Daha önce bağlamdan bağımsız dili tanıttık

$$L = \{w : n_a(w) = n_b(w)\}$$

ve bunun belirleyici olduğunu, ancak doğrusal olmadığını gösterdik. Öte yandan, dil

$$L = \{a^n b^n\} \cup \{a^n b^{2n}\}$$

doğrusal, ancak belirleyici değildir. Bu, düzenli, doğrusal, belirleyici bağlamdan bağımsız ve belirsiz bağlamdan bağımsız diller arasındaki ilişkini n Şekil 11.5'te gösterildiği gibi olduğunu belirtir.



Hala çözülmemiş bir sorun var. Egzersiz 8, Bölüm 10.5'te deterministik lineer sınırlı otomaton kavramını tanıttık. Şimdi, diğer otomatonlarla bağlantılı olarak sorduğumuz soruyu sorabiliriz:

Burada belirsizlik (nondeterminism) ne rol oynuyor? Ne yazık ki, kolay bir cevap yok. Şu anda, deterministik lineer sınırlı otomatonlar tarafından kabul edilen dil ailesinin, bağlam duyarlı dillerin uygun bir alt kümesi olup olmadığı bilinmemektedir.

Özetlemek gerekirse, birkaç dil ailesi ve bunlara bağlı otomatonlar arasındaki ilişkileri keşfettik. Bunu yaparken, dillerin bir hiyerarşisini oluşturduk ve otomatonları dil kabul etme güçlerine göre sınıflandırdık. Turing makineleri, lineer sınırlı otomatonlardan daha güçlüdür. Bunlar da, pushdown otomatonlardan daha güçlüdür. Hiyerarşinin en altında, çalışmamıza başladığımız sonlu kabul edenler bulunmaktadır.

ALISTIRMALAR

1. Bu kitapta verilen örnekleri toplayın ve Şekil 11.4'te gösterilen tüm alt küme ilişkilerinin gerçekten uygun olanlar olduğunu kanıtlayın.
2. Örnek 11.3'teki örneği hariç, lineer ancak deterministik olmayan bağlam serbest dillerden iki örnek bulun.
3. İki örnek bulun (Örnek 11.3'teki örneği hariç) d deterministik bağlam serbest ama lineer olmayan diller.

BÖLÜM

12

ALGORİTMİK HESAPLAMA NIN SINIRLARI

BÖLÜM ÖZETİ

Turing makinelerinin, bilgisayarların yapabileceği her şeyi yapabileceğini iddia ettik. Eğer bir Turing makinesi tarafından yapılamayan bir problem bulursak, o zaman en güçlü bilgisayarın bile gücünde olmayan bir problemi de tanımlamış oluruz. Gördüğümüz gibi, böyle birçok problem vardır, ancak hepsiyle detaylı bir şekilde ilgilenmek yerine, diğerlerinin hepsinin indirgenebileceği bir durumu buluyoruz. Bu, duraklama problemi. Bu oldukça soyut bir konu olmasına rağmen, birkaç pratik olarak önemli sonuç ortaya çıkmaktadır.

Turing makinelerinin neler yapabileceğinden bahsettikten sonra, şimdi neler yapamayacaklarına bakıyoruz. Turing'in tezi, Turing makinesinin gücünde çok az sınırlama olduğunu düşünmemize yol açsa da, belirli problemler için çözüm algoritmalarının var olamayacağını birkaç kez iddia ettik. Şimdi bu iddiamızın ne anlama geldiğini daha açık hale getiriyoruz. Bazı sonuçlar oldukça basit bir şekilde ortaya çıktı; eğer bir dil geri dönüşümsüzse, o zaman tanım gereği onun için bir üyelik algoritması yoktur. Eğer bu konu sadece bu kadar olsaydı, çok ilginç olmazdı; geri dönüşümsüz dillerin pratik değeri pek yoktur. Ancak sorun daha derin. Örneğin, bir bağlam serbest dilin olup olmadığını belirlemek için bir algoritmanın var olmadığını belirttik (ama henüz kanıtlamadık).

belirsiz değildir. Bu soru, programlama dilleri çalışmasında pratik bir öneme sahiptir.

Öncelikle, bir şeyin bir Turing makinesi tarafından yapılamayacağını söylediğimizde ne anlama geldiğini netleştirmek için karar verilebilirlik ve hesaplanabilirlik kavramlarını tanımlıyoruz. Ardından, bu türden birkaç klasik probleme bakıyoruz; bunlar arasında Turing makineleri için iyi bilinen duraklama problemi de bulunmaktadır. Bunun ardından, Turing makineleri ve özyinelemeli sayılabilir diller için bir dizi ilgili problem ortaya çıkmaktadır. Sonrasında, bağlamdan bağımsız dillerle ilgili bazı sorulara bakıyoruz. Burada, ne yazık ki, algoritmaların olmadığı oldukça önemli birçok problem buluyoruz.

12.1 TURING MAKİNELERİ TARAFINDAN ÇÖZÜLEMİYEN BAZI PROBLEMLER

Mekanik hesaplamaların gücünün sınırlı olduğu argümanı şaşırtıcı değildir. İlgüdüsel olarak, birçok belirsiz ve spekülatif sorunun, şu anda inşa edebileceğimiz veya hatta öngörebileceğimiz herhangi bir bilgisayarın kapasitesinin çok ötesinde özel bir içgörü ve akıl yürütme gerektirdiğini biliyoruz. Bilgisayar bilimcileri için daha ilginç olan, açıkça ve basit bir şekilde ifade edilebilen, algoritmik bir çözüm olasılığı görünen ancak herhangi bir bilgisayar tarafından çözülemeyeceği bilinen soruların varlığıdır.

Hesaplanabilirlik ve Karar Verilebilirlik

Tanım 9.4'te, belirli bir alandaki bir fonksiyonun f hesaplanabilir olduğunu, bu alandaki tüm argümanlar için f değerini hesaplayan bir Turing makinesi varsa söylenir. Bir fonksiyon, böyle bir Turing makinesi yoksa hesaplanamaz. Fonksiyonun alanının bir kısmında f hesaplayabilen bir Turing makinesi olabilir, ancak fonksiyonu yalnızca alanının tamamında hesaplayan bir Turing makinesi varsa hesaplanabilir olarak adlandırırız.

Buradan görüyoruz ki, bir fonksiyonu hesaplanabilir veya hesaplanamaz olarak sınıflandırdığımızda, onun tanım kümesinin ne olduğunu net bir şekilde belirtmemiz gerekir.

Buradaki endişemiz, bir hesaplamanın sonucunun basit bir “evet” veya “hayır” olduğu biraz basitleştirilmiş bir ayar olacaktır. Bu durumda, bir problemin karar verilebilir veya karar verilemez olduğunu konuşuyoruz. Bir *problem* ile ilgili ifadelerden oluşan bir küme anlayacağız; her biri ya doğru ya da yanlış olmalıdır. Örneğin, “Bağlamdan bağımsız bir dil bilgisi için” ifadesini ele alıyoruz. G , $\text{dil}(G)$ belirsizdir.” Bazı G için bu doğrudur, diğerleri için bu yanlıştır, ancak açıkça birinin diğerine sahip olması gerekir. Sorun, bize verilen herhangi bir G için ifadenin doğru olup olmadığını belirlemektir. Yine, bir alt alan vardır, tüm bağlamdan bağımsız gramerlerin kümesi. Bir sorunun karar verilebilir olduğunu söyleriz eğer sorunun alanındaki her ifade için doğru cevabı veren bir Turing makinesi varsa.

Karar verilebilirlik veya karar verilemezlik sonuçlarını ifade ettiğimizde , her zaman alanın ne olduğunu bilmemiz gerekir, çünkü bu sonuçları etkil eyebilir. Problem,bir alanda karar verilebilirken başka bir alanda karar veri lemeyebilir. Özellikle,bir problemin tek bir örneği her zaman karar verilebi lir, çünkü cevap yadoğru ya da yanlıştır. İlk durumda, her zaman “doğru” y anıtını veren bir Turingmakinesi doğru cevabı verirken, ikinci durumda he r zaman “yanlış” yanıtını verenbir makine uygundur. Bu, alaycı bir cevap gi bi görünebilir, ancak önemli birnoktayı vurgular. Doğru cevabın ne olduğu nu bilmememiz fark etmez; önemli olan,doğru yanıtı veren bir Turing mak inesinin var olmasıdır.

Turing Makinesi Durdurma Problemi

Tarihi öneme sahip bazı problemlerle başlıyoruz ve bu problemler aynı zamandasonraki sonuçları geliştirmek için bir başlangıç noktası sağlıyor . Bunların eniyi bilinenleri Turing makinesi durdurma problemidir. Kısaca ifade etmek gerekirse,problem şudur: Bir Turing makinesi M 'nin tanı mı ve bir girdi w verildiğinde, M , başlangıç konfigürasyonu q_0w 'de başla tıldığında, sonunda duracak birhesaplama yapar mı? Problemi kısaca ifa de etmek için, M 'nin w 'ye uygulandığında,yani basitçe (M, w) , durup dur madığını soruyoruz. Bu problemin alanı, tüm Turingmakinelere ve tüm w 'lerin kümesi olarak alınmalıdır; yani, rastgele bir M ve w verildiğinde, M 'nin w 'ye uygulandığında hesaplamanın durup durmayacağınıöngöre cek tek bir Turing makinesi arıyoruz.

Bir Turing makinesi M üzerindeki w eylemini simüle ederek yanıtı bulamayız, ö rneğin bunu evrensel bir Turing makinesinde gerçekleştirerek, çünkü hesaplamanı n uzunluğu üzerinde bir sınır yoktur. Eğer M sonsuz bir döngüye girerse, ne kadar beklersek bekleyelim, aslında M 'nin bir döngüde olup olmadığından asla emin ola mayız. Bu, çok uzun bir hesaplama durumu olabilir. İhtiyacımız olan, makinenin ta nımı ve girdisi üzerinde bazı analizler yaparak herhangi bir M ve w için doğru yanıtı belirleyebilen bir algoritmadır. Ancak şimdi gösterdiğimiz gibi, böyle bir algoritma yoktur.

Sonraki tartışmalar için, durma problemi ile neyi kastettiğimiz hakkında kesin bir fikre sahip olmak uygundur; bu nedenle, yukarıda biraz gevşek bir ş ekilde ifade ettiğimiz şeyi belirli bir tanım yapıyoruz.

TANIM 12.1

Let w_M , bir Turing makinesi $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ tanımlayan bir dize olsun ve w , M 'nin alfabesinde bir dize olsun. w_M ve w 'nin 0 ve 1'lerin bir dizisi olarak ko dlandığını varsayacağız, Bölüm 10.4'te önerildiği gibi. Durma probleminin bir çözümü, herhangi bir w_M ve w için hesaplamayı gerçekleştiren bir Turin g makinesi H dir.

$$q_0 w_M w \vdash^* x_1 q_y x_2$$

eğer Mw 'ye uygulandığında durursa, ve

$$q_0 w_M w \vdash^* y_1 q_n y_2$$

eğer Mw 'ye uygulandığında durmazsa. Burada q_y ve q_n her ikisi de H 'nin son durumlarıdır.

TEOREM 12.1

Tanım 12.1'de gerektirdiği gibi davranan herhangi bir Turing makinesi H yoktur. Bu nedenle durma problemi çözümsüzdür.

Kanıt: Aksini varsayıyoruz, yani durma problemini çözen bir algoritmanın, ve dolayısıyla bir Turing makinesi H 'nin var olduğunu varsayıyoruz. Girdi H için dize $w_M w$ olacaktır. Gereksinim, herhangi bir $w_M w$ verildiğinde, Turing makinesi H 'nin evet veya hayır cevabı ile durmasıdır. Bunu, H 'nin iki karşılık gelen son durumdan birinde durmasını isteyerek sağlıyoruz, q_y veya q_n diyelim. Durum, Şekil 12.1 gibi bir blok diyagramı ile görüştürülebilir. Bu diyagramın amacı, eğer H durum q_0 'da $w_M w$ girişi ile başlatılırsa, sonunda durum q_y veya q_n 'da duracağını göstermektir. Tanım 12.1'de gerektirdiği gibi, H 'nin aşağıdaki kurallara göre çalışmasını istiyoruz:

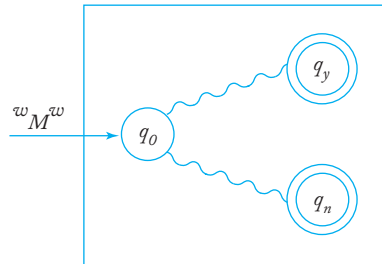
$$q_0 w_M w \vdash^*_{H} x_1 q_y x_2$$

eğer Mw 'ye uygulandığında durursa, ve

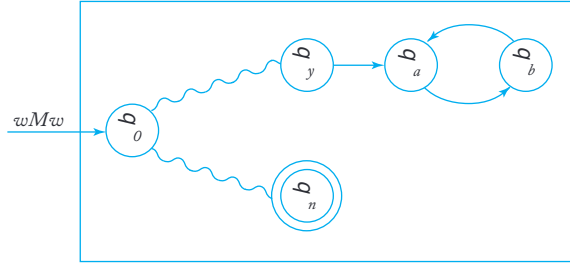
$$q_0 w_M w \vdash^*_{H} y_1 q_n y_2$$

eğer Mw 'ye uygulandığında durmazsa.

Sonra, Şekil 12.2'de gösterilen yapıya sahip bir Turing makinesi H' üretmek için H 'yi değiştiriyoruz. Şekil 12.2'deki ek durumlarla,



ŞEKİL 12.1



ŞEKİL 12.2

durumlar arasında geçişlerin q_y ve yeni durumlar q_a ve q_b yapılacağını, bant sembolüne bakılmaksızın, bantın değişmeden kalacak şekilde yapılması gerektiğini iletin. Bunun nasıl yapıldığı açıktır. Karşılaştırma H ve H' durumlarında H nin q_y ye ulaştığı ve durduğu durumlarda, modifiye edilmiş makine H' sonsuz bir döngüye girecektir. Resmi olarak, H' nin eylemi

$$q_0 w_M w \vdash_{H'}^* \infty$$

eğer Mw 'ye uygulandığında durursa, ve

$$q_0 w_M w \vdash_{H'}^* y_1 q_n y_2$$

eğer Mw 'ye uygulandığında durmazsa.

H' den başka bir Turing makinesi $\hat{H}$ inşa ediyoruz. Bu yeni makine w_M gi rdi olarak alır ve kopyalar, başlangıç durumu q_0 da sona erer. Daha sonra, H' g ibi tam olarak davranır. O zaman $\hat{H}$ nin eylemi şöyle olur ki

$$q_0 w_M \vdash_{\hat{H}}^* q_0 w_M w_M \vdash_{\hat{H}}^* \infty$$

eğer Mw_M üzerine uygulandığında durursa, ve

$$q_0 w_M \vdash_{\hat{H}}^* q_0 w_M w_M \vdash_{\hat{H}}^* y_1 q_n y_2$$

eğer Mw_M üzerine uygulandığında durmazsa.

Şimdi $\hat{H}$ bir Turing makinesidir, bu yüzden $\{0,1\}^*$ de bir tanımı vardır, diyelim ki, $\hat{w}$. Bu dize, $\hat{H}$ nin tanımı olmanın yanı sıra, girdi dizesi olarak da kullanılabilir. Bu n edenle, $\hat{H}$ nin $\hat{w}$ üzerine uygulanması durumunda ne olacağını meşru bir şekil de sorabiliriz. Yukarıdakilerden, M ile $\hat{H}$ yi tanımlayarak, elde ederiz

$$q_0 \hat{w} \vdash_{\hat{H}}^* \infty$$

eğer $\hat{H}\hat{w}$ üzerine uygulandığında durursa, ve

$$q_0 \hat{w} \vdash_{\hat{H}}^* y_1 q_n y_2$$

eğer $\hat{H}^w$ 'ye uygulandığında durmazsa. Bu açıkça saçmalık. Çelişki, H' 'nin varlığı varsayımımızın ve dolayısıyla durma probleminin karar verilebilirliği varsayımının yanlış olduğunu bize söyler. ■

Tanım 12.1'e itiraz edilebilir, çünkü durma problemini çözmek için H' 'nin çok belirli konfigürasyonlarda başlaması ve bitmesi gerektiğini talep ettik. Ancak, bu biraz keyfi olarak seçilmiş koşulların argümanda yalnızca küçük bir rol oynadığını görmek zor değildir ve esasen aynı akl yürütme, herhangi bir başka başlangıç ve bitiş konfigürasyonları ile de kullanılabilir. Problemi tartışma amacıyla belirli bir tanıma bağladık, ancak bu sonuca etki etmez.

Teorem 12.1'in ne söylediğini akılda tutmak önemlidir. Bu, belirli durumlar için durma problemini çözmeyi engellemez; genellikle M ve w 'yi analiz ederek Turing makinesinin durup durmayacağını söyleyebiliriz. Teorem, bunun her zaman yapılamayacağını söyler; tüm w^M ve w için doğru bir karar verebilecek bir algoritma yoktur.

Teorem 12.1'i kanıtlamak için verilen argümanlar klasik ve tarihi ilgi taşımaktadır. Teoremin sonucu aslında önceki sonuçlarda ima edilmiştir, aşağıdaki argüman bunu göstermektedir.

TEOREM 12.2

Eğer durma problemi karar verilebilir olsaydı, o zaman her rekürsif olarak sayılabilir dil rekürsif olurdu. Sonuç olarak, durma problemi karar verilemez.

Kanıt: Bunu görmek için $L \Sigma$ üzerinde rekürsif olarak sayılabilir bir dil olsun ve M_L 'yi kabul eden bir Turing makinesi olsun. Durma problemini çözen Turing makinesi H olsun. Bununla aşağıdaki prosedürü inşa ediyoruz:

1. H^{wiwMw} 'ye uygula. Eğer H "hayır" derse, o zaman tanım gereği wL 'ye ait değildir.
2. Eğer H "evet" derse, o zaman w 'ye M 'yi uygula. Ama M durmak zorundadır, bu yüzden $sonun\ da\ w$ 'ün L 'ye ait olup olmadığını bize söyleyecektir.

Bu, bir üyelik algoritması oluşturur ve L 'yi rekürsif hale getirir. Ama zaten rekürsif olmayan rekürsif olarak sayılabilir dillerin olduğunu biliyoruz. Çelişki, bu durumda H' 'nin var olamayacağını, yani durma probleminin karar verilemez olduğunu ima eder. ■

Durma probleminin Teorem 11.5'ten ne kadar basit bir şekilde elde edilebileceği, durma problemi ile rekürsif olarak sayılabilir diller için üyelik probleminin neredeyse özdeş olmasından kaynaklanmaktadır. Tek fark, durma probleminde

sonlu ve sonsuz durumda durmayı ayırt ederken, üyelik probleminde b unu yapıyoruz. Teorem 11.5'in (Teorem 11.3 aracılığıyla) ve 12.1'in kanıtı arı yakından ilişkilidir, her ikisi de diyagonalizasyonun bir versiyonudur.

Bir Karar Verilemeyen Problemi Diğerine İndirmek

Yukarıdaki argüman, durma problemini üyelik problemiyle ilişkilendirerek, çok önemli bir azaltma tekniğini göstermektedir. Bir problem A , eğer A 'nın karar verilebilirliği B 'nin karar verilebilirliğinden geliyorsa, problem B 'ye indirilmiştir. O zaman, eğer A 'nın karar verilemez olduğunu biliyorsak, o zaman B 'nin de karar verilemez olduğunu sonucuna varabiliriz. Bu fikri göstermek için birkaç örnek yapalım.

ÖRNEK 12.1

Durum-giriş problemi şu şekildedir. Herhangi bir Turing makinesi $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ ve herhangi bir $q \in Q, w \in \Sigma^+$ verildiğinde, durum q 'nin M 'ye w uygunluğunda hiç girilip girilmediğini karar verin. Bu problem karar verilemez.

Durma problemini durum-giriş problemine indirmek için, durum-giriş problemini çözen bir algoritmamız A olduğunu varsayalım. O zaman bunu durma problemini çözmek için kullanabiliriz. Örneğin, herhangi bir M ve w verildiğinde, önce M 'yi $\widehat{M}$ 'ye öyle bir şekilde değiştiriyoruz ki $\widehat{M}$, yalnızca M durursa durum q 'da durur. Bunu, yalnızca M 'nin geçiş fonksiyonu δ 'ya bakarak yapabiliriz. Eğer M durursa, bunu bazı $\delta(q_i, a)$ nedeniyle yapar. tanımsızdır. Her tanımsız δ için $\widehat{M}$ elde etmek üzere,

$$\delta(\quad) = (q, a, R$$

burada q son bir durumdur. Durum-giriş algoritması A uygularız. $(\widehat{M}, q, w)$. Eğer A evet derse, yani durum q girilmişse, o zaman (M, w) durur. Eğer A hayır derse, o zaman (M, w) durmaz.

Böylece, durum-giriş probleminin karar verilebilir olduğu varsa yımı, durdurma problemi için bir algoritma sağlar. Çünkü durma problemi karar verilemezdir, durum-giriş problemi de karar verilemez olmalıdır.

ÖRNEK 12.2

Boş-bant durma problemi, durma probleminin indirgenebileceği bir diğer problemdir. Verilen bir Turing makinesi M ,

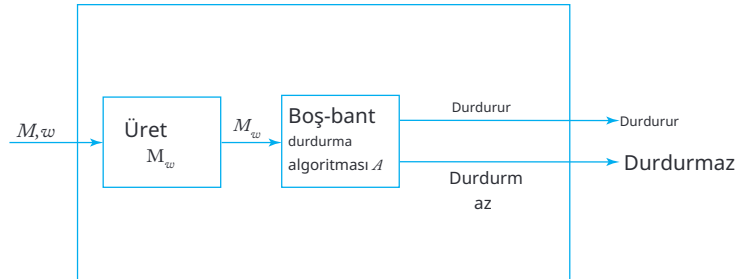
boş bir bantla başlatıldığında M durur mu yoksa durmaz mı belirleyin. Bu kararsızdır.

Bu indirimin nasıl gerçekleştirildiğini göstermek için, bazı M ve bazı w verildiğini varsayalım. Öncelikle M 'ye dayalı yeni bir makine M_w oluşturuyoruz, bu makine boş bir bantla başlar, üzerine w yazar, ardından kendisini bir konfigürasyona q_0w yerleştirir. Daha sonra, M_wM gibi davranır. Açıkça M_w yalnızca M w üzerinde duruyorsa boş bir bantta duracaktır.

Şimdi boş bant durma probleminin karara bağlanabilir olduğunu varsayalım. Verilen herhangi bir (M, w) için, önce M_w 'yi inşa ediyoruz, ardından boş bant durma problemi algoritmasını buna uyguluyoruz. Sonuç, M 'nin w 'ye uygulandığında durup durmayacağını bize söyler. Bu, herhangi bir M ve w için yapılabileceğinden, boş bant durma problemi için bir algoritma, durma problemi için bir algoritmaya dönüştürülebilir. İkincisinin kararsız olduğu bilindiğinden, boş bant durma problemi için de aynı şey geçerli olmalıdır.

Bu iki örneğin argümanlarındaki yapı, kararsızlık sonuçlarını belirleme de yaygın bir yaklaşımı göstermektedir. Bir blok diyagramı genellikle süreci görselleştirmemize yardımcı olur. Örnek 12.2'deki yapı, Şekil 12.3'te özetlenmiştir. O diyagramda, önce (M, w) 'yi M_w 'ye dönüştüren bir algoritma kullanıyoruz; böyle bir algoritmanın var olduğu açıktır. Sonra, var olduğunu varsaydığımız boş bant durma problemi için algoritmayı kullanıyoruz. İkisini bir araya getirmek, durma problemi için bir algoritma sağlar. Ancak bu imkansızdır ve A 'nın var olamayacağını sonucuna varabiliriz.

Bir karar problemi, etkili bir şekilde bir aralık $\{0,1\}$ olan bir fonksiyondur; yani, doğru veya yanlış bir cevap. Ayrıca, hesaplanabilir olup olmadıklarını görmek için daha genel fonksiyonlara da bakabiliriz; bunu yapmak için, durma problemini (veya kararsız olduğu bilinen herhangi bir problemi) söz konusu fonksiyonu hesaplama problemine indirgeriz. Turing'in tezi nedeniyle, pratik koşullarda karşılaşılan fonksiyonların hesaplanabilir olmasını bekliyoruz, bu nedenle hesaplanamaz fonksiyon örnekleri için biraz daha ileri gitmemiz gerekiyor. Hesaplanamaz fonksiyonların çoğu, Turing makinesinin davranışını tahmin etme girişimleriyle ilişkilidir.



ŞEKİL 12.3 Durdurma problemi için algoritma.

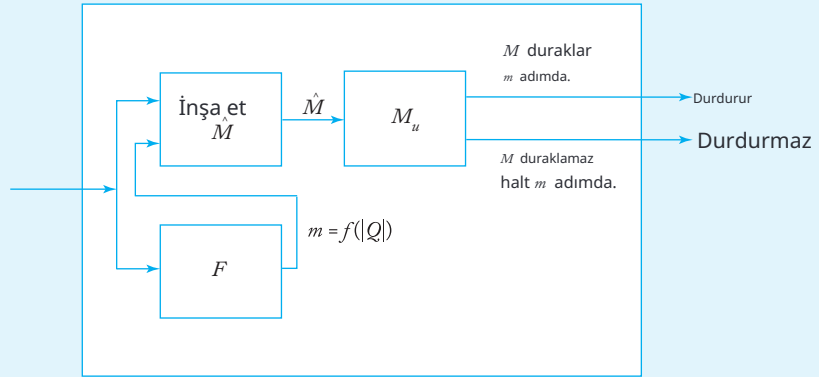
ÖRNEK 12.3

Let $\Gamma = \{0,1\}$. Consider the function $f(n)$ değeri herhangi bir n durum Turing makinesi tarafından yapılabilecek maksimum hareket sayısıdır boş bir bantla başlatıldığında durur. Bu fonksiyon, bu n sonucunda, hesaplanamaz.

Bu gösterime başlamadan önce, $f(n)$ olduğundan emin olalım. tüm n için tanımlanmıştır. Öncelikle, yalnızca sonlu sayıda

Turing makinesi olduğunu unutmayın n durumlarıyla. Bunun nedeni Q ve Γ 'nin sonlu olmasıdır, bu yüzden δ sonlu bir tanım alanına ve aralığa sahiptir. Bu da, yalnızca sonlu sayıda farklı δ olduğunu ve dolayısıyla sonlu sayıda farklı n durumlu Turing makineleri olduğunu ima eder.

Herhangi bir n durum makinesinin tümü arasında, her zaman duracak bazıları vardır; örneğin, yalnızca son durumları olan ve bu nedenle hiç hareket etmeyen makineler. Bazı n durum makineleri, boş bir bantla başlatıldıklarında durmayacaklardır, ancak bunlar f tanımına girmez. Duran her makine belirli sayıda hareket gerçekleştirecektir; bunlardan en büyüğünü alarak $f(n)$ veririz.



ŞEKİL 12.4 Boş bant duraklama problemi için algoritma.

Herhangi bir Turing makinesi M ve pozitif sayı m al. Kolaydır değiştirmek için $\widehat{M}$ böyle bir şekilde ki, ikincisi her zaman duracaktır ki cevapla: M boş bir bantta uygulanırsa en fazla m hareketle durur, ya da M boş bir bantta uygulanırsa m hareketten fazla yapar.

Bunun için yapmamız gereken tek şey M hareketlerini sayması ve sonlandırmasıdır bu sayı m 'yi aştığında. Şimdi varsayalım ki $f(n)$ bazı Turing makinesi F tarafından hesaplanabilir.

O zaman $\widehat{M}$ ve F 'yi aşağıda gösterildiği gibi bir araya getirebiliriz Şekil 12.4'te. Öncelikle $f(|Q|)$ hesaplıyoruz, burada Q M 'nin durum kümesidir.

Bu, M 'nin durması durumunda yapabileceği maksimum hareket sayısını bize söyler. Aldığımız değer daha sonra m olarak kullanılır ve aşağıdaki gibi inşa etmek için kullanılır

özetlendi ve $\widehat{M}$ 'nin bir evrensel Turing makinesi için yürütülmesi için bir tanımı verilir. Bu, M 'nin boş bir bantta uygulanmasının durup durmadığını bize söyler $f(|Q|)$ adımından daha az bir sürede. Eğer M 'nin boş bir bantta uygulanması $f(|Q|)$ hareketten fazla yapıyorsa, o zaman f 'nin tanımından dolayı, M 'nin asla durmadığı anlamına gelir. Böylece boş bant durma problemi için bir çözüm elde etmiş oluyoruz. Sonucun imkansızlığı, bize f 'nin hesaplanabilir olmadığını kabul ettiriyor.

ALISTIRMALAR

1. Teorem 12.1'deki H 'nin nasıl değiştirilebileceğini detaylı bir şekilde açıklayın H' üretmek için.
2. “Bir Turing makinesi tüm çift uzunlukta dizeleri kabul eder mi?” sorusunun kararsız olduğunu gösterin.
3. Aşağıdaki problemin kararsız olduğunu gösterin. Herhangi bir Turing makinesi verildiğinde $M, a \in \Gamma$ ve $w \in \Sigma^+$, sembolüne a yazılıp yazılmadığını belirleyin M w 'ye uygulandığında.
4. Genel durdurma probleminde, herhangi bir M ve w için doğru yanıt veren bir algoritma istiyoruz. Bu genel geçerliliği gevşetebiliriz, örneğin, tüm M için çalışsan ancak yalnızca tek bir w için çalışan bir algoritma arayarak. Böyle bir problem kararlı olduğunu söyleriz eğer her w için, (M, w) durup durmadığını belirleyen (muhtemelen farklı) bir algoritma varsa. Bu kısıtlı ortamda bile problemi n kararsız olduğunu gösterin.
5. Herhangi bir Turing makinesinin tüm girdilerde durup durmadığını belirlemek için bir algoritma olmadığını gösterin.
6. “Bir Turing makinesi bir hesaplama sürecinde başlangıç hüccresine (yani, hesaplamanın başında okuma-yazma başlığının altında bulunan hüccreye) geri döner mi?” sorusunu düşünün. Bu kararlı bir soru mu?
7. Herhangi iki Turing makinesinin M_1 ve M_2 aynı dili kabul etmediğini belirlemek için bir algoritma olmadığını gösterin.
8. Sonuç 7'nin, M_2 'nin sonlu bir otomaton olması durumunda nasıl etkilendiğini?
9. Deterministik yığın otomatonları için durma probleminin çözülebilir olup olmadığını; yani, Tanım 7.3'teki gibi bir pda verildiğinde, otomatonun girdi w üzerinde durup durmayacağını her zaman tahmin edebilir miyiz?
10. M herhangi bir Turing makinesi ve x ile y onun iki olası anlık tanımı olsun. Bunun durup durmadığını belirleme probleminin

$$x \vdash_M^* y$$

kararsız olduğunu gösterin.

11. Örnek 12.3'te, $f(1)$ ve $f(2)$ değerlerini verin.
12. Bir Turing makinesinin herhangi bir girdi üzerinde durup durmadığını belirleme probleminin kararsız olduğunu gösterin.
13. B , boş bir bantla başlatıldığında duran tüm Turing makinelerinin kümesi olsun. Bu kümenin tekrar üretilebilir olduğunu, ancak tekrar üretilebilir olmadığını gösterin.
14. Consider the set of all n -state Turing machines with tape alphabet $\Gamma = \{0, 1, \sqcup\}$. Give an expression for $m(n)$, the number of distinct Turing machines with this Γ .
15. Let $\Gamma = \{0, 1, \sqcup\}$ and let $b(n)$ be the maximum number of tape cells examined by any n -state Turing machine that halts when started with a blank tape. Gösterin ki $b(n)$ hesaplanamaz.
16. Aşağıdaki ifadenin doğru olup olmadığını belirleyin: Alanı sonlu olan herhangi bir problem karara bağlanabilir.

12.2 HESAPLANAMAYAN PROBLEMLER REKÜRSİF SAYILABİLİR DİLLER İÇİN

Rekürsif sayılabilir diller için bir üyelik algoritması olmadığını belirledik. Bazı özellikleri karara bağlayacak bir algoritmanın olmaması, rekürsif sayılabilir diller için olağanüstü bir durum değildir, aksine genel bir kuraldır. Şimdi gösterdiğimiz gibi, bu diller hakkında söyleyebileceğimiz çok az şey var. Rekürsif sayılabilir diller o kadar genel ki, aslında onlarınla ilgili sorduğumuz her soru hesaplanamaz. Sürekli olarak, rekürsif sayılabilir diller hakkında bir soru sorduğumuzda, bu soruya durma problemini indirgeme yolunun olduğunu buluyoruz. Burada bunun nasıl yapıldığını göstermek için bazı örnekler veriyoruz ve bu örneklerden genel durumu çıkarıyoruz.

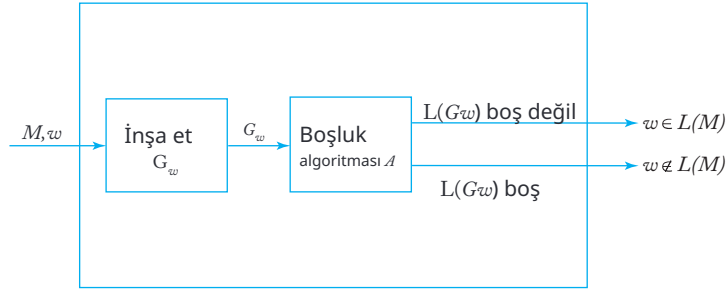
TEOREM 12.3

G serbest bir gramer olsun. O zaman, belirleme problemi

$$L(G) = \emptyset$$

kararsız olduğunu gösterin.

Kanıt: Rekürsif sayılabilir diller için üyelik problemini bu probleme indirgeceğiz. Bir Turing makinesi M ve bazı bir dize w verildiğini varsayalım. M 'yi aşağıdaki gibi değiştirebiliriz. M önce girişini banttaki özel bir kısımda kaydeder. Sonra, her final durumuna girdiğinde, kontrol eder.



ŞEKİL 12.5 Üyelik algoritması.

kaydedilen girişi alır ve yalnızca eğer w ise kabul eder. Bunu yapmak için δ 'yi basit bir şekilde değiştirerek, her w için bir makine M_w oluşturabiliriz, böylece

$$L(M_w) = L(M) \cap \{w\}.$$

Teorem 11.7'yi kullanarak, ardından karşılık gelen bir dil bilgisi G_w inşa ederiz. Açıkça, M ve w 'den G_w 'ye giden yapının her zaman yapılabileceği görülmektedir. Eşit derecede açıktır ki, $L(G_w)$ yalnızca $w \in L(M)$ olduğunda boştur.

Şimdi, $L(G) = \emptyset$ olup olmadığını belirlemek için bir algoritma A 'nın var olduğunu varsayalım. G_w 'yi ürettiğimiz bir algoritmayı T ile gösterelim, o zaman T ve A 'yı Şekil 12.5'te gösterildiği gibi bir araya getirebiliriz. Şekil 12.5, herhangi bir M ve w için w 'nin $L(M)$ içinde olup olmadığını bize söyleyen bir Turing makinesidir. Böyle bir Turing makinesi var olsaydı, herhangi bir özyinelemeli sayılabilir dil için bir üyelik algoritmamız olurdu, bu da daha önce belirlenmiş bir sonuca doğrudan çelişir. Bu nedenle, belirtilen " $L(G) = \emptyset$ " problemi karar verilebilir değildir. ■

TEOREM 12.4

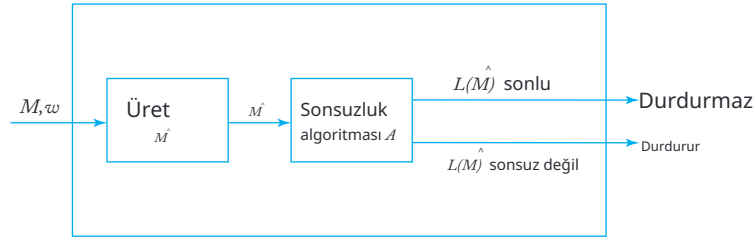
M herhangi bir Turing makinesi olsun. O zaman $L(M)$ 'nin sonlu olup olmadığı sorusu kararsızdır.

Kanıt: Durdurma problemini (M, w) düşünün. M 'den, aşağıdaki işlemleri yapan başka bir Turing makinesi $\widehat{M}$ inşa ediyoruz. Öncelikle, M 'nin durma durumları değiştirilir, böylece herhangi birine ulaşırsa, tüm girişler $\widehat{M}$ tarafından kabul edilir.

Bu, herhangi bir durma konumunun son bir duruma gitmesi sağlanarak yapılabilir.

İkincisi, orijinal makine, yeni makine $\widehat{M}$ önce w 'yi bantta üretmesi, ardından M ile aynı hesaplamaları yapması için değiştirilir; yeni oluşturulan w ve başka kullanılmayan bir alan kullanılarak. Diğer bir deyişle, M 'nin bantta w 'yi yazdıktan sonraki hareketleri, orijinal konfigürasyon q_0w ile başlamış olsaydı M tarafından yapılacak olanlarla aynıdır. Eğer

M herhangi bir konfigürasyonda durursa, o zaman $\widehat{M}$ son bir durumda duracaktır.



ŞEKİL 12.6

Bu nedenle, eğer (M, w) durursa, $\widehat{M}$ tüm girdi için bir son duruma ulaşacaktır. Eğer (M, w) durmazsa, o zaman $\widehat{M}$ da durmayacak ve dolayısıyla hiçbir şeyi kabul etmeyecektir. Diğer bir deyişle, $\widehat{M}$ ya sonsuz dil Σ^+ ya da sonlu dil $\emptyset$ kabul eder.

Şimdi, L 'nin sonlu olup olmadığını bize söyleyen bir algoritmanın A var olduğunu var sayarsak, $(\widehat{M})$ sonlu olup olmadığını belirlemek için bir çözüm oluşturabiliriz. Şekil 12.6'da gösterildiği gibi. Bu nedenle, $L(M)$ 'nin sonlu olup olmadığını belirleyen bir algoritma mevcut olamaz. ■

Teorem 12.4'ün kanıtında, sorunun özel doğası, yani " $L(M)$ sonlu mu?" önemsizdir. Problemin doğasını değiştirebiliriz, argümanı önemli ölçüde etkilemeden.

ÖRNEK 12.4

$\Sigma = \{a, b\}$ olan herhangi bir Turing makinesi M için " $L(M)$ iki farklı aynı uzunluktaki dize içeriyor mu?" problemine karar verilemeyeceğini gösterin.

Bunu göstermek için, Teorem 12.4'tekiyle tam olarak aynı yaklaşımı kullanıyoruz, ancak $\widehat{M}$ bir durma konumuna ulaştığında, iki dize a ve b 'yi kabul edecek şekilde değiştirilecektir. Bunun için, başlangıç girişi kaydedilir ve hesaplamamanın sonunda a ve b ile karşılaştırılır, sadece bu iki dize kabul edilir. Böylece, eğer (M, w) durursa, $\widehat{M}$ eşit uzunlukta iki dize kabul edecektir, aksi takdirde $\widehat{M}$ hiçbir şey kabul etmeyecektir. Argümanın geri kalanı daha sonra Teorem 12.4'teki gibi devam eder.

Aynı şekilde, " $L(M)$ beş uzunluğunda herhangi bir dize içeriyor mu?" veya " $L(M)$ düzenli mi?" gibi diğer soruları da argümanı esasen etkilemeden değiştirebiliriz. Bu sorular ve benzeri sorular hepsi karar verilemezdir. Bunu formelleştiren genel bir sonuç, Rice teoremi olarak bilinir. Bu teorem, herhangi bir önemsiz özelliğin

Özyinelemeli olarak sayılabilir dil kararsızdır. “Önemsiz olmayan” sıfatı, b azı ama tüm özyinelemeli olarak sayılabilir dillerin sahip olduğu bir özelliği ifade eder. Rice teoreminin kesin bir ifadesi ve kanıtı Hopcroft ve Ullman (1979) içinde bulunabilir.

ALISTIRMALAR

1. Teorem 12.4'teki makine $\widehat{M}$ nasıl inşa edildiğini detaylı bir şekilde gösterin.
2. “Bir Turing makinesi herhangi bir çift uzunlukta string kabul eder mi?” sorusunun kararsız olduğunu gösterin.
3. Önceki bölümün sonunda bahsedilen iki problemin, şunlar olduğunu gösterin:
 - (a) $L(M)$ beş uzunluğunda herhangi bir string içerir.
 - (b) $L(M)$ düzenlidir.
 kararsızdırlar.
4. Let M_1 ve M_2 rastgele Turing makineleri olsun. “ $L(M_1) \subseteq L(M_2)$ ” probleminin kararsız olduğunu gösterin.
5. Let G herhangi bir kısıtlamasız gramer olsun. $L(G)^R$ 'nin özyinelemeli olarak sayılabilir olup olmadığını belirlemek için bir algoritma var mı?
6. Let G herhangi bir kısıtlamasız gramer olsun. $L(G) = L(G)^R$ olup olmadığını belirlemek için bir algoritma var mı?
7. Let G_1 herhangi bir kısıtlamasız gramer olsun ve G_2 herhangi bir düzenli gramer olsun. Problemi gösterin

$$L(G_1) \cap L(G_2) = \emptyset$$
 kararsız olduğunu gösterin.
8. Let G_1 ve $G_2 \Sigma$ üzerindeki kısıtlamasız gramerler olsun. Problemi gösterin

$$L(G_1) \cup L(G_2) = \Sigma^*$$
 kararsız olduğunu gösterin.
9. Gösterin ki, Egzersiz 6'daki soru, $L(G_2)$ boş olmadığı sürece, herhangi bir sabit G_2 için kararsızdır.
10. Sınırsız bir dilbilgisi G için, “ $L(G) = L(G)^*$ mi?” sorusunun kararsız olduğunu gösterin. (a) Rice teoremi üzerinden ve (b) ilk ilkelerden argüman geliştirin.

12.3 POST İLETİŞİM PROBLEMİ

Durdurma probleminin kararsızlığı, özellikle bağlamdan bağımsız diller alanında birçok pratik sonuç doğurmaktadır. Ancak birçok durumda, durdurma problemi ile doğrudan çalışmak zahmetlidir ve durdurma problemi ile diğer problemler arasındaki boşluğu kapatan bazı ara sonuçlar elde etmek uygundur. Bu ara sonuçlar, durdurma probleminin kararsızlığından kaynaklanır, ancak incelemek istediğimiz problemlerle daha yakından ilişkilidir ve bu nedenle argümanları daha kolay hale getirir. Bu tür bir ara sonuç, Post iletişim problemidir.

Post iletişim problemi şu şekilde ifade edilebilir. Belirli bir alfabe Σ , n dizisinin iki dizisi verildiğinde,

$$A = w_1, w_2, \dots, w_n$$

ve

$$B = v_1, v_2, \dots, v_n,$$

bir A, B çifti için bir Post iletişim çözümünün (PC-çözümü) var olduğunu, $i, j, \dots, k$ gibi boş olmayan bir tam sayı dizisi varsa söyleriz.

$$w_i w_j \cdots w_k = v_i v_j \cdots v_k$$

Posta eşleşme problemi, herhangi bir (A, B) için bir PC çözümünün olup olmadığını bize söyleyecek bir algoritma geliştirmektir.

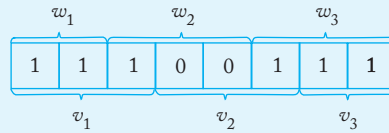
ÖRNEK 12.5

$\Sigma = \{0,1\}$ alalım ve A ile B olarak

$$w_1 = 11, w_2 = 100, w_3 = 111,$$

$$v_1 = 111, v_2 = 001, v_3 = 11.$$

Bu durumda, Şekil 12.7'nin gösterdiği gibi bir PC çözümü vardır.



ŞEKİL 12.7

Alırsak

$$w_1 = 00, w_2 = 001, w_3 = 1000,$$

$$v_1 = 0, v_2 = 11, v_3 = 011,$$

herhangi bir PC çözümü olamaz çünkü A 'nın elemanlarından oluşan herhangi bir dize, B 'den gelen karşılık gelen diziden daha uzun olacaktır.

Belirli durumlarda, açık bir yapı ile bir çift (A, B) için bir PC çözümünün mümkün olduğunu gösterebiliriz veya daha önce yaptığımız gibi, böyle bir çözümün var olamayacağını savunabiliriz. Ancak genel olarak, bu soruyu her koşulda çözmek için bir algoritma yoktur. Bu nedenle, Post karşılık problemi çözümsüzdür.

Bunu göstermek biraz uzun bir süreçtir. Açıklık adına, iki parçaya ayırıyoruz. İlk bölümde, değiştirilmiş Post karşılık problemini tanıtlıyoruz. Bir çift (A, B) değiştirilmiş Post karşılık çözümüne (MPC çözümü) sahipse, öyle bir tam sayı dizisi $i, j, \dots, k$ vardır ki

$$w_1 w_i w_j \cdots w_k = v_1 v_i v_j \cdots v_k.$$

Değiştirilmiş Post karşılık probleminde, dizilerin A ve B ilk elemanları özel bir rol oynar. Bir MPC çözümü, sol tarafta w_1 ile ve sağ tarafta v_1 ile başlamalıdır. Eğer bir MPC çözümü varsa, o zaman bir PC çözümü de vardır, ancak tersinin doğru olduğu söylenemez.

Değiştirilmiş Post karşılaştırma problemi, rastgele bir çiftin (A, B) bir MPC çözümü olup olmadığını belirlemek için bir algoritma geliştirmektir. Bu problem aynı zamanda kararsızdır. Değiştirilmiş Post karşılaştırma probleminin kararsızlığını, bilinen bir kararsız problem olan, tekrar edilebilir diller için üyelik problemini buna indirgemek suretiyle göstereceğiz. Bu amaçla, aşağıdaki yapıyı tanıtıyoruz. Farz edelim ki bize bir

A	B	
$FS \Rightarrow$	F	F, V 'de olmayan bir semboldür.
a	a	her bir $a \in \Sigma$
V_i	V_i	her bir $V_i \notin \Sigma$
E	$\Rightarrow wE$	E, V 'de olmayan bir semboldür.
y_i	x_i	her bir x_i, y_i içinde P
$\Rightarrow$	$\Rightarrow$	

ŞEKİL 12.8

kısıtlanmasız dil $G = (V, T, S, P)$ ve bir hedef dize w . Bunlarla, çift (A, B) oluşturunuz, Şekil 12.8'de gösterildiği gibi. Şekil 12.8'de, dize $FS \Rightarrow w_1$ olarak alınmalı ve dize Fv_1 olarak alınmalıdır. Diğer dizelerin sırası önemsizdir.

Sonunda $w \in L(G)$ olduğunu iddia etmek istiyoruz, eğer ve yalnızca bu şekilde oluşturulan A ve B kümeleri bir MPC çözümüne sahipse. Bu belki de hemen açık değildir, bu yüzden basit bir örnekle açıklayalım.

ÖRNEK 12.6

Let $G = (\{A, B, C\}, \{a, b, c\}, S, P)$ üretimleri ile

$$S \rightarrow aABb | Bbb,$$

$$Bb \rightarrow C,$$

$$AC \rightarrow aac,$$

ve $w = aaac$ alalım. Önerilen yapıdan elde edilen diziler A ve B Şekil 12.9'da verilmiştir. Dizi $w = aaac$

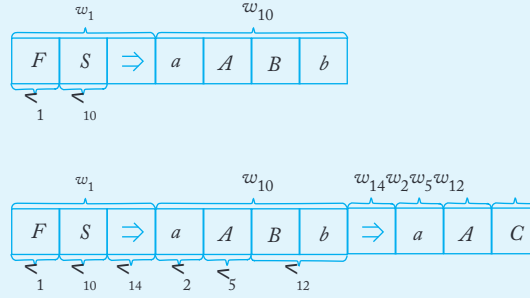
$L(G)$ içindedir ve bir türetimi vardır

$$S \Rightarrow aABb \Rightarrow aAC \Rightarrow aaac.$$

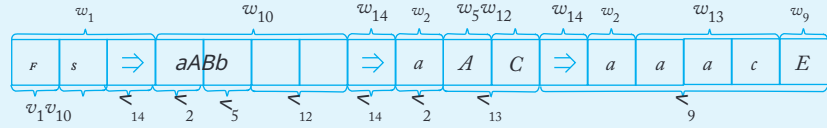
i	w_i	$\prec_i$
1	$FS \Rightarrow$	F
2	a	a
3	b	b
4	c	c
5	A	A
6	B	B
7	C	C
8	S	S
9	E	$\Rightarrow aaacE$
10	$aABb$	S
11	$Bb b$	S
12	C	Bb
13	aac	AC
14	$\Rightarrow$	$\Rightarrow$

ŞEKİL 12.9

Bu türetimin, oluşturulan kümelerle birlikte bir MPC çözümüyle nasıl paralel olduğunu Şekil 12.10'da görebiliriz; burada türetimin ilk iki adımı gösterilmektedir. Türetim dizisinin üstündeki ve altındaki tam sayılar, w_i için indeksleri göstermektedir ve v_i , sırasıyla, oluşturmak için kullanılır string.



ŞEKİL 12.10



ŞEKİL 12.11

Şekil 12.10'u dikkatlice inceleyin ve neler olduğunu görün. Bir MPC çözümü oluşturmak istiyoruz, bu yüzden w_1 ile başlamalıyız, yani $FS \Rightarrow$. Bu dize S içeriyor, bu nedenle eşleştirmek için v_{10} veya v_{11} kullanmalıyız. In bu durumda, v_{10} kullanıyoruz; bu, w_{10} getiriyor ve bizi kısmi türevdeki ikinci stringe yönlendiriyor. Birkaç adım daha bakarak, stringin $w_1w_iw_j\dots$ her zaman karşılık gelen stringden daha uzun olduğunu görüyoruz. $v_1v_iv_j\dots$ ve ilkinin türetimde tam olarak bir adım önde olduğunu. Tek istisna, son adımda w_9 uygulanması gerektiğidir. the v -string yakalamak. Tam MPC çözümü Şekil 12.11'de gösterilmektedir. Yapı, örnekle birlikte, bir sonraki sonucun belirlendiği hatları göstermektedir.

TEOREM 12.5

Let $G = (V, T, S, P)$ herhangi bir kısıtlanmasız gramer olsun, w herhangi bir dize olsun T^+ . Let (A, B) G ve w 'den oluşturulan karşılık çiftidir t sürecinin Şekil 12.8'de gösterildiği gibi. O zaman çift (A, B) bir MPC çözümünü sağlar eğer ve ancak $w \in L(G)$.

Kanıt:Kanıt, özetlenen akıl yürütmeye dayanan resmi bir indüktif argümanı içerir. Ayrıntıları atlayacağız. ■

Bu sonuçla, üyelik problemini özyinelemeli olarak sayılabilir diller için değiştirilmiş Post eşleşme problemine indirebiliriz ve böylece ikincisinin belirsizliğini gösterebiliriz.

TEOREM 12.6

Değiştirilmiş Post yazışma problemi karara bağlanamaz.

Kanıt: Verilen herhangi bir kısıtlamasız dilbilgisi $G = (V, T, S, P)$ ve $w \in T^+$, biz küme A ve B 'yi yukarıda önerildiği gibi oluşturuyoruz. Teorem 12.5'e göre, çift (A, B) bir MPC çözümüne sahipse ancak ve ancak $w \in L(G)$.

Şimdi, değiştirilmiş Post mektup eşleşmesi probleminin karar verilebilir olduğunu varsayalım. O zaman, Şekil 12.12'de taslağı verilen G 'nin üyelik problemi için bir algoritma oluşturabiliriz.

ABG ve w arasında açıkça vardır, ancak G ve w için bir üyelik algoritması yoktur. Bu nedenle, değiştirilmiş Post karşılaştırma problemini çözmek için herhangi bir algoritmanın olamayacağını sonucuna varmalıyız. ■

Bu ön çalışma ile şimdi Post eşleşme problemini orijinal biçiminde kanıtlamaya hazırız.

TEOREM 12.7

Post yazışma problemi kararsızdır.

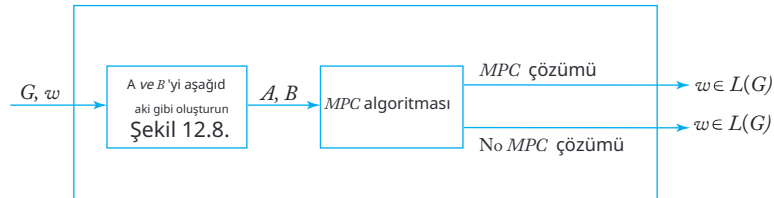
Kanıt: Eğer Post yazışma problemi kararlı olsaydı, modifiye edilmiş Post yazışma problemi de kararlı olurdu.

Diyelim ki, bazı bir alfabede $\Sigma, A = w_1, w_2, \dots, w_n$ ve $B = v_1, v_2, \dots, v_n$ sıraları verilmiştir. O zaman yeni semboller y_i ve yeni

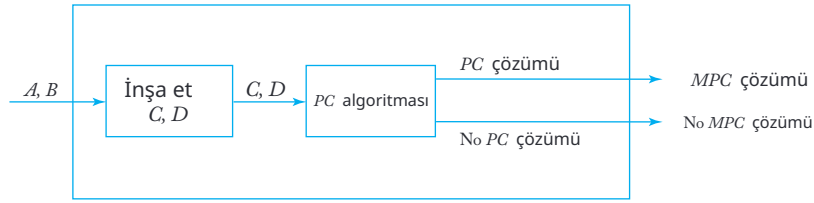
sıraları tanıtlıyoruz.

$$C = y_0, y_1, \dots, y_{n+1},$$

$$D = z_0, z_1, \dots, z_{n+1},$$



ŞEKİL 12.12 Üyelik algoritması.



ŞEKİL 12.13 MPC algoritması.

şu şekilde tanımlanır. İçin $i = 1, 2, \dots, n$

$$y_i = w_{i1} \natural w_{i2} \natural \dots \natural w_{im_i} \natural,$$

$$z_i = \natural v_{i1} \natural v_{i2} \natural \dots \natural v_{ir_i},$$

burada w_{ij} ve v_{ij} sırasıyla w_i ve v_i 'nin j inci harfini belirtir ve $m_i = |w_i|$, $r_i = |v_i|$. Sözcüklerle y_i , her bir karaktere ekleme yapılarak w_i 'den oluşturulur, oysa z_i , her bir karakterin v_i 'ye eklenmesiyle elde edilir. Tanımın tamamlanması için C ve D alırsız

$$y_0 = \natural y_1,$$

$$y_{n+1} = \S,$$

$$z_0 = \natural z_1,$$

$$z_{n+1} = \natural \S.$$

Şimdi (C, D) çiftini düşünelim ve bunun bir PC çözümü olduğunu varsayalım. Yerleştirme nedeniyle ve $\S$, böyle bir çözümün solda y_0 ve sağda y_{n+1} olması gerekir ve bu nedenle şöyle görünmelidir

$$\natural w_{11} \natural w_{12} \dots \natural w_{j1} \natural \dots \natural w_{k1} \dots \natural \S = \natural v_{11} \natural v_{12} \dots \natural v_{j1} \natural \dots \natural v_{k1} \dots \natural \S.$$

Karakterleri ve $\S$ göz ardı edersek, bunun şunu ima ettiğini görüyoruz

$$w_1 w_j \dots w_k = v_1 v_j \dots v_k,$$

bu nedenle (A, B) çifti bir MPC çözümüne izin verir.

Argümanı tersine çevirerek, (A, B) için bir MPC çözümü varsa, (C, D) çifti için bir PC çözümü olduğunu gösterebiliriz.

Şimdi Post yazışma probleminin karar verilebilir olduğunu varsayalım. O zaman Şekil 12.13'te gösterilen makineyi inşa edebiliriz. Bu makine açıkça değiştirilmiş Post yazışma problemini karar verir. Ancak değiştirilmiş Post yazışma problemi karar verilemez; dolayısıyla, Post yazışma problemini karar vermek için bir algoritmamız olamaz. ■

ALISTIRMALAR

1. $A = \{10, 00, 11, 01\}$ ve $B = \{0, 001, 1, 101\}$ için bir PC çözümünün var olduğunu gösterin.
2. $A = \{001, 0011, 11, 101\}$ ve $B = \{01, 111, 111, 010\}$ olarak alalım. Çiftin (A, B) bir PC çözümü var mı? Bir MPC çözümü var mı?
3. Teorem 12.5'in kanıtının detaylarını sağlayın.
4. $|\Sigma| = 1$ için, Post karşılaştırma probleminin karar verilebilir olduğunu gösterin; yani, herhangi bir (A, B) için bir algoritmanın (A, B) bir PC çözümüne sahip olup olmadığını karar verebileceği anlamına gelir.
5. Post karşılaştırma probleminin alanını yalnızca tam olarak iki sembol içeren alfabelerle sınırladığımızı varsayalım. Ortaya çıkan karşılaştırmalı problemi karar verilebilir mi?
6. Post karşılaştırma probleminin aşağıdaki değişikliklerinin karar verilemez olduğunu gösterin:
 - (a) Eğer $w_i w_j \cdots w_k w_1 = v_i v_j \cdots v_k v_1$ olan bir tam sayı dizisi varsa bir MPC çözümü vardır.
 - (b) Eğer bir dizi tam sayı varsa, o zaman bir MPC çözümü vardır böylece $w_1 w_2 w_i w_j \cdots w_k = v_1 v_2 v_i v_j \cdots v_k$.
7. İki dizi (A, B) çiftinin çift PC çözümü olduğu söylenir eğer ve yalnızca, boş olmayan bir çift tam sayı dizisi $i, j, \dots, k$ varsa öyle ki $w_i w_j \cdots w_k = v_i v_j \cdots v_k$. R astgele bir çiftin (A, B) çift PC çözümüne sahip olup olmadığını belirleme problemi kararsızdır.

12.4 KARARSIZ PROBLEMLER İÇİN KONTEKST-SIZ DİLLE

Post yazışma problemi, kontekst-sız diller için kararsız soruları incelemek için uygun bir araçtır. Bunu birkaç seçilmiş sonuçla gösteriyoruz.

TEOREM 12.8

Herhangi bir verilen kontekst-sız dilin belirsiz olup olmadığını belirlemek için bir algoritma yoktur.

Kanıt: İki dizi $A = (w_1, w_2, \dots, w_n)$ ve $B = (v_1, v_2, \dots, v_n)$ bazı bir alfabe Σ .

Yeni bir farklı sembol seti seçin $a_1, a_2, \dots, a_n$, öyle ki

$$\{a_1, a_2, \dots, a_n\} \cap \Sigma = \emptyset,$$

ve iki dili göz önünde bulundurun

$$L_A = \{w_i w_j \cdots w_l w_k a_k a_l \cdots a_j a_i\}$$

ve

$$L_B = \{v_i v_j \cdots v_l v_k a_k a_l \cdots a_j a_i\}.$$

Şimdi bağlamdan bağımsız grameri inceleyin

$$G = (\{S, S_A, S\}, \Sigma \cup \{a_1, a_2, \dots, a_n\}, P, S),$$

üretim kümesi P , iki alt kümenin birleşimidir: İlk

küme P_A şunlardan oluşur

$$\begin{aligned} &\rightarrow S_A, \\ S_A &\rightarrow w_i S_A a_i | w_i a_i, \quad i = 1, 2, \dots, n, \end{aligned}$$

ikinci küme P_B ise üretilere sahiptir

$$\begin{aligned} S &\rightarrow S_B, \\ S_B &\rightarrow v_i S_B a_i | v_i a_i, \quad i = 1, 2, \dots, n. \end{aligned}$$

Alın

$$G_A = (\{S, S_A\}, \Sigma \cup \{a_1, a_2, \dots, a_n\}, P_A, S)$$

ve

$$G_B = (\{S, S_B\}, \Sigma \cup \{a_1, a_2, \dots, a_n\}, P_B, S);$$

o zaman açıkça

$$\begin{aligned} L_A &= L(G_A), \\ L_B &= L(G_B), \end{aligned}$$

ve

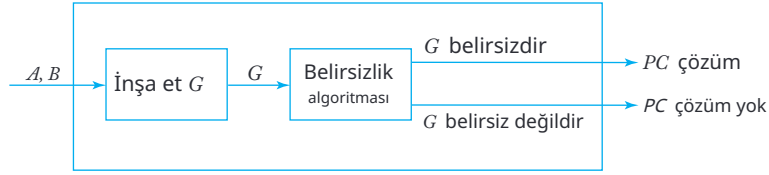
$$L(G) = L(G_A) \cup L(G_B).$$

G_A ve G_B kendileri açısından belirsiz olmadığını görmek kolaydır. Eğer $L(G)$ içindeki bir dize a_i ile bitiyorsa, o zaman G_A grameri ile türetilmesi $S \Rightarrow w_i S a_i$ ile başlamalıdır. Benzer şekilde, daha sonraki herhangi bir aşamada herhangi bir a_i uygulaması gerektiğini söyleyebiliriz. Böylece, eğer G belirsizse, bunun nedeni türetilmesi olan bir w olmasıdır

$$S \Rightarrow S_A \Rightarrow w_i S_A a_i \xrightarrow{*} w_i a_j \cdots w_k a_k \cdots a_j a_i = w$$

ve

$$S \Rightarrow S_B \Rightarrow v_i S_B a_i \xrightarrow{*} v_i v_j \cdots v_k a_k \cdots a_j a_i = w.$$



ŞEKİL 12.14PC algoritması.

Sonuç olarak, eğer G belirsizse, o zaman Post yazışma problemi çift (A, B) ile bir çözümü vardır. Tersine, eğer G belirsiz değilse, o zaman Post yazışma problemi için bir çözüm olamaz.

Belirsizlik sorununu çözmek için bir algoritma var olsaydı, bunu Şekil 12.14'te gösterildiği gibi Post yazışma sorununu çözmek için uyarlayabilirdik. Ancak Post yazışma sorunu için bir algoritma olmadığından, belirsizlik sorununun kararsız olduğunu sonucuna varıyoruz. ■

TEOREM 12.9

Karar vermek için bir algoritma yoktur

$$L(G_1) \cap L(G_2) = \emptyset$$

keyfi bağlamdan bağımsız gramerler G_1 ve G_2 .

Kanıt: G_1 olarak G_A gramerini ve G_2 olarak G_B gramerini alalım. Teorem 12.8'in kanıtında tanımlandığı gibi. Varsayalım ki $L(G_A)$ ve $L(G_B)$ bir ortak elemana sahiptir, yani,

$$S_A \xRightarrow{*} w_i w_j \cdots w_k a_k \cdots a_j a_i$$

ve

$$S_B \xRightarrow{*} v_i v_j \cdots v_k a_k \cdots a_j a_i.$$

O zaman (A, B) çifti bir PC çözümüne sahiptir. Tersine, eğer çiftin bir PC çözümü yoksa, o zaman $L(G_A)$ ve $L(G_B)$ ortak bir elemana sahip olamaz. Sonuç olarak, $L(G_A) \cap L(G_B)$ yalnızca (A, B) bir PC çözümüne sahipse boştur. Bu indirgeme teoremi kanıtlar. ■

Bu yönde bilinen başka çeşitli sonuçlar vardır. Bunların bazıları Post eşleşme problemine indirgenebilirken, diğerleri farklı ara sonuçlar belirleyerek daha kolay çözülebilir. Burada argümanları vermeyeceğiz, sadece bazı ek sonuçları belirteceğiz.

İşte bağlamdan bağımsız diller için bilinen bazı kararsız problemler:

- Eğer G_1 ve G_2 bağlamdan bağımsız gramerlerse, $L(G_1) = L(G_2)$ mi?
- Eğer G_1 ve G_2 bağlamdan bağımsız gramerlerse, $L(G_1) \subseteq L(G_2)$ mi?
- Eğer G_1 bir bağlamdan bağımsız gramerse, $L(G_1)$ düzenli mi?
- Eğer G_1 bir bağlamdan bağımsız gramerse ve G_2 düzenli ise, $L(G_1) \subseteq L(G_2)$ mi?
- Eğer G_1 bir bağlamdan bağımsız gramerse ve G_2 düzenli ise, $L(G_1) \cap L(G_2) = \emptyset$ mi?

Bu sonuçların çoğunun kanıtları uzun ve oldukça teknik olduğundan, burada atlayacağız.

Bağlamdan bağımsız dillerle ilgili birçok kararsız problemin varlığı ilk başta şaşırtıcı görünüyor ve bu, algoritmik bir yaklaşım denemek is teyebileceğimiz bir alanda hesaplamaların sınırlamaları olduğunu gösteriyor. Örneğin, BNF'de tanımlı bir programlama dilinin belirsiz olup olmadığını veya bir dilin iki farklı spesifikasyonunun aslında eşdeğer olup olmadığını söyleyebilirsek faydalı olurdu. Ancak, belirlenen sonuçlar bunun mümkün olmadığını söylüyor ve bu görevlerden her biri için bir algoritma aramak zaman kaybı olacaktır. Bunun, belirli durumlar veya belki de en ilginç olanlar için cevabı bulmanın yolları olabileceği olasılığını ortadan kaldırmadığını unutmayın. Kararsızlık sonuçları, tamam en genel bir algoritmanın olmadığını ve bir yöntem ne kadar farklı durumu ele alabilirse olsun, her zaman bozulacağı bazı durumların olacağını bize söylüyor.

12.5 ETKİNLİK SORUSU

Sadece hesaplanabilirlik veya karar verilebilirlik ile ilgilendiğimiz sürece, hangi Turing makinesi modelini kullandığımızın pek bir önemi yoktur. Ancak uygulama kolaylığı veya verimlilik gibi olası pratik endişelere bakmaya başladığımızda, önemli farklılıklar hızla ortaya çıkar. İşte bu konulara ilk bakışımızı veren iki örnek.

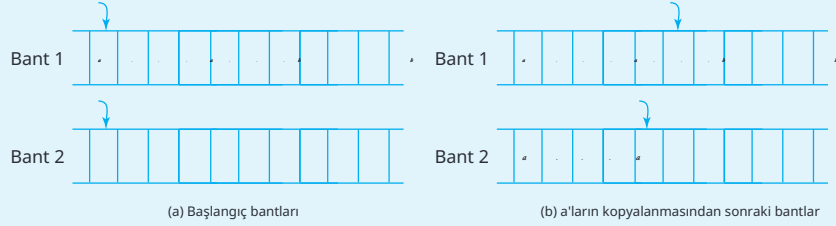
ÖRNEK 12.7

Örnek 9.7'de, dil için tek bantlı bir Turing makinesi inşa ettik.

$$L = \{a^n b^n : n \geq 1\}.$$

O algoritmaya bir bakış, her a için karşılık gelen b ile eşleşmek için yaklaşık olarak $2n$ adım gerektiğini gösterecektir. Bu nedenle, tüm hesaplama $O(n^2)$ hareket alır.

Ancak, daha sonra Örnek 10.1'de belirttiğimiz gibi, iki bantlı bir makine ile farklı bir algoritma kullanabiliriz. Öncelikle tüm a'ları ikinci bantta kopyalarız, ardından bunları birincideki b'lerle eşleştiririz. Kopyalama öncesi ve sonrası durum Şekil 12.15'te gösterilmiştir. Hem kopyalama hem de eşleştirme $O(n)$ hareketle yapılabilir ve bu nedenle çok daha verimlidir.



ŞEKİL 12.15

ÖRNEK 12.8

5.2 ve 6.3. Bölümlerde, bağlamdan bağımsız diller için üyelik problemi tartıştık. Girdi dizisinin uzunluğunu w problem boyutunu n olarak alırsak, kapsamlı arama $O(n^M)$ adım alır, burada M gramerle ilgili. Daha verimli CYK algoritması çalışma miktarını $O(n^3)$ gerektirir. Her iki algoritma da belirleyicidir.

Bu problem için bir belirsiz algoritma, w 'nin türetilmesinde hangi üretim dizisinin uygulandığını basitçe tahmin ederek ilerler.

Birim veya λ üretimleri olmayan bir gramerle çalışıyorsak, türetim uzunluğu esasen $|w|$ olduğundan, bu durumda bir $O(n)$

algoritmamız vardır.

Bu örnekler, verimlilik sorularının kullandığımız Turing makinesinin türünden etkilendiğini ve belirleyicilik ile belirsizlik meselesinin özellikle kritik bir konu olduğunu önermektedir. Bunu 14. Bölümde daha ayrıntılı olarak inceleyeceğiz.

ALISTIRMALAR

$\Sigma = \{a, b, c\}$ dilleri için algoritmaların verimliliğini değerlendirin.

1. $L = \{ww^R\}$.
2. $L = \{ww\}$.

3. $L = \{a^n b^n c^n, n \geq 1\}$.
4. $L = \{w : n_a(w) = n_b(w) = n_c(w)\}$.
5. $L = \{w_1 w_2 : |w_1| = |w_2|, w_1 = w_2\}$.

aşağıdaki durumların her biri için:

- Standart bir Turing makinesinde
 - İki bantlı deterministik Turing makinesinde
 - Tek bantlı belirsiz Turing makinesinde
 - İki bantlı belirsiz Turing makinesinde
-

BÖLÜM

13

DIĞER MODEL HESAPLAMALAR

BÖLÜM ÖZETİ

Dilbilgileri, bir dizeyi ardışık olarak yeniden yazmak için bir dizi kuraldan oluşan yeniden yazım sisteminin örnekleridir; bu kurallar, diğer dizeleri üretmek ve böylece dilleri tanımlamak için kullanılır. Bu bölüm, bazı tipik yeniden yazım sistemlerini kısaca özetler ve bunların Turing makineleri ile bağlantısını keşfeder.

Turing makineleri, inşa edebileceğimiz en genel hesaplama modelleri olsalar da, tek başına değil. Farklı zamanlarda, bazıları ilk bakışta Turing makinelerinden radikal bir şekilde farklı gibi görünen diğer modeller önerilmiştir. Ancak sonunda, tüm modellerin eş değeri olduğu bulunmuştur. Bu alandaki öncü çalışmaların çoğu 1930 ile 1940 yılları arasında yapılmış ve A. M. Turing gibi birçok matematikçi buna katkıda bulunmuştur. Bulunan sonuçlar, yalnızca mekanik hesaplama kavramına değil, aynı zamanda matematiğin tamamına ışık tutmuştur.

Turing'in çalışması 1936'da yayımlandı. O zamanlar ticari bilgisayarlar mevcut değildi. Aslında, bu fikir çok yüzeysel bir şekilde düşünülmüştü. Turing'in fikirleri sonunda bilgisayar biliminde çok önemli hale gelse de, onun asıl amacı dijital bilgisayarların incelenmesi için bir temel sağlamaktı. Turing'in ne yapmaya çalıştığını anlamak için, o dönemde matematiğin durumuna kısaca bakmalıyız.

Newton ve Leibniz'in on yedinci ve on sekizinci yüzyıllarda diferan siyel ve integral hesaplamayı keşfetmesiyle birlikte, matematiğe olan ilgi arttı ve disiplin patlayıcı bir büyüme dönemine girdi. Birçok farklı alan incelendi ve neredeyse hepsinde önemli ilerlemeler kaydedildi. On dokuzuncu yüzyılın sonuna gelindiğinde, matematiksel bilgi biriki mi oldukça büyük hale gelmişti. Matematikçiler, bazı mantıksal zorluk ların ortaya çıktığını ve daha dikkatli bir yaklaşım gerektirdiğini tanıya cak kadar sofistike hale gelmişlerdi. Bu, akıl yürütmede titizlikle ilgili b ir endişeye yol açtı ve süreç içinde matematiksel bilginin temellerinin i ncelenmesine neden oldu. Bunun neden gerekli olduğunu görmek içi n, matematiksel konularla ilgili hemen hemen her kitap ve makalede tipik bir kanıtın neyi içerdiğini düşünelim. Mantıklı iddiaların bir dizisi yapılıp, "kolayca görülebilir" ve "bunun sonucudur" gibi ifadelerle iç içe geçer. Bu tür ifadeler gelenekseldir ve bunlarla ne kastedildiği, eğer böyle bir talep olursa, daha ayrıntılı bir akıl yürütme sunulabileceğidir .Elbette, bu çok tehlikelidir, çünkü bazı şeyleri gözden kaçırmak, hatalı gizli varsayımlar kullanmak veya yanlış çıkarımlar yapmak mümkündür. Bu tür argümanları her gördüğümüzde, bize verilen kanıtın gerçe kten doğru olup olmadığını merak etmeden edemeyiz.

Sıklıkla, bunun nasıl anlaşılacağına dair bir yol yoktur ve uzun ve karmaşık kanıtlar yayımlanmış ve ancak önemli bir süre sonra hatalı olduğu bulunmuştur. Ancak pratik sınırlamalar nedeniyle, bu tür bir akıl yürütme çoğu matematikçi tarafından kabul edilmektedir. Argümanlar konuya ışık tutar ve en azından sonucun doğru olduğuna olan güvenimizi artırır. Ancak tamamen güvenilirli k talep edenler için, bunlar kabul edilemez.

Bu tür "dağınık" matematiğe bir alternatif, mümkün olduğunca formel leştirmektir. Bir dizi varsayılan verilere, aksiyomlar olarak adlandırılan, ve mantıksal çıkarım ve tümdengelim için kesin tanımlı kurallara başlarız. Kur allar, her biri bizi bir kanıtlanmış gerçekten diğerine götüren bir dizi adım da kullanılır. Kurallar, uygulanmalarının doğruluğunun rutin ve tamamen mekanik bir şekilde kontrol edilebileceği şekilde olmalıdır. Bir önerme, aks iyomlardan sonlu bir mantıksal adım dizisi ile türetilirse doğru olarak kabu l edilir. Eğer önerme, doğru olduğu kanıtlanabilen başka bir önerme ile çel işiyorsa, o zaman yanlış kabul edilir.

Böyle formal sistemler bulmak, on dokuzuncu yüzyılın sonunda mat ematiğin önemli bir hedefiydi. İki endişe hemen ortaya çıktı. Birincisi, si stemin tutarlı olması gerektiğiydi. Bununla, bir adım dizisi ile doğru old uğu kanıtlanabilen hiçbir önermenin, başka bir eşit derecede geçerli arg ümanla yanlış olduğu gösterilmemesi gerektiğini kastediyoruz. Matematikte tutarlılık vazgeçilmezdir ve tutarsız bir sistemden türetilen herh angi bir şey, üzerinde anlaştığımız her şeye aykırı olacaktır. İkinci bir endişe, bir sistemin tam olup olmadığıydı; bu, sistemde ifade edilebilen herhangi bir önermenin doğru veya yanlış olarak kanıtlanabileceği anlamına gelir. Bir sür e boyunca, tüm matematik için tutarlı ve tam sistemlerin geliştirilebileceği u muluyordu, böylece katı ama tamamen mekanik teorem kanıtlamaya kapı aç ılacaktı. Ancak bu umut, K. Gödel's'in çalışmalarıyla sarsıldı. Onun

ünlü eksiklik teoremi, Gödels, ilginç herhangi bir tutarlı sistemin eksik olması gerektiğini gösterdi; yani, bazı kanıtlanamaz önermeleri içermesi gerekir. Gödels'in devrim niteliğindeki sonucu 1931'de yayımlandı.

Gödels'in çalışması, kanıtlanamaz ifadelerin bir şekilde kanıtlanabilir olanlardan ayırt edilip edilemeyeceği sorusunu yanıtsız bıraktı, böylece matematiğin çoğunun mekanik olarak doğrulanabilir kanıtlarla kesin hale getirilebileceği umudu hâlâ vardı. Bu sorun, Turing ve o dönemin diğer matematikçileri, özellikle A. Church, S. C. Kleene ve E. Post tarafından ele alındı. Soruyu incelemek için çeşitli hesaplama formal modelleri oluşturuldu. Bunlar arasında Church ve Kleene'nin özyinelemeli fonksiyonları ve Post sistemleri öne çıkmaktadır, ancak incelenen birçok başka sistem de vardır. Bu bölümde, bu çalışmalardan ortaya çıkan bazı fikirleri kısaca gözden geçireceğiz. Burada ele alamayacağımız zengin bir materyal bulunmaktadır. Sadece çok kısa bir sunum yapacağız ve okuyucuyu detaylar için diğer kaynaklara yönlendireceğiz. Özyinelemeli fonksiyonlar ve Post sistemleri hakkında oldukça erişilebilir bir hesap, Denning, Dennis ve Qualitz (1978) tarafından bulunabilirken, çeşitli diğer yeniden yazım sistemleri hakkında iyi bir tartışma Salomaa (1973) ve Salomaa (1985) tarafından verilmiştir.

Burada incelediğimiz hesaplama modelleri ve önerilen diğerleri, çeşitli kökenlere sahiptir. Ancak sonunda, bunların hepsinin hesaplamaları gerçekleştirme gücü açısından eşdeğer olduğu bulunmuştur. Bu gözlemin ruhuna genellikle Church tezi denir. Bu tez, yeterince geniş oldukları takdirde, tüm olası hesaplama modellerinin eşdeğer olması gerektiğini belirtir. Ayrıca, bunun içinde doğuştan gelen bir sınırlama olduğunu ve hesaplamaları için açık bir yöntem vermeyen bazı fonksiyonların olduğunu ima eder. Bu iddia elbette Turing tezine çok yakındır ve ilişkilidir ve birleşik kavram bazen

Church-Turing tezi olarak adlandırılır. Bu, algoritmik hesaplama için genel bir ilke sağlar ve kanıtlanabilir olmamakla birlikte, daha güçlü modellerin bulunamayacağına dair güçlü kanıtlar sunar.

13.1 REKÜRSİF FONKSİYONLAR

Bir fonksiyon kavramı, matematiğin birçok alanı için temeldir. Bölüm 1.1'de özetlendiği gibi, bir fonksiyon, bir kümenin bir elemanına, fonksiyonun tanım kümesi olarak adlandırılan, başka bir kümede benzersiz bir değer atayan bir kuraldır. Bu çok geniş ve genel bir tanımdır ve bu ilişkiyi nasıl açıkça temsil edebileceğimiz sorusunu hemen gündeme getirir. Fonksiyonların tanımlanabileceği birçok yol vardır. Bazılarını sıkça kullanırken, diğerleri daha az yaygındır.

Fonksiyonel notasyona aşinayız; burada ifadeleri şöyle yazarız:

$$f(n) = n^2 + 1.$$

Bu, fonksiyonu f hesaplamak için bir tarif aracılığıyla tanımlar: Argüman n için herhangi bir değer verildiğinde, bu değeri kendisiyle çarpın ve ardından bir ekleyin. Fonksiyon bu açık şekilde tanımlandığı için, değerlerini tamamen mekanik bir şekilde hesaplayabiliriz. Fonksiyonun tanımını tamamlamak için f 'nin tanım kümesini de belirtmemiz gerekir. Örneğin, tanım kümesini tüm tam sayılar kümesi olarak alırsak, o zaman f 'nin aralığı pozitif tam sayılar kümesinin bir alt kümesi olacaktır.

Birçok çok karmaşık fonksiyon bu şekilde belirtilebileceğinden, notasyonun ne ölçüde evrensel olduğunu sormak mantıklıdır. Eğer bir fonksiyon tanımlanmışsa (yani, tanım kümesinin elemanları ile aralığı arasındaki ilişkiyi biliyorsak), bu fonksiyon böyle bir fonksiyonel biçimde ifade edilebilir mi? Bu soruyu yanıtlamak için, önce izin verilen biçimlerin ne olduğunu netleştirmeliyiz. Bunun için bazı temel fonksiyonları tanıtlıyoruz ve bunlardan daha karmaşık olanları inşa etmek için kurallar ekliyoruz.

Primitif Rekürsif Fonksiyonlar

Konuşmayı basit tutmak için, yalnızca bir veya iki değişkenin fonksiyonlarını ele alacağız, bunların tanım kümesi ya I , tüm negatif olmayan tam sayıların kümesi ya da $I \times I$, ve aralığı I içindedir. Bu bağlamda, temel

fonksiyonlarla başlıyoruz:

1. Sıfır fonksiyonu $z(x) = 0$, tüm $x \in I$ için.
2. Sonraki fonksiyon $s(x)$, değeri x 'ye göre sıradaki tam sayıdır, yani, yaygın notasyonda, $s(x) = x + 1$.
3. Projeksiyon cihazı işlevleri

$$p_k(x_1, x_2) = x_k, \quad k = 1, 2$$

Bu fonksiyonlardan daha karmaşık fonksiyonlar oluşturmanın iki yolu vardır:

1. Bileşim, bu sayede inşa ediyoruz

$$f(x, y) = h(g_1(x, y), g_2(x, y))$$

tanımlı fonksiyonlardan g_1, g_2, h .

2. İlkel özyineleme, bir fonksiyonun özyinelemeli olarak tanımlanabileceği üzerinden

$$\begin{aligned} f(x, 0) &= g_1(x), \\ f(x, y + 1) &= h(g_2(x, y), f(x, y)), \end{aligned}$$

tanımlı fonksiyonlardan g_1, g_2 ve h .

Bu işlemin nasıl çalıştığını, tam sayı aritmetiğinin temel işlemlerinin bu şekilde nasıl inşa edilebileceğini göstererek açıklıyoruz.

ÖRNEK 13.1

Tam sayılar x ve y toplaması, aşağıdaki gibi tanımlanan $\text{add}(x, y)$ fonksiyonu ile uygulanabilir

$$\begin{aligned}\text{add}(x, 0) &= x, \\ \text{add}(x, y + 1) &= \text{add}(x, y) + 1.\end{aligned}$$

2 ve 3'ü toplamak için, bu kuralları ardışık olarak uyguluyoruz:

$$\begin{aligned}\text{add}(3, 2) &= \text{add}(3, 1) + 1 \\ &= (\text{add}(3, 0) + 1) + 1 \\ &= (3 + 1) + 1 \\ &= 4 + 1 = 5.\end{aligned}$$

ÖRNEK 13.2

Örnek 13.1'de tanımlanan add fonksiyonunu kullanarak, çarpımı aşağıdaki gibi tanımlayabiliriz

$$\begin{aligned}\text{mult}(x, 0) &= 0, \\ \text{mult}(x, y + 1) &= \text{add}(x, \text{mult}(x, y)).\end{aligned}$$

Resmi olarak, ikinci adım, h 'nin add fonksiyonu ile tanımlandığı ve $\text{g2}(x, y)$ fonksiyonunun projector fonksiyonu $\text{p1}(x, y)$ olduğu ilkel rekürsiyonun bir uygulamasıdır.

ÖRNEK 13.3

Çıkarma o kadar da belirgin değildir. Öncelikle, sistemimizde negatif sayıların izin verilmediğini dikkate alarak bunu tanımlamalıyız. Alışılmış çıkarma işlemi ile tanımlanan bir tür çıkarma vardır

$$\begin{aligned}x \dot{-} y &= x - y \text{ if } x \geq y, \\ x \dot{-} y &= 0 \text{ if } x < y.\end{aligned}$$

Operatör $\dot{-}$ bazen monus olarak adlandırılır; çıkarma işlemini tanımlar, böylece aralığı I olur.

Şimdi öncül fonksiyonu tanımlıyoruz

$$\begin{aligned} \text{pred}(0) &= 0, \\ \text{pred}(y+1) &= y, \end{aligned}$$

ve buradan çıkarma fonksiyonunu elde ediyoruz

$$\begin{aligned} \text{subtr}(x, 0) &= x, \\ \text{subtr}(x, y+1) &= \text{pred}(\text{subtr}(x, y)). \end{aligned}$$

$5 - 3 = 2$ olduğunu kanıtlamak için, tanımları bir dizi kez uygulayarak önermeyi azaltıyoruz:

$$\begin{aligned} \text{subtr}(5, 3) &= \text{pred}(\text{subtr}(5, 2)) \\ &= \text{pred}(\text{pred}(\text{subtr}(5, 1))) \\ &= \text{pred}(\text{pred}(\text{pred}(\text{subtr}(5, 0)))) \\ &= \text{pred}(\text{pred}(\text{pred}(5))) \\ &= \text{pred}(\text{pred}(4)) \\ &= \text{pred}(3) \\ &= 2. \end{aligned}$$

Aynı şekilde, tam sayı bölmesini tanımlayabiliriz, ancak bunu bir alıştırma olarak bırakacağız. Bunu verili olarak kabul edersek, temel aritmetik işlemlerin hepsinin tanımlanan temel süreçlerle inşa edilebileceğini görürüz. Cebirsel işlemler kesin bir şekilde tanımlandığında, şimdi daha karmaşık olanlar inşa edilebilir ve basit olanlardan çok karmaşık hesaplamalar oluşturulabilir. Bu şekilde inşa edilebilen fonksiyonlara ilkel özyinelemeli (primitive recursive) diyoruz.

DEFINITION 13.1

Bir fonksiyon, yalnızca temel fonksiyonlardan z , s , pk , ardışık bileşim ve ilkel rekürsif fonksiyon ile inşa edilebiliyorsa ilkel rekürsif olarak adlandırılır.

Dikkat edin ki, eğer g_1, g_2 ve h toplam fonksiyonlarsa, o zaman f , bileşim ve ilkel rekürsif fonksiyon ile tanımlanan bir toplam fonksiyondur. Bu durumdan, her ilkel rekürsif fonksiyonun I veya $I \times I$ üzerinde bir toplam fonksiyon olduğu sonucuna varılır.

İlkel rekürsif fonksiyonların ifade gücü oldukça fazladır, ve en yaygın fonksiyonların çoğu ilkel rekürsiftir. Ancak, tüm fonksiyonlar bu sınıfta değildir, aşağıdaki argüman bunu göstermektedir.

THEOREM 13.1

F kümesini I' 'ye I' 'ye giden tüm fonksiyonların kümesi olarak tanımlayalım. O zaman F için de ilkel rekürsif olmayan bazı fonksiyonlar vardır.

Kanıt: Her ilkel rekürsif fonksiyon, nasıl tanımlandığını gösteren sonlu bir dizi ile tanımlanabilir. Bu tür diziler kodlanabilir ve standart sırada düzenlenebilir. Bu nedenle, tüm ilkel rekürsif fonksiyonların kümesi sayılabilir.

Şimdi tüm fonksiyonların kümesinin de sayılabilir olduğunu varsayalım. O zaman tüm fonksiyonları bir sırada yazabiliriz, diyelim ki, $f_1, f_2, \dots$. Sonra bir fonksiyon g tanım layarak inşa ederiz

$$g(i) = f_i(i) + 1, \quad i = 1,$$

Açıkça, g iyi tanımlanmıştır ve bu nedenle F içindedir, ancak aynı şekilde, g her f_i ile çapraz pozisyonda farklıdır. Bu çelişki, F 'nin sayılabilir olmayacağını kanıtlar.

Bu iki gözlemi birleştirmek, F içinde ilkel özyinelemeli olmayan bazı fonksiyonlar olması gerektiğini kanıtlar. ■

Aslında, bu daha da ileri gidiyor; sadece ilkel özyinelemeli olmayan fonksiyonlar değil, aslında ilkel özyinelemeli olmayan hesaplanabilir fonksiyonlar da vardır.

THEOREM 13.2

Let C, I 'den I' 'ye giden tüm toplam hesaplanabilir fonksiyonların kümesi olsun. O zaman C içinde ilkel özyinelemeli olmayan bazı fonksiyonlar vardır.

Kanıt: Önceki teoremin argümanı ile, tüm ilkel özyinelemeli fonksiyonların kümesi sayılabilir. Bu kümedeki fonksiyonları $r_1, r_2, \dots$ olarak adlandıralım ve bir fonksiyon g tanımlayalım

$$g(i) = r_i(i) + 1.$$

Yapı itibarıyla, fonksiyon g her r_i ile farklıdır ve bu nedenle ilkel özyinelemeli değildir. Ancak açıkça g hesaplanabilir, teoremi kanıtlar. ■

İlkel özyinelemeli olmayan hesaplanabilir fonksiyonların varlığına dair yapısal olmayan kanıt, çaprazlama ile oldukça basit bir alıştırma değildir. Böyle bir fonksiyonun örneğinin gerçek yapımı çok daha karmaşık bir meseledir. Burada oldukça basit görünen bir örnek vereceğiz; ancak, bunun ilkel özyinelemeli olmadığını gösterme süreci oldukça uzundur.

Ackermann Fonksiyonu

Ackermann fonksiyonu, $I \times I$ 'den I 'ye giden bir fonksiyondur ve şöyle tanımlanır:

$$\begin{aligned} A(0, y) &= y, \\ A(x, 0) &= A(x-1, 1), \\ A(x, y+1) &= A(x-1, A(x, y)) \end{aligned}$$

A'nın toplam, hesaplanabilir bir fonksiyon olduğunu görmek zor değildir. Aslında, onun hesaplanması için bir özyinelemeli bilgisayar programı yazmak oldukça temel bir işlemdir. Ancak, görünüşteki basitliğine rağmen, Ackermann fonksiyonu ilkel özyinelemeli değildir.

Elbette, A'nın tanımından doğrudan argüman üretemeyiz. Bu tanım ilkel özyinelemeli bir fonksiyon için gereken biçimde olmasa da, uygun bir alternatif tanımın var olabileceği mümkündür.

Buradaki durum, bir dilin düzenli olmadığını veya bağlamdan bağımsız olmadığını kanıtlamaya çalıştığımızda karşılaştığımız duruma benzer. Tüm ilkel özyinelemeli fonksiyonlar sınıfının bazı genel özelliklerine başvurmalıyız ve Ackermann fonksiyonunun bu özelliği ihlal ettiğini göstermeliyiz. İlkel özyinelemeli fonksiyonlar için, böyle bir özellik büyüme oranıdır. İlkel özyinelemeli bir fonksiyon $f(n)$ 'nin $n \rightarrow \infty$ olarak ne kadar hızlı büyüebileceğine bir sınır vardır ve Ackermann fonksiyonu bu sınırı ihlal eder. Ackermann fonksiyonunun çok hızlı büyüdüğü kolayca gösterilebilir; bu bölümün sonunda, örneğin, 9'dan 11'e kadar olan alıştırmalara bakın. Bunun, ilkel özyinelemeli fonksiyonlar için büyüme sınırıyla nasıl ilişkili olduğu aşağıdaki teoremden netleştirilmiştir. Kanıtı, zahmetli ve teknik olduğu için atlanacaktır.

TEOREM 13.3

f herhangi bir ilkel rekürsif fonksiyon olsun. O zaman bazı bir tam sayı n vardır ki

$$f(i) < A(n, i),$$

tüm $i = n, n+1, \dots$ için.

Kanıt: Argümanın detayları için bkz. Denning, Dennis ve Qualitz (1978, s. 534). ■

Eğer bu sonucu kabul edersek, Ackermann fonksiyonunun

ilkel rekürsif olmadığı kolayca ortaya çıkar.

TEOREM 13.4

Ackermann fonksiyonu ilkel rekürsif değildir.

Kanıt: Fonksiyonu dikkate alalım

$$g(i) = A(i, i).$$

Eğer A ilkel özyinelemeli olsaydı, g de öyle olurdu. Ama o zaman, Teorem 13.3'e göre, öyle bir n vardır ki

$$g(i) < A(n, i),$$

tüm i için. Eğer şimdi i = n seçersek, çelişki elde ederiz

$$\begin{aligned} g(n) &= A(n, n) \\ &< A(n, n), \end{aligned}$$

A'nın ilkel özyinelemeli olamayacağını kanıtlar. ■

μ Özyinelemeli Fonksiyonlar

Özyinelemeli fonksiyonların fikrini Ackermann fonksiyonu ve diğer hesaplanabilir fonksiyonları kapsayacak şekilde genişletmek için, bu tür fonksiyonların inşa edilebileceği kurallara bir şey eklememiz gerekir. Bir yol, μ veya minimalizasyon operatörünü tanıtmaktır, bu da

$$\mu y (g(x, y)) = \text{smallest } y \text{ such that } g(x, y) = 0.$$

Bu tanımda, g'nin bir toplam fonksiyon olduğunu varsayıyoruz.

ÖRNEK 13.4

Let

$$g(x, y) = x + y - 3,$$

toplam bir fonksiyondur. Eğer $x \leq 3$ ise,

$$y = 3 - x$$

minimalizasyonun sonucudur, ancak eğer $x > 3$ ise, öyle bir y yoktur ki $y \in I$ ve $x + y - 3 = 0$. Bu nedenle,

$$\begin{aligned} \mu y (g(x, y)) &= 3 - x, & \text{for } x \leq 3, \\ &= \text{undefined}, & \text{for } x > 3. \end{aligned}$$

Bu durumdan, $g(x, y)$ toplam bir fonksiyon olmasına rağmen, $\mu y (g(x, y))$ yalnızca kısmi olabilir.

Önceki örneğin gösterdiği gibi, minimalizasyon işlemi, kısmi fonksiyonların özyinelemeli olarak tanımlanması olasılığını açar. Ancak, toplam fonksiyonları tanımlama gücünü de genişlettiği ve tüm hesaplanabilir fonksiyonları kapsayacak şekilde dahil ettiği ortaya çıkıyor. Yine, yalnızca ana sonucu alıntı yapıyoruz ve detayların bulunabileceği literatüre referans veriyoruz.

TANIM 13.2

Bir fonksiyon, μ -recursive olarak adlandırılırsa, temel fonksiyonlardan μ -operatorü ve bileşim ile ilkel özyineleme işlemlerinin bir dizisi ile inşa edilebiliyorsa.

TEOREM 13.5

Bir fonksiyon, yalnızca hesaplanabilir olduğunda μ -recursive'dir.

Kanıt: Bir kanıt için, Denning, Dennis ve Qualitz (1978, Bölüm 13) bakınız. ■

Bu μ -özyinelemeli fonksiyonlar, bize algoritmik hesaplama için başka bir model sunar.

ALISTIRMALAR

1. Örnekler 13.1 ve 13.2'deki tanımları kullanarak $3 + 4 = 7$ ve $2 * 3 = 6$.

2. Fonksiyonu tanımlayın

$$\begin{aligned} greater(x, y) &= 1 \text{ if } x > y, \\ &= 0 \text{ if } x \leq y. \end{aligned}$$

Bu fonksiyonun ilkel özyinelemeli olduğunu gösterin.

3. Fonksiyonu tanımlayın

$$\begin{aligned} less(x, y) &= x \text{ if } x < y, \\ &= 0 \text{ if } x \geq y. \end{aligned}$$

Bu fonksiyonun ilkel özyinelemeli olduğunu gösterin.

4. Fonksiyonu dikkate alın

$$\begin{aligned} \text{equals}(x, y) &= 1 \quad \text{if } x = y, \\ &= 0 \quad \text{if } x \neq y. \end{aligned}$$

Bu fonksiyonun ilkel özyinelemeli olduğunu gösterin.

5. Let f be defined by

$$\begin{aligned} f(x, y) &= x \quad \text{if } x \neq y, \\ &= 0 \quad \text{if } x = y. \end{aligned}$$

Bu fonksiyonun ilkel özyinelemeli olduğunu gösterin.

6. Let f be defined by

$$\begin{aligned} f(x, y) &= xy \quad \text{if } x = 2y, \\ &= 0 \quad \text{if } x \neq 2y. \end{aligned}$$

Bu fonksiyonun ilkel özyinelemeli olduğunu gösterin.

7. Tam sayı bölümü iki fonksiyon div ve rem ile tanımlanabilir:

$$\text{div}(x, y) = n,$$

buradan n , öyle bir en büyük tam sayıdır ki $x \geq ny$, ve

$$\text{rem}(x, y) = x - ny.$$

Fonksiyonların div ve rem ilkel rekürsif olduğunu gösterin.

8. Gösterin ki

$$f(n) = 2^n$$

9. Gösterin ki

$$f(n) = 2^{2^n}$$

ilkel rekürsiftir.

10. Fonksiyonun gösterin

$$g(x, y) = x^y$$

ilkel rekürsiftir.

11. Ackermann fonksiyonunu hesaplamak için bir bilgisayar programı yazın. Bunu kullanarak

$\text{evaluate}.A(2, 5)$ ve $A(3, 3)$ değerlerini hesaplayın.

12. Ackermann fonksiyonu için aşağıdakini kanıtlayın:

$$(a) \quad A(1, y) = y + 2.$$

$$(b) \quad A(2, y) = 2y + 3.$$

$$(c) \quad A(3, y) = 2^{y+3} - 3.$$

13. Egzersiz 12'yi kullanarak $A(4, 1)$ ve $A(4, 2)$ 'yi hesaplayın.

14. Genel bir ifade verin $A(4, y)$ için.
15. $A(5, 2)$ 'nin hesaplanmasında özyinelemeli çağrılar dizisini gösterin.
16. Ackermann fonksiyonunun $I \times I$ içinde bir toplam fonksiyon olduğunu gösterin.
17. Egzersiz 11 için oluşturulan programı kullanarak $A(5, 5)$ 'i değerlendirmeye çalışın. Gözlemlediğiniz şeyi açıklayabilir misiniz?
18. Aşağıdaki her g için, hesaplayın $\mu y(g(x, y))$, ve tanım kümesini belirleyin.
 - (a) $g(x, y) = xy$.
 - (b) $g(x, y) = 2^x + 2y^2 - 2$.
 - (c) $g(x, y) = \text{tam sayı kısmı } (x-1)/(y+1)$.
 - (d) $g(x, y) = x \bmod (y+1)$.
19. Örnek 13.3'teki pred tanımı, sezgisel olarak net olmasına rağmen, bir ilkel rekürsif fonksiyon tanımına katı bir şekilde uymamaktadır. Tanımın, doğru forma sahip olacak şekilde nasıl yeniden yazılabileceğini gösterin.

13.2 POST SİSTEMLERİ

Bir Post sistemi, ardışık dizelerin türetilbileceği bir alfabaden ve bazı üretim kurallarından oluşan sınırsız bir gramer gibi görünmektedir. Ancak, üretimlerin uygulanma şekli açısından önemli farklılıklar vardır.

TANIM 13.3

Bir Post sistemi Π şu şekilde tanımlanır

$$\Pi = (C, V, A, P),$$

burada

C , iki ayrık kümeden oluşan, sonlu bir sabitler kümesidir C_N , yani terminal olmayan sabitler ve C_T , terminal sabitler kümesi,

V sonlu bir değişkenler kümesidir,

A, C^* 'den bir sonlu kümedir ve aksiyomlar olarak adlandırılır,

P , sonlu bir üretim kümesidir.

Bir Post sistemindeki üretimler belirli kısıtlamaları karşılamalıdır. Onlar şu formda olmalıdır

$$x_1 V_1 x_2 \cdots V_{n+1} \rightarrow y_1 W_1 \cdots W_{m+1}, \quad (13.1)$$

burada $x_i, y_i \in C^*$ ve $V_i, W_i \in V$ olup, herhangi bir

değişkenin solda en fazla bir kez görünebileceği gereksinimine tabidir, böylece

$$V_i \neq V_j \text{ for } i \neq j,$$

ve sağdaki her değişkenin solda görünmesi gerektiği, yani,

$$\bigcup_{i=1}^m W_i \subseteq \bigcup_{i=1}^n V_i.$$

Terminal dizisinin şu biçimde olduğunu varsayalım $x_1 w_1 x_2 w_2 \cdots w_n x_n$ +1, alt diziler $x_1, x_2, \dots$ (13.1)'deki karşılık gelen dizelerle eşleşir ve $w_i \in C^*$. Bu durumda $w_1 = V_1, w_2 = V_2, \dots$ olarak tanımlayabiliriz, ve bu değerleri (13.1)'deki W 'lerin sağında yerine koyabiliriz. Her W , solda bulunan bir V_i olduğu için, ona benzersiz bir değer atanır ve yeni dizeyi $y_1 w_i y_2 w_j \cdots y_{m+1}$ olarak elde ederiz. Bunu şöyle yazarız

$$x_1 w_1 x_2 w_2 \cdots x_{n+1} \Rightarrow y_1 w_i y_2 w_j \cdots y_{m+1}.$$

Bir dilbilgisi açısından, şimdi bir Post sisteminin türettiği dilden bahsedebiliriz.

TANIM 13.4

Post sistemi $\Pi = (C, V, A, P)$ tarafından üretilen dil

$$L(\Pi) = \left\{ w \in C_T^* : w_0 \xRightarrow{*} w \text{ for some } w_0 \in A \right\}.$$

ÖRNEK 13.5

Post sistemini düşünelim

$$C_T = \{a, b\},$$

$$C_N = \emptyset,$$

$$V = \{V_1\},$$

$$A = \{\lambda\},$$

ve üretim

$$V_1 \rightarrow aV_1b.$$

Bu, türetime izin verir

$$\lambda \Rightarrow ab \Rightarrow aabb.$$

İlk adımda, (13.1) uyguluyoruz ve tanımlama $x_1=\lambda, V_1=\lambda, x_2=\lambda, y_1=a, W_1=V_1$, ve $y_2=b$. İkinci adımda, yeniden tanımlıyoruz $V_1=ab$, diğer her şeyi aynı bırakıyoruz. Eğer buna devam ederseniz, yakında bu özel Post sisteminin ürettiği dilin $\{a^n b^n : n \geq 0\}$ olduğunu kendinize hızlıca kanıtlayacaksınız.

ÖRNEK 13.6

Post sistemini düşünelim

$$\begin{aligned} C_T &= \{1, +, =\}, \\ C_N &= \emptyset, \\ V &= \{V_1, V_2, V_3\}, \\ A &= \{1 + 1 = 11\}, \end{aligned}$$

ve üretimler

$$\begin{aligned} V_1 + V_2 &= V_3 \rightarrow V_1 1 + V_2 = V_3 1, \\ V_1 + V_2 &= V_3 \rightarrow V_1 + V_2 1 = V_3 1. \end{aligned}$$

Sistem, türetim yapılmasına olanak tanır

$$\begin{aligned} 1 + 1 &= 11 \Rightarrow 11 + 1 = 111 \\ &\Rightarrow 11 + 11 = 1111. \end{aligned}$$

1'lerin dizilerini tam sayıların tekil temsilleri olarak yorumlayarak, türetim şu şekilde yazılabilir

$$1 + 1 = 2 \Rightarrow 2 + 1 = 3 \Rightarrow 2 + 2 = 4.$$

Bu Post sisteminin ürettiği dil, $1 + 1 = 2$ aksiyomundan türetilen $2 + 2 = 4$ gibi tam sayı toplama işlemlerinin tüm kimliklerinin kümesidir.

Örnek 13.6, Post sistemlerinin matematiksel ifadeleri bir aksiyonlar kümesi nden titizlikle kanıtlamak için bir mekanizma olarak orijinal amacını basit bir şekilde göstermektedir. Ayrıca, böyle tamamen titiz bir yaklaşımın doğasında var olan garipliği ve neden nadiren kullanıldığını da göstermektedir. Ancak Post sistemleri, karmaşık teoremleri kanıtlamak için hantal olsalar da, bir sonraki teoremin gösterdiği gibi, hesaplama için genel modellerdir.

TEOREM 13.6

Bir dil, yalnızca bazı Post sistemlerinin onu ürettiği durumlarda, özyinel emeli olarak sayılabilir.

Kanıt: Buradaki argümanlar nispeten basittir ve bunları kısaca özetliyoruz. Öncelikle, bir Post sisteminin türetilmesi tamamen mekanik olduğundan, bu bir Turing makinesinde gerçekleştirilebilir. Bu nedenle, bir Post sistemi tarafından üretilen herhangi bir dil, özyinelemeli olarak sayılabilir.

Karşıtını düşünersek, herhangi bir özyinelemeli olarak sayılabilir dilin, tüm üretileri aşağıdaki formda olan bazı sınırsız bir dilbilgisi G tarafından üretildiğini unutmayın.

$$x \rightarrow y,$$

with $x, y \in (VUT)^*$. Herhangi bir kısıtlamasız dilbilgisi G verildiğinde, bir Post sistemi $\Pi = (V_\Pi, C, A, P_\Pi)$ oluşturuyoruz, burada $V_\Pi = \{V_1, V_2\}$, $C_N = V$, $C_T = T$, $A = \{S\}$ ve üretimlerle

$$V_1 x V_2 \rightarrow y V_2,$$

her üretim $x \rightarrow y$ dilbilgisi için. O zaman, bir w 'nin Post sistemi Π tarafından üretilebileceğini göstermek kolaydır, eğer ve yalnızca G tarafından üretilen dilde bulunuyorsa. ■

ALISTIRMALAR

1. Post sistemleri tarafından türetilen ilk dört cümleyi bulun:

- (a) $V \rightarrow aVVb$, aksiyom $\{\lambda\}$ ile.
- (b) $aV \rightarrow aVVb$, aksiyom $\{a\}$ ile.
- (c) $aVb \rightarrow aVVb$, aksiyom $\{ab\}$ ile.

2. $\Sigma = \{a, b, c\}$ için, aşağıdaki dilleri üreten bir Post sistemi bulun:

- (a) $L(a^*b + ab^*c)$.
- (b) $L = \{ww\}$.
- (c) $L = a^n b^n c$

3. Bir Post sistemi bulun ki üretsin

$$L = \{ww^R : w \in \{a, b\}^*\}.$$

4. $\Sigma = \{a\}$ için, $\{a\}$ aksiyomu ve

üretim ile Post sistemi hangi dili kullanır?

$$V_1 \rightarrow V_1 V_1.$$

oluştur?

5. Egzersiz 4'teki Post sistemi, $\Sigma = \{a, b\}$ ile birlikte, eğer aksiyom kümesi $\{a, ab\}$ ise hangi dili üretir?6. Bir Post sistemi bulunarak, $1 * 1 = 1$ aksiyomundan başlayarak, tam sayı çarpımının kimliklerini unary notasyon kullanarak kanıtlayın.

7. Teorem 13.6'nın kanıtının detaylarını verin.

8. Post sistemi hangi dili üretir

$$V \rightarrow aVV$$

ve aksiyom kümesi $\{ab\}$?

9. Sınırlı bir Post sistemi, her üretimin $x \rightarrow y$ sağladığı, alışılmış gerekliliklerin yanı sıra, sağ ve soldaki değişkenlerin sayısının aynı olduğu, yani (13.1)'de $en=m$ olduğu ek kısıtlamayı da karşılayan bir sistemdir.

Bir Post sistemi tarafından üretilen her dil L için, L' 'yi üretmek üzere bir sınırlı Post sisteminin var olduğunu gösterin.

13.3 YENİDEN YAZIM SİSTEMLERİ

İncelediğimiz çeşitli dil bilgileri, Post sistemleriyle birçok ortak noktaya sahiptir. Hepsi, dizelerin yazıldığı bir alfabe ve bir dizinin diğerinden elde edilmesini sağlayan bazı kurallara dayanır.

Bir Turing makinesi de bu şekilde görülebilir, çünkü anlık tanımı, konfigürasyonunu tamamen tanımlayan bir dizidir. Program, bir dizden bir diğerini üretmek için kurallar kümesi olarak düşünülebilir. Bu gözlemler, bir yeniden yazım sistemi kavramında biçimlendirilebilir. Genel olarak, bir yeniden yazım sistemi, bir dizi üretim veya kural ile birlikte bir alfabeden Σ oluşur ve Σ^+ içindeki bir dize başka bir dize üretebilir. Bir yeniden yazım sistemini diğerinden ayıran, Σ 'nin doğası ve üretimlerin uygulanması için kısıtlamalardır.

Fikir oldukça geniştir ve daha önce karşılaştığımız durumların yanı sıra herhangi bir sayıda özel duruma da izin verir. Burada, ilginç olan ve ayrıca hesaplama için genel modeller sağlayan daha az bilinen bazılarını kısaca tanıtlıyoruz. Ayrıntılar için Salomaa (1973) ve Salomaa (1985) bakınız.

Matris Dil bilgileri

Matris dil bilgileri, üretimlerin nasıl uygulanabileceği açısından daha önce incelediğimiz dil bilgilerinden (genellikle ifade yapısı dil bilgileri olarak adlandırılır) farklıdır. Matris dil bilgileri için, üretimlerin kümesi, her biri sıralı bir diziden oluşan alt kümelerden oluşur.

alt kümeler $P_1, P_2, \dots, P_n$, her biri sıralı bir dizidir.

$$x_1 \rightarrow y_1, x_2 \rightarrow y_2, \dots$$

Bir setin ilk üretimi P_i uygulandığında, bir sonraki olarak oluşturulan dizeye ikinci üretimi uygulamalıyız, ardından üçüncü üretimi ve devamını. Bu setin içindeki diğer tüm üretimler de uygulanmadıkça, P_i 'nin ilk üretimini uygulayamayız.

ÖRNEK 13.7

Matris dilbilgisini düşünün

$$\begin{aligned} P_1 : S &\rightarrow S_1 S_2, \\ P_2 : S_1 &\rightarrow a S_1, S_2 \rightarrow b S_2 c, \\ P_3 : S_1 &\rightarrow \lambda, S_2 \rightarrow \lambda. \end{aligned}$$

Bu gramerle bir türetim

$$S \Rightarrow S_1 S_2 \Rightarrow a S_1 b S_2 c \Rightarrow a a S_1 b b S_2 c c \Rightarrow a a b b c c.$$

Not edin ki, P_2 'nin birinci kuralı bir a oluşturmak için kullanıldığında, ikinci kuralın da kullanılması gerekir; bu da karşılık gelen bir b ve c üretir. Bu, bu matris grameri tarafından üretilen terminal dizelerin kümesinin kolayca görülebilmesini sağlar.

$$L = \{a^n b^n c^n : n \geq 0\}.$$

Matris gramerleri, her bir P_i 'nin tam olarak bir üretim içerdiği özel bir durum olarak ifade yapısı gramerlerini içerir. Ayrıca, matris gramerleri algoritmik süreçleri temsil ettiğinden, Church'un tezi tarafından yönetilmektedir. Bundan, matris gramerleri ve ifade yapısı gramerlerinin hesaplama modelleri olarak aynı güce sahip olduğunu sonuçlandırıyoruz. Ancak, Örnek 13.7'nin gösterdiği gibi, bazen bir matris gramerinin kullanımı, sınırsız bir ifade yapısı grameri ile elde edebileceğimizden çok daha basit bir çözüm sunar.

Markov Algoritmaları

Bir Markov algoritması, üretimlerinin sıralı olarak kabul edildiği bir yeniden yazım sistemidir.

$$x \rightarrow y$$

Bir türetimde, ilk uygulanabilir üretim kullanılmalıdır. Ayrıca, alt dizedeki en soldaki x bulunması gereken yer y ile değiştirilmelidir.

Bazı üretimler terminal üretimler olarak belirlenebilir; bunlar

$$x \rightarrow \cdot y.$$

Bir türetim, bazı dize $w \in \Sigma$ ile başlar ve ya bir terminal üretim kullanılır dışında ya da uygulanabilir üretim kalmadığında devam eder.

Dil kabulü için, terminal bir dize ile başlayarak, boş dize üretilene kadar üretimler uygulanır.

DEFINITION 13.5

Let M be a Markov algorithm with alphabet Σ and terminals T . Then the set

$$L(M) = \left\{ w \in T^* : w \xrightarrow{*} \lambda \right\}$$

is the language accepted by M .

EXAMPLE 13.8

Consider the Markov algorithm with $\Sigma = T = \{a, b\}$ and productions

$$\begin{aligned} ab &\rightarrow \lambda, \\ ba &\rightarrow \lambda. \end{aligned}$$

Every step in the derivation annihilates a substring ab or ba , so

$$L(M) = \left\{ w \in \{a, b\}^* : n_a(w) = n_b(w) \right\}.$$

EXAMPLE 13.9

Find a Markov algorithm for

$$L = \{a^n b^n : n \geq 0\}.$$

An answer is

$$\begin{aligned} ab &\rightarrow S, \\ aSb &\rightarrow S, \\ S &\rightarrow \cdot \lambda. \end{aligned}$$

Son örnekte, ilk iki üretimi alıp sol ve sağ tarafları tersine çevirirsek, $\text{dil } L$ üreten bir bağlamdan bağımsız gramer elde ederiz. Belirli bir anlamda, Markov algoritmaları, geriye doğru çalışan ifadeleri yapısı gramerleridir. Bu, son üretimle ne yapılacağı belirsiz olduğundan çok da harfi harfine alınmamalıdır. Ancak bu gözlem, Markov algoritmalarının gücünü tanımlayan aşağıdaki teoremin kanıtı için bir başlangıç noktası sağlar.

TEOREM 13.7

Bir dil, yalnızca bir Markov algoritması varsa, yinelemeli olarak sayılabilir.

Kanıt: Salomaa (1985, s. 35) için bakınız. ■

L-Sistemleri

L-sistemlerinin kökenleri, beklediğimizden oldukça farklıdır.

Geliştiricisi A. Lindenmayer, bunları belirli organizmaların büyüme desenlerini modellemek için kullandı. L-sistemleri esasen paralel yeniden yazım sistemleridir.

Bu, bir türetim adımında her sembolün yeniden yazılması gerektiği anlamına gelir. Bunun mantıklı olması için, bir L-sisteminin üretimleri aşağıdaki biçimde olmalıdır

$$a \rightarrow u, \quad (13.2)$$

burada $a \in \Sigma$ ve $u \in \Sigma^*$. Bir dize yeniden yazıldığında, yeni dize üretilmeden önce dizenin her sembolüne böyle bir üretim uygulanmalıdır.

ÖRNEK 13.10

$\Sigma = \{a\}$ olarak alalım ve

$$a \rightarrow aa$$

bir L-sistemi tanımlayalım. Dize a ile başlayarak, türetimi yapılabiliriz

$$a \Rightarrow aa \Rightarrow aaaa \Rightarrow aaaaaaaaa.$$

Bu şekilde türetilen dize kümesi açıkça

$$L = \{a^{2^n} : n \geq 0\}.$$

L-sistemlerinin (13.2) biçimindeki üretimlerle yeterince genel olmadığı bilinmektedir. Tüm algoritmik hesaplamalar için gerekli genelleştirme yi sağlayan bir genişletme fikri vardır. Genişletilmiş bir L-sisteminde, üretimler aşağıdaki biçimdedir

$$(x, a, y) \rightarrow u,$$

burada $a \in \Sigma$ ve $x, y, u \in \Sigma^*$, burada a yalnızca xay dizisinin bir parçası olarak bulunduğu u ile değiştirilebilir. Böyle genişletilmiş L-sistemlerinin genel hesaplama modelleri olduğu bilinmektedir. Ayrıntılar için Salomaa (1985) bakınız.

ALISTIRMALAR

1. Bir matris grameri bulun

$$L = \{a^n b^b c^2 : n \geq 0\}.$$

2. Bir matris grameri bulun

$$L = \{ww : w \in \{a, b\}^*\}.$$

3. Matris grameri tarafından hangi dil üretiliyor

$$\begin{aligned} P_1 : S &\rightarrow S_1 S_2, \\ P_2 : S_1 &\rightarrow a S_1 b, S_2 \rightarrow b S_2 a, \\ P_3 : S_1 &\rightarrow \lambda, S_2 \rightarrow \lambda? \end{aligned}$$

4. Diyelim ki Örnek 13.7'de son üretim grubunu değiştiriyoruz

$$P_3 : S_1 \rightarrow \lambda, S_2 \rightarrow S.$$

Bu matris dilini hangi dil oluşturur?

5. Örnek 13.9'daki Markov algoritması neden $abab^*$ 'i kabul etmez?

6. Dili türeten bir Markov algoritması bulun $L = \{a^n b^n c^n : n \geq 1\}$.

7. Kabul eden bir Markov algoritması bulun

$$L = \{a^n b^m a^{nm} : n \geq 1, m \geq 1\}.$$

8. Üreten bir L-sistemi bulun $L(aa^*)$.

9. için bir L-gramer bulun

$$L = \{a^n b^b c^n : n \geq 0\}.$$

İpucu: Genişletilmiş bir L-gramer kullanmanız gerekebilir.

BÖLÜM

14

BİLGİSAYAR KOMPLEKSİTESİNE GENEL BAKIŞ

BÖLÜM ÖZETİ

Daha önceki Turing makineleri tartışmalarında mesele yalnızca bir şeyin prensipte yapılıp yapılamayacağıydı, burada odak pratikte şeylerin ne kadar iyi yapılabileceğine kayıyor. Bu son bölümde, hesaplama teorisinin ikinci kısmını tanıtıyoruz: karmaşıklık teorisi, hesaplamanın verimliliği ile ilgilenen kapsamlı bir konudur. Kompleksite teorisindeki bazı tipik sorunları kısaca tanımlıyoruz, bunlar arasında

belirsizlik (nondeterminism) rolü de bulunmaktadır.

Şimdi hesaplama karmaşıklığını, algoritmaların verimliliğini incelemeyi yeniden ele alıyoruz. Karmaşıklık, Bölüm 11'de kısaca bahsedildiği gibi, pratikte çözülebilen problemleri, prensipte çözülebilenlerden ayırarak hesaplanabilirliği tamamlar.

Karmaşıklığı incelerken, donanım, yazılım, veri yapıları ve uygulama gibi birçok detayı göz ardı etmekve ortak, temel sorunlara odaklanmak gereklidir. Bu nedenle, genellikle büyüklük sıraları ifadeleriyle çalışıyoruz. Ancak, göreceğimiz gibi, böyle çok yüksek seviyeli bir bakış açısı bile çok faydalı sonuçlar verir.

Verimlilik, zaman ve alan gibi kaynak gereksinimleri ile ölçülür, bu nedenle zaman karmaşıklığı ve alan karmaşıklığı hakkında konuşabiliriz. Burada zaman karmaşıklığı ile sınırlı kalacağız, bu da belirli bir hesaplamanın aldığı time ölçüsüdür. Bununla ilgili birçok sonuç vardır.

Aynı zamanda alan karmaşıklığı ile ilgili de birçok sonuç vardır, ancak zaman karmaşıklığı biraz daha erişilebilir ve aynı zamanda daha kullanışlıdır.

Hesaplama karmaşıklığı geniş bir konudur, çoğu bu metnin kapsamının çok ötesindedir. Ancak, basitçe ifade edilen ve kolayca takdir edilen bazı sonuçlar vardır ve bu da dillerin ve hesaplamanın doğası hakkında daha fazla ışık tutar. Bu bölümde, karmaşıklıkta en belirgin sonuçların kısa bir özetini sunuyoruz. Birçok kanıt zordur ve bunları uygun kaynaklara atıfta bulunarak geçiştireceğiz. Buradaki amacımız, konunun özünü sunmak ve detaylara takılmamaktır. Bu nedenle, hem konuyu seçiminde hem de tartışmanın resmîyetinde büyük bir esneklik ta niyacağız.

14.1 HESAPLAMADA VERİMLİLİK

Somut bir örnekle başlayalım. Bin tam sayının bir listesini verelim, bu sayıları bir şekilde, diyelim ki artan sırayla sıralamak istiyoruz. Sıralama, basit bir problem olmasına rağmen bilgisayar biliminde çok temel bir problemdir. Eğer şimdi “Bu görevi yapmak ne kadar zaman alacak?” sorusunu sorarsak, bunun yanıtını verebilmek için çok daha fazla bilgiye ihtiyaç duyduğumuzu hemen görürüz. Açıkça, listedeki öğe sayısı, ne kadar zaman alacağı üzerinde önemli bir rol oynamaktadır, bununla birlikte başka faktörler de vardır. Hangi bilgisayarı kullandığımız ve programı nasıl yazdığımız sorusu da vardır. Ayrıca, birçok sıralama yöntemi bulunmaktadır, bu nedenle algoritmanın seçimi önemlidir. Bu zaman gereksinimlerinin kaba bir tahminini yapmadan önce, dikkate alınması gereken birkaç şey daha düşünebilirsiniz. Eğer sıralamanın genel bir resmini oluşturma umudumuz varsa, bu sorunların çoğunu göz ardı etmemiz gerekiyor ve temel olanlara odaklanmalıyız.

Hesaplama karmaşıklığı tartışmamız için aşağıdaki basitleştirici varsayımları yapacağız.

1. Çalışmamız için modelimiz bir Turing makinesi olacaktır. Kullanılacak Turing makinesinin kesin türü aşağıda tartışılacaktır.
2. Problemin boyutu n ile gösterilecektir. Sıralama problemimiz için, n liste içindeki öğe sayısıdır. Bir problemin boyutu her zaman bu kadar kolay tanımlanamaz, ancak genel olarak pozitif bir tam sayıya bir şekilde ilişkilendirebiliriz.
3. Bir algoritmayı analiz ederken, belirli bir durumdaki performansın çok genel davranışına daha az ilgi duyarız. Özellikle, algoritmanın problem boyutu arttıkça nasıl davrandığıyla ilgileniyoruz. Bu nedenle, ana soru, kaynak gereksinimlerinin n büyüdükçe ne kadar hızlı arttığıdır.

Hedefimiz, bir problemi boyutuna bağlı olarak zaman gereksinimi olarak tanımlamak olacak ve bir Turing makinesini bilgisayar modeli olarak kullanacağız.

Öncelikle, bir Turing makinesi için zaman kavramına biraz anlam katıyoruz. Bir Turing makinesinin her zaman bir hareket yaptığını düşünürüz, bu nedenle bir hesaplamanın aldığı zaman, yapılan hareketlerin sayısıdır. Belirttiğimiz gibi, problemin boyutuyla birlikte hesaplama gereksinimlerinin nasıl büyüdüğünü incelemek istiyoruz. Normalde, belirli bir boyuttaki tüm problemler setinde bazı varyasyonlar vardır. Burada yalnızca en yüksek kaynak gereksinimlerine sahip en kötü durum ile ilgileniyoruz. Bir hesaplamanın zaman karmaşıklığı $T(n)$ olduğunu söyleyerek, boyut n olan herhangi bir problemin, bazı Turing makinelerinde en fazla $T(n)$ hareketle tamamlanabileceğini kastediyoruz.

Belirli bir tür Turing makinesi olarak hesaplama modeline karar verdikten sonra, algoritmaları açık programlar yazarak analiz edebiliriz ve problemi çözmek için gereken adım sayısını sayabiliriz. Ancak, çeşitli nedenlerden dolayı bu pek kârlı değildir. Öncelikle, gerçekleştirilen işlem sayısı programın küçük detaylarına bağlı olarak değişebilir ve bu da programcıya güçlü bir şekilde bağlı olabilir. İkincisi, pratik bir açıdan, algoritmanın gerçek dünyada nasıl performans gösterdiğiyle ilgileniyoruz, bu da Turing makinesinde gösterdiğinden önemli ölçüde farklı olabilir. Umabileceğimiz en iyi şey, Turing makinesi analizinin gerçek yaşam performansının ana yönlerini temsil etmesidir; örneğin, zaman karmaşıklığının asimptotik büyüme oranı. Bu nedenle, bir algoritmanın kaynak gereksinimlerini anlamaya yönelik ilk girişimimiz, Bölüm 1'de tanıtılan O , Θ ve Ω notasyonunu kullandığımız bir büyüklük sırası analizidir. Bu yaklaşımın görünürdeki resmi olmamasına rağmen, genellikle çok faydalı bilgiler elde ederiz.

ÖRNEK 14.1

Bir dizi n sayısı $x_1, x_2, \dots, x_n$ ve bir anahtar sayı x verildiğinde, kümenin x içerip içermediğini belirleyin.

Bir küme belirli bir şekilde düzenlenmedikçe, en basit algoritma sadece bir lineer arama olup, burada x 'yi sırasıyla $x_1, x_2, \dots$, karşılaştırırız veya bir eşleşme bulana kadar ya da kümenin son elemanına ulaşana kadar devam ederiz. İlk karşılaştırmada veya son karşılaştırmada bir eşleşme bulabileceğimiz için, ne kadar işin gerektiğini tahmin edemeyiz, ancak en kötü durumda n karşılaştırma yapmak zorunda olduğumuzu biliyoruz. Bu durumda, bu lineer aramanın zaman karmaşıklığının $O(n)$ olduğunu veya daha da iyisi, $\Theta(n)$ olduğunu söyleyebiliriz. Bu analizi yaparken, hangi makinede çalıştığına veya algoritmanın nasıl uygulandığına dair özel varsayımlar yapmadık. Ancak, eğer sayı kümesinin boyutunu iki katına çıkarırsa k , hesaplama süresinin y

aklaşık olarak iki katına çıkacağı anlamına geliyor. Bu, arama hakkında bize çok şey söylüyor.

ALISTIRMALAR

1. Farz edelim ki, size bir kümenin sayısı $x_1, x_2, \dots, x_n$ verildi ve bu kümenin herhangi bir kopya içerip içermediğini belirlemeniz istendi.
 - (a) Bir algoritma önerin ve zaman karmaşıklığı için bir büyüklük sırası ifadesi bulun.
 - (b) Algoritmanın bir Turing makinesinde uygulanmasının sonuçlarınızı etkileyip etkilemediğini inceleyin.
2. Egzersiz 1'i tekrarlayın, bu sefer kümenin herhangi bir üçlü içerip içermediğini belirleyin. Algoritma mümkün olduğunca verimli mi?
3. Algoritma seçiminin sıralamanın verimliliğini nasıl etkilediğini gözden geçirin. En verimli sıralama algoritmalarının zaman karmaşıklığı nedir?

14.2 TURING MAKİNESİ MODELLERİ VE KARMANLIK

Hesaplanabilirlik çalışmasında, hangi belirli Turing makinesi modelini kullandığımızın pek önemi yoktur, ancak bir hesaplamanın verimliliğinin makinenin bant sayısı ve deterministik veya nondeterministik olup olmadığı tarafından etkilenebileceğini zaten gördük. Örnek 12.8'de gösterildiği gibi, nondeterministik çözümler genellikle deterministik alternatiflerden çok daha verimlidir. Bir sonraki örnek bunu daha da net bir şekilde göstermektedir.

ÖRNEK 14.2

Şimdi, karmaşıklık teorisinde önemli bir rol oynayan tatmin edilebilirlik problemini (SAT) tanıtıyoruz.

Bir mantık veya boolean sabiti veya değişkeni, tam olarak iki değer alabilen, doğru veya yanlış olan bir değerdir; bunları sırasıyla 1 ve 0 ile gösteririz. Boolean operatörleri, boolean sabitlerini ve değişkenlerini boolean ifadelerine birleştirmek için kullanılır. En basit boolean operatörleri veya, $\vee$ ile gösterilir ve şöyle tanımlanır:

$$0 \vee 1 = 1 \vee 0 = 1 \vee 1 = 1,$$

$$0 \vee 0 = 0,$$

ve ve operatörü ($\wedge$), şöyle tanımlanır:

$$0 \wedge 0 = 0 \wedge 1 = 1 \wedge 0 = 0,$$

$$1 \wedge 1 = 1.$$

Ayrıca bir çubuğun gösterdiği inkar gereklidir ve bu şöyle tanımlanır:

$$\begin{aligned}\bar{0} &= 1, \\ \bar{1} &= 0.\end{aligned}$$

Şimdi konjunktif normal formda

(CNF) boolean ifadelerini ele alıyoruz. Bu formda, ifadeleri değişkenlerden $x_1, x_2, \dots, x_n$ oluşturarak yapıyoruz, şu şekilde başlayarak

$$e = t_i \wedge t_j \wedge \dots \wedge t_k. \quad (14.1)$$

Terimler $t_i, t_j, \dots, t_k$, değişkenleri ve onların

inkarını birleştirerek oluşturulur, yani,

$$t_i = s_l \vee s_m \vee \dots \vee s_p, \quad (14.2)$$

her $s_l, s_m, \dots, s_p$ bir değişkeni veya bir değişkenin inkarını temsil eder. s_i , 'literal' olarak adlandırılacakken, t_i 'clause' olarak adlandırılır bir CNF ifadesi e .

Sağlanabilirlik problemi basitçe şöyle ifade edilir: Sağlanabilir bir ifade e verildiğinde, değişkenlere $x_1, x_2, \dots, x_n$ değerleri atayarak e değerini doğru yapacak bir atama bulun. belirli durum, bak

$$e_1 = (\bar{x}_1 \vee x_2) \wedge (x_1 \vee x_3).$$

Atama $x_1 = 0, x_2 = 1, x_3 = 1$, e_1 'yi doğru yapar, böylece bu ifade tatmin edilebilir. Öte yandan,

$$e_2 = (x_1 \vee x_2) \wedge \bar{x}_1 \wedge \bar{x}_2 \quad (14.3)$$

tatmin edilemez çünkü değişkenler için her atama x_1 ve x_2 e_2 'yi yanlış yapar.

Tatmin edilebilirlik problemi için deterministik bir algoritma keşfetmek kolaydır.

Değişkenler için tüm olası değerleri alırız $x_1, x_2, \dots, x_n$ ve her biri için ifadeyi değerlendiririz. Çünkü 2^n böyle olasılık vardır, bu kapsamlı yaklaşımın üstel zaman karmaşıklığı vardır.

Yine, belirsiz alternatif işleri basitleştirir. Eğer e tatmin edilebilir ise, her x_i değerini tahmin ederiz ve ardından e 'yi değerlendiririz. Bu esas itibarıyla bir $O(n)$ algoritmasıdır. Örnek 12.8'de olduğu gibi, karmaşıklığı üstel olan bir belirleyici kapsamlı arama algoritmamız ve lineer zamanlı bir belirsiz algoritmamız vardır. Ancak, Örnek 12.8'den farklı olarak, üstel olmayan herhangi bir belirleyici algoritma bildiğimiz yoktur.

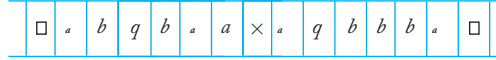
Bu örnek ve Örnekler 12.7 ve 12.8, karmaşıklık sorularının kullandığımız Turing makinesinin türünden etkilendiğini ve belirleyicilik ile belirsizlik meselesinin özellikle kritik bir konu olduğunu önermektedir. Bu gözlemlerle tutarlı bazı genel sonuçlar çıkarılabilir.

TEOREM 14.1

İki bantlı bir makinenin n adımda bir hesaplama gerçekleştirebileceğini varsayalım. O zaman bu hesaplama, standart bir Turing makinesi tarafından

$O(n^2)$ adımda simüle edilebilir.

Kanıt: İki bantlı makinedeki hesaplamanın simülasyonu için, standart makine iki bantlı makinenin anlık tanımını kendi bantında tutar, Şekil 14.1' de gösterildiği gibi. Bir hareketi simüle etmek için, standart makinenin bantının tüm aktif alanını araması gerekir. Ancak, iki bantlı makinenin bir hareketi aktif alanı en fazla iki hücre kadar genişletebileceğinden, n hareketten sonra aktif alan en fazla $O(n)$ uzunluğuna sahip olur. Bu nedenle, tüm simülasyon $O(n^2)$ hareketle gerçekleştirilebilir.



ŞEKİL 14.1

Bu sonuç, iki banttan daha fazlasına kolayca genişletilebilir (Bu bölümün sonunda Egzersiz 6). ■

TEOREM 14.2

Diyelim ki, bir belirsiz Turing makinesi M_n adımda bir hesaplama gerçekleştirebilir. O zaman, standart bir Turing makinesi aynı hesaplamayı $O(k^{an})$ adımda gerçekleştirebilir; burada k ve a 'ye bağımsızdır.

Kanıt: Standart bir Turing makinesi, tüm olası yapılandırmaları takip ederek, sürekli olarak arama yaparak ve tüm aktif alanı güncelleyerek belirsiz bir makineyi simüle edebilir. Eğer k belirsizlik için maksimum dallanma faktörü ise, o zaman n adımından sonra en fazla k^n olası yapılandırma vardır. Her yapılandırmaya tek bir hareketle en fazla bir sembol eklenebileceğinden, n hareketten sonra bir yapılandırmanın uzunluğu $O(n)$ olur.

Bu nedenle, bir hamleyi simüle etmek için standart makinenin, istenen sonuca ulaşmak için $O(k^n)$ uzunluğunda bir aktif alanı araması gerekir. Bazı detaylar için, bu bölümün sonunda yer alan Egzersiz 7'ye bakın. ■

Bu teoremi dikkatlice yorumlamalıyız. Bu, bir belirsiz hesaplamanın, gerekli olan zamanın üstel bir artışını dikkate almayı göze alıyorsak, her zaman deterministik bir makinede gerçekleştirilebileceğini söyler.

Ancak bu sonuç, özellikle basit bir simülasyondan gelmektedir ve daha iyi sonuçlar elde etme umudu hâlâ vardır. Bu konunun araştırılması,

karmaşıklık teorisinin özüdür.

Örnek 12.7, çoklu bant makinesi için algoritmaların, standart Turing makinesi için kullanılan hantal yöntemden daha pratik olabileceğini öneriyor. Bu nedenle, karmaşıklık sorunlarını incelemek için bir çoklu bant Turing makinesini model olarak kullanacağız, ancak göreceğimiz gibi, bu küçük bir meseledir.

ALISTIRMALAR

1. İki bantlı bir Turing makinesi kullanarak $\{ww : w \in \{a, b\}^*\}$ kümesinin üyeliği için lineer zamanlı bir algoritma bulun. Tek bantlı bir makinede en iyi neyi bekleyebilirsiniz?
2. Tek bantlı, çevrimdışı bir Turing makinesinde zaman $O(T(n))$ içinde gerçekleştirilebilen herhangi bir hesaplamanın, standart bir Turing makinesinde de zaman $O(T(n))$ içinde gerçekleştirilebileceğini gösterin.
3. Standart bir Turing makinesinde zaman $O(T(n))$ içinde gerçekleştirilebilen herhangi bir hesaplamanın, bir yarı sonsuz bantlı Turing makinesinde de zaman $O(T(n))$ içinde gerçekleştirilebileceğini gösterin.
4. Boolean ifadesini

$$(x_1 \wedge x_2) \vee x_3$$

bağlayıcı normal formda yeniden yazın.

5. İfadenin

$$(x_1 \vee \bar{x}_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \bar{x}_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_3)$$

tatmin edilebilir olup olmadığını belirleyin.

6. Teorem 14.1'i k bantlar için genelleştirerek, n hareketin bir k bantlı makinede standart bir makinede $O(n^2)$ hareketle simüle edilebileceğini gösterin.
7. Teorem 14.2'nin kanıtında bir ince noktayı göz ardı ettik. Bir yapılandırma büyüdüğünde, şeridin geri kalan içeriği taşınmak zorundadır. Bu gözden kaçırma sonucu etkiler mi?

14.3 DİL AİLELERİ VE KARMANIZ SINIFLARI

Chomsky hiyerarşisinde dil sınıflandırması için, dil ailelerini otomata sınıflarıyla ilişkilendiriyoruz, burada her otomata sınıfı geçici depolama doğasıyla tanımlanır. Dilleri sınıflandırmanın bir diğer olasılığı, bir Turing makinesi kullanmak ve zaman karmaşıklığını ayırt edici bir faktör olarak dikkate almaktır. Bunu yapmak için, bir dilin zaman karmaşıklığını önce tanımlıyoruz.

TANIM 14.1

Bir Turing makinesi M bir dili L zaman $T(n)$ içinde karar veriyorsa, her $w \in L$ içinde $|w| = n$ olan, $T(n)$ hareketinde karar verilir. Eğer M belirsizse, bu, her $w \in L$ için, kabul etmeye götüren uzunluğu $T(|w|)$ olan en az bir hareket dizisi olduğu anlamına gelir ve Turing makinesi tüm girdilerde zaman $T(|w|)$ içinde durur.

TANIM 14.2

Bir dil L DTIME $(T(n))$ sınıfının bir üyesi olarak kabul edilir, eğer L 'yi zaman içinde karar veren bir deterministik çok bantlı Turing makinesi varsa. $O(T(n))$.

Bir dil L , eğer NTIME $(T(n))$ sınıfının bir üyesi olarak kabul edilir, eğer bir nondeterministik çok bantlı Turing makinesi L 'yi $O(T(n))$ zamanında karar verabiliyorsa.

Bu karmaşıklık sınıfları arasındaki bazı ilişkiler, örneğin

$$DTIME(T(n)) \subseteq NTIME(T(n)),$$

ve

$$T_1(n) = O(T_2(n))$$

anlamına gelir

$$DTIME(T_1(n)) \subseteq DTIME(T_2(n)),$$

belirgin, ancak buradan itibaren durum hızla belirsizleşiyor. Söyleyebileceğimiz şey, $T(n)$ sırası arttıkça, giderek daha fazla dil aldığımızdır.

TEOREM 14.3

Her tam sayı için $k \geq 1$,

$$DTIME(n^k) \subset DTIME(n^{k+1}).$$

Kanıt: Bu, Hopcroft ve Ullman'daki (1979, s. 299) bir sonuçtan gelmektedir. ■

Buradan çıkarabileceğimiz sonuç, zamanında karar verilebilen bazı dillerin $O(n^2)$ için doğrusal zamanlı bir üyelik algoritması olmayan diller olduğudur, ayrıca $DTIME(n^3)$ içinde olmayan diller de vardır $DTIME(n^2)$, ve benzeri. Bu, bize sonsuz sayıda iç içe geçmiş karmaşıklık sınıfı verir. Eğer üstel zaman karmaşıklığına izin verirse, daha da fazlasını elde ederiz. Aslında, bunun bir sınırı yoktur; karmaşıklık fonksiyonu $T(n)$ ne kadar hızlı büyürse büyüsün, her zaman $DTIME(T(n))$ dışında bir şey vardır.

TEOREM 14.4

Her rekürsif dilin zaman $f(n)$ içinde karar verilebileceği, toplam Turing hesaplanabilir bir fonksiyon $f(n)$ olmadığıdır; buradan, girdi dizisinin uzunluğu dur.

Kanıt: Alfabe $\Sigma = \{0,1\}$ ile, Σ^+ içindeki tüm dizeler uygun sırada $w_1, w_2, \dots$ düzenlenmiştir. Ayrıca, Turing makineleri için $M_1, M_2, \dots$ uygun bir sıralama olduğunu varsayalım.

Şimdi teoremin ifadesindeki fonksiyon $f(n)$ var olduğunu varsayalım. O zaman dili tanımlayabiliriz

$$L = \{w_i : M_i \text{ does not decide } w_i \text{ in } f(|w_i|) \text{ steps}\}. \quad (14.4)$$

Biz L 'nin rekürsif olduğunu iddia ediyoruz. Bunu görmek için, herhangi bir $w \in L$ alalım ve önce $f(|w|)$ hesaplayalım. Fonksiyonun toplam Turing hesaplanabilir bir fonksiyon olduğunu varsayarak, bu mümkündür. Ardından, dizide $w_1, w_2, \dots$ dizisinin konumunu i buluyoruz. Bu da mümkündür çünkü dizi uygun sıradadır. Elimizde olduğunda, M_i buluyoruz ve w üzerinde $f(|w|)$ adım çalıştırmasına izin veriyoruz. Bu, w 'nin L içinde olup olmadığını bize söyleyecektir, dolayısıyla rekürsiftir.

Ancak şimdi L 'nin zaman $f(n)$ içinde karar verilemeyeceğini gösterebiliriz. Farz edelim ki öyle. Çünkü L rekürsiftir, bazı M_k vardır, böylece $L = L(M_k)$. Peki $w_k \in L$ içinde mi? Eğer $w_k \in L$ içinde olduğunu iddia edersek, o zaman M_k 'nin $f(|w_k|)$ adımda karar verir. Ancak bu (14.4) ile çelişiyor. Tersine, eğer $w_k \notin L$ varsayımını kabul edersek bir çelişki elde ederiz. Bu sorunu çözme yeteneğinin olmaması tipik bir diyagonalizasyon sonucudur ve bizi orijinal varsayımın, yani hesaplanabilir bir $f(n)$ varlığının yanlış olması gerektiği sonucuna götürür. ■

Teorem 14.3, örneğin, $DTIME(n^4)$ içinde yer alan ve $DTIME(n^3)$ içinde yer almayan bir dilin var olduğunu iddia etmemize olanak tanır. Bu teorem olarak ilginç olsa da, böyle bir sonucun pratik bir önemi olup olmadığı belirsizdir. Bu noktada, $DTIME(n^4)$ içinde yer alan bir dilin özelliklerinin ne olabileceğine dair hiçbir fikrimiz yok. Konuya biraz daha fazla iç içe örnek kazanabiliriz eğer karmaşıklık sınıflandırmasını Chomsky hiyerarşisi içindeki dillere ilişkilendirirsek. Daha belirgin sonuçlar veren bazı basit örneklerle bakacağız.

ÖRNEK 14.3

Her düzenli dil, girdi uzunluğuna orantılı bir zamanda belirleyici s onlu bir otomaton tarafından tanınabilir. Bu nedenle,

$$L_{REG} \subseteq DTIME(n).$$

Ancak $DTIME(n)$ çok daha fazlasını L_{REG} içermektedir. We have al- Örnek 13.7'de bağlamdan bağımsız dilin hazır kurulduğu $\{a^n b^n : n \geq 0\}$ iki bantlı bir makine tarafından $O(n)$ zamanında tanınabilir. Orada verilen argüman, daha karmaşık diller için de kullanılabilir.

ÖRNEK 14.4

Bağlamdan bağımsız olmayan dil $L = \{ww : w \in \{a, b\}^*\}$ non-NTIME(n) içindedir. Bu açıktır, çünkü bu dildeki dizeleri aşağıdaki algoritma ile tanıyabiliriz:

1. Girişi giriş dosyasından bant 1'e kopyalayın. Bu dizinin ortasını b elirsiz bir şekilde tahmin edin.
2. İkinci kısmı bant 2'ye kopyalayın.
3. Bant 1 ve bant 2'deki sembolleri birer birer karşılaştırın.

Açıkça, tüm adımlar $O(|w|)$ zamanında yapılabilir, bu nedenle $L \in NTIME(n)$.

Aslında, bir dizinin ortasını bulmak için $O(n)$ zamanında bir algoritma geliştirebilirsek, $L \in DTIME(n)$ olduğunu gösterebiliriz. Bu şöyle yapılabilir: Bant 1'deki her sembole bakarız, bant 2'de bir sayım tutarız, ancak yalnızca her ikinci sembolü sayarız. Ayrıntıları bir alıştırma olarak bırakıyoruz.

ÖRNEK 14.5

Örnek 12.8'den şu sonuç çıkar:

$$L_{CF} \subseteq DTIME(n^3)$$

ve

$$L_{CF} \subseteq NTIME(n).$$

Şimdi bağlam duyarlı diller ailesini düşünelim. Burada da kapsamlı arama ayrıştırması mümkündür çünkü her adımda yalnızca sınırlı sayıda üretim uygulanabilir. Denklem (5.2)'ye götüren analizi takip ederek, maksimum cümle biçimlerinin sayısını görüyoruz:

$$N = |P| + |P|^2 + \dots |P|^{cn} = O(|P|^{cn+1}).$$

Ancak, bunun üzerinden iddia edemeyeceğimizi unutmayın

$$L_{CS} \subseteq DTIME(|P|^{cn+1}),$$

çünkü $|P|$ ve c için üst bir sınır koyamıyoruz.

Bu örneklerden bir eğilim gözlemliyoruz: $T(n)$ arttıkça, daha fazla aile L_{REG}, L_{CF}, L_{CS} kapsanıyor. Ancak Chomsky hiyerarşisi ile karmaşıklık sınıfları arasındaki bağlantı zayıf ve çok net değil.

ALISTIRMALAR

1. Örnek 14.4'teki argümanı tamamlayın.
2. Şunu gösterin $L = \{ww^Rw : w \in \{a, b\}^+\} \in DTIME(n)$ içindedir.
3. Şunu gösterin $L = \{www : w \in \{a, b\}^+\} \in DTIME(n)$ içindedir.
4. $NTIME(2^n)$ içinde olmayan diller olduğunu gösterin.

14.4 Karmaşıklık Sınıfları P ve NP

Bu noktada, formel diller için yararlı karmaşıklık sınıfları bulmaya çalışırken karşılaştığımız zorlukları özetlemek ve birkaç sonuç çıkarmak öğreticidir.

1. Doğru bir şekilde iç içe geçmiş sonsuz sayıda karmaşıklık sınıfı vardır. $DTIME(n^k)$, $k = 1, 2, \dots$. Bu karmaşıklık sınıflarının tanınmış Chomsky hiyerarşisi ile pek az bağlantısı vardır ve bu sınıfların doğası hakkında herhangi bir içgörü elde etmek zor görünmektedir. Belki de bu, dilleri sınıflandırmamızın iyi bir yolu değildir.
2. Turing makinesinin belirli bir modeli, deterministik makinelerle sınırlı kalsak bile, karmaşıklığı etkiler. Gerçek bir bilgisayarın en iyi modeli hangi tür Turing makinesi olduğu net değildir, bu nedenle bir analiz herhangi bir belirli Turing makinesi türüne bağlı olmamalıdır.

3. Birçok dili, birbelirsiz Turing makinesi tarafından verimli bir şekilde karar verilebilecek şekilde bulduk. Bazıları için makul belirleyici algoritmalar da vardır, ancak diğerleri için yalnızca verimsiz, kaba kuvvet yöntemlerini biliyoruz. Bu örneklerin anlamı nedir?

Zaman karmaşıklıkları ile farklı büyüme oranlarına sahip anlamlı dil hiyerarşileri üretme girişimi verimsiz görüldüğünden, daha az önemli bazı faktörleri göz ardı edelim, örneğin, bazı ilginç olmayan ayrımları kaldırarak, $DTIME(n^k)$ ve $DTIME(n^{k+1})$ arasındaki ayrım gibi. Örneğin,

$DTIME(n)$ ve $DTIME(n^2)$ arasındaki farkın temel olmadığını savunabiliriz, çünkü bunun bir kısmı sahip olduğumuz Turing makinesinin belirli modeline bağlıdır (örneğin, kaç bant olduğu). Bu, bizi ünlü karmaşıklık sınıfını düşünmeye yönlendiriyor.

$$P = \bigcup_{i \geq 1} DTIME(n^i).$$

Bu sınıf, herhangi bir polinomun derecesine bakılmaksızın, bazı belirleyici Turing makineleri tarafından polinom zamanda kabul edilen tüm dilleri içerir. Daha önce gördüğümüz gibi, L_{REG} ve $L_{CF}P$ içindedir.

Belirleyici ve belirsiz karmaşıklık sınıfları arasındaki ayrımın temel görüldüğü için, ayrıca tanıtıyoruz

$$NP = \bigcup_{i \geq 1} NTIME(n^i).$$

Açıkça

$$P \subseteq NP,$$

Ancak bilinmeyen, bu sınırlamanın uygun olup olmadığıdır. Genel olarak, NP 'de bazı dillerin P 'de olmadığına inanılmaktadır, ancak bu konuda henüz bir örnek bulunmamıştır.

Bu karmaşıklık sınıflarına, özellikle P sınıfına olan ilgi, gerçekçi ve gerçekçi olmayan hesaplamalar arasında ayrım yapma çabasıyla kaynaklanmaktadır. Belirli hesaplamalar, teorik olarak mümkün olsalar da, pratikte mevcut bilgisayarlar ve henüz tasarlanmamış süper bilgisayarlar gerçekçi olmaya yüksek kaynak gereksinimlerine sahip olduklarından reddedilmelidir.

Böyle problemler bazen, ilkesel olarak hesaplanabilir olmalarına rağmen, pratik bir algoritmanın gerçekçi bir umudunun olmadığına işaret etmek için intractable (çözumsuz) olarak adlandırılmaktadır.

Bunu daha iyi anlamak için, bilgisayar bilimcileri intractability (çözumsuzlük) kavramını resmi bir temele oturtmaya çalışmışlardır. Intractable terimini tanımlamak için yapılan bir girişim, genellikle Cook-Karp tezi olarak adlandırılan çalışmadır. Cook-Karp tezinde, P 'de olan bir problem tractable (çözülebilir) olarak adlandırılırken, olmayan bir problem intractable (çözumsuz) olarak tanımlanmaktadır.

Cook-Karp tezi, gerçekçi bir şekilde çözebileceğimiz problemleri çözemeyeceğimiz olanlardan ayırmanın iyi bir yolu mu? Cevap net değildir. Açıkça, P 'de olmayan herhangi bir hesaplamanın zaman karmaşıklığı, daha hızlı büyümektedir.

herhangi bir polinom ve gereksinimleri problem boyutuyla çok hızlı bir şekilde artacaktır. Büyük n için, $2^{0.1n}$ gibi bir fonksiyon için bile bu aşırı olacaktır. n , say $n \geq 1000$, bu karmaşıklıkta bir problemi çözmenin imkansız olduğunu söyl emekte haklı hissedebiliriz. Ama ya DT IME(n^{100}) olan problemler ne olacak? Cook-Karp tezi böyle bir problemi çözülebilir olarak adlandırırsa da, küçük n için bile bununla pek bir şey yapılamaz. Cook-Karp tezinin gerekçesi, P'deki çoğu pratik problemin DT IME(n), DT IME(n^2) veya DT IME(n^3) içinde olduğu, bu sınıfın dışındaki problemlerin ise genellikle üstel karmaşıklıklara sahip olduğu yönünde ki ampirik gözlemde yatıyor gibi görünmektedir. Pratik problemler arasında, P'deki problemler ile P'de olmayanlar arasında belirgin bir ayrım vardır.

14.5 BİRKAÇ NP SORUNU

Bilgisayar bilimcileri birçok NP problemi, yani polinom zamanda belirsiz bir şekilde çözülebilen problemleri inceledi. Bu konuyla ilgili bazı argümanlar oldukça teknik olup, çözülmesi gereken birçok detay içermektedir.

Geleneksel olarak, karmaşıklık soruları diller olarak incelenir, öyle ki belirtilen koşulları sağlayan durumlar bazı dillerdeki dizelerle tanımlanırken, bunları sağlamayanlar L içindedir. Bu nedenle, genellikle ya pılması gereken ilk şey, problemin sezgisel anlayışını bir dil açısından yeniden ifade etmektir.

ÖRNEK 14.6

SAT problemini yeniden düşünelim. Bu problemin, bir belirsiz Turing makinesi tarafından verimli bir şekilde ve, oldukça verimsiz bir şekilde, bir kaba kuvvet üstel arama ile çözülebileceğini iddia etmek için bazı temel argümanlar sunduk. O argümanda bazı küçük noktalar göz ardı edildi.

Diyelim ki bir CNF ifadesinin uzunluğu n , m farklı literal ile birlikte. Açıktaki $m < n$ olduğundan, problemi boyut olarak n alabiliriz. Sonra, CNF ifadesini bir Turing makinesi için bir dize olarak kodlamamız gerekiyor. Bunu, örneğin, $\Sigma = \{x, \vee, \wedge, (,), -, 0, 1\}$ olarak x 'in alt simgesini ikili bir sayı olarak kodlayarak yapabiliriz. Bu sistemde, CNF ifadesi $(x_1 \vee x_2) \wedge (x_3 \vee x_4)$ şu şekilde kodlanır

$$(x1 \vee x - 10) \wedge (x11 \vee x100).$$

Alt simge m 'den büyük olamayacağından, herhangi bir alt simgenin maksimum uzunluğu $\log_2 m$ 'dir. Sonuç olarak, bir n sembolü CNF'nin maksimum kodlanmış uzunluğu $O(n \log n)$ olacaktır.

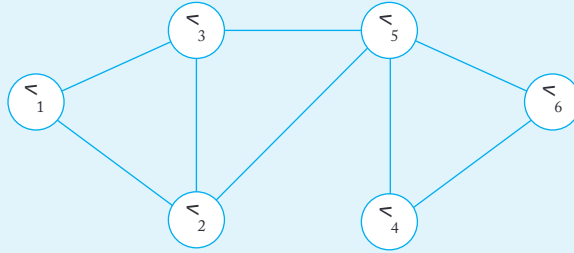
Sonraki adım, değişkenler için bir deneme çözümü üretmektir. B elirsiz bir şekilde, bu $O(n)$ zamanında yapılabilir. (Bu bölümün sonund a Egzersiz 1'e bakın.) Bu deneme çözümü daha sonra giriş dizisine ye rleştirilir. Bu, $O(n^2 \log n)$ zamanında yapılabilir*. Dolayısıyla, tüm süreç $O(n^2 \log n)$ veya $O(n^3)$ zamanında gerçekleştirilebilir ve $SAT \in NP$.

Birçok grafik problemi incelenmiş ve NP içinde olduğu bilinmekte dir.

ÖRNEK 14.7

Hamilton Yolu Problemi Verilen bir yönsüz grafik, köşeleri $v_1, v_2, \dots, v_n$ olan bir Hamilton yolu, tüm köşelerden geçen basit bir yoldur. Şekil 14.2'deki grafikte bir Hamilton yolu vardır (v_2, v_1), (v_1, v_3) , (v_3, v_5) , (v_5, v_4) , (v_4, v_6) . Hamilton yolu problemi (HAMPATH), verilen bir grafikte bir Hamilton yolunun olup olmadığını belirlemektir.

Belirli bir algoritma kolayca bulunabilir, çünkü herhangi bir Hamilton yolu, köşelerin bir permütasyonudur $v_1, v_2, \dots, v_n$. Bu kadar $n!$ permütasyon vardır ve bunların hepsinin kaba kuvvet araması cevabı verecektir. Ne yazık ki, bu, mütevazı n için bile büyük bir maliyetle gelir.



ŞEKİL 14.2

Belirsiz çözümü keşfetmek için önce bir grafiği bir dize ile temsil etmenin bir yolunu bulmalıyız. Grafikleri kodlamanın en basit ve en uygun yollarından biri, bir komşuluk matrisidir. Yönlendirilmiş düğüm kümesi $v_1, v_2, \dots, v_n$ ve kenar kümesi E , bir komşuluk matrisidir is an $n \times n$ array in which $a(i, j)$, the entry in the i th row and j th kolon karşılar

$$a(i, j) = \begin{cases} 1 & \text{if } (v_i, v_j) \in E, \\ 0 & \text{if } (v_i, v_j) \notin E. \end{cases}$$

Yönsüz bir kenar, karşıt yönlerde iki ayrı kenar olarak düşünülebilir. Dizi

```

0 1 1 0 0 0
1 0 1 0 1 0
1 1 0 0 1 0
0 0 0 0 1 1
0 1 1 1 0 1
0 0 0 1 1 0

```

Şekil 14.2'deki grafiği temsil eder.

Bir grafikte n düğüm varsa, bu grafiğin temsil edilmesi için n^2 uzunluğunda bir dize gereklidir. Yönsüz bir grafik için matris simetriktir, bu nedenle depolama gereksinimi $(n+1)n/2$ 'ye düşürülebilir, ancak her durumda, giriş dizisinin uzunluğu $O(n^2)$ olacaktır.

Sonraki adımda, düğümlerin bir permütasyonunu belirsiz bir şekilde üretiyorduk. Bu, $O(n^3)$ zamanında yapılabilir. Son olarak, permütasyonu kontrol ederek bir yol oluşturup oluşturmadığını kontrol ediyoruz. Bunun için $O(n^4)$ zaman yeterlidir.

Bu nedenle, $HAMPATH \in NP$.

ÖRNEK 14.8

Klik Problemi G , düğümleri $v_1, v_2, \dots, v_n$ olan bir yönsüz graf olsun. Ak-kliği, her düğüm çifti $v_i, v_j \in V_k$ arasında bir kenar bulunan bir alt küme $V_k \subseteq 2V$. Klik problemi (CLIQ), verilen bir k için G 'nin bir k -kliği olup olmadığını belirlemektir.

Belirleyici bir arama, 2^V öğelerinin tümünü inceleyebilir. Buba sit bir işlemdir, ancak üstel zaman karmaşıklığına sahiptir. Belirli z-lık algoritması sadece doğru alt küme tahmin eder. Grafiğin temsil ve kontrolü, önceki örneğe benzer, bu nedenle daha fazla ayrıntıya girmeden, klik probleminin $O(n^4)$ zamanında çözülebileceğini iddia ediyoruz ve

$$CLIQ \in NP.$$

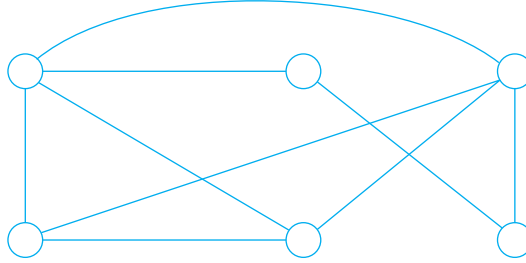
Bu tür birçok başka problem var: bazıları örneklerimize benzer, diğerleri oldukça farklı, ancak hepsi aynı özellikleri paylaşıyor.

1. Tüm problemler NP'dedir ve basit belirsiz çözümleri vardır.
2. Tüm problemler, deterministik çözümlere sahiptir ve bu çözümlerin üstel zaman karmaşıklığı vardır, ancak bunların çözülebilir olup olmadığı bilinmemektedir.

Bu çeşitli problemler arasındaki bağlantıyı daha iyi anlamak için, tüm bu görünüşte farklı durumlar için bazı ortak noktalar bulmamız gerekiyor.

ALISTIRMALAR

1. Örnek 14.6'da, bir deneme çözümünün nasıl $O(n)$ zamanında üretilebileceğini gösterin. Bu, tüm 2^n olasılıklarının $O(n)$ yükseklikteki bir karar ağacında üretilmesi gerektiği anlamına gelir.
2. Örnek 14.6'da deneme çözümünün kontrolünün nasıl yapılabileceğini gösterin. $O(n^2 \log n)$ zamanında.
3. HAMPATH'ta bir permütasyonun belirsiz bir şekilde $O(n^4)$ zamanında nasıl üretilebileceğini tartışın.
4. HAMPATH'ta, Hamiltonian yolunun kontrolü nasıl $O(n^4)$ zamanında yapılabilir? $O(n^4)$ zamanında nasıl yapılabilir?
5. Bir k -kliğin tam olarak $k(k-1)/2$ kenara sahip olması gerektiğini gösterin.
6. Aşağıdaki grafikte bir 4-kliğin bulun. Grafiğin 5-kliğe sahip olmadığını kanıtlayın.



7. Örnek 14.8'deki argümanın detaylarını verin.
8. P 'nin birleşim, kesişim ve tamamlayıcı altında kapalı olduğunu gösterin.

14.6 POLİNOMİYAL-ZAMAN REDÜKSİYONU

Farklı durumları birleştirmenin bir yolu, eğer birisi çözülebiliyorsa diğerlerinin de çözülebileceği anlamında, bunları birbirine indirip indiremediğimize bakmaktır.

TANIM 14.3

Bir dil L_1 , başka bir dile L_2 polinom-zaman indirgenabilir olarak adlandırılır, eğer herhangi bir w_1 için bir deterministik Turing makinesi varsa

L_1 alfabesindeki w_1 , L_2 alfabesindeki w_2 'ye polinom zamanda dönüştürülebiliyorsa ve bu dönüşüm, yalnızca $w_1 \in L_1$ olduğunda $w_2 \in L_2$ olduğu şekilde yapılır.

ÖRNEK 14.9

Memnuniyet probleminin içinde bir maddenin uzunluğu üzerinde herhangi bir kısıtlama yoktur. Kısıtlı bir memnuniyet türü, her maddenin en fazla üç literale sahip olabileceği üç-memnuniyet problemi (3SAT)dir.

SAT problemi, 3SAT'a polinom zamanında indirgenebilir.

Basit bir 4-literal ifadesi ile indirgemeyi gösteriyoruz.

$$e_1 = (x_1 \vee x_2 \vee x_3 \vee x_4).$$

Yeni bir değişken z tanıtıyoruz ve inşa ediyoruz.

$$e_2 = (x_1 \vee x_2 \vee z) \wedge (x_3 \vee x_4 \vee \bar{z}).$$

Eğer e_1 doğruysa, x_1, x_2, x_3, x_4 'ten biri doğru olmalıdır. Eğer $x_1 \vee x_2$ doğruysa, $z = 0$ 'i seçiyoruz ve e_2 doğru olur. Eğer $x_3 \vee x_4 = 1$ ise, $z = 1$ 'i seçebiliriz e_2 'yi sağlamak için. Tersine, eğer e_2 doğruysa, e_1 de doğru olmalıdır, bu nedenle memnuniyet için, e_1 ve e_2 eşdeğerdir.

Bu desenin dört litreden fazla terim içeren maddelere genişletilebileceğini iddia ediyoruz, ancak bu argümanı bir alıştırmaya bırakacağız. Bir CNF ifadesinin SAT formundan 3SAT formuna dönüştürülmesinin deterministik olarak polinom zamanda yapılabileceğini göstermek için bir alıştırmaya ekliyoruz.

ÖRNEK 14.10

3SAT problemi, CLIQUE'e polinom zamanında indirgenebilir.

Herhangi bir 3SAT ifadesinde her bir maddenin tam olarak üç literali olduğunu varsayabiliriz. Eğer bazı ifadelerde bu durum geçerli değilse, sadece tatmin edilebilirliği değiştirmeyen ek literaller ekleriz. Örneğin, $(x_1 \vee x_2)$ ifadesi, $(x_1 \vee x_1 \vee x_2)$ ile eşdeğerdir. Şimdi ifadeyi düşünelim

$$e = (x_1 \vee \bar{x}_2 \vee \bar{x}_3) \wedge (\bar{x}_1 \vee x_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3).$$

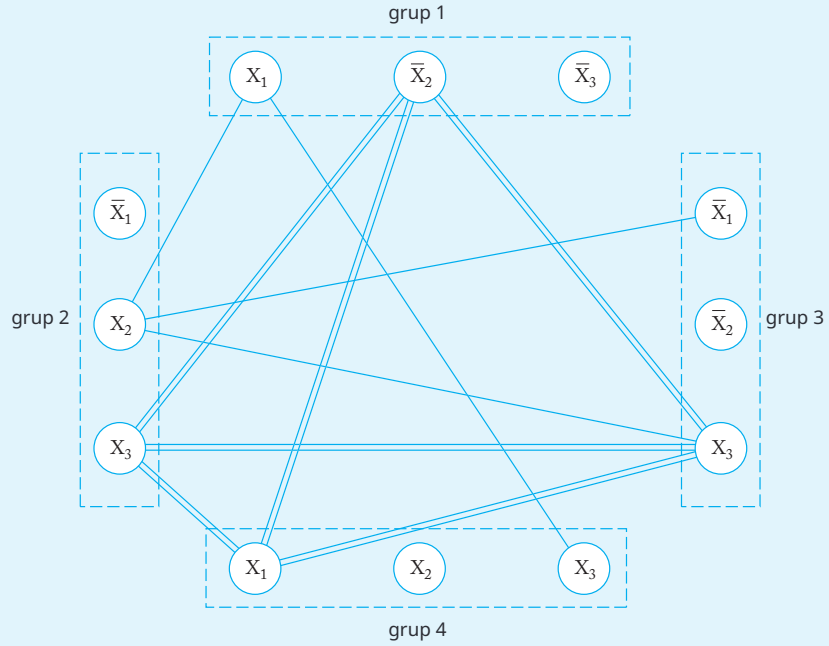
Her bir cümlemin üç köşeden oluşan bir grup ile temsil edildiği ve her bir ifadenin bu köşelerden biri ile ilişkilendirildiği bir grafik çizeriz (Şekil 14.3).

Bir gruptaki her bir köşe için, iki ilişkili literalin tamamlayıcı olmadığı sürece, diğer gruptaki tüm köşelere bir kenar ekliyoruz. Bu nedenle

Şekil 14.3'te bir kenar* $(x_2)_1$ ile $(x_3)_2$ arasında ve $(x_2)_1$ ile $(x_3)_3$, ancak $(x_2)_1$ ile $(x_2)_2$ arasında değildir. Şekil 14.3, kenarların bir alt kümesini göstermektedir (tam küme çok dağınık görünmektedir). $(x_2)_1, (x_3)_2, (x_3)_3$, ve $(x_1)_4$ olan köşeleri içeren alt grafın bir 4-klige olduğunu ve

$$\overline{x_2} = x_3 = x_1 = 1$$

bir değişken atamasıdır.



ŞEKİL 14.3

Bu yaklaşım, herhangi bir 3SAT ifadesi için genelleştirilebilir k klausları ile. 3SAT probleminin, yalnızca ilişkili grafikte k -clique. Ayrıca, bu zor değil 3SAT ifadesinin grafiğe dönüşümünün belirleyici bir şekilde polinom zamanda yapılabileceğini görmek.

Bu indirimlerin amacı, artık belirli bir probleme birkaç farklı açıdan baka bilmemizdir. Diyelim ki SAT'ın çözülebilir olduğunu varsayıyoruz. Eğer bunu k kanıtlamak zor ise, daha basit 3SAT durumunu deneyebiliriz. Eğer bu da

*İkinci alt indeksler, köşelerin ait olduğu grubu tanımlar.

Çalışmıyor, klika problemi için verimli bir algoritma bulmaya çalışabiliriz, veya başka bir ilgili grafik problemi. Seçeneklerden herhangi biri işlenebilir olduğunu gösterilebilirse, SAT'nin işlenebilir olduğunu iddia edebiliriz.

ALISTIRMALAR

1. Beş literalli cümlelerden oluşan bir CNF ifadesinin 3SAT formuna nasıl indirgenebileceğini gösterin.
Bir dizi literalı olan cümleler için yöntemini genelleştir.
2. 3SAT'e SAT'ın nasıl polinom zamanda indirgenebileceğini gösterin.
3. Örnek 14.10'daki ifadenin, bir 3SAT ifadesinin k cümlesinin sadece ve sadece ilişkili grafikte bir k -klığın bulunması durumunda tatmin edilebilir olduğunu gerektirir.
4. Bir 3SAT ifadesi ile ilişkili grafiğin inşasının polinom zamanda deterministik olarak yapılabileceğini gösterin.
5. Seyahat Satıcısı Problemi (TSP) şu şekilde ifade edilebilir. G , kenarları negatif olmayan ağırlıklarla atanmış bir tam graf olsun. Basit bir yolun ağırlığı, tüm kenar ağırlıklarının toplamıdır. TSP problemi, herhangi bir verilen $k \geq 0$ için, grafikte G 'nin ağırlığı k 'den küçük veya eşit olan bir Hamiltonian yolunun olup olmadığını belirlemektir.
HAMPATH'ın TSP'ye polinom zamanda indirgenebilir olduğunu gösterin.

14.7 NP-TAMLIK VE AÇIK BİR SORU

Karmaşıklık çalışmaları için merkezi olan ve eğer bunlardan birini tamamen anlasaydık, işlenebilirlik ile ilgili ana konuyu anlayacağımız birçok problem vardır.

TANIM 14.4

Bir dil L , eğer $L \in \text{NP}$ ve her $L_1 \in \text{NP}$, polinom zamanda L 'ye indirgenebiliyorsa NP-tam olarak adlandırılır.

Bu tanımdan, eğer $L \in \text{NP}$ -tamamlayıcı ise ve L_1 'e polinom-zaman indirgenbilir ise, o zaman L_1 de NP-tamamlayıcıdır. Yani, eğer herhangi bir NP-tamamlayıcı dil için bir deterministik polinom-zaman algoritması bulabilirsek, o zaman NP içindeki her dil de P içindedir, yani,

$$\text{P} = \text{NP}.$$

Burada, böyle problemler için etkili algoritmaların var olduğu umudunu taşıyoruz, henüz hiçbirini bulunmamış olsa bile. Öte yandan, bilinen birçok NP-tamamlayıcı problemin çözülemez olduğunu kanıtlayabilseydik, o zaman birçok ilginç problem pratikte çözülemez hale gelir. Bu, NP-tamamlayıcılığı birçok önemli problemin çözülebilirliği sorusunun merkezine yerleştirir. Bu noktada, bazı NP-tamamlayıcı problemleri çalışmamız gerekiyor.

Bir sonraki sonuç, Cook teoremi olarak bilinir, bu çalışmaya giriş noktasını sağlar.

TEOREM 14.5

Doğruluk Problemi (SAT) NP-tamamlayıcıdır.

Kanıt: Kanıtın arkasındaki fikir, bir Turing makinesinin her yapılandırma dizisi için, ancak kabul eden bir yapılandırma dizisi varsa tatmin edilebilir bir CNF ifadesi oluşturulabileceğidir.

Detaylar, ne yazık ki, uzun ve karmaşıktır. Tekniktirler ve NP problemi hakkında pek az ışık sağlarlar, bu yüzden burada onları atlayacağız. Cook teoremi hakkında kapsamlı tartışmalar, karmaşıklığa adanmış kitaplarda bulunabilir. ■

Eğer Cook teoremini kabul edersek, hemen birçok NP-tamamlayıcı problemimiz olur.

ÖRNEK 14.11

SAT'ın 3SAT'a indirgenebileceğini ve 3SAT'ın CLIQ'e indirgenebileceğini gösterdik. Bu nedenle 3SAT ve CLIQ her ikisi de NP-tamamlayıcıdır.

HAMPATH'ın da NP-tamamlayıcı olduğu ortaya çıkıyor, ancak bunu göstermek için gereken indirgemeler daha az belirgindir.

SAT, 3SAT, CLIQ ve HAMPATH'a ek olarak, NP-tamamlayıcı olduğu bilinen daha birçok problem vardır. Bu problemlerin herhangi biri için verilen imli algoritmalar bulmaya yönelik önemli bir çaba harcanmıştır, ancak şu ana kadar başarılı olunamamıştır. Bu, bize

$$P \neq NP$$

ve birçok önemli problemin çözümsüz olduğunu varsaymamıza yol açıyor. Ancak bu makul bir çalışma varsayımdır, henüz kanıtlanmamıştır. Bu, karmaşıklık teorisindeki temel açık problem olmaya devam etmektedir.

ALISTIRMALAR

1. TSP'nin NP-tamamlayıcı olduğunu gösterin.
 2. Bir G yönsüz graf olsun. Grafın bir Euler döngüsü, tüm kenarları içeren basit bir döngüdür. Euler Döngüsü Problemi (EULER), G 'nin bir Euler döngüsüne sahip olup olmadığını belirlemektir. G bir Euler döngüsüne sahip midir? EULER'in NP-tamamlayıcı olmadığını gösterin.
 3. NP-tamamlayıcı problemleri derlemek için karmaşıklık teorisi üzerine kitaplara da nişin.
 4. $P=NP$ 'nin kararsız olması mümkün mü?
-

EK

A

SONLU DURUM DÖNÜŞTÜRÜCÜLER

Sonlu kabul ediciler, biçimsel dillerin incelenmesinde merkezi bir rol oynamaktadır, ancak dijital tasarım gibi diğer alanlarda, dönüştürücüler daha önemlidir.

Bu konuya derinlemesine giremeyecek olsak da, en azından ana fikirleri özetleyebiliriz. Tam bir inceleme, birçok pratik uygulama örneği ile birlikte Kohavi ve Jha, 2010'da bulunabilir.

A.1 GENEL BİR ÇERÇEVE

Sonlu durum dönüştürücüleri (fst'ler), sonlu kabul edicilerle birçok ortak özelliğe sahiptir. Bir fst, sonlu bir iç durum kümesine Q sahiptir ve i ki durum arasında geçişlerin yapıldığı ayrık bir zaman diliminde çalışır t_n ve t_{n+1} . Bir fst, bir girdi alfabesinden Σ bir dize içeren ve belirli bir girdi karşısında bir çıktı alfabesinden Γ bir dize üreten, yalnızca bir kez o kulan bir girdi dosyası ile ilişkilidir. Her zaman adımında bir girdi sembolü kullanıldığı ve bir çıktı sembolü üretildiği varsayılacaktır (aynı zamanda basıldığını da söyleyebiliriz).

Bir fst, belirli dizeleri diğer dizelere çevirdiğinden, fst'yi bir fonksiyonu n uygulanması olarak görebiliriz. Eğer M bir fst ise, F_M ile M tarafından temsil edilen fonksiyonu göstereceğiz, bu nedenle

$$F_M : D \rightarrow R,$$

burada D , Σ^* 'nin bir alt kümesidir ve R , Γ^* 'nin bir alt kümesidir. Çoğu tartışmada $D = \Sigma^*$ olduğunu varsayıyoruz.

Bir fst 'yi bir fonksiyon olarak yorumlamak, onun belirleyici olduğunu ve bunun da çıktının girdi tarafından benzersiz bir şekilde belirlendiği anlamına gelir. Belirsizlik, dil teorisinde önemli bir konudur, ancak sonlu durum dönüştürücülerinin çalışmasında önemli bir rol oynamaz.

Bir girdi sembolünün bir çıktı sembolü ile sonuçlanması kuralı, $\text{mapping } F_M$ 'nin uzunluk koruyucu olduğunu, yani

$$|F_M(w)| = |w|.$$

Ancak bu daha çok görünüştedir. Örneğin, her zaman boş dize λ 'yi Γ 'ye dahil edebiliriz, böylece

$$|F_M(w)| < |w|$$

mümkün hale gelir. Uzunluk koruyucu kısıtlamanın aşılabileceği başka yollar da vardır.

Çok sayıda türde fst 'ler kapsamlı bir şekilde incelenmiştir. Onlar arasındaki ana fark, çıktının nasıl üretildiğidir.

A.2 MEALY MAKİNELERİ

Bir Mealy makinesinde, her geçiş tarafından üretilen çıktı, geçişten önceki iç duruma ve geçişte kullanılan girdi sembolüne bağlıdır, bu nedenle geçiş sırasında üretilen çıktıyı düşünebiliriz.

DEFINITION A.1

A Mealy makinesi, altılı ile tanımlanır

$$M = (Q, \Sigma, \Gamma, \delta, \theta, q_0),$$

burada

Q sonlu bir iç durumlar kümesidir,
 Σ giriş alfabetidir,
 Γ çıkış alfabetidir,
 $\delta: Q \times \Sigma \rightarrow Q$ geçiş fonksiyonudur,
 $\theta: Q \times \Sigma \rightarrow \Gamma$ çıkış fonksiyonudur,
 $q_0 \in Q$ M 'nin başlangıç durumudur.

Makine, tüm girişlerin işlenmesi için mevcut olduğu q_0 durumunda başlar. Eğer t n zamanında Mealy makinesi q_i durumundaysa, mevcut giriş

sembolü a ve $\delta(q_i, a) = q_j, \theta(q_i, a) = b$ olduğunda, makine q_j durumuna geçecek ve çıkış b üretecektir. Sürecin, girişin sonuna ulaşıldığında sona erdiği varsayılmaktadır. Dikkat edilmelidir ki, bir dönüştürücü ile ilişkili son durumlar yoktur.

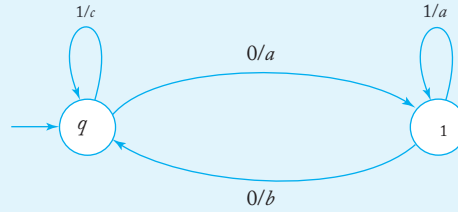
Geçiş grafikleri, sonlu kabul edenler için olduğu kadar burada da faydalıdır. Aslında, tek fark, geçiş kenarlarının artık a/b ile etiketlenmiş olmasıdır; burada a mevcut giriş sembolüdür ve b geçiş tarafından üretilen çıktıdır.

ÖRNEK A.1

fst ile $Q = \{q_0, q_1\}$, $\Sigma = \{0, 1\}$, $\Gamma = \{a, b, c\}$, başlangıç durumu q_0 , ve

$$\begin{aligned}\delta(q_0, 0) &= q_1, & \delta(q_0, 1) &= q_0, \\ \delta(q_1, 0) &= q_0, & \delta(q_1, 1) &= q_1, \\ \theta(q_0, 0) &= a, & \theta(q_0, 1) &= c, \\ \theta(q_1, 0) &= b, & \theta(q_1, 1) &= a\end{aligned}$$

Şekil A.1'deki grafikte temsil edilmektedir. Bu Mealy makinesi, 1010 giriş dizisi verildiğinde $caab$ yazdırır.

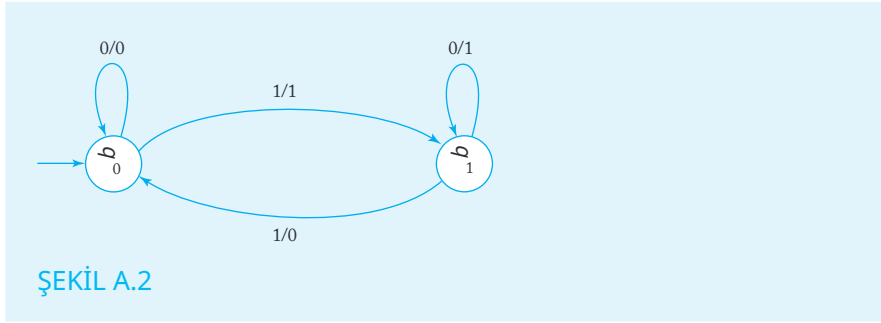


ŞEKİL A.1

ÖRNEK A.2

Bir Mealy makinesi M oluşturun, bu makine 0 ve

1'lerden oluşan dizeleri girdi olarak alır. Çıktısı, girdi içinde ilk 1 meydana gelene kadar 0'lardan oluşan bir dize olacaktır, bu noktada 1'leri yazmaya geçecektir. Bu devam edecektir. girişte bir sonraki 1 ile karşılaşılana kadar, çıktının 0'a geri döndüğü durumdur. Her 1 ile karşılaşıldığında alternasyon devam eder. Örneğin, $F_M(0010010) = 0011100$. Bu fst, bir flip-flop devresi için basit bir modeldir. Şekil A.2 bir çözümü göstermektedir.



A.3 MOORE MAKİNELERİ

Moore makineleri, çıktı üretme şekli bakımından Mealy makinelerinden farklıdır. Bir Moore makinesinde her durum, çıktı alfabesinin bir ögesi ile ilişkilidir. Duruma girildiğinde, bu çıktı sembolü basılır.

Çıktı yalnızca bir geçiş gerçekleştiğinde üretilir; bu nedenle, başlangıç durumuna bağlı sembol başlangıçta basılmaz, ancak bu duruma daha sonraki bir aşamada girilirse üretilebilir.

DEFINITION A.2

A Moore makinesi, altılı ile tanımlanır

$$M = (Q, \Sigma, \Gamma, \delta, \theta, q_0),$$

burada

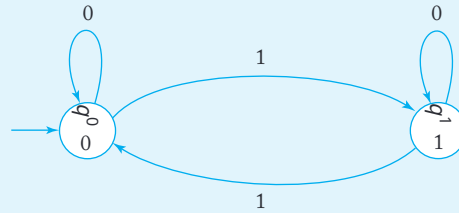
Q sonlu bir iç durumlar kümesidir,
 Σ giriş alfabesidir,
 Γ çıkış alfabesidir,
 $\delta: Q \times \Sigma \rightarrow Q$ geçiş fonksiyonudur,
 $\theta: Q \rightarrow \Gamma$ çıkış fonksiyonudur,
 $q_0 \in Q$ başlangıç durumudur.

Makine, q_0 durumunda başlar; bu sırada tüm girdi işlenmek üzere mevcuttur. Eğer t_n zamanında Moore makinesi q_i durumundaysa, mevcut girdi sembolü a ise, ve $\delta(q_i, a) = q_j$, $\theta(q_j) = b$ ise, makine q_j durumuna geçecek ve çıkış b üretecektir.

Bir Moore makinesinin geçiş grafiğinde, her bir köşe artık iki etikete sahiptir: durum adı ve duruma bağlı çıkış sembolleri.

ÖRNEK A.3

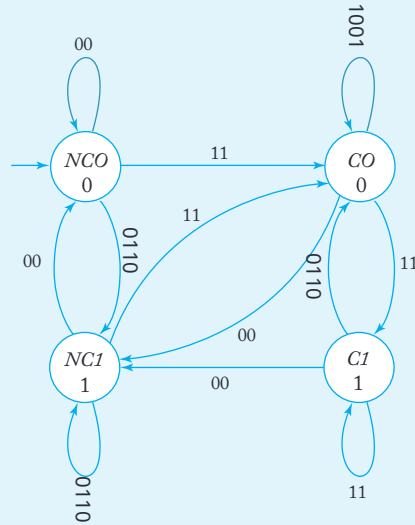
Örnek A.2'deki problem için bir Moore makinesi çözümü Şekil A.3'te verilmiştir.



ŞEKİL A.3

ÖRNEK A.4

Örnek 1.17'de iki pozitif ikili sayıyı toplamak için bir dönüştürücü oluşturduk. Şekil 1.9, aslında oluşturduğumuz şeyin iki durumlu Mealy makinesi olduğunu göstermektedir. Bu problem için bir Moore makinesi de kolayca oluşturulabilir, ancak şimdi hem taşıma hem de çıkış sembolünü takip etmek için dört duruma ihtiyacımız var. Bir çözüm Şekil A.4'te gösterilmektedir.



ŞEKİL A.4

A.4 MOORE VE MEALY MAKİNESİ EŞDEĞERLİĞİ

Önceki bölümdeki örnekler, Moore ve Mealy makineleri arasındaki farkı göstermektedir, ancak aynı zamanda bir problemin bir tür makine ile çözülebileceğini, diğer tür makine ile de çözülebileceğini önermektedir. Bu anlamda, iki tür dönüştürücü muhtemelen eşdeğerdir.

TANIM A.3

İki sonlu durum dönüştürücüsü M ve N eşdeğer olarak kabul edilir, eğer aynı fonksiyonu uygularlarsa, yani aynı tanıma sahiplerse ve eğer

$$F_M(w) = F_N(w),$$

ortak tanımlarındaki tüm w için.

TANIM A.4

Let C_1 and C_2 be two classes of finite-state transducers. We say that C_1 and C_2 are equivalent if for every fst M in one class there exists an equivalent fst N in the other class, and vice versa.

Şimdi Mealy ve Moore makine sınıflarının eşdeğer olduğunu göstereceğiz. Bunun için genişletilmiş geçiş fonksiyonu δ^* ve genişletilmiş çıktı fonksiyonu θ^* tanıtmalıyız. İfade

$$\delta^*(q_i, w) = q$$

fst'nin durum q_i 'den durum q_j 'ye geçiş yaptığını belirtmek içindir w dizisinin işlerken. Benzer şekilde,

$$\theta^*(q_i, w) =$$

fst'nin durum q_i 'den başlayarak ve girdi w verildiğinde çıktı ürettiğini göstermek içindir. Hem Mealy hem de Moore makineleri için δ^* resmi olarak

$$\delta^*(q_i, a) = \delta(q_i, a),$$

$$\delta^*(q_i, wa) = \delta(\delta^*(q_i, w), a),$$

tüm $a \in \Sigma$ ve tüm $w \in \Sigma^+$ için. Mealy makineleri için

$$\theta^*(q_i, a) = \theta(q_i, a),$$

$$\theta^*(q_i, wa) = \theta^*(q_i, w) \theta(\delta^*(q_i, w), a),$$

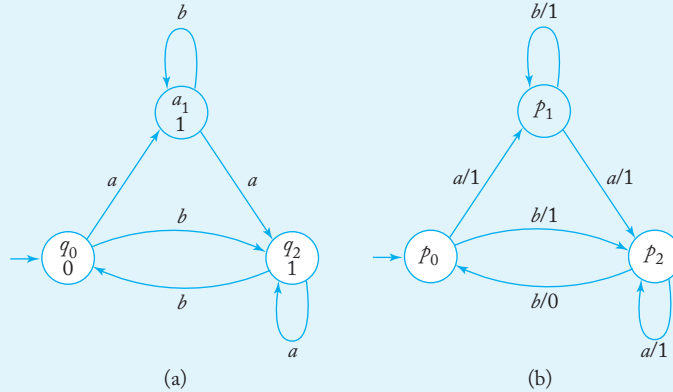
Moore makineleri için

$$\begin{aligned}\theta^*(q_i, a) &= \theta(\delta(q_i, a)), \\ \theta^*(q_i, wa) &= \theta^*(q_i) \theta(\delta^*(q_i, wa)).\end{aligned}$$

Bir Moore makinesinin eşdeğer bir Mealy makinesine dönüştürülmesi basittir. İki makinenin durumları aynıdır ve Moore makinesi tarafından basılacak sembol, o duruma giden Mealy makinesi geçişine atanır.

ÖRNEK A.5

Şekil A.5(a) içindeki Moore makinesi, Şekil A.5(b) içindeki Mealy makinesi ile eşdeğerdir.



ŞEKİL A.5

TEOREM A.1

Her Moore makinesi için eşdeğer bir Mealy makinesi vardır.

Kanıt: $M = (Q, \Sigma, \Gamma, \delta_M, \theta_M, q_0)$ bir Moore makinesi olsun. O zaman bir Mealy makinesi $N = (Q, \Sigma, \Gamma, \delta_N, \theta_N, q_0)$ inşa ederiz, burada

$$\delta_N = \delta_M$$

ve

$$\theta_N(q_i, a) = \theta_M(\delta(q_i, a)).$$

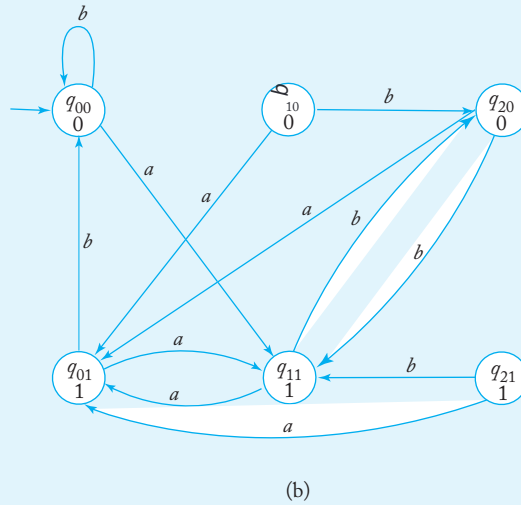
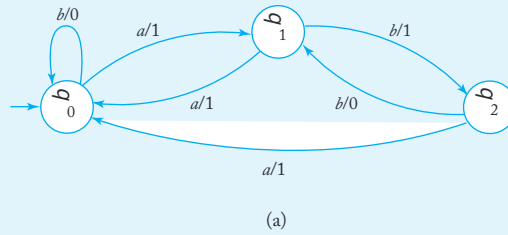
İçgüdüsel olarak, M ve N eşdeğerdir. Her iki makine de belirli bir girdi karşısında aynı durumları geçer. M bir duruma girildiğinde bir sembol yazdırır, N geçiş sırasında aynı sembolü yazdırır.

o duruma. Daha açık bir kanıt, okuyucuya bıraktığımız kolay bir induksiyonu içerir. ■

Bir Mealy makinesinden eşdeğer bir Moore makinesine dönüşüm biraz daha karmaşıktır çünkü Moore makinesinin durumları artık iki parça bilgi taşımak zorundadır: ilgili Mealy makinesinin iç durumu ve Mealy makinesinin o duruma geçişiyle ürettiği çıktı sembolü. Oluşturduğumuz yapıda, her durum için q_i Mealy makinesinin, $|\Gamma|$ Moore makinesinin durumları q_{ia} , $a \in \Gamma$ ile etiketlenmiştir. Moore makinesi durumları için çıktı fonksiyonu $\theta(q_{ja}) = a$ olacaktır. Mealy makinesi durum q_j 'ye geçtiğinde ve a yazdığında, Moore makinesi durum q_{ja} 'ya geçecek ve böylece a da yazacaktır.

ÖRNEK A.6

Şekil A.6(a)'daki Mealy makinesi ve Şekil A.6(b)'deki Moore makinesi eşdeğerdir.



ŞEKİL A.6

TEOREM A.2

Her Mealy makinesi N için eşdeğer bir Moore makinesi M vardır.

Kanıt: $N = (Q_N, \Sigma, \Gamma, \delta_N, \theta_N, q_0)$ olan bir Mealy makinesi alalım. Daha sonra bir Moore makinesi $M = (Q_M, \Sigma, \Gamma, \delta_M, \theta_M, q_{0r})$ aşağıdaki gibi inşa edelim: M için durumları ve çıktı fonksiyonunu oluşturun

$$q_{ia} \in Q$$

ve

$$\theta(q_{ia}) = a,$$

her $i = 1, 2, \dots, n$ ve tüm $a \in \Gamma$. Her geçiş kuralı için

$$\delta_N(q_i, a) = q_j$$

ve karşılık gelen çıktı fonksiyonu

$$\theta(q_i, a) = b$$

tanıtlıyoruz $|\Gamma|$ kurallar

$$\delta_M(q_{ic}, a) = q_j$$

tüm $c \in \Gamma$. Başlangıç durumu sembolü ilk geçişten önce yazılmadığı için, M için başlangıç durumu herhangi bir durum q_{0r} , $r \in \Gamma$ olabilir. Bu, inşayı tamamlar.

N ve M 'nin eşdeğer olduğunu göstermek için, önce

$$\delta_N^*(q_0, w) = q \quad , \quad (A.1)$$

o zaman $c \in \Gamma$ böyle ki

$$\delta_M^*(q_0, w) = q_{kc}. \quad (A.2)$$

Bu ve tersinin, w uzunluğu üzerinde bir induksiyon ile kanıtlanabilir.

Sonraki durumu düşünün

$$\theta_N^*(q_0, w a \theta^*(q, w) \delta_N^*(q_0, w), a)) \quad (A.3)$$

ve varsayalım ki $\delta_N^*(q_0, w) = q_k$, $\delta_N(q_k, a) = q_l$ ve $\theta_N(q_k, a) = b$. O zaman

$$\theta_N(\delta_N^*(q, w)) = b.$$

(A.2)'den ve

$$\delta_M(q_{kc}, a) = q_{lb}$$

şu sonuç çıkar

$$\begin{aligned} \theta_M^*(q_{0r}, w a) &= \theta_M^*(q, w \theta_M(q_{lb})) \\ &= \theta_M^*(q, w) b. \end{aligned}$$

Şimdi (A.3) ile geri dönüyoruz

$$\begin{aligned}\theta_N^*(q_0, wa) &= \theta_N^*(q_0, w)\theta_N(\delta_N^*(q_0, w), a) \\ &= \theta_N^*(q_0, w)b.\end{aligned}$$

Eğer şimdi indüktif varsayımı yaparsak

$$\theta_N^*(q_0, w) = \theta_M^*(q_{0r}, w) \quad (\text{A.4})$$

tüm $|w| \leq m$ ve herhangi bir $r \in \Gamma$ için o zaman

$$\begin{aligned}\theta_N^*(q_0, wa) &= \theta_M^*(q_{0r}, w)b \\ &= \theta_M(q_{0r}, wa).\end{aligned}$$

ve böylece (A.4) tüm m için geçerlidir.

İki yapıyı bir araya getirerek temel eşdeğerlik

sonucunu elde ediyoruz.

TEOREM A.3

Mealy makineleri ve Moore makinelerinin sınıfları eşdeğerdir.

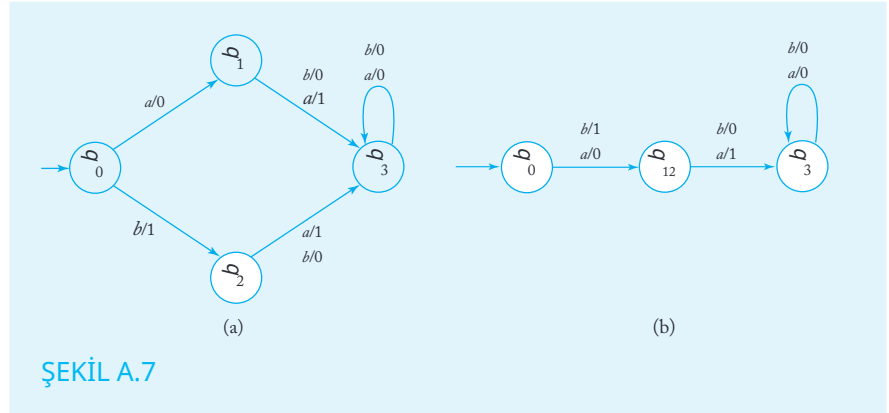
A.5 MEALY MAKİNESİ MİNİMİZASYONU

Verilen bir fonksiyon $\Sigma^* \rightarrow \Gamma^*$ için birçok eşdeğer sonlu durum dönüştürücü vardır; bunlardan bazıları iç durum sayısı bakımından farklılık gösterir. Pratik nedenlerden dolayı, genellikle en az sayıda iç duruma sahip olan minimal fst'yi bulmak önemlidir.

Bir Mealy makinesini minimize etmenin ilk adımı, başlangıç durumunda ulaşılabilen ve herhangi bir hesaplamada rol oynamayan durumları kaldırmaktır. Ulaşılabilen durumlar yoksa, fst'nin bağlı olduğu söylenir. Ancak bir Mealy makinesi bağlı olabilir ve yine de minimal olmayabilir; bu, bir sonraki örnekte gösterilmektedir.

ÖRNEK A.7

Şekil A.7(a)'daki Mealy makinesi bağlıdır, ancak q_1 ve q_2 durumlarını aynı amaca hizmet ettiği ve Şekil A.7(b)'deki makineyi vermek için birleştirilebileceği açıktır.

**DEFINITION A.5**

Bir $M = (Q, \Sigma, \Gamma, \delta, \theta, q_0)$ Mealy makinesi olsun. Durumların q_i ve q_j eşdeğer olduğunu söyleriz eğer ve yalnızca

$$\theta^*(q_i, w) = \theta^*(q_j, w)$$

tüm $w \in \Sigma^*$ için. Eşdeğer olmayan durumlara ayırt edilebilir denir.

DEFINITION A.6

İki durum q_i ve q_j k -eşdeğer

$$\theta^*(q_i, w) = \theta^*(q_j, w)$$

tüm $|w| \leq k$ için. İki durum k -ayrılabilir eğer $|w| \leq k$

olan bir w vardır böylece $\theta^*(w, q_i) \neq \theta^*(w, q_j)$.

THEOREM A.4

(a) Bir Mealy makinesinin iki durumu q_i ve q_j 1-ayrılabilir eğer $a \in \Sigma$ vardır böylece

$$\theta(q_i, a) \neq \theta(q_j, a).$$

(b) İki durum k -ayrılabilir eğer $(k - 1)$ -ayrılabilirler veya $a \in \Sigma$ vardır böylece

$$\delta(q_i, a) = q_r$$

$$\delta(q_j, a) = q_s,$$

burada q_r ve q_s $(k - 1)$ -ayrılabilir.

Kanıt: Bölüm(a) tanımdan doğrudan gelmektedir ve dolayısıyla, durumların k -ayırıcı olduğunu belirten sonuç da öyledir, eğer $(k-1)$ -ayırıcı iseler. Bölüm(b) için, böyle bir $|w| \leq k-1$ olduğunu biliyoruz, öyle ki

$$\theta^*(q_r, w) \neq \theta^*(q_s, w).$$

Şimdi

$$\theta^*(q_i, aw) = \theta(q_i, a)\theta^*(q_r, w)$$

ve

$$\theta^*(q_j, aw) = \theta(q_j, a)\theta^*(q_s, w).$$

Sonuç buradan çıkar. ■

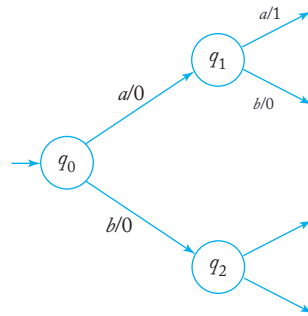
TEOREM A.5

Bir $M_1 = (Q, \Sigma, \Gamma, \delta, \theta, q_0)$ Mealy makinesi olsun, burada tüm durumlar ayırıcıdır. O zaman M_1 minimal ve benzersizdir (durumların yeniden etiketlenmesi içinde).

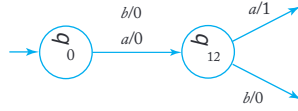
Kanıt: Varsayalım ki, daha az duruma sahip eşdeğer bir makine M_2 vardır. M_1 'den daha az duruma sahip. Eğer M_2 , M_1 'den daha az duruma sahipse, güvercin yuvası ilkesine göre, M_2 'nin en az bir durumu, M_1 'nin durumlarının birkaç fonksiyonunu birleştirmelidir. M_1 . Özellikle, her biri M_1 'de farklı bir duruma götüren iki dize olmalıdır, bunlar M_2 'de aynı duruma götürmektedir. M_2 'nin ne tür bir yapısı olabilir?

Örnek olarak, M_1 'nin Şekil A.8'de gösterilen kısmi yapısına sahip olduğunu varsayalım ve M_2 için durumları q_1 ve q_2 birleştirmeye çalışalım. Eşdeğerliliği korumak için, M_2 'nin kısmi yapıları Şekil A.9'da gösterildiği gibi olmalıdır. Ancak, q_1 ve q_2 M_1 'de ayırt edilebilir olduğundan, bazı w olmalıdır ki

$$\theta^*(q_1, w) \neq \theta^*(q_2, w).$$



ŞEKİL A.8



ŞEKİL A.9

Bu nedenle, M_1 'de giriş dizeleri aw ve bw farklı çıktılar üretmelidir. Ancak M_2 'de aynı şeyi açıkça yazıyorlar. Bu çelişki, bu seviyede iki makinenin de aynı olması gerektiğini ima ediyor. Bu akıl yürütme, iki makinenin her hangi bir parçasına genişletilebileceğinden, teorem ortaya çıkar. ■

Mealy makinesinin minimize edilmesi, bu nedenle eşdeğer durumların tanımlanmasıyla başlar.

Bölme Algoritması

1. Durumlar ile $Q = \{q_0, q_1, \dots, q_n\}$ q_0 'a 1-eşit olan tüm durumları bul ve Q 'yu iki kümeye ayır $\{q_0, q_i, \dots, q_j\}$ ve $\{q_k, \dots, q_l\}$. Bu kümelerin ilki q_0 'a 1-eşit olan tüm durumları içerecek, ikincisi ise ondan 1-ayrılabilir olan tüm durumları içerecek. Sonra, bu süreci $q_1, q_2, \dots, q_n$ durumları ile tekrarlıyoruz. Tekrar eden kümeleri kaldırarak, 1-eşitliğe dayalı bir bölme ile kalıyoruz.
2. Her eşdeğer sınıfta bulunan her durum çifti q_i ve q_j için, Theorem A.4'te olduğu gibi geçişlerin olup olmadığını belirleyin eğer durumlar q_r ve q_s farklı eşdeğer sınıflarında. Eğer öyleyse, onları ayırmak için yeni eşdeğer sınıfları oluşturun. Tüm çiftleri kontrol ederek uygun eşdeğer sınıflarını bulun.
3. Yeni eşdeğer sınıflar oluşturulmadığı sürece adım 2'yi tekrarlayın.

Bu prosedürün sonunda, belirlenen durum Q eşdeğerlik sınıflarına ayrılmış olacaktır $E_1, E_2, \dots, E_q$ böylece her sınıfın tüm üyeleri Tanım A.5 anlamında eşdeğerdir.

Prosedürü haklı çıkarmak için birkaç noktaya değinilmesi gerekiyor. İlk olarak, k 'inci geçişten sonra adım 2'de mevcut bir eşdeğerlik sınıfının tüm elemanlarının $(k + 1)$ -eşdeğer olduğu görülmektedir. Bu, Teorem A.4'ün (b) kısmını kullanarak induksiyonla takip edilmektedir.

İkinci nokta, sürecin sona ermesi gerektiğidir. Bu açıktır çünkü adım 2'de her geçiş en az bir yeni eşdeğerlik sınıfı oluşturur ve en fazla $|Q|$ kadar böyle sınıf olabilir.

Son olarak, sürecin durduğunda tam bir eşdeğerlik bölümlendirmesinin sağlandığını göstermeliyiz. Bu, Teorem A.4'ün (b) kısmından görülebilir. Bir çift (q_i, q_j) k -ayırıcı olması için $(k - 1)$ -ayırıcı durumlar q_r ve q_s olmalıdır; eğer böyle durumlar yoksa, daha uzun dizelerle ayırt edilebilecek durumlar da olamaz. Bu nedenle, o eşdeğerlik bölümlendirmesi tamam olmalıdır.

Minimum Mealy Makinesi Yapımı $M = (Q, \Sigma, \Gamma, \delta, \theta, q_0)$ olsun, minimal eşdeğer bir makine $P = (Q_P, \Sigma, \Gamma, \delta_P, \theta_P, Q_P)$ inşa etmek istediğimiz Mealy makinesi. İlk olarak, eşdeğerlik sınıflarını $E_1, E_2, \dots, E_m$ kullanarak bölme prosedürünü buluyoruz ve P için $E_1, E_2, \dots, E_m$ etiketli durumlar oluşturuyoruz. Eşdeğerlik sınıfından bir eleman q_i seçin ve diğer eşdeğerlik sınıfından bir eleman q_j seçin. Eğer $\delta(q_i, a) = q_j$ ve $\theta(q_i, a) = b$ ise, P için geçiş fonksiyonunu tanımlayın.

$$\delta_P(E_r, a) = E_s$$

ve tarafından çıktı

$$\theta_P(E_r, a) = b.$$

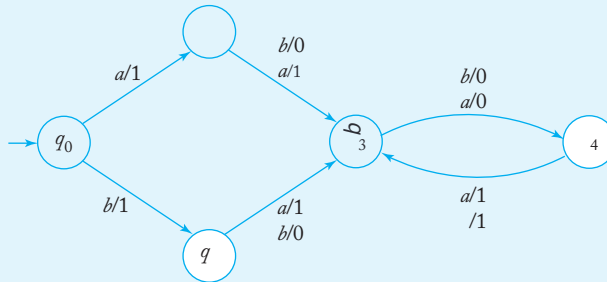
Başlangıç durumu için $M q_0$ ise, P için başlangıç durumu E_p olacak şekilde q_0 eşdeğerlik sınıfında E_p bulunur. P 'nin M 'nin minimal eşdeğeri olduğunu göstermek basit bir alıştırmadır. Ayrıca, Teorem A.5'ten de, minimal Mealy makinesinin durumların basit bir yeniden etiketlenmesi içinde benzersiz olduğu sonucuna varılır.

ÖRNEK A.8

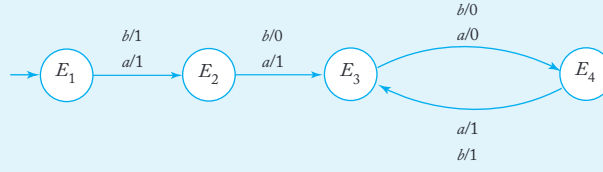
Şekil A.10'daki Mealy makinesini düşünün. Adım 1, eşdeğerlik bölümlendirilmesini üretir $\{q_0, q_4\}, \{q_1, q_2\}, \{q_3\}$. İkinci adımda şunu buluyoruz $\delta(q_0, a) = q_1, \delta(q_4, a) = q_3$. Çünkü q_1 ve q_3 ayırt edilebilir, bu nedenle q_0 ve q_4 de ayırt edilebilir ve yeni bölüm

$$E_0 = \{q_0, q_4\}, E_1 = \{q_1, q_2\}, E_2 = \{q_3\}, E_3 = \{q_3\}, E_4 = \{q_4\}.$$

Adım 2'den bir geçiş daha, daha fazla iyileştirme sağlamaz ve bölümlendirme tamamlanır. Buradan, Şekil A.11'deki minimal Mealy makinesini inşa ederiz.



ŞEKİL A.10



ŞEKİL A.11

A.6 MOORE MAKİNESİ MİNİMİZASYONU

Bir Moore makinesinin minimizasyonu, Mealy makinelerinin minimizasyonu desenini takip eder, ancak bazı farklılıklar vardır. Tanım A.5 ve A.6 Moore makinelerine uygulanırken, Teorem A.4'ün değiştirilmesi gerekir.

TEOREM A.6

(a) İki durum q_i ve q_j bir Moore makinesinde 0-ayırıcıdır eğer

$$\theta(q_i) \neq \theta(q_j).$$

(b) İki durum k -ayırıcıdır eğer $(k-1)$ -ayırıcıdır veya öyle bir $a \in \Sigma$ vardır ki $\delta(q_i, a) = q_r$ ve $\delta(q_j, a) = q_s$, burada q_r ve q_s $(k-1)$ -ayırıcıdır.

Kanıt: Buradaki argüman esasen Teorem A.4'tekiyle aynıdır. ■

Moore makinesi minimizasyonu için önce Mealy makinesi bölme prosedürünü kullanarak durumları eşdeğer sınıflara ayırıyoruz ve minimizasyon sürecinde bunları kullanıyoruz.

Minimal Moore Makinesi Yapımı

Let $M = (Q, \Sigma, \Gamma, \delta, \theta, q_0)$ minimize edilecek Moore makinesi olsun. Öncelikle eşdeğer sınıfları $E_1, E_2, \dots, E_m$ bölme algoritmasıyla belirliyoruz ve minimize edilmiş makine P için $E_1, E_2, \dots, E_m$ etiketli durumlar oluşturuyoruz. Bir eleman q_i seçin E_r içinden ve bir eleman q_j seçin E_s içinden. Eğer $\delta(q_i, a) = q_j$ ve $\theta(q_j) = b$ ise, o zaman P için geçiş fonksiyonu

$$\delta_P(E_r, a) = E_s$$

ve

$$\theta_P(E_s) = b.$$

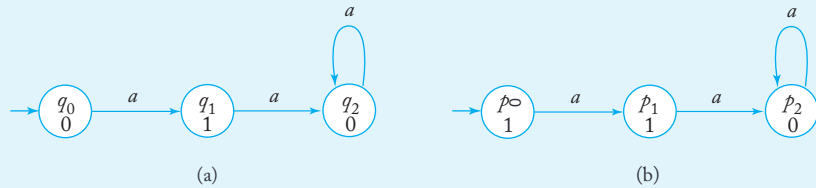
Bu, minimal Moore makinesini verecektir.

Minimal Moore makinesi inşasıyla ilgili küçük bir komplikasyon vardır. Minimizasyon süreci, eşdeğer duruma $\theta(q_0)$ atar q_0 ile ilişkilidir. Bazı durumlarda bu, benzersizlik sorununa yol açabilir.

ÖRNEK A.9

Şekil A.12'deki iki Moore makinesine bakın. Açıkça, bunlar eşdeğer ve ayrıca açıkça minimaldir. Ancak, iki başlangıç durumu için çıktı farklı olduğundan, sadece yeniden etiketleme ile aynı hale getirilemezler. Zorluk, başlangıç durumunun yeniden girilemediği sürece çıktının keyfi olmasından kaynaklanmaktadır.

Ancak bu, göz ardı edilebilecek önemsiz bir meseledir.



ŞEKİL A.12

A.7 SONLU DURUM DÖNÜŞTÜRÜCÜLERİN SINIRLARI

Mealy ve Moore makineleri sonlu durum otomatalarıdır, bu nedenle yeteneklerinin sınırlı olduğunu doğru bir şekilde şüpheleniyoruz, tıpkı sonlu kabul edenlerin sınırlı olduğu gibi. Bu sınırlamaları keşfetmek için düzenli diller için pompalama lemmasına benzer bir şey e ihtiyacımız var.

TEOREM A.7

Bir $M = (Q, \Sigma, \Gamma, \delta, \theta, q_0)$ Mealy makinesi olsun. O zaman bir durum $q_i \in Q$ ve bir $w \in \Sigma^+$ vardır, öyle ki

$$\delta^*(q_i, w) = q_i.$$

Kanıt: Bu, kuş yuvası ilkesinden gelmektedir, $|Q|$ 'nin sonlu olduğunu, ancak w 'nin keyfi olarak uzun olabileceğini not ederek. ■

ÖRNEK A.10

Fonksiyonu $F\{a\}^* \rightarrow \{a, b\}^*$ olarak tanımlayın

$$\begin{aligned} F(a^{2n}) &= a^n b^n, \\ F(a^{2n+1}) &= a^n b^{n+1}. \end{aligned}$$

Bu fonksiyonu uygulayan bir Mealy makinesi var mı? Birkaç deneme hızla cevabın hayır olduğunu önermektedir.

Bir m durumu olan böyle bir makine var ise, girdi olarak $w = a^{2m}$ se çebiliriz. Bu dizinin ilk kısmının işlenmesi sırasında, Teorem A.7'ye göre, makine, a girdisinin a çıktısını ürettiği bir döngüye girmek zorunda kalacaktır. Döngüden çıkmak için a dışında bir girdi gerekecektir. Ancak başka bir girdi olmadığı için, makine sürekli olarak a yazmaya devam edecek ve böylece fonksiyonu temsil edemeyecektir. Çelişki, böyle bir makinenin var olamayacağını göstermektedir.

Bu örnekten çıkarılan sonuç şaşırtıcı olmamalıdır, çünkü bu, düzenli bir kümenin düzensiz bir kümeye çevrilmesini içermektedir. Aslında, fst'ler ile sonlu kabul ediciler arasındaki yapısal benzerlik, düzenli diller ile sonlu durum dönüştürücüler tarafından üretilen çıktı arasında bir bağlantı olduğunu önermektedir.

TANIM A.7

M , fonksiyonu F_M uygulayan bir sonlu durum dönüştürücüsü olsun ve L herhangi bir dil olsun. O zaman

$$T_M(L) = \{F_M(w) : w \in L\}$$

L 'nin M -çevirisi dir.

Artık bir fst, genellikle sınırlı olmayan girdi üretebileceğinden her türlü çıktıyı üretebilir. Ancak girdi bir düzenli dil ise, çıktı da öyle olacaktır.

TEOREM A.8

$M = (Q, \Sigma, \Gamma, \delta_M, \theta_M, q_0)$ bir Mealy makinesi olsun ve L bir düzenli dil olsun. O zaman $T_M(L)$ de düzenlidir.

Kanıt: Eğer L düzenli ise, o zaman bir dfa $N = (P, \Sigma, \delta_N, p_0, F)$ vardır ki $L = L(N)$. Şimdi M ve N ile bir sonlu kabul edici (muhtemelen

belirsiz) $H = (Q_H, \Sigma, \delta_H, q_H, F_H)$ aşağıdaki gibi inşa edelim:

$$Q = \{q_{ij} : p_i \in P, q_j \in Q\}$$

Eğer $\delta_N(p_i, a) = p_k$, $\delta_M(q_j, a) = q_l$, ve $\theta_M(q_j, a) = b$, o zaman

$$\delta_H(q_{ij}, b) = q_{kl}. \quad (\text{A.5})$$

H için başlangıç durumu q_{00} olacak ve son durum kümesi

$$F_H = \{q_{ij} : p_i \in F, q_j \in Q\}.$$

O zaman $T_H(L)$ düzenlidir.

Bu ifadeyi savunmak için, önce şunu gözlemleyelim ki eğer $\delta_N^*(p_i, w) = p_k$ ve $\delta_M^*(q_j, w) = q_l$, o zaman

$$\delta_H^*(q_{ij}, \theta_M^*(q_{ij}, w)) = q_{kl}.$$

Bu, basit bir induksiyonla takip eder. Dolayısıyla, eğer $\delta_N(p_0, w) \in F$,
öyleyse

$$\delta_H(q_{00}, \theta_M^*(q_{00}, w)) \in F,$$

ve eğer $w \in L$, o zaman $F_M(w) \in L(H)$.

Argümanı tamamlamak için, eğer bir dize $v \in H$ tarafından kabul ediliyorsa, o zaman $w \in L$ olmalıdır, öyle ki $v = F_M(w)$. Şimdi H yerine, H' ile aynı olan başka bir dfa H' inşa ettiğimizi varsayalım, tek farkı (A.5)'in yerine geçmesidir.

$$\delta_H(q_{ij}, a) = q_{kl}.$$

N ve H' eşdeğerdir ve bir dize $w \in H'$ tarafından kabul edilir, eğer ve yalnızca $w \in L$. Şimdi eğer $v \in L(H)$ ise, o zaman H için geçiş grafi içinde q_{00} 'dan v etiketli bir son duruma giden bir yol vardır. Ancak H' içindeki aynı yol w ile etiketlenmiştir, bu nedenle $v = F_M(w)$. Bu nedenle $w \in L$. ■

EK

B

JFLAP: FAYDALI BİR ARAÇ

Bu kitabın temel önermesi, zor soyut kavramların anlaşılmasının en iyi şekilde örneklerle ve zorlu alıştırmalarla sağlandığıdır, bu nedenle problem çözme, yaklaşımımızın merkezi bir temasını oluşturur.

Zor bir problemi çözmek genellikle iki ayrı adım içerir. Öncelikle, sorunu anlamalı, hangi teoremlerin ve sonuçların geçerli olduğunu belirlemeli ve hepsini bir araya getirerek bir çözüme ulaşmalıyız. Bu genellikle en zor kısımdır ve sıklıkla içgörü ve yaratıcılık gerektirir. Ancak çözüm sürecini net bir şekilde anladıktan sonra, somut sonuçlar üretmek için daha rutin bir adım gereklidir. Çalışmamızda, bu otomata veya dil bilgisi inşa etmek ve bunları doğruluk açısından test etmekle ilgilidir. Bu adım daha az zorlu olabilir ama sıkıcı ve hata yapmaya yatkındır. İşte burada yazılım şeklindeki mekanik yardım çok faydalı olabilir. Deneyimlerime göre, JFLAP bu amaca mükemmel bir şekilde hizmet etmektedir.

JFLAP, bu kitapta yer alan kavramlar üzerine inşa edilmiş etkileşimli bir araçtır. Duke Üniversitesi'nden Profesör Susan Rodger ve öğrencileri tarafından oluşturulmuştur. Birçok üniversitede yıllar boyunca başarıyla kullanılmıştır. JFLAP ile ilgili bazı materyaller JPAK'ta toplanmıştır, çevrimiçi olarak şu adreste mevcuttur: go.jblearning.com/LinzJPAK. JPAK, JFLAP'a kısa bir giriş, nasıl edineceği ve nasıl kullanılacağı hakkında bilgi verir. Ayrıca, JFLAP'ın gücünü gösteren birçok alıştırma örneği ve faydalı bir fonksiyon kütüphanesi içerir.

JFLAP birçok açıdan faydalıdır. Öğrenci için, JFLAP soyut kavramların pratikte nasıl uygulandığını görme imkanı sunar. Bir

zor inşaat, bir dfa'nın düzenli bir ifadeye dönüştürülmesi gibi, uygulandı ğında, aksi takdirde kavraması zor olabilecek bir şeyi hayata geçirir. No nintuitif kavramlar, örneğin belirsizlik, pratik bir şekilde gösterilmektedi r. JFLAP ayrıca büyük bir zaman tasarrufu sağlar. Otomatları ve gramerl eri inşa etmek, test etmek ve değiştirmek, daha geleneksel kalem ve ka ğıt yöntemine göre çok daha kısa bir sürede yapılabilir. Kapsamlı testler in kolay olması, nihai ürünün kalitesini de artıracaktır.

Öğretmenler JFLAP'tan da faydalanabilir. Elektronik teslimat ve toplu notlandırma, çok fazla çaba tasarrufu sağlarken, aynı zamanda değerlendirmelerin doğruluğunu ve adaletini artıracaktır. Öğretici olan, fakat genellikle büyük miktarda gereksiz iş nedeniyle kaçınılan alıştırmalar şimdi mümkündür. Burada bir örnek, sağ-lineer bir dilbilgisinin eşdeğer sol-lineer bir dilbilgisine dönüştürülmesidir. Turing makineleriyle çalışmak bilinen şekilde zahmetli ve hata yapmaya açıktır, ancak JFLAP ile çok daha zorlayıcı ödevler makul hale gelmektedir. JPAK, bu tür alıştırmaların büyük bir sayısına sahiptir. Son olarak, JPAK, kitapta yer alan birçok örneğin JFLAP uygulamalarını içerdiğinden, bu örneklerin dinamik bir sınıf sunumu için bir fırsat vardır.

JFLAP, oldukça kendini açıklayıcıdır ve kullanmayı öğrenmek için çok az çaba g erektirir. Hem öğrenciler hem de eğitimler için kullanımını şiddetle tavsiye ediyor um.

CEVAPLAR: SEÇİLMİŞ EGZERSİZLER İÇİN ÇÖZÜMLER VE İPUÇLARI

Bölüm 1

Bölüm 1.1

5. Diyelim ki $x \in S - T$. O zaman $x \in S$ ve $x \notin T$, bu da $x \in S$ ve $x \notin T$ olduğu anlamına gelir, yani $x \in S \cap T$. Yani $S - T \subseteq S \cap T$. Tersine, eğer $x \in S \cap T$, o zaman $x \in S$ ve $x \notin T$, bu da $x \in S - T$ olduğu anlamına gelir, yani $S \cap T \subseteq S - T$. Bu nedenle $S - T = S \cap T$.
6. Denklem (1.2) için, diyelim ki $x \in S_1 \cup S_2$. O zaman $x \in S_1 \cup S_2$, bu da $x \in S_1$ veya $x \in S_2$ içinde olamaz, yani $x \in S_1 \cap S_2$. Yani $S_1 \cup S_2 \subseteq S_1 \cap S_2$. Tersine, eğer $x \in S_1 \cap S_2$, o zaman $x \in S_1$ içinde değildir ve $x \in S_2$ içinde değildir, yani $x \in S_1 \cup S_2$. Bu nedenle $S_1 \cap S_2 \subseteq S_1 \cup S_2$. Bu nedenle $S_1 \cup S_2 = S_1 \cap S_2$.
12. (a) yansıtıcılık: Çünkü $|S_1| = |S_1|$, bu nedenle yansıtıcılık geçerlidir.
(b) simetri: eğer $|S_1| = |S_2|$ ise $|S_2| = |S_1|$ böylece simetri sağlanır.
(c) geçişlilik: Çünkü $|S_1| = |S_3| = |S_2|$. geçişlilik geçerlidir ve ilişki bir eşdeğerliktir.
20. Üç eşdeğer sınıf şunlardır: $E_0 = \{6, 9, 24\}$, $E_1 = \{1, 4, 25, 31, 37\}$, ve $E_2 = \{2, 5, 23\}$.
26. Normal aritmetik işlemler uygulanamaz. Örneğin, alalım $x = n^3$ ve $y = \frac{1}{n^2}$, o zaman tanıma göre $x = O(n^4)$, $y = O(n^2)$. Ancak, $x/y = n^5$.

30. Genel olarak, sıradan aritmetik işlemler büyüklük sırasına uygun olmaz. Yani $O(n^2) - O(n^2)$ mutlaka 0'a eşit değildir. Örneğin, bu problemde eğer $g(n) = n^2$ ise, o zaman $f(n) - g(n) = n^2 + n = O(n^2)$.

39. (a) $f(n) < 2(2^n)$ ifadesi $n = 1$ için geçerlidir. Farz edelim ki $f(k) < 2(2^k)$ ifadesi $k = 1, 2, \dots, n, n + 1$ için doğrudur. O zaman

$$f(n+2) = f(n+1) + f(n) < 2(2^{n+1}) + 2(2^n) < 2^{n+2} + 2^{n+1} < 2(2^{n+2}).$$

(b) $f(n) > 1.5^n/10$ ifadesi $n = 1$ için doğrudur. Farz edelim ki bu ifade $k = 1, 2, \dots, n + 1$ için doğrudur, o zaman

$$\begin{aligned} f(n+2) &= f(n+1) + f(n) > 1.5^{n+1}/10 + 1.5^n/10 \\ &= 1.5^{n+1}(1 + \frac{1}{1.5})/10 > 1.5^{n+1}(1 + 0.5)/10 = 1.5^{n+2}/10. \end{aligned}$$

41. Varsayalım $2 - \sqrt{2}$ bir rasyonel sayıdır, o zaman

$$2 - \sqrt{2} = \frac{n}{m},$$

buradan n ve m ortak çarpanı olmayan tam sayılardır. Eşdeğer olarak, şunu elde ederiz

$$\sqrt{2} = 2 - \frac{n}{m} = \frac{2m - n}{m}$$

bu da demektir ki $\sqrt{2}$ bir rasyonel sayıdır. Ancak, zaten biliyoruz ki $\sqrt{2}$ rasyonel değildir, bu yüzden $2 - \sqrt{2}$ irrasyonel olmalıdır.

43. (a) Doğru. Eğer a rasyonel ve b irrasyonel ise, ancak $c = a + b$ rasyonel ise, o zaman $b = c - a$ rasyonel olmalıdır, bu bir çelişkidir.

(b) Yanlış. Örneğin, $a = 2 + \sqrt{2}$ ve $b = 2 - \sqrt{2}$ her ikisi de pozitif irrasyonellerdir, ancak $a + b = 4$ rasyoneldir.

(c) Doğru. Eğer $a = 0$ rasyonel ve $b = 0$ irrasyonel ise ama $c = ab$ rasyonel ise, o zaman $b = c/a$ rasyonel olmalıdır.

Bölüm 1.2

2. İçin $n = 1, |u^1| = |u|$. Varsayalım $|u^k| = k|u|$ $k = 1, 2, \dots, n$ için geçerlidir, o zaman

$$|u^{n+1}| = |u^n u| = |u^n| + |u| = n|u| + |u| = (n+1)|u|.$$

6. $\bar{L} = \{\lambda, a, b, ab, ba\} \cup \{w \in \{a, b\}^+ : |w| \geq 3\}$.

8. Hayır, böyle bir dil yoktur, çünkü $\lambda / \in L^*$ ama $\lambda \in (L)^*$.

11. (a) Doğru. Varsayalım $w \in (L_1 \cup L_2)^R$, o zaman $w^R \in (L_1 \cup L_2)$, bu nedenle $w^R \in L_1$ veya $w^R \in L_2$. ve $w \in (L_1^R \cup L_2^R)$. Bu nedenle, $(L_1 \cup L_2)^R \subseteq (L_1^R \cup L_2^R)$.

Benzer şekilde, $(L_1^R \cup L_2^R) \subseteq (L_1 \cup L_2)^R$ olduğunu kanıtlayabiliriz. Bu nedenle, $(L_1 \cup L_2)^R = L_1^R \cup L_2^R$.

13.

$$\begin{aligned} S &\rightarrow aaaaA, \\ A &\rightarrow aAa|\lambda. \end{aligned}$$

14. (a)

$$\begin{aligned} S &\rightarrow AaAaA, \\ A &\rightarrow bA|\lambda. \end{aligned}$$

(f) Öncelikle b 'lerin çiftlerini oluşturuyoruz, ardından rastgele sayıda a 'ları herhangi bir yere ekliyoruz.

$$\begin{aligned} S &\rightarrow SbSbS|A|\lambda, \\ A &\rightarrow aA|\lambda. \end{aligned}$$

17. (a) Öncelikle rastgele sayıda a üretiriz, ardından eşit sayıda a ve b ekleriz.

$$\begin{aligned} S_1 &\rightarrow A_1B_1, \\ A_1 &\rightarrow aA_1|a, \\ B_1 &\rightarrow aB_1b|\lambda. \end{aligned}$$

(f) İçin $L_1 \cup L_2$, kullanın

$$S \rightarrow S_1|S_2.$$

Burada S_1 ve S_2 (a) ve (b) için gramerlerdir.18. (a) Problem, iki duruma ayırırsanız basitleşir, $|w| \bmod 3 = 1$ ve $|w| \bmod 3 = 2$.

$$\begin{aligned} S &\rightarrow S_1|S_2, \\ S_1 &\rightarrow aaaS_1|a, \\ S_2 &\rightarrow aaaS_2|aa. \end{aligned}$$

21. Birinci Örnek 1.14'teki gramerin tüm cümle biçimlerinin türetililebileceğini, a sayısına göre indüksiyonla gösterebiliriz.

$$S \Rightarrow aAb \Rightarrow a^2Ab^2 \Rightarrow a^3Ab^3 \xrightarrow{*} a^nAb^n,$$

için $n = 1, 2, \dots$. Bu, yalnızca üretim $A \rightarrow aAb$ kullanarak bir cümle biçimi türetebileceğimiz içindir. Bir cümle elde etmek için, üretim $A \rightarrow \lambda$ uygulayarak $a^n b^n$ dizesini elde ederiz. Böylece gramer, $n \geq 1$ için $a^n b^n$ biçiminde dizeler üretir. Üretim $S \rightarrow \lambda$ gramerde bulunduğundan, λ dilin içindedir. Bu nedenle, gramer, $L(G_1)$ dilini Örnek 1.14'te üretir.

24. İlk gramer, $S \Rightarrow SS^* \Rightarrow aa$ türetebilir, ancak ikinci gramer bu dizeyi türetemez. Bu nedenle eşdeğer değildir.

Bölüm 1.3

1.

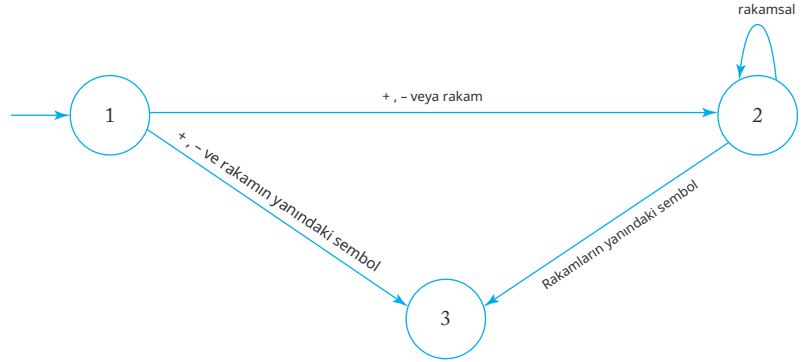
$$\text{şifre} \rightarrow \Omega \text{ harf } \Omega \text{ rakam } \Omega | \Omega \text{ rakam } \Omega \text{ harf } \Omega$$

$$\text{harf} \rightarrow a|b|\dots|z$$

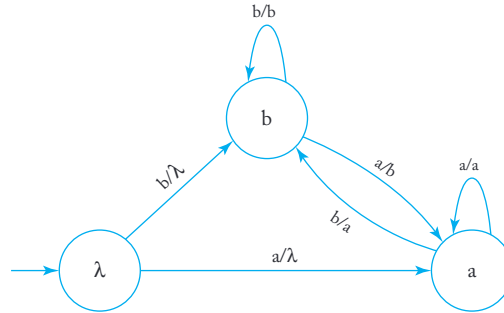
$$\text{rakam} \rightarrow 0|1|2|3|4|5|6|7|8|9$$

$$\Omega \rightarrow \text{harf } \Omega | \text{rakam } \Omega | \lambda.$$

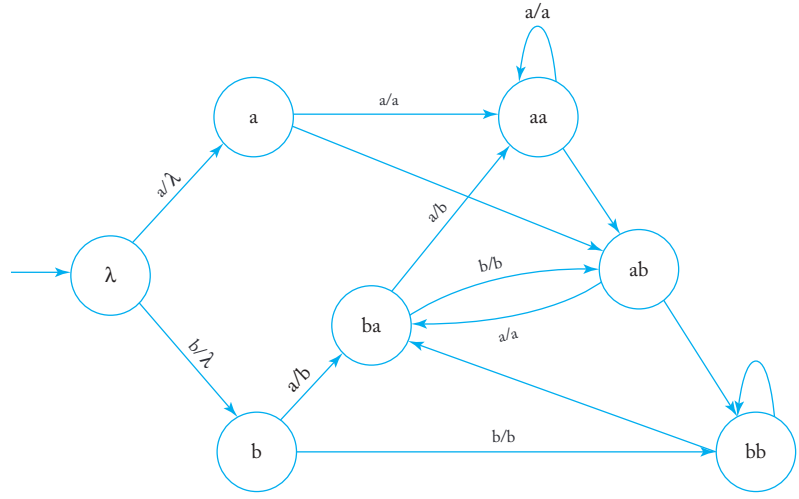
4. Tüm C tam sayılarını kabul eden bir otomaton aşağıda verilmiştir



9. Otomaton, girişi bir zaman dilimi boyunca hatırlamalıdır. Yenide nhatırlama, durumu uygun bilgi ile etiketleyerek yapılabilir. Dur umun etiketi daha sonra çıktı olarak üretilir.



10. (a) Egzersiz 9'a benzer şekilde, otomatonun girişi iki zaman dilimi boyu nca hatırlaması gerekiyor.Bu nedenle durumları iki sembolle etiketliyor uz. Problemin otomatonu aşağıda verilmiştir.



(b) n birim gecikme transdüseri için, girişı n zaman dilimi boyunca hatırlamamız gerekiyor. Durumları mnemonik olarak $a_1, a_2, \dots, a_n$ ile etiketliyoruz, burada $a_i \in \Sigma$. Her pozisyon için $|\Sigma|$ seçim olduğundan, en az $|\Sigma|^n$ durum olmalıdır.

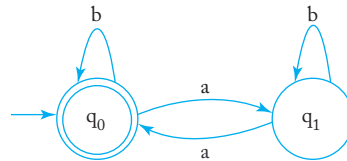
Bölüm 2

Bölüm 2.1

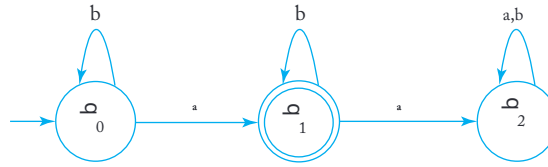
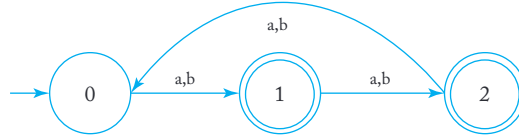
2.

$$\begin{aligned}
 \delta(\lambda, 1) &= \lambda, & \delta(\lambda, 0) &= 0, \\
 \delta(0, 1) &= \lambda, & \delta(0, 0) &= 00, \\
 \delta(00, 0) &= 00, & \delta(00, 1) &= 001, \\
 \delta(001, 0) &= 001, & \delta(001, 1) &= 001.
 \end{aligned}$$

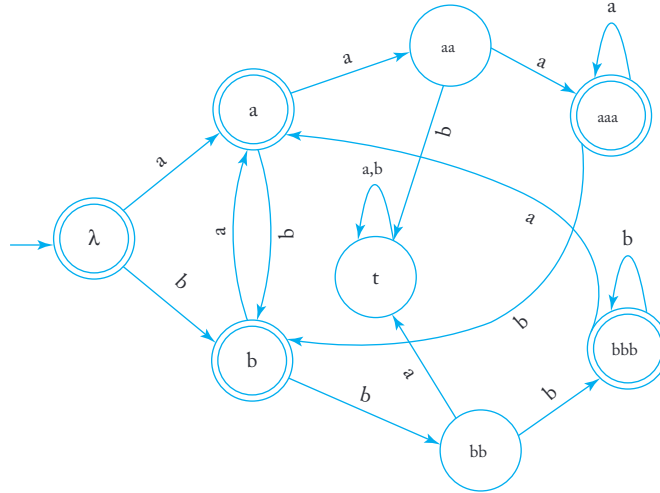
3. (c)



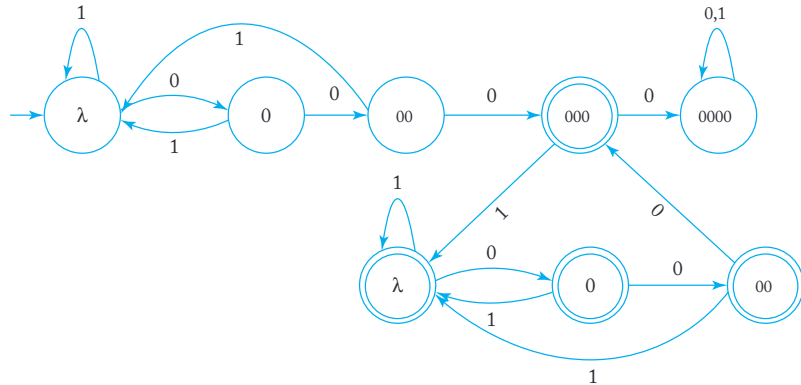
4. (a)

7. (a) Durumları $|w| \bmod 3$ ile etiketleyin.

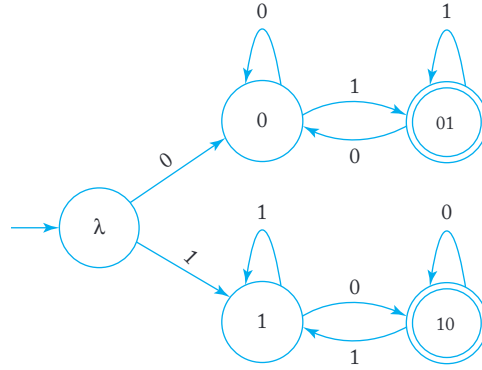
8. (a) Durumları etiketlemek için uygun a'lar ve b'ler kullanın.



11. (b)



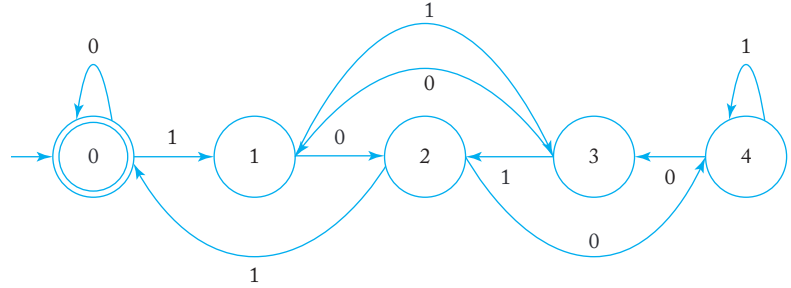
(c) En soldaki sembolü hatırlamamız ve farklı bir sembolle karşılaştığımızda dfa'yı son duruma koymamız gerekiyor



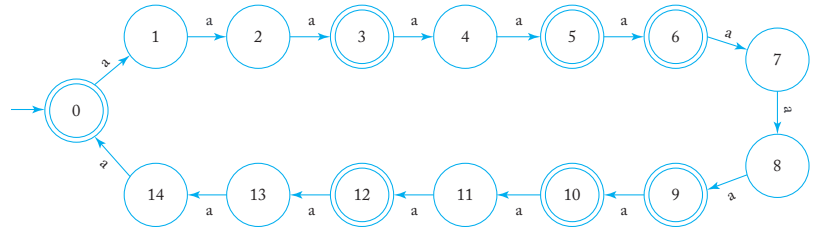
12. Hile, durumları kısmi bit dizisinin değeri (mod 5) etiketlemek ve bir sonraki biti

$$(2n + 1) \bmod 5 = (2n \bmod 5 + 1) \bmod 5.$$

bu, aşağıda gösterilen çözüme yol açar.



16. 3 ve 5'in en küçük ortak çarpanı 15 olduğundan, böyle bir dfa oluşturmak için 15 duruma ihtiyacımız var.



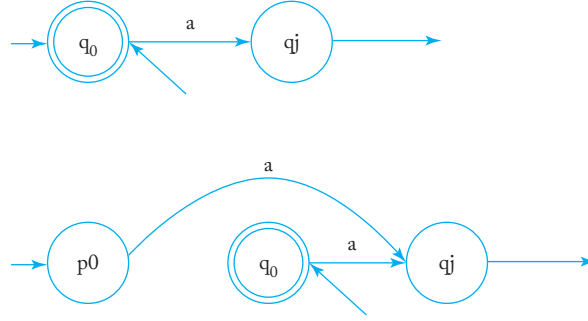
19. Yakalama, eğer λ dilde ise, başlangıç durumunun da son durum olması gerektiğidir. Ancak, diğer girdiler bu duruma ulaşabileceğinden, onu sadece son olmayan bir durum yapamayız. Bu zorluğun üstesinden gelmek için, orijinal dfa'yı yeni bir başlangıç durumu p_0 ve yeni geçişler oluşturarak değiştiriyoruz.

$$\delta(p_0, a) = q_j$$

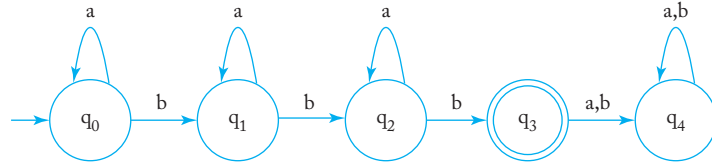
tüm orijinal geçişler için

$$\delta(q_0, a) = q_j.$$

Bu sürecin grafiksel bir temsili aşağıda verilmiştir.

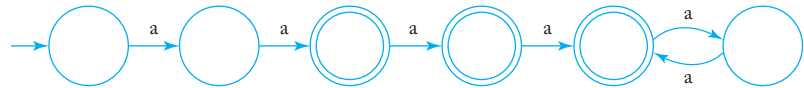


22. $L^3 = \{a^n b a^m b a^p b : n, m, p \geq 0\}$. Bu dili kabul eden bir dfa aşağıda verilmiştir



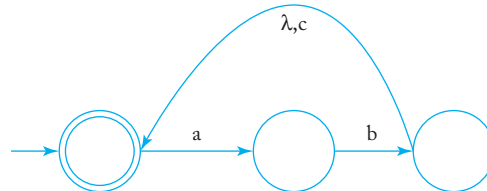
Bölüm 2.2

3. Şekil 2.8'deki nfa, dili kabul eder $\{a^n : n = 3 \text{ veya } n \text{ çift}\}$.
Bu dil için bir dfa şudur



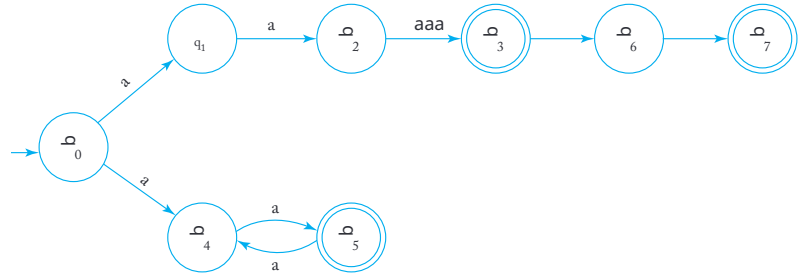
6. $\delta^*(q_0, a) = \{q_0, q_1, q_2\}$ ve $\delta^*(q_1, \lambda) = \{q_0, q_1, q_2\}$.

9.

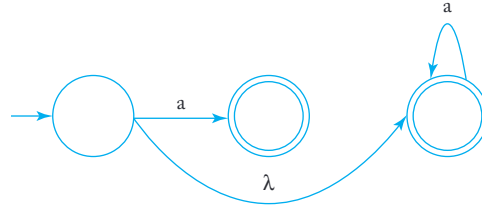


13. 01001 ve 000 kabul edilen tek iki dizedir.

15. nfa'nın a^3 ile kabul etmesinden sonra iki durum ekleyin her iki yeni kenar a ile etiketlenmiştir.



17. Aşağıdaki grafikten λ -kenarını kaldırın.

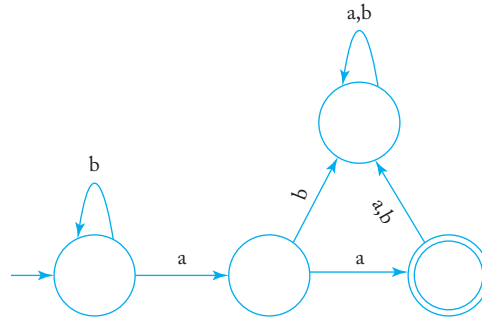


19. Yeni bir başlangıç durumu p_0 tanıtin. Ardından bir geçiş ekleyin

$$\delta(p_0, \lambda) = Q_0.$$

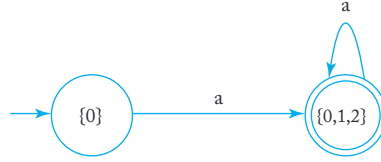
Sonra, başlangıç durumu durumunu Q_0 'dan kaldırın. Yeni nfa'nın o rijinal olanla eşdeğer olduğunu görmek basittir.

22. Kabul etmeyen bir tuzak durumu tanıtin ve tüm tanımsız geçişleri bu yeni duruma yapın.



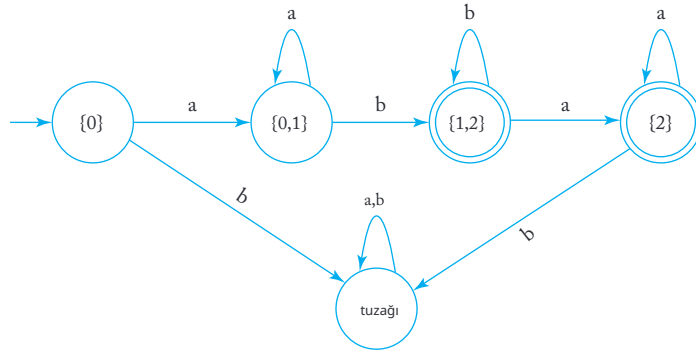
Bölüm 2.3

1.



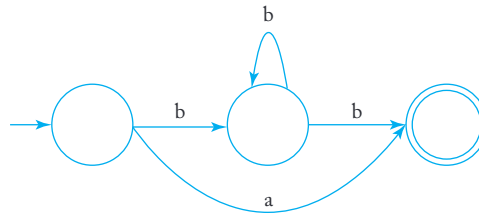
Basit bir cevap için, dilin $\{a^n : n > 1\}$ olduğunu unutmayın.

3.



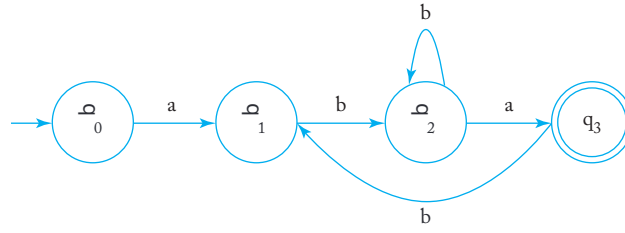
7. Evet, bu doğru. Tanıma göre, $w \in L$ yalnızca $\delta^*(q_0, w) \cap F \neq \emptyset$.
Sonuç olarak, eğer $\delta^*(q_0, w) \cap F = \emptyset$, o zaman $w \notin L$.

10.

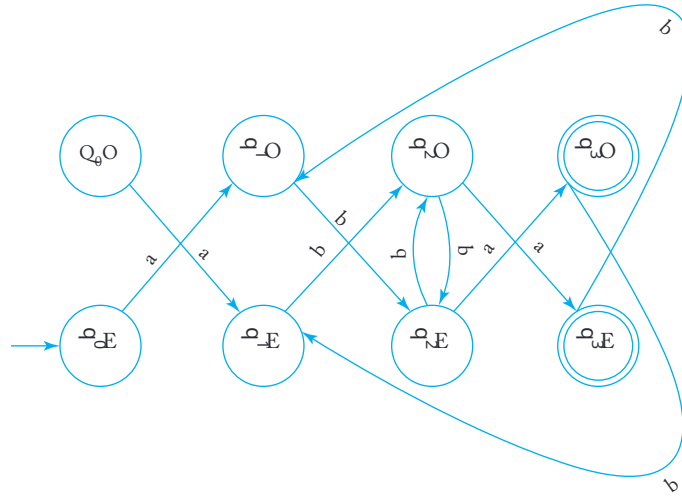


12. Tek bir başlangıç durumu tanıtın ve bunu önceki başlangıç durumlarına λ -geçişleri ile bağlayın. Ardından bir dfa'ya geri dönün ve Teorem 2.2'nin inşasının tek başlangıç durumunu koruduğunu not edin.

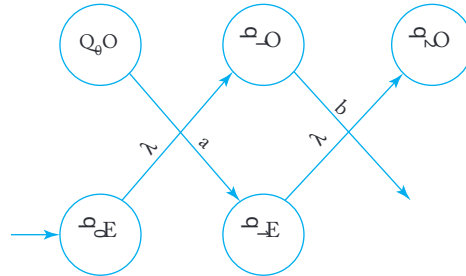
16. Numaraların çift mi yoksa tek mi okunduğunu hatırlayacak şekilde bir dfa kullanmak ve bunu değiştirmek hiledir. Bu, durum sayısını i ki katına çıkararak ve etiketlere O veya E ekleyerek yapılabilir. Örneğin, eğer dfa'nın bir kısmı



eşdeğeri



Şimdi her E durumundan O durumuna geçişi λ -geçişler.



Birkaç örnek, orijinal dfa'nın

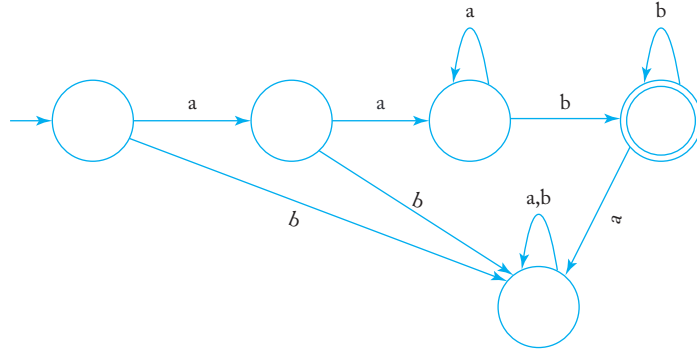
$a_1a_2a_3a_4$ kabul ettiğini gösterirse, yeni otomatonun $\lambda a_2\lambda a_4\ldots$ kabul edeceğini ve dolayısıyla $even(L)$ olduğunu ikna edecektir.

Bölüm 2.4

1. Prosedürü *mark* kullanarak, eşdeğer sınıfları $\{q_0, q_1\}$, $\{q_2\}, \{q_3\}$ oluşturuyoruz. Ardından prosedür *reduce* şunu verir

$$\begin{aligned}\hat{\delta}(01, a) &= 2, & \hat{\delta}(01, b) &= 2, \\ \hat{\delta}(2, a) &= 3, & \hat{\delta}(2, b) &= 3, \\ \hat{\delta}(3, a) &= 3, & \hat{\delta}(3, b) &= 01.\end{aligned}$$

3. (a) Aşağıdaki grafik beş durumlu bir dfa'dır. dfa'nın dilin $L = \{a^n b^m : n \geq 2, m \geq 1\}$ kabul ettiğini görmek kolay olmalıdır.



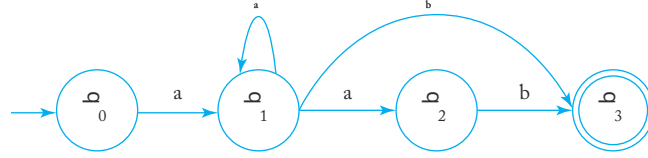
Biz bunun minimal bir dfa olduğunu iddia ediyoruz. Çünkü $q_3 \in F$ ve $q_4 \notin F$, bu yüzden q_3 ve q_4 ayırt edilebilir. Sonra $\delta(q_4, b) = q_4 \in F$ ve $\delta(q_2, b) = q_3 \in F$, bu yüzden q_2 ve q_4 ayırt edilebilir. Benzer şekilde, $\delta^*(q_0, ab) = q_4 \in F$ ve $\delta^*(q_1, ab) = q_3 \in F$, bu yüzden q_0 ve q_1 ayırt edilebilir. Bu şekilde devam edersek, beş durumun birbirine ayırt edilebilir olduğunu görebiliriz. Bu nedenle, dfa minimaldir.

6. Evet, bu doğru. Çelişki ile argüman yapıyoruz. Farz edelim ki $\widehat{M}$ minimal değildir. O zaman L kabul eden daha küçük bir dfa $\widetilde{M}$ inşa edebiliriz. $\widetilde{M}$ içinde, son durum kümesini tamamlayarak L için bir dfa elde edin. Ancak bu dfa M 'ye göre daha küçüktür, bu da M 'nin minimal olduğu varsayımıyla çelişiyor.
9. Farz edelim ki q_b ve q_c ayırt edilemez. Çünkü q_a ve q_b ayırt edilemez ve ayırt edilemezlik, Egzersiz 7'de gösterildiği gibi bir eşitlik ilişkisidir, bu yüzden q_a ve q_c ayırt edilemez olmalıdır. Bu varsayımla çelişiyor.

Bölüm 3

Bölüm 3.1

3.



5. Evet, çünkü $((0 + 1)(0 + 1)^*)^*$ 0 ve 1'lerin herhangi bir dizisini belirtir.

7. Bir düzenli ifade, $aaaa^*(bb)^*b$.

9. (a) Durumları ayırın $m = 0, 1, 2, 3, 4$. 3 veya daha fazla a üretin, ardından gerekli sayıda b ekleyin. L1 için bir düzenli ifade $aaaa^*(\lambda + b + bb + bbb + bbbb)$

16. Tüm durumları $|v| = 2$ ile sayın

$$aa(a+b)^*aa + ab(a+b)^*ab + ba(a+b)^*ba + bb(a+b)^*bb.$$

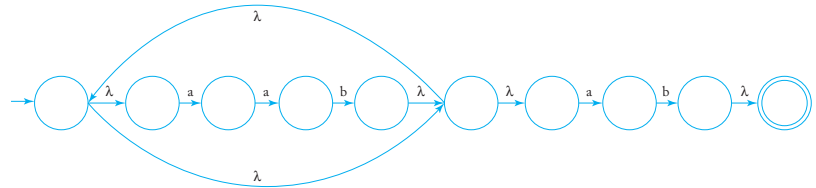
19. (a) Bir düzenli ifade, $(b+c)^*a(b+c)^*a(b+c)^*$.

26. $(rd)^*$ için grafik

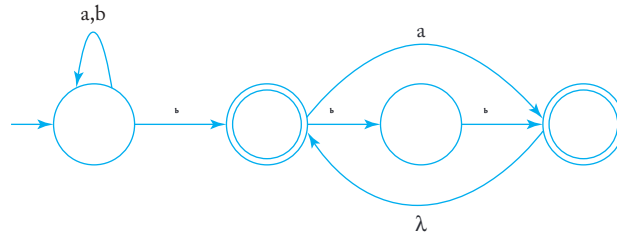


Bölüm 3.2

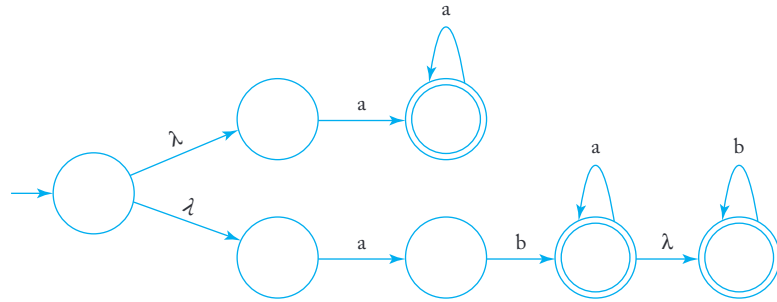
2.



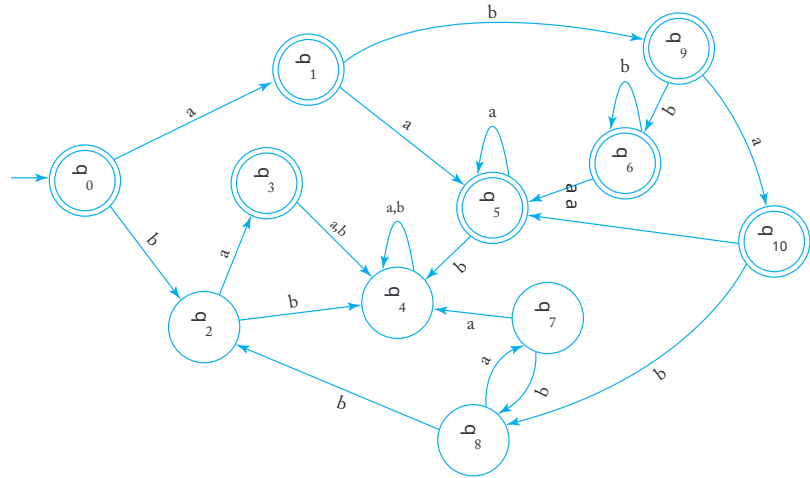
5. Bu, düzenli ifadeyi nfa yapısına geçmeden ilk ilkelerden çözülebilir. İkincisi, elbette, çalışır ama daha karmaşık bir cevap verir.



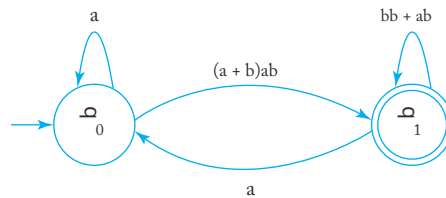
6. (a)



7. (a)



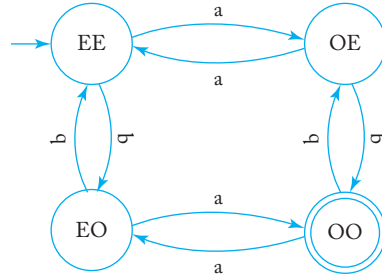
10. (a) Orta köşenin çıkarılması



(b) Denklem (3.1) ile kabul edilen $\text{dil}L(r)$ dir, burada

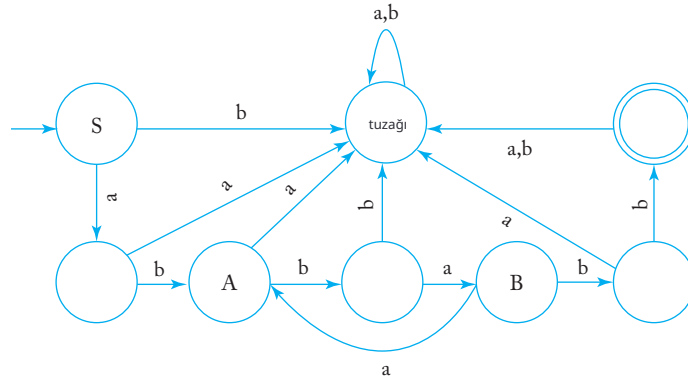
$$r = a^*(a + b)ab(bb + ab + aa^*(a + b)ab)^*.$$

15. (a) Köşeleri, a'nın çift sayıda ve b'nin çift sayıda olduğunu belirtmek için EE ile, a'nın çift sayıda ama b'nin tek sayıda olduğunu belirtmek için OE ile etiketleyin ve devam edin. Böylece genel geçiş grafiğini elde ederiz



Bölüm 3.3

1.



4. Egzersiz 1'deki $\text{dil}L = \{abba(aba)^*bb\}$. Bir soldan-lineer gramer, düzenli ifadeyi ters sırayla alarak oluşturulabilir.

$$S \rightarrow Abb,$$

$$A \rightarrow Aaba|B,$$

$$B \rightarrow abba.$$

6. $L(aaab^*ab)$ için bir gramer elde etmek oldukça basittir. Bu dilin kapanışı yalnızca küçük bir değişiklik gerektirir.

$$S \rightarrow aaaA|\lambda,$$

$$A \rightarrow bA|B,$$

$$B \rightarrow ab|abS.$$

8. İndüksiyonla gösterebiliriz ki eğer w ,

$\hat{G}$ ile türetilmiş bir cümle biçimiye, o zaman w^R aynı adım sayısı ile $\hat{G}$ ile türetilir.

Çünkü w soldan lineer türetmelerle oluşturulmuştur, bu nedenle formu $w = Aw_1$ şeklindedir, burada $A \in V$ ve $w_1 \in T^*$. İndüktif varsayıma göre $w^R = w_1^R A^R$ ile türetilir. Şimdi $A \rightarrow Bv$ uygularsak, o zaman

$$w \Rightarrow Bvw_1.$$

Ancak $\hat{G}$ kuralı $A \rightarrow v^R B$ içerdiğinden, türetmeyi yapabiliriz

$$\begin{aligned} w^R &\rightarrow w_1^R \\ &= (Bvw_1)^R \end{aligned}$$

indüktif adımı tamamlama.

12. Mnemonic etiketler kullanarak bir nfa çizin. Ardından

Teorem 3.4'teki yapıyı kullanın. Bir cevap

$$EE \rightarrow aOE|bEO,$$

$$OE \rightarrow aEE|bOO|\lambda,$$

$$OO \rightarrow bEO,$$

$$EO \rightarrow bOO|\lambda.$$

14. (b) a ve b için çiftleri belirtmek üzere mnemonic etiketler kullanarak, problem için bir dfa elde ederiz. Dfa'dan, grameri elde ederiz

$$EE \rightarrow aOE|bEO|\lambda,$$

$$OE \rightarrow aEE|bOO,$$

$$EO \rightarrow aOO|bEE,$$

$$OO \rightarrow aEO|bOE.$$

15. Birden fazla terminale sahip olan sağ-lineer bir gramerde her kural için, terminalleri sırasıyla değişkenler ve yeni kurallarla değiştirin. Örneğin, için

$$A \rightarrow a_1 _2 B,$$

değişken C'yi tanıtır ve yeni üretimler

$$A \rightarrow a_1 C,$$

$$C \rightarrow _2 B.$$

Bölüm 4

Bölüm 4.1

2. $(L_1$

$\bigcup_{L_2 \in L_1} L_2$ için düzenli bir ifade

$$((ab^*aa) + (a^*bba^*))^*(a^*bba^*).$$

4. DeMorgan yasasına göre, $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$. Yani, eğer dil ailesi tamamlayıcı ve birleşim altında kapalıysa, $L_1 \cap L_2$ bu

ailede olmalıdır.

9. İndüksiyonla. Teorem 4.1'den, iki düzenli dilin birleşiminin düzenli bir dil olduğunu biliyoruz. Şimdi bunun herhangi bir $k \leq n-1$ için doğru olduğunu varsayalım,

$$\bigcup_{i \in \{1, \dots, k\}} L_i$$

düzenli bir dildir. O zaman

$$L_U = \bigcup_{i \in \{1, \dots, n\}} L_i = \left(\bigcup_{i \in \{1, 2, \dots, n-1\}} L_i \right) \cup L_n$$

iki düzenli dilin birleşimidir. Bu nedenle L_U düzenli bir dildir.

Düzenli dillerin kesişimi için benzer bir argüman L_I bir düzenli dildir. Yani düzenli diller sonlu

kesişim altında kapalıdır.

11. Dikkat edin ki

$$\text{nor}(L_1, L_2) = \overline{L_1 \cup L_2}.$$

Sonuç, kesişim ve tamamlayıcı altında kapalı olmaktan kaynaklanır.

16. Cevap evet. Küme kimliğinden başlayarak elde edilebilir.

$$L_2 = ((L_1 \cup L_2) \cap \overline{L_1}) \cup (L_2 \cap \overline{L_1}).$$

Ana gözlem, L_1 sonlu olduğundan, $L_1 \cap L_2$ sonludur ve bu nedenle tüm L_2 için düzenlidir. Geri kalan ise birleşim ve tamamlayıcı altında bilinen kapalı durumlardan kolayca çıkar.

17. If $r \in L$ için bir düzenli ifade ise, ve $s \in \Sigma$ için bir düzenli ifade ise, then $rs \in L$ için bir düzenli ifade ve L düzenlidir.

20. Kullan $L_1 = \Sigma^*$. O zaman, herhangi bir L_2 için, $L_1 \cup L_2 = \Sigma^*$, bu da düzenlidir. Verilen ifade, herhangi bir L_2 'nin düzenli olduğunu ima eder.

22. Aşağıdaki yapıyı kullanabiliriz. Başlangıç köşesinden P 'nin bazı elemanlarına giden bir yol olan tüm durumları P bul ve o elemandan bir son duruma git. Sonra P 'nin her elemanını bir son durumu yap.

27. Varsayalım $G_1 = (V_1, T, S_1, P_1)$ ve $G_2 = (V_2, T, S_2, P_2)$. Genel bir kayıp olmaksızın, V_1 ve V_2 'nin ayık olduğunu varsayabiliriz. İki grameri birleştir ve

(a) S yeni başlangıç sembolü yap ve üretimleri ekle $S \rightarrow S_1 | S_2$.

(b) P_1 'de, her üretimi $A \rightarrow x$, ile değiştir, $A \in V_1$ ve $x \in T^*$, $A \rightarrow x S_2$.

(c) P_1 'de, her üretimi $A \rightarrow x$, ile değiştir, $A \in V_1$,
ve $x \in T^*$, $A \rightarrow x S_1, S_1 \rightarrow \lambda$.

Bölüm 4.2

1. $\widehat{M}^y i L$ için kullanın, başlangıç ve son durumları değiştirin ve kenarları ters çevirin $\widehat{M}^y i L^R$ için elde etmek için. Teorem 4.5'e göre, $w \in L^R$ 'yi kontrol edebiliriz, ya da eşdeğer olarak $w^R \in L$ için.
4. Çünkü $L_1 - L_2$ düzenlidir, Teorem 4.5'e göre, böyle bir algoritma vardır.
7. Bir dfa oluşturun. O zaman $\lambda \in L$ eğer ve ancak $q_0 \in F$.
10. Çünkü L^* ve L her ikisi de düzenlidir, Teorem 4.7'ye göre, düzenli dillerin eşitliğini test eden bir algoritmamız var.
13. Eğer L içinde çift uzunlukta dizeler yoksa, o zaman

$$L((aa + ab + ba + bb)^*) \cap L = \emptyset.$$

Bu nedenle, Teorem 4.6'ya göre sonuç ortaya çıkar

17. $L(G_1)$ için bir dfa M_1 ve $L(G_2)$ için bir dfa M_2 oluşturun. Sonra Teorem 3.1'deki yapıyı kullanarak $L = L(G_1) \cup L(G_2)$ için bir nfa elde edin. L ve Σ^* için eşitliği kontrol edin

Bölüm 4.3

Varsayalım L düzenlidir ve bize m verildi, o zaman $w = a^m b c^m$ seçiy oruz L içinde. Şimdi çünkü $|xy| \leq m$ den büyük olamaz, açıkça $y = a^k b$ azı $k \geq 1$ için tek olası seçimdir. O zaman pompalı dizeler

$$w_i = a^{m+(i-1)k} b c^m \in L,$$

tümü $i = 0, 1, \dots$. Ancak, $w_2 = a^{m+k} b c^m \notin L$. Bu çelişki L 'nin düzenli bir dil olmadığını gösterir.

3. Homomorfizma altında kapanışı düzenli diller için kullanın. Alın

$$h(a) = a, \quad h(b) = a, \quad h(c) = c, \quad h(d) = c,$$

sonra

$$\begin{aligned} h(L) &= \{a^{n+k} c^{n+k} : n+k > 0\} \\ &= \{a^i c^i : i > 0\} \end{aligned}$$

Örnek 4.7'deki argümana göre, $h(L)$ düzenli değildir. Bu nedenle L düzenli değildir.

5. (a) Varsayalım L düzenlidir ve m verilmiştir, o zaman $w = a^m b^m c^{2m} L$ içinde seçiyoruz. Şimdi çünkü $|xy|_m$ ' den büyük olamaz, bu nedenle $y = a^k$ bazı $k \geq 1$ için tek olası seçimdir. Bu nedenle pompalama lemması ile

$$w_i = a^{m-k} b^m c^{2m} \in L,$$

tümü $i = 0, 1, \dots$ Ancak, $w_0 = a^{m-k} b^m c^{2m} \in L$ çünkü $m - k + m < 2m$. Bu nedenle L düzenli değildir.

- (f) Varsayalım L düzenlidir ve m verilmiştir. O zaman $w = a^m b a^m b L$ içinde seçiyoruz. String y o zaman a^k olmalı ve pompalanmış stringler

$$w_i = a^{m+(i-1)k} b a^m b \in L,$$

için $i = 0, 1, \dots$ Ancak, $w_0 = a^m b a^m b \notin L$. Bu nedenle L düzenli değildir.

6. (a) Varsayalım L düzenlidir ve m verilmiştir. En küçük asal p sayısı $p \geq m$ olacak şekilde olsun. O zaman $w = a^p L$ içinde seçiyoruz. String y o zaman a^k olmalı ve pompalanmış stringler

$$w_i = a^{p+(i-1)k} \in L,$$

için $i = 0, 1, \dots$ Ancak, $w_{p+1} = a^{p+p} = a^{p(1+k)} \notin L$. Bu nedenle L düzenli değildir.

7. (a) Çünkü

$$L = \{a^n b^n : n \geq 1\} \cup \{a^n b^m : n \geq 1, m \geq 1\}$$

bu açıkça düzenlidir.

12. Diyelim ki L düzenlidir ve m verilmiştir. Biz $w = a^{m!}$ seçiyoruz ki bu L içindedir. Dize y o zaman bazı $1 \leq k \leq m$ için a^k olmalıdır ve pompalanmış

stringler şunlar olacaktır

$$w_i = a^{m! + (i-1)k} \in L,$$

for $i = 0, 1, \dots$ Ancak, $w_2 = a^{m!+k} \notin L$, çünkü $m! + k \leq m! + m < (m+1)!$. Bu nedenle L düzenli değildir.

18. (a) Dil düzenlidir. Bu, en kolay şekilde problemi $l = 0, k = 0, n > 5$ gibi durumlara ayırarak görülebilir; bu durumda düzenli ifa deler kolayca oluşturulabilir.

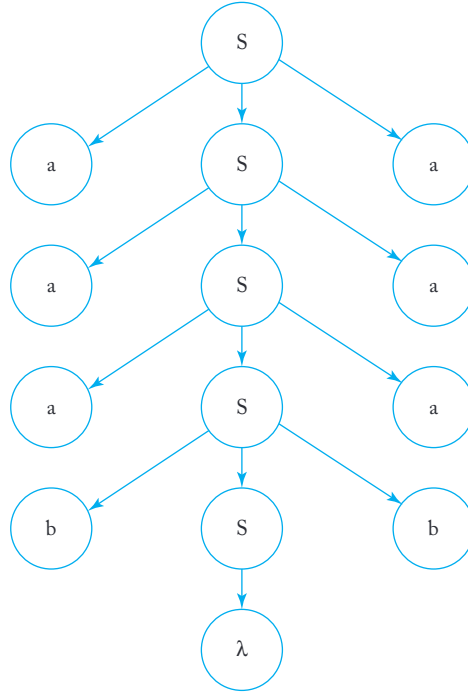
- (e) Bu dil düzenli değildir. Farz edelim m verildi, biz $w = a^m b^{m+1}$ olarak seçiyoruz. Böylece, rakibimiz yalnızca $y = a^k$ seçebilir, bazı $1 \leq k \leq m$. Pompalanan dizeler $w_i = a^{m+(i-1)k} b^{m+1}$. Yani, örneğin, $w_3 = a^{m+2k} b^{m+1}$ gerekli koşulu ihlal eder çünkü $m+1 < m+2k$.

24. $AlL_i = \{a^i b^i\}, i = 0, 1, \dots$. Her i için, L_i sonludur ve bu nedenle düzenlidir, ancak tüm dillerin birleşimi düzensiz dil $L = \{a^n b^n : n \geq 0\}$.

Bölüm 5

Bölüm 5.1

- (b) $S \rightarrow aAb, A \rightarrow aaAbb | \lambda$.
- Bir türetim ağacı şudur:



- (a) $S \rightarrow aSb | A | B, A \rightarrow \lambda | a | aa | aaa, B \rightarrow bB | b$.
 (e) İki parçaya ayırılır: (i) $n_a(w) > n_b(w)$ ve (ii) $n_b(w) > n_a(w)$.
 Parça (i) için, eşit sayıda a ve b üretin, ardından daha fazla a ekleyin.
 Parça (ii) için aynı şeyi yaparak daha fazla b ekleyin.

$$\begin{aligned}
 S &\rightarrow S_1 | S_2, \\
 S_1 &\rightarrow S_1 S_1 | a S_1 b S_1 \quad A, \\
 &\rightarrow a A | a, \\
 S_2 &\rightarrow S_2 S_2 | a S_2 | b S_2 | B, \\
 &\rightarrow b B |
 \end{aligned}$$

10. Gramer tarafından üretilen dil, özyinelemeli olarak tanımlanabilir

şu şekilde

$$L = \{w : w = aub \text{ or } w = bua \text{ with } u \in L, \lambda \in L\}.$$

12. (a) Problemi iki parçaya ayırıyoruz: (i) $L_1 = \{a^n b^m c^k : n=m\}$ ve (ii) $L_2 = \{a^n b^m c^k : m \leq k\}$. Ardından $S \rightarrow S_1 | S_2$ kullanın, burada $S_1 L_1$ türetir ve $S_2 L_2$ türetir. Aşağıda bir gramer verilmiştir.

$$S \rightarrow S_1 C | A S_2,$$

$$S_1 \rightarrow a S_1 b | \lambda,$$

$$S_2 \rightarrow b S_2 c | C,$$

$$A \rightarrow a A | \lambda,$$

$$C \rightarrow c C | \lambda.$$

(e)

$$S \rightarrow a S c | B, \quad B \rightarrow b B c c | \lambda.$$

- 15.

$$S \rightarrow a S b | S_1,$$

$$S_1 \rightarrow a S_1 a | b S_1 b | \lambda.$$

16. (a) Eğer SL türetiyorsa, o zaman $S_2 \rightarrow S S L^2$ türetir.

26. Gramerin değişkenleri $\{S, V, R, T\}$ ve terminalleri

$$T = \{a, b, A, B, C, \rightarrow\}$$

üretimlerle

$$S \rightarrow V \rightarrow R$$

$$R \rightarrow T R | \lambda$$

$$V \rightarrow A | B | C,$$

$$T \rightarrow a | b | A | B | C | \lambda.$$

Bölüm 5.2

2. Herhangi bir $w \in L(G)$ için, G bir s-gramer olduğunda, en soldaki türetme için benzersiz bir seçim olduğu açıktır. Bu nedenle her s-gramer belirgin değildir.

- 4.

$$S \rightarrow a S_1, \quad S_1 \rightarrow a A B, \quad A \rightarrow a A B | b, \quad B \rightarrow b.$$

8. Burada $w = aaab$ için iki en soldaki türetme bulunmaktadır.

$$S \Rightarrow aa a B \Rightarrow aa a b,$$

$$S \Rightarrow A B \Rightarrow A a B \Rightarrow A a a B \Rightarrow aa a B \Rightarrow aa a b.$$

13. Bir düzenli dil için dfa'dan, Teorem 3.4 yöntemini kullanarak bir düzenli gramer elde edebiliriz. Gramer, $q_f \rightarrow \lambda$ dışındaki tüm kurallarıyla bir s-g ramerdir. Ancak bu kural herhangi bir belirsizlik yaratmaz. Çünkü dfa a sla bir seçim yapmaz, bu nedenle uygulanabilecek üretimde asla bir seçim yoktur.
15. Evet, örneğin, $S \rightarrow aS_1 | ab$. $S_1 \rightarrow b$. $\text{Dil } L(ab)$ sonludur, bu nedenle düzenlidir. Ama $S \Rightarrow ab$ ve $S \Rightarrow aS_1 \Rightarrow ab$, ab için iki farklı türetmedir.

17. abab dizesi ya

$$S \Rightarrow aSbS \Rightarrow abS \Rightarrow abaSbS \Rightarrow ababS \Rightarrow abab,$$

ya da

$$S \Rightarrow aSbS \Rightarrow abSaSbS \Rightarrow abaSbS \Rightarrow ababS \Rightarrow abab.$$

Bu nedenle, gramer belirsizdir.

Bölüm 6

Bölüm 6.1

2. Değişken B 'yi ortadan kaldırırsak, ikinci grameri elde ederiz.
6. Öncelikle, bir terminal dizesine yol açabilecek değişkenler kümesini tanımlarız. Çünkü S tek böyle değişkendir, A ve B kaldırılabilir ve biz şunu elde ederiz

$$S \rightarrow aS | \lambda$$

Bu gramer $L(a^*)$ türetir.

7. İlk olarak, C dışında her değişkenin bir terminal

string'e yol açabileceğini fark ediyoruz. Bu nedenle, dilbilgisi

$$S \rightarrow a | aA | B,$$

$$A \rightarrow aB | \lambda,$$

$$B \rightarrow Aa,$$

$$D \rightarrow ddd.$$

Sonra, başlangıç değişkeninden ulaşılamayan değişkenleri ortadan kaldırmak istiyoruz. Açıkça, D işe yaramaz ve üretim listesinden çıkarıyoruz. Ortaya çıkan dilbilgisi, işe yaramaz üretilere sahip değildir.

$$S \rightarrow a | aA | B,$$

$$A \rightarrow aB | \lambda,$$

$$B \rightarrow Aa.$$

10. Tek nullable değişken A 'dır, bu nedenle λ -üretimlerini kaldırmak şunları verir

$$\begin{aligned} S &\rightarrow aA|a|aBB, \\ A &\rightarrow aaA|aa, \\ B &\rightarrow bC|bbC, \\ C &\rightarrow B. \end{aligned}$$

$C \rightarrow B$ tek bir birim üretimdir ve onu kaldırmak şuna yol açar

$$\begin{aligned} S &\rightarrow aA|a|aBB, \\ A &\rightarrow aaA|aa, \\ B &\rightarrow bC|bbC, \\ C &\rightarrow bC|bbC. \end{aligned}$$

Son olarak, B ve C işe yaramaz, bu nedenle şunu elde ediyoruz

$$\begin{aligned} S &\rightarrow aA|a, \\ A &\rightarrow aaA|aa. \end{aligned}$$

Bu dilbilgisi tarafından üretilen dil $L((aa)^*a)$.

17. Bir örnek şudur

$$\begin{aligned} S &\rightarrow aA, \\ A &\rightarrow BB, \\ B &\rightarrow aBb|\lambda. \end{aligned}$$

λ -produksiyonlarını kaldırdığımızda elde ederiz

$$\begin{aligned} S &\rightarrow aA|a, \\ A &\rightarrow BB|B, \\ B &\rightarrow aBb|ab. \end{aligned}$$

21. Bu oldukça basit ikame, doğrudan bir argümanla haklı çıkarılabilir. İlk gramerle türetilen her dize ikinci gramerle de türetililebilir ve tersine.
25. Zor bir problem çünkü sonuç sezgisel olarak zor görünmektedir. Kanıt, bir gramer tarafından üretilen kritik cümle biçimlerinin öteki gramer tarafından da üretililebileceğini göstererek yapılabilir. Bu konudaki önemli adım, eğer

$$A^* Ax_k \dots x_j x_i \Rightarrow y_r x_k \dots x_j x_i,$$

değiştirilmiş gramerle türetim yapabileceğimizi göstermektir

$$A \Rightarrow y_r Z \Rightarrow \dots \xRightarrow{*} y_r x_k \dots x_j x_i.$$

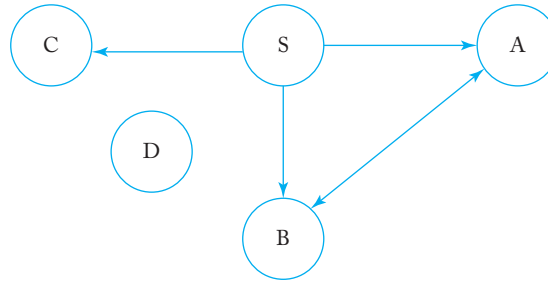
Bu aslında önemli bir sonuçtur, belirli sol-özyinelemeli üretimlerin gramerden çıkarılmasıyla ilgilidir ve bir gramerin Greibach normal formuna dönüştürülmesi için genel algoritmanın tartışılması gerekiyorsa gereklidir.

Bölüm 6.2

3. Birinci birim üretimini ortadan kaldır: $S \rightarrow aSaaA|abA|bb$, $A \rightarrow abA|bb$.
Sonra Teorem 6.6'yı uygulayın.

$$\begin{aligned} S &\rightarrow V_a D_1 | V_a E_1 | V_b V_b, \\ D_1 &\rightarrow S D_2, \\ D_2 &\rightarrow V_a D_3, \\ D_3 &\rightarrow V_a A, \\ E_1 &\rightarrow V_b A, \\ A &\rightarrow V_a E_1 | V_b V_b, \\ V_a &\rightarrow a, \\ V_b &\rightarrow b. \end{aligned}$$

7.



8. Üretimlerle yeni bir değişken V_1 tanıtırın

$$V_1 \rightarrow a_2 \dots a_n B b_1 b_2 \dots b_m$$

ve

$$A \rightarrow a_1 V_1.$$

Bu süreci devam ettirin, V_2 ve

$$V_2 \rightarrow a_3 \dots a_n B b_1 b_2 \dots b_m$$

ve devam edin, sol tarafta terminal kalmayana kadar. Sonra sağdaki terminalleri kaldırmak için benzer bir süreç kullanın.

10. Yeni deęişkenler $V_a \rightarrow a$ ve $V_b \rightarrow b$ tanıtarak, Greibach

normal formunu hemen elde ediyoruz.

$$S \rightarrow aSV_b | bSV_a | a | b | aV_b,$$

$$V_a \rightarrow a,$$

$$V_b \rightarrow b.$$

13. Gramerdeki birim üretimi $A \rightarrow B$ kaldırarak, yeni üretim kuralı $A \rightarrow aaA | bAb$ elde ediyoruz. Sonra yeni üretimleri S içine yerleştirerek, eşdeğer bir gramer elde ediyoruz.

$$S \rightarrow aaABb | bAbBb | a | b,$$

$$A \rightarrow aaA | bAb,$$

$$B \rightarrow bAb.$$

Üretimleri V_a, V_b tanıtarak, grameri

Greibach normal formuna dönüştürebiliriz.

$$S \rightarrow aV_aABV_b | bAV_bBV_b | a | b,$$

$$A \rightarrow aV_aA | bAV_b,$$

$$B \rightarrow bAV_b,$$

$$V_a \rightarrow a,$$

$$V_b \rightarrow b.$$

Bölüm 6.3**4. Öncelikle grameri Chomsky normal formuna dönüştürün.**

$$S \rightarrow AC | b,$$

$$C \rightarrow SB,$$

$$B \rightarrow b,$$

$$A \rightarrow a.$$

Daha sonra dönüştürülmüş gramere göre V_{ij} 'leri $aaabbbb$ dizesi için hesaplaya biliriz.

$$V_{11} = V_{22} = V_{33} = \{A\}, \quad V_{44} = V_{55} = V_{66} = V_{77} = \{S, B\},$$

$$V_{12} = V_{23} = V_{34} = \emptyset, \quad V_{45} = V_{56} = V_{67} = \{C\},$$

$$V_{13} = V_{24} = V_{46} = V_{57} = \emptyset, \quad V_{35} = \{S\},$$

$$V_{14} = V_{25} = V_{47} = \emptyset, \quad V_{36} = \{C\},$$

$$V_{15} = V_{37} = \emptyset, \quad V_{26} = \{S\},$$

$$V_{16} = \emptyset, \quad V_{27} = \{C\},$$

$$V_{17} = \{S\}.$$

Çünkü $S \in V_{17}$, dize $aaabbbb$ verilen gramer tarafından üretilen dildir.

Bölüm 7

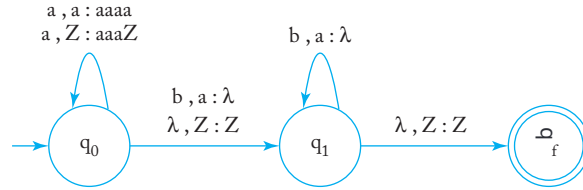
Bölüm 7.1

3. (c) Yeni pda'nın başlangıç durumu p_0 olsun ve q ile q' sırasıyla (a) ve (b)'nin başlangıç durumlarını belirtmektedir. O zaman ayarlıyoruz

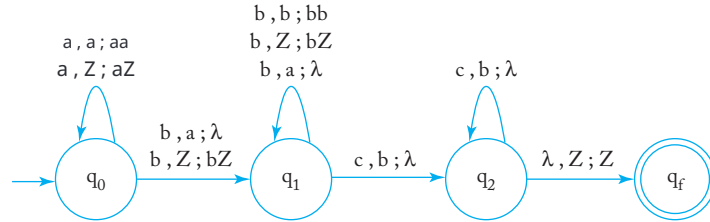
$$\delta(p_0, \lambda, z) = \{(q_0, z), (q'_0, z)\}.$$

4. Durum q_1 için bir ihtiyaç yoktur. Nerede q_1 geçiyorsa q_0 ile değiştirin. Ana zorluk, bu nun işe yaradığını ikna edici bir şekilde savunmaktır.

6. (a)



- (d) Yığıtta başlangıçta bir z ile başlayın. Giriş sembolü bir a olduğunda yığıta bir işaret a koyun giriş bir a olduğunda bir işaret a tüketin b . Yığın içindeki tüm a 'lar tüketildiğinde, girdi b olduğunda yığında bir işaret b koyun. Girdi c olduğunda, yığındaki bir işaret b kaldırın. Girdi tamamlandığında yığında yalnızca z kaldıysa, dize kabul edilecektir.



8. İlk bakışta bu basit bir problem gibi görünüyor: yığına w_1 koyun, sonra w_2 ile bir uyumsuzluk tespit edildiğinde son bir tuzak durumun a geçin. Ancak bu, iki alt dize eşit uzunlukta değilse eksiktir, örneği n , eğer $w_2 = w_1^R v$. Bunu çözmenin bir yolu, problemi $w_1 = w_2$ ve $w_1 / = w_2$ olarak belirsiz bir şekilde bölmektir.

18. Burada yığına konulacak sembolleri hatırlamak için iç durumlar kullanıyoruz. Örneğin,

$$\delta(q_i, a, b) = \{(q_j, cde)\}$$

şu ile değiştirilir

$$\begin{aligned} \delta(q_i, a, b) &= \{(q_{jc}, de)\}, \\ \delta(q_{jc}, \lambda, d) &= \{(q_j, cd)\}. \end{aligned}$$

δ yalnızca sonlu sayıda eleman içerebileceğinden ve her biri yığına yalnızca sonlu miktarda bilgi ekleyebileceğinden, bu yapılandırma herhangi bir pda için uygulanabilir.

Bölüm 7.2

2. Grameri Greibach normal formuna dönüştürün.

$$S \rightarrow aSSSA|\lambda,$$

$$A \rightarrow aB,$$

$$B \rightarrow b.$$

Teorem 7.1'in yapısının ardından, çözümü elde ederiz

$$\delta(q_0, \lambda, z) = \{(q_1, Sz)\},$$

$$\delta(q_1, a, S) = \{(q_1, SSSA)\},$$

$$\delta(q_1, a, A) = \{(q_1, B)\},$$

$$\delta(q_1, \lambda, S) = \{(q_1, \lambda)\},$$

$$\delta(q_1, b, B) = \{(q_1, \lambda)\},$$

$$\delta(q_1, \lambda, z) = \{(q_f, z)\}.$$

with $F = \{q_f\}$

4. $aabb$ dizesi için npda'nın yaptığı hareketler şunlardır

$$(q_0, aabb, z) \vdash (q_1, aabb, Sz) \vdash (q_1, abb, SAz) \vdash (q_1, bb, Az) \\ \vdash (q_1, b, Bz) \vdash (q_1, \lambda, z) \vdash (q_2, \lambda, \lambda).$$

q_2 son durum olduğu için, pda $aabb$ dizesini kabul eder. Benzer şekilde $aaabbbb$ iç in de buluyoruz

$$(q_0, aaabbbb, z) \vdash (q_1, aaabbbb, Sz) \vdash (q_1, aabbbb, SAz) \vdash (q_1, abbbb, SAAz) \\ \vdash (q_1, bbbb, AAz) \vdash (q_1, bbb, BAz) \vdash (q_1, bb, Az) \\ \vdash (q_1, b, Bz) \vdash (q_1, \lambda, z) \vdash (q_2, \lambda, \lambda).$$

Bu nedenle, $aaabbbb$ pda tarafından da kabul edilir.

6. Öncelikle grameri Greibach normal formuna dönüştürün, bu da $S \rightarrow aSSS; S \rightarrow bA; A \rightarrow a$. Ardından Teorem 7.1'in yapısını izleyin.

$$\delta(q_0, \lambda, z) = \{(q_1, Sz)\},$$

$$\delta(q_1, a, S) = \{(q_1, SSS)\},$$

$$\delta(q_1, b, S) = \{(q_1, A)\},$$

$$\delta(q_1, a, A) = \{(q_1, \lambda)\},$$

$$\delta(q_1, \lambda, z) = \{(q_f, z)\}.$$

9. Teorem 7.2'den, herhangi bir npda verildiğinde, eşdeğer birbağlamdan b ağımsız gramer inşa edebiliriz. O gramere dayanarak, Teorem 7.1'i kullanarak eşdeğer üç durumlu npda inşa edebiliriz.
13. Başlamak için en az bir a olmalıdır. Bunun ardından, $\delta(q_0, a, A) = \{(q_0, A)\}$ sadece yığın değiştirmeden a 'ları okur. Son olarak, ilk b ile karşılaşıldığında, pda q_1 durumuna geçer, buradan yalnızca son duruma λ geçişi yapabilir. Bu nedenle, bir dize yalnızca bir veya daha fazla a içeriyorsa ve ardından tek bir b geliyorsa kabul edilir.

14. Örnek 7.8'deki gramerden bir türetme buluyoruz

$$\begin{aligned}
 (q_0 z a_2) &\Rightarrow a(q_0 A q_3)(q_3 z q_2) \\
 &\Rightarrow aa(q_3 z a_2) \\
 &\Rightarrow aa(q_0 A q_3)(q_3 z q_2) \\
 &\Rightarrow aaa(q_0 A q_3)(q_3 z q_2) \\
 &\xrightarrow{*} aa^*(q_0 A q_3)(q_3 z q_2) \\
 &\Rightarrow aa^*(q_3 z q_2) \\
 &\Rightarrow aa^*(q_0 A q_1)(q_1 z q_2) \\
 &\Rightarrow aa^*(q_1 z q_2) \\
 &\Rightarrow aa^*b.
 \end{aligned}$$

19. Anahtar, terminallerin büyük ölçüde değişkenler olarak ele alınabilmesidir. Örneğin, eğer

$$\rightarrow abBBc$$

o zaman pda'nın geçişlere sahip olması gerekir

$$(q_1, bBBc) \in \delta(q_1, a, A),$$

ve

$$(q_1, BBC) \in \delta(q_1, b, b),$$

where $a, b, c \in T \cup \{\lambda\}$.

Bölüm 7.3

2. Örnek 7.4'ün basit bir modifikasyonu. Girdi bir a olduğunda iki token koyun ve girdi bir b olduğunda yalnızca bir token çıkarın. Çözüm şudur:

$$\begin{aligned}
 \delta(q_0, a, z) &= \{(q_1, AAz)\}, \\
 \delta(q_1, a, A) &= \{(q_1, AAA)\}, \\
 \delta(q_1, b, A) &= \{(q_2, \lambda)\}, \\
 \delta(q_2, b, A) &= \{(q_2, \lambda)\}, \\
 \delta(q_2, \lambda, z) &= \{(q_0, z)\},
 \end{aligned}$$

where $F = \{q_0\}$.

4. Bu dpda, ön ek $a^n b^{n+3}$ okunduktan sonra son duruma geçer. Eğer daha fazla a gelirse, bu durumda kalır ve böylece herhangi bir $m > n + 3$ için $a^m b^n$ kabul eder. Tam bir çözüm aşağıda verilmiştir.

$$\begin{aligned}\delta(q_0, \lambda, z) &= \{(q_1, AAz)\}, \\ \delta(q_1, a, A) &= \{(q_1, AA)\}, \\ \delta(q_1, b, A) &= \{(q_2, \lambda)\}, \\ \delta(q_2, b, A) &= \{(q_2, \lambda)\}, \\ \delta(q_2, b, z) &= \{(q_3, z)\}, \\ \delta(q_3, b, z) &= \{(q_3, z)\},\end{aligned}$$

where $F = \{q_3\}$.

6. İlk bakışta, bu bir belirsiz dil gibi görünebilir, çünkü ön ek a iki farklı türde son ek talep ediyor. Yine de, dil deterministiktir, çünkü bir dpda inşa edebiliriz. Bu dpda, ilk girdi sembolü a olduğunda son duruma geçer. Daha fazla sembol gelirse, bu durumdan çıkar ve ardından $a^n b^n$ kabul eder. Tam bir çözüm şudur:

$$\begin{aligned}\delta(q_0, a, z) &= \{(q_3, Az)\}, \\ \delta(q_3, a, A) &= \{(q_1, AA)\}, \\ \delta(q_1, a, A) &= \{(q_1, AA)\}, \\ \delta(q_1, b, A) &= \{(q_2, \lambda)\}, \\ \delta(q_2, b, A) &= \{(q_2, \lambda)\}, \\ \delta(q_2, \lambda, z) &= \{(q_3, z)\}, \\ \delta(q_3, b, A) &= \{(q_2, \lambda)\},\end{aligned}$$

where $F = \{q_3\}$.

9. Sezgisel olarak, dize başlangıcında hangi durumu istediğimizi biliyorsak $n = m$ veya $m = k$ kontrol edebiliriz. Ancak bunu deterministik olarak karar vermemiz yoktur.
11. Çözüm basit. a 'ları ve b 'leri yığına koyun. c , kaydetmeden eşleştirmeye geçişi işaret eder, bu nedenle her şey deterministik olarak yapılabilir.
16. Let $L_1 = \{a^n b^n : n \geq 0\}$ ve $L_2 = \{a^n b^{2n} c : n \geq 0\}$. O zaman açıkça $L_1 \cup L_2$ belirsizdir. Ama bunun tersine $\{b^n a^n\} \cup \{c b^{2n} a^n\}$, ilk sembole bakarak belirli bir şekilde tanınabilir.

Bölüm 7.4

1. (c) L içindeki dizeleri şu şekilde yeniden yazabiliriz:

$$a^n b^{n+2} c^m = a^n b^n b^2 c^2 c^{m-2} = a^n b^n b^2 c^2 c^k$$

buradan $n \geq 1$, $k \geq 1$. L için bir dilbilgisi şu şekilde elde edilebilir.

$$\begin{aligned} S &\rightarrow ABC, \\ A &\rightarrow aAb|ab, \\ B &\rightarrow bbcc, \\ C &\rightarrow cC|c. \end{aligned}$$

Gramerin $LL(2)$ olduğunu iddia ediyoruz. Öncelikle, başlangıç üretimi şu olmalıdır: $S \rightarrow ABC$. $anbn$ kısmı için, eğer bakış açısı aa ise, o zaman $A \rightarrow aAb$ kullanmalıyız; eğer ab ise o zaman sadece $A \rightarrow \lambda$ kullanabiliriz. Geri kalan için, asla bir üretim seçeneği yoktur, bu nedenle gramer $LL(2)$ dir.

3. $aabb$ ve $aabbbbaa$ dizelerini düşünün. İlk durumda, türetim $S \Rightarrow aSb$ ile başlamalıdır, ikinci durumda ise $S \Rightarrow SS$ gerekli ilk adımdır. Am a sadece ilk dört sembolü görürsek, hangi durumun geçerli olduğunu belirleyemeyiz. Bu nedenle gramer $LL(4)$ te değildir. Benzer örnekler, keyfi uzunluktaki dizeler için de yapılabileceğinden, gramer herhangi bir k için $LL(k)$ değildir.

Bölüm 8

Bölüm 8.1

2. Verilen m , biz $w = a^m c^m b^m$ seçiyoruz L içinde. Rakip şimdibirkaç seçeneğe sahip. Eğer vxy yalnız cab 'leri, ya da a 'ları veya c 'leri içerecek şekilde seçerse, o zaman pompalanan dize w_i açıkça L içinde olmayacaktır bazı $i \geq 1$ için. Bu nedenle, rakip $v = a^l$ seçer. O zaman pompalanan dize $v^{m+(-1)^k} c^{m+(-1)^l} b^m$. Ancak, eğer $l = 0$ ise, $nc(w_2) = m + l > m = n_b(w_0)$, bu yüzden $w_2 \notin L$. Öte yandan, eğer $l = 0$ ise, o zaman pompalanan dize $n_a(w_0) = m - k = m = n_b(w_0)$, bu yüzden $w_0 \in L$. Benzer şekilde, eğer rakip vxy 'leri vcb 'leri içerecek şekilde seçerse, o da kazanamaz. Böylece pompalama lemması baş arısız olur ve L bağlam serbest değildir.

6. Verilen m , biz $w = a^m b^{2^m}$ seçiyoruz L içinde. Açıkça, rakip için tek seçeneğe $v = a^k$, $y = b^l$. Pompalanan dize $w_i = a^{m+(i-1)k} b^{2^{m+(i-1)l}}$. Biz w_0 ve w_2 'nin her ikisinin de L içinde olamayacağını iddia ediyoruz, bu nedenle pompalama lemması başarısız olur ve L bağlam serbest değildir. Bunu göstermek için, diyelim ki w_0 ve w_2 her ikisi de L içindedir, o zaman şunları almalıyız

$$2^{m-k} = 2^m - l,$$

ve

$$2^{m+k} = 2^m + l.$$

Şimdi yukarıdaki iki denklemi yan yana çarpalım, şu sonuca varıyoruz

$$2^{m-k} 2^{m+k} = (2^m - l)(2^m + l)$$

Yani

$$2^{2m} = 2^{2m} - l^2.$$

$l = 0$ olması $k = 0$ anlamına gelir ve bu da $|vy| = 0$ anlamına gelir. Bir çelişki var.

7. (c) m verildiğinde, $w = ambmc^2$ seçiyoruz ki bu L 'de bulunmaktadır. Sadece rakip için sorunlu seçimün $v = bk$, $y = cl$ olduğu kolayca görülebilir, burada $k = 0$, $l = 0$. O zaman pompalanan dize $w_i = ambm + (i - 1)kc^2 + (i - 1)l$ 'dir. Ancak, $w_0 = ambm - kc^2 - l$ için,

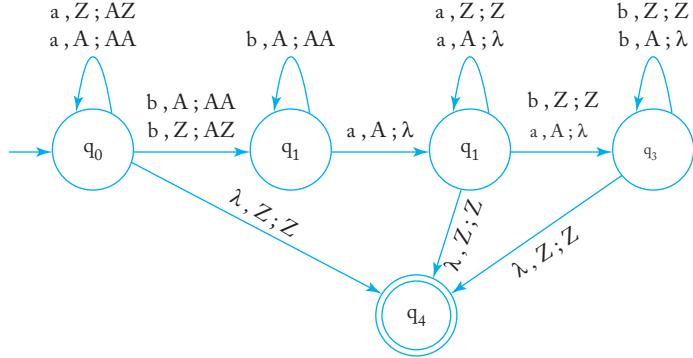
$$m(m - k) = m^2 - mk < m^2 -$$

çünkü $l \leq m$ ve $k \geq 1$. Yani $w_0 \notin L$, bu nedenle L bağlamdan bağımsız değildir.

8. (a) Dil bağlamdan bağımsızdır. Bunun için bir gramer şudur

$$\begin{aligned} S &\rightarrow aSb | S_1 \\ S_1 &\rightarrow aS_1 | bS_1 | \lambda. \end{aligned}$$

(d) Dil bağlamdan bağımsızdır ve aşağıda bir npda verilmiştir.



(f) Dil bağlamdan bağımsız değildir. Pompalama lemmasını kullanın $w = a^m b^m c^m$ ve çeşitli v veya seçimlerini inceleyin.

11. Dil bağlamdan bağımsızdır ve bir gramerle tanımlanmıştır.

$$\begin{aligned} S &\rightarrow S_1 S_1, \\ S_1 &\rightarrow a S_1 | \lambda. \end{aligned}$$

Sonraki adımda dilin lineer olup olmadığını kontrol ediyoruz. Verilen m için $w = a^m b^m a^m b^m$ şeklindedir ve bu L içindedir. Açıkça, ne olursa olsun vxy için ayrıştırma, şu formda olmalıdır $v = a^k$, $y = b^l$. O zaman pompalanan dize şudur $w_i = a^{m+(i-1)k} b^m a^m b^{m+(i-1)l}$ bazı $1 \leq k+l \leq m$ için. İçin $w_0 = a^{m-k} b^m a^m b^{m-l} \notin L$. Bu nedenle, L bağlamdan bağımsızdır ama lineer değildir.

Bölüm 8.2

3. Her $a \in T$ için, onu $h(a)$ ile değiştirin. Yeni üretim kümesi, bir bağlamdan bağımsız gramer $\hat{G}$ verir. Basit bir induksiyon, $L(\hat{G}) = h(L)$ olduğunu gösterir.

7. Argümanlar Teorem 8.5'teki ile benzerdir. $\Sigma^* - L = \overline{L}$. Eğer bağlamdan bağımsız diller fark altında kapalı olsaydı, o zaman $\overline{L}$ her zaman bağlamdan bağımsız olacaktır, bu da yerleşik sonuçlarla çelişmektedir.

Kapama sonucunun L_2 düzenli için geçerli olduğunu göstermek için, düzenliliğin tamlayıcı altında kapalı olduğunu fark edin. L_2 düzenlidir, (L_2) de öyledir. Bu nedenle, $L_1 - L_2 = L_1 \cap (L_2)$ Teorem 8.5'e göre bağlamdan bağımsızdır.

9. İki lineer dilbilgisi $G_1 = (V_1, T, S_1, P_1)$ ve $G_2 = (V_2, T, S_2, P_2)$ ile $V_1 \cap V_2 = \emptyset$, birleşik dilbilgisini oluşturun.

$$\hat{G} = (V_1 \cup V_2, T, S, P_1 \cup P_2 \cup S \rightarrow S_1 | S_2)$$

O zaman $\hat{G}$ lineerdir ve $L(\hat{G}) = L(G_1) \cup L(G_2)$. Lineer dillerin birleştirme altında kapalı olmadığını göstermek için, lineer dili $L = \{a^n b^n : n \geq 1\}$ alın. Dil L^2 lineer değildir, bu da pompalama lemmasının uygulanmasıyla gösterilebilir.

15. Diller $L_1 = \{a^n b^n c^m\}$ ve $L_2 = \{a^n b^m c^m\}$ her ikisi de belirsizdir. Ancak kesişimleri $\{a^n b^n c^n\}$ bağlamdan bağımsız değildir.

21. $\lambda \in L(G)$ yalnızca S nullable ise geçerlidir.

23. Verilen bağlamdan bağımsız dil L_1 olsun ve $L_2 = \{w \in \Sigma^* : |w| \text{ çift}\}$. Çünkü L_2 düzenlidir, Teorem 8.5'e göre, $L = L_1 \cap L_2$ bağlamdan bağımsızdır.

Bu nedenle, Teorem 8.6'ya göre, L 'nin boş olup olmadığını belirlemek için bir algoritma vardır. Bu, L_1 'in herhangi bir çift uzunluk içerip içermediğini belirlemek için bir algoritma olduğu anlamına gelir.

Bölüm 9

Bölüm 9.1

1. L dilini kabul eden bir Turing makinesi şudur:

$$\delta(q_0, a) = (q_1, a, R),$$

$$\delta(q_1, a) = (q_2, a, R),$$

$$\delta(q_2, a) = (q_3, a, R),$$

$$\delta(q_3, a) = (q_3, a, L),$$

$$\delta(q_3, b) = (q_4, b, R),$$

$$\delta(q_3, \sqcup) = (q_5, \sqcup, L),$$

$$\delta(q_4, b) = (q_4, b, R),$$

$$\delta(q_4, \sqcup) = (q_5, \sqcup, L),$$

with $F = \{q_5\}$

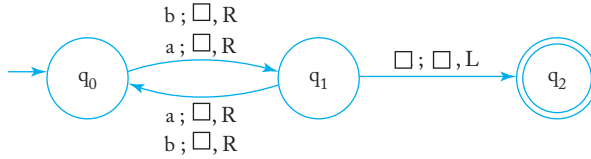
4. Her iki durumda da, Turing makinesi son olmayan durumda q_0 'da duracaktır. Anlık tanımlar şunlardır

$$q_0aba \vdash xq_1ba \vdash xq_2ya \vdash q_2xya \vdash xq_0ya,$$

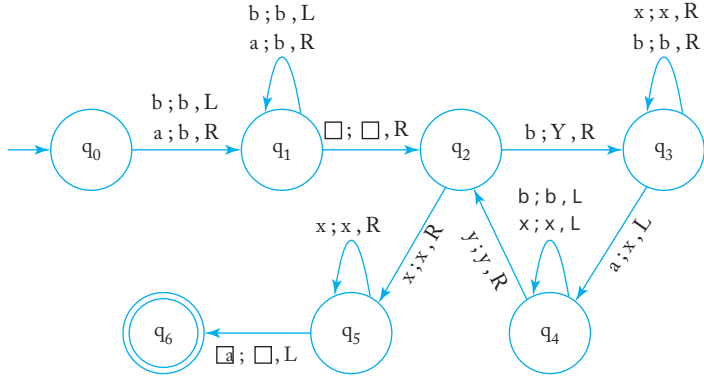
ve

$$q_0aaabbbb \vdash^* xaaq_2ybbb \vdash^* xxxq_2yyyb \vdash^* xxxq_0yyyb.$$

8. (b) $F=\{q_2\}$ olan bir geçiş grafiği



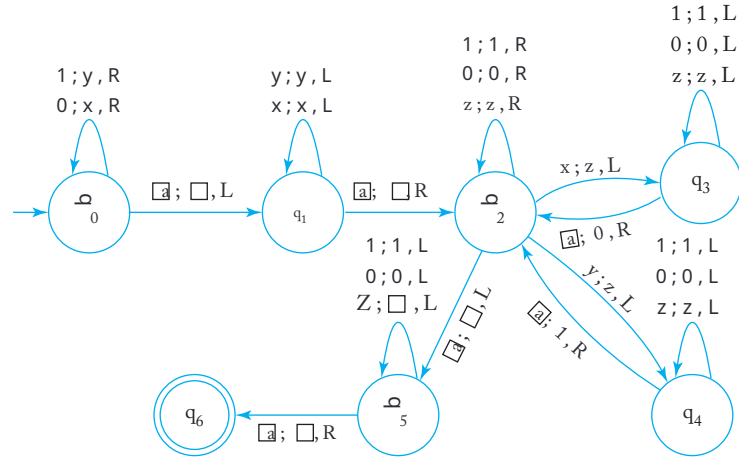
- (f) $F=\{q_6\}$ olan bir geçiş grafiği



10. Problemi çözmek için aşağıdaki süreci uyguluyoruz:

1. Her 0'ı x ile ve 1'i y ile değiştirin.
2. w 'nin en soldaki z olmayan sembolünü bulun, uygun iç duruma girerek bunu hatırlayın ve ardından onu z ile değiştirin.
3. İlk boş hücreye git ve hatırlanan sembolle ilişkili iç durum a göre 0 veya 1 oluşturun.
4. Adım 2 ve adım 3'ü, daha fazla x veya y kalmayana kadar tekrarlayın.

Çözüm için Turing makinesi biraz karmaşıktır.



12. Unary temsil kullanarak w , bu problemin çözümü için Turing makinesi.

$$\delta(q_0, 1) = (q_1, \sqcup, R),$$

$$\delta(q_0, \sqcup) = (q_4, \sqcup, L),$$

$$\delta(q_2, 1) = (q_3, 1, L),$$

$$\delta(q_2, \sqcup) = (q_4, \sqcup, L),$$

$$\delta(q_3, \sqcup) = (q_4, \sqcup, R),$$

$F = \{q_4\}$ ile.

13. (a) Örnek 9.10'daki yapıyı kullanarak x 'yi kopyalayın, ardından elde edilen dizeye bir sembol 1 ekleyin. Turing makinesinin aşağıdaki geçiş fonksiyonları vardır.

$$\delta(q_0, 1) = (q_0, x, R),$$

$$\delta(q_0, \sqcup) = (q_1, \sqcup, L),$$

$$\delta(q_1, 1) = (q_1, 1, L),$$

$$\delta(q_1, x) = (q_2, 1, R),$$

$$\delta(q_2, 1) = (q_2, 1, R),$$

$$\delta(q_2, \sqcup) = (q_1, 1, L),$$

$$\delta(q_1, \sqcup) = (q_3, 1, L),$$

$$\delta(q_3, \sqcup) = (q_4, \sqcup, R),$$

$F = \{q_4\}$ ile.

Bölüm 9.2

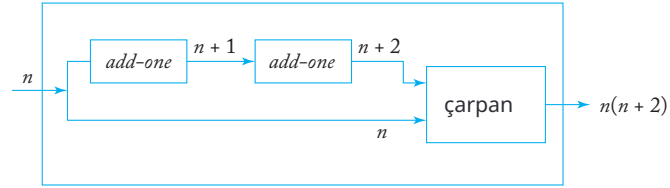
2. Varsayalım ki, eksi işaretini belirtmek için sembol s kullanıyoruz. Örneğin, $s111 = -3$. Ayrıca, tekil çıkarma işlemi için girdi olarak $x -$

y $0w(x)sw(y)0$ ile temsil ediliyorsa, o zaman aşğıdaki taslak çıkarma işlemi için çıkarıcı hesaplamayı gerçekleştirecektir

$$\begin{aligned} q_0 0w(x)sw(y)0 &\vdash^* q_f(w(x) - w(y)) \quad \text{if } x \geq \\ &\vdash^* q_s(w(y) - w(x)) \quad \text{if } x < y \end{aligned}$$

$F = \{q_f\}$ ile. Çıkarma işlemi şu şekilde gerçekleştirilebilir

1. Aşğıdaki adımları, ya x ya da y daha fazla 1 içermediği sürece tekrarlayın.
Her 1 için $w(x)$ içinde $w(y)$ içinde karşılık gelen bir 1 bulun, ardından her iki 1'i de bir token z ile değiştirin.
2. Eğer $w(y)$ daha fazla 1 içermiyorsa, s ve tüm z 'leri kaldırarak tahmin edilen sonucu elde edin, aksi takdirde sadece tüm z 'leri kaldırarak (negatif) sonucu elde edin.
3. (a) Makineyi iki ana parçadan oluşan bir yapı olarak düşünebiliriz, bir bir ekleme makinesi, girdiğe bir ekleyen ve iki sayıyı çarpan bir çarpan. Şematik olarak basit bir şekilde birleştirilmiştir.



5. (a) Makroları tanımlayın:

$3split$: convert $w_1w_2w_3$ into $w_1xw_2xw_3$.
 $reverse - compare$: compare w against w^R .

O zaman bir Turing makinesi inşa edilebilir

- adım1 : 3 ayrıştır girdi.
 adım2 : w ile w^R karşılaştırın, ardından
 terk - w^R ile w karşılaştırın.

Girdiyi yalnızca her iki adım başarılı olduğunda kabul edin.

Bölüm 10

Bölüm 10.1

2. Çevrimdışı Turing makinesi, geçiş fonksiyonu olan bir Turing makinesi dir

$$\delta : Q \times \Sigma \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

Örneğin,

$$\delta(q_i, a, b) = (q_j, c, L)$$

geçiş kuralının yalnızca makine durumunda q_i olduğunda ve giriş dosyasının salt okunur başlığı a gördüğünde ve banttaki okuma-yazma başlığı b gördüğünde uygulanabileceği anlamına gelir. Giriş dosyasındaki b sembolü c ile değiştirilecek, ardından okuma-yazma başlığı sola hareket edecektir.

Bu noktada, giriş dosyasının salt okunur başlığı sağa hareket eder ve kontrol birimi durumunu q_j 'ye değiştirir.

Standart bir Turing makinesi tarafından simülasyon için, simüle edilen makinenin bandı iki parçaya ayrılır; biri giriş dosyası için, diğeri çalışma bandı içindir. Herhangi bir belirli adımda, Turing makinesi giriş kısmını bulur, oradaki sembolü hatırlar ve siler, ardından çalışma kısmına geri döner.

5. Makinenin bir geçiş fonksiyonu vardır

$$\hat{\delta} : Q \times \Gamma \rightarrow Q \times \Gamma \times I \times \{L, R\}$$

burada $I = \{1, 2, \dots\}$.

Örneğin, bir geçiş $\hat{\delta}(q_i, a) = (q_j, b, 5, R)$ standart Turing makinesi tarafından simüle edilebilir

$$\hat{\delta}(q_i, a) = (q_{iR_1}, b, R)$$

$$\hat{\delta}(q_{iR_1}, c) = (q_{iR_2}, c, R)$$

.....

$$\hat{\delta}(q_{iR_4}, c) = (q_j, c, R)$$

tüm $c \in \Sigma$.

10. Makinenin geçiş fonksiyonu

$$\hat{\delta} : Q \times \Gamma \times \Gamma \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}.$$

Örneğin, $\hat{\delta}(q_i, b, a, c) = (q_j, d, R)$ makinenin durumu q_i 'de olduğunu, okuma-yazma kafasının bir sembol a gördüğünü, solunda b ve sağında c olduğunu gösterir. O zaman sembol a ile değiştirilecek ve okuma-yazma kafası sağa hareket edecektir. Bu noktada kontrol birimi durumunu q_j 'ye değiştirecektir.

Geçiş $\hat{\delta}(q_i, (b, a, c)) = (q_j, d, R)$ standart bir Turing makinesi tarafından simüle edilebilir

$$\delta(q_i, a) = (q_{iL}, a, L)$$

$$\delta(q_{iL}, b) = (q_{iR}, b, R)$$

$$\delta(q_{iR}, a) = (q_{iR}, a, R)$$

$$\delta(q_{iR}, c) = (q_{ic}, c, L)$$

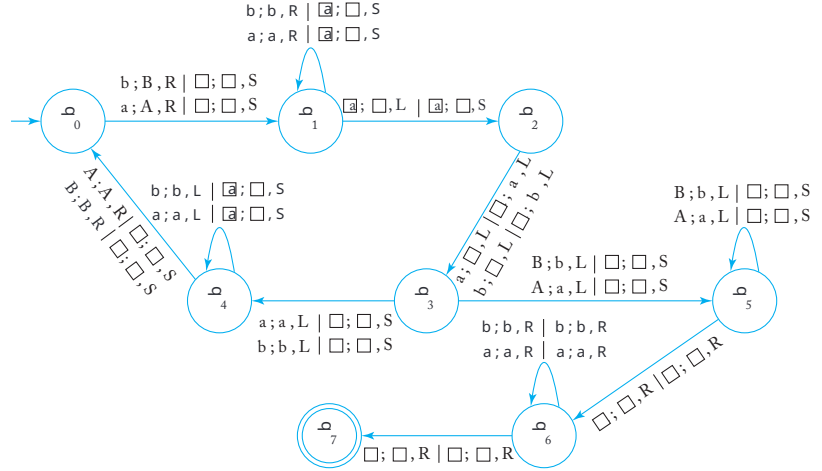
$$\delta(q_{ic}, a) = (q_j, d, R).$$

Bölüm 10.2

1. (c) Anahtar, girişı iki eşit uzunlukta paraya ayırmaktır. İlk parayı bant 1'de tutun ve ikinci parayı bant 2'ye taşıyın. Ardından, standart makinedeki gibi ileri geri gitmeden, iki bantı sembol sembol karşılaştırabiliriz.

Şerit 1'deki en soldaki sembolden başlıyoruz ve onu A ile işaretliyoruz a ve B ile işaretliyoruz b . Ardından, en sağdaki sembolü boş bir alanla değiştiriyoruz. 2. şeride ters sırayla hareket ettirdikten sonra. Bu işlemi 1. şeritte, en soldaki işaretlenmemiş sembolü işaretlemek için tekrarlıyoruz ve ardından en sağdaki sembolü 2. şeride taşıyıp boş ile değiştiriyoruz. İşlem, 1. şeritte işaretlenmemiş sembol kalmadığında başarılı bir şekilde tamamlandı. Şimdi, giridi dizelerini iki eşit uzunlukta alt dizine ayırıyoruz.

$F = \{q_7\}$ olan bir grafik aşağıda gösterilmiştir.



2. Bir çok bantlı çevrimdişı Turing makinesi, yalnızca okuma başlığına sahip tek bir giriş dosyası ve her biri okuma-yazma başlığına sahip n bant ile çalışan bir makinedir. Geçiş fonksiyonu şu şekildedir

$$\delta : Q \times \Sigma \times \Gamma^n \rightarrow Q \times \Gamma^n \times \{L, R\}^n.$$

Örneğin, eğer $n = 2$ ise, geçiş

$$\delta(q_i, a, b, c) = (q_j, d, e, R, L)$$

geçiş kuralının yalnızca makine q_i durumunda olduğunda uygulanabileceği anlamına gelir; giriş dosyasının okuma başlığı bir a görür ve bant 1'in okuma-yazma başlığı bir b görür, bant 2 ise bir c görür. Bant 1'deki sembol bd ile değiştirilir ve okuma-yazma başlığı

sağa hareket eder. Aynı zamanda, bant 2'deki sembol c bir e ile yeniden yazılır ve okuma-yazma başlığı sola hareket eder. Son olarak, okuma başlığı sağa hareket eder ve kontrol birimi ardından durumunu q_j 'ye değiştirir.

Standart bir makine tarafından n -takımlı çevrimdışı makinenin simülasyonu, $n+1$ şeritli bir bant kullanarak, yalnızca okuma başlığına karşılık gelen ilk şeridin her zaman sağa hareket edeceği ve hiçbir durumda içeriğini değiştirmeyeceği tek istisna ile, $(n+1)$ -şeritli Turing makinesi için yapılan simülasyonla aynıdır. Çok bantlı Turing makinesi için simülasyonun tanımı bu bölümün başında verilmiştir.

Bölüm 11

Bölüm 11.1

2. İki sayılabilir kümenin birleşimi sayılabilir ve tüm tekrar-üretilebilir dillerin kümesi sayılabilir. Eğer tekrar-üretilebilir olmayan tüm dillerin kümesi de sayılabilir olsaydı, o zaman tüm dillerin kümesi sayılabilir olurdu. Ancak durum böyle değildir.
4. Şekil 11.1'deki deseni kullanarak w_{ij} 'yi listeleyin, burada w_{ij} , L^j 'deki i -inci kelimedir. Bu, her L^j 'nin sayılabilir olmasından dolayı çalışır.
11. Bir bağlamdan bağımsız dil, tekrarlıdır, bu nedenle Teorem 11.4'e göre tam amlayanı da tekrarlıdır. Ancak, tamamlayanın mutlaka bağlamdan bağımsız olmadığını unutmayın.
16. Eğer $S_2 - S_1$ sonlu bir küme ise, o zaman sayılabilir. Bu, $S_2 = (S_2 - S_1) \cup S_1$ 'nin sayılabilir olması gerektiği anlamına gelir. Bu imkansızdır çünkü S_2 'nin sayılmadığı bilinmektedir. Bu nedenle, S_2 , S_1 'de olmayan sonsuz sayıda eleman içermelidir.
19. Her pozitif rasyonel sayının iki tam sayı ile temsil edilebileceğini biliyoruz ve Şekil 10.1'de gösterilen sıralama prosedüründen, pozitif rasyonel sayıların kümesinin sayılabilir olduğunu biliyoruz. Negatif olmayan rasyonel sayıların kümesi de öyledir. Bu nedenle, rasyonel sayıların kümesi sayılabilir.

Bölüm 11.2

1. Tipik bir türetim:

$$\begin{aligned} S &\Rightarrow S_1 B \Rightarrow a S_1 b B \xRightarrow{*} a^n S_1 b^n B \Rightarrow a^{n-1} a S_1 b b^{n-1} B \\ &\Rightarrow a^{n-1} a a b^{n-1} B \Rightarrow a^{n+1} b^{n-2} b b B \\ &\xRightarrow{*} a^{n+1} b^{n-2} b^{2m-1} b B \Rightarrow a^{n+1} b^{n-1} b^{2m}. \end{aligned}$$

Bundan, $L(M) = \{a^{n+1}b^{n-1}b^{2m} : n \geq 1, m \geq 0\}$.

3. Resmi olarak, dilbilgisi $G = (V, S, T, P)$ ile tanımlanabilir, burada $S \subseteq (V \cup T)^+$ ve

$$L(G) = \left\{ x \in T^* : s \xRightarrow{*}_G x \text{ for any } s \in S \right\}.$$

Tanım 11.3'teki sınırsız dilbilgileri, bu genişlemeye eşdeğerdir çünkü herhangi bir verilen sınırsız dilbilgisine her zaman başlangıç kuralları $S_0 \rightarrow s_i$ ekleyebiliriz, burada tüm $s_i \in S$.

6. Bu tür sınırsız gramerleri elde etmek için, sağa sahte değişkenler ekleyin her zaman $|u| > |v|$. Örneğin,

$$AB \rightarrow C$$

şu ile değiştirilebilir

$$\begin{aligned} AB &\rightarrow CD, \\ D &\rightarrow \lambda. \end{aligned}$$

Eşdeğerlik argümanı açıktır.

Bölüm 11.3

2. (a) Bir bağlama duyarlı gramer

$$\begin{aligned} S &\rightarrow aaAbbc|aab, \\ Ab &\rightarrow bA, \\ Ac &\rightarrow Bcc, \\ bB &\rightarrow Bb, \\ aB &\rightarrow aaAb, \\ aA &\rightarrow aa. \end{aligned}$$

3. (a) Çözüm, oluşturacakları terminal ile alt yazılmış değişkenler kullanılarak daha sezgisel hale getirilebilir. Bir cevap şudur:

$$\begin{aligned} S &\rightarrow SV_aV_bV_c|V_aV_bV_c, \\ V_aV_b &\rightarrow V_bV_a, \\ V_aV_c &\rightarrow V_cV_a, \\ V_bV_a &\rightarrow V_aV_b, \\ V_bV_c &\rightarrow V_cV_b, \\ V_cV_a &\rightarrow V_aV_c, \\ V_cV_b &\rightarrow V_bV_c, \\ V_a &\rightarrow a, \\ V_b &\rightarrow b, \\ V_c &\rightarrow c. \end{aligned}$$

Bölüm 12

Bölüm 12.1

2. $\widehat{M}$ ilk olarak girdiğini kaydeder ve bunu w ile değiştirir, ardından M gibi devam eder. If (M, w) durursa, $\widehat{M}$ orijinal girdiğini kontrol eder ve eğer bu girdi çift uzunluktaysa kabul eder.
3. Verilen M ve w , M 'yi değiştirerek $\widehat{M}$ elde edin, bu yalnızca biröz el sembol, diyelim ki tanıtılan sembol $\#$ yazıldığında durur. Bunu M 'nin durma konfigürasyonlarını değiştirerek yapabiliriz, böylece her durma konfigürasyonu $\#$ yazar, ardından durur. Böylece, M durursa, $\widehat{M}\#$ yazar ve $\widehat{M}\#$ yazarsa, M durur. Böylece, belirli bir sembol a 'nın hiç yazılıp yazılmadığını söyleyen bir algoritmamız varsa, bunu $\widehat{M}^y e a = \#$ ile uygularız. Bu, durma problemine çözüm olur.
9. Evet, bu karara bağlanabilir. Bir $dpda$ 'nın sonsuz bir döngüye girmesinin tek yolu λ -geçişleridir. Bir dizi olup olmadığını bulabiliriz λ -geçişlerinin (a) nihayetinde aynı durumu ve yığın üstünü üretip üretmediğini ve (b) yığının uzunluğunu azaltmadığını. Eğer böyle bir dizi yoksa, o zaman $dpda$ durmalıdır. Eğer böyle bir dizi varsa, bu diziyi başlatan konfigürasyonun erişilebilir olup olmadığını belirleyin. Çünkü, herhangi bir verilen w için, yalnızca sonlu sayıda hamleyi analiz etmemiz gerektiğinden, bu her zaman yapılabilir.

Bölüm 12.2

4. Farz edelim ki $L(M_1) \subseteq L(M_2)$. Böylece M_2 makinesini inşa edebiliriz öyle ki $L(M_2) =$ ve algoritmayı uygulayabiliriz. O zaman $L(M_1) \subseteq L(M_2)$ yalnızca eğer $L(M_1) =$. Ancak bu, Teorem 12.3 ile çelişiyor, çünkü herhangi bir verilen dilbilgisi G' den M_1 'i inşa edebiliriz.
7. Eğer $L(G_2) = \Sigma^*$ alırsak, problem Teorem 12.3 sorusuna dönüşür ve bu nedenle kararsızdır.
9. Yine, Teorem 12.4 ve Örnek 12.4'teki aynı tür argüman, ancak problemi biraz daha zorlaştıran birkaç adım vardır.

(a) Durdurma problemi ile başlayın (M, w) .

(b) Herhangi bir G_2 verildiğinde $L(G_2) / =$, bazı $v \in L(G_2)$ üretin.

Bu her zaman yapılabilir çünkü G_2 düzenlidir.

(c) Teorem 12.4'te olduğu gibi, eğer (M, w)

durdurursa, o zaman M v'yi kabul edecek şekilde M 'yi $\widehat{M}$ olarak değiştirin.

(d) $\widehat{M}$ 'den G_1 üretin, böylece $L(\widehat{M}) = L(G_1)$.

Şimdi, eğer (M, w) durursa, o zaman $\widehat{M}$ v kabul eder ve $L(G_1) \cap L(G_2) / = \emptyset$.
 Eğer (M, w) durmazsa, o zaman $\widehat{M}$ hiçbir şeyi kabul etmez, bu nedenle $L(G_1) \cap L(G_2) = \emptyset$. Bu yapı, herhangi bir G_2 için yapılabilir, bu nedenle karar verilebilir bir problem için herhangi bir G_2 var olamaz.

Bölüm 12.3

2. Bir PC çözümü $w_3w_4w_1=v_3v_4v_1$. Bir MPC çözümü yoktur çünkü bir dize 001 ön eki alırken, diğeri 01 alır.
6. (a) Problem kararsızdır. Eğer kararlı olsaydı, orijinal MPC problemini çözmek için bir algoritmamız olurdu. Verilen $w_1, w_2, \dots, w_n$, $w^R_1, w^R_2, \dots, w^R_n$ oluştururuz ve varsayılan algoritmayı kullanırız. Çünkü $w_1w_2\dots w_k = (w^R_1 \dots w^R_k w^R_1)^R$, orijinal MPC problemi bir çözüme sahipse, yalnızca yeni MPC problemi bir çözüme sahipse vardır.

Bölüm 12.5

2.
 - Standart bir makinede. Dizenin ortasını bulun, ardından sembolleri eşleştirmek için ileri geri gidin. Her iki parça $O(n^2)$ hamle alır.
 - İki bantlı deterministik bir makinede. Sembollerin sayısını sayın. Eğer sayı birim sisteminde yapılırsa, bu bir $O(n)$ işlemdir. Sonra, ilk yarıyı ikinci bantta yazın. Bu da bir $O(n)$ işlemdir. Son olarak, $O(n)$ hamlede karşılaştırabilirsiniz.
 - Tek bantlı nondeterministik bir makinede. Dizenin ortasını bir hamlede nondeterministik olarak tahmin edebilirsiniz. Ancak eşleştirme hala $O(n^2)$ hamle alır.
 - İki bantlı nondeterministik bir makinede. Dizenin ortasını tahmini edin, ardından iki bantlı deterministik makinedeki gibi devam edin. Gerekli toplam çaba $O(n)$ dir.

Bölüm 13

Bölüm 13.1

2. Örnek 13.3'teki *subtr* fonksiyonunu kullanarak, çözümü elde ediyoruz

$$\text{greater}(x, y) = \text{subtr}(1, \text{subtr}(1, \text{subtr}(x, y)))$$

4. Örnek 13.2'deki *mult* ve Örnek 13.3'teki *subtr* fonksiyonunu kullanarak, çözümü elde ediyoruz

$$\text{eşittir}(x, y) = \text{mult}(\text{subtr}(1, \text{subtr}(x, y)), \text{subtr}(1, \text{subtr}(y, x))).$$

8. Fonksiyon şu şekilde tanımlanabilir

$$\begin{aligned} f(0) &= 1, \\ f(n+1) &= \text{mult}(2, f(n)). \end{aligned}$$

12. (a) Ackermann fonksiyonunun tanımından doğrudan türetme verir

$$\begin{aligned} A(1, y) &= A(0, A(1, y-1)) \\ &= A(1, y-1) + 1 \\ &= A(1, y-2) + 2 \\ &\vdots \\ &= A(1, 0) + y \\ &= y + 2. \end{aligned}$$

Bölüm 13.2

1. (a) λ , ab , $aababb$, $aaababbaababbb$

8. Aksiyomdan türetilen ilk birkaç cümle şunlardır

$$ab, a(ab)^2, a(a(ab)^2)^2, a(a(a(ab)^2)^2)^2 \dots$$

Bu nedenle, dil

$$L = \{ab\} \cup L_1$$

nerede

$$L_1 = \{(a_n(a_{n-1} \dots (a_1(ab)^{k_1}) \dots)^{k_{n-1}})^{k_n} : k_i = 2, a_i = a, i \leq n = 1, 2, \dots\}.$$

Bölüm 13.3

1.

$$\begin{aligned} P_1 &: S \rightarrow S_1 S_2, \\ P_2 &: S_1 \rightarrow a S_1, S_2 \rightarrow b S_2 c, \\ P_3 &: S_1 \rightarrow \lambda, S_2 \rightarrow \lambda. \end{aligned}$$

6. Buradaki çözüm, bağlam-duyarlı gramerlerle haberci kullanımını a nımsatmaktadır.

$$\begin{aligned} ab &\rightarrow x, \\ xb &\rightarrow bx, \\ xc &\rightarrow \lambda. \end{aligned}$$

REFERANSLAR

DAHA FAZLA OKUMA İÇİN

- A. V. Aho ve J. D. Ullman. 1972. Ayrıştırma, Çeviri, ve Derleme Teorisi. Cilt 1. Englewood Cliffs, N.J.: Prentice Hall.
- P. J. Denning, J. B. Dennis ve J. E. Qualitz. 1978. Makineler, Diller, ve Hesaplama. Englewood Cliffs, N.J.: Prentice Hall.
- M. R. Garey ve D. Johnson. 1979. Bilgisayarlar ve Çözülmesi Zor Problemler. Yeni York: Freeman.
- M. A. Harrison. 1978. Resmi Dil Teorisine Giriş. Reading, Mass.: Addison-Wesley.
- J. E. Hopcroft ve J. D. Ullman. 1979. Otomata Teorisi, Diller ve Hesaplama. Reading, Mass.: Addison-Wesley.
- R. Hunter. 1981. Derleyicilerin Tasarımı ve İnşası. Chichester, New York: John Wiley.
- R. Johnsonbaugh. 1996. Ayrık Matematik. Dördüncü Baskı. New York: Macmillan.
- Z. Kohavi ve N. K. Jha. 2010. Anahtarlama ve Sonlu Otomata Teorisi. Üçüncü Baskı. New York: Cambridge University Press.
- C. H. Papadimitriou. 1994. Hesaplama Karmaşıklığı. Reading, Mass.: Addison-Wesley.
- G. E. Revesz. 1983. Resmi Dillere Giriş. New York: McGraw-Hill.
- A. Salomaa. 1973. Resmi Diller. New York: Akademik Basım.
- A. Salomaa. 1985. "Hesaplamalar ve Otomatlar," in Matematik ve Uygulamaları Ansiklopedisi. Cambridge: Cambridge University Press.

DİZİN

Not: Sayfa numaralarının ardından *f* gelenler, şekillerdeki materyali belirtir.

A

özet, 395
kabul eden, 28
Ackermann fonksiyonu, 342–343
algoritmalar, 3, 257
 boş bant durma
 problemi için, 317*f*
 Markov, 351–354
 üye, 320*f*, 327*f*
alfabeler, 17
belirsizlik, 140–149
 gramerler ve dillerde,
 145–149
 doğal, 148–149
automata, 2, 17, 26–28
 konfigürasyon, 27
 kontrol birimi, 27
 belirleyici, 27–28
 general özellikler, 26–28
 iç durumlar, 27
 belirsiz, 28
automata sınıfları, eşdeğerliği,
 260–261
otomaton, 2
aksiyomlar, 346, 348

B

Backus-Naur formu (BNF), 151,
 152
döngü tabanı, 9
indüksiyon için temel, 10, 11
ikili toplayıcı, 33
ikili ağaç, 10–11
boş, 233
boş-bant durdurma problemi,
 315–316
 algoritma için, 317
BNF. Backus-Naur formuna bakınız
Boolean operatörleri, 358
zorla ayırıştırma, 141

C

Kartezyen çarpım, 5
ağaçtaki çocuk-ebeveyn ilişkisi, 9
Chomsky hiyerarşisi, 305–307,
 306 *f*, 362–365
Chomsky normal formu, 156,
 171–174
Church-Turing tezi, 337
Church tezi, 337

clique problemi (CLIQ), 369, 374
 kapama
 bağlamdan bağımsız dillerin,
 223–229
 pozitif, 20
 yıldız, 20
 düzenli dillerin kapama ö
 zellikleri
 diğer işlemler, 105–111
 basit küme işlemleri, 102–105
 kapama sorusu, 101–102
 CNF. Bakınız bağlayıcı normal
 form
 kümenin tamamlayıcı, 4
 tam sistemler, 336
 karmaşıklık, 355
 sınıflar, 362–365
 alan, 355
 zaman, 355
 Turing makinesi modelleri,
 358–360
 NP karmaşıklık sınıfı, 365–367
 P karmaşıklık sınıfı, 365–367
 bileşim, 338
 hesaplanabilirlik, 310–311
 hesaplanabilir fonksiyon, 341, 342
 hesaplama modelleri, 238, 337
 hesaplama karmaşıklığı,
 355–356
 P ve NP karmaşıklık sınıfları,
 365–367
 hesaplamanın verimliliği,
 356–357
 dil aileleri ve
 karmaşıklık sınıfları, 362–365
 NP-tamamlayıcı problemler,
 373–374
 NP problemleri, 367–370
 polinom zamanlı indirgeme,
 370–373
 Turing makinesi modelleri,
 358–360
 dillerin birleştiril
 mesi, 20stringlerin b
 irleştirilmesi, 17

otomatonun konfigürasyonu, 27
 bağlayıcı normal form (CNF),
 359
 tutarlı sistemler, 336
 bağlamdan bağımsız gramerler, 130–138,
 151–153, 155–156
 üyelik algoritması için,
 178–180
 gramerleri dönüştürme yöntemleri,
 156–167
 bağlamdan bağımsız diller, 129–153,
 191–201, 306–307, 307f
 deterministik, 203–206
 özellikler, 213
 kapama özellikleri ve karar
 algoritmaları için, 223–229
 dua pompalama lemması, 214–221
 teoremi, 193–195, 200–201
 karar verilemez problemler için,
 329–332
 bağlamdan duyarlı gramerler,
 299–304
 bağlamdan duyarlı diller,
 300–302
 özyinelemeli *us.*, 302–304
 çelişki, kanıt teknikleri
 ile, 13–14
 otomatonun kontrol birimi, 27
 Cook-Karp tezi, 366
 Cook'un teoremi, 374
 sayılabilir kümeler, 278
 graf üzerindeki döngüler, 9
 CYK algoritması, 178–180

D
 ölü yapılandırma, 55
 karar verilebilirlik, 310–311
 bir dili karar verme, 275
 bağlamdan bağımsız diller için
 karar algoritmaları,
 223–229
 DeMorgan yasaları, 4, 104
 bağımlılık grafikleri, 160
 üretilme, 22
 en soldaki, 133–134

en sağdaki, 133–134
 cümle biçimleri, 22
 türetilme ağaçları, 134–136, 135_f
 kısmi, 135
 cümle biçimleri ve, 137
 verim, 136
 türetilir, 22
 belirlenim, 27–28
 belirleyici algoritmalar, 56
 belirleyici bağlamdan bağımsız
 diller
 tanım, 203
 için gramerler, 207–211
 belirleyici sonlu kabul edenler
 (dfa), 38–48, 89
 eşdeğerliği, 58–64
 uzatılmış geçiş fonksiyonu
 için, 40
 diller ve, 41–45
 durum sayısının azaltılması
 , 66–71
 normal diller, 46–48
 ve geçiş grafikleri, 39–41
 belirleyici yığın kabul edici
 (dpda), 203
 belirleyici yığın
 otomatlar, 203–206
 dfa. Bak belirleyici sonlu
 kabul ediciler
 çizgisel, 289
 fark, küme işlemleri, 4
 dijital bilgisayarlar, 56
 ayrık kümeler, 5
 ayırıcı durumlar, 67, 68
 fonksiyonun tanım kümesi, 6, 337
 dpda, 203–206

E

grafin kenarı, 8
 hesaplamanın verimliliği,
 356–357
 sorunun verimliliği, 332–333
 temel sorular, normal
 diller, 114–115
 elipsler, 4

boş küme, 4
 lba için son işaretleyiciler, 281
 numaralandırma prosedürü, 279
 eşdeğerlik
 otomat sınıflarının, 260–261
 sınıflar, 8
 dfa'lar ve nfa'lar, 58–64
 gramerlerin, 26
 Mealy ve Moore makineleri,
 382–386
 düzenli ifadeler, 78
 ilişki, 7
 kapsamlı arama ayırıştırması, 141,
 142
 genişletilmiş çıktı fonksiyonu, 382
 genişletilmiş geçiş fonksiyonu
 dfa'lar için, 40
 sonlu durum dönüştürücüleri için, 382
 nfa'lar için, 53–55

F

diller ailesi, 46
 son durumlar, 39
 son köşeler, 40
 sonlu otomata, 37–71, 88, 95
 sonlu kümeler, 5
 sonlu durum dönüştürücüleri (fst'ler),
 377–378
 Mealy makinesi, 378–380
 Moore makinesi, 380–381
 formel diller, 3, 30
 formel diller teorisi, 151
 fst'ler. Bakınız sonlu durum dönüştürücüleri
 fonksiyonlar, 6–8
 Ackermann'ın, 342–343
 hesaplanabilir, 243, 342
 kavramı, 337, 344
 domain, 6, 337
 μ özyinelemeli, 338
 kısmi, 6
 öncül, 340
 ilkel özyinelemeli, 338–341
 projeksiyon, 338
 aralık, 6, 337
 ardıl, 338

fonksiyonlar (*Cont.*)

tam, 6
sıfır, 338

G

oyun oynama programı, 56
genelleştirilmiş geçiş grafikleri
(GTG), 83–87
Gödel, K., 336–337
grammerlar, 17, 20–26, 30b
 elirsizlik, 145–149bağlam
 dan bağımsız, 130–138b
 ağlamdan bağımlı, 299–304b
 elirleyici bağlamdan bağımsız dill
 er için, 207–211eşdeğ
 er, 26
 örnek, 195–196
 kalbinde, 21
 sol-lineer, 92, 97
 lineer, 93
 matris, 350–351
 üretimleri, 21
 düzenli, 92
 sağ-lineer, 92–99
 basit, 144–145
 kısıtlamasız, 293–298
grafikler, 8–9
 etiketli, 8–9
Greibach normal formu, 156,
 174–176, 191–193, 195, 196
GTG. Genel geçiş grafiklerine
 bakınız

H

bir Turing makinesinin durma durumu,
 234
durma problemi, 311
 algoritması için, 316f
Hamilton yolu problemi
 (HAMPATH), 368, 374
ağaç yüksekliği, 9
resmi dillerin hiyerarşisi
 Chomsky hiyerarşisi, 305–307,
 306 f

bağlam duyarlı gramerler
ve diller, 299–304

özyinelemeli ve özyinelemeli
sayılabilir diller,
 286–292

Turing makineleri, 285–286
kısıtlamasız gramerler,
 293–298

homomorfik görüntü, 105
homomorfizma, 105

I

tamamlanamazlık teoremi, 337
ayırt edilemez durumlar, 67
indüksiyon, kanıt teknikleri ile,
 10–13
indüktif varsayım, 10
indüktif akıl yürütme, 11
indüktif adım, 10
sonsuz döngü, 236
sonsuz kümeler, 5
doğası gereği belirsiz dil,
 148–149
ilk durum, 39
ilk köşe, 40
girdi alfabesi, 39, 377
girdi dosyası, 27
bir yığın otomatonunun
 anlık tanımı, 186bir Turing ma
 kinesinin anlık tanımı, 236–2
37iç durumlar, 39
 otomaton, 27
kesişim, küme işlemleri, 4
çözümlemeyen problemler, 366
irrasyonel sayı, 13

J

JFLAP, 395–396

L

L-sistemleri, 353–354
 λ -prodüksiyonları, 163–165
dil aileleri, 362–365
diller, 17–20

bir dfa tarafından kabul edilen, 41–45

bir pda tarafından kabul edilen, 192,
 199–200
 bir Turing makinesi tarafından kabul edilen,
 239–242
 lba tarafından kabul edilen, 282–283
 nfa tarafından kabul edilen, 55
 npda tarafından kabul edilen, 187–189
belirsizlik, 145–149
düzenli ifadelerle ilişkili
, 75–78
 complement of, 19
 birleştirme, 20bağlamda
 n bağımsız, 129–153bir gr
 amer tarafından üretilen, 22bir Ma
 rkov algoritması tarafından
 üretilen, 352–353Post s
 istemi tarafından üretilen,
 347–349
pozitif kapanış, 20
düzenli, 37, 46–48
tersi, 19
star-kapanışı, 20
 lba, 281
ağaç yaprakları, 9sol uç işaret
eti, 281sol-lineer gramerler,
92, 97sol en çok türetme, 1
33–134sonlu durum dönüşt
ürücülerinin sınırlamaları, 3
92–394lineer sınırlı oto
mata, 281–283, 300–302line
er gramer, 93lineer dill
er, pompalamalemma için,
219–221lineer zaman ayrışt
ırma algoritması,

 144
literal, 359
LL gramerleri, 152, 208
 döngü, 9
LR gramerleri, 152, 211

M
makro talimat, 251–252
Markov algoritmaları, 351–354

matematik ön koşulları ve
notasyon
fonksiyonlar ve ilişkiler, 6–8
grafikler ve ağaçlar, 8–9
kanıt teknikleri, 9–14
 kümeler, 3–6
matris grameri, 350–351
Mealy makineleri, 378–380
eşdeğerlik, 382–386
minimizasyon, 386–391
üye algoritması, 129,
320 f, 327f
 bağlamdan bağımsız diller için,
 178–180
bağlamdan duyarlı diller için,
363
parsing ve, 141–145
düzenli diller için, 114
 minimal dfa, 69
 minimalizasyon operatörü, 343Tu
 ring makinesindeki küçük varya
 syonlar
 otomata sınıflarının eşdeğe
 rliliği, 260–261çev
 rimdışı Turing makinesi,
 265–267
 yarı sonsuz bantlı Turi
 ng makineleri, 263–265kalm
 a seçeneği olan Turing
 makineleri, 261–263
 μ -özyinelemeli fonksiyonlar, 343–344
değiştirilmiş Post eşleşme
problemi, 324
 monus, 339
Moore makineleri, 380–381
eşdeğerlilik, 382–386
minimizasyonu, 391–392
automaton hareketi, 27
MPC algoritması, 328f
MPC çözümü, 324
çok boyutlu Turing
makineleri, 271–272
 birden fazla bant, 263
 çok bantlı Turing makineleri,
 268–271

N

son-durum fonksiyonu, 27
 nfa. Bak belirsiz sonlu
 kabul edenler
 kontratsız gramerler, 300
 belirsizlik, 37, 51, 52,
 55–56, 396
 belirsiz algoritma, 56
 belirsiz otomaton, 28
 belirsiz sonlu kabul ediciler
 (nfa), 51–56, 80
 eşdeğerliği, 58–64
 uzatılmış geçiş fonksiyonu
 için, 53–55
 belirsiz yığın
 kabul edici (npda), 183
 örneği, 184–185, 198–199
 geçiş grafiği, 185
 belirsiz yığın
 otomata, 182–189
 belirsiz Turing
 makinelere, 273–275
 anlaşılması zor, 396
 belirsiz diller
 pompalama lemması, 118–125
 güvercin yuvası ilkesini kullanarak,
 117–118
 sondör sabitler, 346
 normal formlar
 dilbilgisi, 155–156
 önemi, 171–176
 NP-tamamlayıcı problemler, 373–374
 NP problemleri, 367–370
 npda, 183
 boş küme, 4
 nullable, 163

O

çevrimdışı Turing makinesi, 265–267
 Ogden'in lemması, 223
 sıra
 büyüklük notasyonu, 6, 7
 ağaçtaki ilişki, 9
 en azından notasyonu,
 fonksiyonlar, 6

en çok notasyonu,
 fonksiyonlar, 6
 çıkış alfabesi, 377

P

parse tree, 134–136
 parsing, 129, 130, 140–149
 brute force, 141
 context-free grammars, 178
 exhaustive search, 141
 and membership algorithm,
 141–145
 top-down, 141
 partial derivation tree, 135
 partial function, 6
 partition, 6
 partition algorithm, 389
 path
 in graph, 9
 labeled, 9
 simple, 9
 pattern matching, 88–89
 PC algorithm, 331f
 PC-solution. *Seepost*
 correspondence solution
 phrase-structure grammars, 350
 pigeonhole principle, 388
 kullanım, 117–118
 polinom-zaman indirgeme,
 370–373
 pozitif dil kapanışı, 20
 post correspondence problemi,
 323–328
 post correspondence çözümü
 (PC-çözümü), 323
 post sistemler, 346–349
 kuvvet kümesi, 5
 önceki fonksiyon, 340
 ön ek, 18
 ilkel rekürsion, 338
 ilkel rekürsif fonksiyonlar,
 338–341
 ilkel düzenli ifadeler, 74
 dilbilgisi üretimleri, 21

bir Turing makinesinin programı,
234
programlama dilleri, 31–33,
130, 151–153
projektör fonksiyonu, 338
kanıt teknikleri, 9–14
çelişki, 9, 13–14
indüksiyon, 9, 10–13
uygun sıra, 279
uygun alt küme, 5
pompalama lemması
bağlamdan bağımsız diller için,
214–218
doğrusal diller için, 219–221
düzenli diller için, 118–125
itme otomata (pda),
181–182, 182f
ve bağlamdan bağımsız diller,
191–201
tanımı, 183belir
leyici, 203–206belirle
yici bağlamdan bağımsız
diller için dilbilgileri,
207–211
dil tarafından kabul edilen, 186
belirsiz, 182–189

R

fonksiyonun aralığı, 6, 337
rasyonel sayı, 13
okuma-yazma kafası, 232
fonksiyonun özyinelemesi, 11
özyinelemeli fonksiyonlar, 337–338
ilkel, 338–341
özyinelemeli diller, 286–292
vs. bağlam duyarlı, 302–304
tanımı, 287
özyinelemeli olarak sayılabilir
diller, 286–292
tanımı, 286
indirgeme
dfa'daki durum sayısı,
66–71
polinom zaman, 370–373

karar verilemeyen problemler,
315–318
eşdeğerlik için yansıtma kuralı
ilişkisi, 7
düzenli ifadeler
definasyonu, 74
ilişkili dil,
74–78
ve düzenli diller, 80–91
basit desenler için, 88–89
düzenli gramerler, 92. Ayrıca bakınız
gramerler
eşdeğerliği, 97–99
düzenli kesişim, 226
düzenli diller, 37, 46–48,
130, 131
eşdeğerliği, 97–99
düzenli ifadeler ve, 80–91
sağ-lineer gramerler için,
93–97
düzenli dillerin özellikleri, 101
kapama özellikleri. Bak
kapama özellikleri düzenli
dillerin
temel sorular,
114–115
normal olmayan diller,
117–125
ilişkiler, 7
yeniden programlanabilir Turing makinesi.
Bakınız evrensel Turing
makinesi
ters
dil, 19
string, 17
yeniden yazım sistemleri, 350–354
Rice teoremi, 321
sağ uç işareti, 281
sağ-lineer gramerler, 92
normal diller için, 93–99
bir dilin sağ bölüm, 106,
107
en sağdan türetim, 133–134
ağacın kökü, 9

S

s-grammer, 144–145, 208
 satisfiability problemi (SAT), 358
 3SAT, 371–373
 programlama dilinin anlamsal yapısı
 , 153
 yarı sonsuz bant, Turing
 makinelere ile, 263–265
 cümle, 3, 19
 cümle biçimleri, 22, 131
 ve türetim ağaçları, 137
 seri toplayıcı, 34, 34^f
 küme işlemleri, 4
 kümeler, 3–6
 sayılabilir, 278
 boş, 4
 sonsuz, 5
 null, 4
 güç, 5
 büyüklük, 5
 sayısız, 278
 basit küme işlemleri, 102–105
 simülasyon, 261
 tek bantlı Turing makinesi,
 332–333
 alan karmaşıklığı, 355
 yığın, 183
 alfabe, 183
 başlangıç sembolü, 183
 standart temsil
 bir düzenli dilin, 114
 standart Turing makinesi, 232,
 236
 tanımı, 232–239
 dil kabul edenler, 239–242
 dönüştürücüler, 242–247
 dilin yıldız-kapatılması, 20
 durum-giriş problemi, 315
 kalma seçeneği, Turing makineleri
 ile, 261–263
 depolama cihazı, 27
 karmaşık Turing makineleri
 çok boyutlu Turing
 makinelere, 271–272

çok bantlı Turing makineleri,
 268–271
 otomatonun depolanması, 27
 string, 17
 birleştirme, 17
 boş, 17
 uzunluk, 17
 işlemler, 17
 ön ek, 18
 tersi, 17
 son ek, 18
 alt programlar, 252
 alt küme, 4
 doğru, 5
 yerine koyma kuralı, 157–159
 alt dize, 18
 ardışık dizeler, 22
 sonuç fonksiyonu, 338
 son ek, 18
 eşdeğerlik için simetri kuralı
 ilişki, 7
 programlama dilinin sözdizimi,
 152

T

şerit alfabesi, 233
 Turing makinesinin şeridi, 232
 son sabitler, 346
 son sembol, 21
 hesaplama teorisi, 1–3
 uygulamalar, 30–34
 automata, 26–28
 gramerler, 20–26
 diller, 17–20
 matematiksel ön bilgiler
 ve notasyon, 3–14
 üçlü doyurulabilirlik problemi
 (3SAT), 371–373
 zaman karmaşıklığı, 355
 üstten aşağı ayırıştırma, 141
 tam fonksiyon, 6
 şeritteki izler, 263
 çözülebilir problemler, 366, 367
 çevrimci, 28, 33, 242–247

dönüştürme gramerleri, yöntemler için, 156–167
geçiş fonksiyonu, 27, 39, 233
 genişletilmiş, 40, 53
 nfa'lar için, 97
geçiş grafikleri, 245f, 379so
 nlu kabul edici, 39–41ge
 nelleştirilmiş, 83–8
 7Moore makinelerinin,
 380bir yığın otomatonun, 185bi
 r Turing makinesinin, 235, 235f
eşdeğerlik ilişkisi için geçişlilik k
 uralı, 7
 tuzağa düşme durumu, 42
treler, 8–9
Turing hesaplanabilir, 243
Turing makinesi, 231–232,
 259–260
 karmaşık depolama cihazı ile,
 268–272
 talimat seti, 255
 sezgisel görselleştirme, 233f
 doğrusal sınırlı otomata,
 281–283
 küçük varyasyonlar. Bak Turing
 makinesindeki küçük
 varyasyonlar
çok boyutlu, 271–272
birden fazla parça ile, 263
çok bantlı, 268–271
belirsiz, 273–275
açık, 265–267
yarı sonsuz bant ile,
 263–265
standart, 232–247
kalma seçeneği ile, 261–263
görevler, 249–253
evrensel, 276–280
Turing makinesi durma problemi,
 311–315
Turing makinesi modelleri, 358–361
Turing makineleri, 285–286
sorunları, 310–318

hesaplanabilirlik ve karar verilebilirlik,
 310–311
Turing makinesi durma problemi,
 311–315
karar verilemez problem, 315–318
Turing'in tezi, 232, 255–260, 273
iki boyutlu Turing makinesi,
 271

U

sayılabılır olmayan kümeler, 278
karar verilemez, 310
bağlamdan bağımsız diller için
 karar verilemez problemler,
 329–332
birleşim, küme işlemleri, 4
birim üretimler, 165–167
evrensel küme, 4
evrensel Turing makinesi,
 276–280
sınırsız gramerler, 293–298
gereksiz üretimler, 159–163

V

değişkenler
 gramer, 21
 nullable, 163
 başlangıç, 21
 gereksiz, 159
tepe
 son, 40
 grafik, 8
 ilk, 40

W

grafikte yürüyüş, 9

Y

bir türetim ağacının verimi, 136

Z

sıfır fonksiyonu, 338